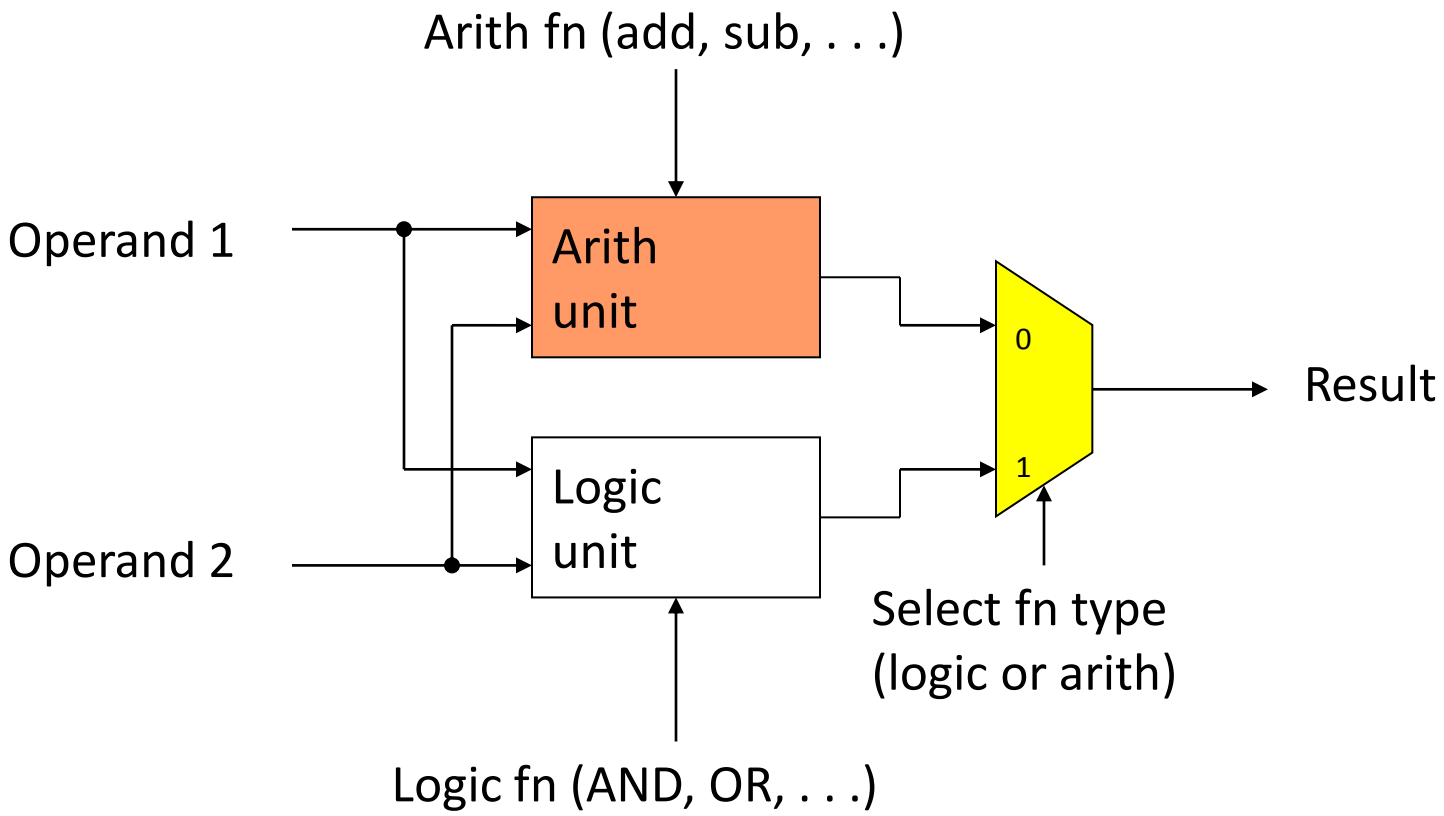
Calculatoare Numerice

– Cursul 6 –

Operații aritmetice (2)

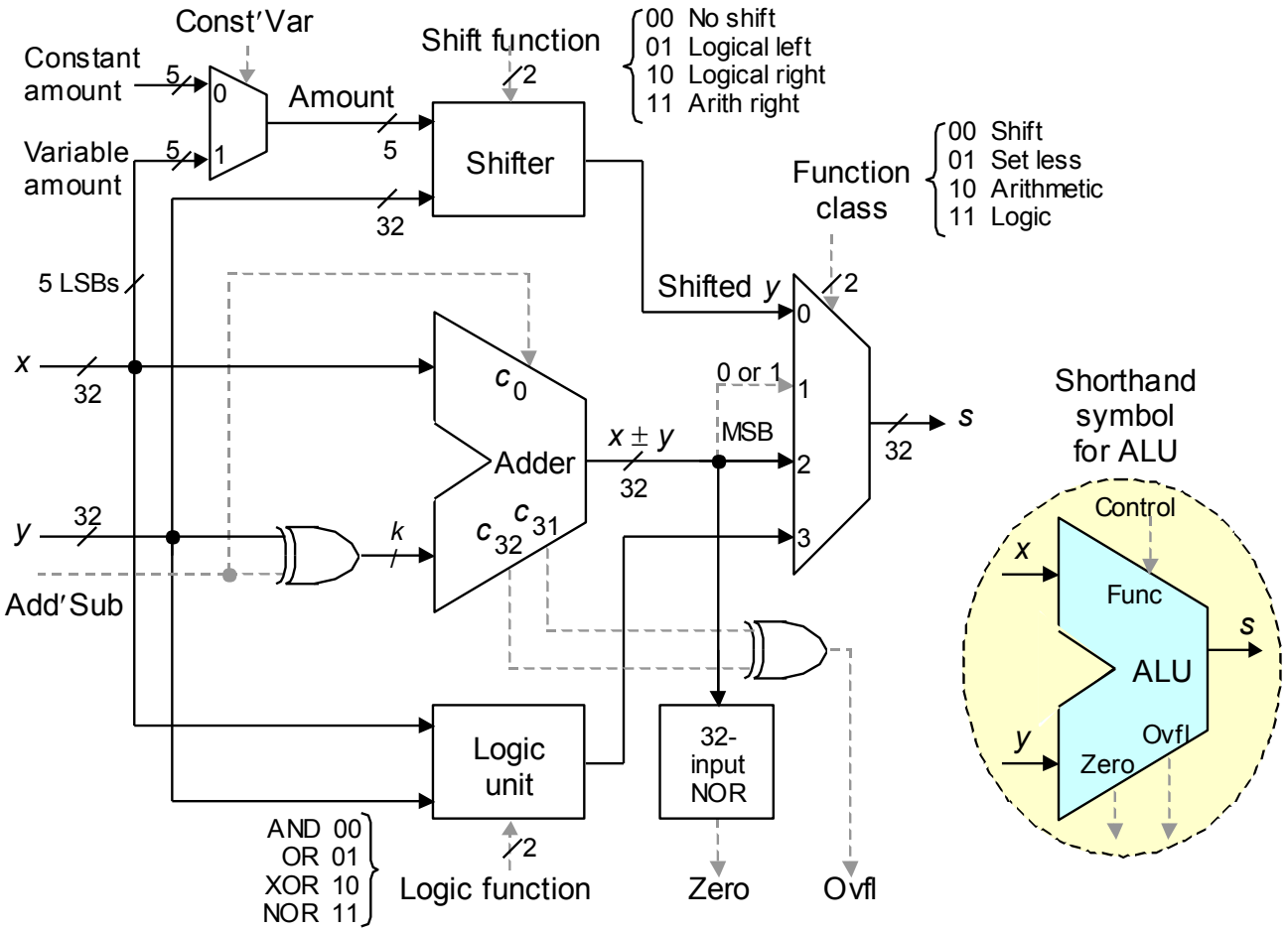
Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

UAL-uri multifuncționale



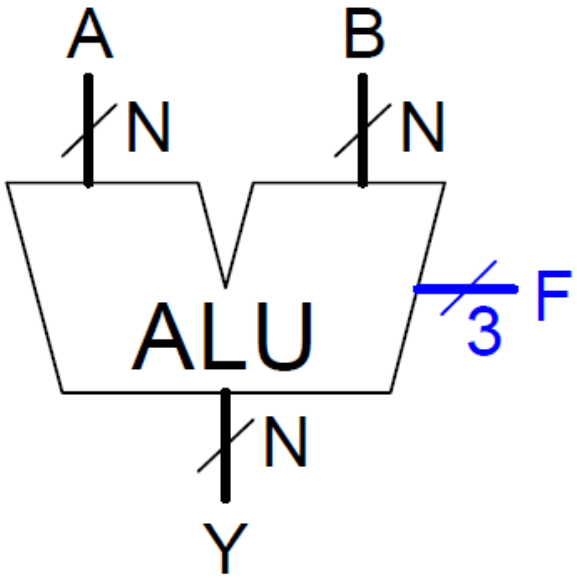
Structura generală a unei unități aritmetice-logice simple

Un UAL complet



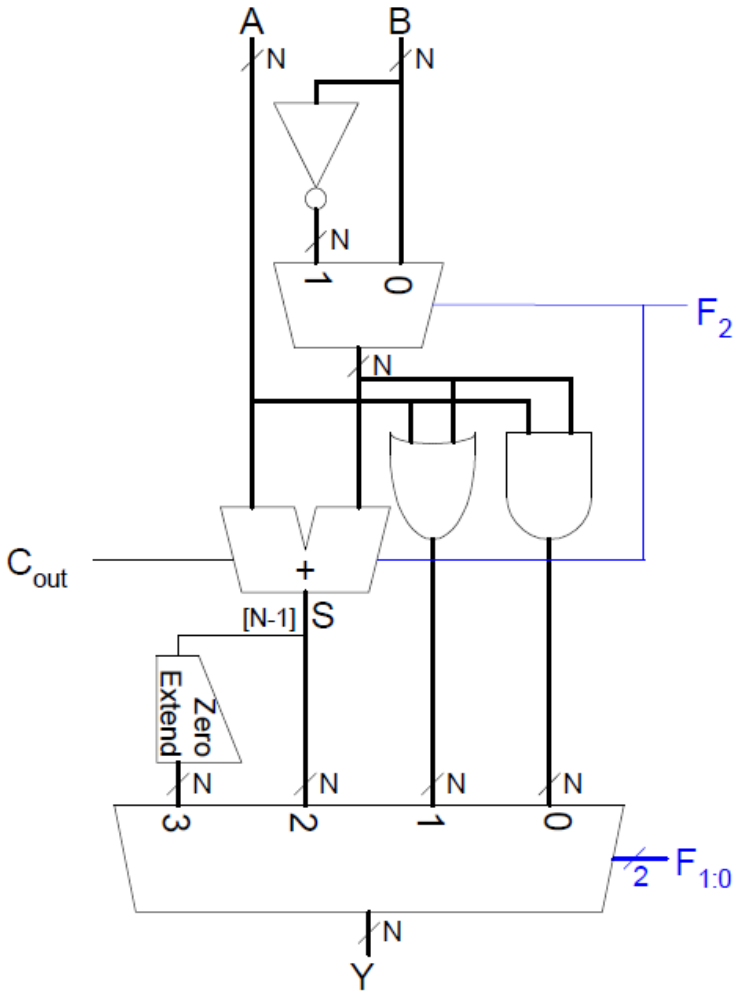
Un UAL multifuncțional cu 8 semnale de control (2 pentru tipul funcției, 1 pt aritmetică, 3 pt shiftare, 2 pt. op. logice) care specifică tipul operației.

Arithmetic Logic Unit (ALU)



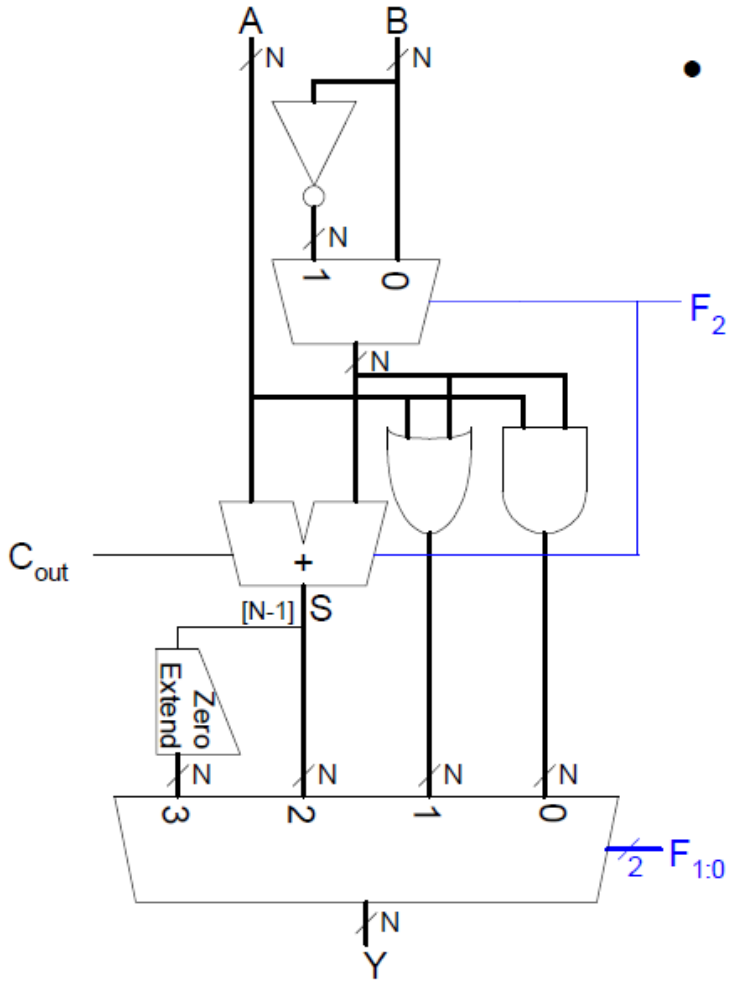
$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

ALU Design



$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

Exemplu Set Less Than (SLT)



Configurăm un ALU pe 32-biți pentru operația SLT: $A = 25$ și $B = 32$

- $A < B$, deci Y trebuie să fie 1 pe 32-biți (0x00000001)
- $F_{2:0} = 111$
- $F_2 = 1$ (sumatorul se comportă ca scăzător), deci 25 - 32 = -7
- -7 are 1 ca MSB ($S_{31} = 1$)
- $F_{1:0} = 11$, mux-ul selectează $Y = S_{31}$ (zero extended) = 0x00000001.

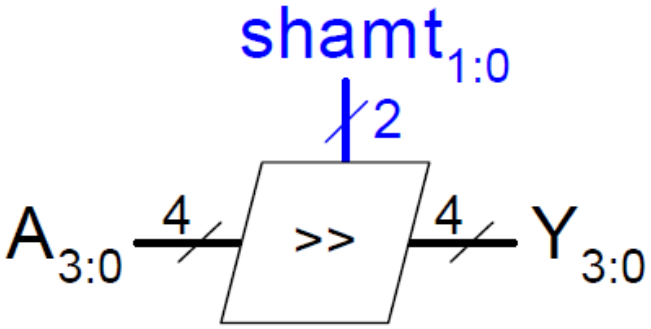
Deplasare

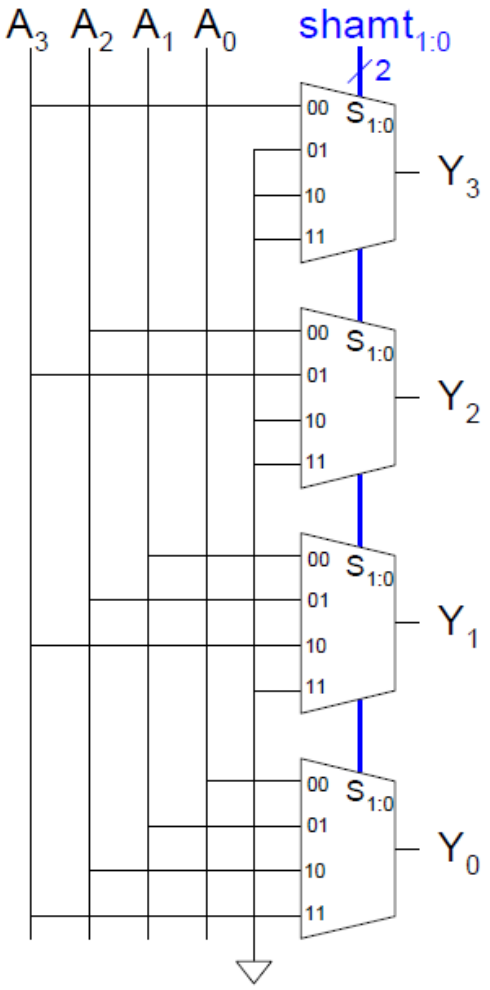
- **Logical shifter:** deplasează valoarea la stânga sau dreapta și umple spațiile goale cu 0-uri
  - Ex: 11001 >> 2 =
  - Ex: 11001 << 2 =
- **Arithmetic shifter:** la fel ca logical shifter, dar la shift dreapta umple spațiile cu vechiul MSB.
  - Ex: 11001 >>> 2 =
  - Ex: 11001 <<< 2 =
- **Rotator:** rotește biții în cerc a.î. biții care “ies” pe o parte “intră” pe partea cealaltă
  - Ex: 11001 ROR 2 =
  - Ex: 11001 ROL 2 =

Deplasare

- **Logical shifter:**
  - Ex: 11001 >> 2 = 00110
  - Ex: 11001 << 2 = 00100
- **Arithmetic shifter:**
  - Ex: 11001 >>> 2 = **11**110
  - Ex: 11001 <<< 2 = 00100
- **Rotator:**
  - Ex: 11001 ROR 2 = 01110
  - Ex: 11001 ROL 2 = 00111

Shifter Design





Deplasatoare ca Multiplicatoare, Divizoare

- $A << N = A \times 2^N$
  - **Exemplu:** 00001 << 2 = 00100 ($1 \times 2^2 = 4$)
  - **Exemplu:** 11101 << 2 = 10100 ($-3 \times 2^2 = -12$)
- $A >> N = A \div 2^N$
  - **Exemplu:** 01000 >> 2 = 00010 ($8 \div 2^2 = 2$)
  - **Exemplu:** 10000 >> 2 = 11100 ($-16 \div 2^2 = -4$)

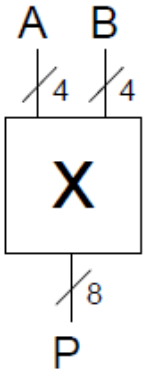
Multiplicatoare

- **Produse parțiale** formate prin multiplicarea unui singur rang binar
- **Deplasăm** produsele parțiale și le **sumăm** pentru a ajunge la rezultat

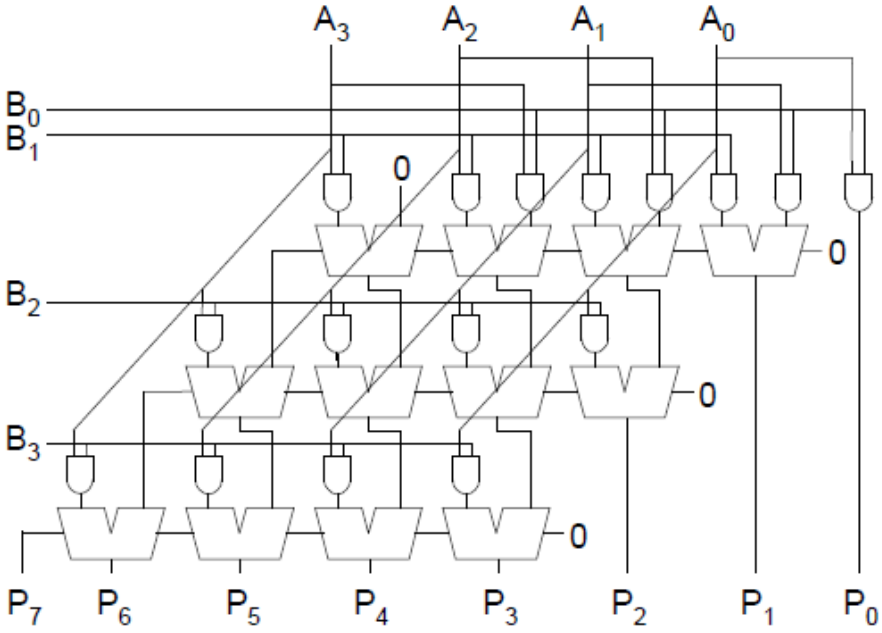
Decimal		Binary
230	multiplicand	0101
x 42	multiplier	x 0111
460	partial products	0101
+ 920		0101
9660		0101
		+ 0000
	result	0100011

230 x 42 = 9660 5 x 7 = 35

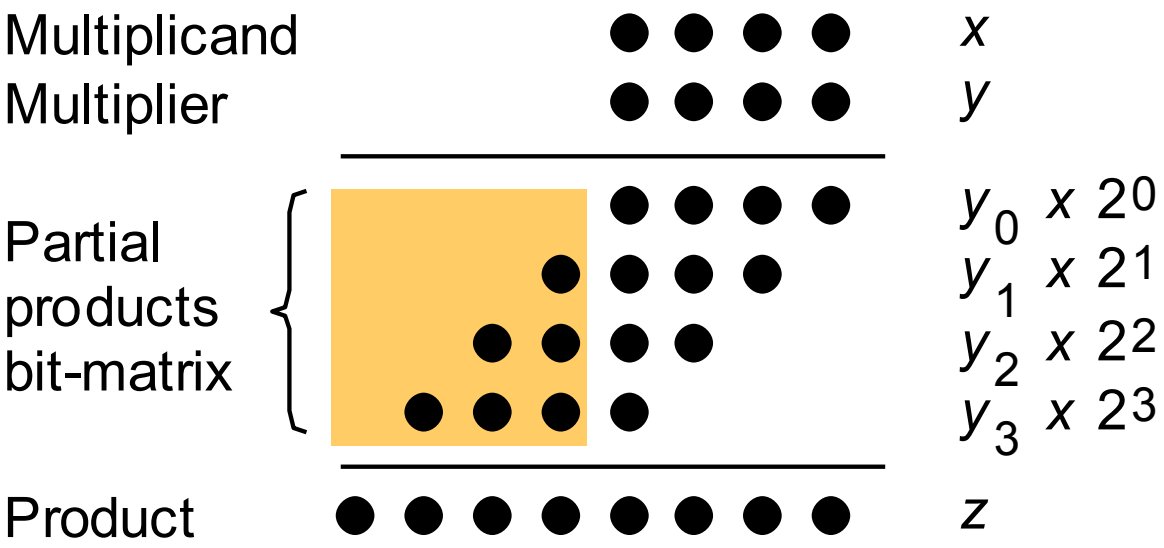
Multiplicator 4 x 4



$$
\begin{array}{rcccccccc}
 & & & A_3 & A_2 & A_1 & A_0 & & \\
\times & & & B_3 & B_2 & B_1 & B_0 & & \\
\hline
 & & & A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 & & \\
 & & A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 & & & \\
 & A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 & & & & \\
+ & A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 & & & & \\
\hline
P_7 & P_6 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0
\end{array}
$$



Înmulțire cu deplasări/adunări repetate



Înmulțirea a două numere de 4 biți în notație punct.

$$z^{(j+1)} = (z^{(j)} + y_j \, x \, 2^k) \, 2^{-1} \quad \text{unde } z^{(0)} = 0 \text{ și } z^{(k)} = z$$

|—— add ——|

|— shift right —|

Înmulțirea binară și zecimală

Position	7	6	5	4	3	2	1	0
$x2^4$		1	0	1	0			
y					0	0	1	1
$z^{(0)}$		0	0	0	0			
$+y_0x2^4$		1	0	1	0			
$2z^{(1)}$	0	1	0	1	0			
$z^{(1)}$		0	1	0	1	0		
$+y_1x2^4$		1	0	1	0			
$2z^{(2)}$	0	1	1	1	1	0		
$z^{(2)}$		0	1	1	1	1	0	
$+y_2x2^4$		0	0	0	0			
$2z^{(3)}$	0	0	1	1	1	1	0	
$z^{(3)}$		0	0	1	1	1	1	0
$+y_3x2^4$		0	0	0	0			
$2z^{(4)}$	0	0	0	1	1	1	1	0
$z^{(4)}$		0	0	0	1	1	1	0

Position	7	6	5	4	3	2	1	0
$x10^4$		3	5	2	8			
y					4	0	6	7
$z^{(0)}$		0	0	0	0			
$+y_0x10^4$	2	4	6	9	6			
$10z^{(1)}$	2	4	6	9	6			
$z^{(1)}$	0	2	4	6	9	6		
$+y_1x10^4$	2	1	1	6	8			
$10z^{(2)}$	2	3	6	3	7	6		
$z^{(2)}$		2	3	6	3	7	6	
$+y_2x10^4$	0	0	0	0	0			
$10z^{(3)}$	0	2	3	6	3	7	6	
$z^{(3)}$		0	2	3	6	3	7	6
$+y_3x10^4$	1	4	1	1	2			
$10z^{(4)}$	1	4	3	4	8	3	7	6
$z^{(4)}$		1	4	3	4	8	3	7	6

Exemple de înmulțire pas cu pas a două numere fără semn pe 4 biți.

Înmulțirea în cod complement al lui 2

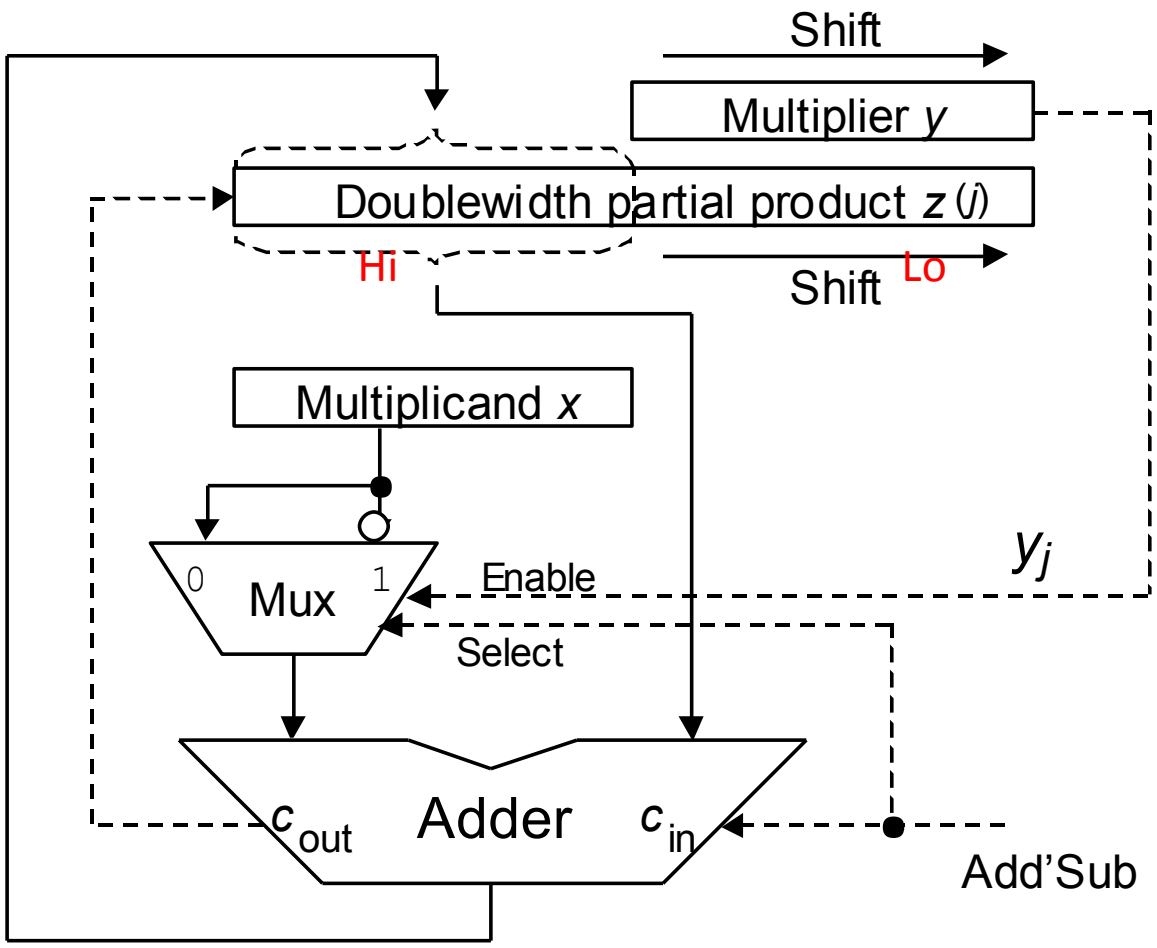
Embedded Systems Laboratory

Position	7	6	5	4	3	2	1	0
$x2^4$	1	0	1	0				
y					0	0	1	1
$z^{(0)}$	0	0	0	0	0			
$+y_0x2^4$	1	1	0	1	0			
$2z^{(1)}$	1	1	0	1	0			
$z^{(1)}$	1	1	1	0	1	0		
$+y_1x2^4$	1	1	0	1	0			
$2z^{(2)}$	1	0	1	1	1	0		
$z^{(2)}$	1	1	0	1	1	1	0	
$+y_2x2^4$	0	0	0	0	0			
$2z^{(3)}$	1	1	0	1	1	1	0	
$z^{(3)}$	1	1	1	0	1	1	1	0
$+(-y_3x2^4)$	0	0	0	0	0			
$2z^{(4)}$	1	1	1	0	1	1	1	0
$z^{(4)}$		1	1	1	0	1	1	1	0

Position	7	6	5	4	3	2	1	0
$x2^4$	1	0	1	0				
y					1	0	1	1
$z^{(0)}$	0	0	0	0	0			
$+y_0x2^4$	1	1	0	1	0			
$2z^{(1)}$	1	1	0	1	0			
$z^{(1)}$	1	1	1	0	1	0		
$+y_1x2^4$	1	1	0	1	0			
$2z^{(2)}$	1	0	1	1	1	0		
$z^{(2)}$	1	1	0	1	1	1	0	
$+y_2x2^4$	0	0	0	0	0			
$2z^{(3)}$	1	1	0	1	1	1	0	
$z^{(3)}$	1	1	1	0	1	1	1	0
$+(-y_3x2^4)$	0	0	1	1	0			
$2z^{(4)}$	0	0	0	1	1	1	1	0
$z^{(4)}$		0	0	0	1	1	1	0

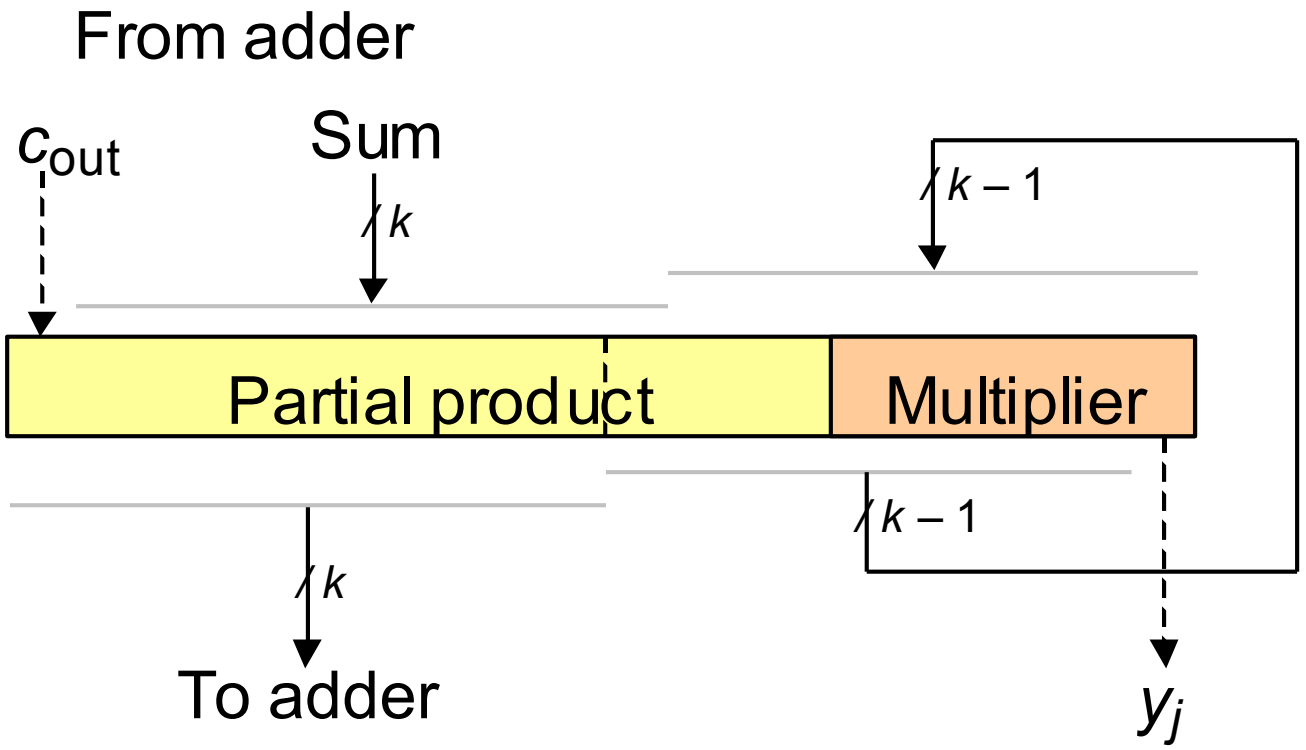
Exemple de înmulțire pentru două numere reprezentate în cod complement al lui 2

Multiplicatoare hardware



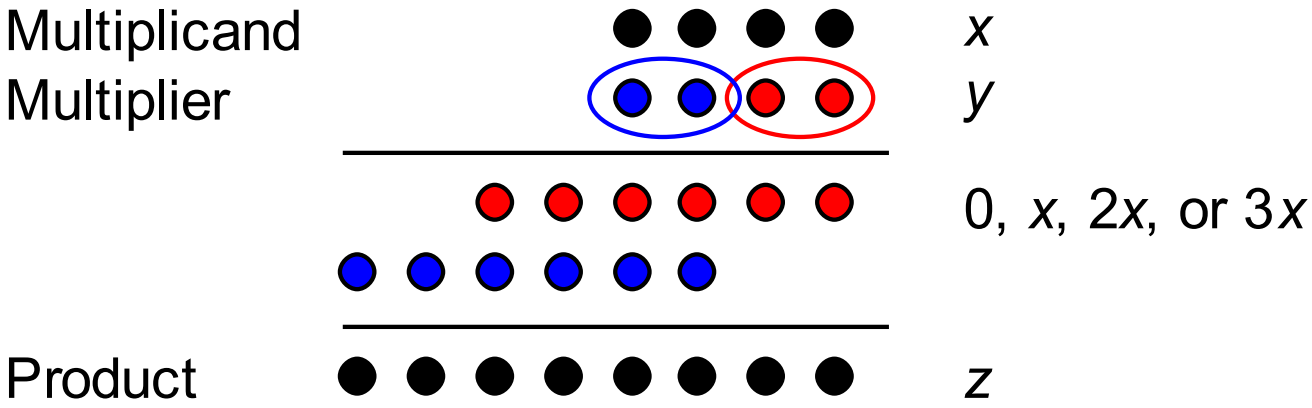
Multiplicator hardware bazat pe shiftări și adunări repetate.

Modulul de shiftare



Shiftarea este încorporată în circuitele de calcul ale produsului parțial pentru un plus de eficiență.

Multplicatoare de ordin superior



Înmulțire de ordin superior.

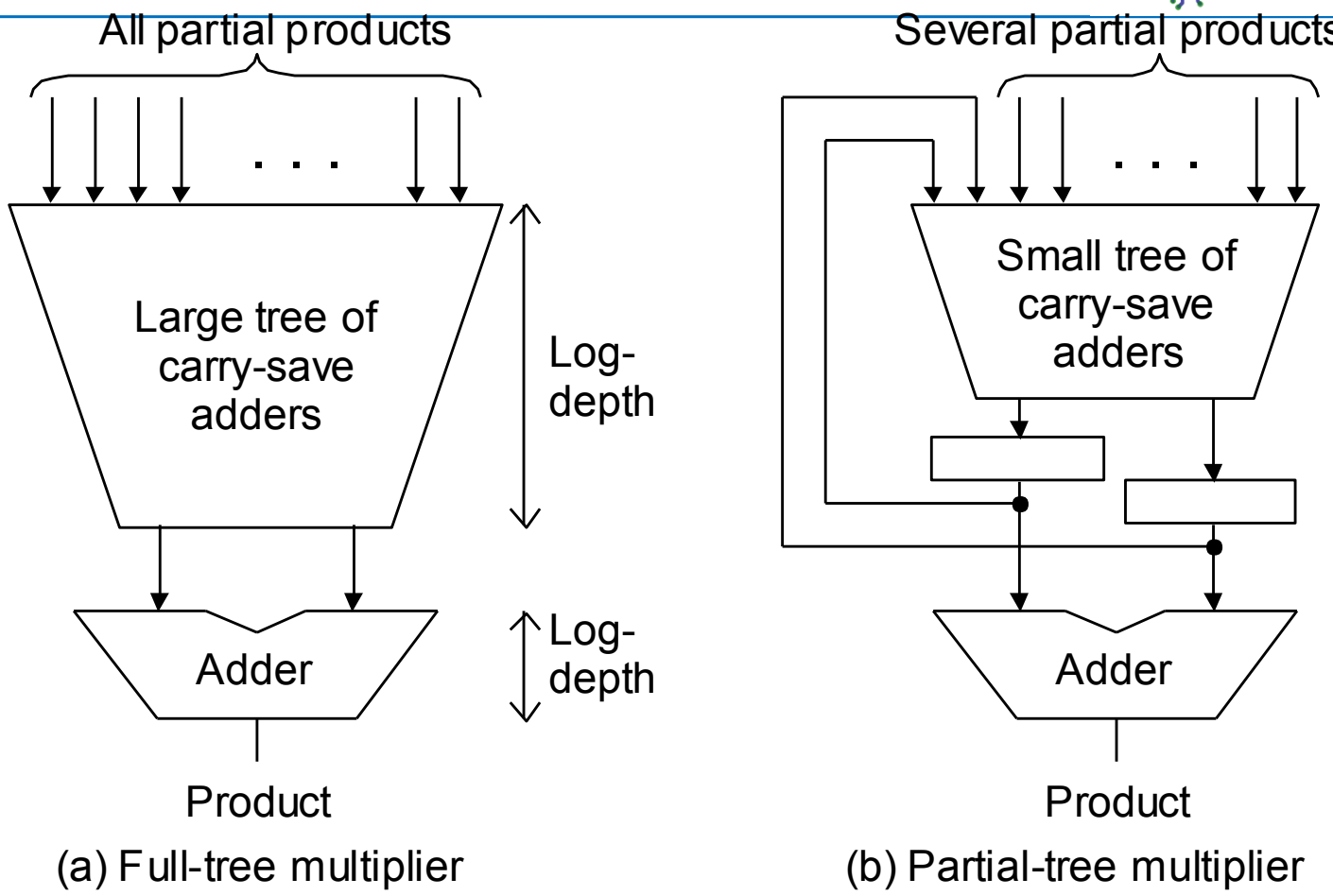
$z^{(j+1)} = (z^{(j)} + y_j \, x \, 2^k) \, 4^{-1}$ unde $z^{(0)} = 0$ și $z^{(k/2)} = z$

|—— add ——|

|— shift right —|

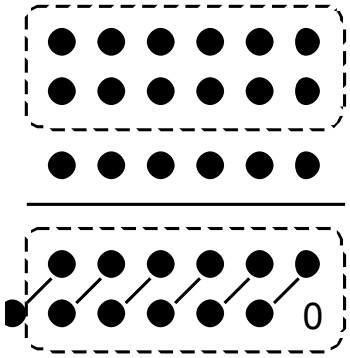
Presupunem *k* este par

Multiplicatoare arborescente

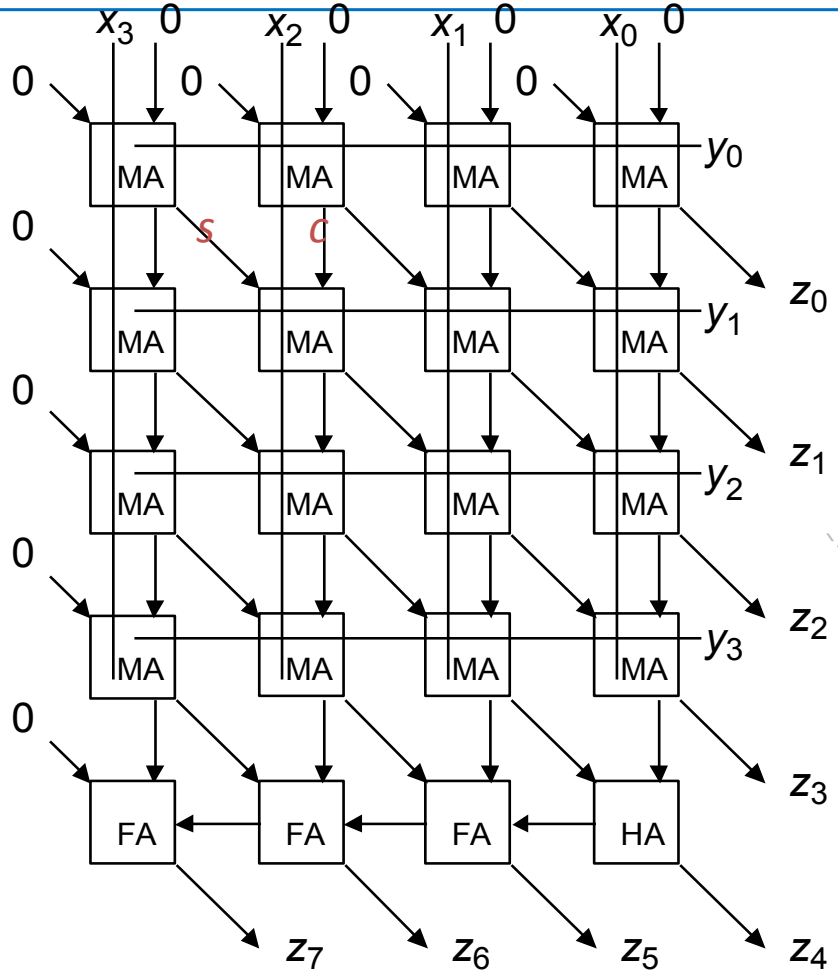


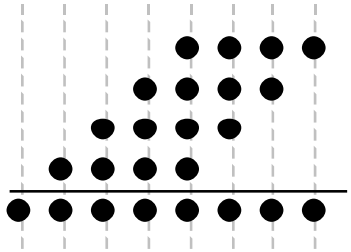
Schema unui multiplicator complet/parțial cu structură arborescentă.

Multiplicatoare matriceale

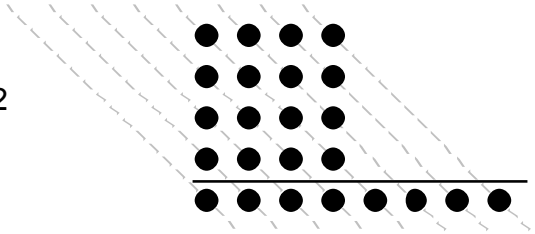


Similar cu sumatoarele carry-save





Our original dot-notation representing multiplication



Straightened dots to depict array multiplier to the left

Multiplicator matriceal pentru operanzi de 4 biți fără semn.

Multiplicatoare programate

Embedded Systems Laboratory

Instrucțiuni pentru înmulțire în assembler

```
mult  $s0,$s1     # set Hi,Lo to ($s0)×($s1); signed
multu $s2,$s3     # set Hi,Lo to ($s2)×($s3); unsigned
mfhi  $t0         # set $t0 to (Hi)
mflo  $t1         # set $t1 to (Lo)
```

Determinarea unui produs de 32 de biți pentru operanzi întregi de 32 de biți

Multiply -> rezultatul este obținut în `Hi,Lo`

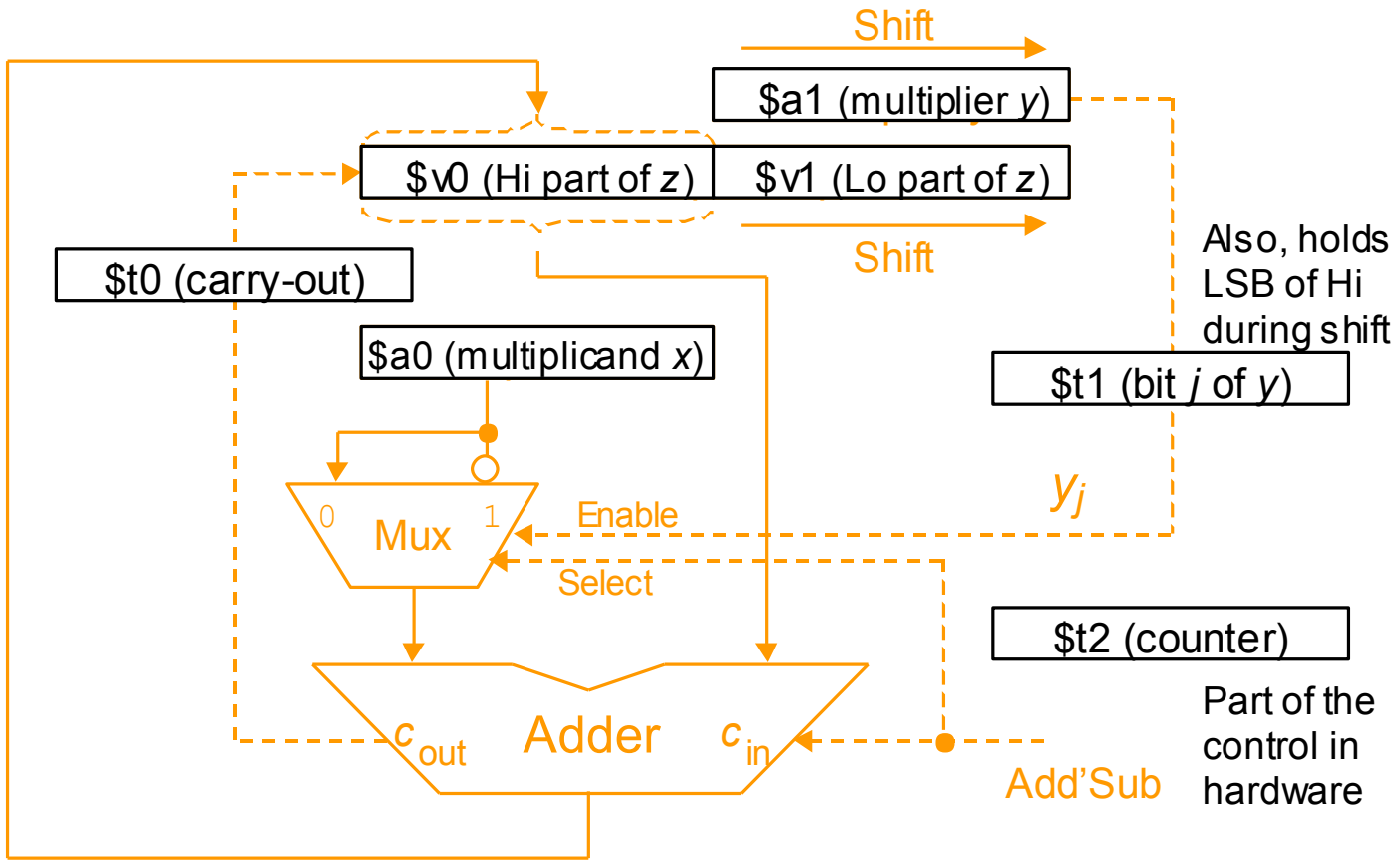
Pentru înmulțire fără semn:

`Hi` trebuie să fie numai 0 și `Lo` stochează rezultatul de 32 de biți

Pentru înmulțire cu semn:

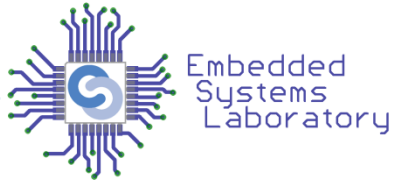
`Hi` trebuie să fie numai 0 sau numai 1, în funcție de semnul operandului din `Lo`

Emularea în softare a unui multiplicator



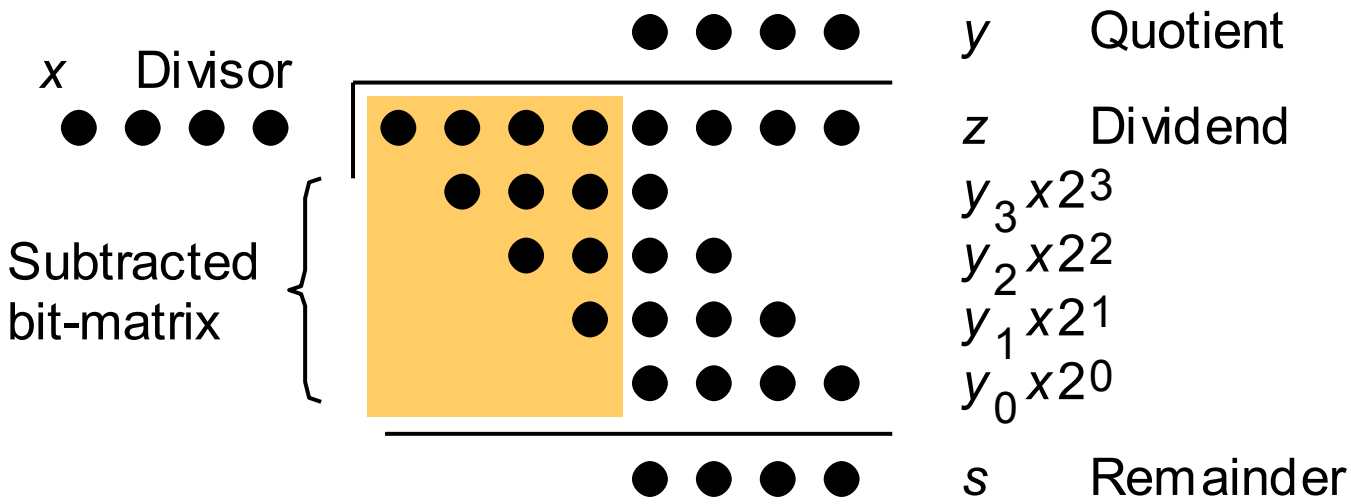
Registrele folosite în program supraimpuse peste structura hardware a unui multiplicator.

Rutină de înmulțire pentru ISA fără operație de mul




```
shamu:  move $v0,$zero              # initialize Hi to 0
        move $v1,$zero              # initialize Lo to 0
        addi $t2,$zero,32 # init repetition counter to 32
mloop:  move $t0,$zero              # set c-out to 0 in case of no add
        move $t1,$a1                # copy ($a1) into $t1
        srl  $a1,1                  # halve the unsigned value in $a1
        subu $t1,$t1,$a1  # subtract ($a1) from ($t1) twice to
        subu $t1,$t1,$a1  # obtain LSB of ($a1), or y[j], in $t1
        beqz $t1,noadd              # no addition needed if y[j] = 0
        addu $v0,$v0,$a0  # add x to upper part of z
        sltu $t0,$v0,$a0  # form carry-out of addition in $t0
noadd:  move $t1,$v0                # copy ($v0) into $t1
        srl  $v0,1                  # halve the unsigned value in $v0
        subu $t1,$t1,$v0  # subtract ($v0) from ($t1) twice to
        subu $t1,$t1,$v0  # obtain LSB of Hi in $t1
        sll  $t0,$t0,31             # carry-out converted to 1 in MSB of $t0
        addu $v0,$v0,$t0  # right-shifted $v0 corrected
        srl  $v1,1                  # halve the unsigned value in $v1
        sll  $t1,$t1,31             # LSB of Hi converted to 1 in MSB of $t1
        addu $v1,$v1,$t1  # right-shifted $v1 corrected
        addi $t2,$t2,-1             # decrement repetition counter by 1
        bne  $t2,$zero,mloop        # if counter > 0, repeat multiply loop
        jr   $ra          # return to the calling program
```

Împărțirea prin shiftări și scăderi repetate



Împărțirea unui număr de 8 biți cu un număr de 4 biți în notație cu puncte.

$$z^{(j)} = 2z^{(j-1)} - y_{k-j}\, x\, 2^k \quad \text{unde } z^{(0)} = z \text{ și } z^{(k)} = 2^k\, s$$

|shift|

|— subtract —|

Împărțirea întregilor și a numerelor fracționare

Embedded Systems Laboratory

Position	7	6	5	4	3	2	1	0	
z	0	1	1	1	0	1	0	1	
$x2^4$	1	0	1	0					
$z^{(0)}$	0	1	1	1	0	1	0	1	
$2z^{(0)}$	0	1	1	1	0	1	0	1	
$-y_3x2^4$	1	0	1	0					$y_3=1$
$z^{(1)}$	0	1	0	0	1	0	1		
$2z^{(1)}$	0	1	0	0	1	0	1		
$-y_2x2^4$	0	0	0	0					$y_2=0$
$z^{(2)}$	1	0	0	1	0	1			
$2z^{(2)}$	1	0	0	1	0	1			
$-y_1x2^4$	1	0	1	0					$y_1=1$
$z^{(3)}$	1	0	0	0	1				
$2z^{(3)}$	1	0	0	0	1				
$-y_0x2^4$	1	0	1	0					$y_0=1$
$z^{(4)}$	0	1	1	1					
s					0	1	1	1	
y					1	0	1	1	

Position	−1	−2	−3	−4	−5	−6	−7	−8	
z	.	1	4	3	5	1	5	0	2
x	.	4	0	6	7				
$z^{(0)}$	.	1	4	3	5	1	5	0	2
$10z^{(0)}$	1	.	4	3	5	1	5	0	2
$-y_{-1}x$	1	.	2	2	0	1			$y_{-1}=3$
$z^{(1)}$			.	2	1	5	0	5	0 2
$10z^{(1)}$			2	.	1	5	0	5	0 2
$-y_{-2}x$	2	.	0	3	3	5			$y_{-2}=5$
$z^{(2)}$				.	1	1	7	0	0 2
$10z^{(2)}$				1	.	1	7	0	0 2
$-y_{-3}x$	0	.	8	1	3	4			$y_{-3}=2$
$z^{(3)}$					.	3	5	6	6 2
$10z^{(3)}$	3	.	5	6	6	2			
$-y_{-4}x$	3	.	2	5	3	6			$y_{-4}=8$
$z^{(4)}$					.	3	1	2	6
s	.	0	0	0	0	3	1	2	6
y	.	3	5	2	8				

Exemple de împărțire pentru întregi și numere fracționare

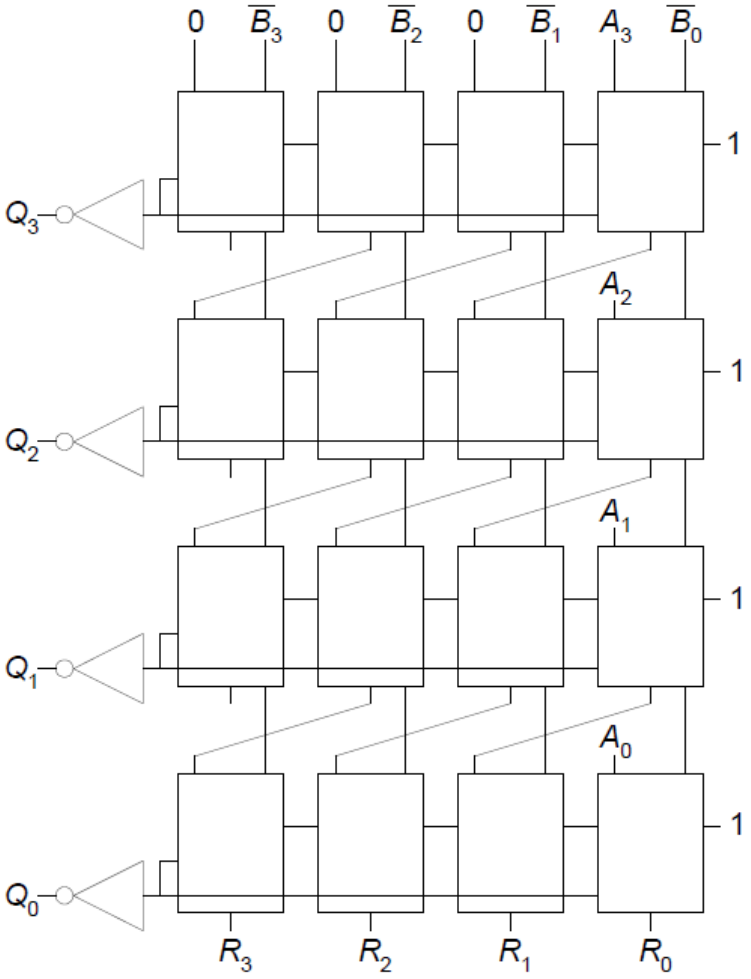
Împărțirea operanzilor cu aceeași dimensiune

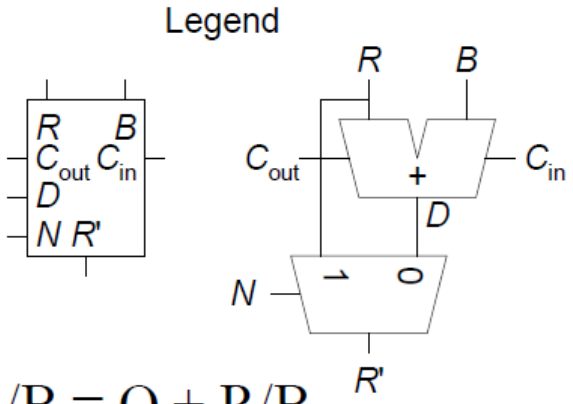
Position	7	6	5	4	3	2	1	0	
z	0	0	0	0	1	1	0	1	
$x2^4$	0	1	0	1					
$z^{(0)}$	0	0	0	0	1	1	0	1	
$2z^{(0)}$	0	0	0	1	1	0	1		
$-y_3x2^4$	0	0	0	0					$y_3=0$
$z^{(1)}$	0	0	0	1	1	0	1		
$2z^{(1)}$	0	0	1	1	0	1			
$-y_2x2^4$	0	0	0	0					$y_2=0$
$z^{(2)}$	0	0	1	1	0	1			
$2z^{(2)}$	0	1	1	0	1				
$-y_1x2^4$	0	1	0	1					$y_1=1$
$z^{(3)}$	0	0	0	1	1				
$2z^{(3)}$	0	0	1	1					
$-y_0x2^4$	1	0	1	0					$y_0=0$
$z^{(4)}$	0	0	1	1					
s					0	0	1	1	
y					0	0	1	0	

Position	−1	−2	−3	−4	−5	−6	−7	−8	
z	.	0	1	0	1				
x	.	1	1	0	1				
$z^{(0)}$	.	0	1	0	1				
$2z^{(0)}$	0	.	1	0	1	0			
$-y_{-1}x$	0	.	0	0	0	0			$y_{-1}=0$
$z^{(1)}$			.	1	0	1	0		
$2z^{(1)}$		1	.	0	1	0	0		
$-y_{-2}x$	0	.	1	1	0	1			$y_{-2}=1$
$z^{(2)}$			.	0	1	1	1		
$2z^{(2)}$		0	.	1	1	1	0		
$-y_{-3}x$	0	.	1	1	0	1			$y_{-3}=1$
$z^{(3)}$			.	0	0	0	1		
$2z^{(3)}$	0	.	0	0	1	0			
$-y_{-4}x$	0	.	0	0	0	0			$y_{-4}=0$
$z^{(4)}$			.	0	0	1	0		
s	.	0	0	0	0	0	0	1	0
y	.	0	1	1	0				

Exemple de împărțire a întregilor și fracțiilor de 4 biți lungime.

Divizor 4 x 4





A/B = Q + R/B

Algorithm:

R' = 0

for i = N-1 to 0

  R = {R' << 1. A_i}

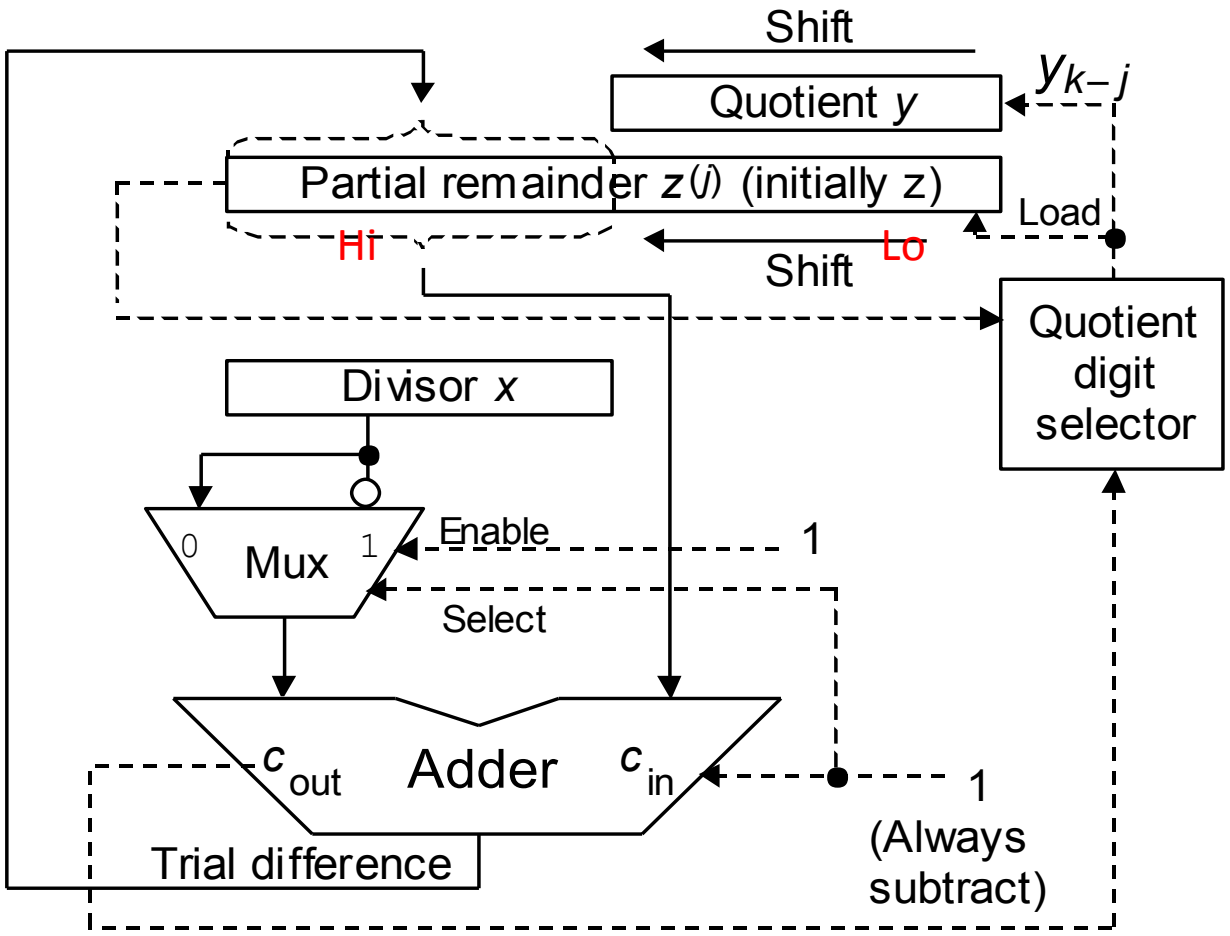
  D = R - B

  if D < 0, Q_i=0, R'=R

  else Q_i=1, R'=D

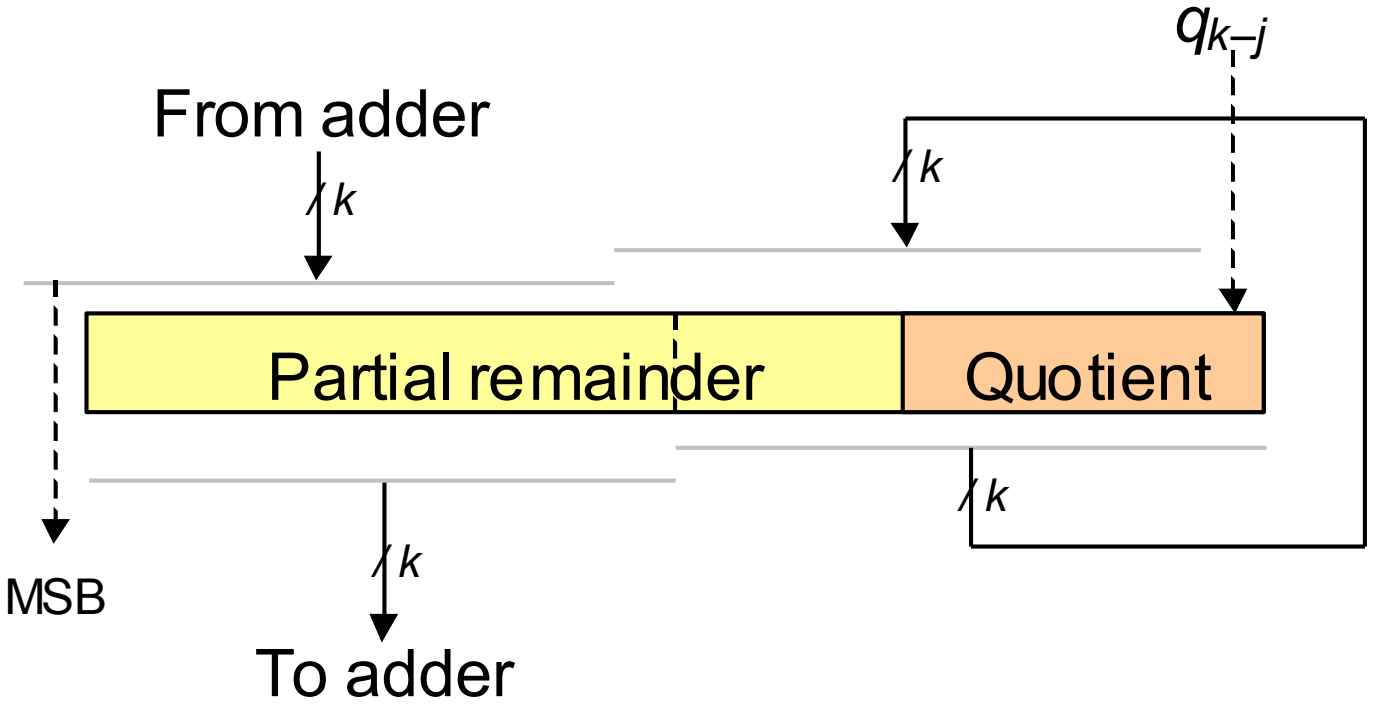
R'=R

Divizoare hardware



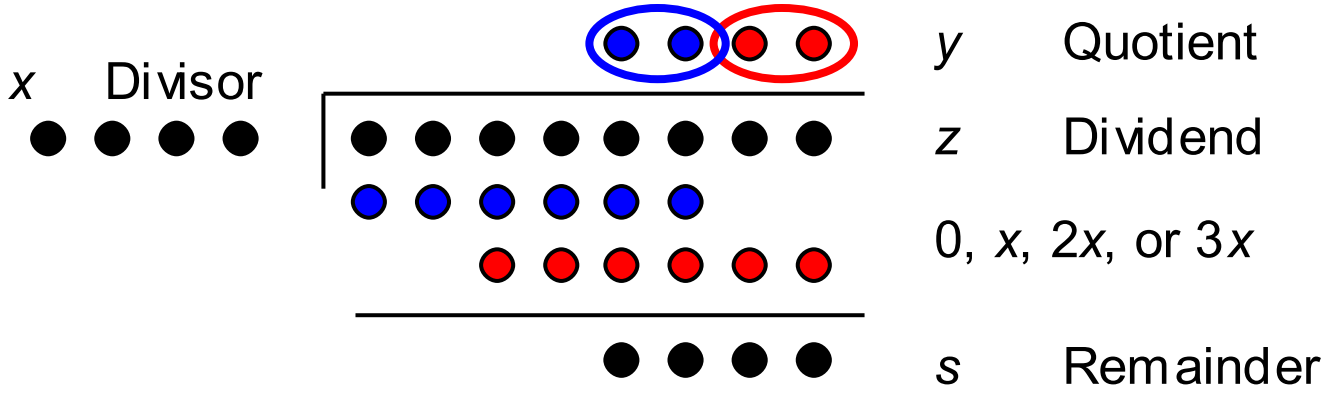
Divizor hardware bazat pe un algoritm cu shiftări și scăderi repetate

Shifter-ul din interiorul divizorului



Deplasarea este încorporată în circuitele pentru calculul rezultatului parțial pentru a aduce eficiență sporită.

Divizoare de ordin superior



Divizor de ordin 4 în notație cu puncte

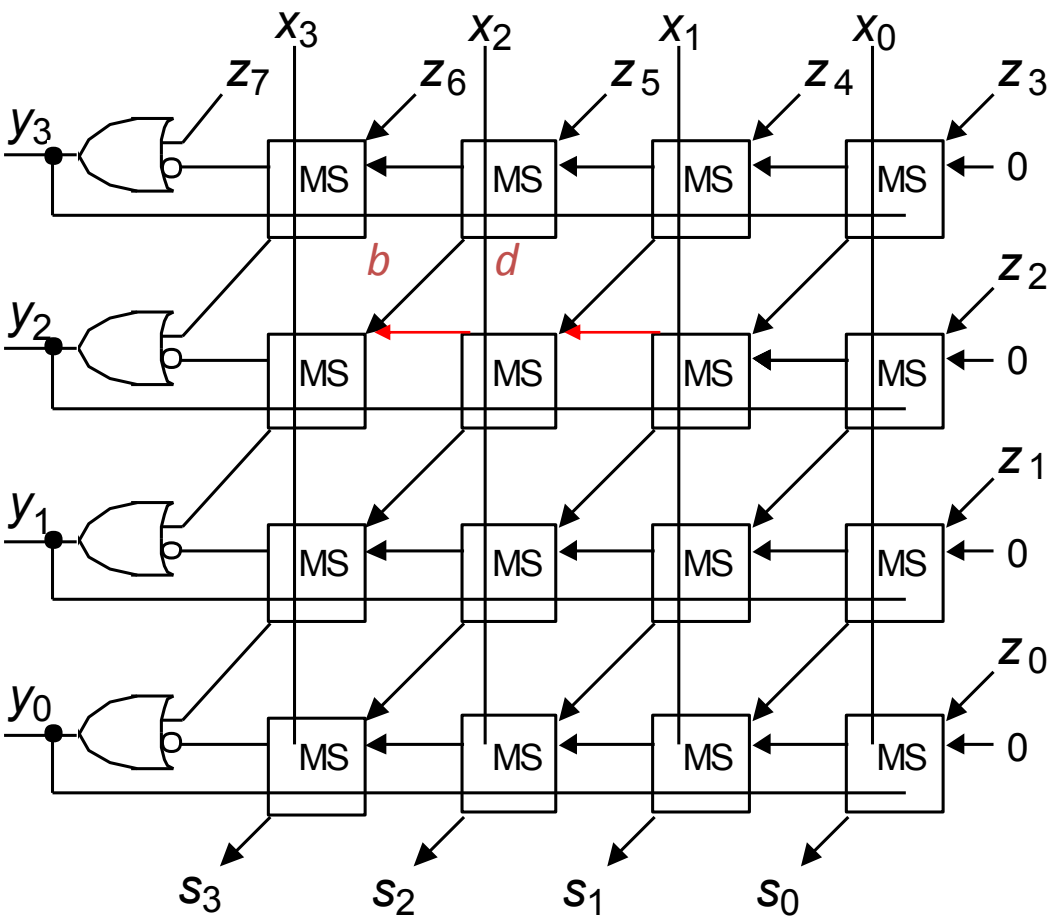
$$z^{(j)} = 4z^{(j-1)} - (y_{k-2j+1}\, y_{k-2j})_{\text{two}}\, x\, 2^k \quad \text{unde} \quad z^{(0)} = z \quad \text{și} \quad z^{(k/2)} = 2^k s$$

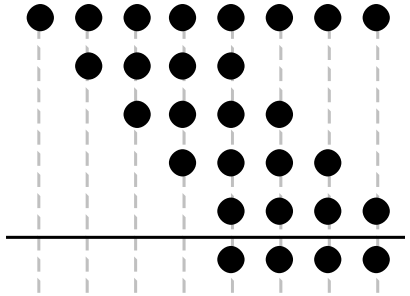
$|\text{shift}|$

$|$——— subtract ———$|$

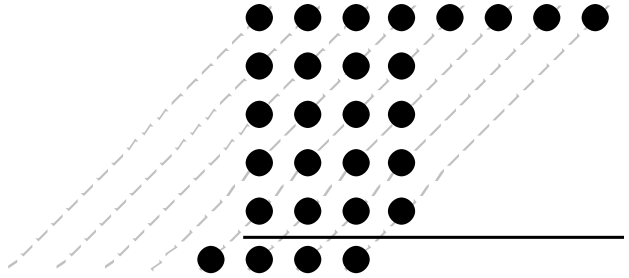
Presupunem k par

Divizoare matriceale





Our original dot-notation for division



Straightened dots to depict an array divider

Divizor matriceal. Un număr de 8 biți este împărțit la altul de 4 biți.

Împărțirea programată

Instrucțiuni în cod mașină pentru împărțire

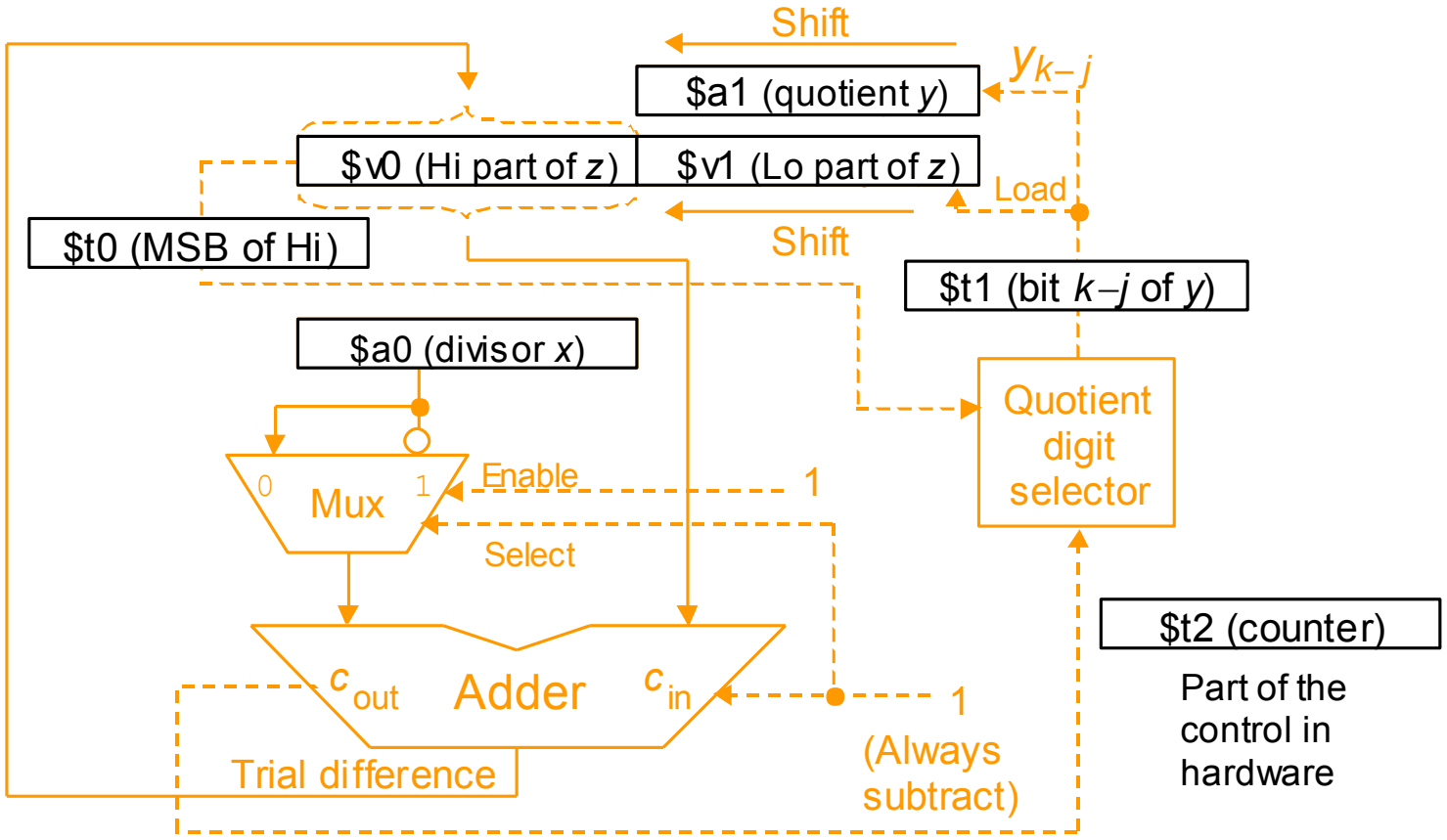
```
div   $s0,$s1     # Lo = quotient, Hi = remainder
divu  $s2,$s3     # unsigned version of division
mfhi  $t0         # set $t0 to (Hi)
mflo  $t1         # set $t1 to (Lo)
```

Calculează *z* mod *x*, unde *z* (cu semn) și *x* > 0 sunt întregi

Divide; remainder will be obtained in `Hi`

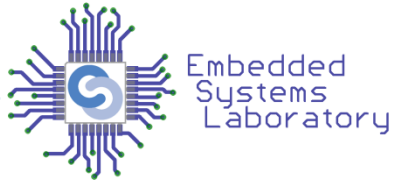
if remainder is negative,
then add |*x*| to (`Hi`) to obtain *z* mod *x*
else `Hi` holds *z* mod *x*

Emularea unui divizor hardware în software



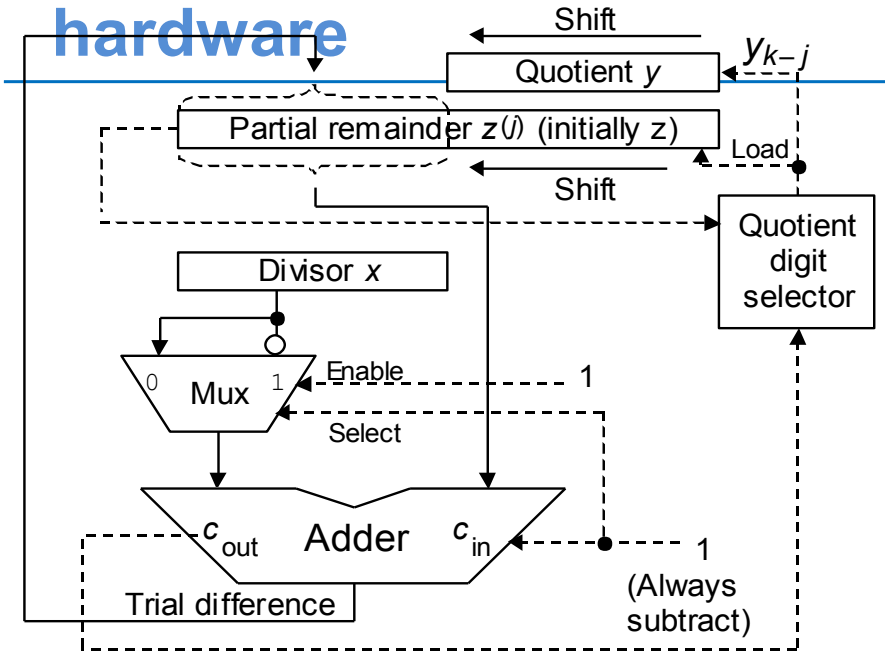
Folosirea registrelor pentru împărțirea programată suprapusă peste diagrama bloc a unui divizor hardware.

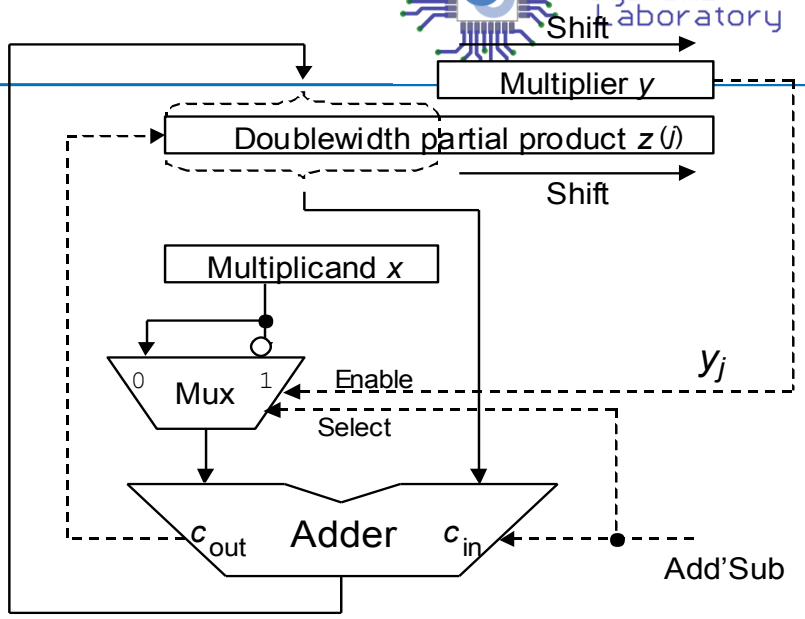
Rutină împărțire pentru ISA fără operația div

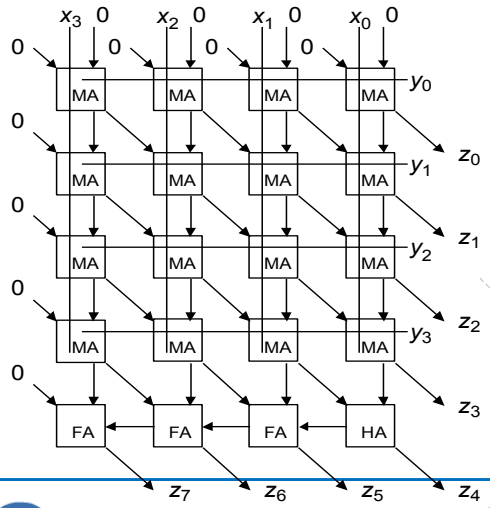


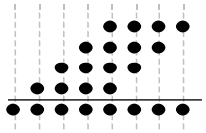

```
shsdi:  move $v0,$a2              # initialize Hi to ($a2)
        move $v1,$a3              # initialize Lo to ($a3)
        addi $t2,$zero,32 # initialize repetition counter to 32
dloop:  slt  $t0,$v0,$zero        # copy MSB of Hi into $t0
        sll  $v0,$v0,1            # left-shift the Hi part of z
        slt  $t1,$v1,$zero        # copy MSB of Lo into $t1
        or   $v0,$v0,$t1  # move MSB of Lo into LSB of Hi
        sll  $v1,$v1,1            # left-shift the Lo part of z
        sge  $t1,$v0,$a0  # quotient digit is 1 if (Hi) ≥ x,
        or   $t1,$t1,$t0  # or if MSB of Hi was 1 before shifting
        sll  $a1,$a1,1            # shift y to make room for new digit
        or   $a1,$a1,$t1  # copy y[k-j] into LSB of $a1
        beq  $t1,$zero,nosub      # if y[k-j] = 0, do not subtract
        subu $v0,$v0,$a0  # subtract divisor x from Hi part of z
nosub:  addi $t2,$t2,-1           # decrement repetition counter by 1
        bne  $t2,$zero,dloop      # if counter > 0, repeat divide loop
        move $v1,$a1              # copy the quotient y into $v1
        jr   $ra          # return to the calling program
```

Divizor vs Multiplicator: Similarități hardware

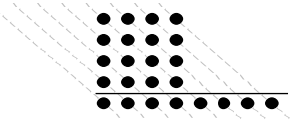




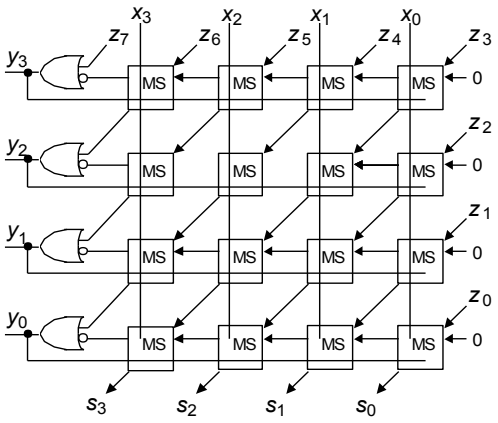


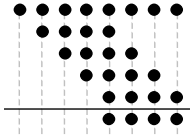


Our original dot-notation representing multiplication

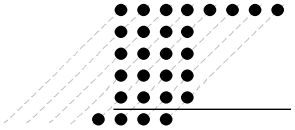


Straightened dots to depict array multiplier to the left





Our original dot-notation for division



Straightened dots to depict an array divider

Acknowledgements

- Aceste slide-uri conțin materiale aparținând:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)
  - Behrooz Parhami (UCSB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

Calculatoare Numerice

-Cursul 7 –

Implementarea unei mașini de calcul

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Comic of the day

http://dilbert.com/strips/comic/1990-04-10/

Din cursurile trecute

- Computer Architecture >> ISAs & RTL
  - Cursul se ocupă de interacțiunea hardware/software și de proiectarea nivelurilro potrivite de abstractizare
- Tehnologia și aplicațiile ei modelează arhitectura sistemelor de calcul
  - Trecutul poate da lecții pentru dezvoltarea tehnicii
- Primii 130 de ani de CompArch, de la Babbage la IBM 360
  - Evoluția de la calculatoare mecanice la mașini complet programabile
  - Schimbarea rapidă în timpul WWII (anii 45), tranziție de la procesoare electro-mecanice la electronice
- Costul software development devine o problemă pentru dezvoltarea arhitecturilor de calcul (era nevoie de compatibilitate)
- IBM 360 introduce noțiunea de “familie de calculatoare” cu același ISA care rulează pe mașini complet diferite hardware
  - Șase mașini diferite lansate în aceeaș zi (7 aprilie 1964)
  - “Future-proofing” pentru generațiile viitoare

IBM 360: Implementarea inițială

	Model 30	. . .	*Model 70*
Memory	8K - 64 KB		256K - 512 KB
Datapath	8-bit		64-bit
Circuit Delay	30 nsec/level		5 nsec/level
Local Store	Main Store		Transistor Registers
Control Store	Read only 1~sec		Conventional circuits

Arhitectura setului de instrucțiuni (ISA) IBM 360 ascundea complet diferențele de tehnologie dintre diferitele modele.

Milestone: Primul ISA proiectat ca o interfață portabilă între hardware și software!

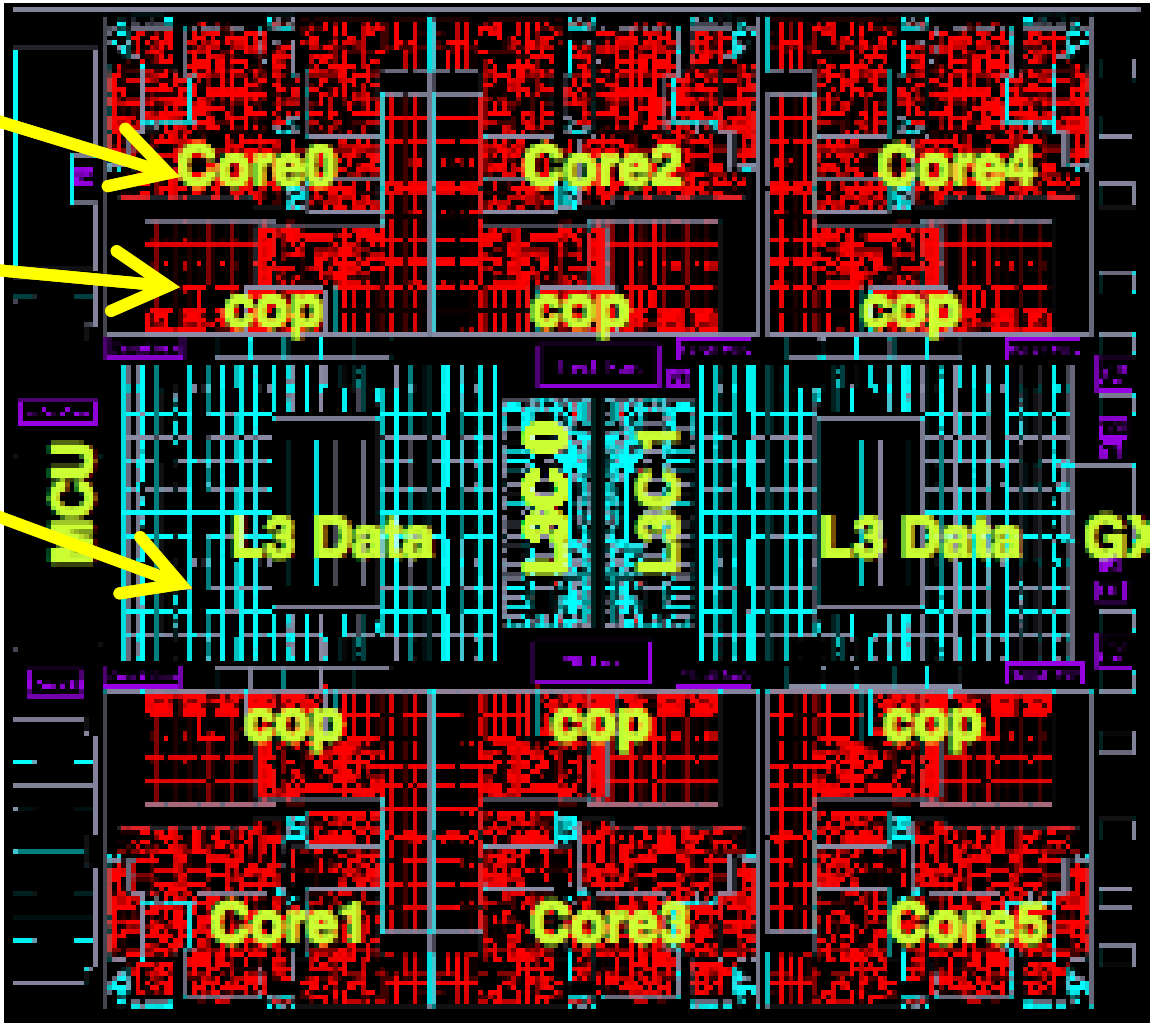
IBM 360 Supraviețuiește și azi: z12 Mainframe Processor

Embedded Systems Laboratory

6 Cores @ 5.5 GHz

Coprocesor special-purpose pe fiecare core

48MB Level-3 cache on chip



32nm SOI Technology
2.75 miliarde tranzistoare
23.7mm x 25.2mm
15 straturi de metal
12.3 km de conexiuni!
10,000 pini de alimentare (!)
1,071 I/O pins

[IBM HotChips24 presentation, August 28, 2012]

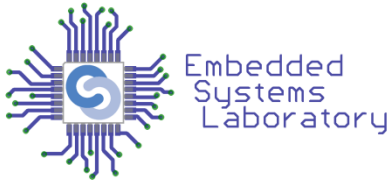
Instruction Set Architecture (ISA)

- Punctul de contact dintre software și hardware
- Descrisă în mod tipic prin punerea la dispoziția programatorilor a registrelor, memoriei și a semanticii instrucțiunilor care operează pe mașina respectivă
- IBM 360 a fost prima familie de mașini de calcul care a separat ISA de implementarea hardware propriu-zisă (aka. *microarchitectură*)
- Mai multe implementări posibile pentru aceeași ISA
  - E.g., sovieticii au construit clone compatibile ale IBM360, precum si Amdahl, după plecarea din IBM.
  - E.g.2., astăzi, procesoarele Intel și AMD folosesc ISA x86-64.
  - E.g.3: majoritatea telefoanelor mobile folosesc ISA ARM cu implementări hardware de la mai multe companii, printre care și TI, Qualcomm, Samsung, Marvell, etc.

Maparea ISA pe microarhitectură

- ISA poate fi proiectată ținând cont tipul de microarhitectură:
  - Acumulator ⇒ hardwired, unpipelined
  - CISC ⇒ microcoded
  - RISC ⇒ hardwired, pipelined
  - VLIW ⇒ fixed-latency in-order parallel pipelines
  - JVM ⇒ software interpretation
- Dar poate fi implementată folosind orice fel de microarhitectură
  - Intel Ivy Bridge: hardwired pipelined CISC (x86)
  - Simics: Software-interpreted SPARC RISC machine
  - ARM Jazelle: Procesor JVM hardware
  - În acest curs: mașină RISC-V cu microcod

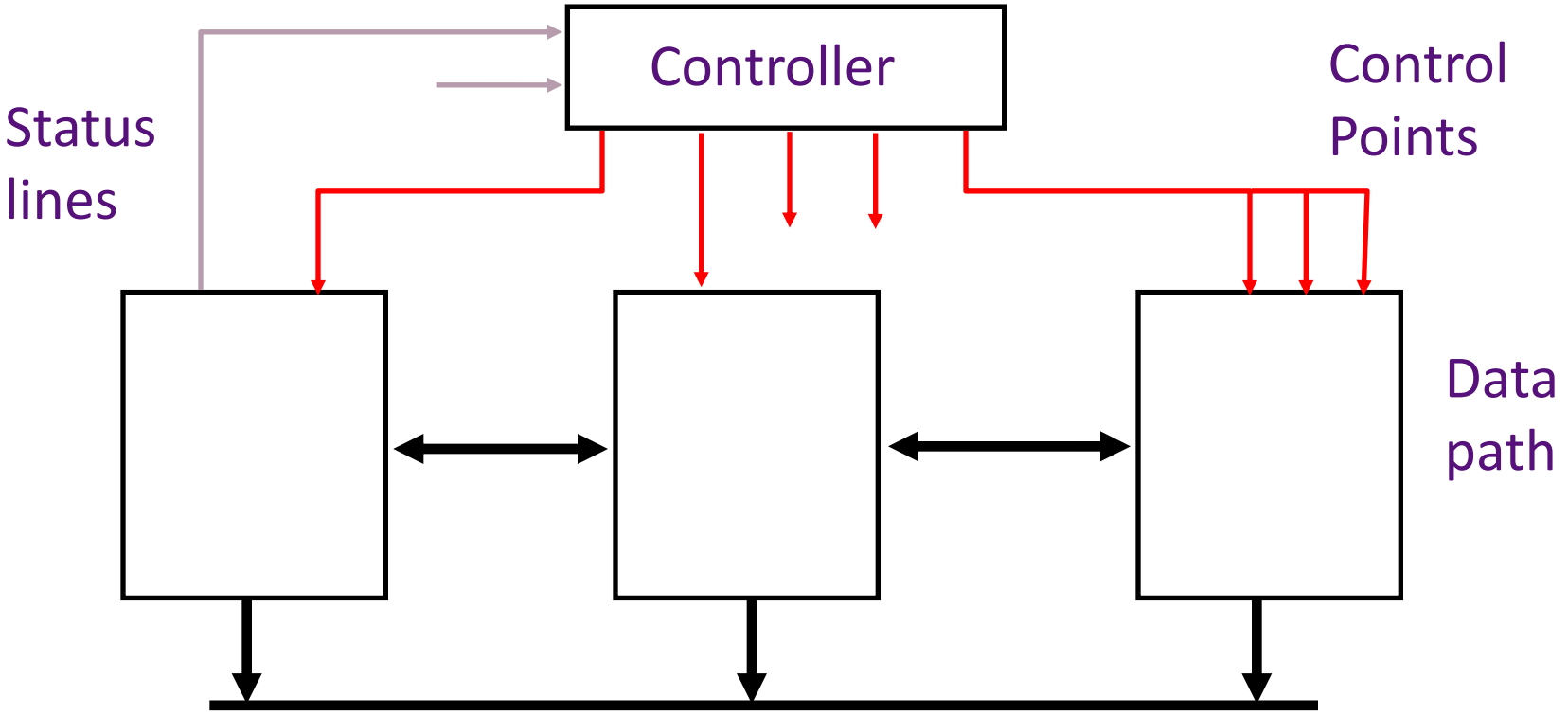
Microprogramarea în cursul de azi



- Pentru a arăta cum pot fi contruite procesoare mici care rulează un ISA complex
- Pentru a vă ajuta să înțelegeți de ce există mașini CISC*
- Pentru că este încă folosită la scară largă (IBM360, x86, PowerPC)
- Servește ca o introducere (mai) blândă în structura mașinilor de calcul
- Pentru a vă face să înțelegeți de ce tehnologia s-a centrat pe RISC*

* “CISC”/”RISC” sunt denumiri mult mai noi decât arhitecturile la care fac referință.

Microarchitectură:
Implementarea unei ISA



Structură: Cum sunt interconectate componentele.

Statică

Comportament: Cum se miscă datele între diferitele componente

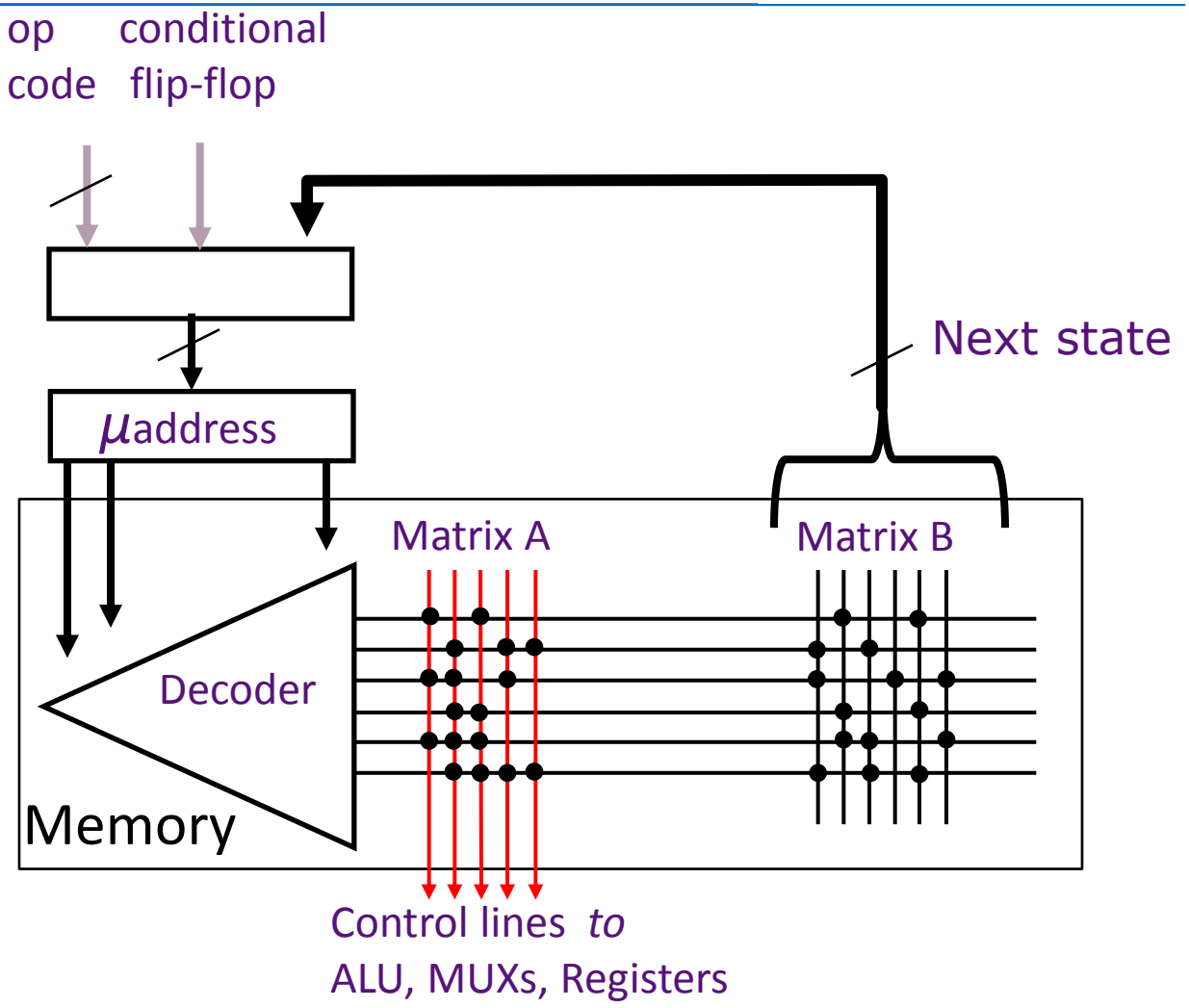
Dinamică

Unitate control microprogramat

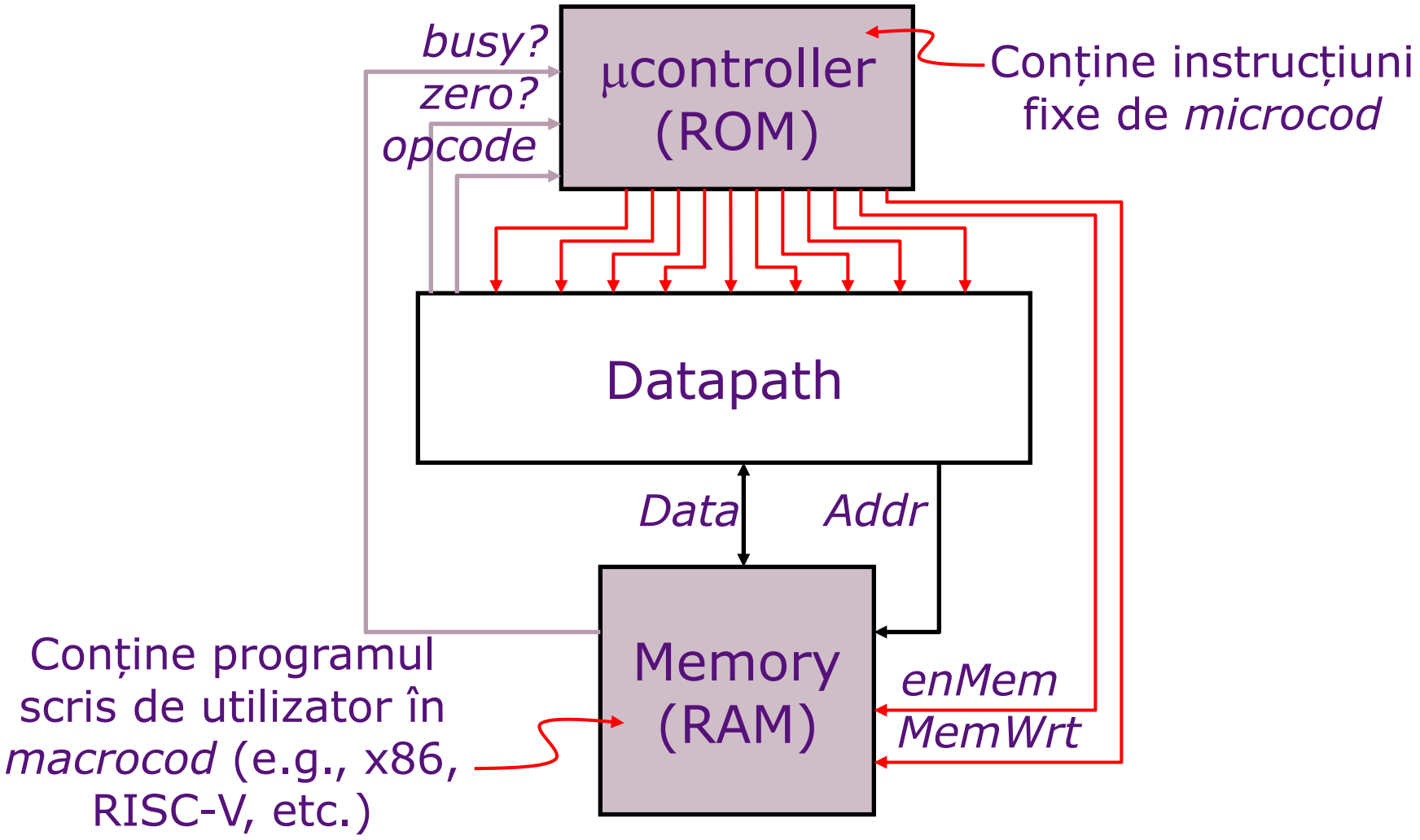
Maurice Wilkes, 1954

Folosită prima dată în EDSAC-2, 1958

Încorporează tabela de stări pentru logica de control într-o memorie



Microarchitectură microcodată



RISC-V ISA

- Design RISC de la UC Berkeley
- ISA complet dar de mici dimensiuni și deschis
- Nu este supra-optimizat pentru un anumit stil de implementare
- Două variante, pentru 32 și 64 de biți
  - RV32 & RV64
- Proiectat pentru multiprocesoare
- Codificare eficientă a instrucțiunilor
- Ușor de extins/restrâns pentru educație/cercetare

- Pentru acest curs vom folosi o arhitectură RISC-V pe 32 de biți, foarte similară cu a procesorului MIPS

RV32 Processor State

Embedded Systems Laboratory

Program counter (**pc**)

32x32-bit registre pt întregi (**x0-x31**)
- **x0** conține întotdeauna 0

32 registre floating-point (FP) (**f0-f31**)
- fiecare poate conține o valoare single- sau double-precision FP (32-bit sau 64-bit IEEE FP)

FP status register (**fsr**), folosit pentru FP rounding mode & raportare excepții

XPRLEN-1 … 0
x0 / zero
x1 / ra
x2
x3
x4
x5
x6
x7
x8
x9
x10
x11
x12
x13
x14
x15
x16
x17
x18
x19
x20
x21
x22
x23
x24
x25
x26
x27
x28
x29
x30
x31

XPRLEN

63 … 0
f0
f1
f2
f3
f4
f5
f6
f7
f8
f9
f10
f11
f12
f13
f14
f15
f16
f17
f18
f19
f20
f21
f22
f23
f24
f25
f26
f27
f28
f29
f30
f31

64

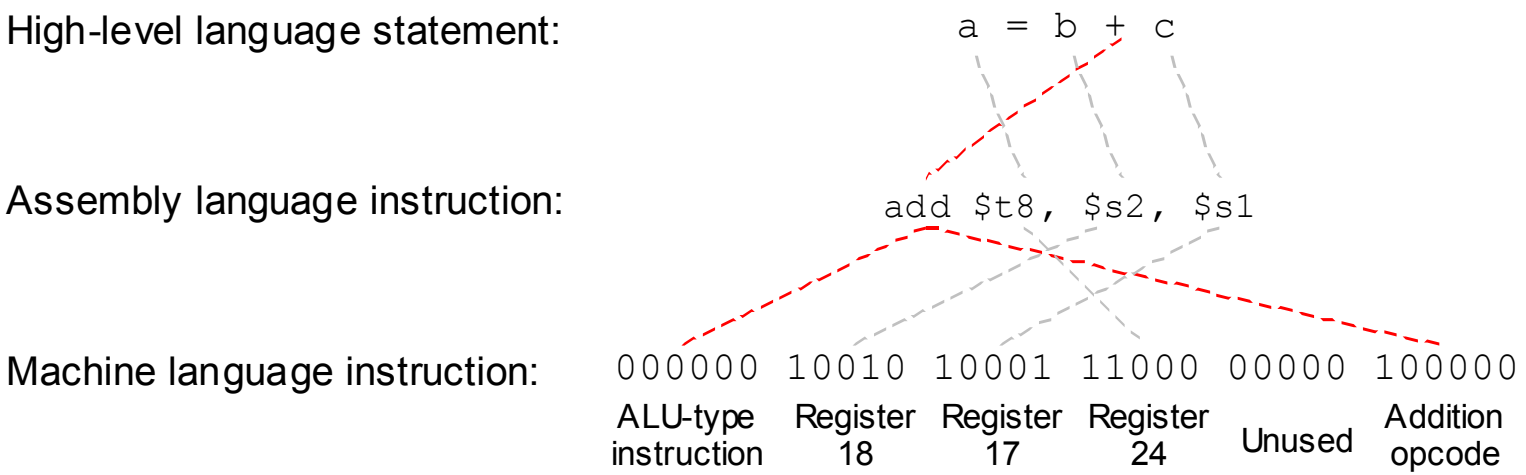
XPRLEN-1 … 0
pc

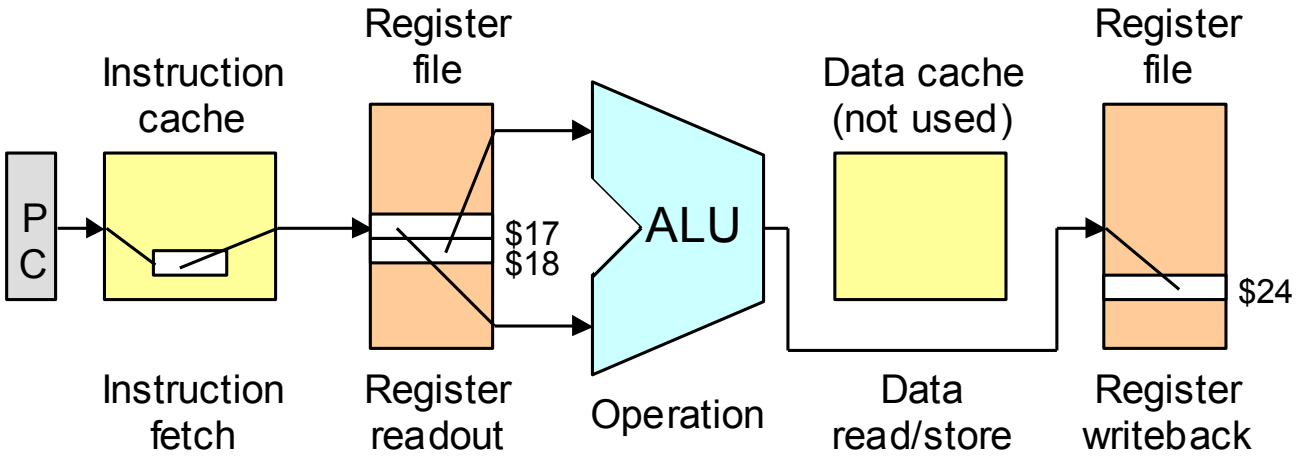
XPRLEN

31 … 0
fsr

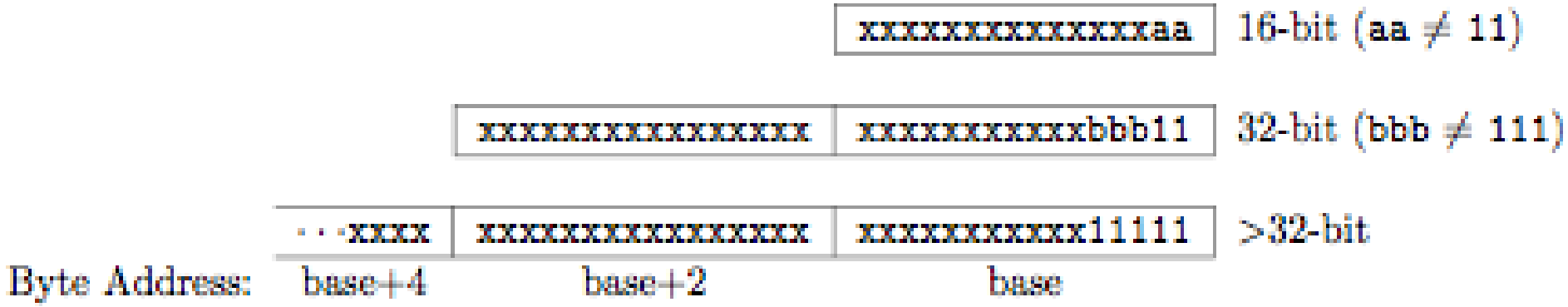
32

Formatul instrucțiunilor



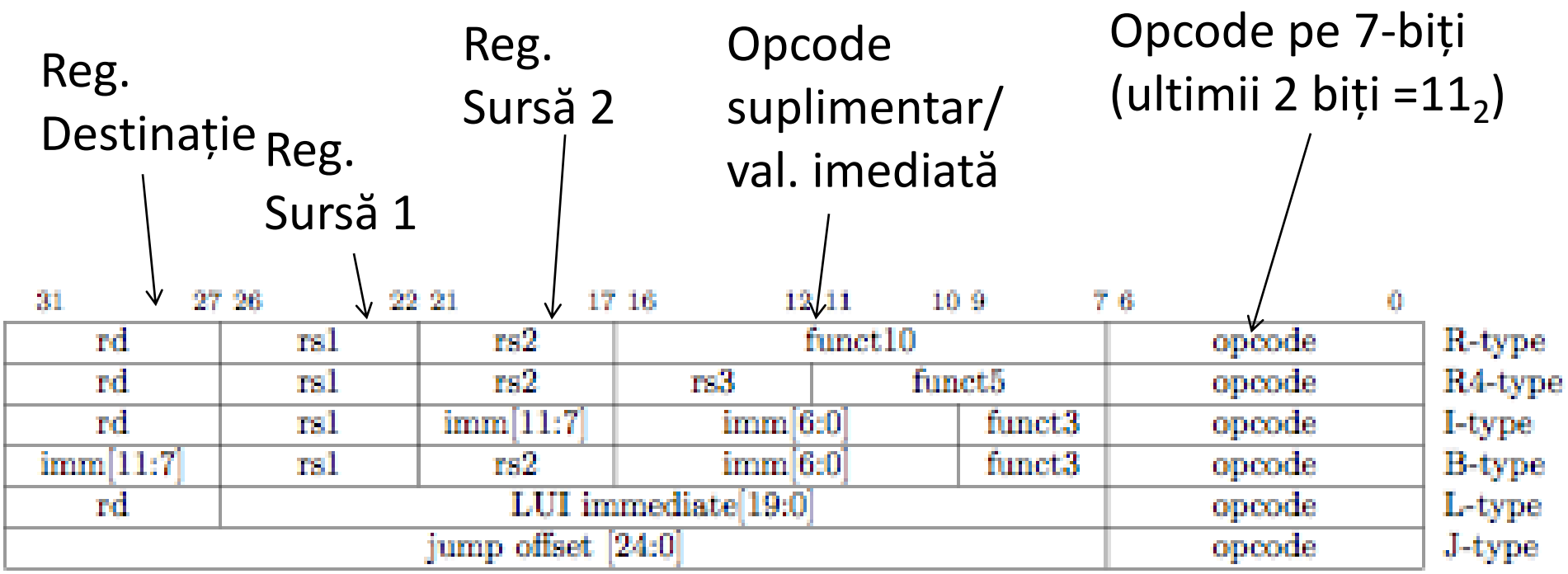


RISC-V Codificarea Instrucțiunilor

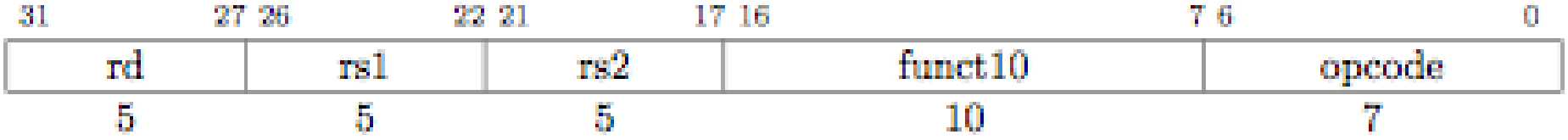


- Suport pentru instrucțiuni de lungime variabilă.
- Setul de bază (RV32) are întotdeauna instrucțiuni de dimensiune fixă, cu ultimii doi biți = 11_2
- Toate instrucțiunile de branch și jump au granularitate de 16 biți (chiar și în ISA de bază, unde toate instrucțiunile au lungimea de 32 de biți)

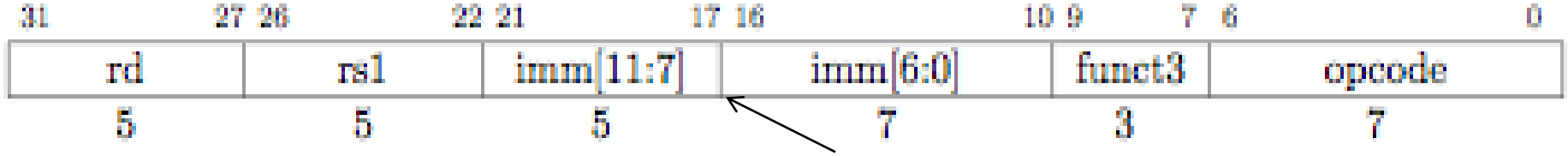
RISC-V Formatul Instrucțiunilor



R-Type/I-Type/R4-Type Formats

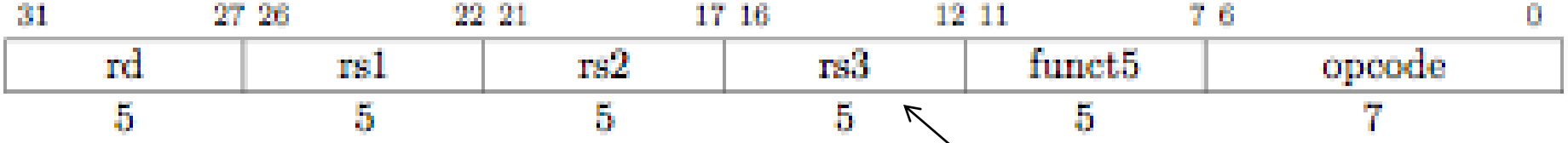


Operații Reg-Reg în ALU



Operații Reg-Imm ALU
Instrucțiuni Load, adresare (rs1 + imediat)

12-bit signed immediate

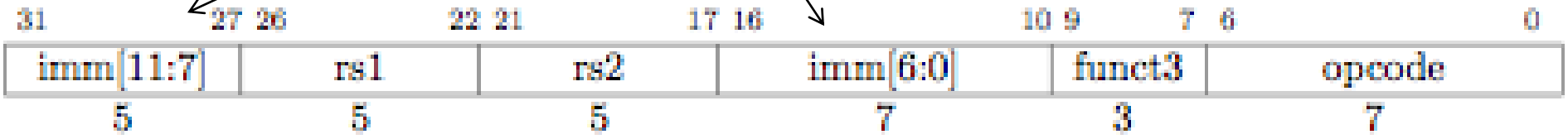


Folosite doar pentru adunări și înmulțiri în virgulă mobilă

Reg. Source 3

B-Type

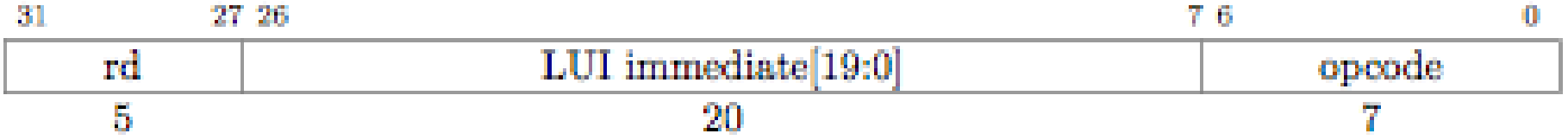
Valoare imediată cu semn pe 12 biți împărțită în 2 câmpuri



Folosită pt. Branch, comparații de registre - PC+(imediat<<1)

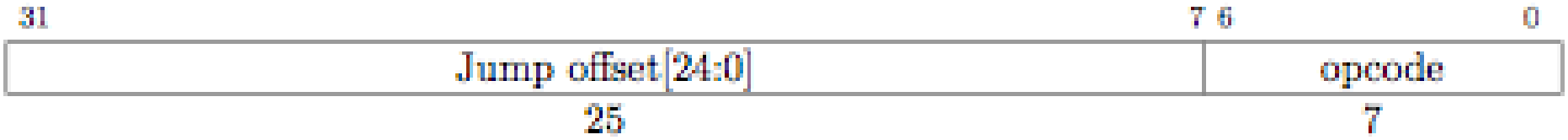
Instrucțiuni Store, adresă (rs1 + imediat) , date în rs2

L-Type



Scrie o valoare imediată de 20 de biți în registrul destinație.
Folosită pentru incărcarea de constante imediate mari.
Valorile imediate pe 12 biți sunt cu semn pentru a putea construi constante pe 32 de biți într-o secvență de doi pași (LUI high-20b, ADDI low-12b)

J-Type



"J" Jump necondiționat, PC+offset target

"JAL" Jump and link, scrie PC+4 în **x1**

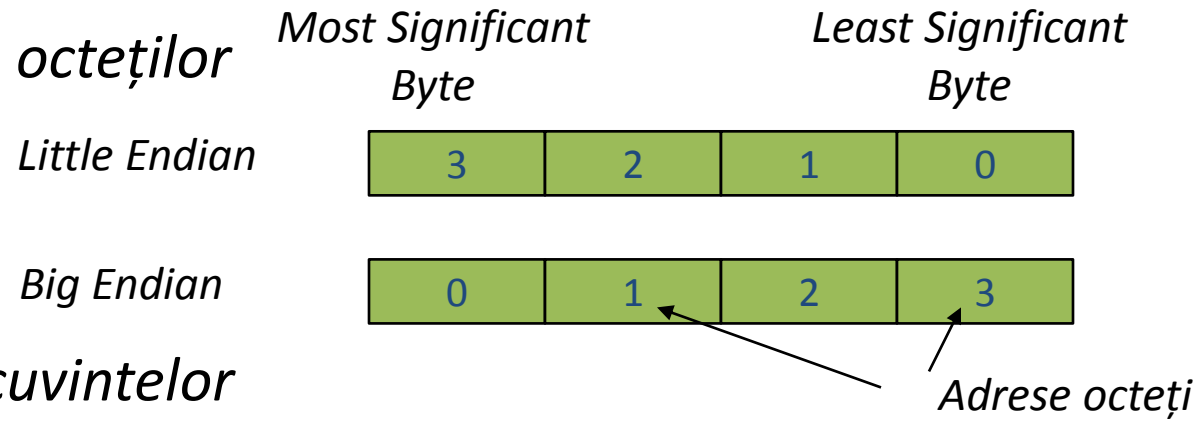
Formatul datelor și adresarea memoriei

Tipuri de date:

8-b Byte, 16-b Half word, 32-b word & 64-b double word

Remarci:

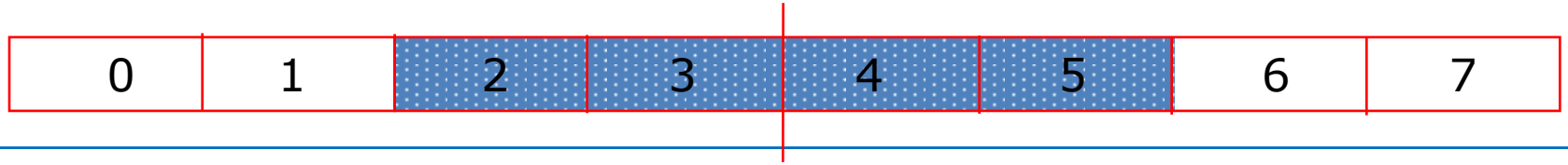
- *Adresarea octeților*



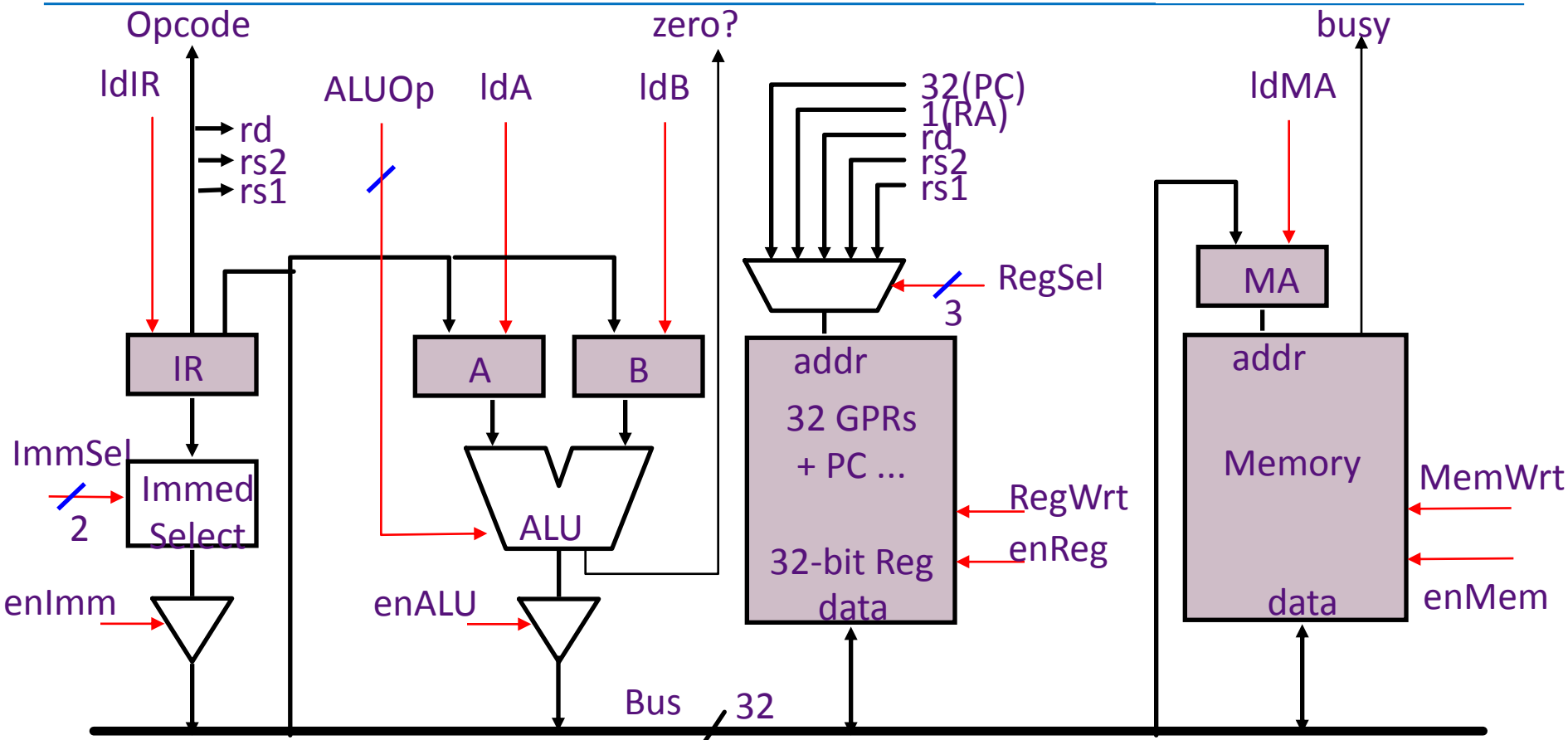
- *Alinierea cuvintelor*

Presupunem că memoria e organizată în cuvinte de 32 de biți.

Poate o adresă de cuvânt să înceapă numai la 0, 4, 8, ... ?



RISC-V – descriere generală



Microinstrucțiune: register to register transfer (17 semnale de control)

MA -> PC se traduce prin RegSel = PC; enReg=yes; ldMA= yes

B <- Reg[rs2] *se traduce prin* RegSel = rs2; enReg=yes; ldB = yes

The Big Picture

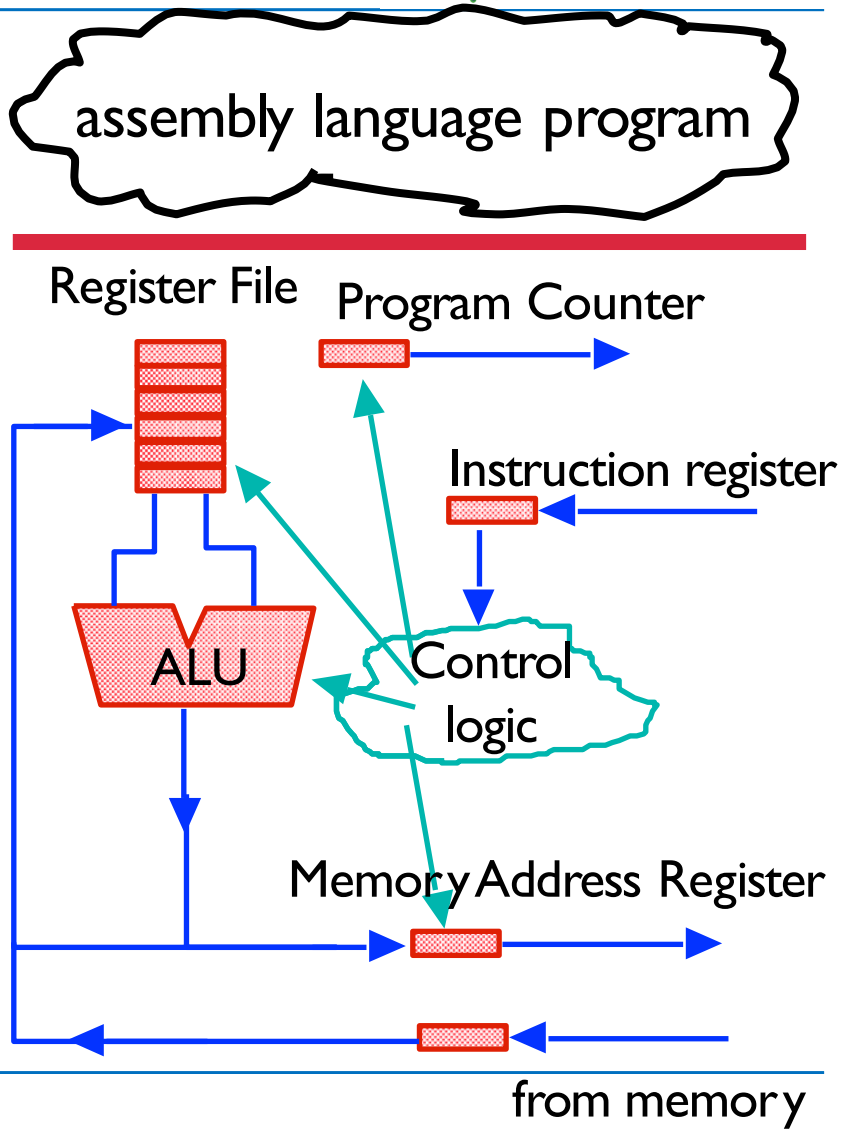
Embedded Systems Laboratory

Limbaj asamblare

Interfața pe care arhitectura o prezintă utilizatorului

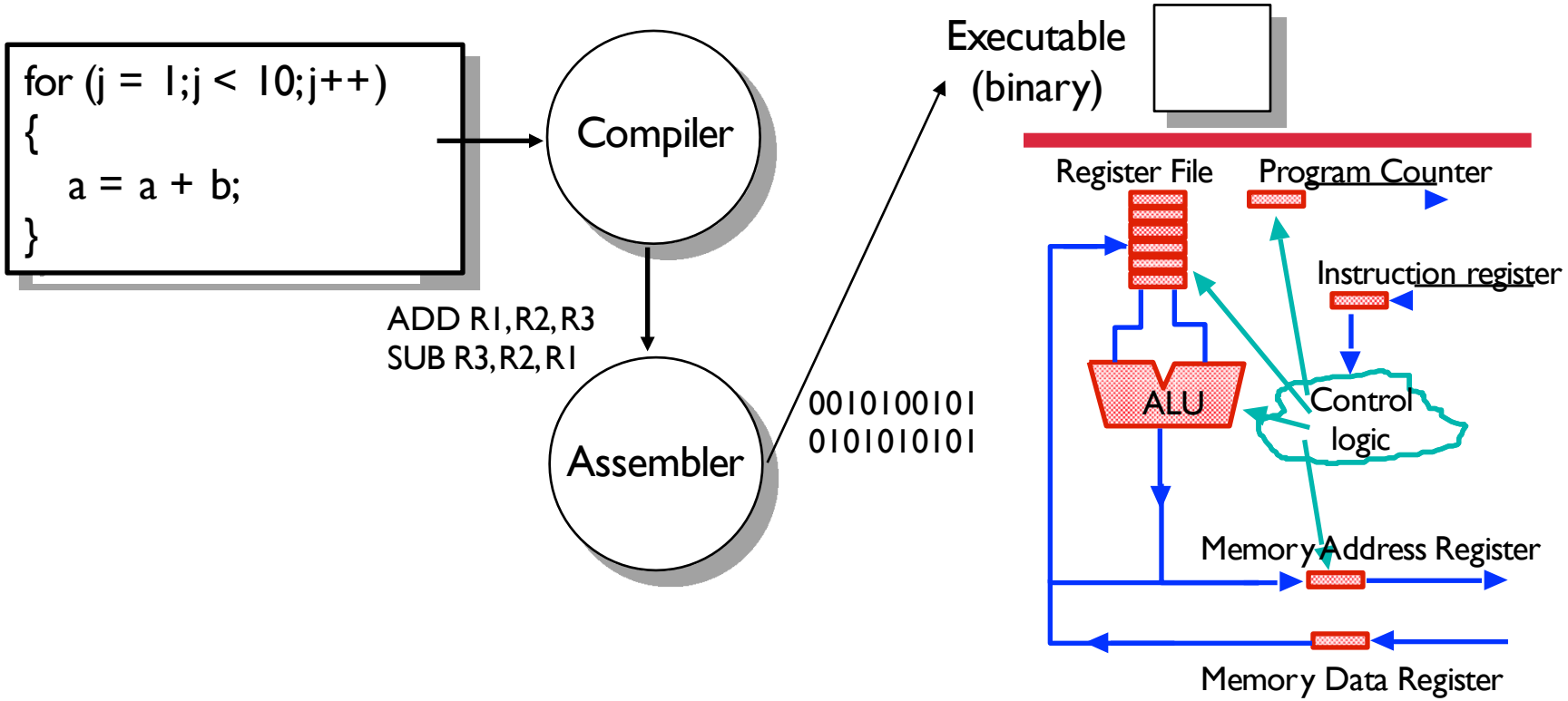
Instrucțiuni ”Low-level” ce folosesc memoria și căile de date pentru a efectua operațiile de bază

- aritmetice: add, sub, mul, div
- logice: and, or, shift
- data transfer: load, store
- (un)conditional branch: jump, branch on condition



Niveluri de abstractizare

- De obicei, oamenii nu scriu software în cod mașină
- Folosesc limbaje de nivel înalt C, C++, JAVA, Python
- Limbajul de nivel înalt este translatat în assembly de un compilator
- Assembly este translatat în cod mașină de asamblor



Clase ISA

Embedded Systems Laboratory

Memory to Memory Machines

- Avem nevoie de stocare pentru variabile temporare
- Memoria este lentă
- Memoria este mare (mulți biți de adresă)

Registre

- Registrele pot stoca variabile temporare
- Registrele sunt mai rapide decât memoria
- Traficul cu memoria este redus, iar execuția programului este accelerată (pentru că registrele sunt mia rapide ca memoria)
- Densitatea codului este îmbunătățită (pentru că registrele sunt adresate mai ușor decât memoria)

Predomina GPR Machines

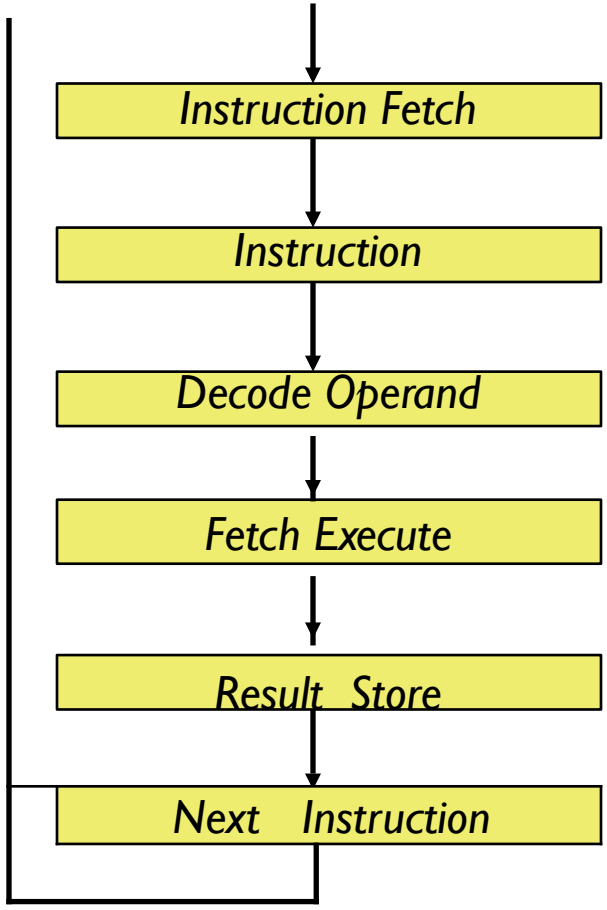
Embedded Systems Laboratory

Toate mașinile de calcul folosesc registre generale

Avantajele registrelor

- Registrele sunt mai rapide decât memoria
  - Traficul cu memoria este redus, iar execuția programului este accelerată (pentru că registrele sunt mai rapide ca memoria)
- Registrele pot stoca variabile
  - Registrele sunt mai ușor de folosit pentru un compilator: e.g., (A*B) – (C*D) – (E*F) poate executa în orice ordine vs. mașină cu stivă

Modelul execuției unei instrucțiuni



Model secvențial de execuție

Programul este o secvență de execuție

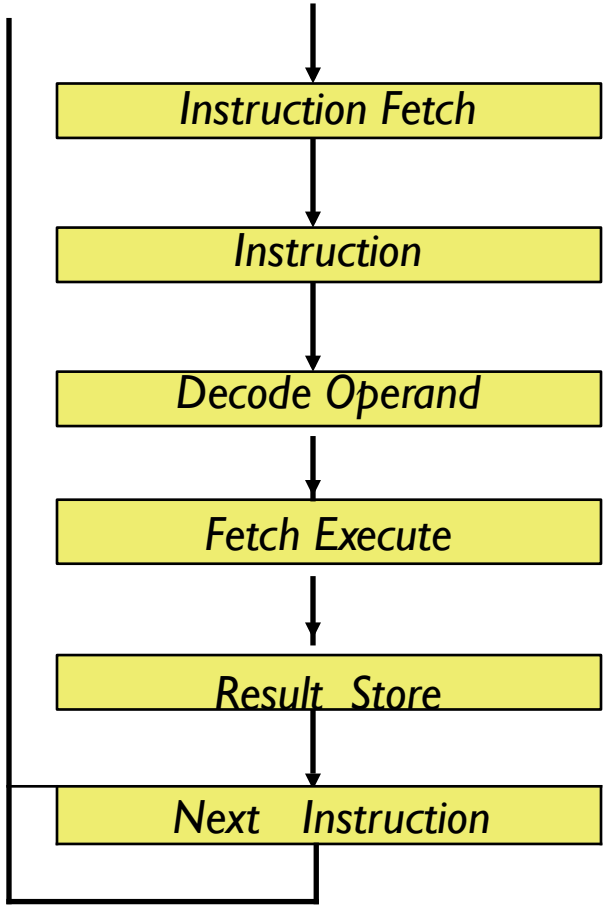
Instrucțiunile sunt atomice și executate secvențial

Conceptul programului stocat

Programul și datele sunt stocate în memorie

Instrucțiunile sunt aduse din memorie pentru execuție

Modelul execuției unei instrucțiuni



ISA Issues

Get instruction from memory

Instruction Format/Encoding

Addressing Modes

Op-codes and DataTypes

Addressing Modes

Instruction Sequencing

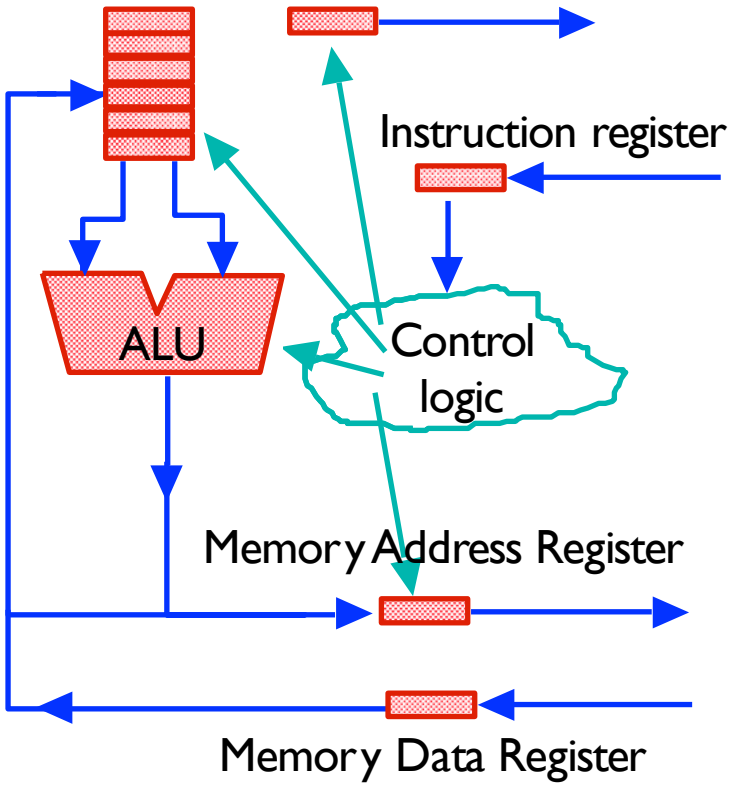
Execuția unei instrucțiuni

Embedded Systems Laboratory

Secvențiatorul (FSM) face fetch la instrucțiune din memorie și o scrie în Instruction Register

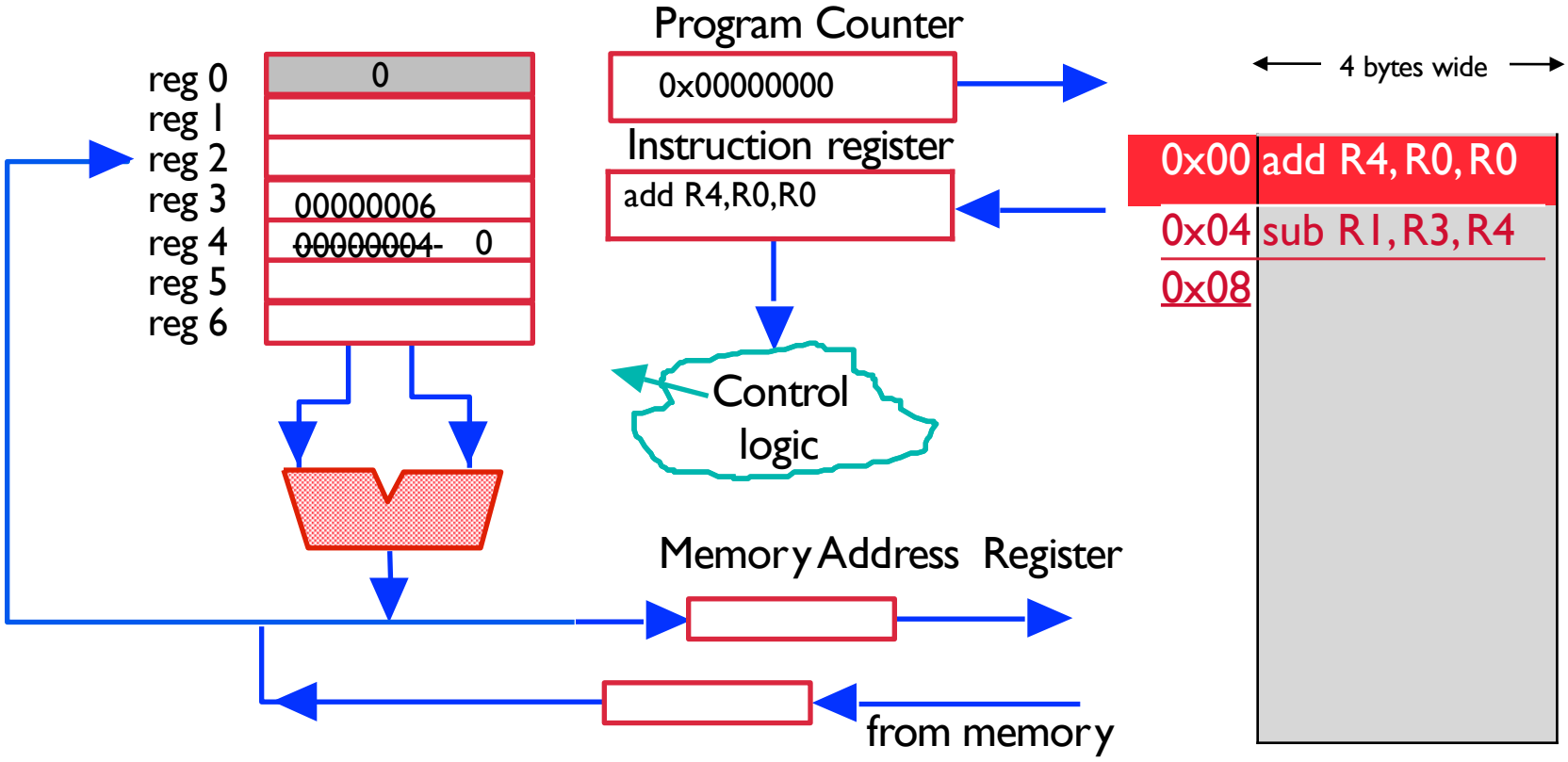
Logica de control decodifică instrucțiunea și emite comenzi către tabela de registre, ALU și celelalte registre

Dacă o operație ALU (e.g. add), datele merg din tabela de registre prin ALU și înapoi în tabela de registre

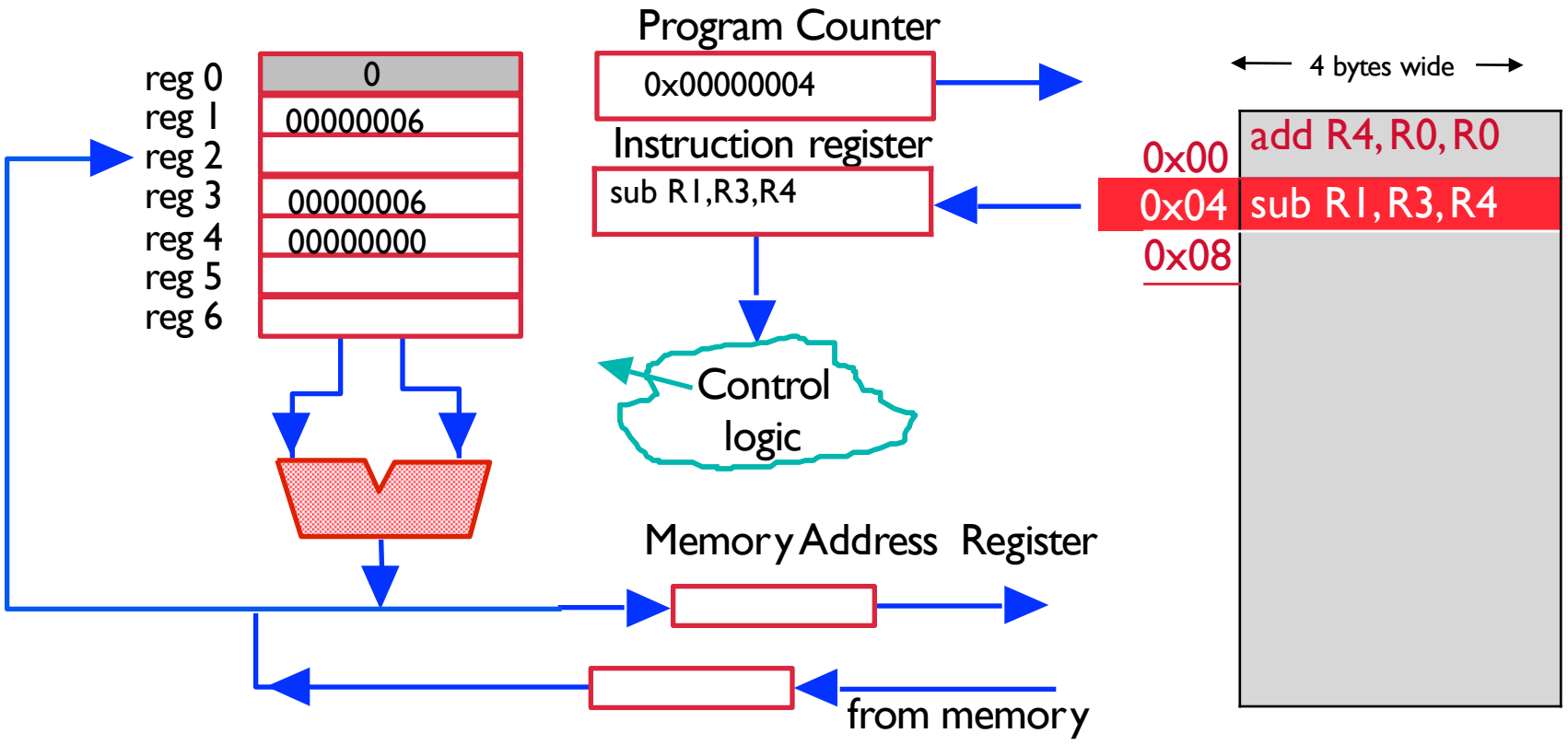


0x00	00101101
0x01	00100001
0x02	00110000
0x03	00001111
0x04	11010101
0x05	01010101
0x06	00101010
0x07	01010101
0x08	11110011
0x09	00111100
0x0A	00001100
0x0B	00000000
0x0C	00011000
0x0D	11111111

Execuție din tabela de registre



Execuție din tabela de registre

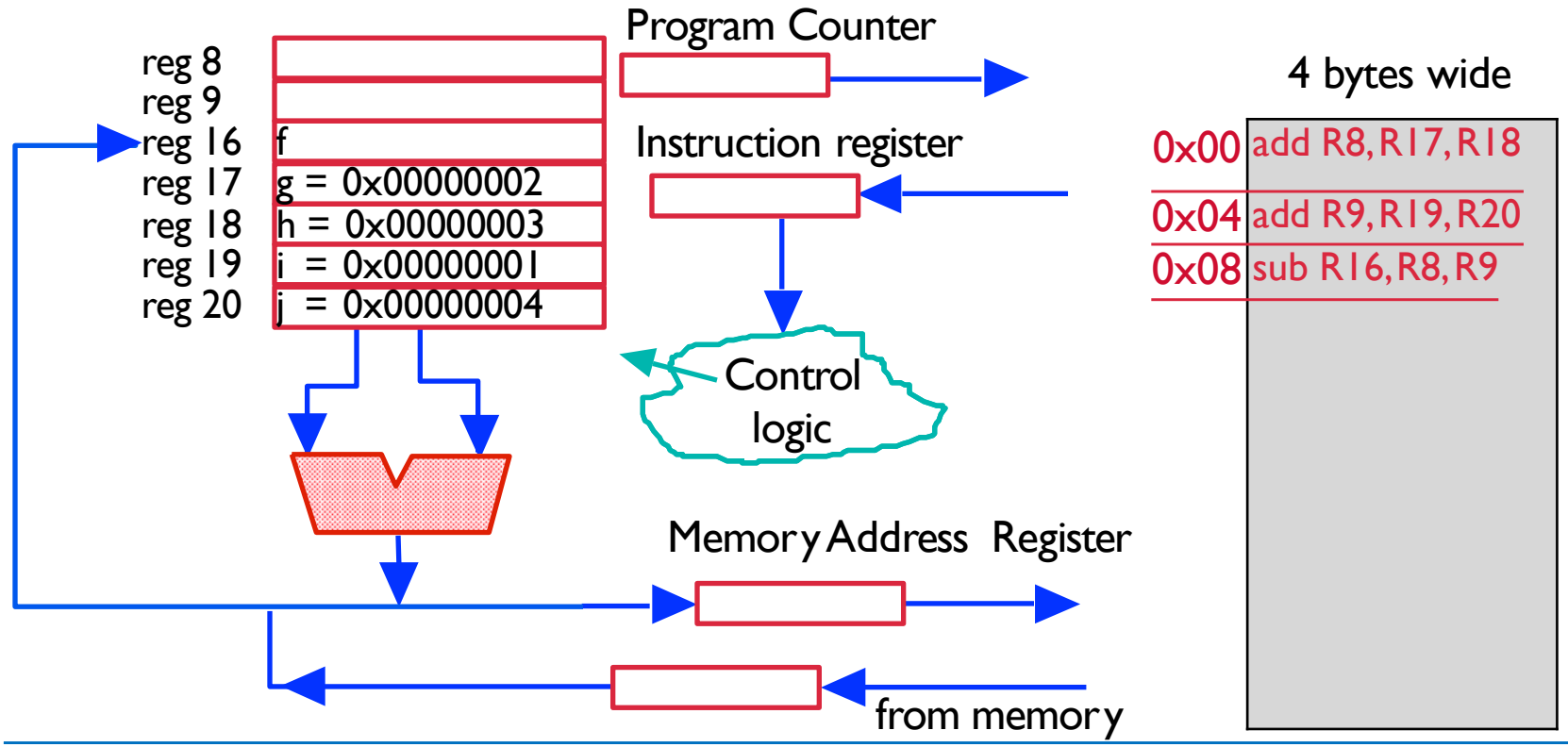


Alt exemplu

Embedded Systems Laboratory

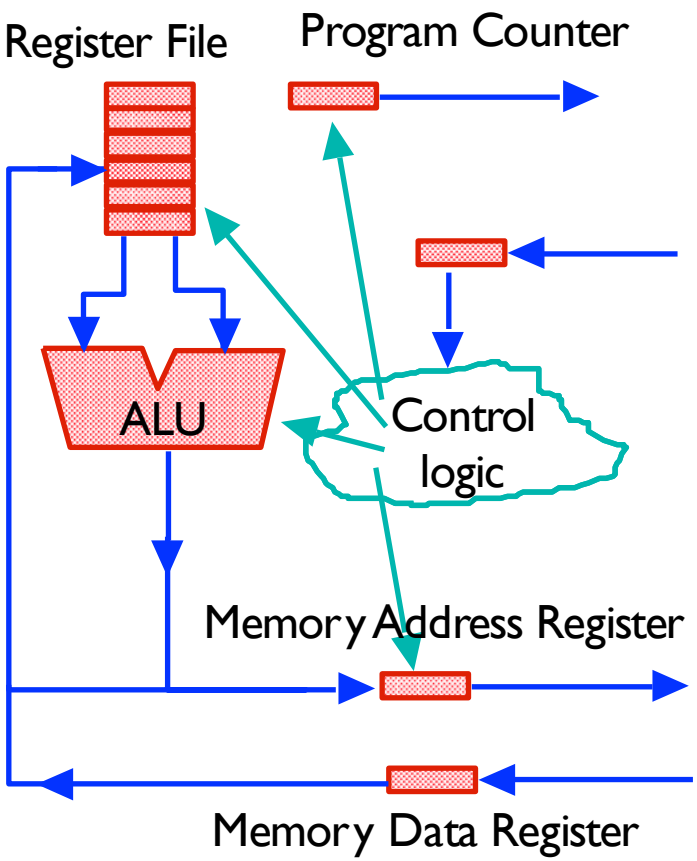
$$f = (g + h) - (i + j)$$

R16 == f, R17 == g, R18 == h, R19 == i, R20 == j



Accesarea datelor

- Adresa generată de ALU
- Adresa ajunge în *Memory Address Register*
- După accesul memoriei, rezultatul este întors în *Memory Data Register*
- Adresele pot fi pentru date sau pentru instrucțiuni, ambele sunt stocate în memorie



0x00	00101101
0x01	00100001
0x02	00110000
0x03	00001111
0x04	11010101
0x05	01010101
0x06	00101010
0x07	01010101
0x08	11110011
0x09	00111100
0x0A	00001100
0x0B	00000000
0x0C	00011000
0x0D	11111111

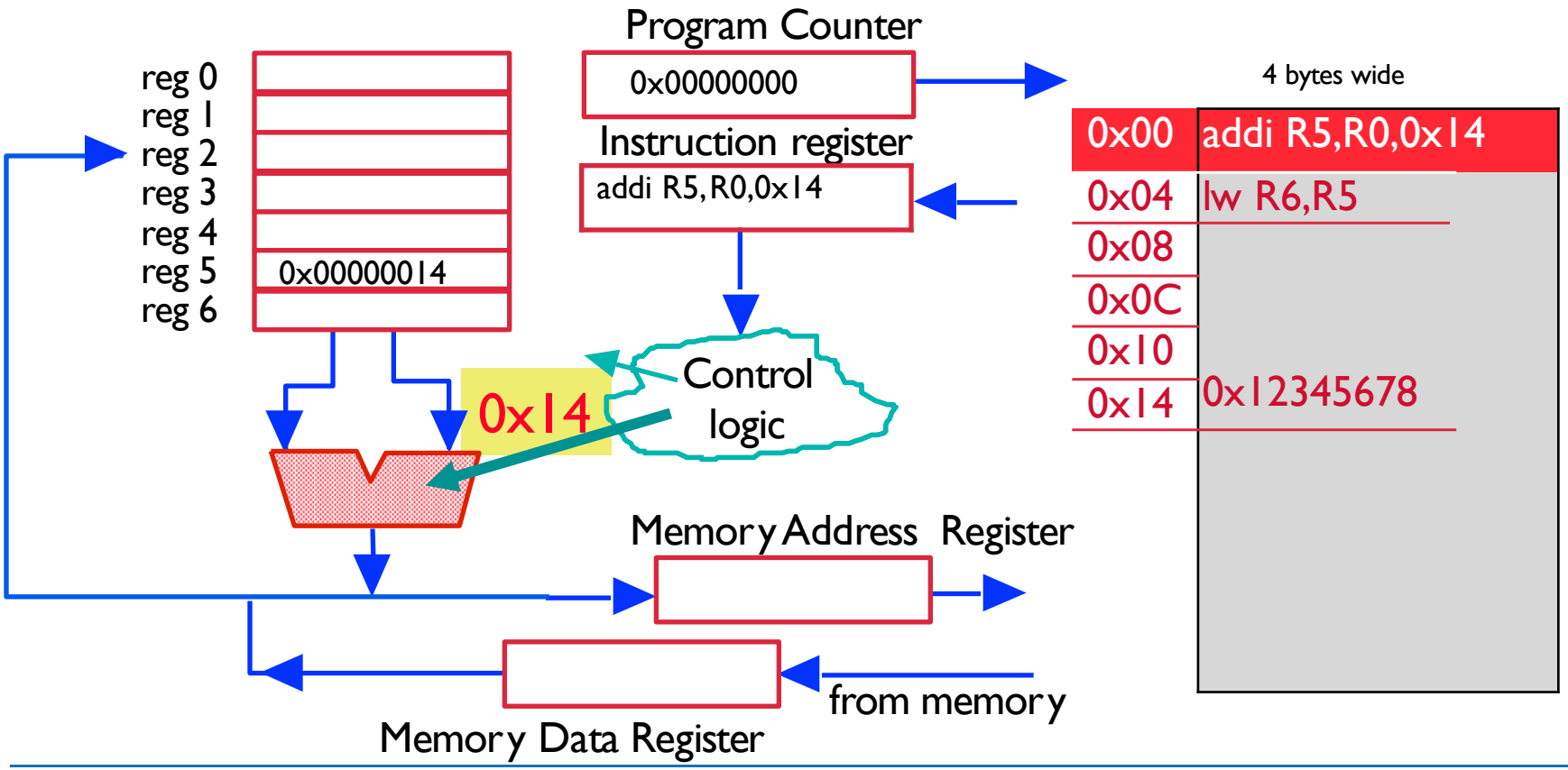
Operații cu memoria - Load

Embedded Systems Laboratory

R6 ⇇mem[0x14]

Presupunem &A = 0x14

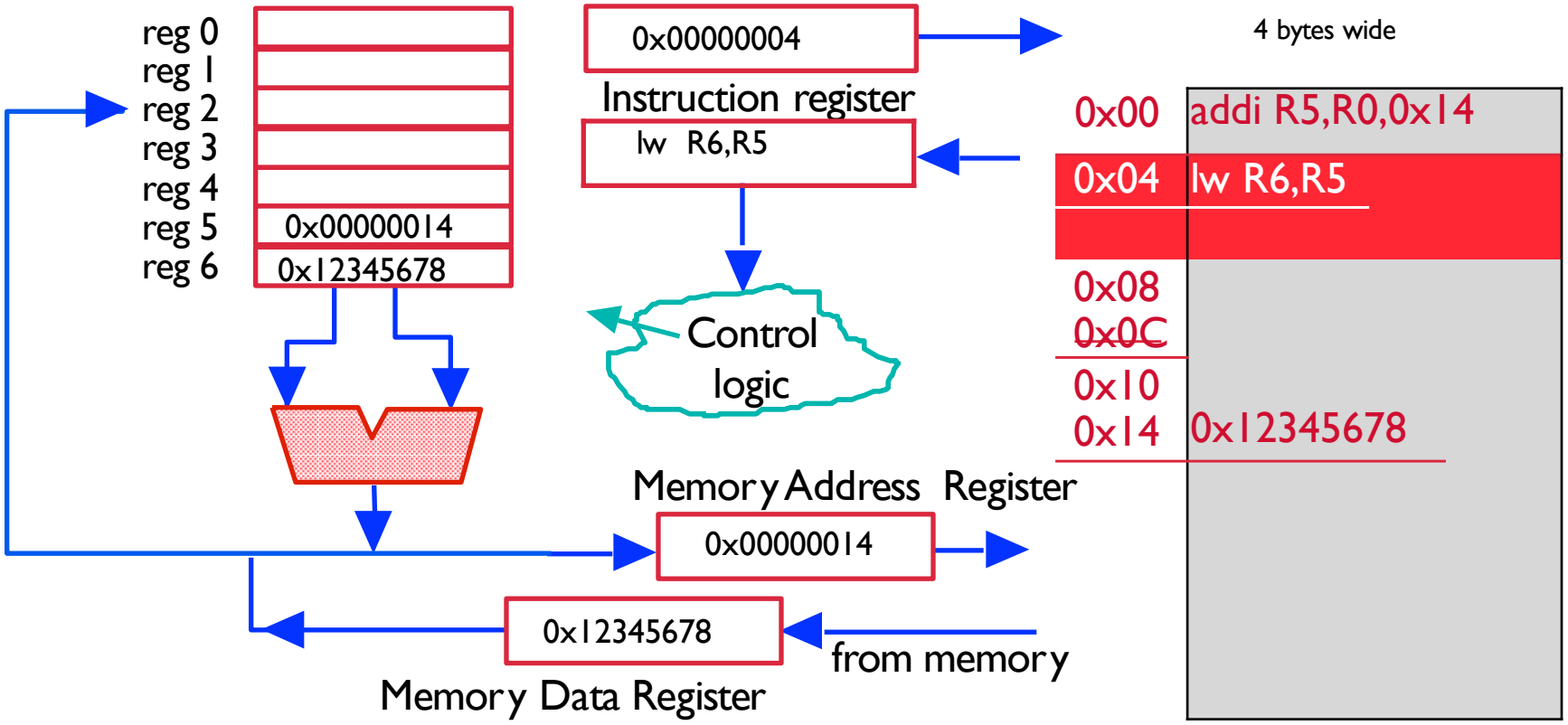
addi: Adună constanta pe 16 biți la registrul sursă



Operații cu memoria - Load

R6 ⇐mem[0x14]

Presupunem &A = 0x14



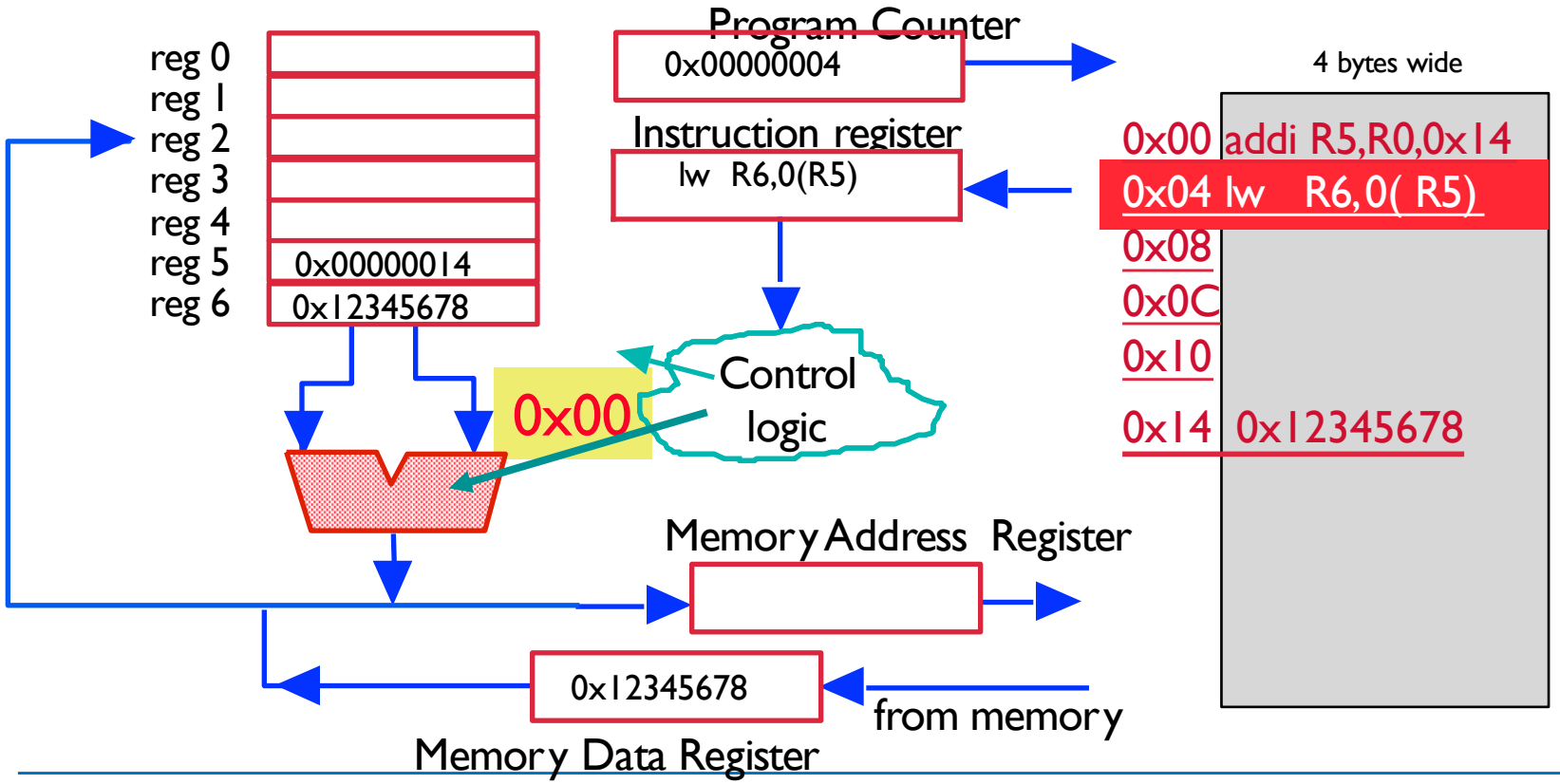
Operații cu memoria - Load

Embedded Systems Laboratory

Adresa poate fi calculată și prin adăugarea unui offset registrului

LW R6, 0(R5)

R6 ⇐ memory[0 + R5]



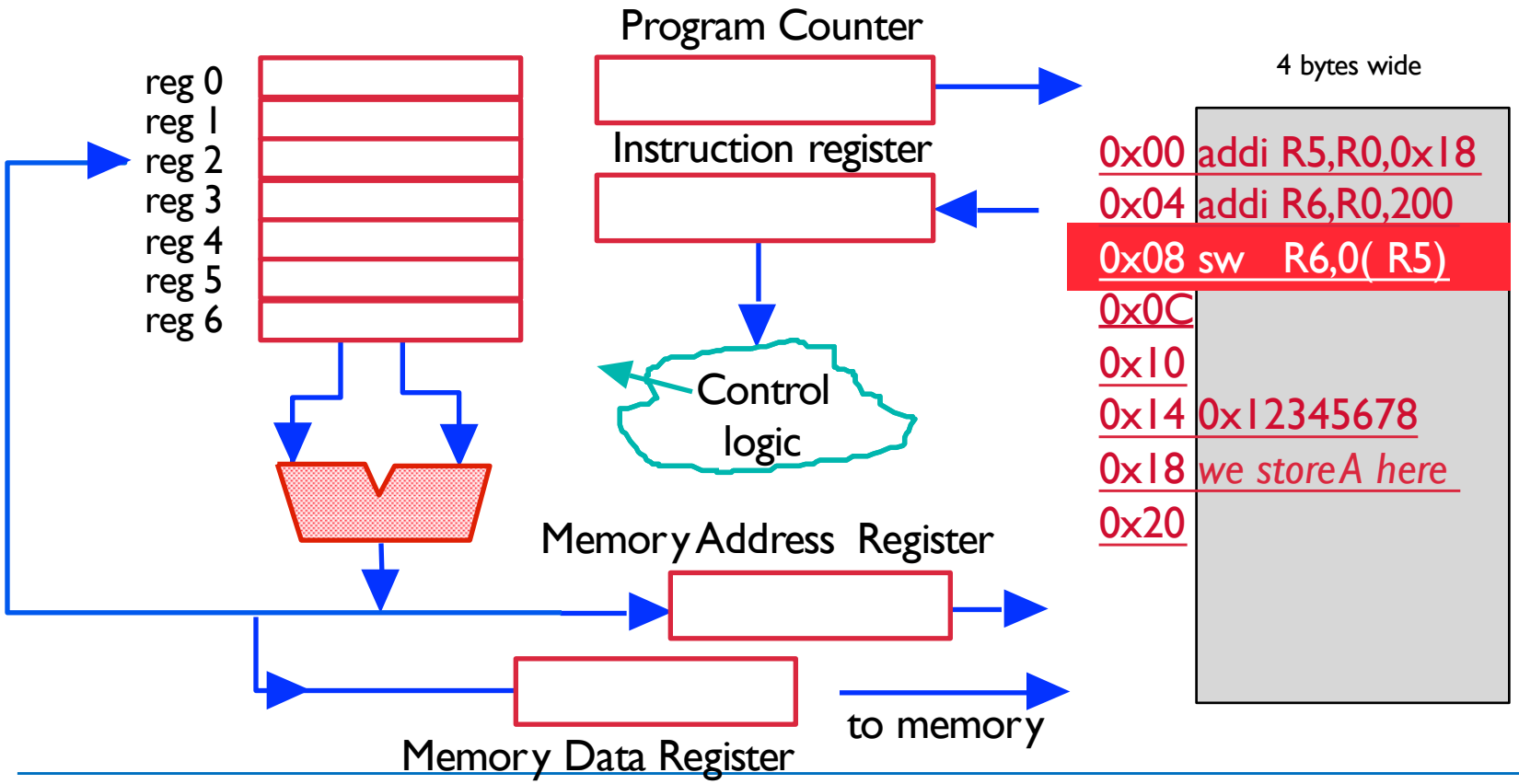
Operații cu memoria - Store

Embedded Systems Laboratory

Stocarea datelor în memorie se face la fel

A = 200; presupunem &A = 0x18

mem[0x18] ⇐200



Formatul instrucțiunilor (cod mașină)

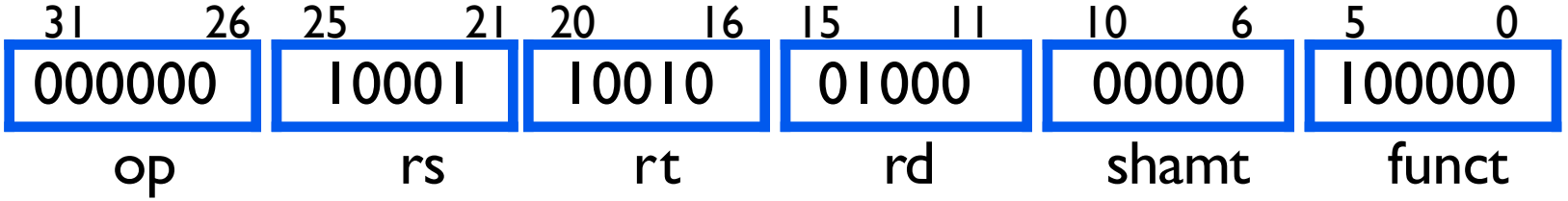
Embedded Systems Laboratory

add R8, R17, R18

Stocat în format binar ca

00000010 00110010 01000000 00100000

Instrucțiunea este împărțită în "câmpuri"



op	Codul operației
rs	Adrea registru sursă 1
rt	Adresa registru sursă 2
rd	Adresa registrul destinație
shamt	shift amount
funct	Funcție (selectează tipul operației)

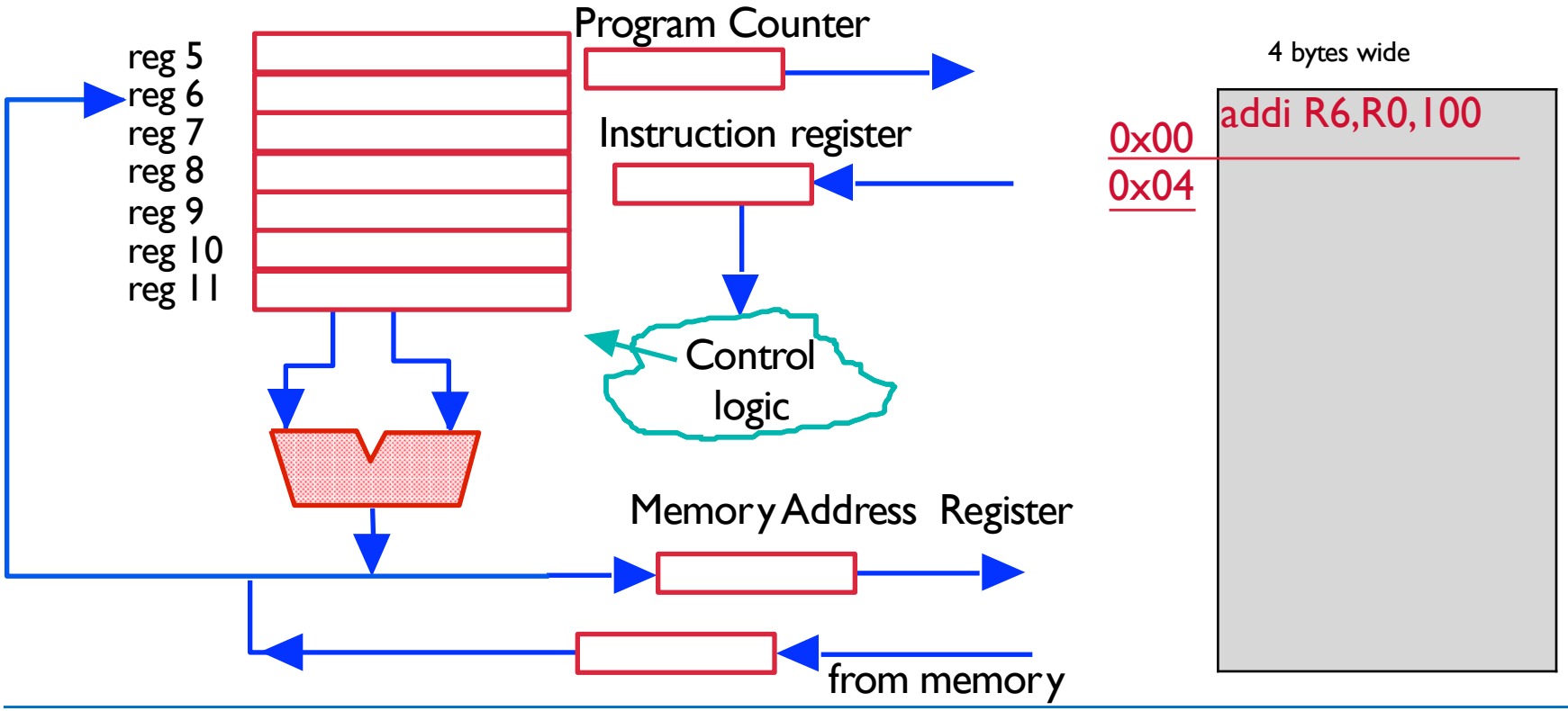
add = 32_{10}

sub = 34_{10}

Încărcarea valorilor imediate

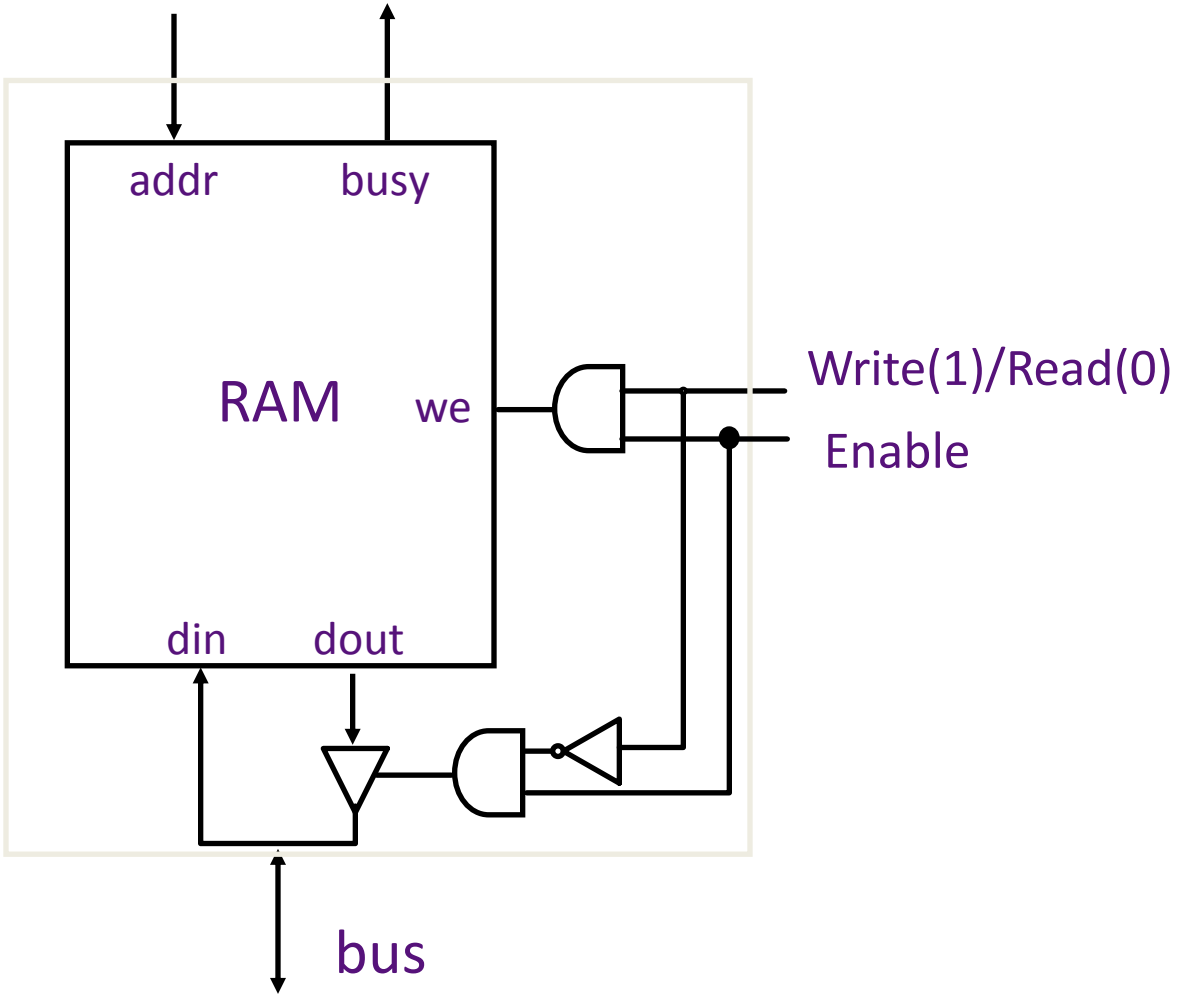
Cum punem o constantă (valoare imediată) în registru?

Scriem valoarea 100 în R6: R6 ⇐ R0 + 100 = 0+100 = 100



Modulul de memorie

Presupunere: memoria operează independent și e mult mai lentă decât operațiile de transfer registru-registru (mai mulți cicli de ceas pentru a accesa datele)



Execuția instrucțiunilor

Execuția unei instrucțiuni RISC-V presupune următorii pași:

1. instruction fetch
2. decode & register fetch
3. operații în UAL
4. operații cu memoria (opțional)
5. write back în tabela de registre generale (opțional)

+ calculul adresei următoarei instrucțiuni

Fragmente de microprogram

Embedded Systems Laboratory

instr fetch:
MA, A <- PC
PC <- A + 4
IR <- Memory
dispatch on Opcode

Poate fi văzută ca
O macroinstrucțiune

ALU:
A <- Reg[rs1]
B <- Reg[rs2]
Reg[rd] <- func(A,B)
do instruction fetch

ALUi:
A <- Reg[rs1]
B <- Imm
Reg[rd] <- Opcode(A,B)
do instruction fetch

sign extension

Fragmente de Microprogram
(cont.)

Embedded Systems Laboratory

LW:
A <- Reg[rs1]
B <-Imm
MA <- A + B
Reg[rd] <- Memory
do instruction fetch

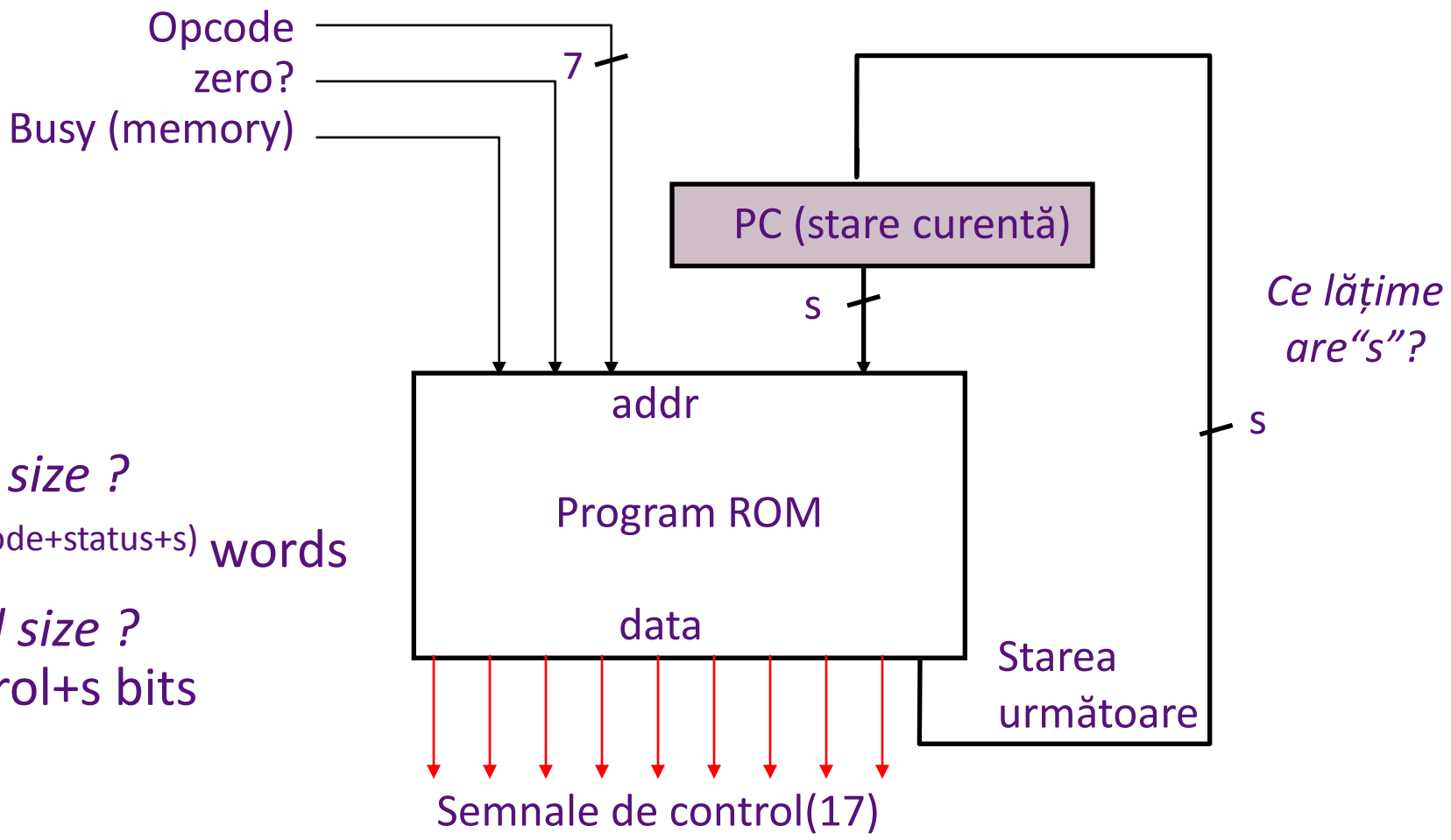
J:
A <- A - 4 Get original PC back in A
B <- IR
PC <- JumpTarg(A,B)
do instruction fetch

JumpTarg(A,B) = {A + (B[31:7]<<1)}

beq:
A <- Reg[rs1]
B <- Reg[rs2]
If A==B *then go to* bz-taken
do instruction fetch

bz-taken:
A <- PC
A <- A - 4 Get original PC back in A
B <- BImm << 1 BImm = IR[31:27,16:10]
PC <- A + B
do instruction fetch

RISC-V Microcontroller: *prima încercare:implementare numai în ROM*



ROM size ?

= $2^{(opcode+status+s)}$ words

Word size ?

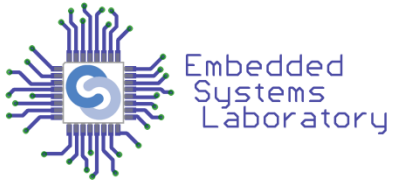
= control+s bits

Microprogramul din ROM

Embedded Systems Laboratory

State	Op	zero?	busy	Control points	next-state
$fetch_0$	*	*	*	MA,A <- PC	$fetch_1$
$fetch_1$	*	*	yes	....	$fetch_1$
$fetch_1$	*	*	no	IR <- Memory	$fetch_2$
$fetch_2$	*	*	*	PC <- A + 4	?
$fetch_2$	ALU	*	*	PC <- A + 4	ALU_0
ALU_0	*	*	*	A <- Reg[rs1]	ALU_1
ALU_1	*	*	*	B <- Reg[rs2]	ALU_2
ALU_2	*	*	*	Reg[rd] <- func(A,B)	$fetch_0$

Microprogramul din ROM



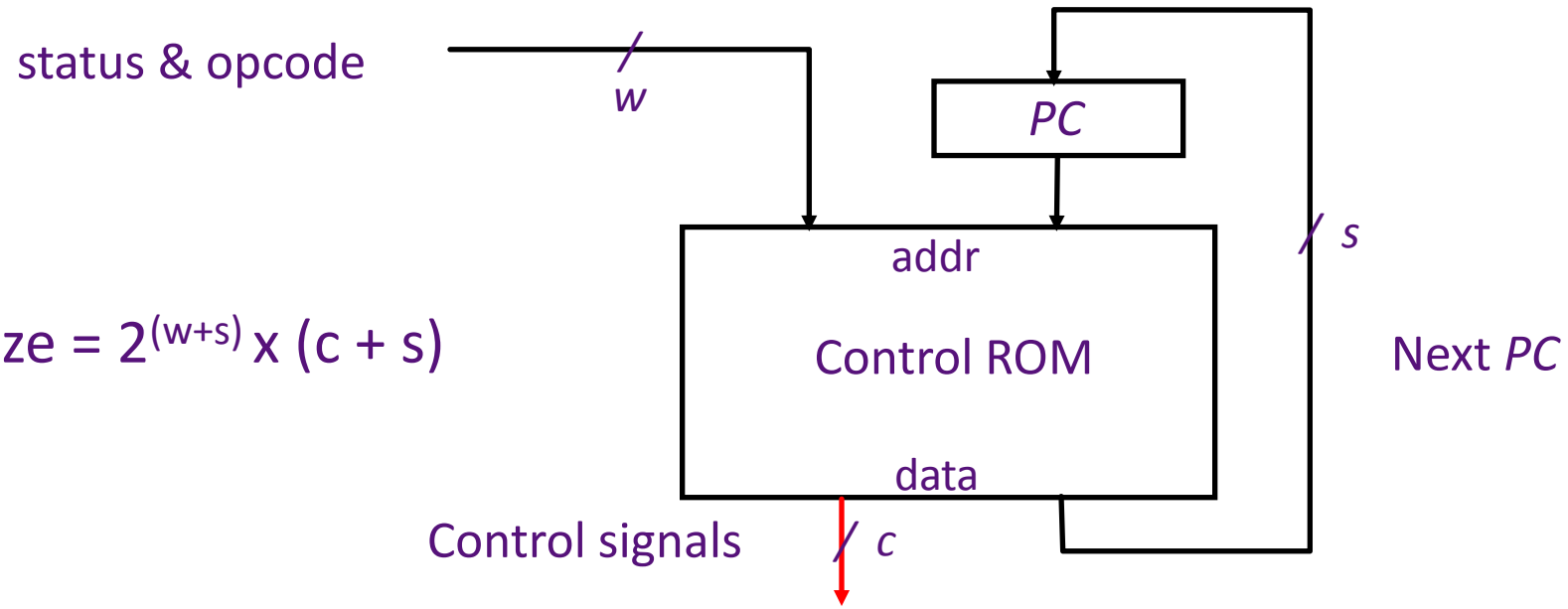
State	Op	zero?	busy	Control points	next-state
$fetch_0$	*	*	*	MA,A <- PC	$fetch_1$
$fetch_1$	*	*	yes	....	$fetch_1$
$fetch_1$	*	*	no	IR ? Memory	$fetch_2$
$fetch_2$	ALU	*	*	PC <- A + 4	ALU_0
$fetch_2$	ALUi	*	*	PC <- A + 4	$ALUi_0$
$fetch_2$	LW	*	*	PC <- A + 4	LW_0
$fetch_2$	SW	*	*	PC <- A + 4	SW_0
$fetch_2$	J	*	*	PC <- A + 4	J_0
$fetch_2$	JAL	*	*	PC <- A + 4	JAL_0
$fetch_2$	JR	*	*	PC <- A + 4	JR_0
$fetch_2$	JALR	*	*	PC <- A + 4	$JALR_0$
$fetch_2$	beq	*	*	PC <- A + 4	beq_0
...					
ALU_0	*	*	*	A <- Reg[rs1]	ALU_1
ALU_1	*	*	*	B <- Reg[rs2]	ALU_2
ALU_2	*	*	*	Reg[rd] <- func(A,B)	$fetch_0$

Microprogramul din ROM *Cont.*

Embedded Systems Laboratory

State	Op	zero?	busy	Control points	next-state
$ALUi_0$	*	*	*	A <- Reg[rs1]	$ALUi_1$
$ALUi_1$	*	*	*	B <- Imm	$ALUi_2$
$ALUi_2$	*	*	*	Reg[rd]<- Op(A,B)$fetch_0$	
...					
J_0	*	*	*	A <- A - 4	J_1
J_1	*	*	*	B <- IR	J_2
J_2	*	*	*	PC <- JumpTarg(A,B)	$fetch_0$
...					
beq_0	*	*	*	A <- Reg[rs1]	beq_1
beq_1	*	*	*	B <- Reg[rs2]	beq_2
beq_2	*	yes	*	A <- PC	beq_3
beq_2	*	no	*	....	$fetch_0$
beq_3	*	*	*	A <- A - 4	beq_4
beq_4	*	*	*	B <- BImm	beq_5
beq_5	*	*	*	PC <- A+B	$fetch_0$

Dimensiunea memoriei ROM



size = $2^{(w+s)}$ x (c + s)

RISC-V: w = 5+2 c = 17 s = ?

no. of steps per opcode = 4 to 6 + fetch-sequence

no. of states -> (4 steps per op-group) x op-groups + secv. comune

= 4 x 8 + 10 states = 42 states => s = 6

Control ROM = $2^{(5+6)}$ x 23 bits ~ 24 Kbytes

Reducerea dimensiunii memoriei de control

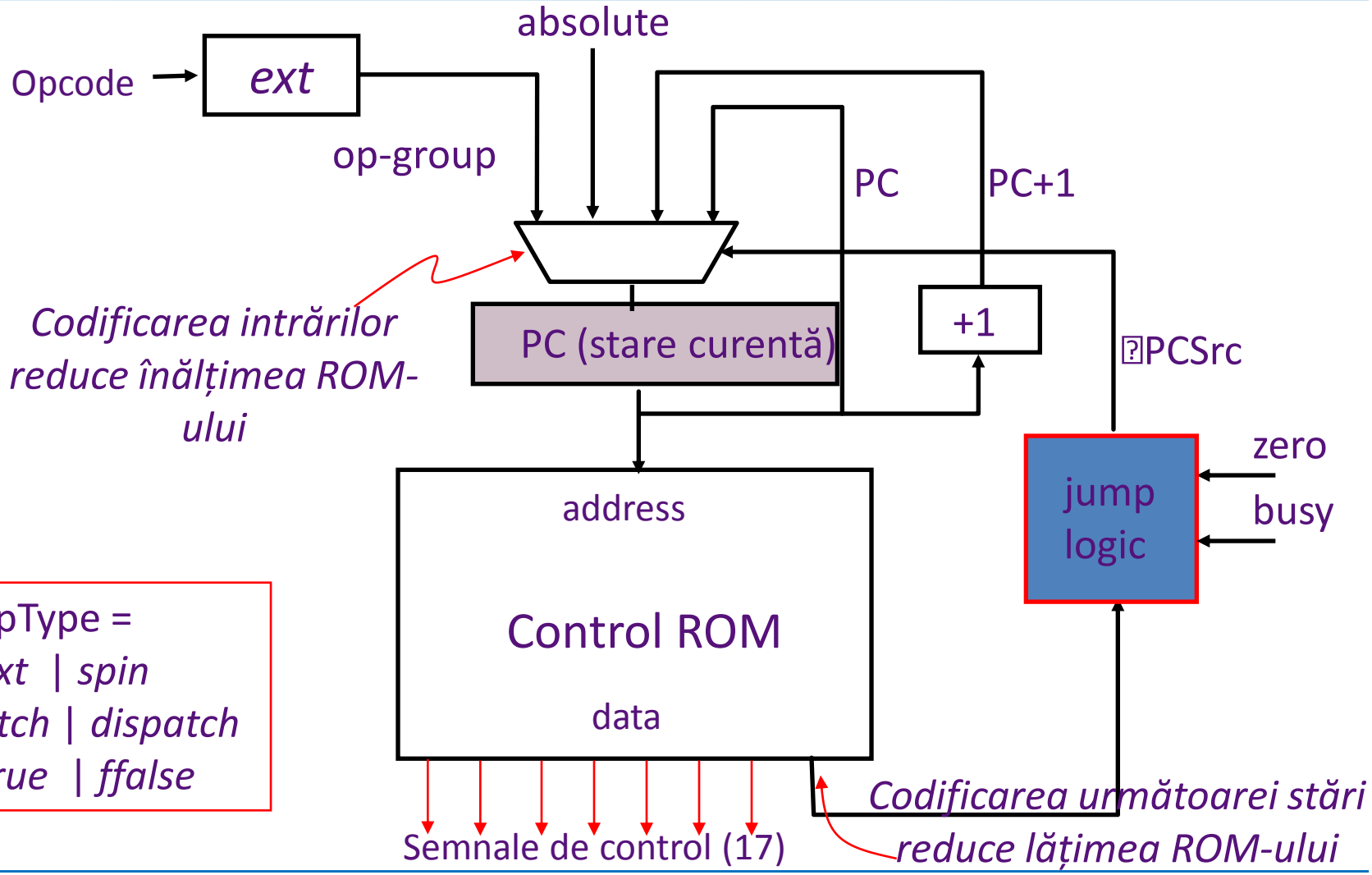
Embedded Systems Laboratory

Memoria de control trebuie să fie *rapidă => scumpă*

- Reducerea înălțimii ROM-ului (= biți de adresă)
  - *reducerea numărului de intrări prin adăugarea de logică externă*
    Fiecare bit de intrare dublează dimensiunea memoriei de control
  - *reducerea numărului de stăriprin gruparea codurilor de operații*
    găsește secvențe comune de acțiuni
  - *comprimă biții de stare pentru intrări*
    combină toate excepțiile într-una singură, i.e., exception/no-exception

- Reducerea lățimii ROM-ului
  - *restricționează codificarea următorii stări*
    Next, Dispatch on opcode, Wait for memory, ...
  - *codifică semnalele de control (microcod verticalizat)*

RISC-V Controller V2



Stările pentru Jump

PCSrc = *Case* JumpTypes

next	->	PC+1
spin	->	if (busy) then PC else PC+1
fetch	->	absolute
dispatch	->	op-group
ftrue	->	if (zero) then absolute else PC+1
ffalse	->	if (zero) then PC+1 else absolute

Instruction Fetch & ALU: *RISC-V-Controller-2*

Embedded Systems Laboratory

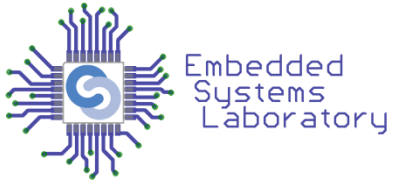
State	Control points	next-state
$fetch_0$	MA,A <- PC	
$fetch_1$	IR <- Memory	
$fetch_2$	PC <- A + 4	
...		
ALU_0	A <- Reg[rs1]	
ALU_1	B <- Reg[rs2]	
ALU_2	Reg[rd]<-func(A,B)	
$ALUi_0$	A <- Reg[rs1]	
$ALUi_1$	B <- Imm	
$ALUi_2$	Reg[rd]<- Op(A,B)	

Load & Store: *RISC-V-Controller-2*

Embedded Systems Laboratory

State	Control points	next-state
LW_0	A <- Reg[rs1]	next
LW_1	B <- Imm	next
LW_2	MA <- A+B	next
LW_3	Reg[rd] <- Memory	spin
LW_4		fetch
SW_0	A <- Reg[rs1]	next
SW_1	B <- BImm	next
SW_2	MA <- A+B	next
SW_3	Memory <- Reg[rs2]	spin
SW_4		fetch

Branches: *RISC-V-Controller-2*



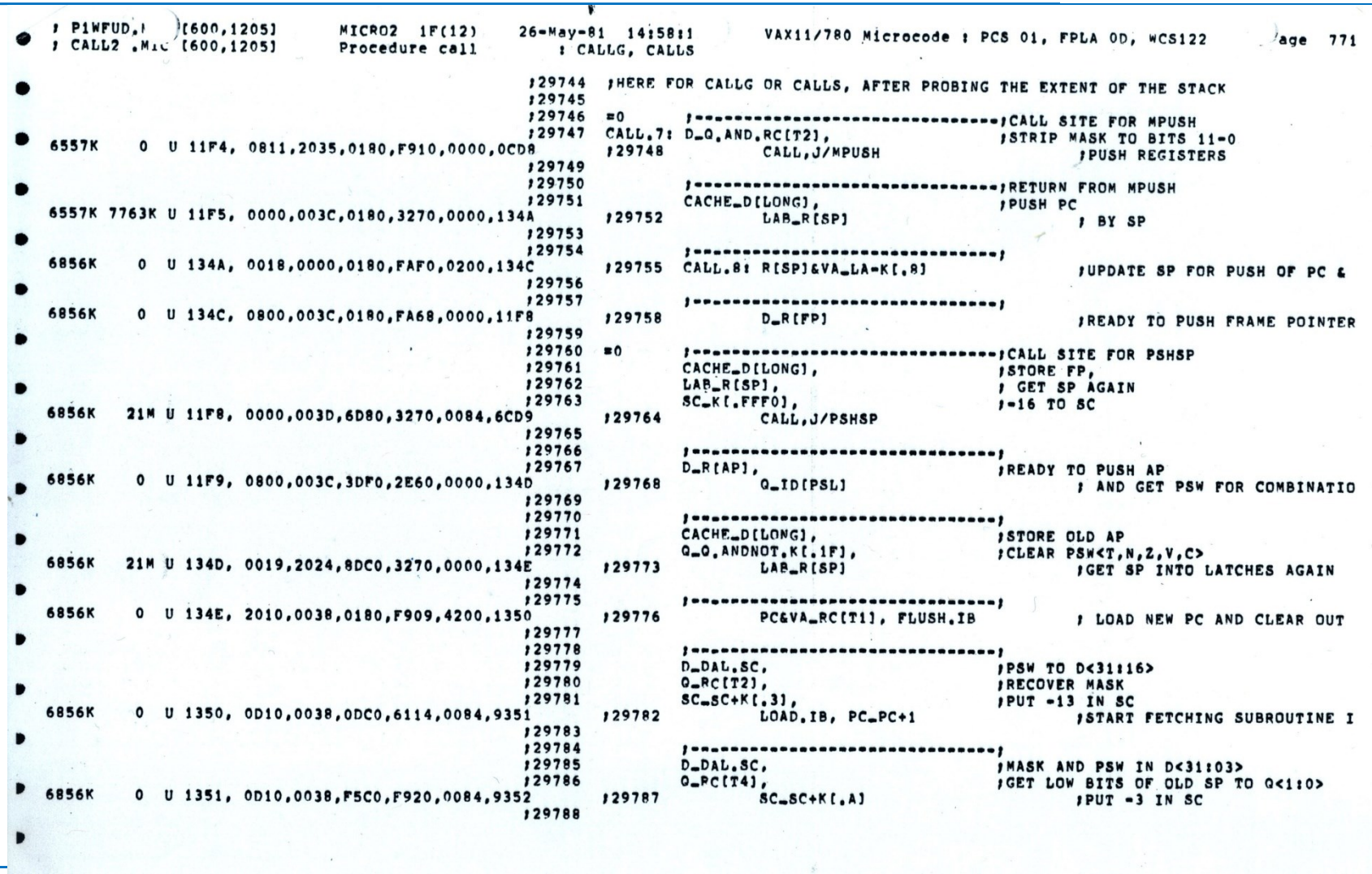
State	Control points	next-state
beq_0	A <- Reg[rs1]	next
beq_1	B <- Reg[rs2]	next
beq_2	A <- PC	ffalse
beq_3	A <- A- 4	next
beq_3	B <- BImm<<1	next
beq_4	PC <- A+B	fetch

Jumps: *RISC-V-Controller-2*

Embedded Systems Laboratory

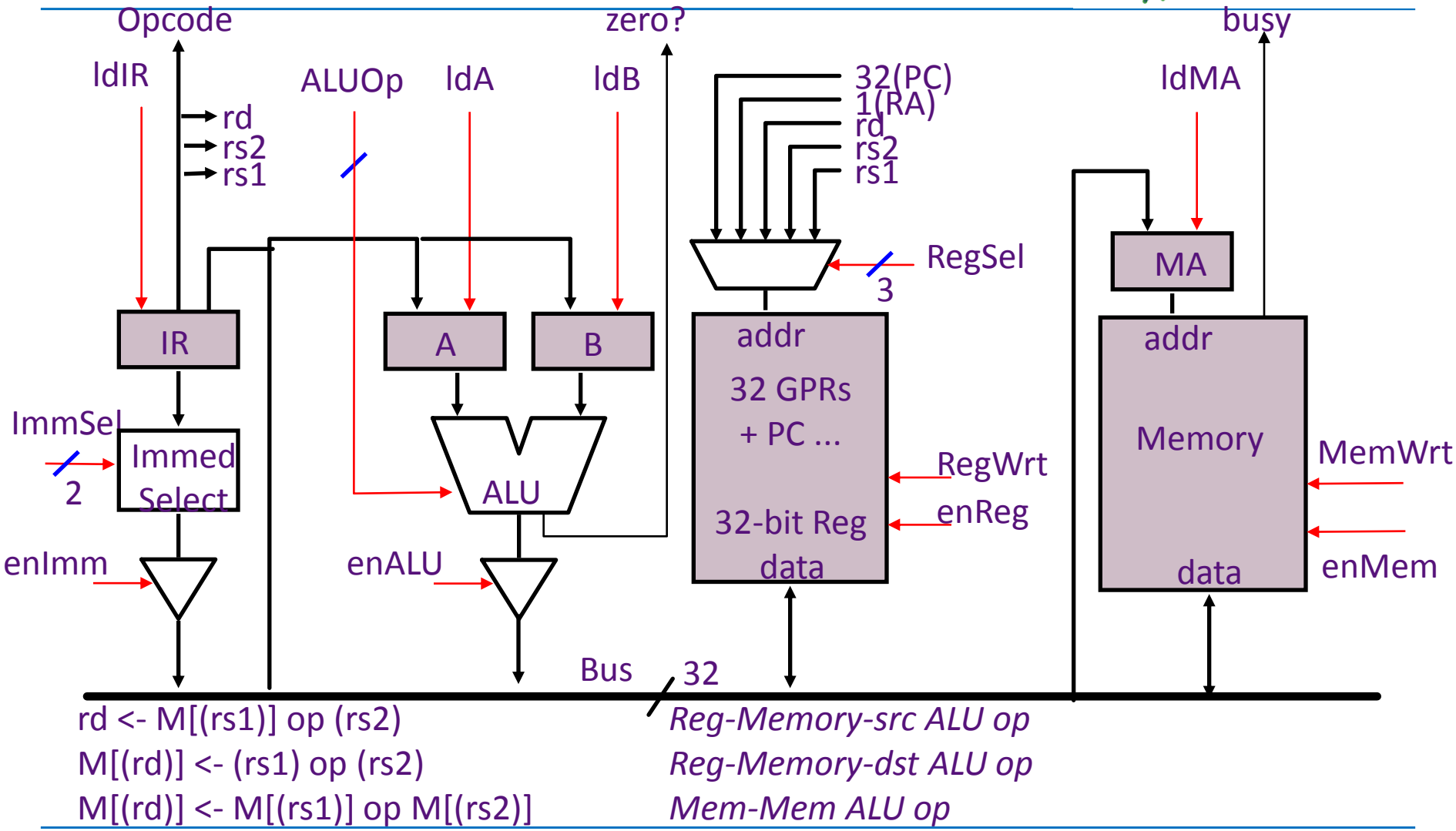
State	Control points	next-state
J_0	A <- A-4	next
J_1	B <- IR	next
J_2	PC <- JumpTarg(A,B)	fetch
JR_0	A <- Reg[rs1]	next
JR_1	PC <- A	fetch
JAL_0	A <- PC	next
JAL_1	Reg[1] <- A	next
JAL_2	A <- A-4	next
JAL_3	B <- IR	next
JAL_4	PC <- JumpTarg(A,B)	fetch

VAX 11-780 Microcode



```
; P1WFUD.!      [600,1205]      MICRO2  1F(12)     26-May-81  14:58:1      VAX11/780 Microcode ! PCS 01, FPLA 0D, WCS122      Page  771
; CALL2 .MIC    [600,1205]      Procedure call        ! CALLG, CALLS

                                                ;29744  ;HERE FOR CALLG OR CALLS, AFTER PROBING THE EXTENT OF THE STACK
                                                ;29745
                                                ;29746  =0      ;-------------------------------;CALL SITE FOR MPUSH
                                                ;29747  CALL.7: D_Q,AND,RC[T2],                 ;STRIP MASK TO BITS 11-0
6557K     0  U 11F4, 0811,2035,0180,F910,0000,0CD8  ;29748          CALL,J/MPUSH                  ;PUSH REGISTERS
                                                ;29749
                                                ;29750          ;-------------------------------;RETURN FROM MPUSH
                                                ;29751          CACHE_D[LONG],                ;PUSH PC
6557K  7763K U 11F5, 0000,003C,0180,3270,0000,134A  ;29752          LAB_R[SP]                     ; BY SP
                                                ;29753
                                                ;29754          ;-------------------------------;
6856K     0  U 134A, 0018,0000,0180,FAF0,0200,134C  ;29755  CALL.8: R[SP]&VA_LA-K[,8]             ;UPDATE SP FOR PUSH OF PC &
                                                ;29756
                                                ;29757          ;-------------------------------;
6856K     0  U 134C, 0800,003C,0180,FA68,0000,11F8  ;29758          D_R[FP]                       ;READY TO PUSH FRAME POINTER
                                                ;29759
                                                ;29760  =0      ;-------------------------------;CALL SITE FOR PSHSP
                                                ;29761          CACHE_D[LONG],                ;STORE FP,
                                                ;29762          LAB_R[SP],                    ; GET SP AGAIN
                                                ;29763          SC_K[,FFF0],                  ;-16 TO SC
6856K    21M U 11F8, 0000,003D,6D80,3270,0084,6CD9  ;29764          CALL,J/PSHSP
                                                ;29765
                                                ;29766          ;-------------------------------;
                                                ;29767          D_R[AP],                      ;READY TO PUSH AP
6856K     0  U 11F9, 0800,003C,3DF0,2E60,0000,134D  ;29768          Q_ID[PSL]                     ; AND GET PSW FOR COMBINATIO
                                                ;29769
                                                ;29770          ;-------------------------------;
                                                ;29771          CACHE_D[LONG],                ;STORE OLD AP
                                                ;29772          Q_Q,ANDNOT,K[,1F],            ;CLEAR PSW<T,N,Z,V,C>
6856K    21M U 134D, 0019,2024,8DC0,3270,0000,134E  ;29773          LAB_R[SP]                     ;GET SP INTO LATCHES AGAIN
                                                ;29774
                                                ;29775          ;-------------------------------;
6856K     0  U 134E, 2010,0038,0180,F909,4200,1350  ;29776          PC&VA_RC[T1], FLUSH,IB        ; LOAD NEW PC AND CLEAR OUT
                                                ;29777
                                                ;29778          ;-------------------------------;
                                                ;29779          D_DAL,SC,                     ;PSW TO D<31:16>
                                                ;29780          Q_RC[T2],                     ;RECOVER MASK
                                                ;29781          SC_SC+K[,3],                  ;PUT -13 IN SC
6856K     0  U 1350, 0D10,0038,0DC0,6114,0084,9351  ;29782          LOAD,IB, PC_PC+1              ;START FETCHING SUBROUTINE I
                                                ;29783
                                                ;29784          ;-------------------------------;
                                                ;29785          D_DAL,SC,                     ;MASK AND PSW IN D<31:03>
                                                ;29786          Q_RC[T4],                     ;GET LOW BITS OF OLD SP TO Q<1:0>
6856K     0  U 1351, 0D10,0038,F5C0,F920,0084,9352  ;29787          SC_SC+K[,A]                   ;PUT -3 IN SC
                                                ;29788
```

C


Opcode
zero?
busy
ldIR
ALUOp
ldA
ldB
32(PC)
1(RA)
rd
rs2
rs1
ldMA
rd
rs2
rs1
RegSel
3
MA
IR
A
B
addr
32 GPRs
+ PC ...
32-bit Reg
data
addr
Memory
data
ImmSel
2
Immed
Select
ALU
RegWrt
enReg
MemWrt
enMem
enImm
enALU
Bus
32
rd <- M[(rs1)] op (rs2)
M[(rd)] <- (rs1) op (rs2)
M[(rd)] <- M[(rs1)] op M[(rs2)]
Reg-Memory-src ALU op
Reg-Memory-dst ALU op
Mem-Mem ALU op

Instrucțiuni Mem-Mem ALU:

RISC-V-Controller-2

Embedded Systems Laboratory

Mem-Mem ALU op	M[(rd)] <- M[(rs1)] op M[(rs2)]	
$ALUMM_0$	MA <- Reg[rs1]	next
$ALUMM_1$	A <- Memory	spin
$ALUMM_2$	MA <- Reg[rs2]	next
$ALUMM_3$	B <- Memory	spin
$ALUMM_4$	MA <- Reg[rd]	next
$ALUMM_5$	Memory <- func(A,B)	spin
$ALUMM_6$		fetch

Instrucțiunile complexe nu necesită de obicei modificări ale structurii unității de control microprogramat

-- doar spațiu suplimentar în memoria ROM de control

Implementarea acestor instrucțiuni folosind un controller hardware dedicat nu poate fi făcută fără modificarea căilor de date din interiorul unității de control.

Considerente de performanță

Unitatea de control microprogramat

-> cicli multipli per instrucțiune

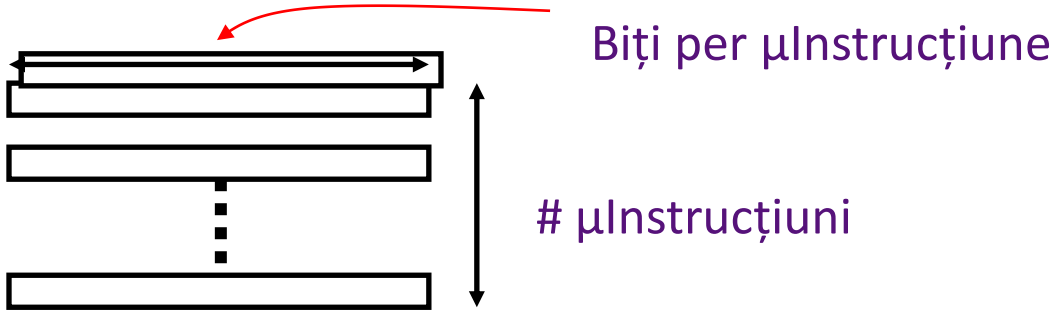
Durata ciclului ?

$t_C > \max(t_{reg\text{-}reg}, t_{ALU}, t_{ROM})$

Presupunem că $10 * t_{ROM} < t_{RAM}$

Performanță mărită, atunci când comparăm cu o implementare pur hardware a UC care execută toate operațiile într-un singur ciclu.

µCod orizontal vs. vertical

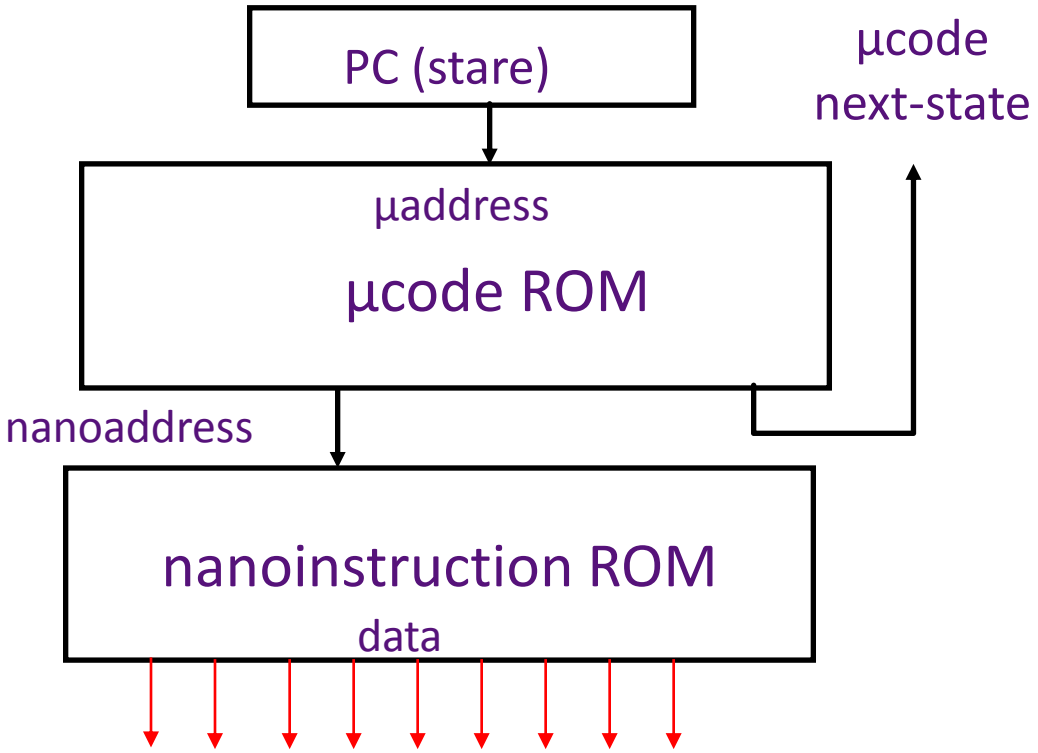


- µcodul orizontalizat are µinstrucțiuni mai lungi
  - Mai multe operații per µinstrucțiune
  - Mai puțini pași în microcod pe macroinstrucțiune
  - Sparse encoding ⇒ mai mulți biți
- µcodul verticalizat are µinstrucțiuni scurte
  - De obicei, o singură operație de transfer date per µinstrucțiune
    - µinstrucțiuni separate pentru branch
  - Mai mulți pași în microcod per macroinstrucțiune
  - Mai compact ⇒ mai puțini biți
- Nanocoding
  - Încearcă să combine cele două practici

Nanocoding

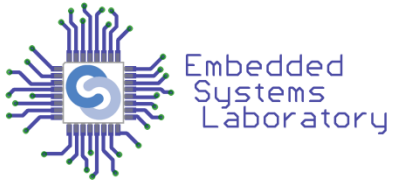
Exploatează existența secvențelor recurente de µcod, e.g.,

ALU_0 A <- Reg[rs1]
...
$ALUi_0$ A <- Reg[rs1]
...



MC68000 avea un microcod pe 17-biți ce conținea un µjump pe 10 biți sau un pointer pe 9 biți la o nanoinstrucțiune
- Nanoinstrucțiunile aveau 68 de biți lățime și decodificate produceau 196 semnale de control

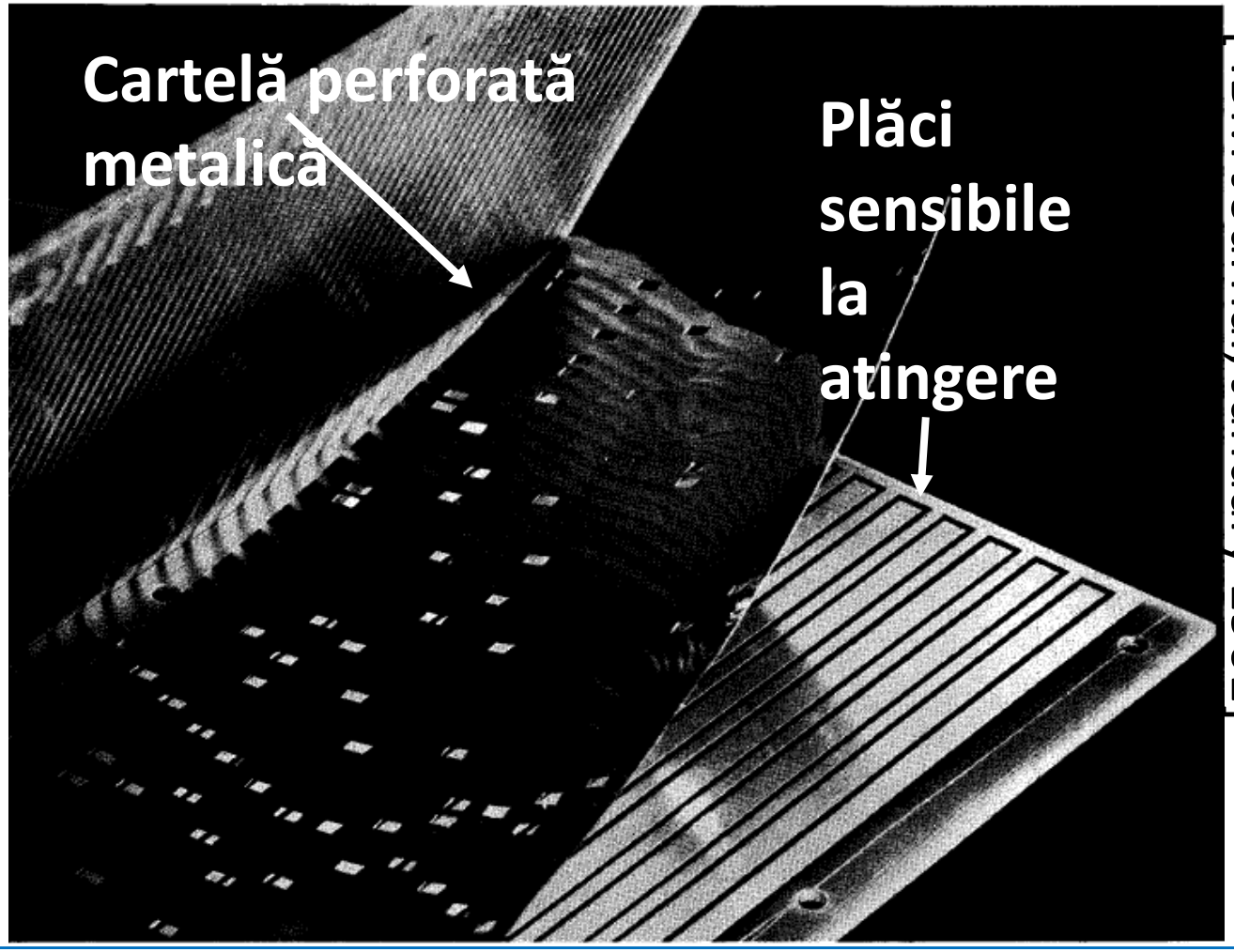
Microprogramarea pentru IBM 360



	M30	M40	M50	M65
Datapath width (bits)	8	16	32	64
µinst width (bits)	50	52	85	87
µcode size (K µinsts)	4	4	2.75	2.75
µstore technology	CCROS	TCROS	BCROS	BCROS
µstore cycle (ns)	750	625	500	200
memory cycle (ns)	1500	2500	2000	750
Rental fee ($K/month)	4	7	15	35

Numai cele mai rapide modele (75 și 95) aveau UC hardwired

IBM Card Capacitor Read-Only Storage



[IBM Journal, January 1961]

Emularea Microcodului

- IBM a făcut o eroare atunci când a subestimat importanța compatibilității software cu modelele mai vechi atunci când a introdus seria 360.
- Honeywell a furat o parte din utilizatorii IBM1401 atunci când a oferit un pachet software de translație a programelor IBM în cod pentru seria Honeywell H200
- IBM a răspuns prin adăugarea de microcod opțional pentru seria 360 care putea să emuleze ISA IBM1401, pe care l-a extins mai apoi pentru seria IBM7000
  - Un program foarte popular pentru 1401 era un simulator al seriei 650 pentru că mulți clienți rulau programe de 650 emulate pe mașini 1401.
    - (*650, simulat pe 1401, emulat pe 360)*

Microprogramarea a atins apogeul în anii ‘70

- Memoriile ROM erau cu mult mai rapide decât cele DRAM
- Pentru un ISA complex, unitatea de control microprogramat era mai ieftină și mai simplu de fabricat
- *Instrucțiunile noi*, e.g., floating point, puteau fi adăugate fără modificări ale magistralelor de date
- *Repararea defectelor* controllerului era mai simplă
- Compatibilitatea ISA a numeroaselor modele de calculatoare putea fi obținută ușor și ieftin

Aproape toate calculatoarele acelei epoci aveau o unitate de control microprogramată

Writable Control Store (WCS)

- Implementăm memoria de control în RAM, nu în ROM
  - Memoriile MOS SRAM sunt acum aproape la fel de rapide ca un ROM (memoriile cu mies de ferită/DRAM erau de 2-10x mai lente)
  - E foarte greu să scrii un microprogram fără erori din prima încercare
- Opțiunea de User-WCS pentru anumite procesoare
  - Permite utilizatorilor să schimbe microcodul pentru procesorul în cauză
- User-WCS *a eșuat*
  - Nu au existat suficiente programe care să folosească această abilitate
  - Dificil de stocat software util într-un spațiu atât de mic
  - Unitatea de control era optimizată pentru microcodul original, mai puțin pentru restul
  - Protecția integrității ISA dificilă dacă utilizatorul poate schimba microcodul
  - Memoria virtuală necesita microcod restartabil

Microprogramarea nu a dispărut

- A jucat un rol crucial în dezvoltarea microprocesoarelor anilor 80
  - DEC uVAX, Motorola 68K series, Intel 286/386
- Joacă un rol secundar, dar important, în microprocesoarele moderne
  - e.g., AMD Bulldozer, Intel Ivy Bridge, Intel Atom, IBM PowerPC, …
  - Majoritatea instrucțiunilor sunt executate direct (logică cablată)
  - Instrucțiunile utilizate mai puțin frecvent și/sau complicate, folosesc microcod
- Se pot aplica patch-uri la microcod pentru a repara anumite erori ISA detectate post-fabricare (ex. Procesoarele Intel încarcă patch-uri de microcod la bootup)

Acknowledgements

- Aceste slide-uri conțin materiale aparținând:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)
  - Behrooz Parhami (UCSB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

Calculatoare Numerice

-Cursul 7 –

Implementarea unei mașini de calcul

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Comic of the day

http://dilbert.com/strips/comic/1990-04-10/

Din cursurile trecute

- Computer Architecture >> ISAs & RTL
  - Cursul se ocupă de interacțiunea hardware/software și de proiectarea nivelurilro potrivite de abstractizare
- Tehnologia și aplicațiile ei modelează arhitectura sistemelor de calcul
  - Trecutul poate da lecții pentru dezvoltarea tehnicii
- Primii 130 de ani de CompArch, de la Babbage la IBM 360
  - Evoluția de la calculatoare mecanice la mașini complet programabile
  - Schimbarea rapidă în timpul WWII (anii 45), tranziție de la procesoare electro-mecanice la electronice
- Costul software development devine o problemă pentru dezvoltarea arhitecturilor de calcul (era nevoie de compatibilitate)
- IBM 360 introduce noțiunea de “familie de calculatoare” cu același ISA care rulează pe mașini complet diferite hardware
  - Șase mașini diferite lansate în aceeaș zi (7 aprilie 1964)
  - “Future-proofing” pentru generațiile viitoare

IBM 360: Implementarea inițială

	Model 30	. . .	*Model 70*
Memory	8K - 64 KB		256K - 512 KB
Datapath	8-bit		64-bit
Circuit Delay	30 nsec/level		5 nsec/level
Local Store	Main Store		Transistor Registers
Control Store	Read only 1~sec		Conventional circuits

Arhitectura setului de instrucțiuni (ISA) IBM 360 ascundea complet diferențele de tehnologie dintre diferitele modele.

Milestone: Primul ISA proiectat ca o interfață portabilă între hardware și software!

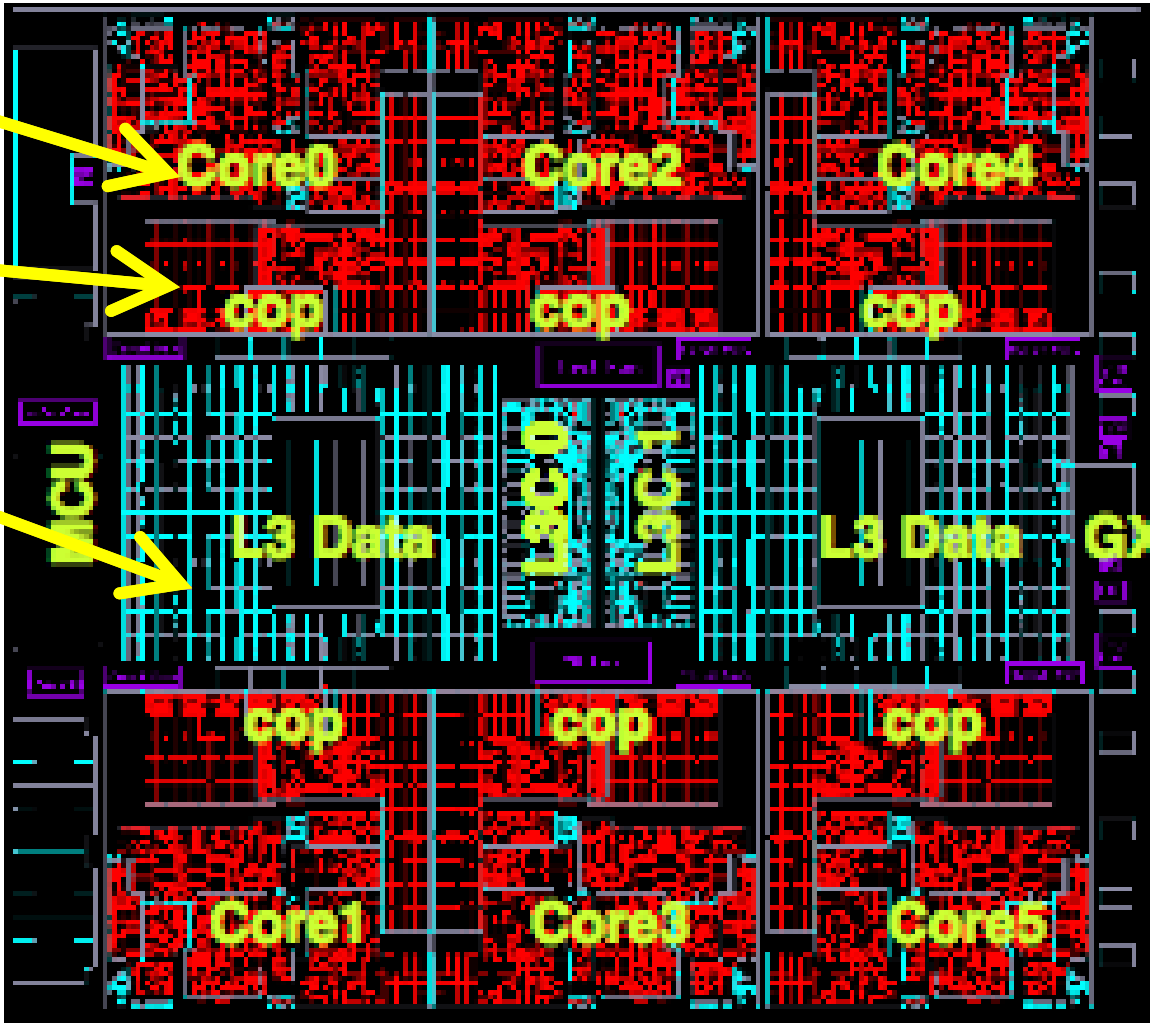
IBM 360 Supraviețuiește și azi: z12 Mainframe Processor

Embedded Systems Laboratory

6 Cores @ 5.5 GHz

Coprocesor special-purpose pe fiecare core

48MB Level-3 cache on chip



32nm SOI Technology
2.75 miliarde tranzistoare
23.7mm x 25.2mm
15 straturi de metal
12.3 km de conexiuni!
10,000 pini de alimentare (!)
1,071 I/O pins

[IBM HotChips24 presentation, August 28, 2012]

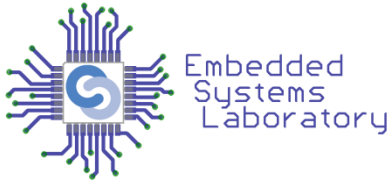
Instruction Set Architecture (ISA)

- Punctul de contact dintre software și hardware
- Descrisă în mod tipic prin punerea la dispoziția programatorilor a registrelor, memoriei și a semanticii instrucțiunilor care operează pe mașina respectivă
- IBM 360 a fost prima familie de mașini de calcul care a separat ISA de implementarea hardware propriu-zisă (aka. *microarchitectură*)
- Mai multe implementări posibile pentru aceeași ISA
  - E.g., sovieticii au construit clone compatibile ale IBM360, precum si Amdahl, după plecarea din IBM.
  - E.g.2., astăzi, procesoarele Intel și AMD folosesc ISA x86-64.
  - E.g.3: majoritatea telefoanelor mobile folosesc ISA ARM cu implementări hardware de la mai multe companii, printre care și TI, Qualcomm, Samsung, Marvell, etc.

Maparea ISA pe microarhitectură

- ISA poate fi proiectată ținând cont tipul de microarhitectură:
  - Acumulator ⇒ hardwired, unpipelined
  - CISC ⇒ microcoded
  - RISC ⇒ hardwired, pipelined
  - VLIW ⇒ fixed-latency in-order parallel pipelines
  - JVM ⇒ software interpretation
- Dar poate fi implementată folosind orice fel de microarhitectură
  - Intel Ivy Bridge: hardwired pipelined CISC (x86)
  - Simics: Software-interpreted SPARC RISC machine
  - ARM Jazelle: Procesor JVM hardware
  - În acest curs: mașină RISC-V cu microcod

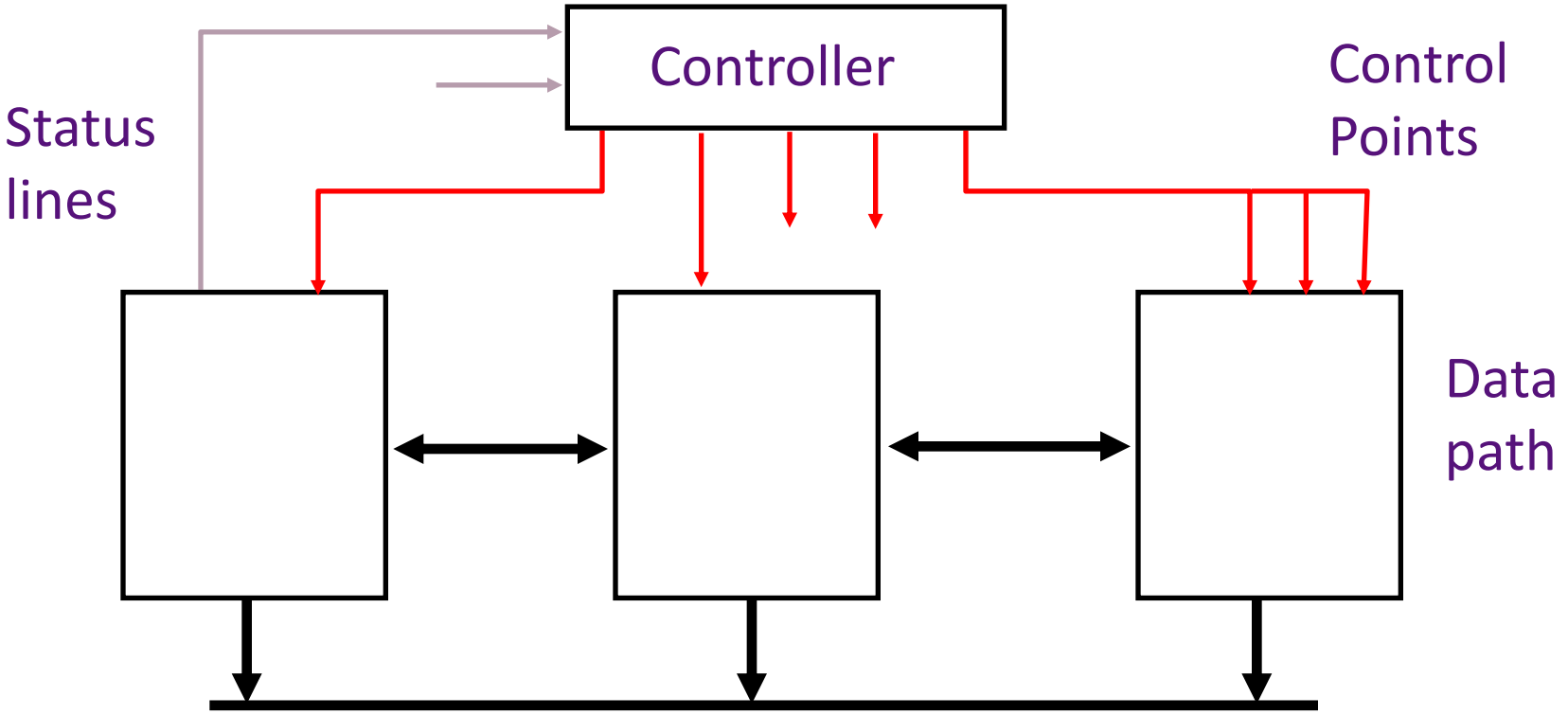
Microprogramarea în cursul de azi



- Pentru a arăta cum pot fi contruite procesoare mici care rulează un ISA complex
- Pentru a vă ajuta să înțelegeți de ce există mașini CISC*
- Pentru că este încă folosită la scară largă (IBM360, x86, PowerPC)
- Servește ca o introducere (mai) blândă în structura mașinilor de calcul
- Pentru a vă face să înțelegeți de ce tehnologia s-a centrat pe RISC*

* “CISC”/”RISC” sunt denumiri mult mai noi decât arhitecturile la care fac referință.

Microarchitectură:
Implementarea unei ISA



Structură: Cum sunt interconectate componentele.

Statică

Comportament: Cum se miscă datele între diferitele componente

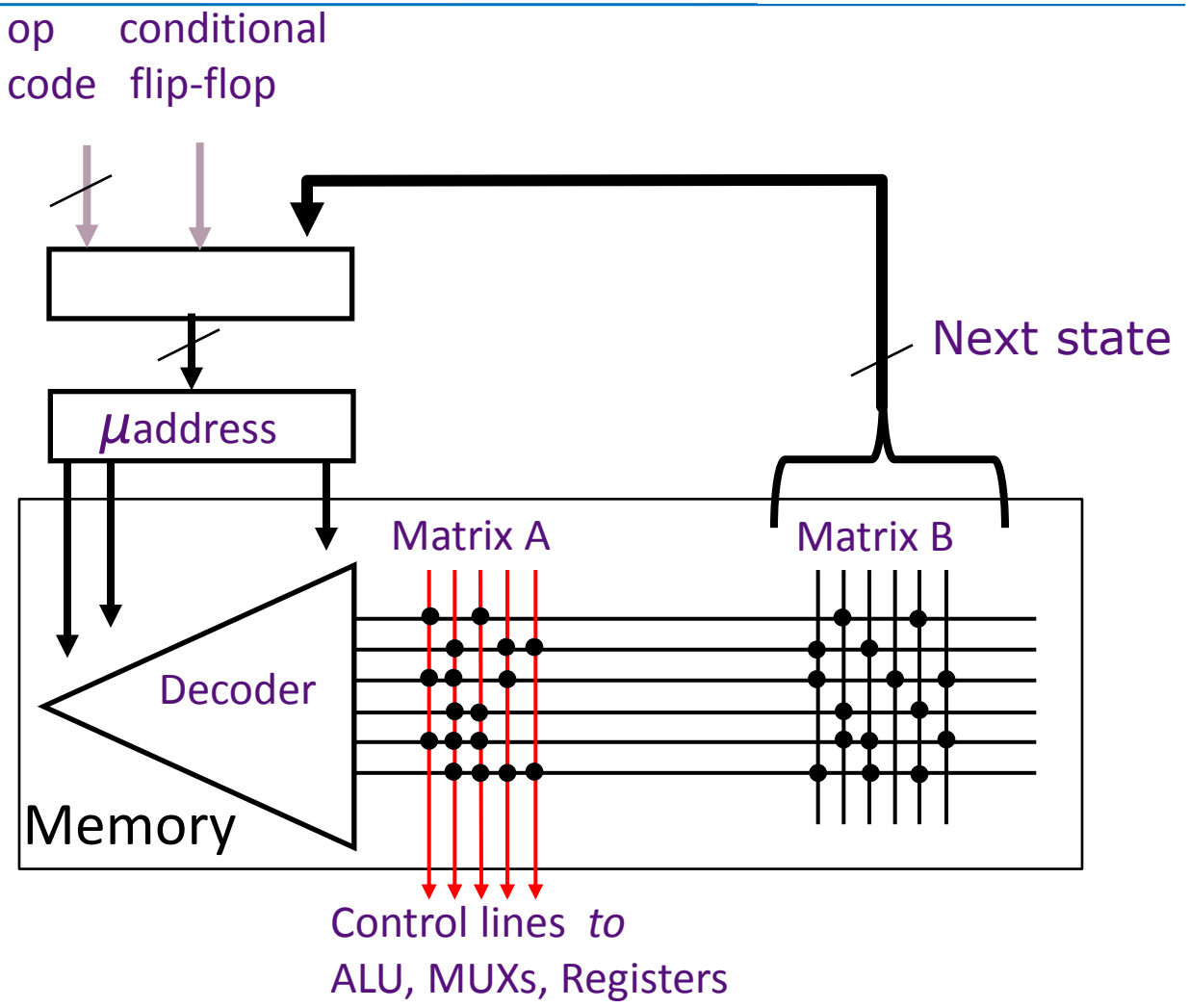
Dinamică

Unitate control microprogramat

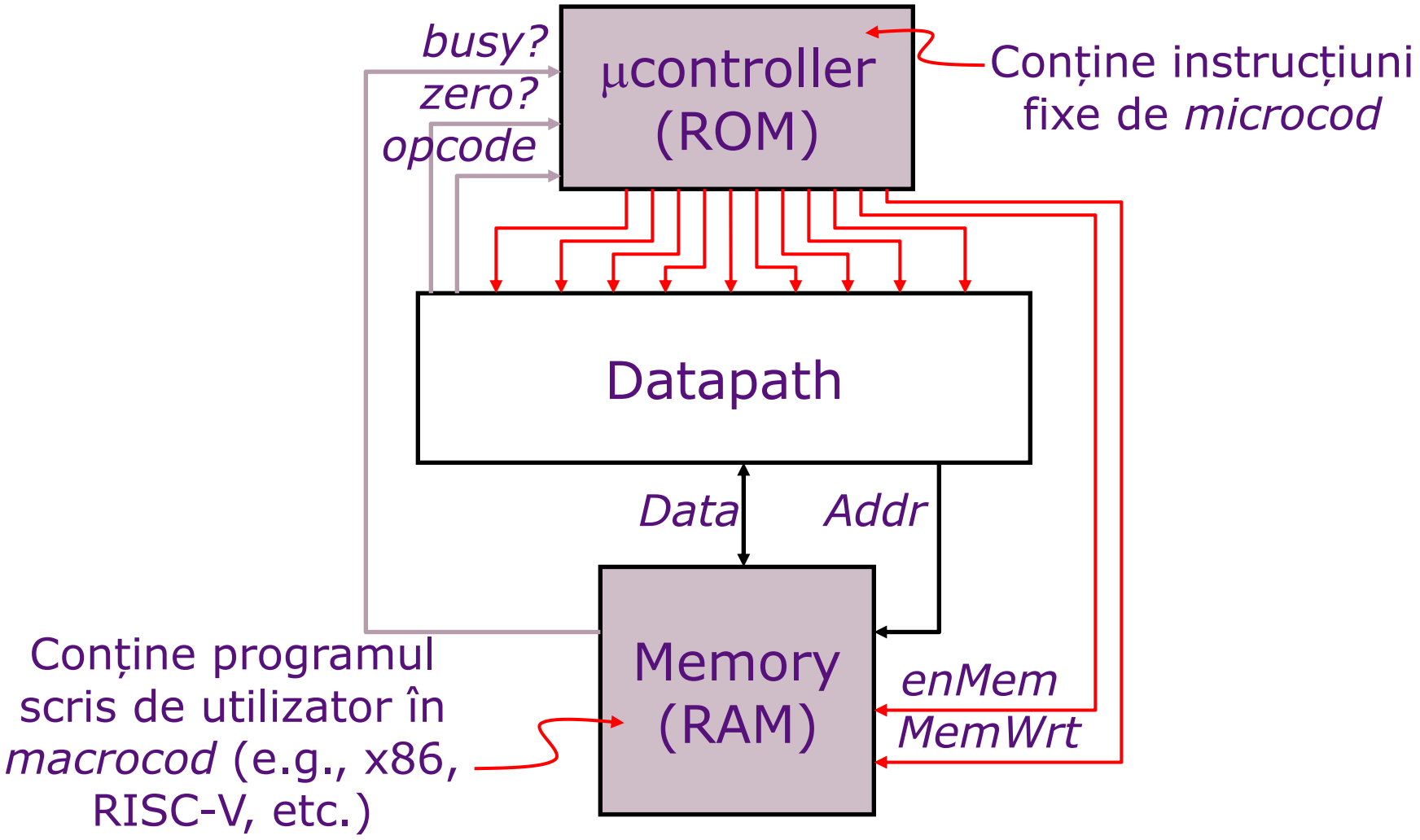
Maurice Wilkes, 1954

Folosită prima dată în EDSAC-2, 1958

Încorporează tabela de stări pentru logica de control într-o memorie



Microarchitectură microcodată



RISC-V ISA

- Design RISC de la UC Berkeley
- ISA complet dar de mici dimensiuni și deschis
- Nu este supra-optimizat pentru un anumit stil de implementare
- Două variante, pentru 32 și 64 de biți
  - RV32 & RV64
- Proiectat pentru multiprocesoare
- Codificare eficientă a instrucțiunilor
- Ușor de extins/restrâns pentru educație/cercetare

- Pentru acest curs vom folosi o arhitectură RISC-V pe 32 de biți, foarte similară cu a procesorului MIPS

RV32 Processor State

Embedded Systems Laboratory

Program counter (**pc**)

32x32-bit registre pt întregi (**x0-x31**)
- **x0** conține întotdeauna 0

32 registre floating-point (FP) (**f0-f31**)
- fiecare poate conține o valoare single- sau double-precision FP (32-bit sau 64-bit IEEE FP)

FP status register (**fsr**), folosit pentru FP rounding mode & raportare excepții

XPRLEN-1 … 0
x0 / zero
x1 / ra
x2
x3
x4
x5
x6
x7
x8
x9
x10
x11
x12
x13
x14
x15
x16
x17
x18
x19
x20
x21
x22
x23
x24
x25
x26
x27
x28
x29
x30
x31

XPRLEN

63 … 0
f0
f1
f2
f3
f4
f5
f6
f7
f8
f9
f10
f11
f12
f13
f14
f15
f16
f17
f18
f19
f20
f21
f22
f23
f24
f25
f26
f27
f28
f29
f30
f31

64

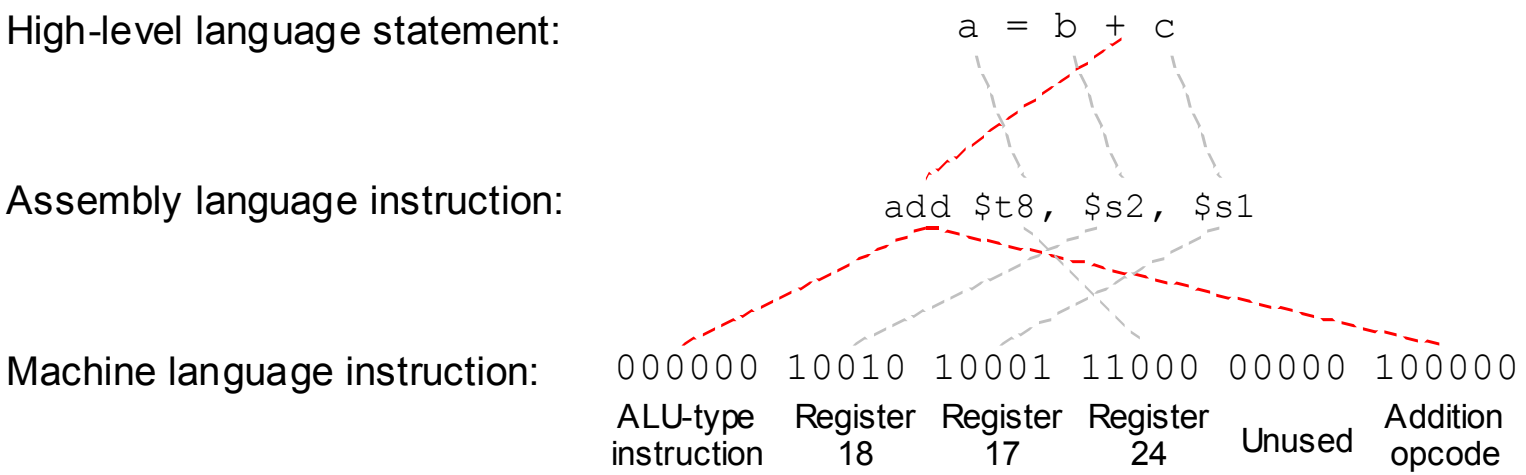
XPRLEN-1 … 0
pc

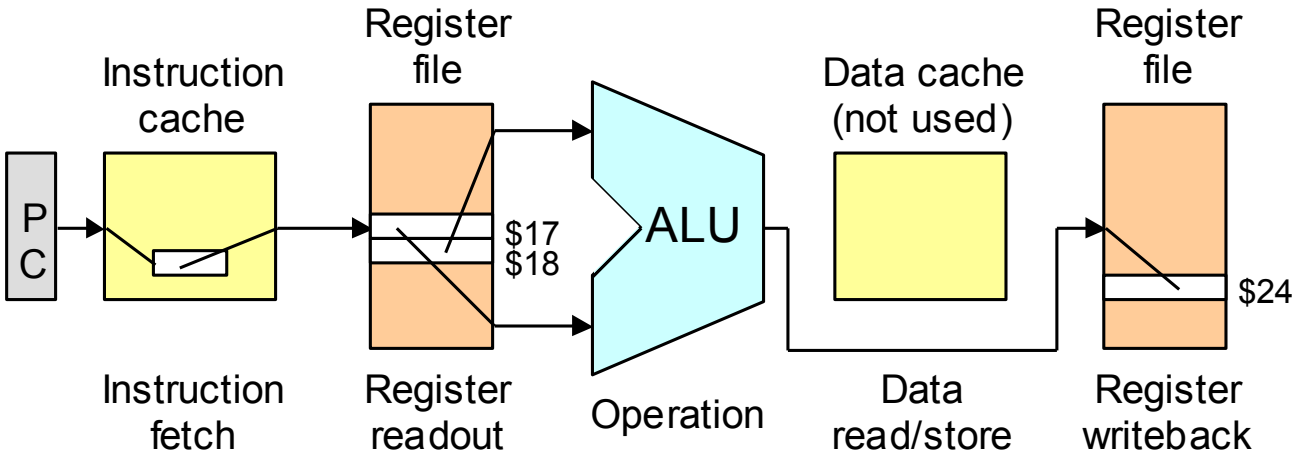
XPRLEN

31 … 0
fsr

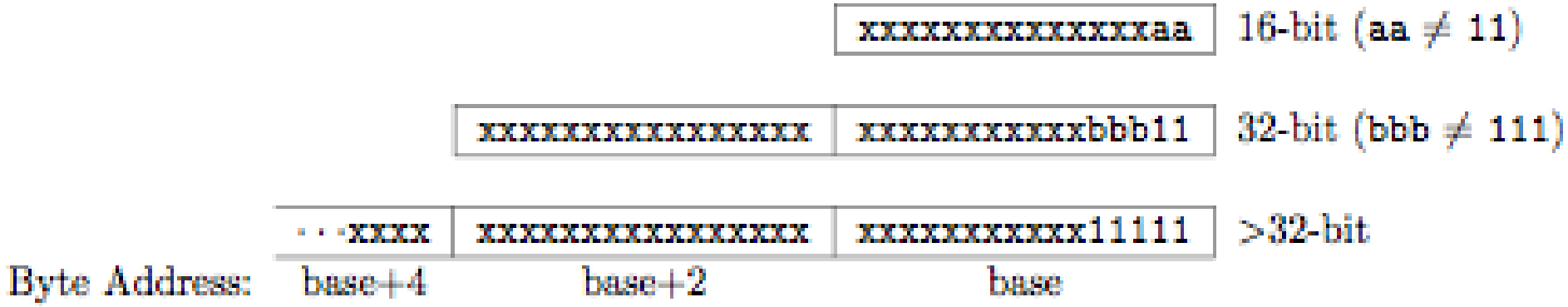
32

Formatul instrucțiunilor



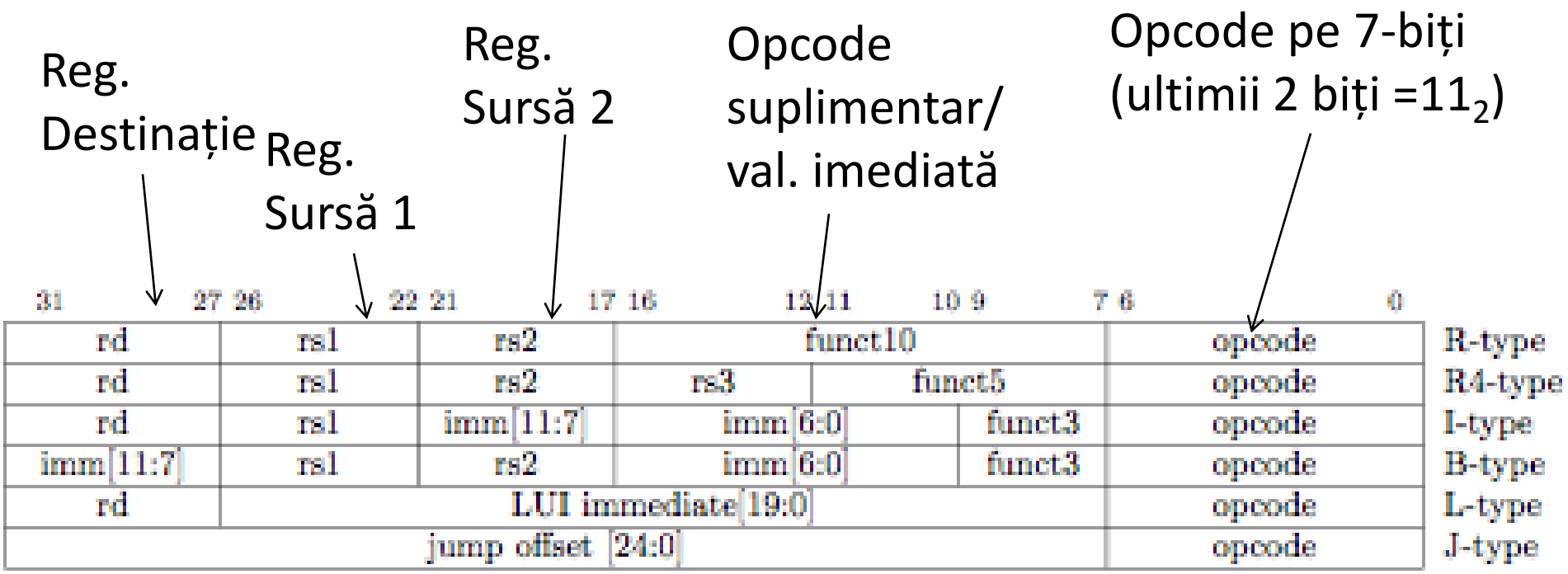


RISC-V Codificarea Instrucțiunilor

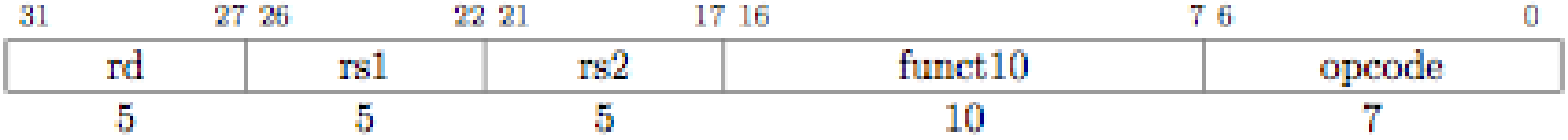


- Suport pentru instrucțiuni de lungime variabilă.
- Setul de bază (RV32) are întotdeauna instrucțiuni de dimensiune fixă, cu ultimii doi biți = 11_2
- Toate instrucțiunile de branch și jump au granularitate de 16 biți (chiar și în ISA de bază, unde toate instrucțiunile au lungimea de 32 de biți)

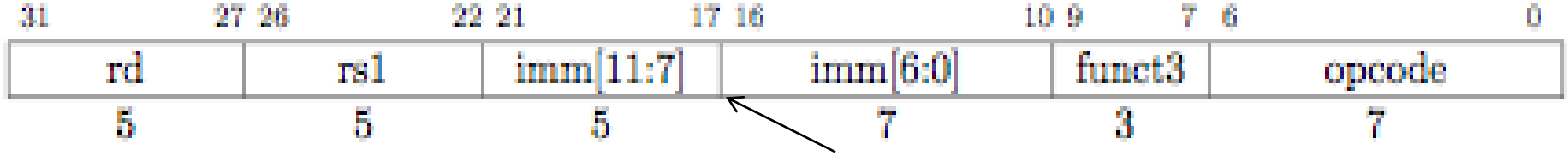
RISC-V Formatul Instrucțiunilor



R-Type/I-Type/R4-Type Formats

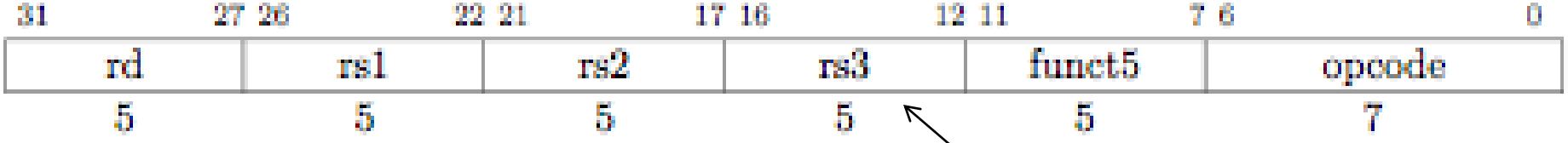


Operații Reg-Reg în ALU



Operații Reg-Imm ALU
Instrucțiuni Load, adresare (rs1 + imediat)

12-bit signed immediate

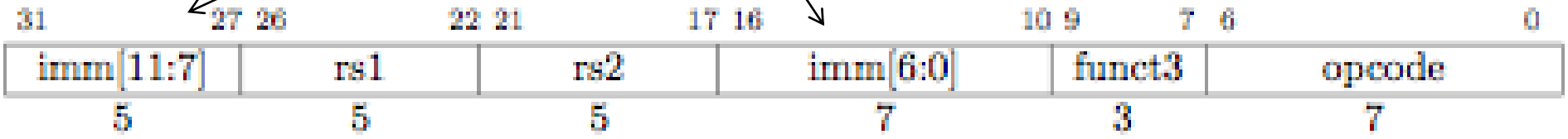


Folosite doar pentru adunări și înmulțiri în virgulă mobilă

Reg. Source 3

B-Type

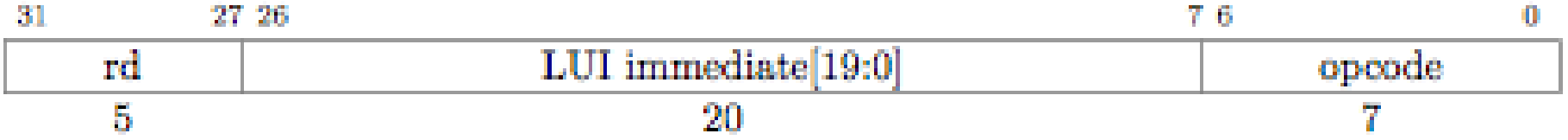
Valoare imediată cu semn pe 12 biți împărțită în 2 câmpuri



Folosită pt. Branch, comparații de registre - PC+(imediat<<1)

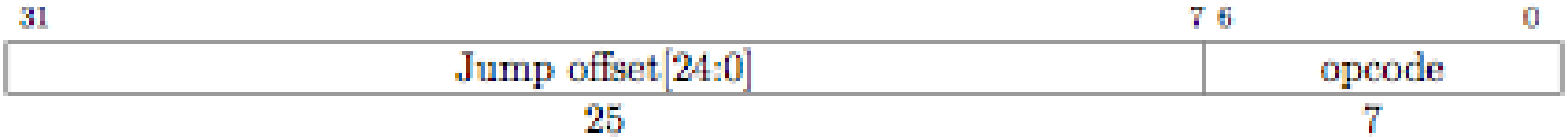
Instrucțiuni Store, adresă (rs1 + imediat) , date în rs2

L-Type



Scrie o valoare imediată de 20 de biți în registrul destinație.
Folosită pentru incărcarea de constante imediate mari.
Valorile imediate pe 12 biți sunt cu semn pentru a putea construi constante pe 32 de biți într-o secvență de doi pași (LUI high-20b, ADDI low-12b)

J-Type



"J" Jump necondiționat, PC+offset target

"JAL" Jump and link, scrie PC+4 în **x1**

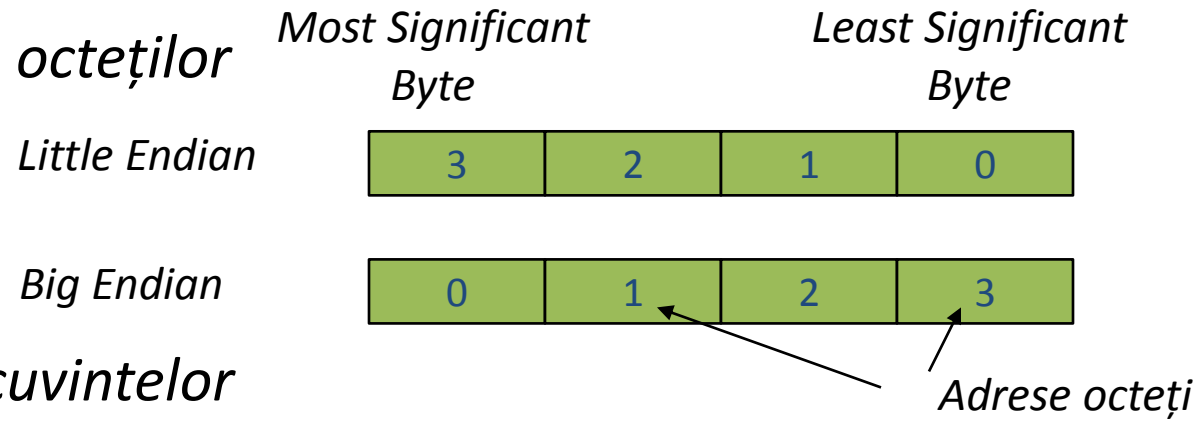
Formatul datelor și adresarea memoriei

Tipuri de date:

8-b Byte, 16-b Half word, 32-b word & 64-b double word

Remarci:

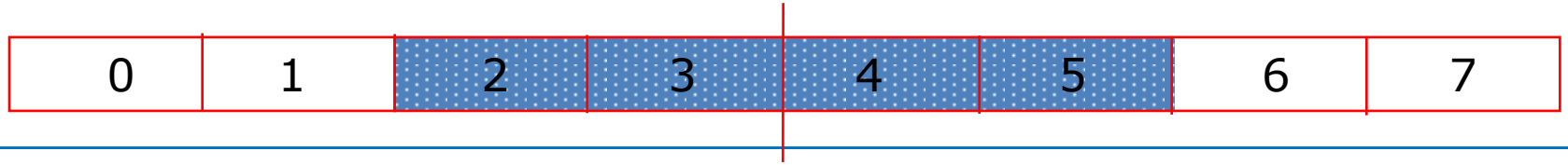
- *Adresarea octeților*



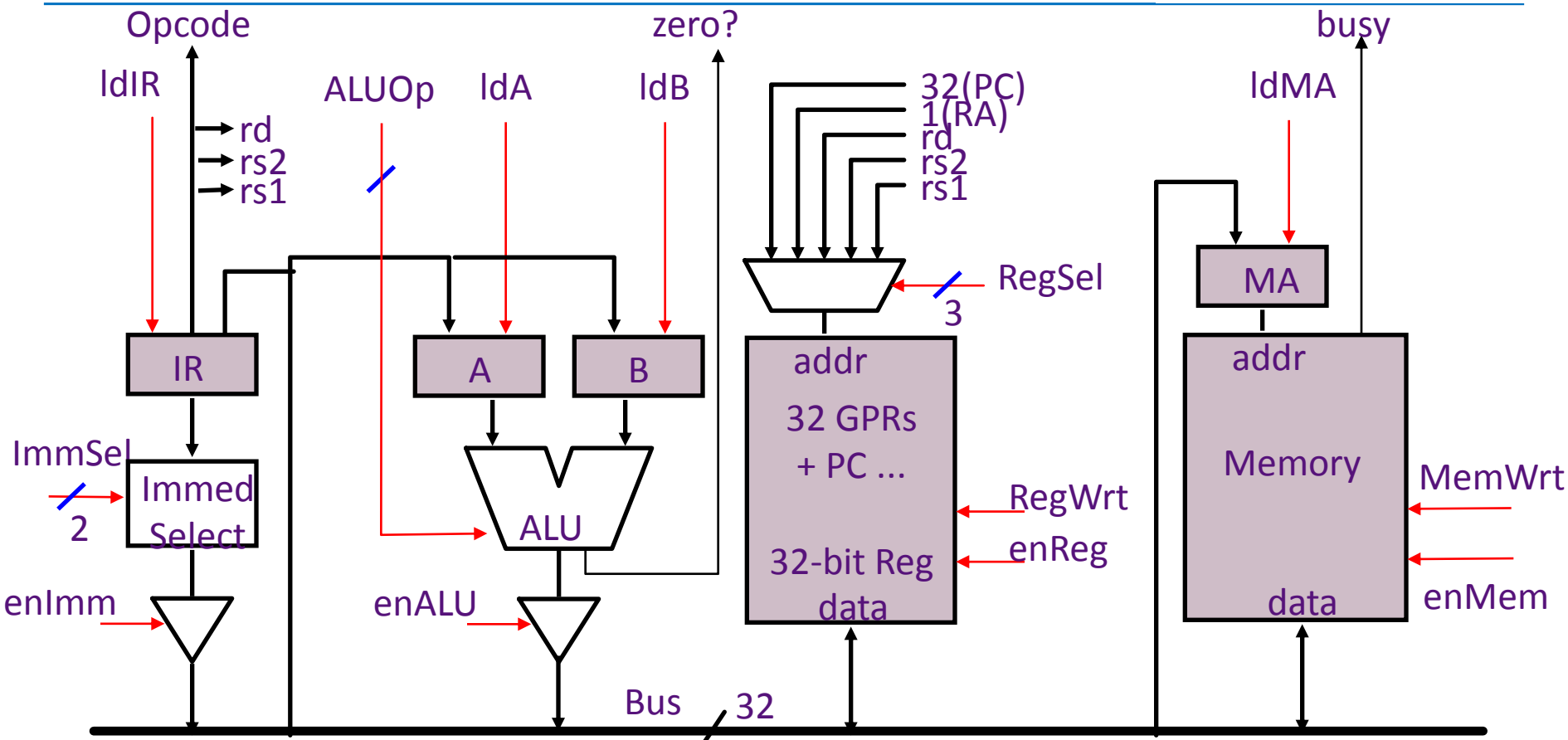
- *Alinierea cuvintelor*

Presupunem că memoria e organizată în cuvinte de 32 de biți.

Poate o adresă de cuvânt să înceapă numai la 0, 4, 8, ... ?



RISC-V – descriere generală



Microinstrucțiune: register to register transfer (17 semnale de control)

MA -> PC se traduce prin RegSel = PC; enReg=yes; ldMA= yes

B <- Reg[rs2] *se traduce prin* RegSel = rs2; enReg=yes; ldB = yes

The Big Picture

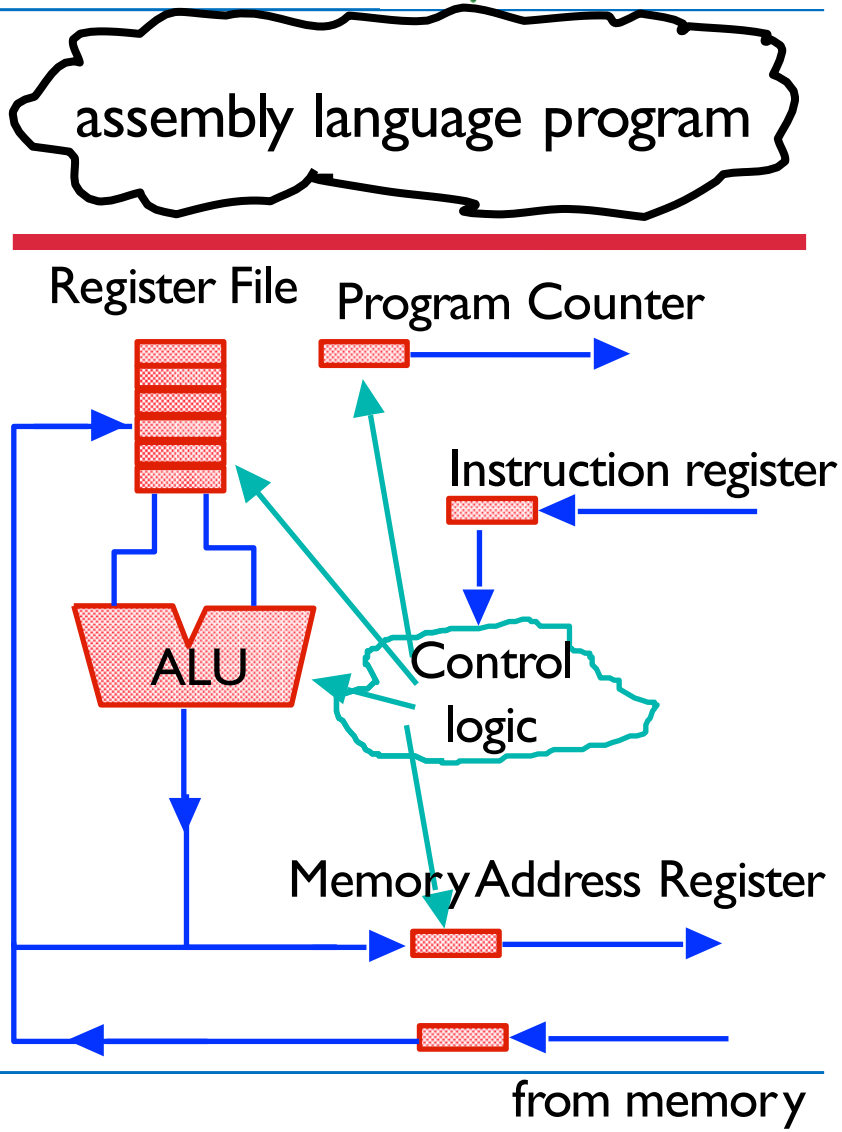
Embedded Systems Laboratory

Limbaj asamblare

Interfața pe care arhitectura o prezintă utilizatorului

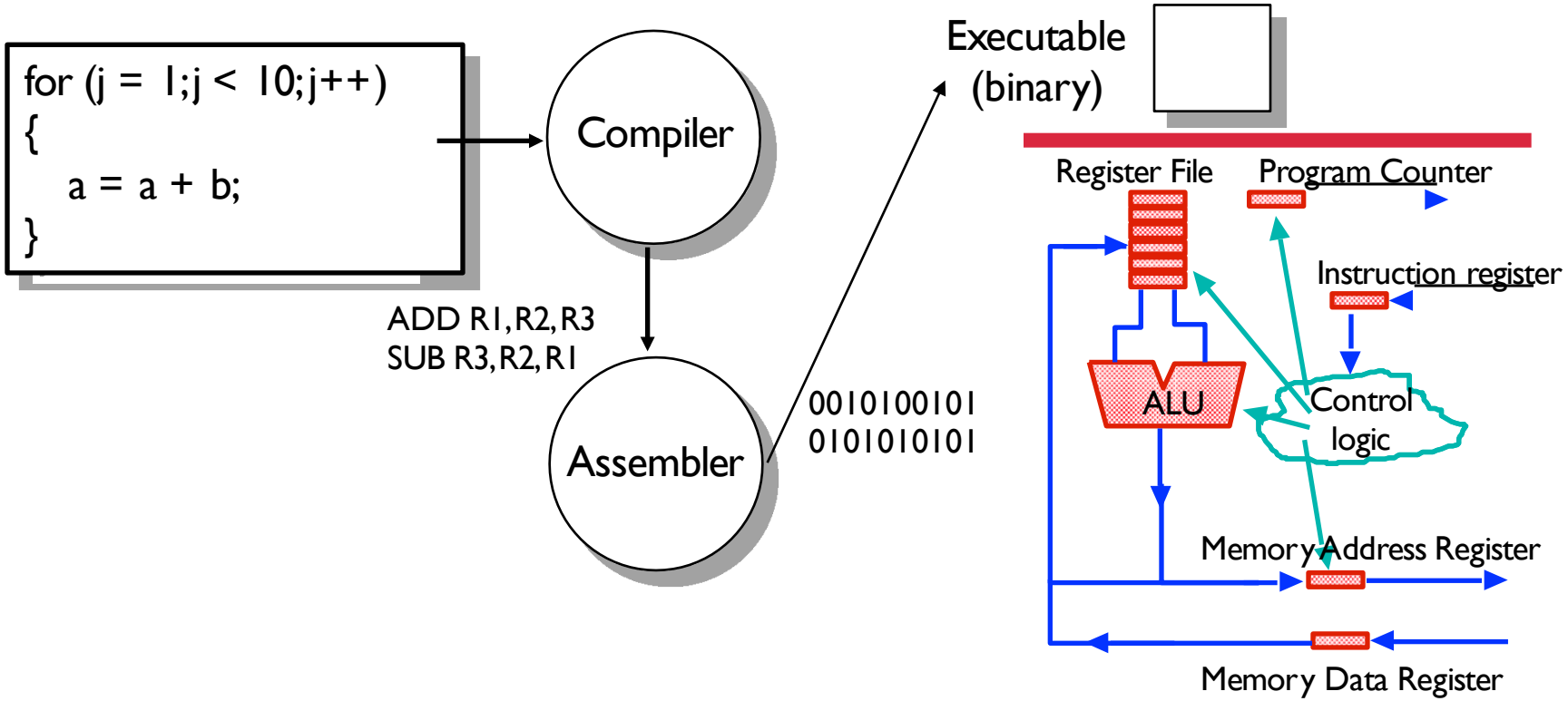
Instrucțiuni ”Low-level” ce folosesc memoria și căile de date pentru a efectua operațiile de bază

- aritmetice: add, sub, mul, div
- logice: and, or, shift
- data transfer: load, store
- (un)conditional branch: jump, branch on condition



Niveluri de abstractizare

- De obicei, oamenii nu scriu software în cod mașină
- Folosesc limbaje de nivel înalt C, C++, JAVA, Python
- Limbajul de nivel înalt este translatat în assembly de un compilator
- Assembly este translatat în cod mașină de asamblor



Clase ISA

Embedded Systems Laboratory

Memory to Memory Machines

- Avem nevoie de stocare pentru variabile temporare
- Memoria este lentă
- Memoria este mare (mulți biți de adresă)

Registre

- Registrele pot stoca variabile temporare
- Registrele sunt mai rapide decât memoria
- Traficul cu memoria este redus, iar execuția programului este accelerată (pentru că registrele sunt mia rapide ca memoria)
- Densitatea codului este îmbunătățită (pentru că registrele sunt adresate mai ușor decât memoria)

Predomina GPR Machines

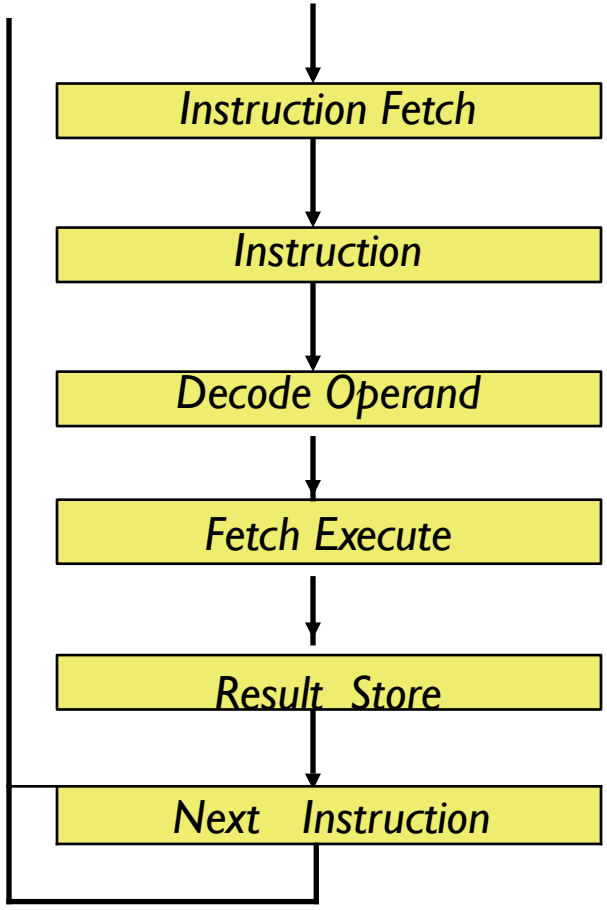
Embedded Systems Laboratory

Toate mașinile de calcul folosesc registre generale

Avantajele registrelor

- Registrele sunt mai rapide decât memoria
  - Traficul cu memoria este redus, iar execuția programului este accelerată (pentru că registrele sunt mai rapide ca memoria)
- Registrele pot stoca variabile
  - Registrele sunt mai ușor de folosit pentru un compilator: e.g., (A*B) – (C*D) – (E*F) poate executa în orice ordine vs. mașină cu stivă

Modelul execuției unei instrucțiuni



Model secvențial de execuție

Programul este o secvență de execuție

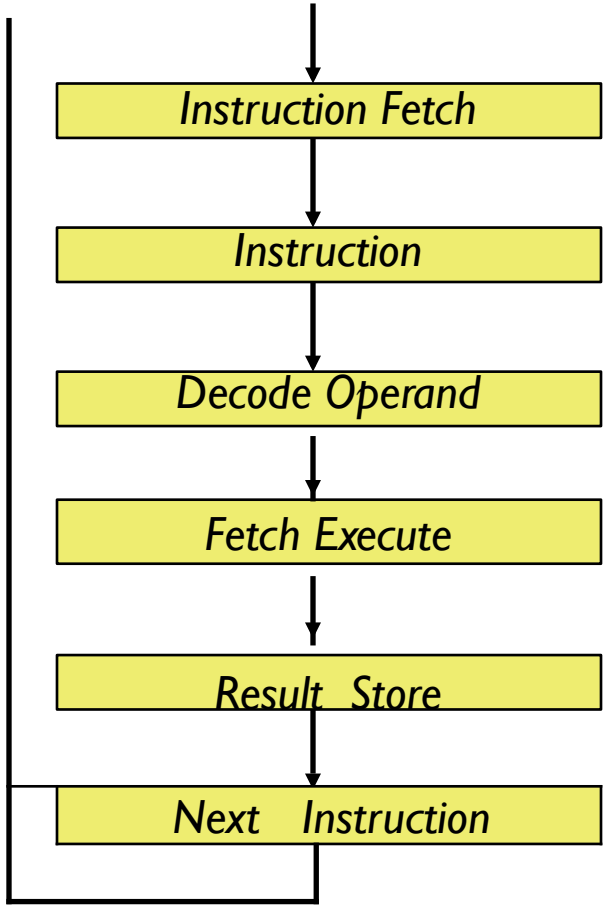
Instrucțiunile sunt atomice și executate secvențial

Conceptul programului stocat

Programul și datele sunt stocate în memorie

Instrucțiunile sunt aduse din memorie pentru execuție

Modelul execuției unei instrucțiuni



ISA Issues

Get instruction from memory

Instruction Format/Encoding

Addressing Modes

Op-codes and DataTypes

Addressing Modes

Instruction Sequencing

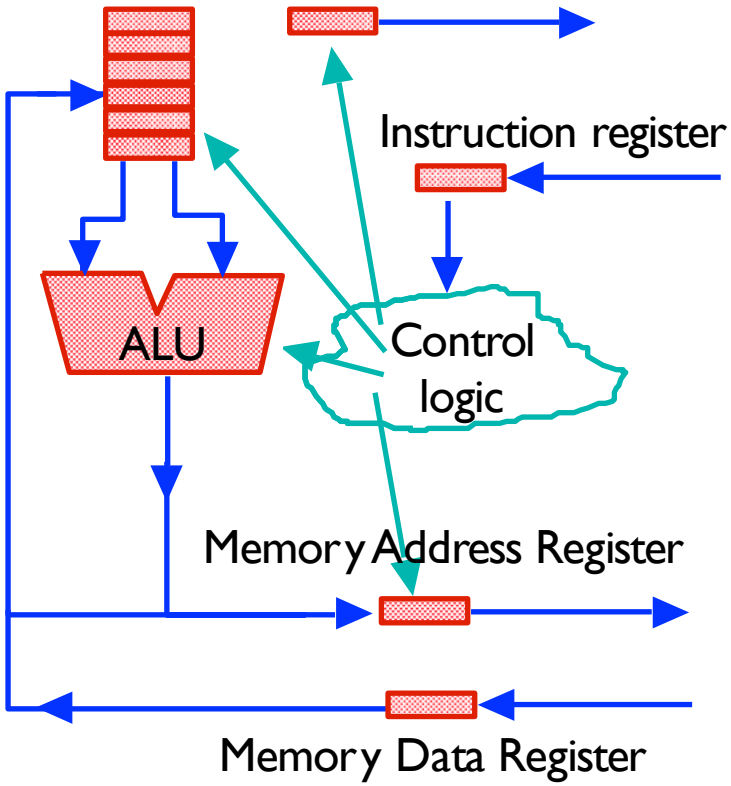
Execuția unei instrucțiuni

Embedded Systems Laboratory

Secvențiatorul (FSM) face fetch la instrucțiune din memorie și o scrie în Instruction Register

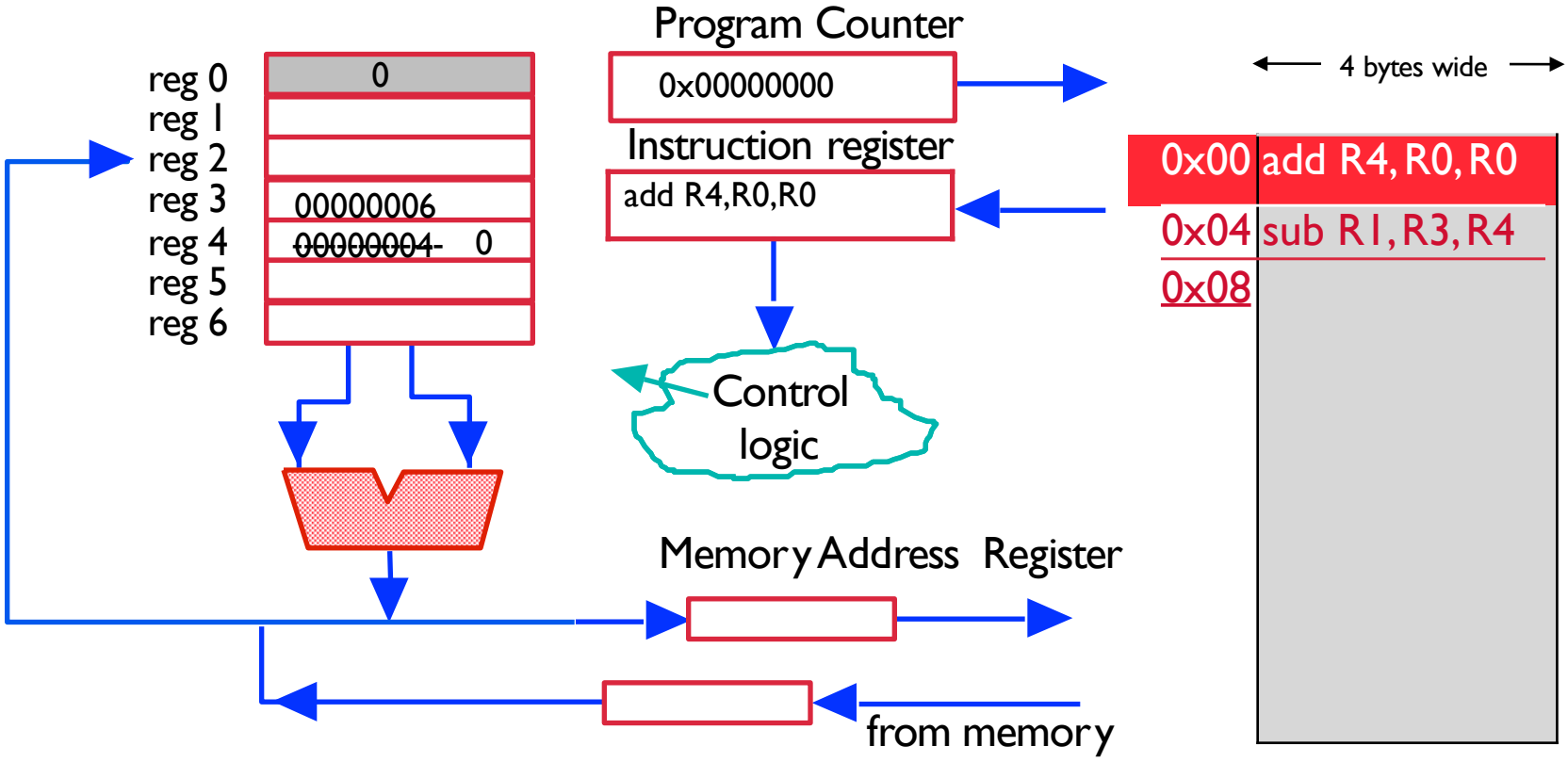
Logica de control decodifică instrucțiunea și emite comenzi către tabela de registre, ALU și celelalte registre

Dacă o operație ALU (e.g. add), datele merg din tabela de registre prin ALU și înapoi în tabela de registre

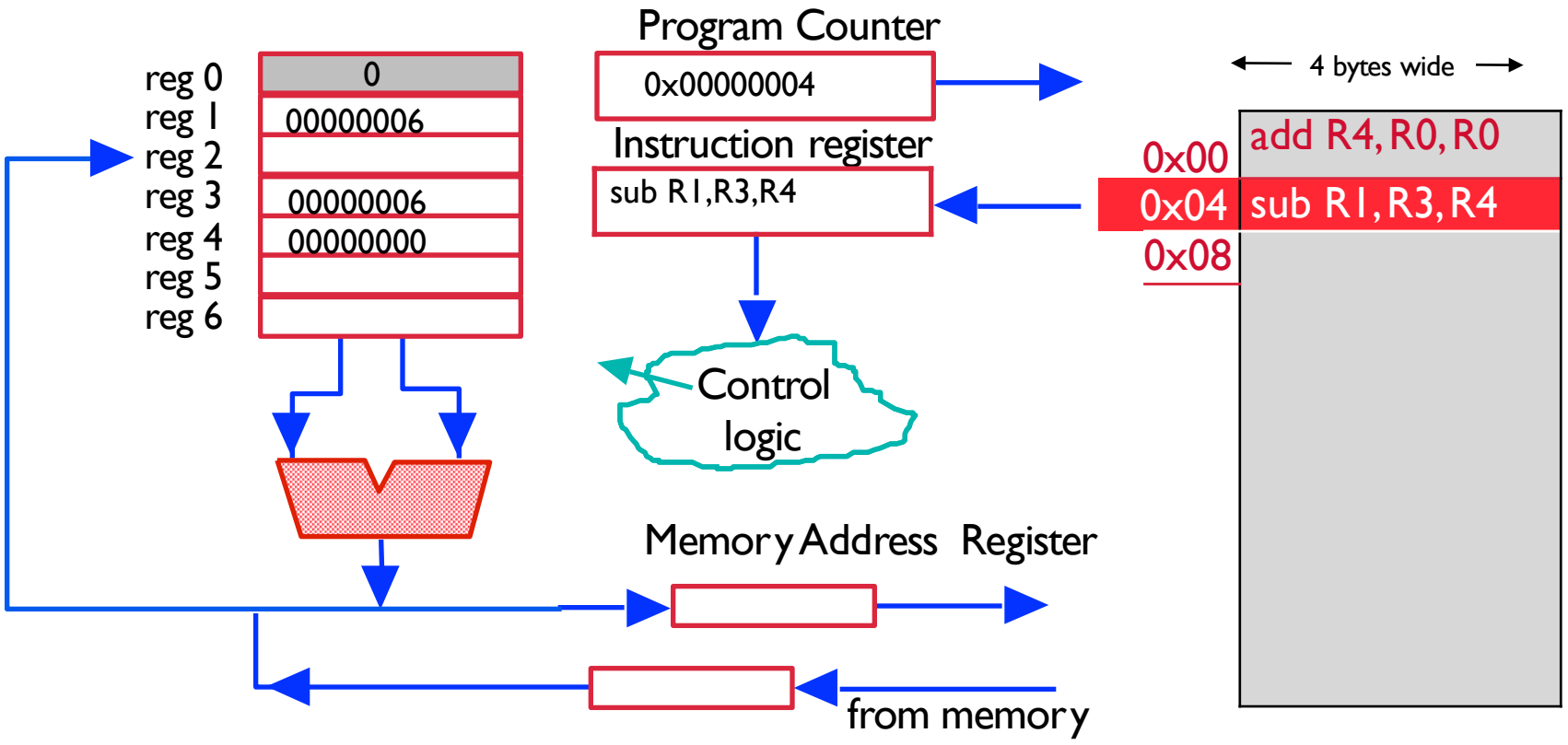


0x00	00101101
0x01	00100001
0x02	00110000
0x03	00001111
0x04	11010101
0x05	01010101
0x06	00101010
0x07	01010101
0x08	11110011
0x09	00111100
0x0A	00001100
0x0B	00000000
0x0C	00011000
0x0D	11111111

Execuție din tabela de registre



Execuție din tabela de registre

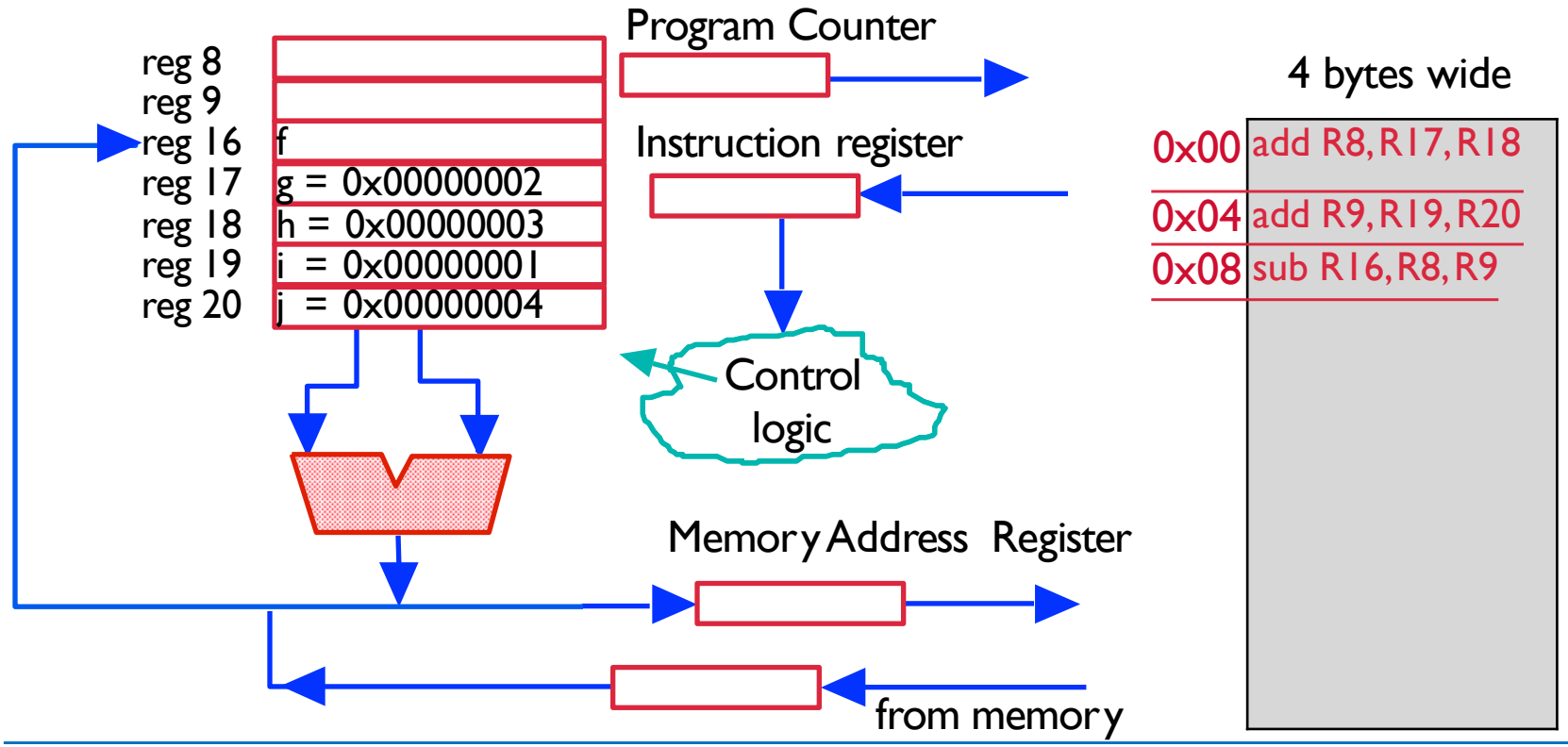


Alt exemplu

Embedded Systems Laboratory

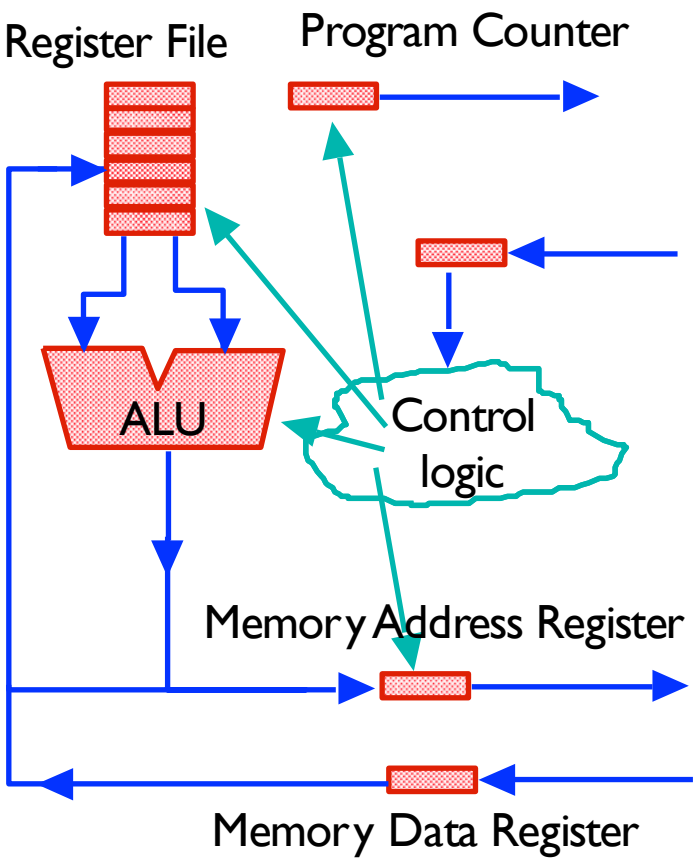
$$f = (g + h) - (i + j)$$

R16 == f, R17 == g, R18 == h, R19 == i, R20 == j



Accesarea datelor

- Adresa generată de ALU
- Adresa ajunge în *Memory Address Register*
- După accesul memoriei, rezultatul este întors în *Memory Data Register*
- Adresele pot fi pentru date sau pentru instrucțiuni, ambele sunt stocate în memorie



0x00	00101101
0x01	00100001
0x02	00110000
0x03	00001111
0x04	11010101
0x05	01010101
0x06	00101010
0x07	01010101
0x08	11110011
0x09	00111100
0x0A	00001100
0x0B	00000000
0x0C	00011000
0x0D	11111111

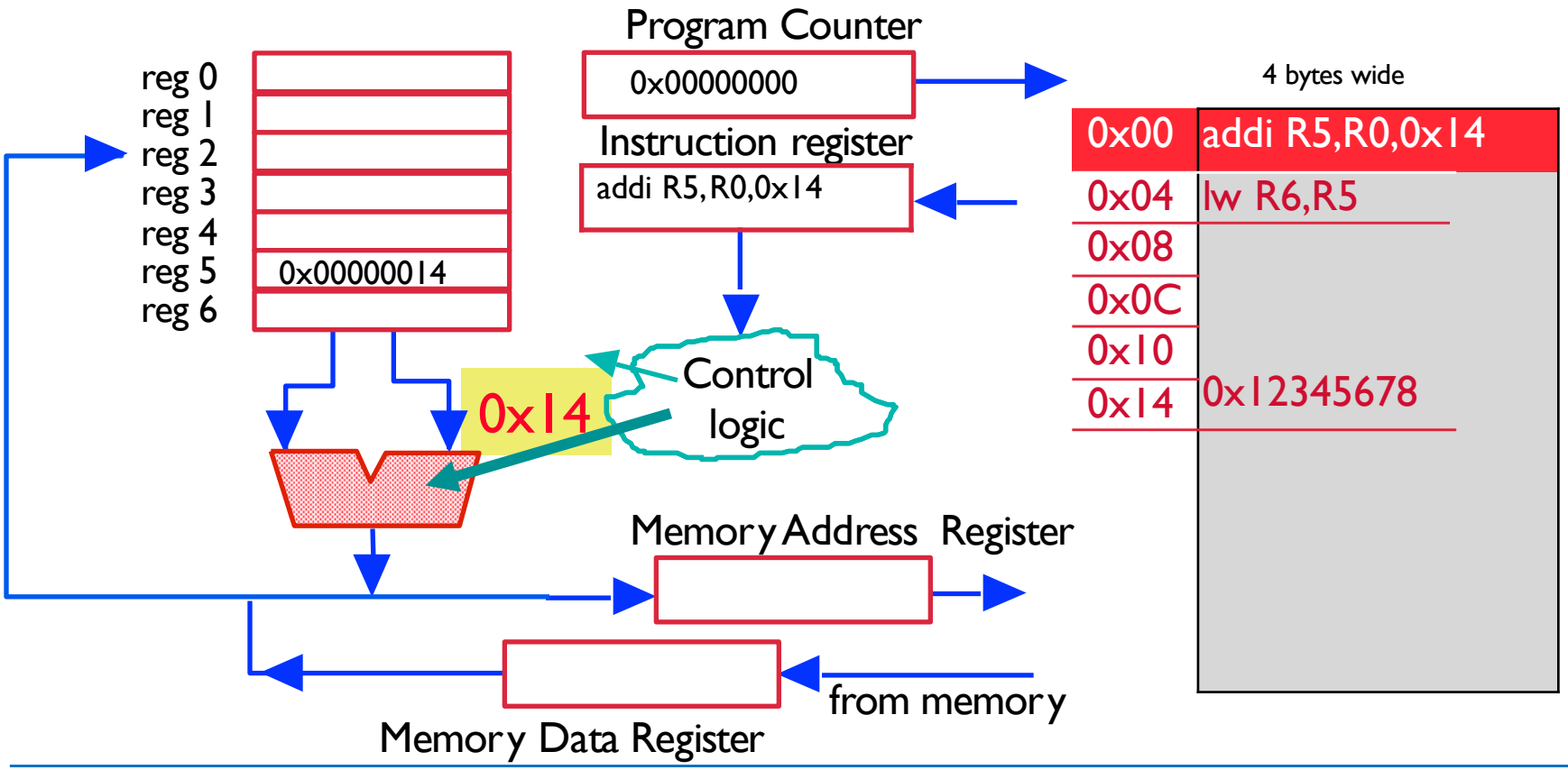
Operații cu memoria - Load

Embedded Systems Laboratory

R6 ⇇mem[0x14]

Presupunem &A = 0x14

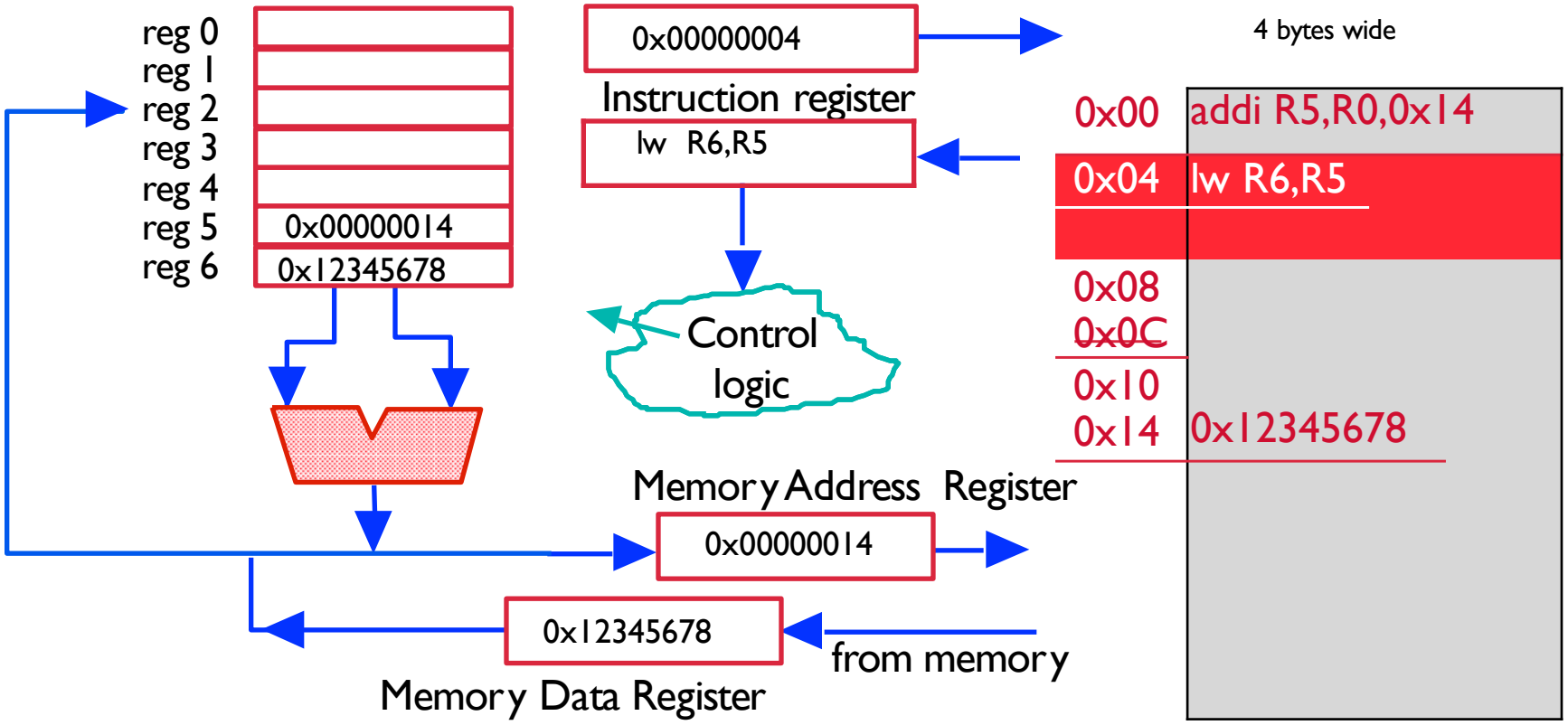
addi: Adună constanta pe 16 biți la registrul sursă



Operații cu memoria - Load

R6 ⇐mem[0x14]

Presupunem &A = 0x14



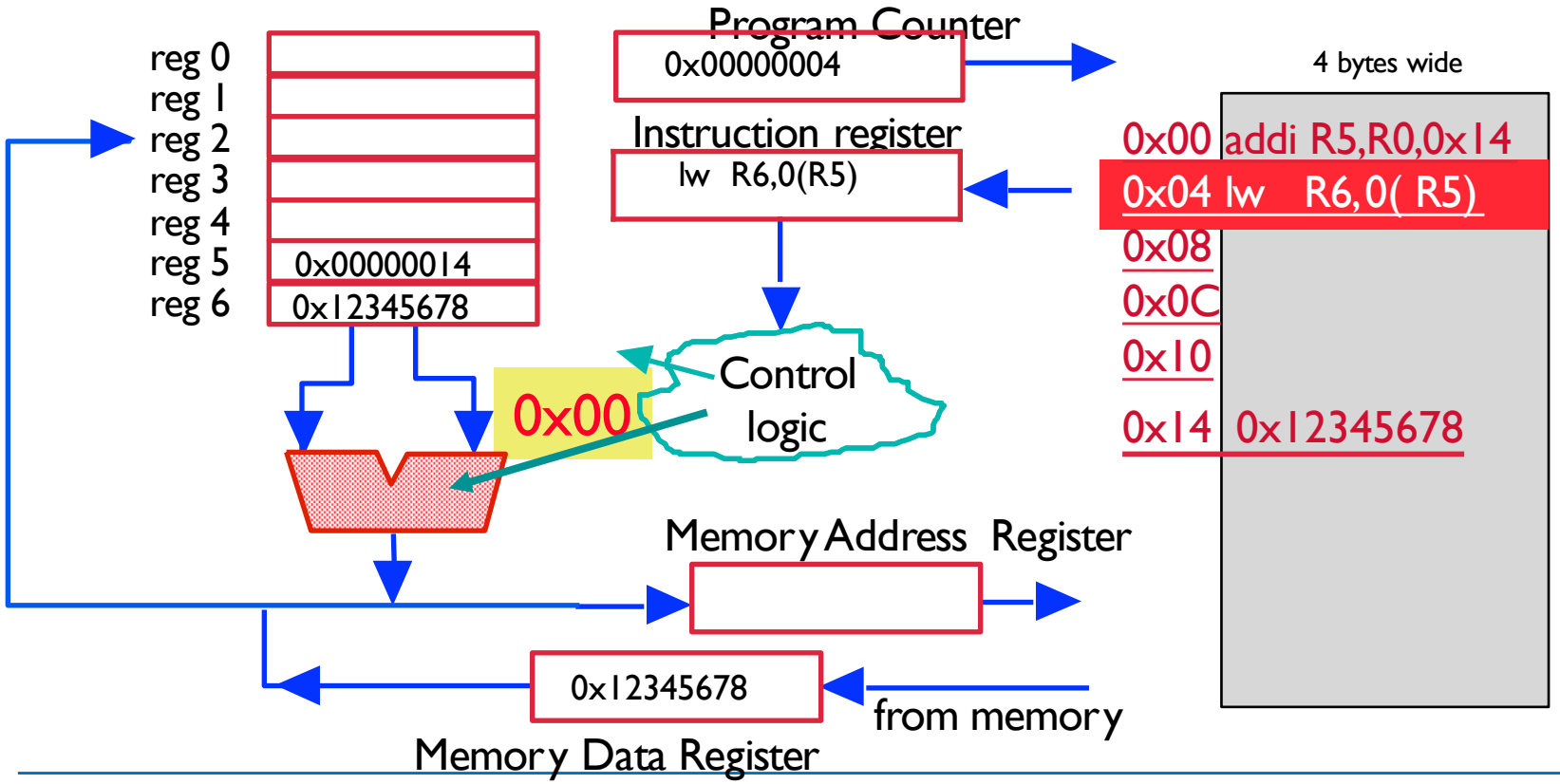
Operații cu memoria - Load

Embedded Systems Laboratory

Adresa poate fi calculată și prin adăugarea unui offset registrului

LW R6, 0(R5)

R6 ⇐ memory[0 + R5]



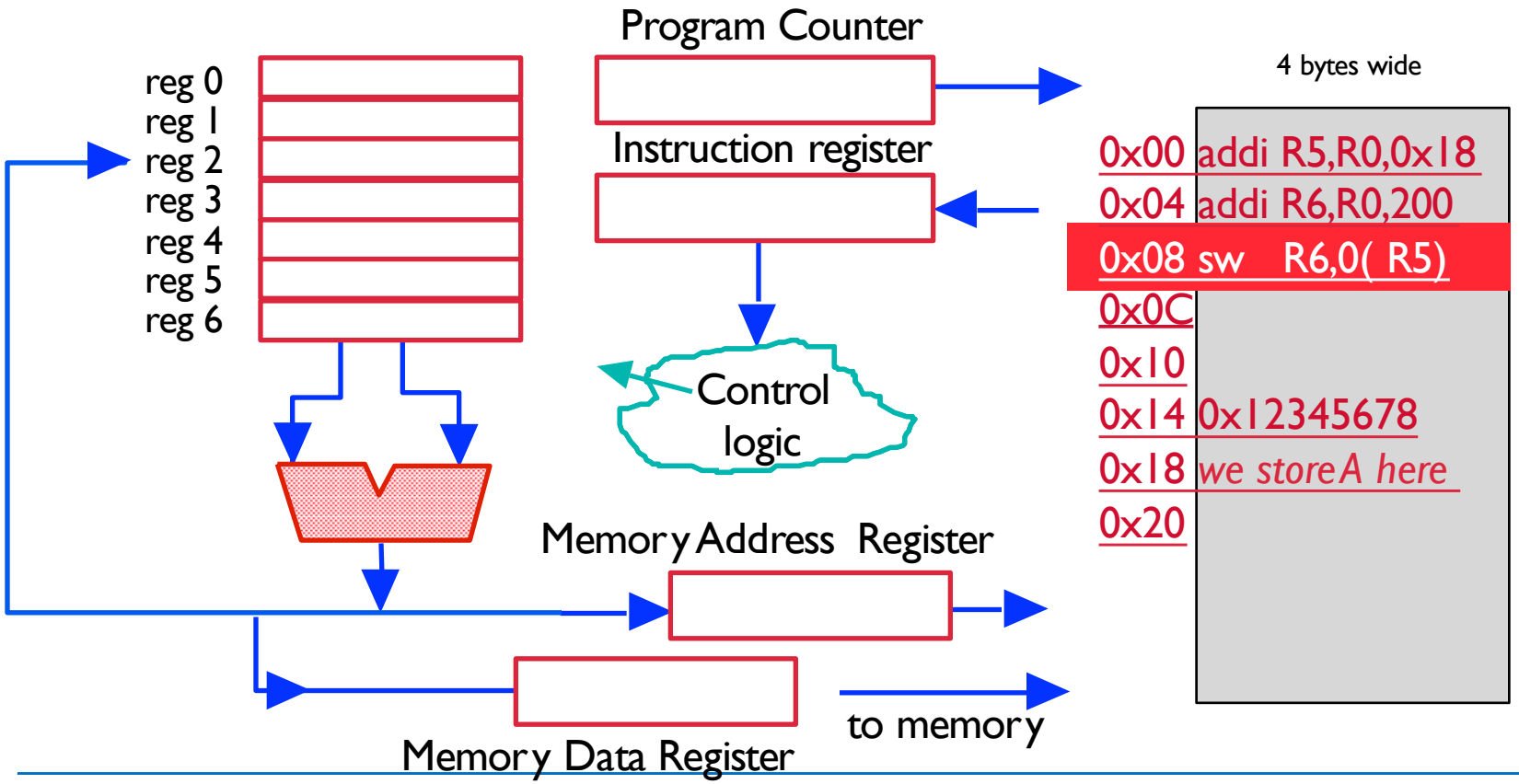
Operații cu memoria - Store

Embedded Systems Laboratory

Stocarea datelor în memorie se face la fel

A = 200; presupunem &A = 0x18

mem[0x18] ⇐200



Formatul instrucțiunilor (cod mașină)

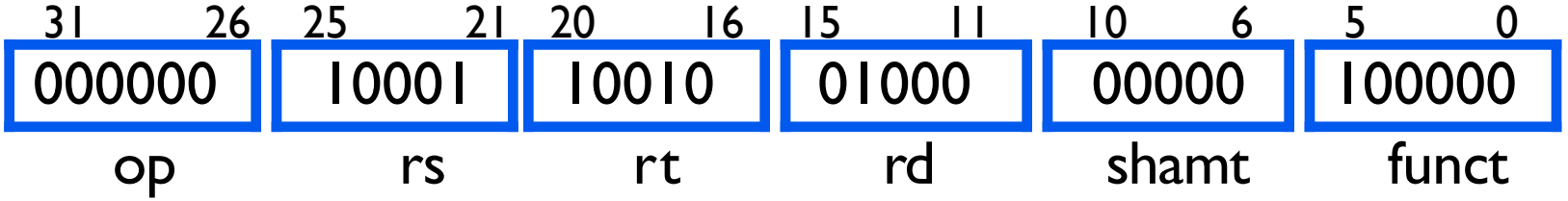
Embedded Systems Laboratory

add R8, R17, R18

Stocat în format binar ca

00000010 00110010 01000000 00100000

Instrucțiunea este împărțită în "câmpuri"



op	Codul operației
rs	Adrea registru sursă 1
rt	Adresa registru sursă 2
rd	Adresa registrul destinație
shamt	shift amount
funct	Funcție (selectează tipul operației)

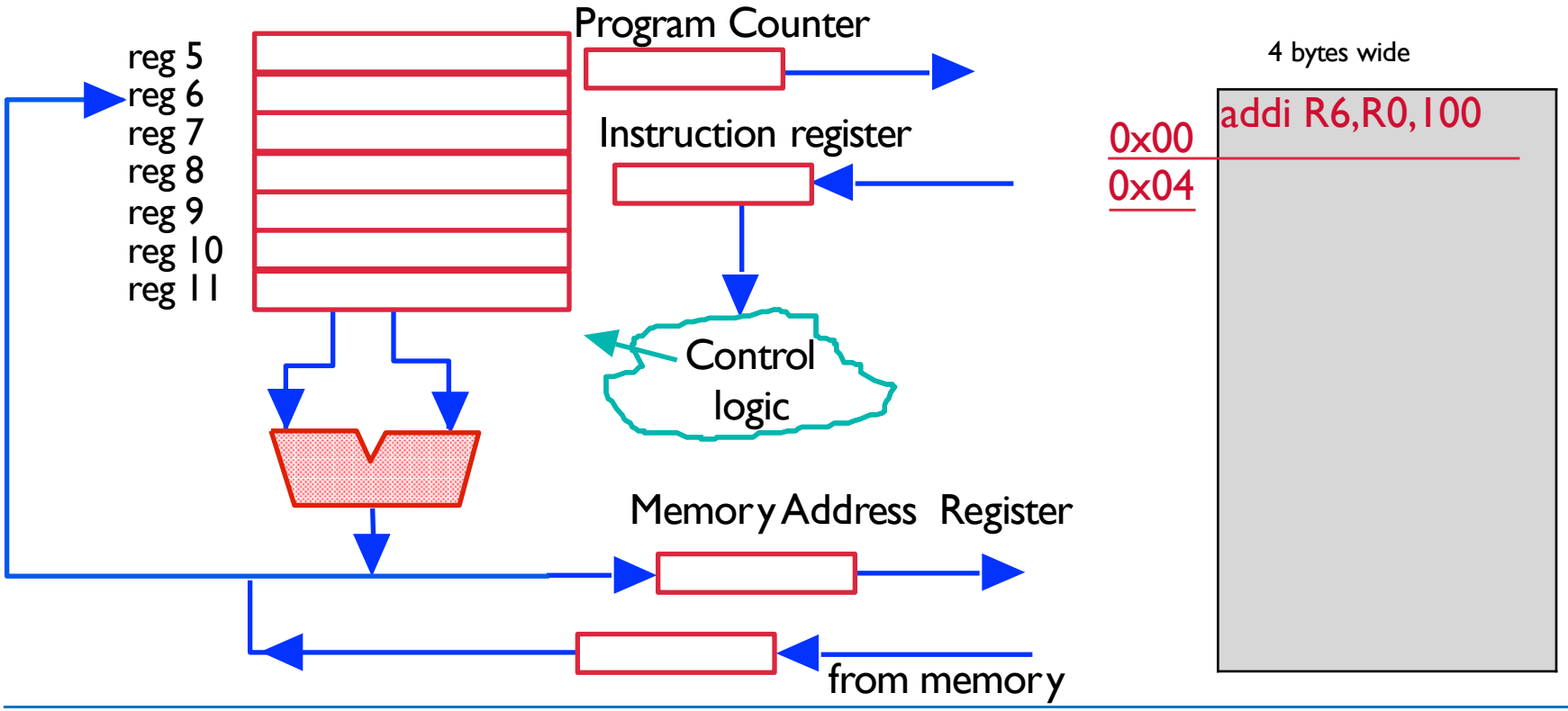
add = 32_{10}

sub = 34_{10}

Încărcarea valorilor imediate

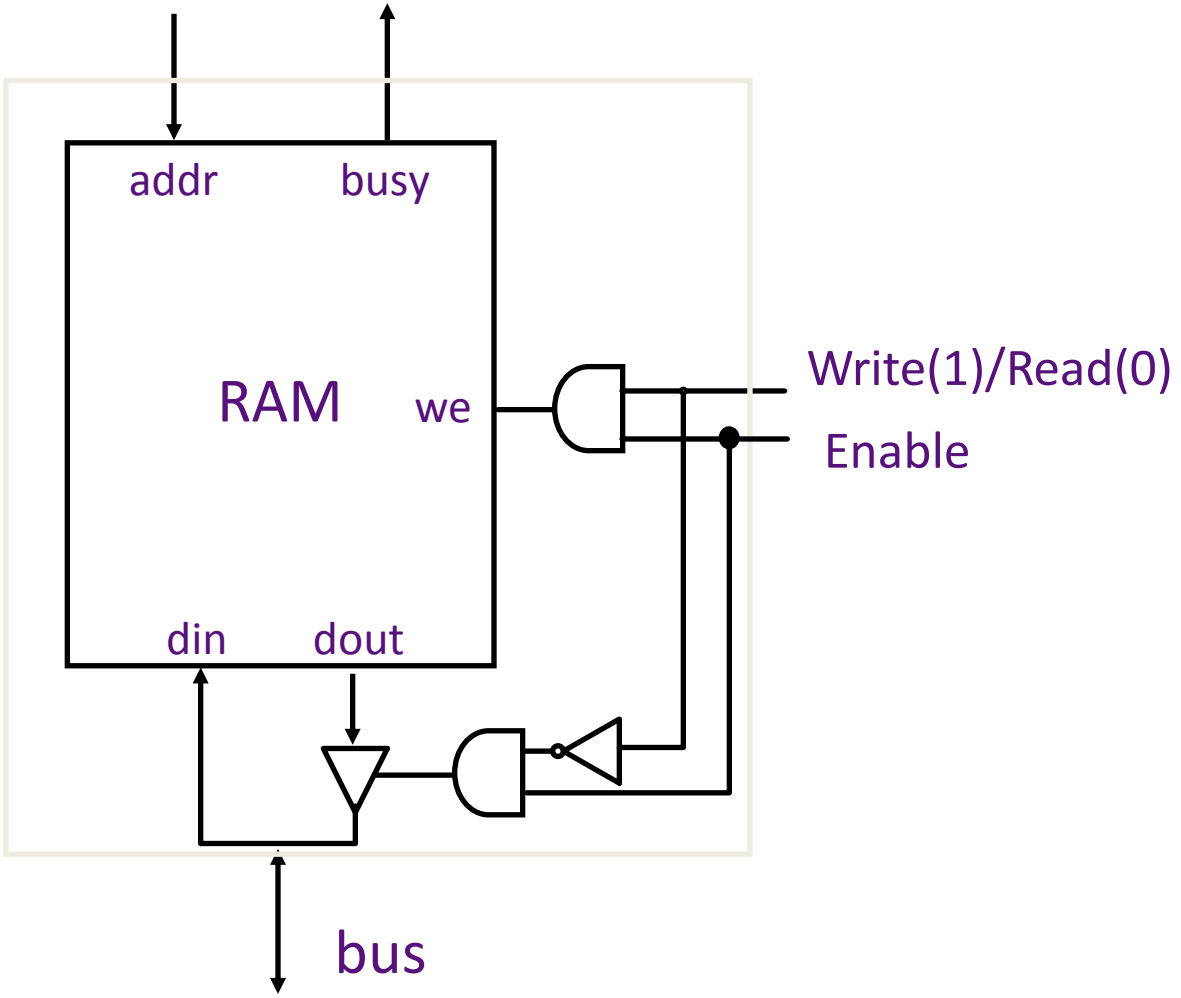
Cum punem o constantă (valoare imediată) în registru?

Scriem valoarea 100 în R6: R6 ⇐ R0 + 100 = 0+100 = 100



Modulul de memorie

Presupunere: memoria operează independent și e mult mai lentă decât operațiile de transfer registru-registru (mai mulți cicli de ceas pentru a accesa datele)



Execuția instrucțiunilor

Execuția unei instrucțiuni RISC-V presupune următorii pași:

1. instruction fetch
2. decode & register fetch
3. operații în UAL
4. operații cu memoria (opțional)
5. write back în tabela de registre generale (opțional)

+ calculul adresei următoarei instrucțiuni

Fragmente de microprogram

Embedded Systems Laboratory

instr fetch:
MA, A <- PC
PC <- A + 4
IR <- Memory
dispatch on Opcode

Poate fi văzută ca
O macroinstrucțiune

ALU:
A <- Reg[rs1]
B <- Reg[rs2]
Reg[rd] <- func(A,B)
do instruction fetch

ALUi:
A <- Reg[rs1]
B <- Imm
Reg[rd] <- Opcode(A,B)
do instruction fetch

sign extension

Fragmente de Microprogram
(cont.)

Embedded Systems Laboratory

LW:
A <- Reg[rs1]
B <-Imm
MA <- A + B
Reg[rd] <- Memory
do instruction fetch

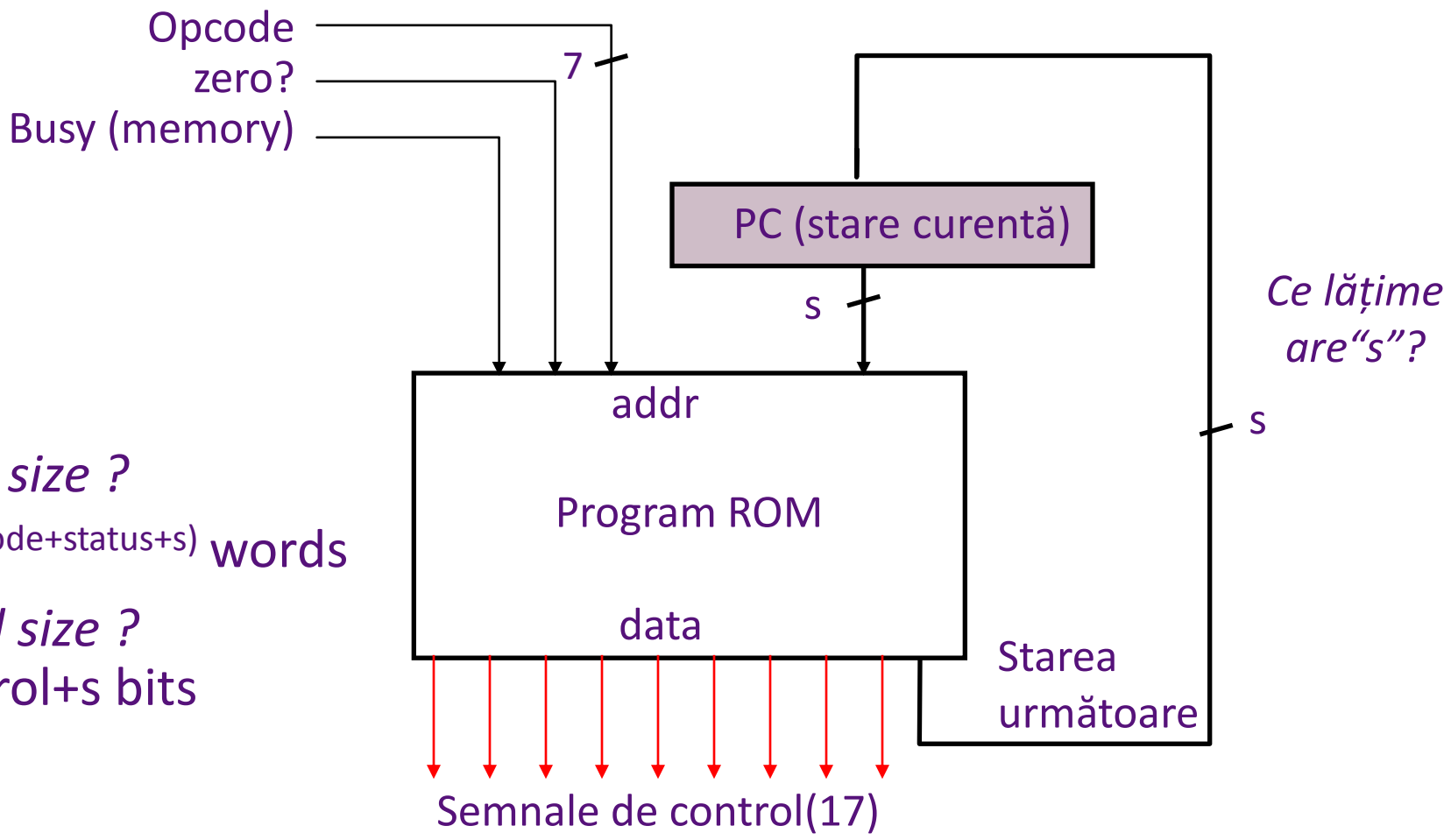
J:
A <- A - 4 Get original PC back in A
B <- IR
PC <- JumpTarg(A,B)
do instruction fetch

JumpTarg(A,B) = {A + (B[31:7]<<1)}

beq:
A <- Reg[rs1]
B <- Reg[rs2]
If A==B *then go to* bz-taken
do instruction fetch

bz-taken:
A <- PC
A <- A - 4 Get original PC back in A
B <- BImm << 1 BImm = IR[31:27,16:10]
PC <- A + B
do instruction fetch

RISC-V Microcontroller: *prima încercare:implementare numai în ROM*



ROM size ?

= $2^{(opcode+status+s)}$ words

Word size ?

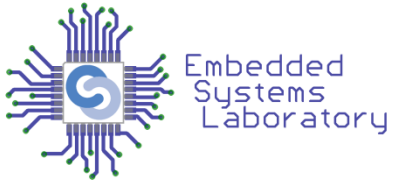
= control+s bits

Microprogramul din ROM

Embedded Systems Laboratory

State	Op	zero?	busy	Control points	next-state
$fetch_0$	*	*	*	MA,A <- PC	$fetch_1$
$fetch_1$	*	*	yes	....	$fetch_1$
$fetch_1$	*	*	no	IR <- Memory	$fetch_2$
$fetch_2$	*	*	*	PC <- A + 4	?
$fetch_2$	ALU	*	*	PC <- A + 4	ALU_0
ALU_0	*	*	*	A <- Reg[rs1]	ALU_1
ALU_1	*	*	*	B <- Reg[rs2]	ALU_2
ALU_2	*	*	*	Reg[rd] <- func(A,B)	$fetch_0$

Microprogramul din ROM



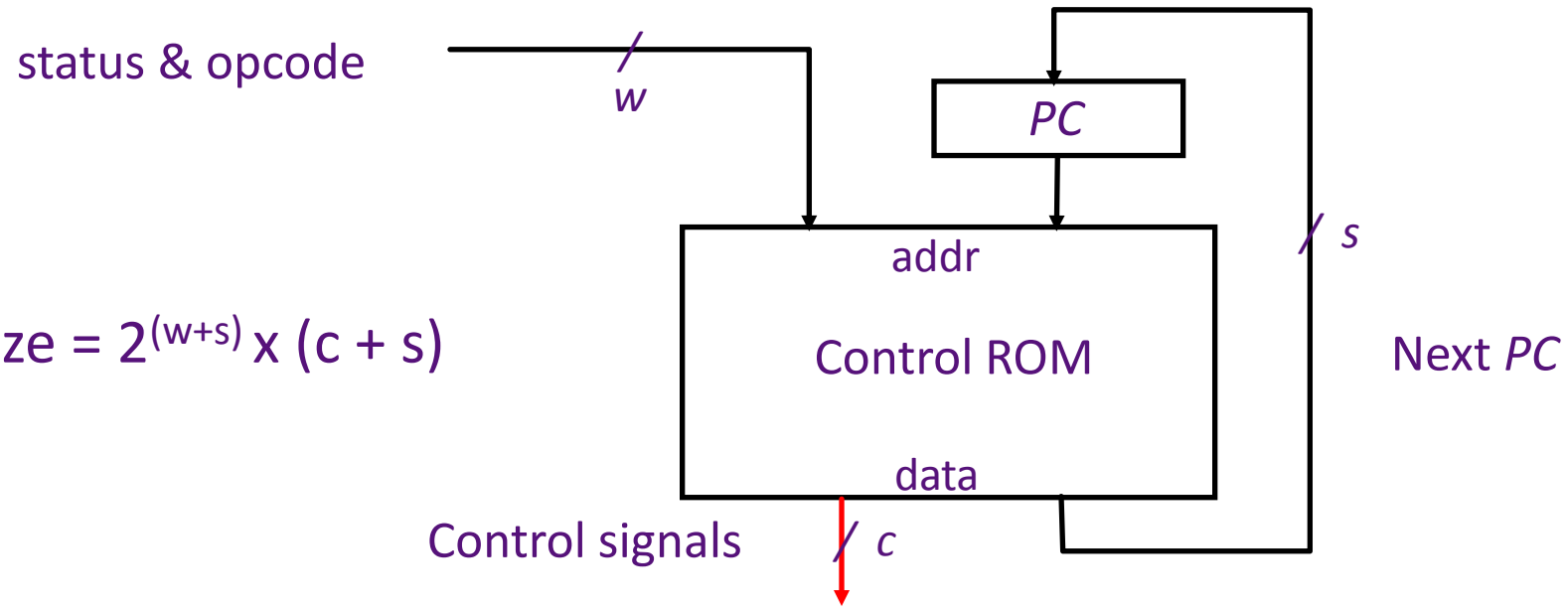
State	Op	zero?	busy	Control points	next-state
$fetch_0$	*	*	*	MA,A <- PC	$fetch_1$
$fetch_1$	*	*	yes	....	$fetch_1$
$fetch_1$	*	*	no	IR ? Memory	$fetch_2$
$fetch_2$	ALU	*	*	PC <- A + 4	ALU_0
$fetch_2$	ALUi	*	*	PC <- A + 4	$ALUi_0$
$fetch_2$	LW	*	*	PC <- A + 4	LW_0
$fetch_2$	SW	*	*	PC <- A + 4	SW_0
$fetch_2$	J	*	*	PC <- A + 4	J_0
$fetch_2$	JAL	*	*	PC <- A + 4	JAL_0
$fetch_2$	JR	*	*	PC <- A + 4	JR_0
$fetch_2$	JALR	*	*	PC <- A + 4	$JALR_0$
$fetch_2$	beq	*	*	PC <- A + 4	beq_0
...					
ALU_0	*	*	*	A <- Reg[rs1]	ALU_1
ALU_1	*	*	*	B <- Reg[rs2]	ALU_2
ALU_2	*	*	*	Reg[rd] <- func(A,B)	$fetch_0$

Microprogramul din ROM *Cont.*

Embedded Systems Laboratory

State	Op	zero?	busy	Control points	next-state
$ALUi_0$	*	*	*	A <- Reg[rs1]	$ALUi_1$
$ALUi_1$	*	*	*	B <- Imm	$ALUi_2$
$ALUi_2$	*	*	*	Reg[rd]<- Op(A,B)$fetch_0$	
...					
J_0	*	*	*	A <- A - 4	J_1
J_1	*	*	*	B <- IR	J_2
J_2	*	*	*	PC <- JumpTarg(A,B)	$fetch_0$
...					
beq_0	*	*	*	A <- Reg[rs1]	beq_1
beq_1	*	*	*	B <- Reg[rs2]	beq_2
beq_2	*	yes	*	A <- PC	beq_3
beq_2	*	no	*	....	$fetch_0$
beq_3	*	*	*	A <- A - 4	beq_4
beq_4	*	*	*	B <- BImm	beq_5
beq_5	*	*	*	PC <- A+B	$fetch_0$

Dimensiunea memoriei ROM



size = $2^{(w+s)}$ x (c + s)

RISC-V: w = 5+2 c = 17 s = ?

no. of steps per opcode = 4 to 6 + fetch-sequence

no. of states -> (4 steps per op-group) x op-groups + secv. comune

= 4 x 8 + 10 states = 42 states => s = 6

Control ROM = $2^{(5+6)}$ x 23 bits ~ 24 Kbytes

Reducerea dimensiunii memoriei de control

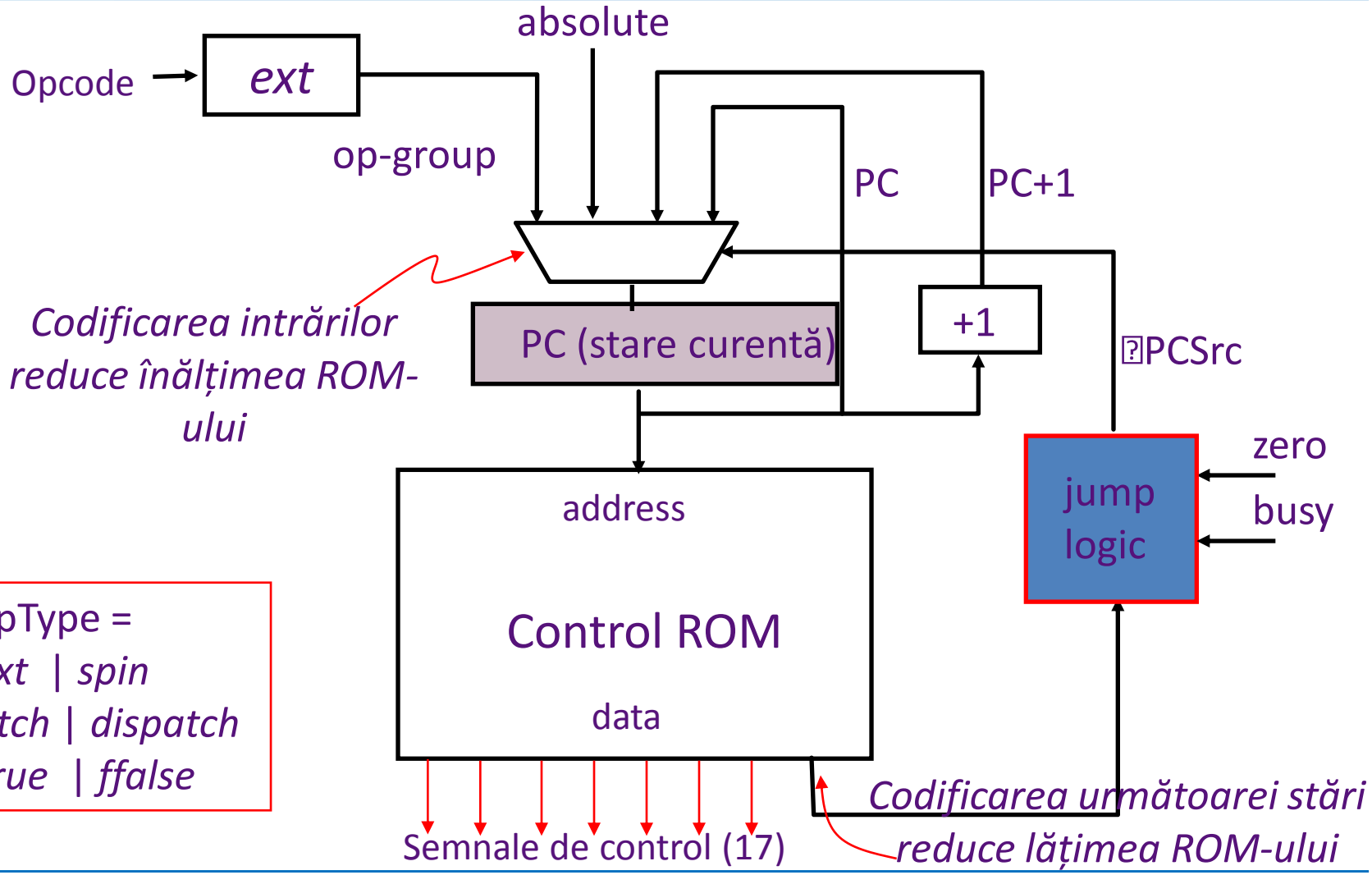
Embedded Systems Laboratory

Memoria de control trebuie să fie *rapidă => scumpă*

- Reducerea înălțimii ROM-ului (= biți de adresă)
  - *reducerea numărului de intrări prin adăugarea de logică externă*
    Fiecare bit de intrare dublează dimensiunea memoriei de control
  - *reducerea numărului de stăriprin gruparea codurilor de operații*
    găsește secvențe comune de acțiuni
  - *comprimă biții de stare pentru intrări*
    combină toate excepțiile într-una singură, i.e., exception/no-exception

- Reducerea lățimii ROM-ului
  - *restricționează codificarea următorii stări*
    Next, Dispatch on opcode, Wait for memory, ...
  - *codifică semnalele de control (microcod verticalizat)*

RISC-V Controller V2



Stările pentru Jump

PCSrc = *Case* JumpTypes

next	->	PC+1
spin	->	if (busy) then PC else PC+1
fetch	->	absolute
dispatch	->	op-group
ftrue	->	if (zero) then absolute else PC+1
ffalse	->	if (zero) then PC+1 else absolute

Instruction Fetch & ALU: *RISC-V-Controller-2*

Embedded Systems Laboratory

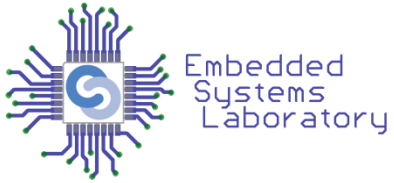
State	Control points	next-state
$fetch_0$	MA,A <- PC	
$fetch_1$	IR <- Memory	
$fetch_2$	PC <- A + 4	
...		
ALU_0	A <- Reg[rs1]	
ALU_1	B <- Reg[rs2]	
ALU_2	Reg[rd]<-func(A,B)	
$ALUi_0$	A <- Reg[rs1]	
$ALUi_1$	B <- Imm	
$ALUi_2$	Reg[rd]<- Op(A,B)	

Load & Store: *RISC-V-Controller-2*

Embedded Systems Laboratory

State	Control points	next-state
LW_0	A <- Reg[rs1]	next
LW_1	B <- Imm	next
LW_2	MA <- A+B	next
LW_3	Reg[rd] <- Memory	spin
LW_4		fetch
SW_0	A <- Reg[rs1]	next
SW_1	B <- BImm	next
SW_2	MA <- A+B	next
SW_3	Memory <- Reg[rs2]	spin
SW_4		fetch

Branches: *RISC-V-Controller-2*



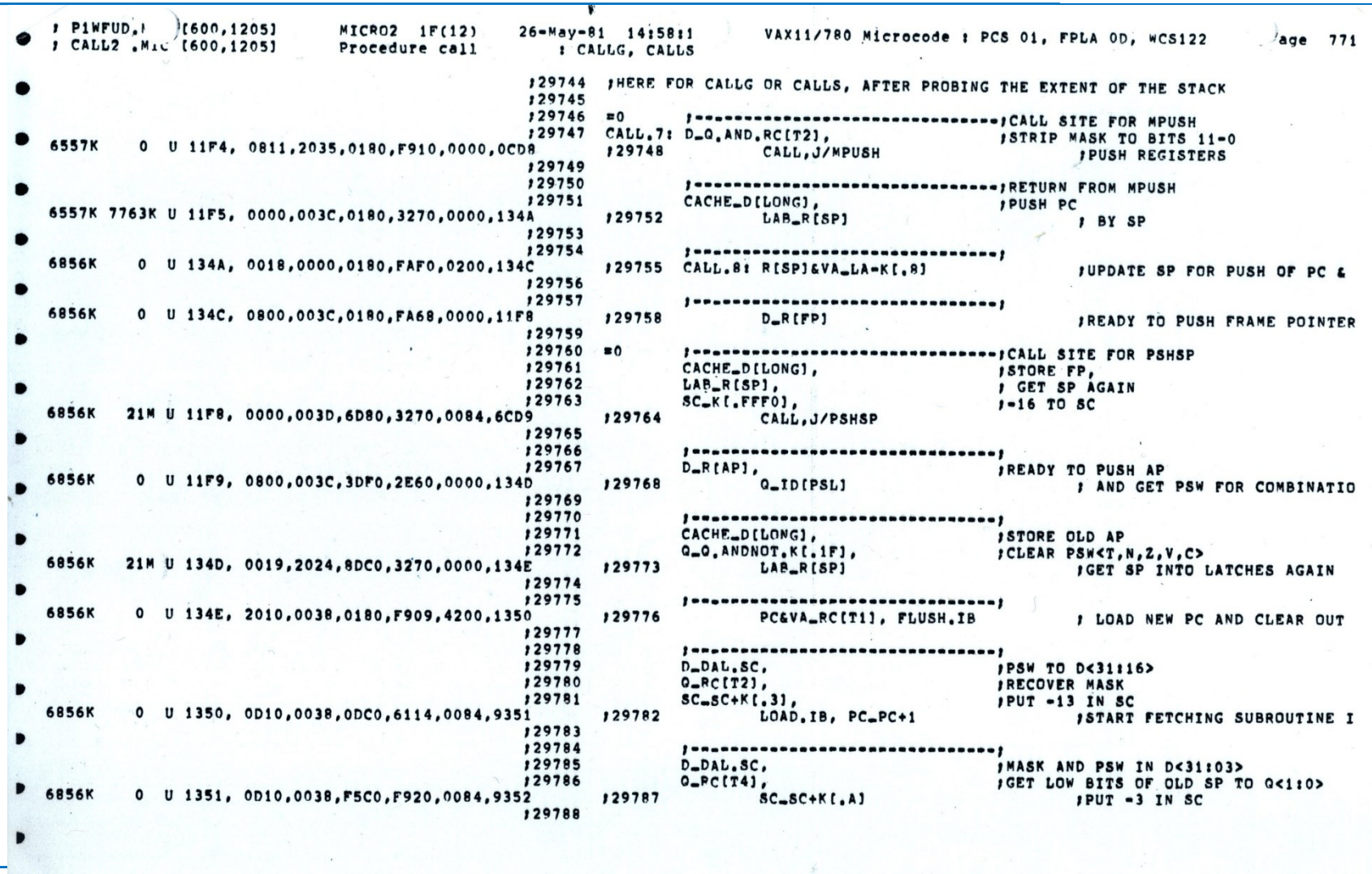
State	Control points	next-state
beq_0	A <- Reg[rs1]	next
beq_1	B <- Reg[rs2]	next
beq_2	A <- PC	ffalse
beq_3	A <- A- 4	next
beq_3	B <- BImm<<1	next
beq_4	PC <- A+B	fetch

Jumps: *RISC-V-Controller-2*

Embedded Systems Laboratory

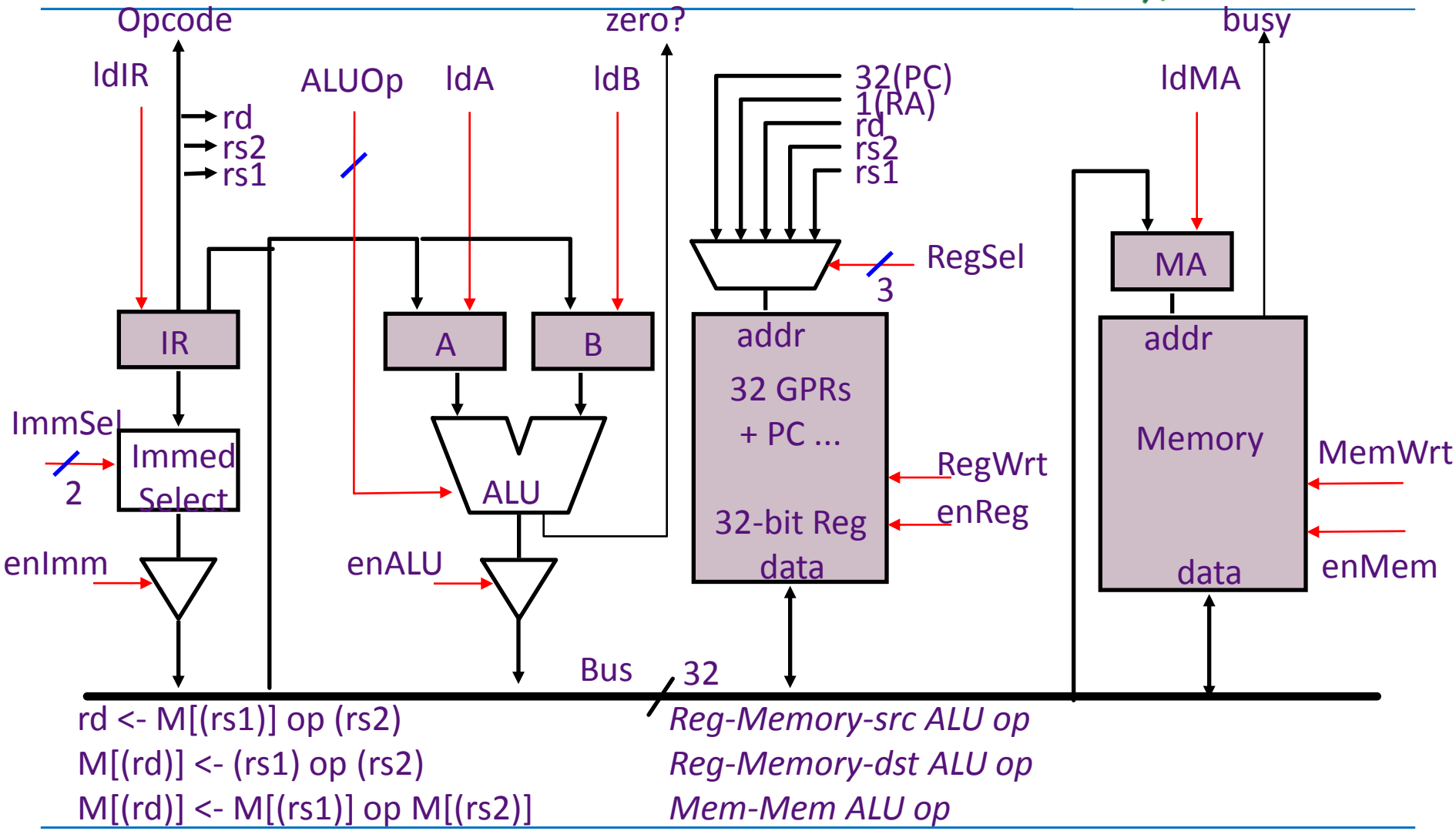
State	Control points	next-state
J_0	A <- A-4	next
J_1	B <- IR	next
J_2	PC <- JumpTarg(A,B)	fetch
JR_0	A <- Reg[rs1]	next
JR_1	PC <- A	fetch
JAL_0	A <- PC	next
JAL_1	Reg[1] <- A	next
JAL_2	A <- A-4	next
JAL_3	B <- IR	next
JAL_4	PC <- JumpTarg(A,B)	fetch

VAX 11-780 Microcode



```
; P1WFUD.!      [600,1205]      MICRO2  1F(12)     26-May-81  14:58:1      VAX11/780 Microcode ! PCS 01, FPLA 0D, WCS122      Page  771
; CALL2 .MIC    [600,1205]      Procedure call        ! CALLG, CALLS

                                                ;29744  ;HERE FOR CALLG OR CALLS, AFTER PROBING THE EXTENT OF THE STACK
                                                ;29745
                                                ;29746  =0      ;-------------------------------;CALL SITE FOR MPUSH
                                                ;29747  CALL.7: D_Q,AND,RC[T2],                 ;STRIP MASK TO BITS 11-0
6557K     0  U 11F4, 0811,2035,0180,F910,0000,0CD8  ;29748          CALL,J/MPUSH                  ;PUSH REGISTERS
                                                ;29749
                                                ;29750          ;-------------------------------;RETURN FROM MPUSH
                                                ;29751          CACHE_D[LONG],                ;PUSH PC
6557K  7763K U 11F5, 0000,003C,0180,3270,0000,134A  ;29752          LAB_R[SP]                     ; BY SP
                                                ;29753
                                                ;29754          ;-------------------------------;
6856K     0  U 134A, 0018,0000,0180,FAF0,0200,134C  ;29755  CALL.8: R[SP]&VA_LA-K[,8]             ;UPDATE SP FOR PUSH OF PC &
                                                ;29756
                                                ;29757          ;-------------------------------;
6856K     0  U 134C, 0800,003C,0180,FA68,0000,11F8  ;29758          D_R[FP]                       ;READY TO PUSH FRAME POINTER
                                                ;29759
                                                ;29760  =0      ;-------------------------------;CALL SITE FOR PSHSP
                                                ;29761          CACHE_D[LONG],                ;STORE FP,
                                                ;29762          LAB_R[SP],                    ; GET SP AGAIN
                                                ;29763          SC_K[,FFF0],                  ;-16 TO SC
6856K    21M U 11F8, 0000,003D,6D80,3270,0084,6CD9  ;29764          CALL,J/PSHSP
                                                ;29765
                                                ;29766          ;-------------------------------;
                                                ;29767          D_R[AP],                      ;READY TO PUSH AP
6856K     0  U 11F9, 0800,003C,3DF0,2E60,0000,134D  ;29768          Q_ID[PSL]                     ; AND GET PSW FOR COMBINATIO
                                                ;29769
                                                ;29770          ;-------------------------------;
                                                ;29771          CACHE_D[LONG],                ;STORE OLD AP
                                                ;29772          Q_Q,ANDNOT,K[,1F],            ;CLEAR PSW<T,N,Z,V,C>
6856K    21M U 134D, 0019,2024,8DC0,3270,0000,134E  ;29773          LAB_R[SP]                     ;GET SP INTO LATCHES AGAIN
                                                ;29774
                                                ;29775          ;-------------------------------;
6856K     0  U 134E, 2010,0038,0180,F909,4200,1350  ;29776          PC&VA_RC[T1], FLUSH,IB        ; LOAD NEW PC AND CLEAR OUT
                                                ;29777
                                                ;29778          ;-------------------------------;
                                                ;29779          D_DAL,SC,                     ;PSW TO D<31:16>
                                                ;29780          Q_RC[T2],                     ;RECOVER MASK
                                                ;29781          SC_SC+K[,3],                  ;PUT -13 IN SC
6856K     0  U 1350, 0D10,0038,0DC0,6114,0084,9351  ;29782          LOAD,IB, PC_PC+1              ;START FETCHING SUBROUTINE I
                                                ;29783
                                                ;29784          ;-------------------------------;
                                                ;29785          D_DAL,SC,                     ;MASK AND PSW IN D<31:03>
                                                ;29786          Q_RC[T4],                     ;GET LOW BITS OF OLD SP TO Q<1:0>
6856K     0  U 1351, 0D10,0038,F5C0,F920,0084,9352  ;29787          SC_SC+K[,A]                   ;PUT -3 IN SC
                                                ;29788
```

C


Opcode
zero?
busy
ldIR
ALUOp
ldA
ldB
32(PC)
1(RA)
rd
rs2
rs1
ldMA
rd
rs2
rs1
RegSel
3
MA
IR
A
B
addr
32 GPRs
+ PC ...
32-bit Reg
data
addr
Memory
data
ImmSel
2
Immed
Select
ALU
RegWrt
enReg
MemWrt
enMem
enImm
enALU
Bus
32
rd <- M[(rs1)] op (rs2)
M[(rd)] <- (rs1) op (rs2)
M[(rd)] <- M[(rs1)] op M[(rs2)]
Reg-Memory-src ALU op
Reg-Memory-dst ALU op
Mem-Mem ALU op

Instrucțiuni Mem-Mem ALU:

RISC-V-Controller-2

Embedded Systems Laboratory

Mem-Mem ALU op	M[(rd)] <- M[(rs1)] op M[(rs2)]	
$ALUMM_0$	MA <- Reg[rs1]	next
$ALUMM_1$	A <- Memory	spin
$ALUMM_2$	MA <- Reg[rs2]	next
$ALUMM_3$	B <- Memory	spin
$ALUMM_4$	MA <- Reg[rd]	next
$ALUMM_5$	Memory <- func(A,B)	spin
$ALUMM_6$		fetch

Instrucțiunile complexe nu necesită de obicei modificări ale structurii unității de control microprogramat

-- doar spațiu suplimentar în memoria ROM de control

Implementarea acestor instrucțiuni folosind un controller hardware dedicat nu poate fi făcută fără modificarea căilor de date din interiorul unității de control.

Considerente de performanță

Unitatea de control microprogramat

-> cicli multipli per instrucțiune

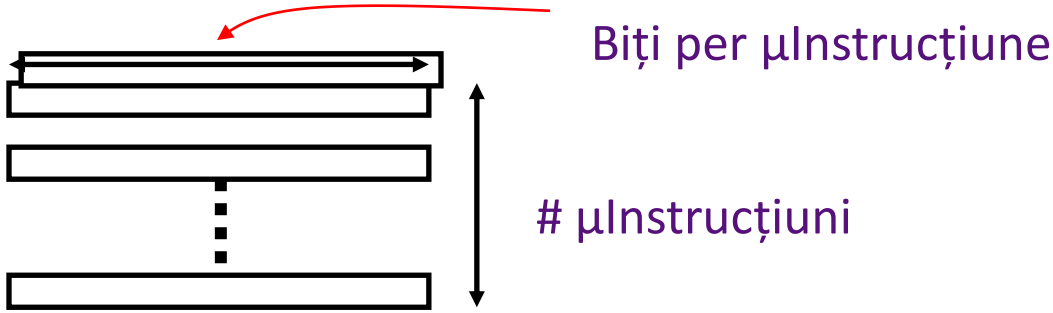
Durata ciclului ?

$t_C > \max(t_{reg\text{-}reg}, t_{ALU}, t_{ROM})$

Presupunem că $10 * t_{ROM} < t_{RAM}$

Performanță mărită, atunci când comparăm cu o implementare pur hardware a UC care execută toate operațiile într-un singur ciclu.

µCod orizontal vs. vertical

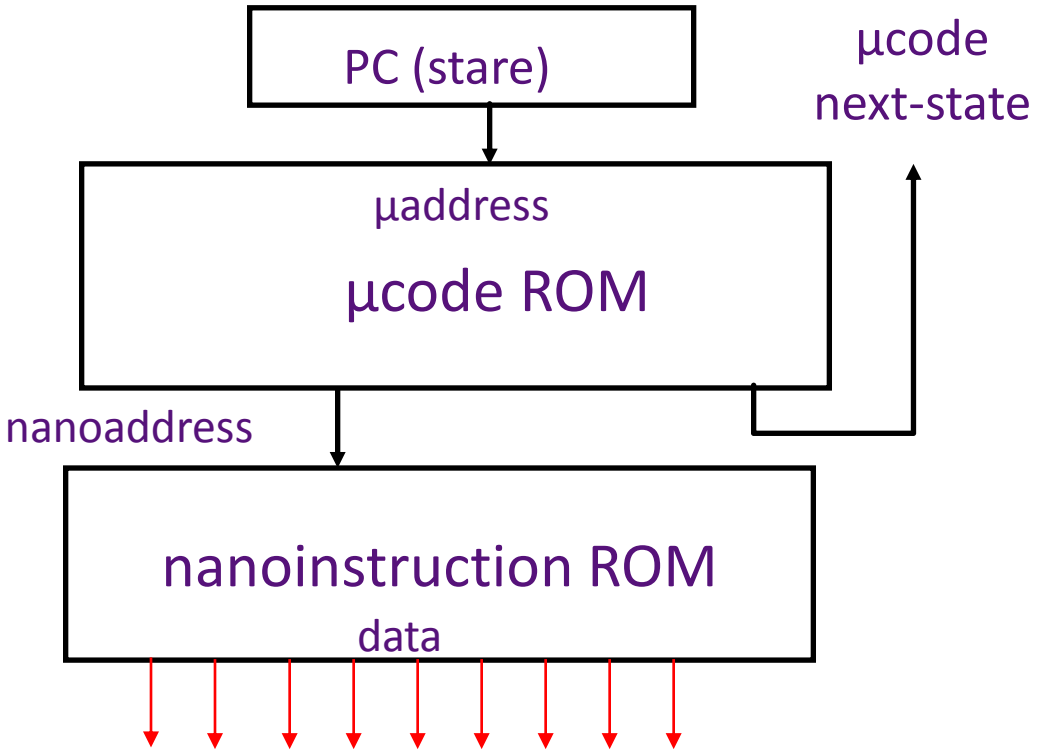


- µcodul orizontalizat are µinstrucțiuni mai lungi
  - Mai multe operații per µinstrucțiune
  - Mai puțini pași în microcod pe macroinstrucțiune
  - Sparse encoding ⇒ mai mulți biți
- µcodul verticalizat are µinstrucțiuni scurte
  - De obicei, o singură operație de transfer date per µinstrucțiune
    - µinstrucțiuni separate pentru branch
  - Mai mulți pași în microcod per macroinstrucțiune
  - Mai compact ⇒ mai puțini biți
- Nanocoding
  - Încearcă să combine cele două practici

Nanocoding

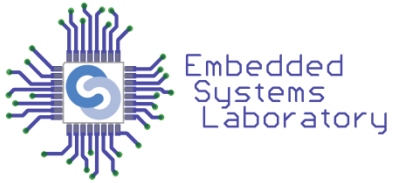
Exploatează existența secvențelor recurente de µcod, e.g.,

ALU_0 A <- Reg[rs1]
...
$ALUi_0$ A <- Reg[rs1]
...



MC68000 avea un microcod pe 17-biți ce conținea un µjump pe 10 biți sau un pointer pe 9 biți la o nanoinstrucțiune
- Nanoinstrucțiunile aveau 68 de biți lățime și decodificate produceau 196 semnale de control

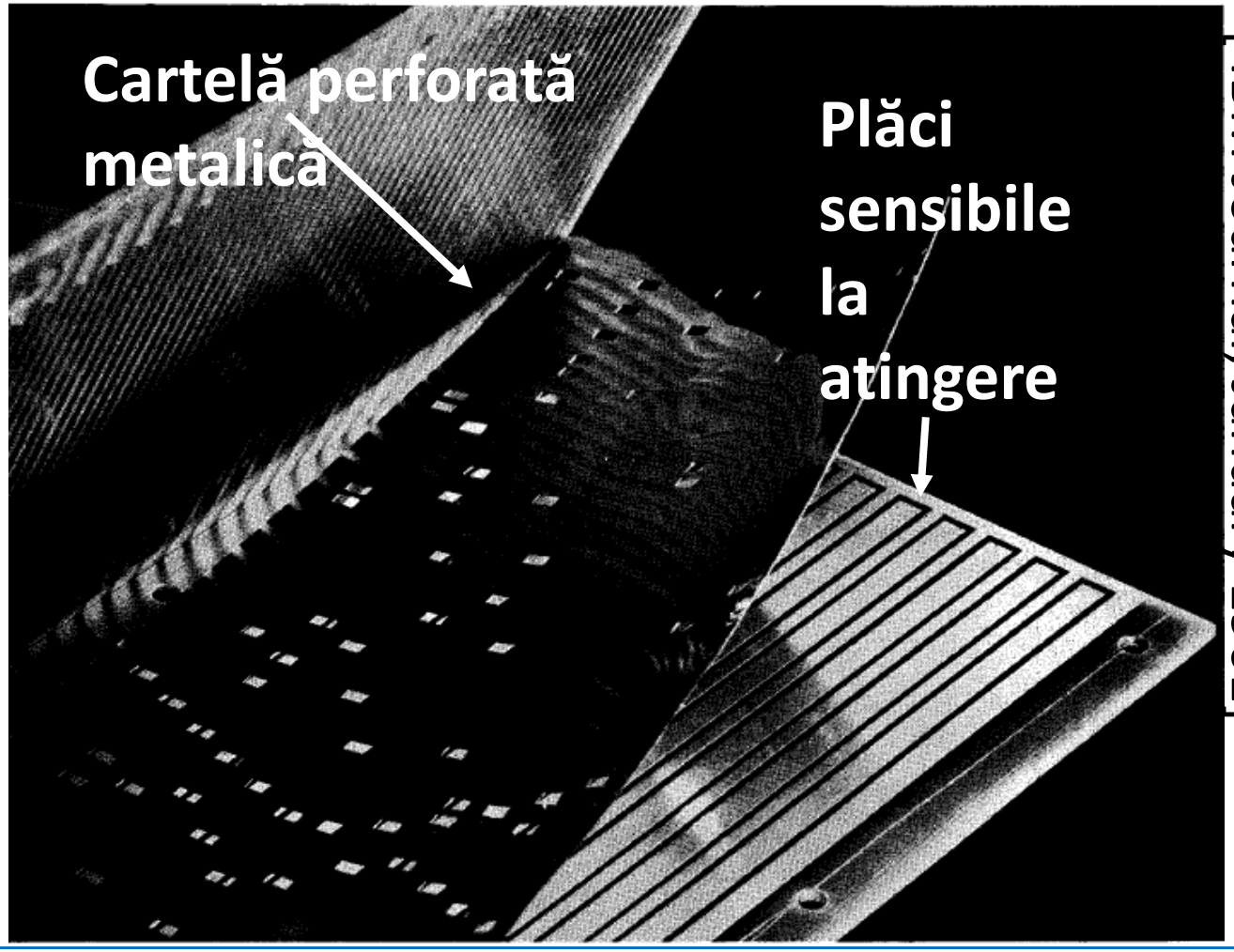
Microprogramarea pentru IBM 360



	M30	M40	M50	M65
Datapath width (bits)	8	16	32	64
µinst width (bits)	50	52	85	87
µcode size (K µinsts)	4	4	2.75	2.75
µstore technology	CCROS	TCROS	BCROS	BCROS
µstore cycle (ns)	750	625	500	200
memory cycle (ns)	1500	2500	2000	750
Rental fee ($K/month)	4	7	15	35

Numai cele mai rapide modele (75 și 95) aveau UC hardwired

IBM Card Capacitor Read-Only Storage



[IBM Journal, January 1961]

Emularea Microcodului

- IBM a făcut o eroare atunci când a subestimat importanța compatibilității software cu modelele mai vechi atunci când a introdus seria 360.
- Honeywell a furat o parte din utilizatorii IBM1401 atunci când a oferit un pachet software de translație a programelor IBM în cod pentru seria Honeywell H200
- IBM a răspuns prin adăugarea de microcod opțional pentru seria 360 care putea să emuleze ISA IBM1401, pe care l-a extins mai apoi pentru seria IBM7000
  - Un program foarte popular pentru 1401 era un simulator al seriei 650 pentru că mulți clienți rulau programe de 650 emulate pe mașini 1401.
    - (*650, simulat pe 1401, emulat pe 360)*

Microprogramarea a atins apogeul în anii ‘70

- Memoriile ROM erau cu mult mai rapide decât cele DRAM
- Pentru un ISA complex, unitatea de control microprogramat era mai ieftină și mai simplu de fabricat
- *Instrucțiunile noi*, e.g., floating point, puteau fi adăugate fără modificări ale magistralelor de date
- *Repararea defectelor* controllerului era mai simplă
- Compatibilitatea ISA a numeroaselor modele de calculatoare putea fi obținută ușor și ieftin

Aproape toate calculatoarele acelei epoci aveau o unitate de control microprogramată

Writable Control Store (WCS)

- Implementăm memoria de control în RAM, nu în ROM
  - Memoriile MOS SRAM sunt acum aproape la fel de rapide ca un ROM (memoriile cu mies de ferită/DRAM erau de 2-10x mai lente)
  - E foarte greu să scrii un microprogram fără erori din prima încercare
- Opțiunea de User-WCS pentru anumite procesoare
  - Permite utilizatorilor să schimbe microcodul pentru procesorul în cauză
- User-WCS *a eșuat*
  - Nu au existat suficiente programe care să folosească această abilitate
  - Dificil de stocat software util într-un spațiu atât de mic
  - Unitatea de control era optimizată pentru microcodul original, mai puțin pentru restul
  - Protecția integrității ISA dificilă dacă utilizatorul poate schimba microcodul
  - Memoria virtuală necesita microcod restartabil

Microprogramarea nu a dispărut

- A jucat un rol crucial în dezvoltarea microprocesoarelor anilor 80
  - DEC uVAX, Motorola 68K series, Intel 286/386
- Joacă un rol secundar, dar important, în microprocesoarele moderne
  - e.g., AMD Bulldozer, Intel Ivy Bridge, Intel Atom, IBM PowerPC, …
  - Majoritatea instrucțiunilor sunt executate direct (logică cablată)
  - Instrucțiunile utilizate mai puțin frecvent și/sau complicate, folosesc microcod
- Se pot aplica patch-uri la microcod pentru a repara anumite erori ISA detectate post-fabricare (ex. Procesoarele Intel încarcă patch-uri de microcod la bootup)

Acknowledgements

- Aceste slide-uri conțin materiale aparținând:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)
  - Behrooz Parhami (UCSB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

Calculatoare Numerice

– Cursul 9 –

Măsurarea Performanței

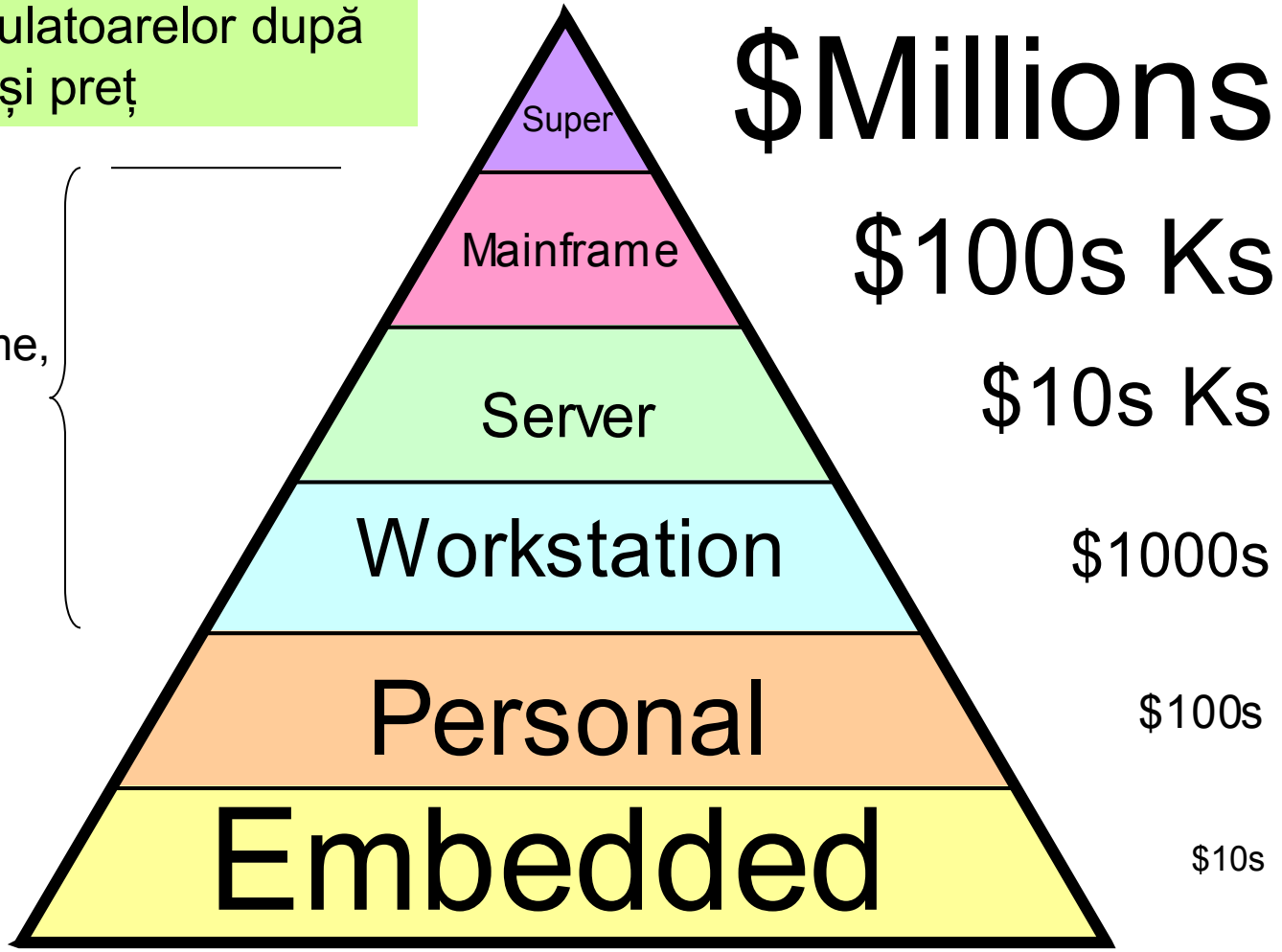
Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Piramida Preț/Performanță

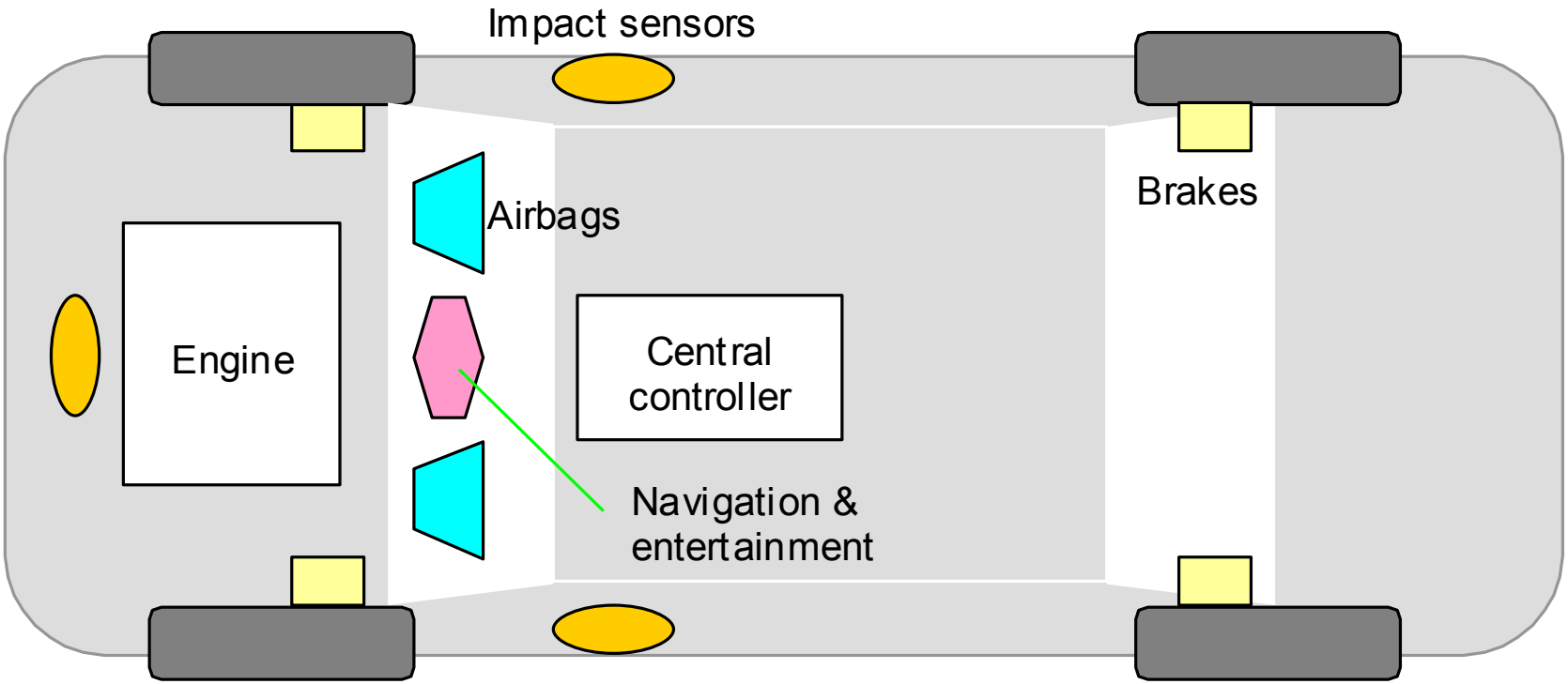
Embedded Systems Laboratory

Clasificarea calculatoarelor după putere de calcul și preț

Diferențe de mărime, performanță

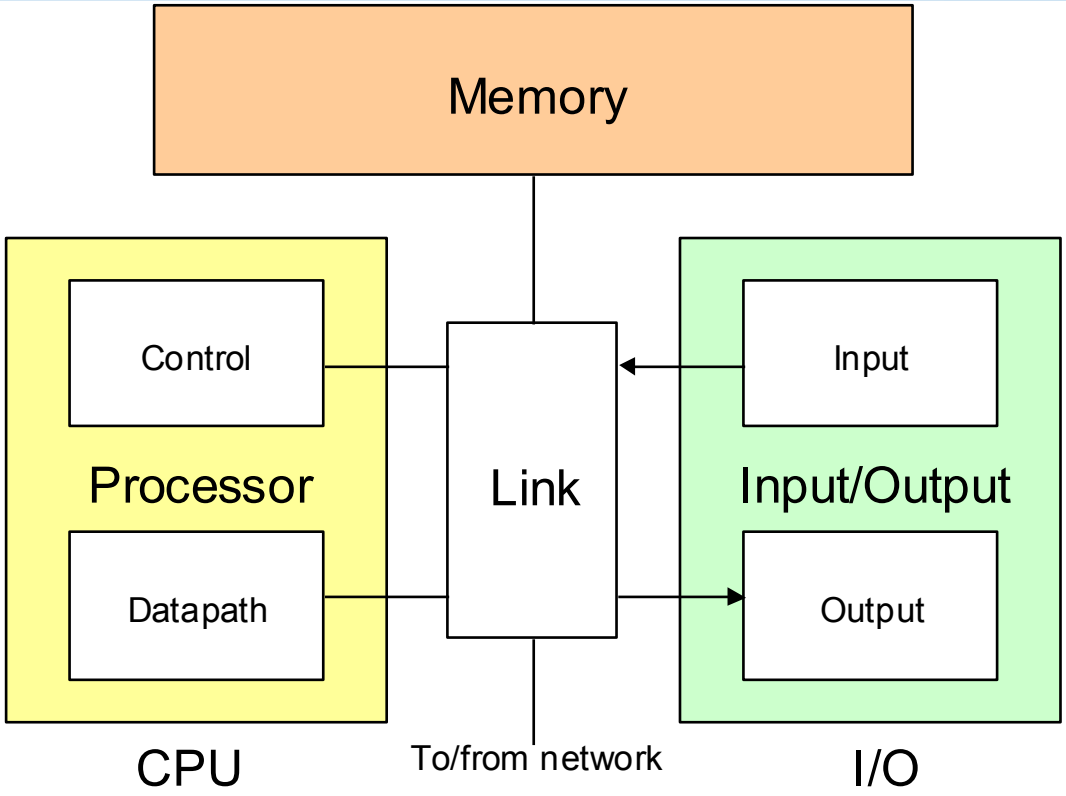


Calculatoare embedded automotive



Procesoarele embedded sunt ubicue și totuși invizibile. Pot fi găsite în automobile, electronice, electrocasnice și multe alte produse.

Subsistemele unui calculator digital



Cele (trei, patru, cinci sau) șase componente principale ale unui calculator numeric. De obicei, unitatea de legătură (link unit), care poate să fie o magistrală simplă sau o matrice de interconexiuni, nu este inclusă în acest tip de diagramă.

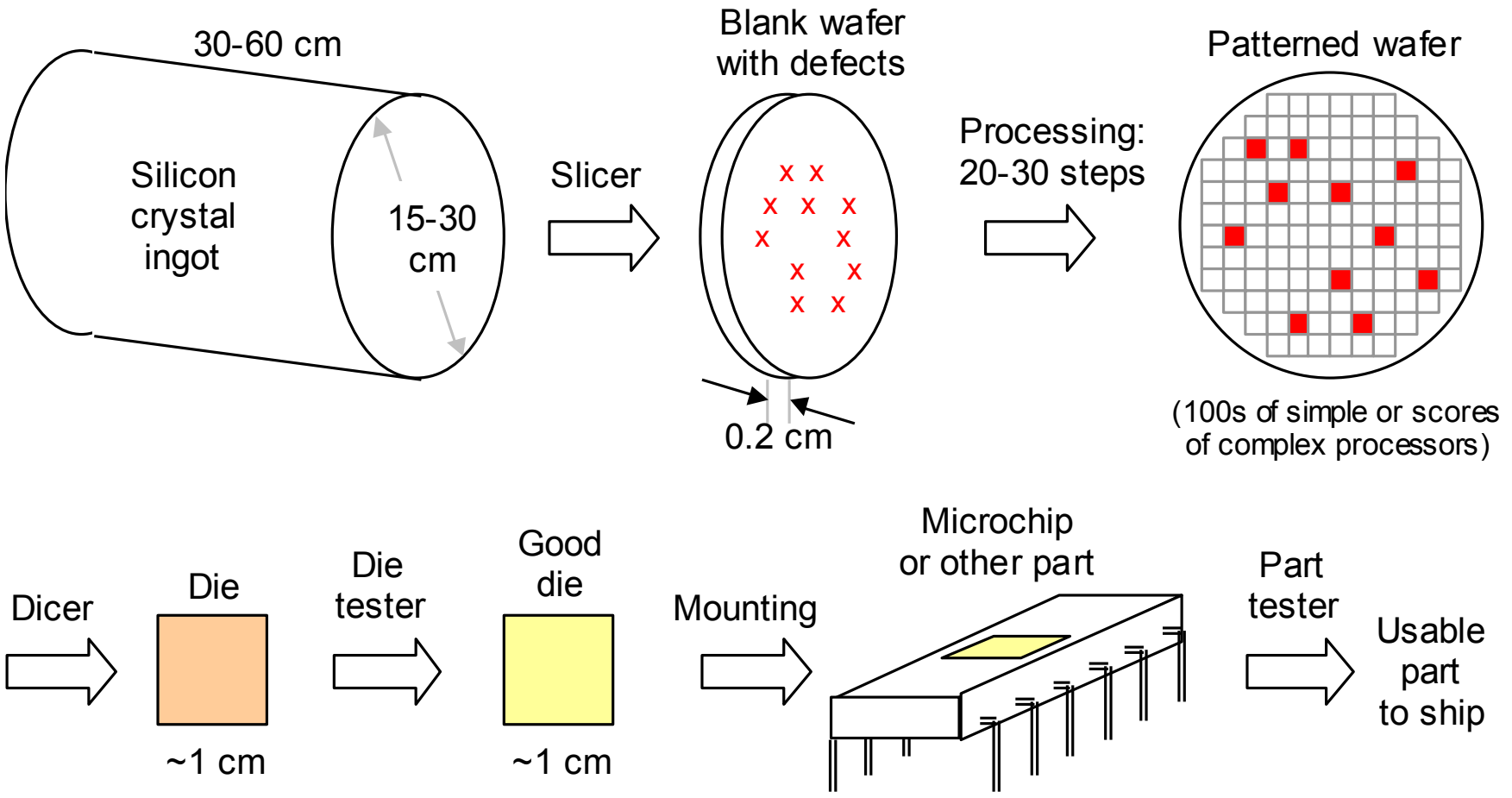
Progresul pe generații

Embedded Systems Laboratory

Cele cinci generații de calculatoare digitale

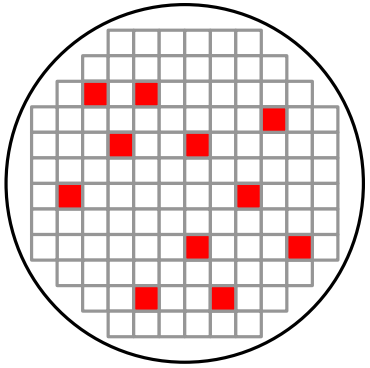
Generația (data început)	Tehnologia procesorului	Sistem memorie	Dispozitive I/O noi	Look & feel
0 (1600s)	(Electro-) mecanic	Roți, cartele	Pârghii, cartele perforate	Echipament de fabrică
1 (1950s)	Tuburi cu vid	Tambur magnetic	Bandă hârtie & magnetică	Echipament cât o sală mare
2 (1960s)	Tranzistoare	Miez de ferită	Imprimantă, terminal text	Cameră server
3 (1970s)	SSI/MSI	RAM/ROM	Disc, tastatură, monitor	Echipament de birou
4 (1980s)	LSI/VLSI	SRAM/DRAM	Rețea, CD, mouse,sunet	Desktop/ laptop micro
5 (1990s)	ULSI/GSI/ WSI, SOC	SDRAM, flash	Senzori/actuatoare , point&click, touch	Invizibil, embedded

Producerea (și productivitatea) circuitelor integrate

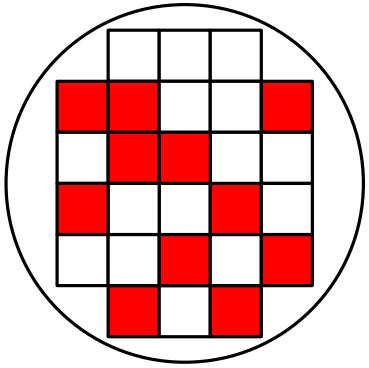


Procesul de fabricație pentru un circuit integrat

Efectele mărimii matriței asupra productivității



120 dies, 109 good



26 dies, 15 good

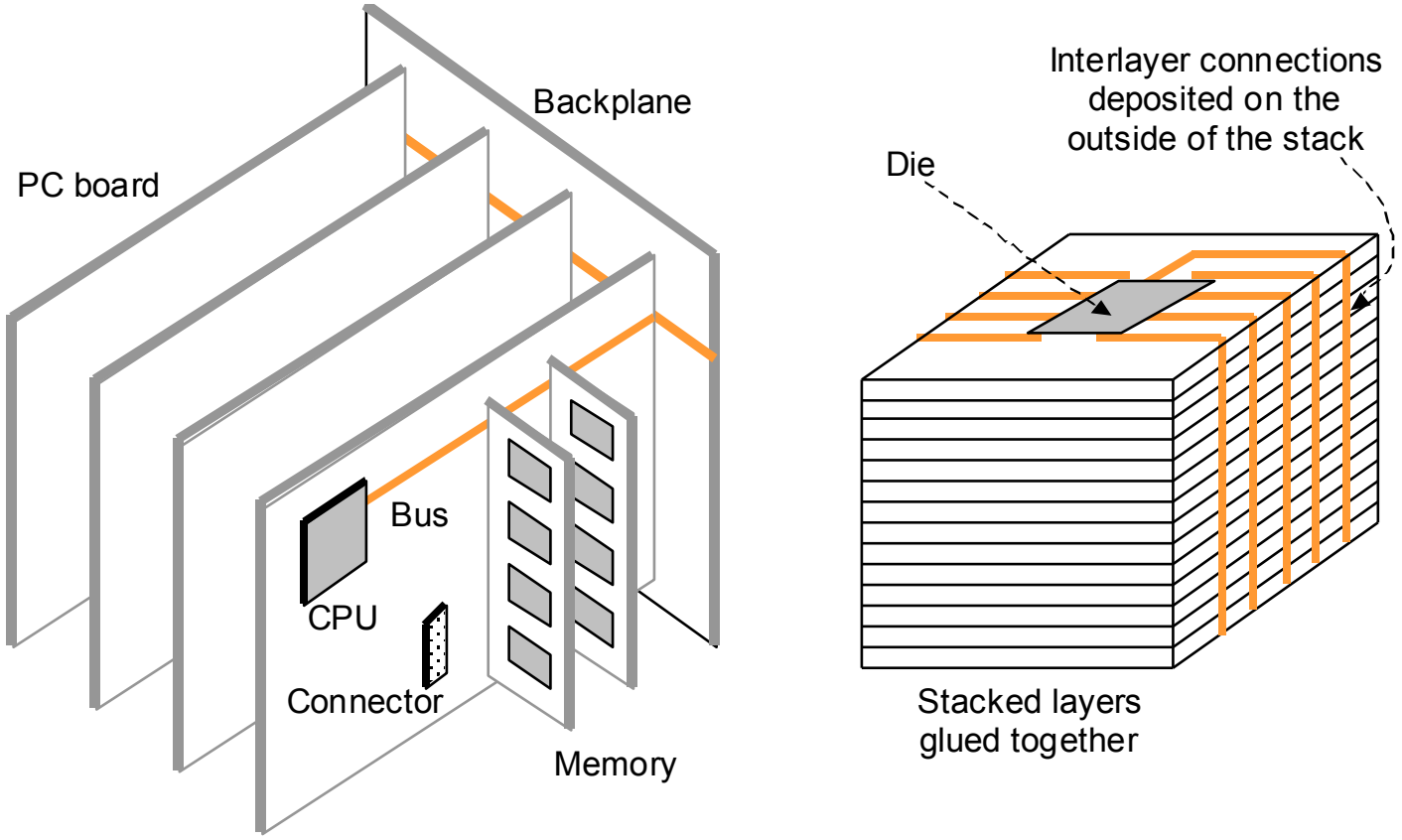
Scăderea dramatică a productivității cu creșterea suprafeței chip-ului

Die yield $=_{def}$ (number of good dies) / (total number of dies)

Die yield = Wafer yield $\times$ [1 + (Defect density $\times$ Die area) / a]$^{-a}$

Die cost = (cost of wafer) / (total number of dies $\times$ die yield)
= (cost of wafer) $\times$ (die area / wafer area) / (die yield)

Tehnologii pentru procesoare și memorii

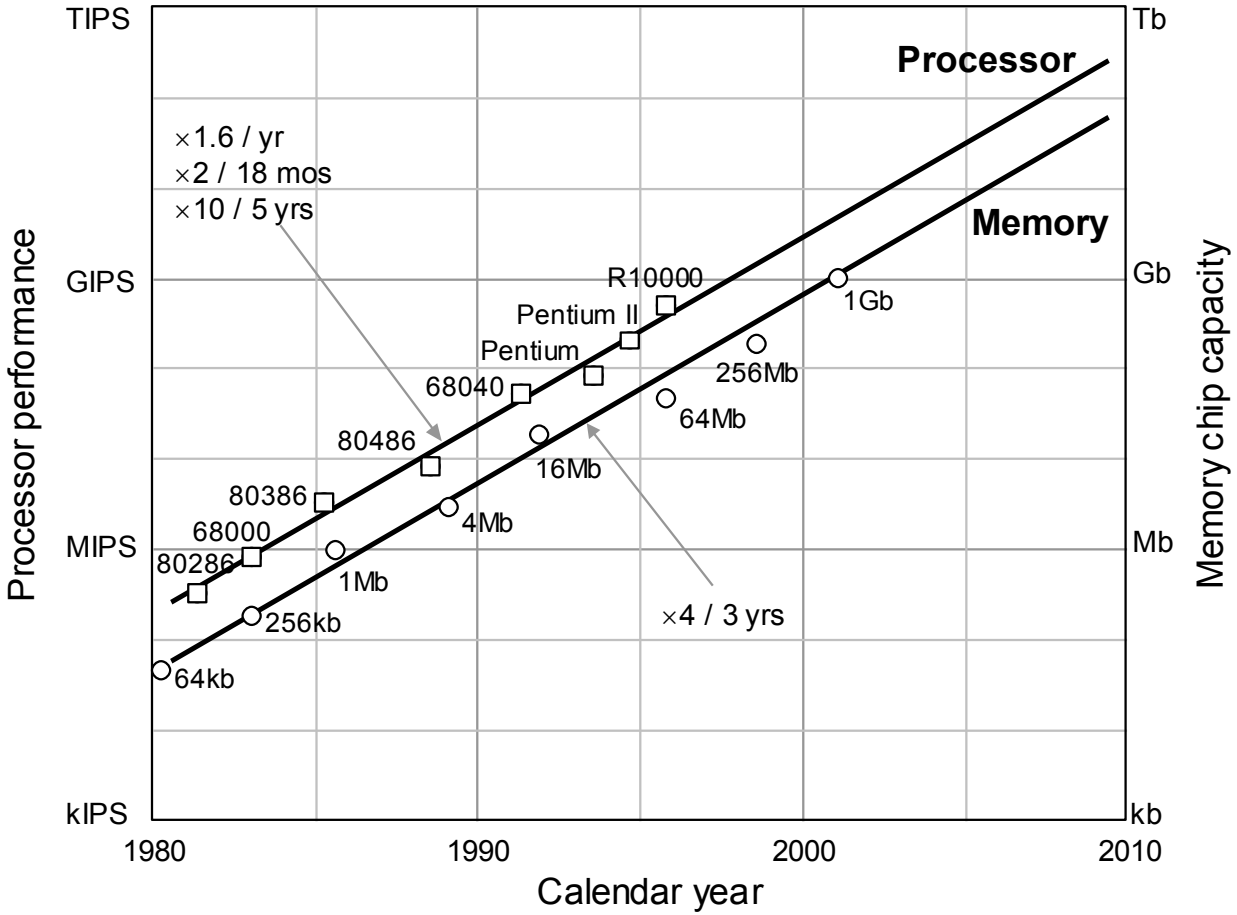


(a) 2D or 2.5D packaging now common

(b) 3D packaging of the future

Încapsularea procesoarelor, memoriilor și a altor componente.

Legea lui Moore



Creșterea performanței procesoarelor și memoriilor de-a lungul anilor.

”Rateuri” în previziunile despre viitorul tehnicii de calcul

Embedded Systems Laboratory

“DOS addresses only 1 MB of RAM because we cannot imagine any applications needing more.” Microsoft, 1980

“640K ought to be enough for anybody.” Bill Gates, 1981

“Computers in the future may weigh no more than 1.5 tons.” *Popular Mechanics*

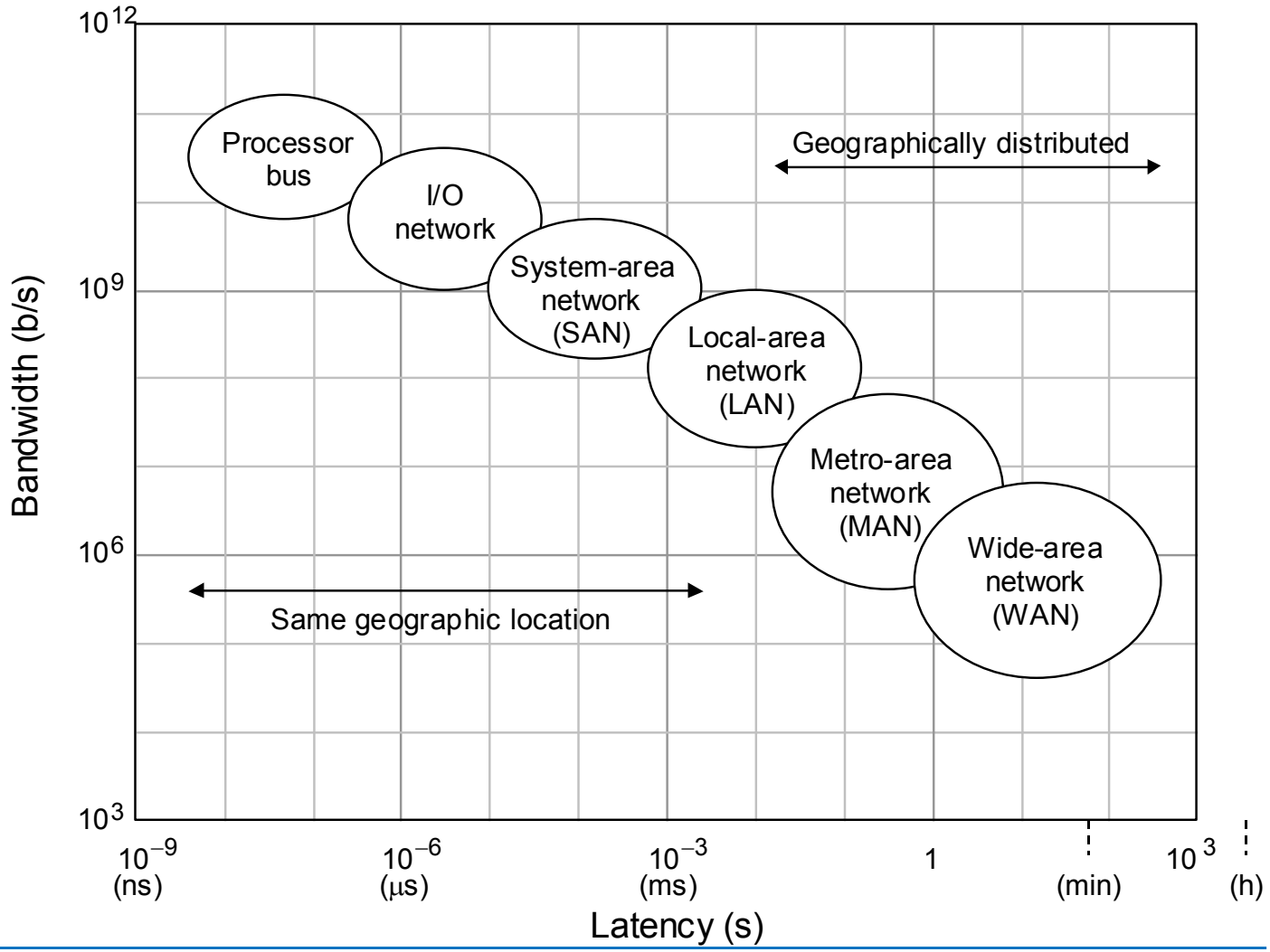
“I think there is a world market for maybe five computers.” Thomas Watson, IBM Chairman, 1943

“There is no reason anyone would want a computer in their home.” Ken Olsen, DEC founder, 1977

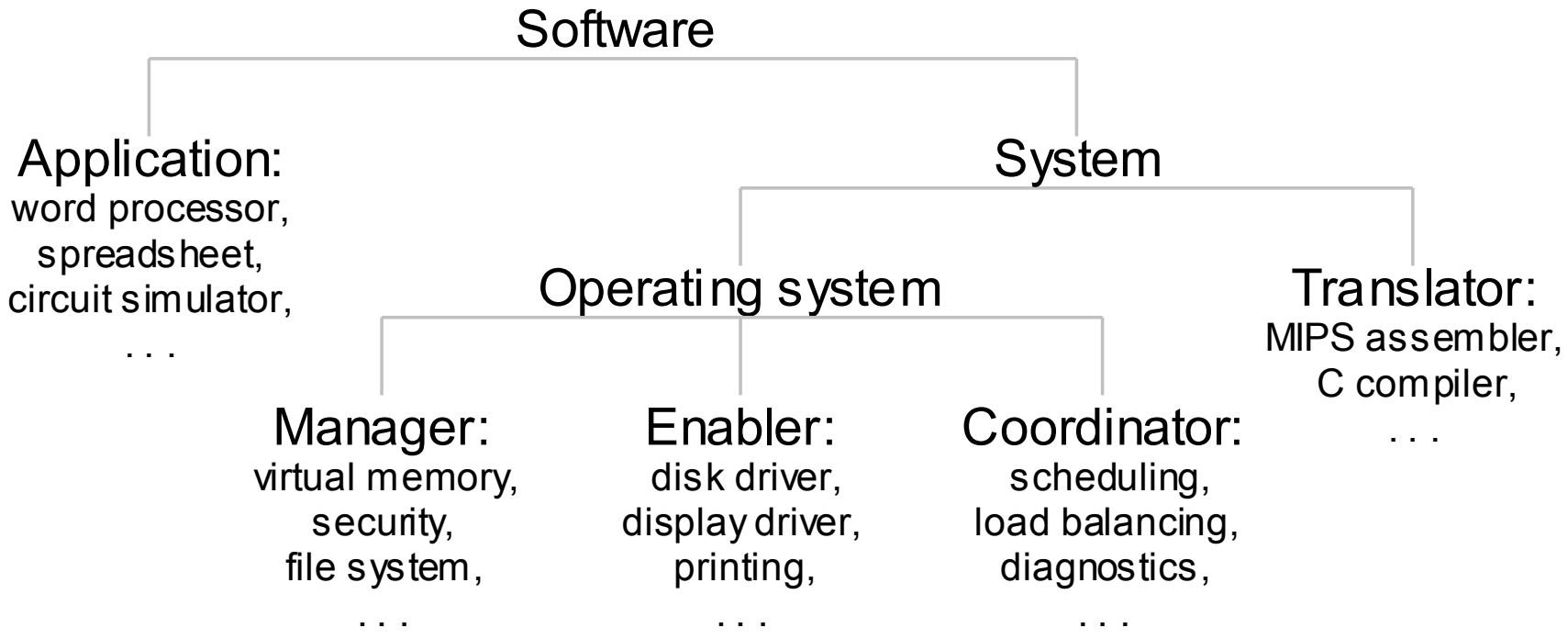
“The 32-bit machine would be an overkill for a personal computer.” Sol Libes, *ByteLines*

Tehnologii de comunicație

Caracteristicile de latență și lățime de bandă ale diferitelor clase de rețele de comunicație.

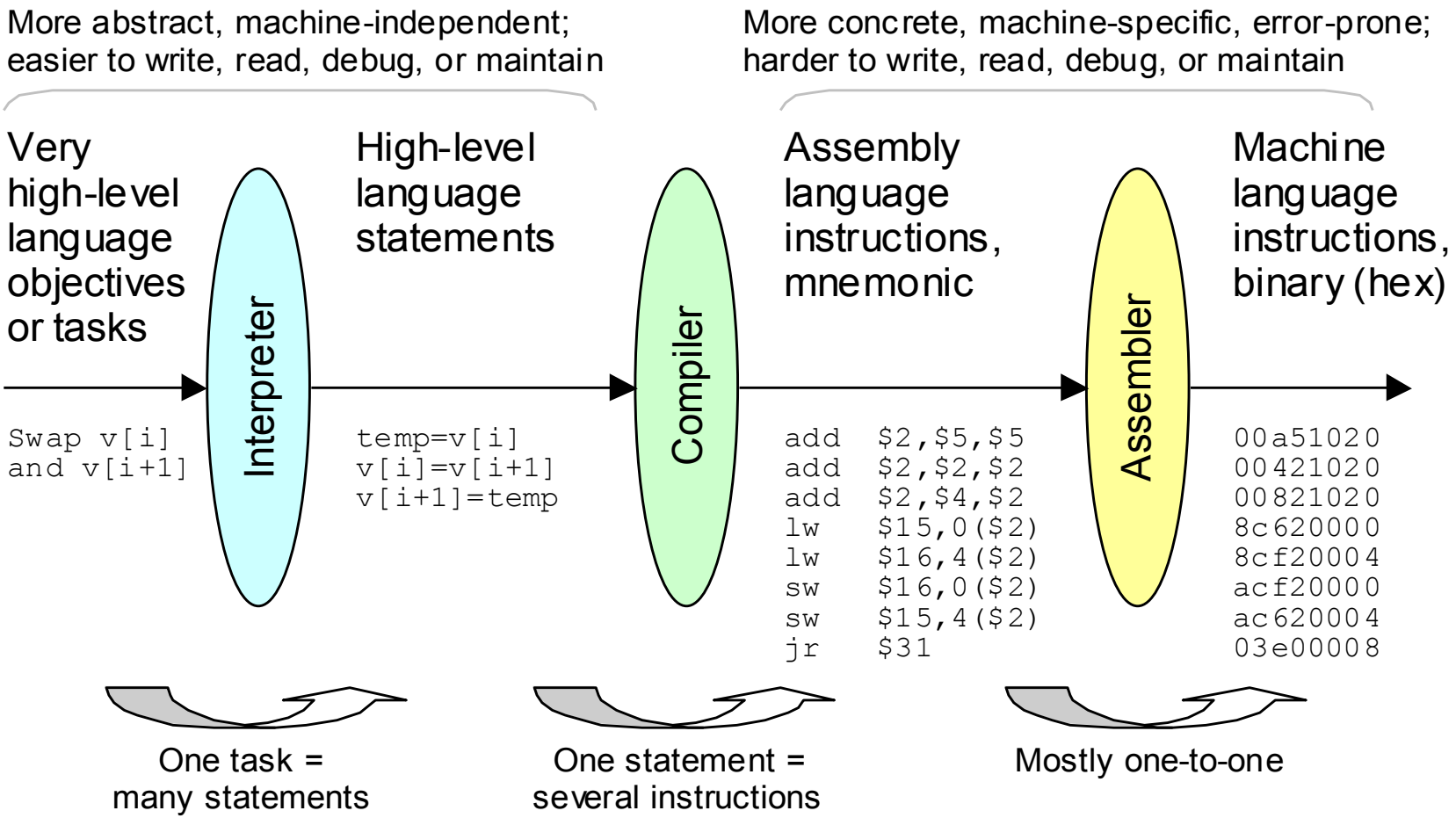


Sisteme software și aplicații



Diferitele clase de software.

Programarea High- vs Low-Level



Modele și abstractizări în limbajele de programare.

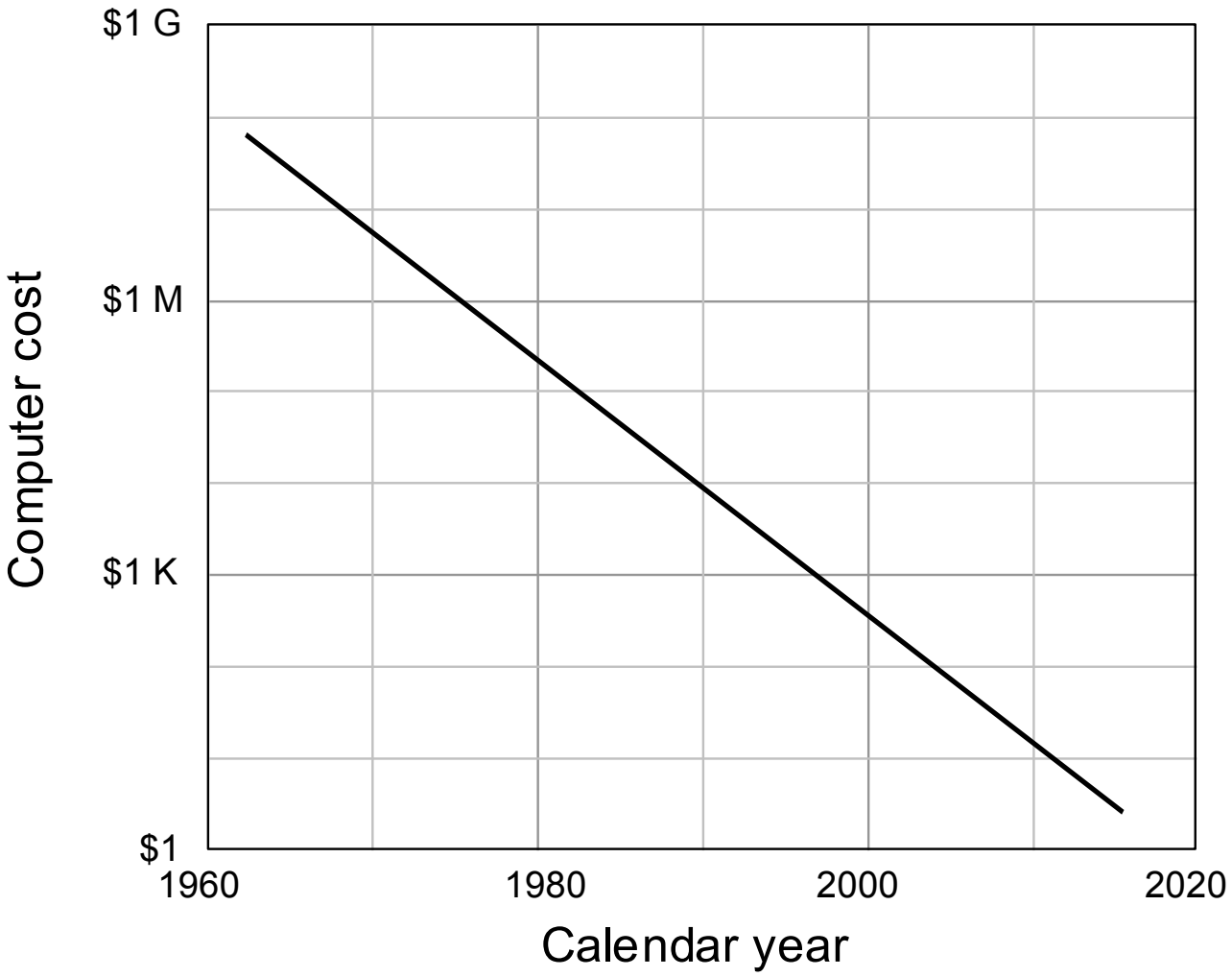
Performanța calculatoarelor

Embedded Systems Laboratory

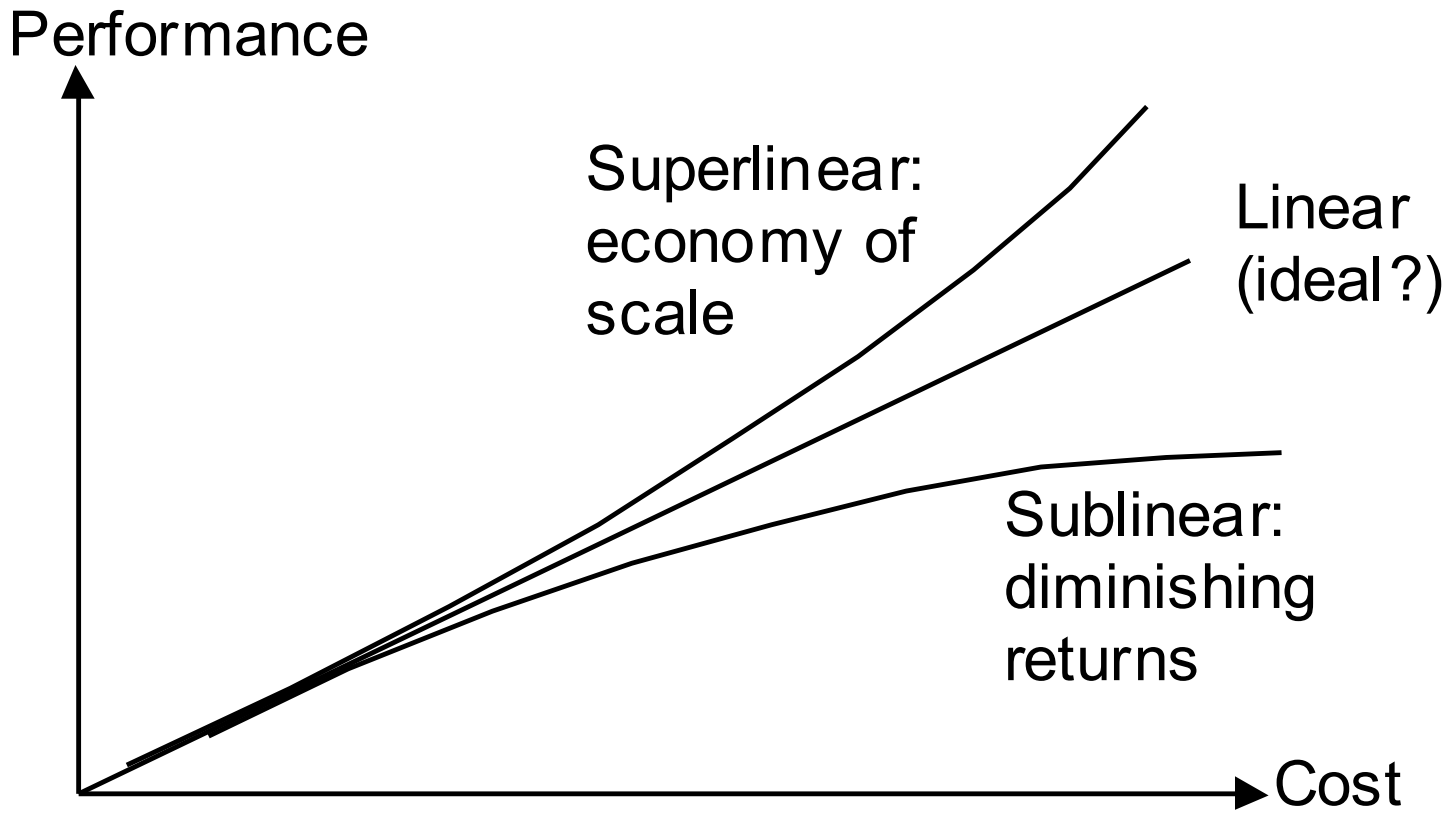
Cuprins

1. Cost, performanță și raportul Cost/Performanță
2. Definirea performanței calculatoarelor
3. Mărirea performanței și Legea lui Amdahl
4. Modelarea și măsurarea performanței
5. Raportarea performanței
6. În căutarea performanței mărite

Cost, performanță și Cost/Performanță

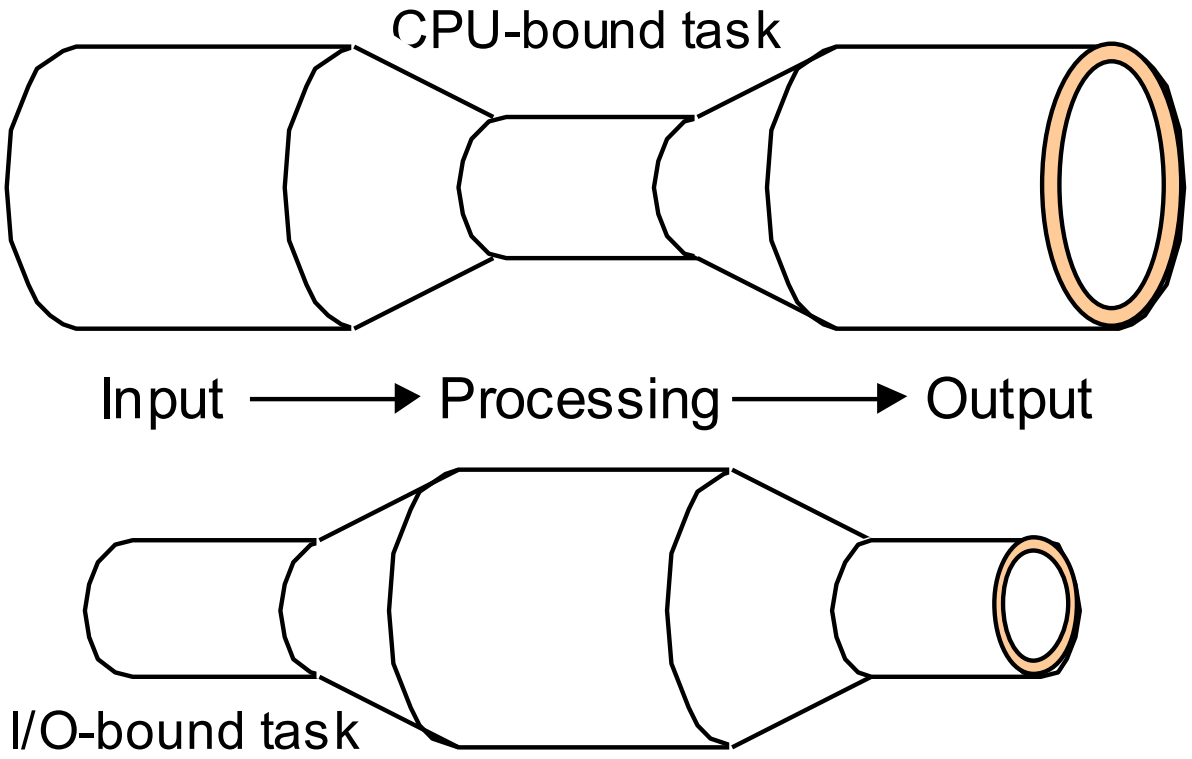


Cost/Performanță



Îmbunătățirea performanței în funcție de cost

Definirea performanței sistemelor de calcul



Analogia cu un pipeline arată că discrepanța dintre puterea de procesare și capabilitățile I/O duce la o constrângere (bottleneck) a performanței.

Analogie: comparăm șase avioane comerciale

Performanța avioanelor

Caracteristicile cheie pentru șase avioane de pasageri (cifrele sunt aproximative și sunt alese ca valori medii ale diferitelor configurații posibile);

Avion	Pasageri	Rază (km)	Viteză (km/h)	Preț ($M)
Airbus A310	250	8 300	895	120
Boeing 747	470	6 700	980	200
Boeing 767	250	12 300	885	120
Boeing 777	375	7 450	980	180
Concorde	130	6 400	2 200	350
DC-8-50	145	14 000	875	80

Diferite definiții ale performanței

Performanța din punctul de vedere al pasagerului: Viteza

Viteza este doar una din variabile. Timpul total al călătoriei este mai mare decât timpul de zbor. Dacă distanța depășește raza de acțiune a unui avion rapid, e de preferat alegerea unui avion mai lent, dar care nu are nevoie de escală pentru realimentare.

Performanța din pct. de vedere al companiei aeriene: Productivitate

Măsurată în pasageri-km/oră (relavantă dacă prețurile biletelor ar fi d.p. cu distanța parcursă, ceea ce nu este chiar adevărat)

Airbus A310	250 × 895 = 0.224 M pasageri-km/hr
Boeing 747	470 × 980 = 0.461 M pasageri-km/hr
Boeing 767	250 × 885 = 0.221 M pasageri-km/hr
Boeing 777	375 × 980 = 0.368 M pasageri-km/hr
Concorde	130 × 2200 = 0.286 M pasageri-km/hr
DC-8-50	145 × 875 = 0.127 M pasageri-km/hr

Performanța din pct. de vedere al FAA: Siguranța

Eficiența: Cost/Performanță

Embedded Systems Laboratory

Caracteristicile cheie pentru șase avioane de pasageri (cifrele sunt aproximative și sunt alese ca valori medii ale diferitelor configurații posibile);

Avion	Pasageri	Rază (km)	Viteză (km/h)	Preț ($M)	Valorile mari sunt ok: Productivitate (M P km/hr)	Valorile mici sunt ok: Cost / Performanță
A310	250	8 300	895	120	0.224	536
B 747	470	6 700	980	200	0.461	434
B 767	250	12 300	885	120	0.221	543
B 777	375	7 450	980	180	0.368	489
Concorde	130	6 400	2 200	350	0.286	1224
DC-8-50	145	14 000	875	80	0.127	630

Concepte de performanță și speedup

Embedded Systems Laboratory

Performanță = 1 / Timp execuție

se poate simplifica

Performanță = 1 / Timp execuție CPU

(Performanța lui M_1) / (Performanța lui M_2) = Speedup M_1 față de M_2
= (Timp execuție M_2) / (Timp execuție M_1)

Terminologie: M_1 is *x* times **as fast as** M_2 (e.g., 1.5 times as fast)
M_1 is 100(*x* – 1)% **faster than** M_2 (e.g., 50% faster)

CPU time = Instructions × (Cycles per instruction) × (Secs per cycle)
= Instructions × CPI / (Clock rate)

Instruction count, CPI, și clock rate nu sunt complet independente, așa că îmbunătățirea uneia de x ori poate duce la îmbunătățirea cu aproape x ori a timpului total de execuție

Dezvoltarea formulei pt timpul de execuție CPU

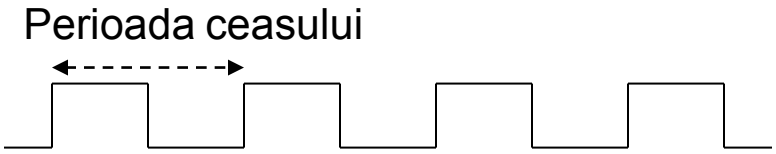
Embedded Systems Laboratory

CPU time = Instructions × (Cycles per instruction) × (Secs per cycle)
= Instructions × Average CPI / (Clock rate)

Instrucțiuni: Numărul de instrucțiuni executate, != numărul de instrucțiuni dintr-un program (estimare dinamică)

Average CPI: Calculat în funcție de estimarea dinamică a numărului de instrucțiuni executate și de numărul de cicli de ceas necesari pentru execuția unei instrucțiuni

Clock rate: 1 GHz = 10^9 cicli / s (1 ciclu = 10^{-9} s = 1 ns)
200 MHz = 200 × 10^6 cicli / s (1 ciclu = 5 ns)



Dynamic Instruction Count

Embedded Systems Laboratory

Cât de multe instrucțiuni sunt executate în acest fragment de cod?

```
250 instrucțiuni
for i = 1, 100 do
20 instrucțiuni
    for j = 1, 100 do
    40 instrucțiuni
        for k = 1, 100 do
        10 instrucțiuni
        endfor
    endfor
endfor
```

Static count = 326

Fiecare ”for” constă din 2 instrucțiuni: incrementarea indexului și verificarea condiției de iesire

12,422,450 Instrucțiuni

2 + 20 + 124,200 instrucțiuni
100 iterații
12,422,200 instrucțuni în total

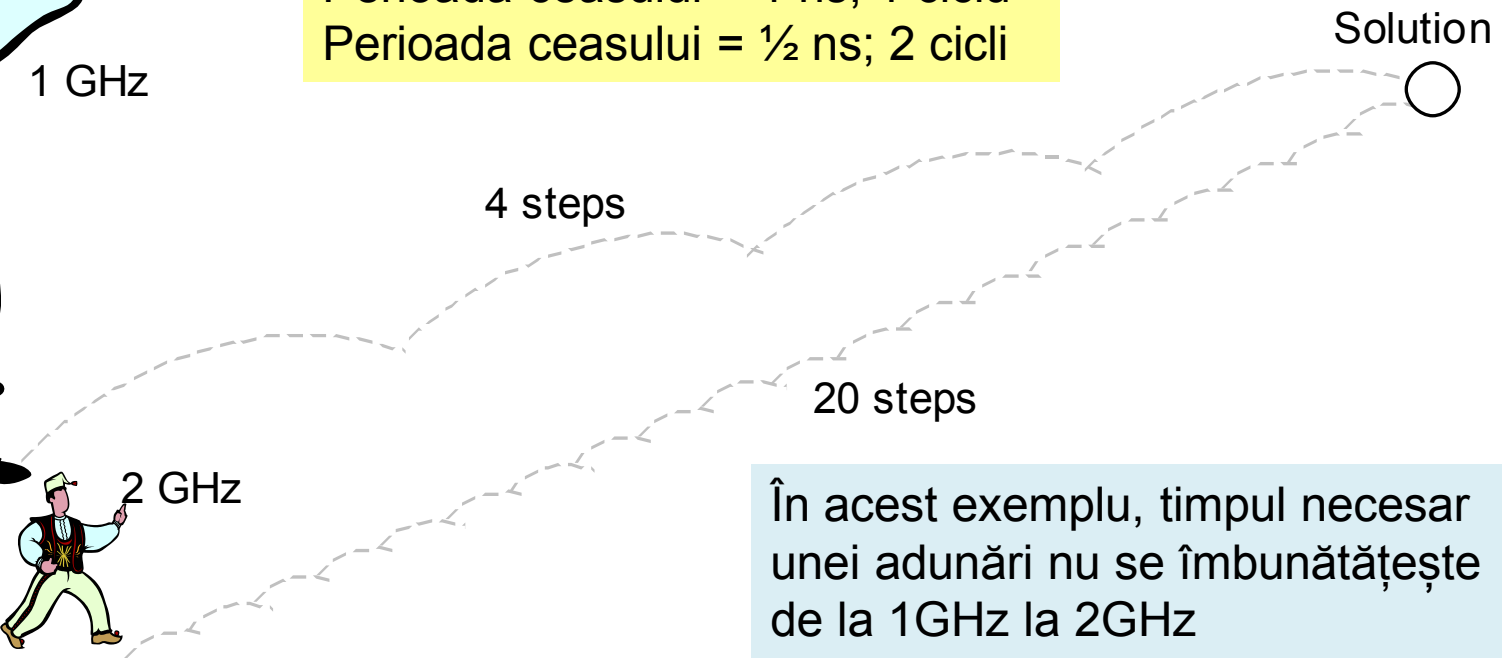
2 + 40 + 1200 instrucțiuni
100 iterații
124,200 instrucțiuni în total

2 + 10 instrucțiuni
100 iterații
1200 instrucțiuni în total

```
for i = 1, n
while x > 0
```

Ceas mai rapid ≠ Timp de execuție mai mic

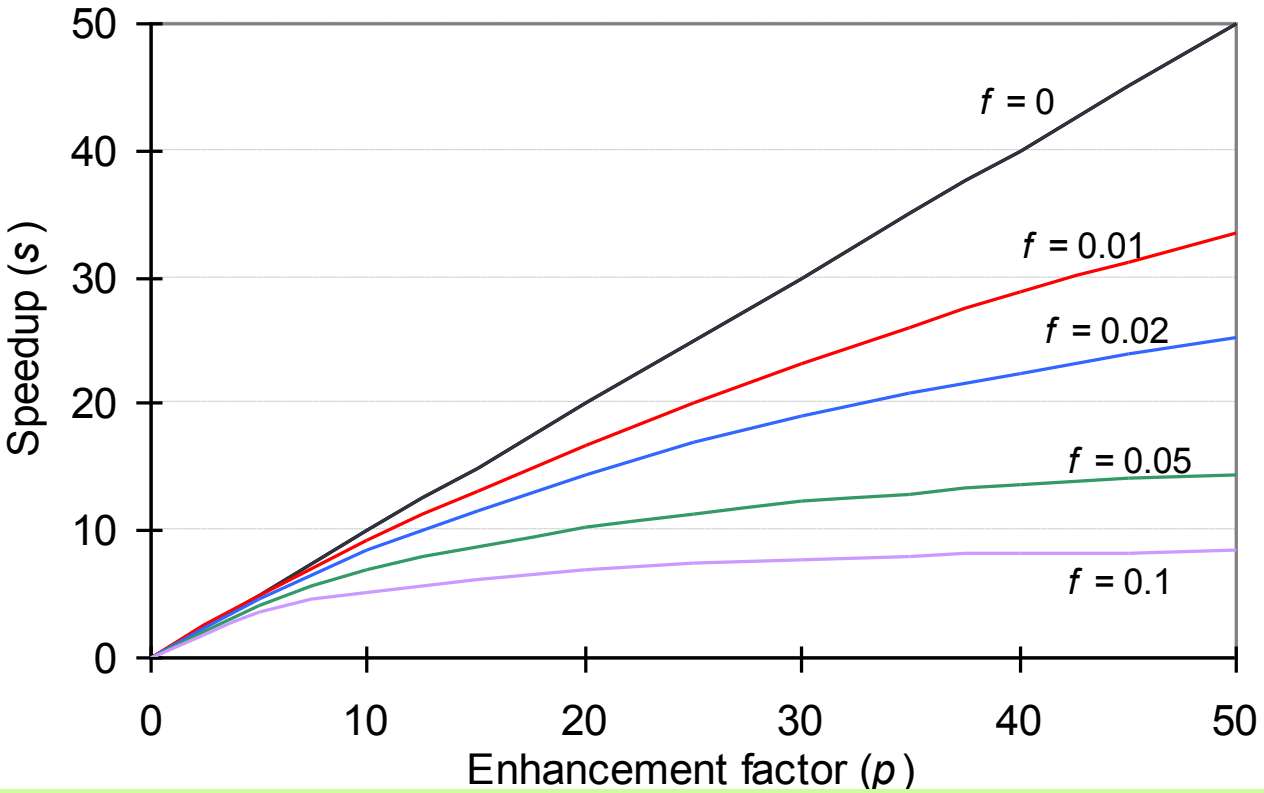
Presupunem că o adunare = 1 ns
Perioada ceasului = 1 ns; 1 ciclu
Perioada ceasului = ½ ns; 2 cicli



În acest exemplu, timpul necesar unei adunări nu se îmbunătățește de la 1GHz la 2GHz

Pași mai rapizi nu înseamnă neapărat că ajungi la destinație într-un timp mai scurt.

Creșterea performanței: Legea lui Amdahl



f = fracțiunea ne-paralelizabilă

p = speedup pentru restul

$$s = \frac{1}{f + (1-f)/p}$$

$$\leq min(p, 1/f)$$

Legea lui Amdahl: Îmbunătățirea vitezei de execuție dacă fracțiunea f a unui task este ne-paralelizabilă și restul de 1-f din task rulează de p ori mai repede.

Legea lui Amdahl în proiectare

Un procesor petrece 30% din timp cu adunări flp, 25% cu înmulțiri flp și 10% cu împărțiri flp. Evaluați îmbunătățirea performanței pentru:

a. Sumatorul flp este de 2x mai rapid
b. Multiplicatorul flp este de 3x mai rapid.
c. Unitatea de împărțire flp este de 10x mai rapidă

Soluție

a.Speedup adder = 1 / [0.7 + 0.3 / 2] = 1.18
b.Speedup multiplier = 1 / [0.75 + 0.25 / 3] = 1.20
c.Speedup divider = 1 / [0.9 + 0.1 / 10] = 1.10

Dar dacă și sumatorul și multiplicatorul sunt reproiectate simultan?

Legea lui Amdahl în management

Membrii unui grup de cercetare vizitează frecvent biblioteca. Fiecare vizită durează 20 de minute. Grupul decide să se aboneze la o suită de publicații care acoperă 90% din vizitele la bibliotecă; timpul de acces la publicații este redus la 2 minute.

a. Cu cât s-a îmbunătățit timpul mediu de acces la publicații?

b. Dacă grupul are 20 de membri, fiecare făcând 2 vizite/săpt. La bibliotecă, care este cheltuiala maxim admisă pt. abonamente? Presupuneți că sunt 50 săpt. de muncă pe an și un cercetător e plătit cu 25$/h.

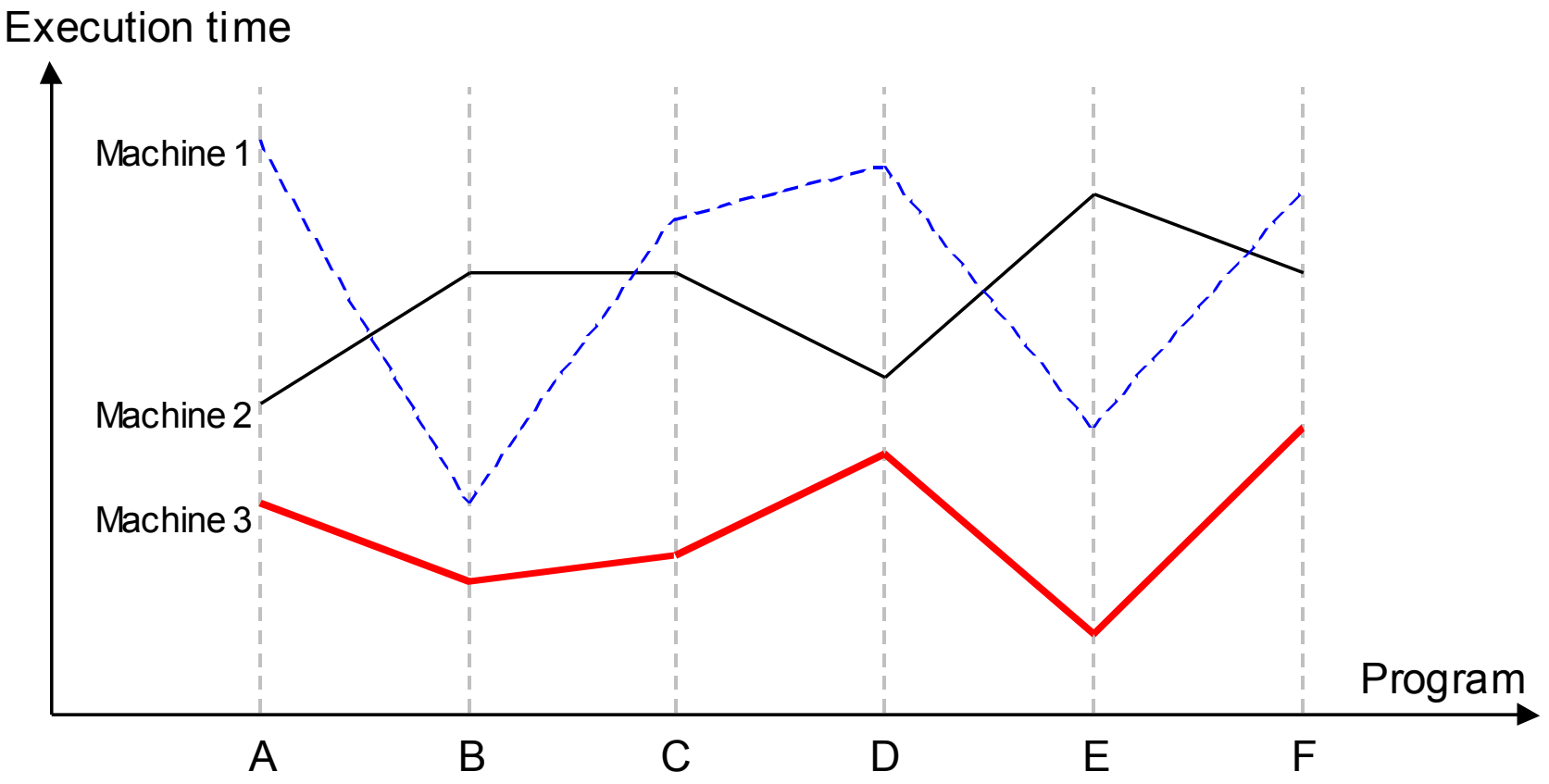
Soluție

a.Speedup în accesul la publicații = 1 / [0.1 + 0.9 / 10] = 5.26

b.Timp economisit = 20×2×50×0.9 (20 – 2) = 32,400 min = 540 h

Costuri = 540 × $25 = $13,500 = Cheltuiala maxim admisă

Măsurarea vs. Modelarea performanței



Timpii de execuție a șase programe pe trei mașini diferite

Legea lui Amdahl generalizată

Timpul inițial de rulare pt un program = 1 = $f_1 + f_2 + \ldots + f_k$

Noul timp de rulare, după ce fracțiunea f_i este rulată de p_i ori mai repede

$$\frac{f_1}{p_1} + \frac{f_2}{p_2} + \ldots + \frac{f_k}{p_k}$$

Formula pentru speedup

$$S = \frac{1}{\frac{f_1}{p_1} + \frac{f_2}{p_2} + \ldots + \frac{f_k}{p_k}}$$

Dacă o fracțiune este încetinită, se folosește $s_j\, f_j$ în locul f_j / p_j, unde $s_j > 1$ este factorul de încetinire (slowdown)

Teste de performanță (benchmarking)

Embedded Systems Laboratory

Ești un inginer la Outtel, companie start-up care dorește să concureze cu Intel folosind noile sale procesoare care sunt mai bune de 2.5x decât Intel pt. operații floating-point. Acest nivel de performanță a putut fi atins printr-un compromis de design care a dus la creșterea cu 20% a timpului de execuție pentru toate celelalte instrțiuni. Job-ul tău este de a selecta benchmark-ul care să arate superioaritatea procesoarelor Outtel.

a. Care este fracția minimă de timp *f* de operații fpu pt un program care ruleaza pe un procesor Intel pt ca Outtel sa aibă un speedup de 2x sau mai bun?

Soluție

a.Folosim o formă generalizată a legii lui Amdahl pt care *f* are un speedup de 2.5 și restul un slowdown de 1.2 (100%+20%): $1 / [1.2(1 - f) + f / 2.5] \geq 2 \Rightarrow f \geq 0.875$

Estimarea performanței

Average CPI = $\sum_{\text{All instruction classes}}$ (Class-*i* fraction) × (Class-*i* CPI)

Machine cycle time = 1 / Clock rate

CPU execution time = Instructions × (Average CPI) / (Clock rate)

Frecvența folosirii, în procente, pt. diferitele tipuri de instrucțiuni în patru aplicații reprezentative

Aplicație → Clasa instr. ↓	Compresie de date	Compilator C	Simulator reactor	Modelarea mișcării atomilor
A: Load/Store	25	37	32	37
B: Integer	32	28	17	5
C: Shift/Logic	16	13	2	1
D: Float	0	0	34	42
E: Branch	19	13	9	10
F: All others	8	9	6	4

Calcule pentru CPI și IPS

Considerăm două implementări M_1 (600 MHz) și M_2 (500 MHz) al unui set de instrucțiuni ce conține următoarele clase:

Clasă	CPI pt. M_1	CPI pt. M_2	Comentarii
F	5.0	4.0	Floating-point
I	2.0	3.8	Integer arithmetic
N	2.4	2.0	Nonarithmetic

a. Care este performanța maximă pt. M_1 și M_2 în MIPS?

b. Dacă 50% din instrucțiuni sunt din clasa N, și restul împărțite egal între F și I, care mașină e mai rapidă? De câte ori?

Soluție

a. MIPS max. pt. M_1 = 600 / 2.0 = 300; pt. M_2 = 500 / 2.0 = 250

b. CPI mediu pt. M_1 = 5.0 / 4 + 2.0 / 4 + 2.4 / 2 = 2.95;

pt. M_2 = 4.0 / 4 + 3.8 / 4 + 2.0 / 2 = 2.95 $\rightarrow$ M_1 mai rapid de 1.2x

Rating-ul MIPS poate să inducă în eroare

Două compilatoare produc cod mașină pentru același program pe o mașină cu două clase de instrucțiuni:

Clasa	CPI	Compilator 1	Compilator 2
A	1	600M	400M
B	2	400M	400M

a. Care sunt timpii de rulare pentru cele 2 programe la un ceas de 1 GHz?
b. Care compilator produce cod mai rapid și de câte ori mai rapid?
c. Care program rulează la o rată MIPS mai mare?

Soluție

a. Timp rulare1 (2) = (600M $\times$ 1 + 400M $\times$ 2) / 10^9 = 1.4 s (1.2 s)
b. Codul generat de C2 e de 1.4 / 1.2 = 1.17x mai rapid
c. MIPS rating 1, CPI = 1.4 (2, CPI = 1.5) = 1000 / 1.4 = 714 (667)

Raportarea performanței

Timpul de execuție măsurat sau estimat pt. trei programe.

	Timp pe mașina X	Timp pe mașina Y	Speedup Y față de X
Program A	20	200	0.1
Program B	1000	100	10.0
Program C	1500	150	10.0
Toate trei	2520	450	5.6

Analogie: Dacă o mașină merge către un oraș aflat la 100km cu 100km/h și se întoarce cu 50km/h, viteza medie nu este (100+50)/2 ci este obținută luând în considerare că a parcurs 200km în 3 ore.

Compararea performanței totale

Timpul măsurat și estimat pentru execuția a trei programe

	Timp pe mașina X	Timp pe mașina Y	Speedup Y față de X	**Speedup X față de Y**
Program A	20	200	0.1	10
Program B	1000	100	10.0	0.1
Program C	1500	150	10.0	0.1
	Medie aritmetică		6.7	3.4
	Medie geometrică		2.15	0.46

Media geometrică nu produce o metrică eficientă a speedup-ului total, dar este un indicator că lucrurile merg în direcția dorită

Efectele amestecării instrucțiunilor asupra performanței

Luăm un exemplu de două aplicații DC și RS și două mașini M_1 și M_2:

Clasă	Data Comp.	Reactor Sim.	M_1's CPI	M_2's CPI
A: Ld/Str	25%	32%	4.0	3.8
B: Integer	32%	17%	1.5	2.5
C: Sh/Logic	16%	2%	1.2	1.2
D: Float	0%	34%	6.0	2.6
E: Branch	19%	9%	2.5	2.2
F: Other	8%	6%	2.0	2.3

a. Aflați CPI efectiv pt. cele două aplicații pe ambele mașini.

Soluție

a.CPI al DC pe M_1: $0.25 \times 4.0 + 0.32 \times 1.5 + 0.16 \times 1.2 + 0 \times 6.0 +$
$0.19 \times 2.5 + 0.08 \times 2.0 = 2.31$

DC pe M_2: 2.54 RS pe M_1: 3.94 RS pe M_2: 2.89

În căutarea performanței mărite

Embedded Systems Laboratory

Puterea de calcul disponibilă la începutul anilor 2000:

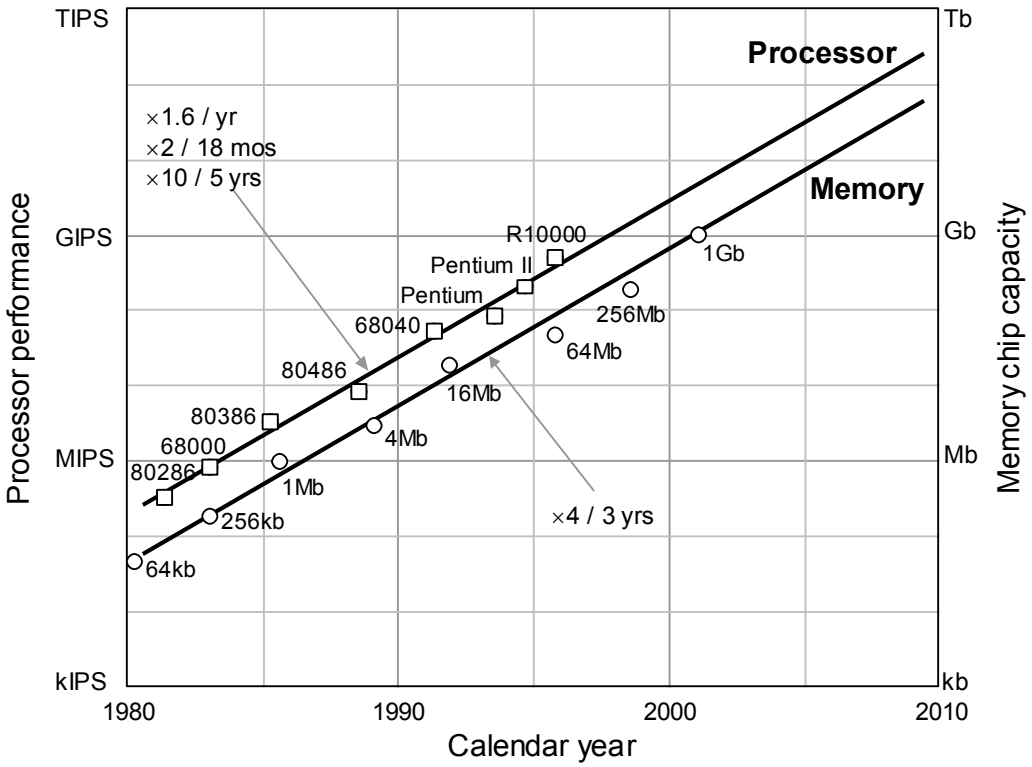
Gigaflops pe un calculator desktop
Teraflops pe un supecomputer
Petaflops în proiectare

Prefixuri pentru numere mari:
Kilo = 10^3, Mega = 10^6, Giga = 10^9, Tera = 10^{12}, Peta = 10^{15}

Pentru memorie:
K = 2^{10} = 1024, M = 2^{20}, G = 2^{30}, T = 2^{40}, P = 2^{50}

Prefixuri pentru numere mici:
micro = 10^{-6}, nano = 10^{-9}, pico = 10^{-12}, femto = 10^{-15}

Trenduri în performanță și învechire

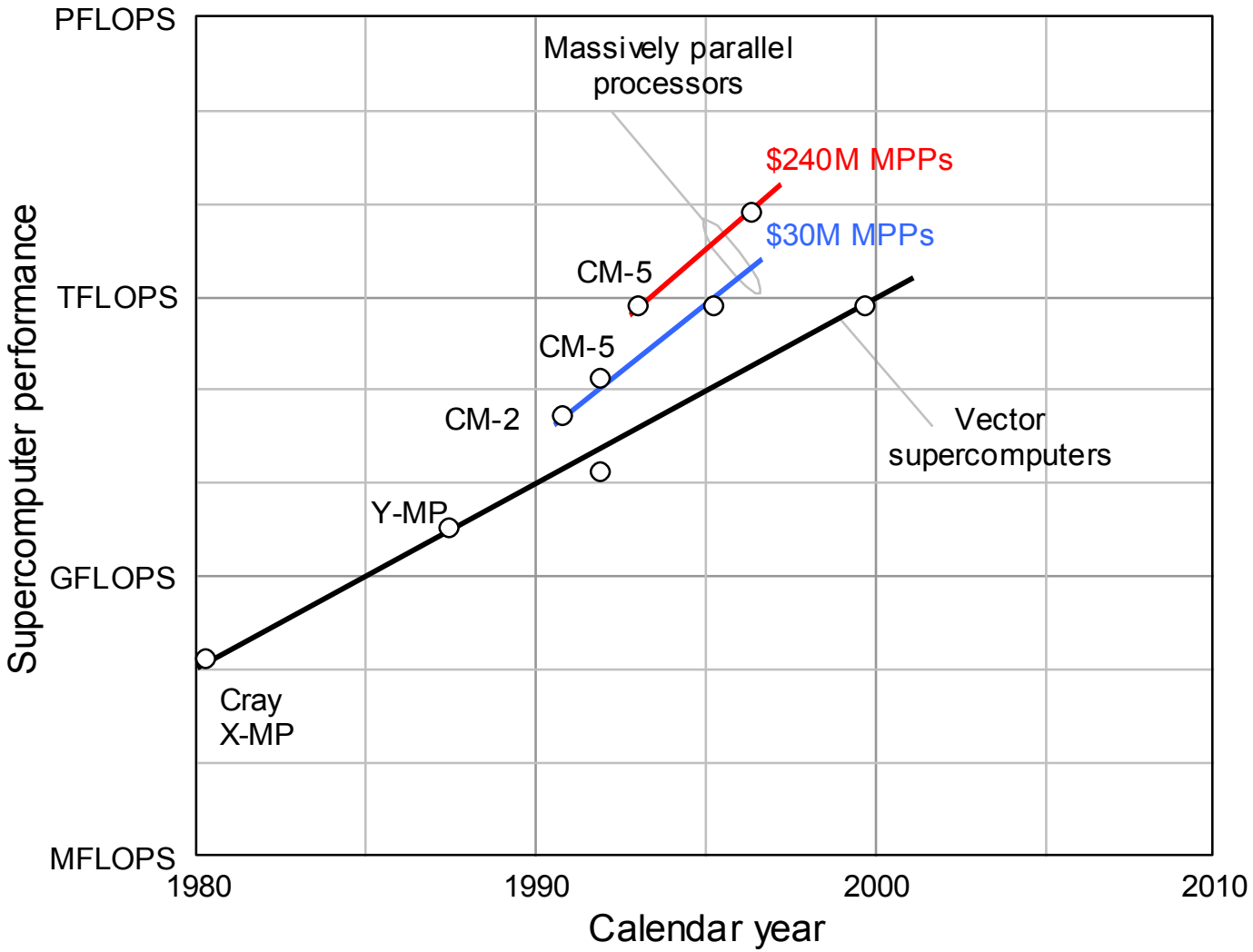


Legea lui Moore pentru procesoare și DRAM

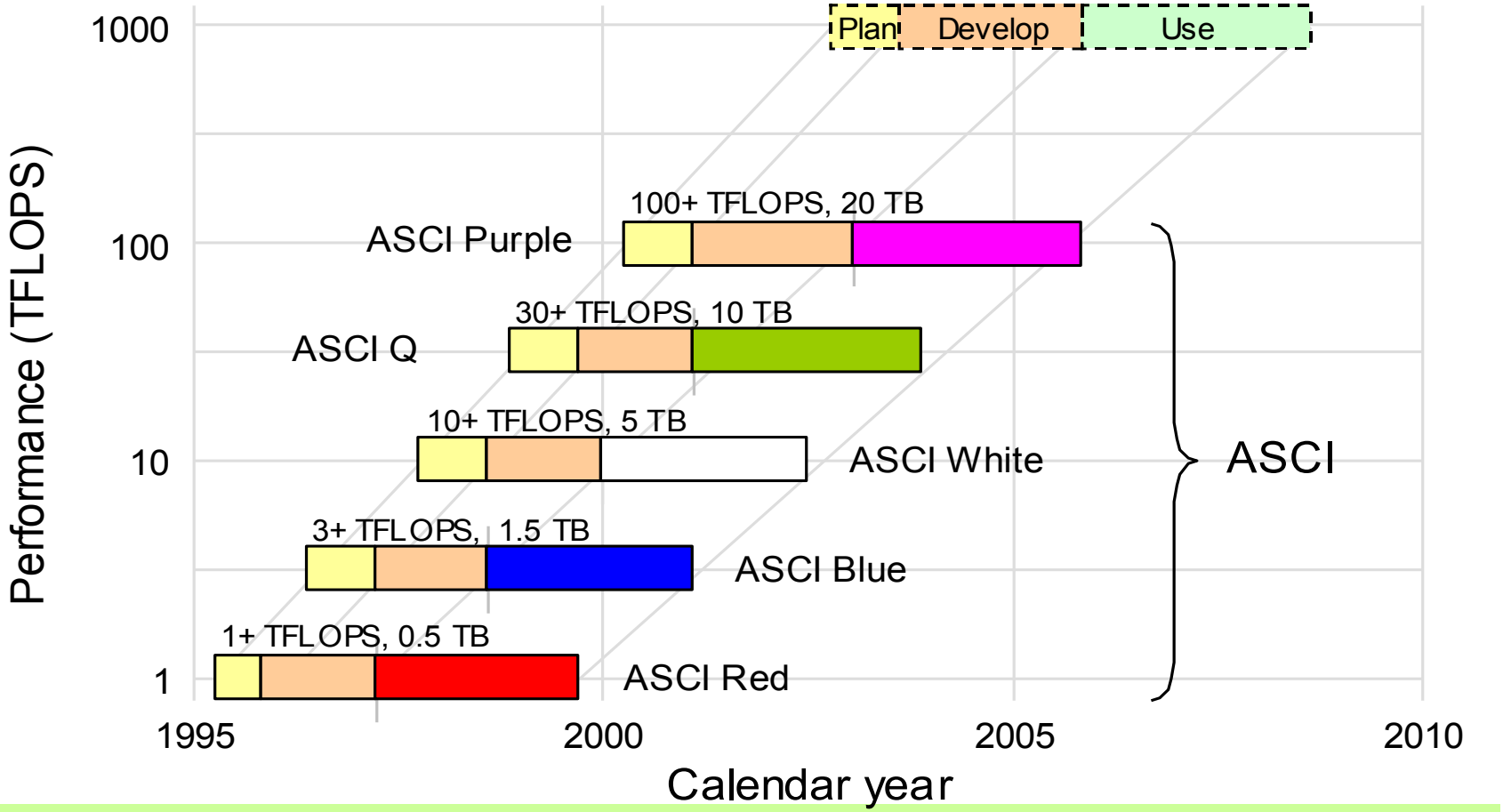
"Can I call you back? We just bought a new computer and we're trying to set it up before it's obsolete."

Super-computere

Creșterea exponențială a puterii de calcul pentru supercalculatoare

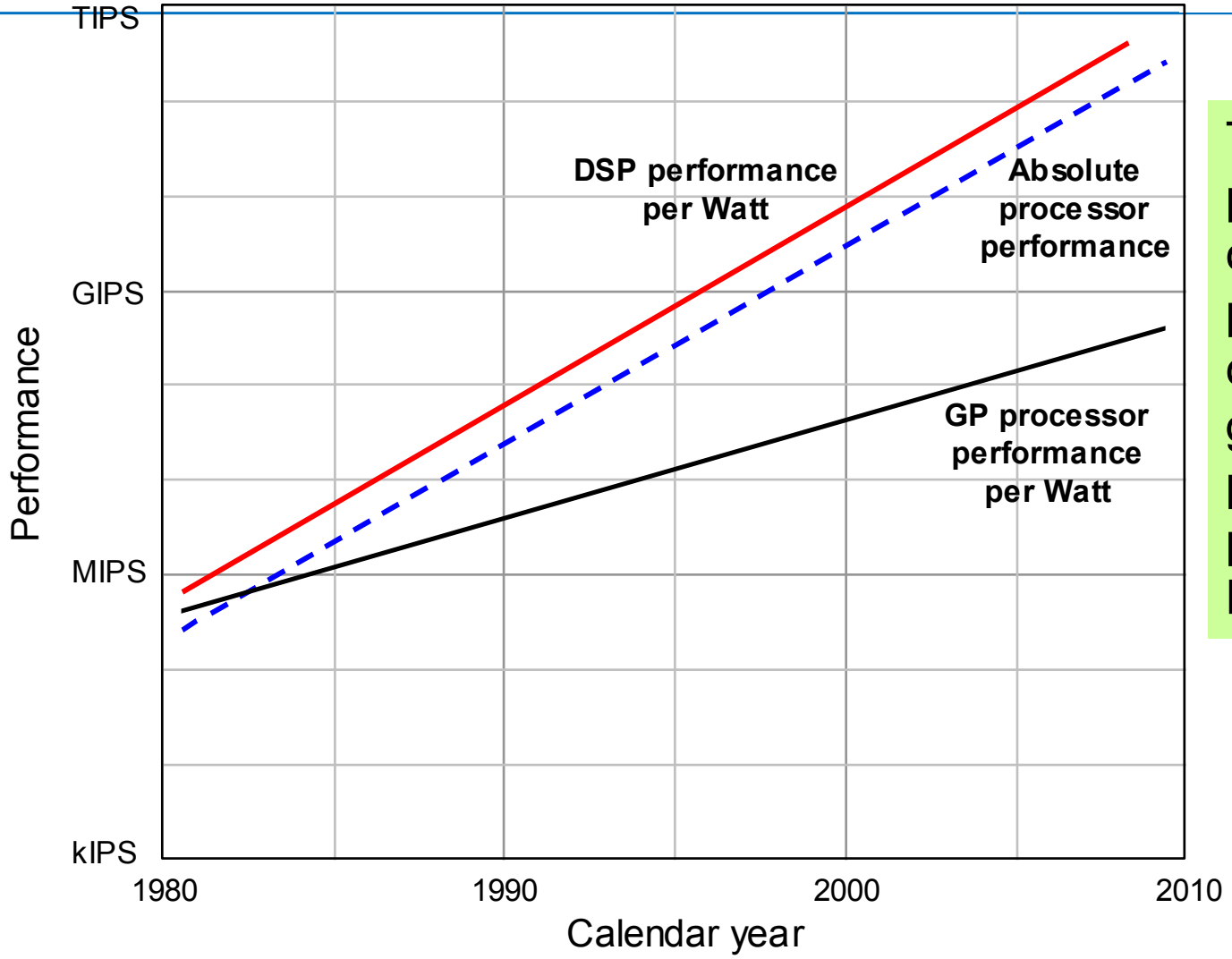


Cele mai puternice calculatoare



Puncte de cotitură pentru DOE Accelerated Strategic Computing Initiative (ASCI) cu extrapolări până la nivelul PFLOPS.

Performanța e importantă, dar nu înseamnă totul!



Tendința în putere de calcul/Watt pentru calculatoare general-purpose și procesoare DSP.

Acknowledgements

- Aceste slide-uri conțin materiale aparținând:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)
  - Behrooz Parhami (UCSB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

Calculatoare Numerice

– Cursul 10 –

De la CISC la RISC

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Comic of the day

http://dilbert.com/strips/comic/2010-05-20/

Din cursurile anterioare

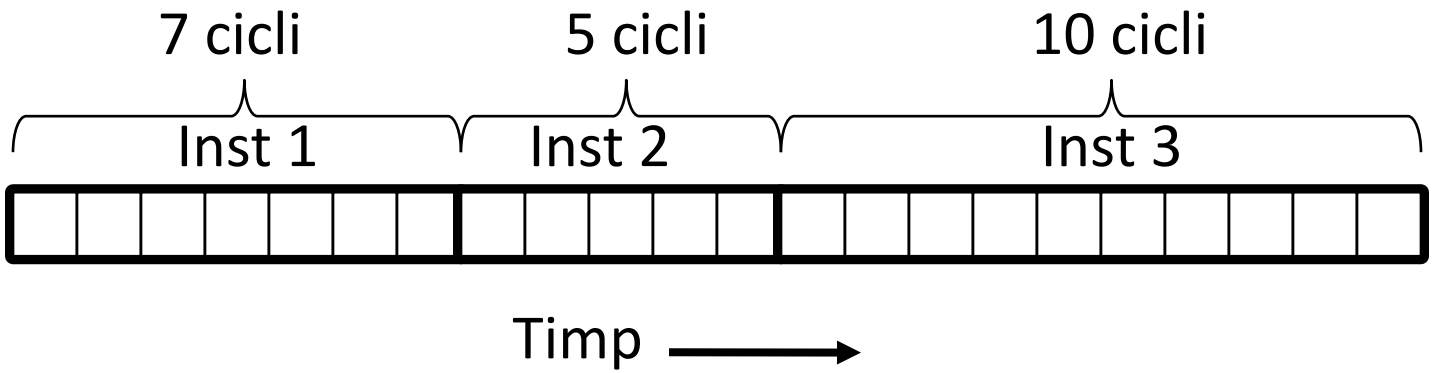
- ISA este interfața hardware/software
  - Definește cum este văzut procesorul de către programator
  - Definește formatul instrucțiunilor (bit encoding) și semnatica lor
  - Exemple: IBM 360, MIPS, RISC-V, x86, JVM
- Mai multe implementări posibile pentru aceeași ISA
  - IBM360: model 30 (c. 1964), z12 (c. 2012)
  - X86: 8086 (c. 1978), 80186, 286, 386, 486, Pentium, Pentium Pro, Pentium-4 (c. 2000), Core 2 Duo, Nehalem, Sandy Bridge, Ivy Bridge, Atom, AMD Athlon, Transmeta Crusoe, SoftPC
  - MIPS: R2000, R4000, R10000, R18K, …
  - JVM: HotSpot, PicoJava, ARM Jazelle, …
- Microcodificare: metodă directă de a implementa mașini de calcul cu puține porți logice și instrucțiuni complexe

“Legea de fier” a performanței calculatoarelor

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} * \frac{\text{Cycles}}{\text{Instruction}} * \frac{\text{Time}}{\text{Cycle}}$$

- Numărul de instrucțiuni per program depinde de codul sursă, compilator și ISA
- Cycles per instructions (CPI) depinde de ISA și de μarchitectură
- Timpul per ciclu depinde de μarchitectură și de tehnologia în care procesorul este construit.

CPI pentru procesoare cu microcod



Numărul total de cicli de ceas = 7+5+10 = 22

Numărul total de instrucțiuni = 3

CPI = 22/3 = 7.33

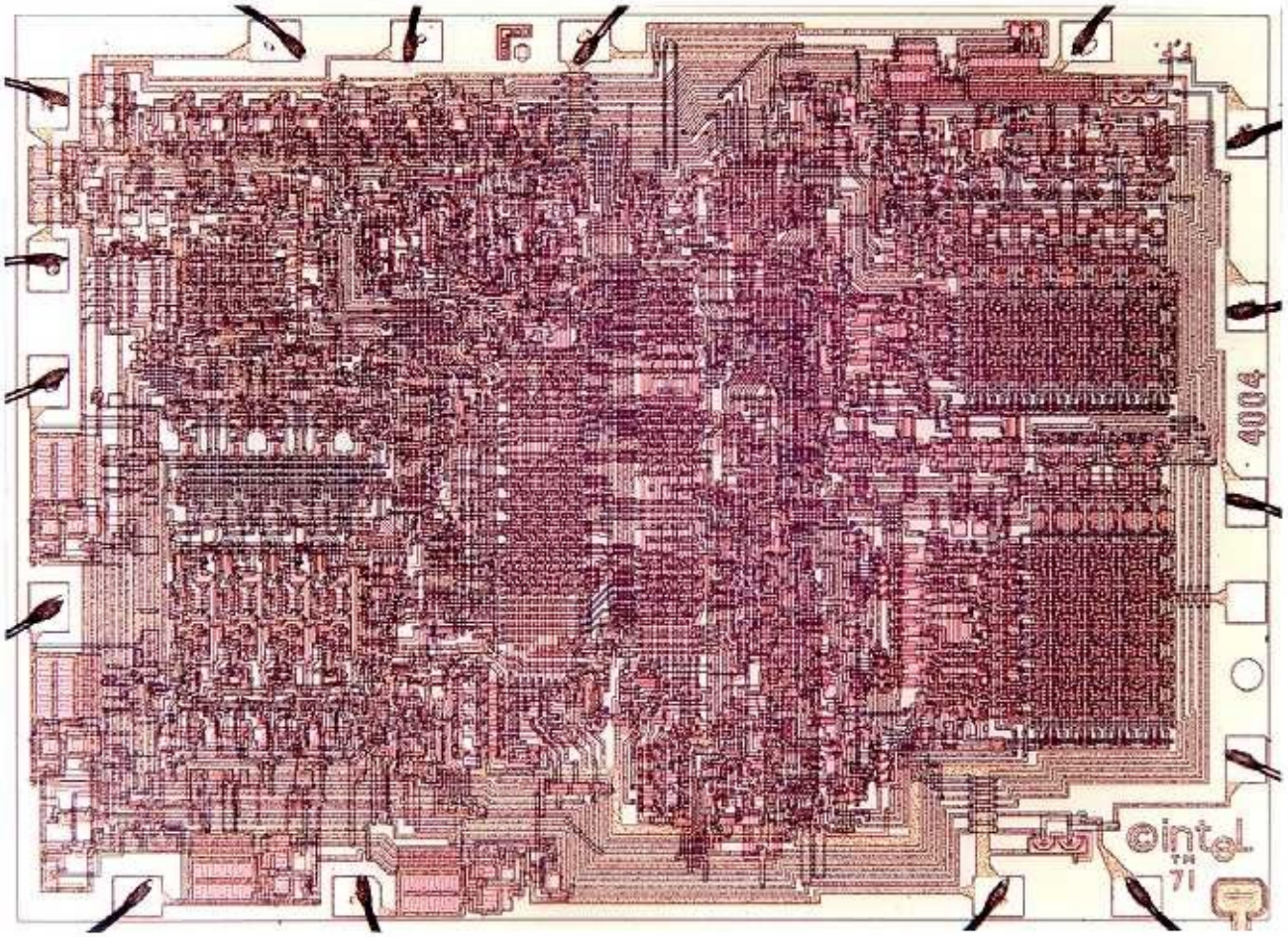
CPI este întotdeauna o valoare medie peste un număr foarte mare de instrucțiuni.

Influențe tehnologice

- Când microprogramarea a apărut pentru prima dată în anii 50, erau tehnologii diferite pentru:
  - Logică: Tuburi cu vid
  - Memorie principală: Miez de ferită
  - Memorie Read-Only: matrice diode, cartele perforate,…
- Logica era foarte scumpă, comparativ cu ROM sau RAM
- ROM mai ieftin ca RAM
- ROM mult mai rapid ca RAM

Anii 70 au adus un avans important în tehnologia de fabricație a circuitelor integrate și a memoriilor semiconductoare...

Primul microprocesor Intel 4004, 1971



- Arhitectură 4-biți cu acumulator
- 8µm pMOS
- 2,300 tranzistoare
- 3 x 4 mm2
- Ceas 750kHz
- 8-16 cicli/inst.

Posibil datorită noii tehnologii de fabricație a circuitelor integrate

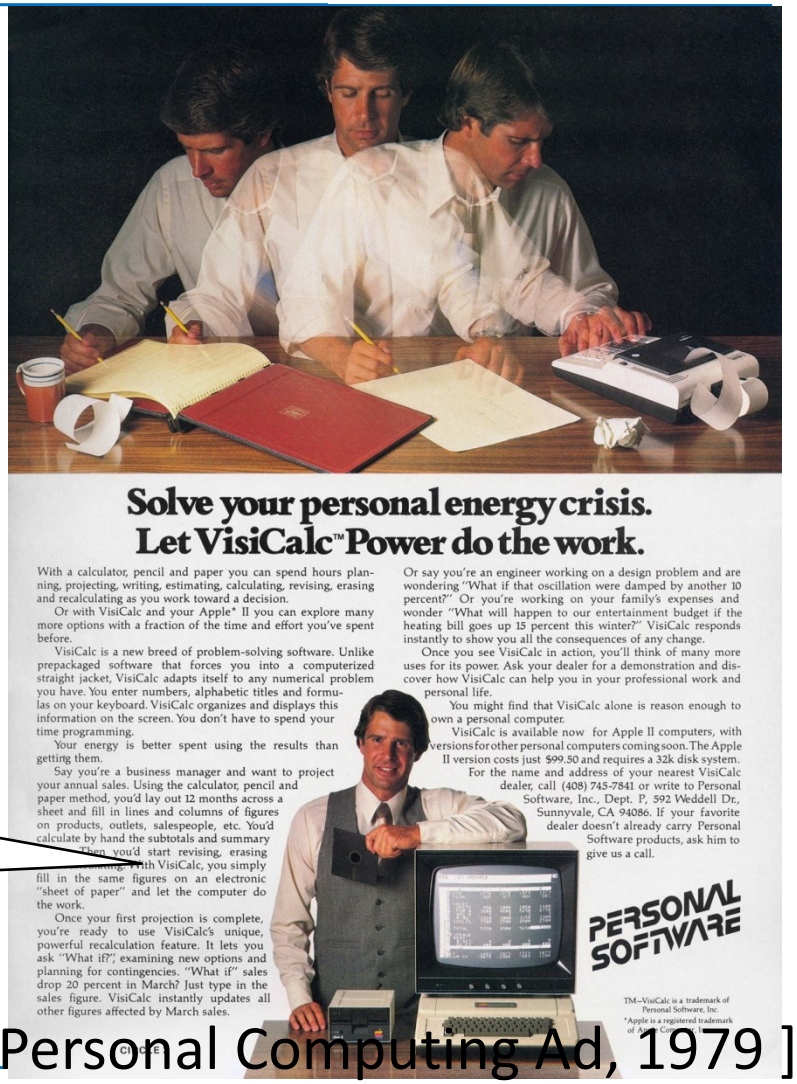
Microprocesoarele în anii 70

- Ținta inițială era controlul disp. embedded
  - Primul micro pe 4 biți, 4004 de la Intel, proiectat pentru un calculator matematic de birou
  - Constrâns de ce se putea integra într-un singur chip
  - Arhitectură cu acumulator, similară cu primele calculatoare
  - Logică cablată de control
- Micro pe 8 biți (8085, 6800, 6502) folosite în PC-uri pentru consumer market
  - Micral, Altair, TRS-80, Apple-II
  - De obicei spațiu de adresă de 16 biți (puteau să adreseze direct maxim 64KB)
  - De obicei veneau cu interpretor BASIC preîncărcat în ROM sau de pe casetă audio.

VisiCalc – primul "killer" app pentru microprocesoare

- Microprocesoarele au avut un impact mic în piața de calculatoare convenționale până la apariția VisiCalc spreadsheet pentru Apple-II
- Apple-II folosea un procesor Mostek 6502 la 1MHz

Discurile floppy au fost inventate de IBM și erau folosite inițial ca un mod de a trimite clienților patch-uri pentru microcodul IBM360!



[Personal Computing Ad, 1979]

Memoria DRAM în anii 70

- Progres dramatic pentru tehnologia de fabricație a memoriilor semiconductoare

- 1970, Intel produce primul DRAM, 1Kbit 1103

- 1979, Fujitsu produce DRAM de 64Kbit

=> La mijlocul anilor șaptezeci era evident că PC-urile vor avea în curând memorii fizice >64KBytes

Evoluția microprocesoarelor

- Progres rapid în anii 70, alimentat de tehnologii noi pentru MOSFET și o piață în continuă creștere
- Intel i432
  - Cel mai ambițios micro al anilor șaptezeci; pornit 1975 - lansat 1981
  - Arhitectură pe 32 de biți orientată obiect
  - Instrucțiuni de lungime variabilă
  - Probleme severe de performanță, complexitate și utilizare
- Motorola 68000 (1979, 8MHz, 68,000 tranzistoare)
  - Puternic microcodat (și nanocodat)
  - Arhitectură pe 32-biți cu registre generale (24 biți adresă)
  - 8 registre adresă, 8 registre de date
- Intel 8086 (1978, 8MHz, 29,000 tranzistoare)
  - Procesor “Stopgap” 16-biți, proiectat în 10 săptămâni
  - Arhitectură cu acumulator, compatibil în assembly cu 8080
  - Adresare segmentată pe 20-biți

IBM PC, 1981

- Hardware
  - Echipa de la IBM ce construia prototipuri de PC-uri în 1979
  - Motorola 68000 ales inițial, dar 68000 nu a fost disponibil la timp
  - IBM construiește prototipuri cu “stopgap” folosind plăci cu 8088 de la procesorul text Display Writer
  - 8088 este versiunea pe 8 biți a lui 8086 => sistem mai ieftin
  - Vânzări estimate inițial la maxim 250,000
  - Vânzări efective: sute de milioane!
- Software
  - Microsoft negociază să producă un OS pentru IBM. Mai târziu cumpără și modifică QDOS de la Seattle Computer Products.
- Sistem deschis
  - Procesor standard, Intel 8088
  - Interfețe standard
  - OS standard, MS-DOS
  - IBM permite clonarea sistemelor lor și existența de software third-party

Presenting the IBM® of Personal Computers.

IBM is proud to announce a product *you* may have a personal interest in. It's a tool that could soon be on your desk, in your home or in your child's schoolroom. It can make a surprising difference in the way you work, learn or otherwise approach the complexities (and some of the simple pleasures) of living.

It's the computer we're making for you.

In the past 30 years, the computer has become faster, smaller, less complicated and less expensive. And IBM has contributed heavily to that evolution.

Today, we've applied what we know to a new product we believe in: the IBM Personal Computer.

It's a computer that has reached a truly personal scale in size and in price: starting at less than $1,600† for a system that, with the addition of one simple device, hooks up to your home TV and uses your audio cassette recorder.

For flexibility, performance and ease of use, no other personal computer offers as many advanced features to please novice and expert alike (see the box).

Features like high resolution color graphics. Ten, user-defined function keys. The kind of expandability that lets you add a printer for word processing, or user memory up to 256KB. Or BASIC and Pascal languages that let you write your own programs. And a growing list of superior programs like VisiCalc,™ selected by IBM to match the quality and thoughtfulness of the system's total design.

This new system will be sold through channels which meet our professional criteria: the nationwide chain of 150 ComputerLand® stores, and Sears Business Systems Centers. Of course, our own IBM Product Centers will sell and service the system. And the IBM Data Processing Division will serve those customers who want to purchase in quantity.

Experience the IBM Personal Computer. You'll be surprised how quickly you feel comfortable with it. And impressed with what it can do for you.

IBM®

IBM PERSONAL COMPUTER SPECIFICATIONS
*ADVANCED FEATURES FOR PERSONAL COMPUTERS

User Memory
16K - 256K bytes*
Permanent Memory
(ROM) 40K bytes*
Microprocessor
High speed, 8088*
Auxiliary Memory
2 optional internal diskette drives, 5¼", 160K bytes per diskette
Keyboard
83 keys, 6 ft. cord attaches to system unit*
10 function keys*
10-key numeric pad
Tactile feedback*

Display Screen
High-resolution (720h x 350v)*
80 characters x 25 lines
Upper and lower case
Green phosphor screen*
Diagnostics
Power-on self testing*
Parity checking
Languages
BASIC, Pascal
Printer
Bidirectional*
80 characters/second
12 character styles, up to 132 characters/line*
9 x 9 character matrix*

Color/Graphics
Text mode:
16 colors*
256 characters and symbols in ROM*
Graphics mode:
4-color resolution: 320h x 200v*
Black & white resolution: 640h x 200v*
Simultaneous graphics & text capability*
Communications
RS-232-C interface
Asynchronous (start/stop) protocol
Up to 9600 bits per second

For the IBM Personal Computer dealer nearest you, call (800) 447-4700. In Illinois, (800) 322-4400.

†This price applies to IBM Product Centers. Prices may vary at other stores.
VisiCalc is a trademark of Personal Software, Inc.

CIRCLE 3

[Personal Computing Ad, 11/81]

Microprogramarea: anii optzeci

- Evoluția a născut mașini de calcul și mai complexe
  - Seturile complexe de instrucțiuni au dus la necesitatea de-a implementa subrutine și lucru cu stiva direct în µcod
  - Necesitatea de-a repara defectele din programele de control era în conflict cu natura read-only a µROM-ului
  - →Writable Control Store (WCS) (B1700, QMachine, Intel i432, …)
- Odată cu apariția tehnologiei VLSI limitările de viteză și mărime pentru ROM & RAM devin invalide →complexitate sporită
- Compilatoarele mai bune făceau ca instrucțiunile complexe să fie mai puțin importante.
- Numeroase inovații la nivel de micro-arhitectură, e.g., pipelining, caches & buffers, au făcut ca operațiile de transfer între registre pe mai mulți cicli de ceas să fie perimate

Analiza mașinilor cu microcod

- John Cocke (& grupul lui) IBM
  - Lucrau pe un procesor cu pipeline simplu, 801 cu compilatoare avansate la IBM
  - Au portat compilatorul experimental PL.8 pe IBM 370; foloseau DOAR instrucțiuni simple de transfer reg-reg și load/store similare 801
  - Codul produs rula mai repede decât codul produs de orice alt compilator existent pe seria 370 care folosea TOATE instrucțiunile!(până la 6MIPS în loc de 2MIPS cu compilatoarele standard)
- Emer, Clark, at DEC
  - Măsura performanța VAX-11/780 folosind hardware extern
  - A descoperit ca era defapt o mașină de calcul de 0.5MIPS, deși se presupunea de obicei că e de 1MIPS
  - 20% din instrucțiunile VAX erau responsabile pentru 60% din microcod, și doar 0.2% din timpul de execuție!
- VAX8800
  - Control Store: 16K*147b RAM, Unified Cache: 64K*8b RAM
  - 4.5x mai mult RAM pentru microcod decât pentru cache RAM!

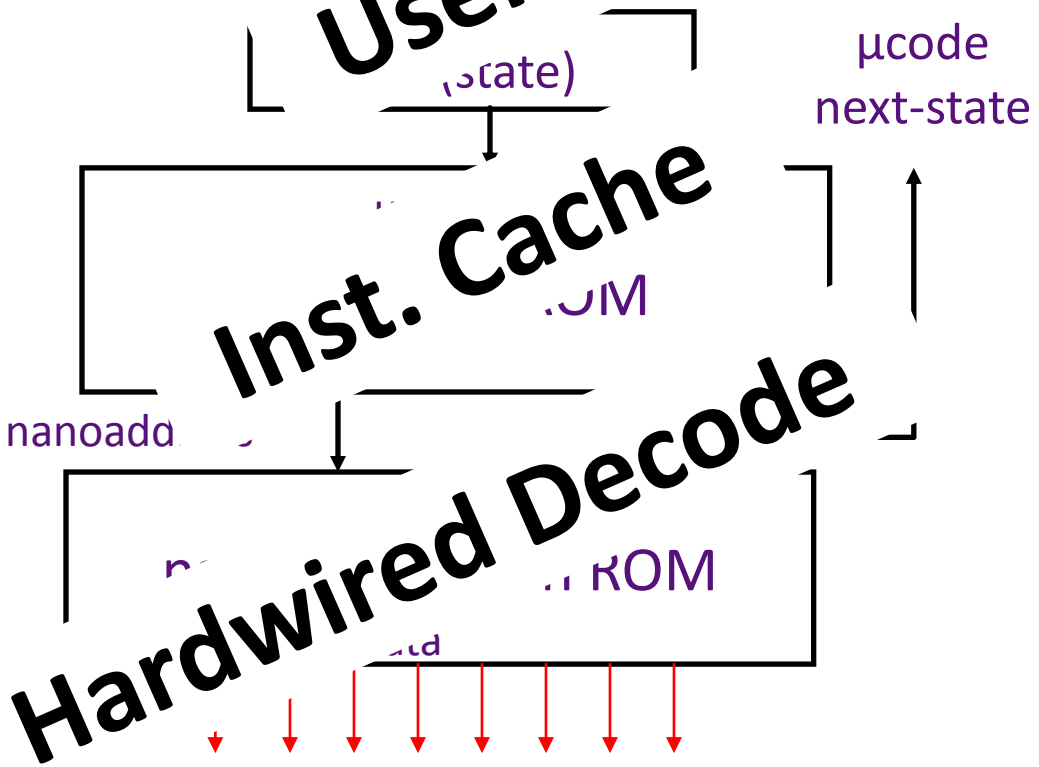
Schimbările tehnologice pentru IC-uri

- Logică, RAM, ROM, toate implementate cu tranzistoare MOS
- RAM semiconductor ~ aceeași viteză ca ROM

Nanocoding

Explo[illegible] nțele
recure[illegible] semnale de
comanda din µcod, e.g.,

ALU_0 A <- Reg[rs1]
...
$ALUi_0$ A <- Reg[rs1]
...



- MC68000 avea un µcod [illegible] 17 biți ce conținea ori un µjump pe 10 biți, ori un pointer de 9-biți la o nanoinstrucțiune
  - Nanoinstrucțiunile aveau 68 biți lățime și decodate dădeau 196 de semnale de control

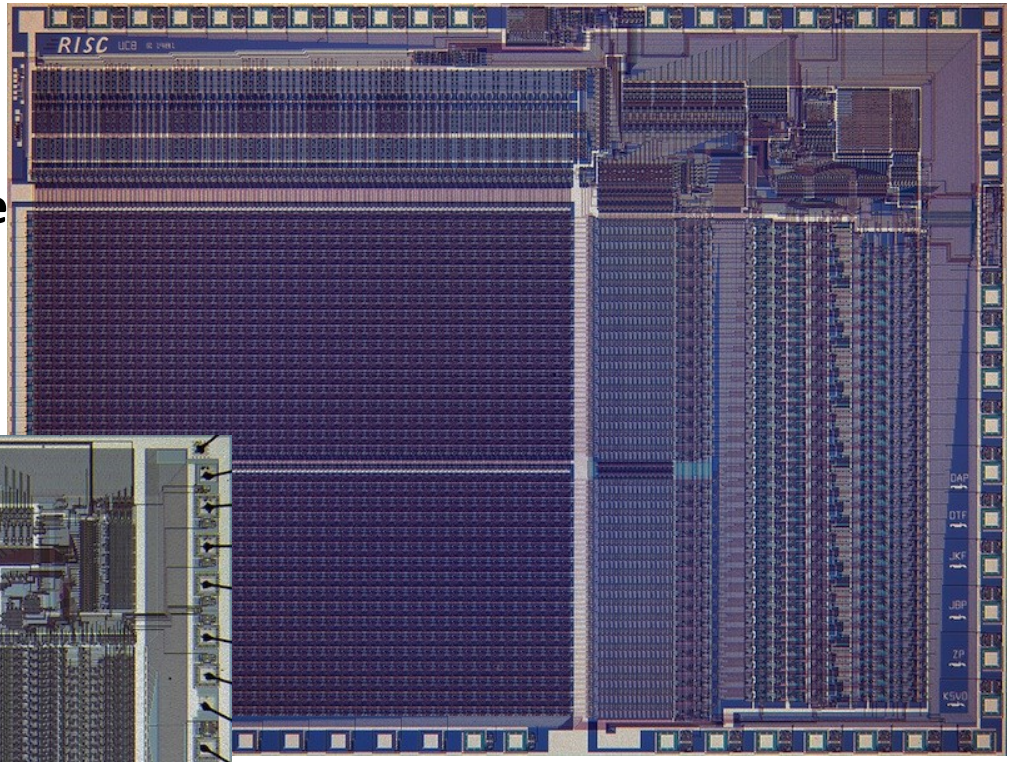
De la CISC la RISC

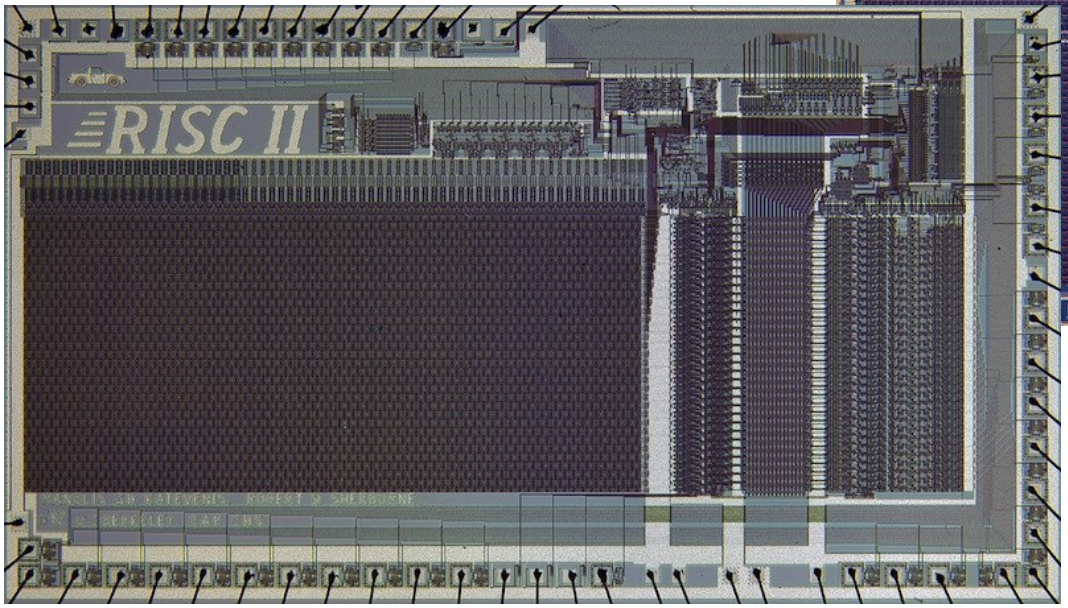
- Folosește RAM rapid pentru a construi un instruction *cache* rapid pentru instrucțiuni vizibile utilizatorului, nu microrutine fixate in hardware
  - Conținutul memoriei rapide se schimbă ca să acomodeze noile nevoi ale aplicațiilor
- Folosește un ISA simplu pentru a permite o implementare cablată de bandă de asamblare
  - Majoritatea codului compilat folosea doar o mică parte din instrucțiunile CISC disponibile
  - Codificarea simpificată permitea implementarea cu pipeline
- Beneficii suplimentare
  - La începutul amilor ‘80, se puteau în sfârșit integra o magistrală de date de 32 de biți și memorii cache de mici dimensiuni într-un singur chip

Chip-uri RISC Berkeley

Embedded Systems Laboratory

RISC-I (1982) Conține 44,420 tranzisotare, fabricat în 5 µm NMOS, suprafață 77 mm², ceas de 1 MHz. Probabil primul RISC VLSI.





RISC-II (1983) conține 40,760 tranzistoare, fabricat în 3 µm NMOS, ceas de 3 MHz, și suprafața de 60 mm².

"Iron Law" of Processor Performance

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} * \frac{\text{Cycles}}{\text{Instruction}} * \frac{\text{Time}}{\text{Cycle}}$$

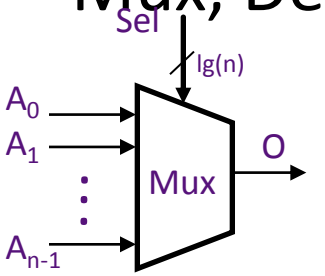
- Instructions per program depends on source code, compiler technology, and ISA
- Cycles per instructions (CPI) depends on ISA and µarchitecture
- Time per cycle depends upon the µarchitecture and base technology

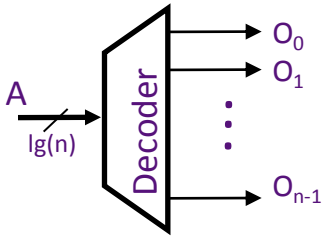
Acest curs →

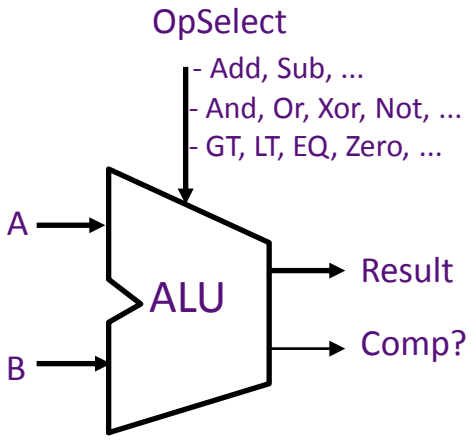
Microarchitectură	CPI	cycle time
Microcodată	>1	short
Single-cycle unpipelined	1	long
Pipelined	1	short

Elemente hardware

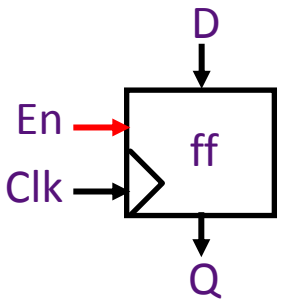
- Circuite combinaționale
  - Mux, Decoder, ALU, ...

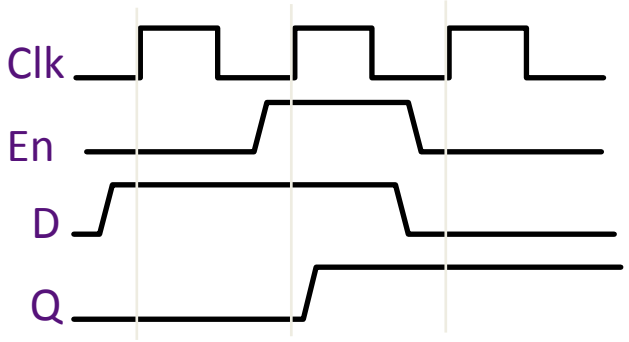






- Elemente sincrone
  - Flipflop, Register, Register file, SRAM, DRAM

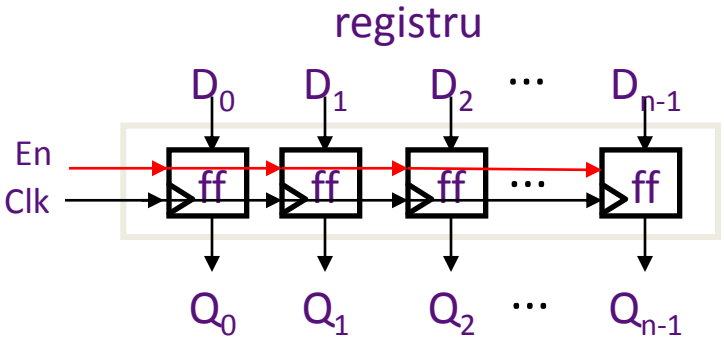


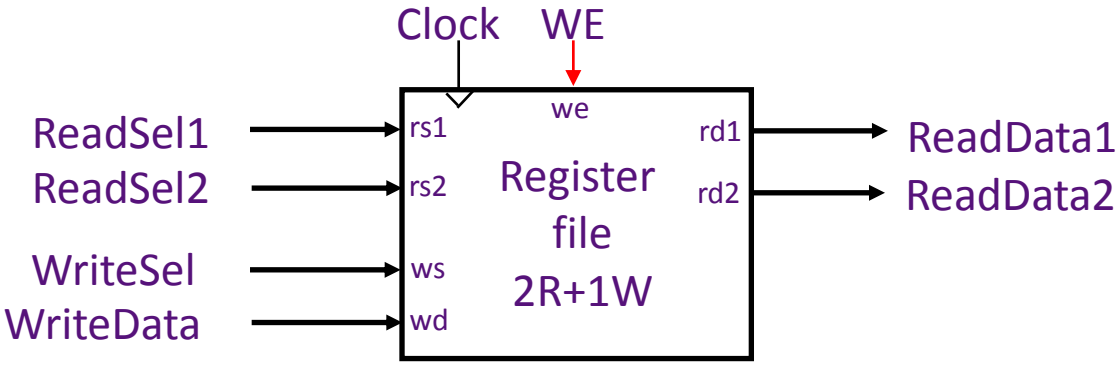


Edge-triggered: Datele eșantionate pe front crescător

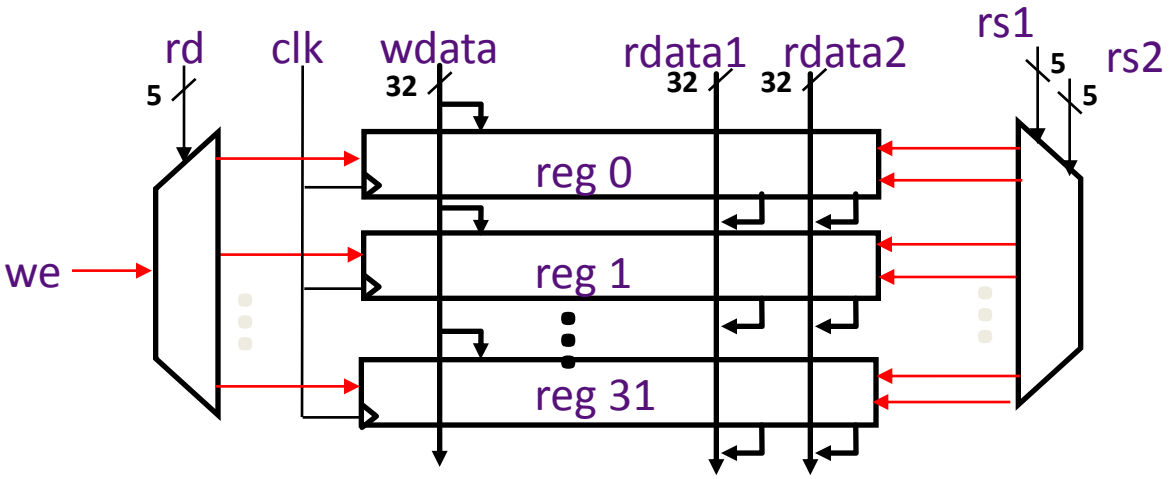
Tabele de registre

- Citirile se fac combinațional



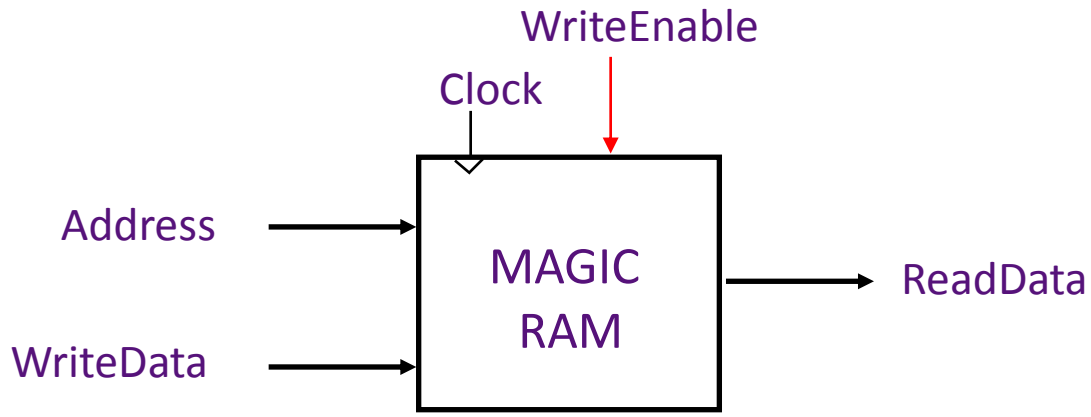


Implementarea unei tabele de registre



- Instrucțiunile cu întregi pentru RISC-V au cel mult doi operanzi sursă în registrele generale

Un model simplu pentru memorie



Citirile și scrierile se fac întotdeauna într-un singur ciclu

- o citire poate fi făcută oricând (i.e. Combinațional)
- o scriere este făcută pe frontul pozitiv al ceasului

liniile de adresă și de date trebuie să fie stabile pe frontul ceasului

Mai târziu vom prezenta un model mai realist de memorie

Implementarea RISC-V

Single-cycle per instruction datapath & control logic

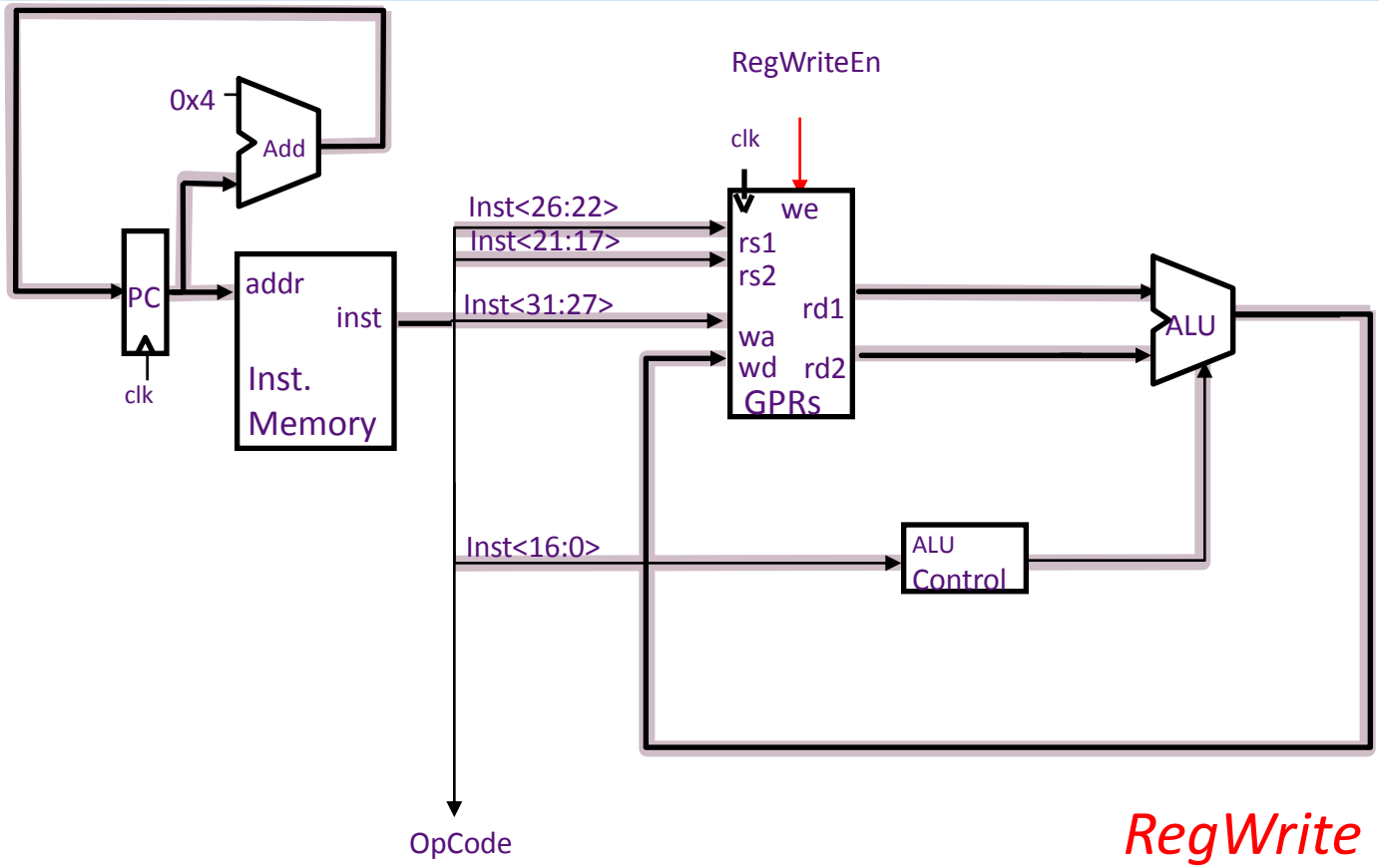
Execuția instrucțiunilor

O execuție implică

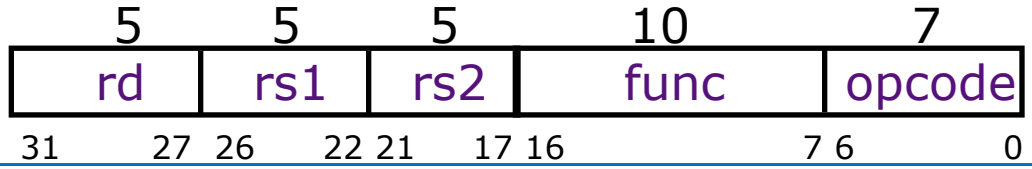
1. Instruction fetch
2. Decode and register fetch
3. Operații UAL
4. Operații cu memoria (opțional)
5. Write back (opțional)

și calculul adresei instrucțiunii următoare

Transfer date: Instrucțiuni Reg-Reg ALU

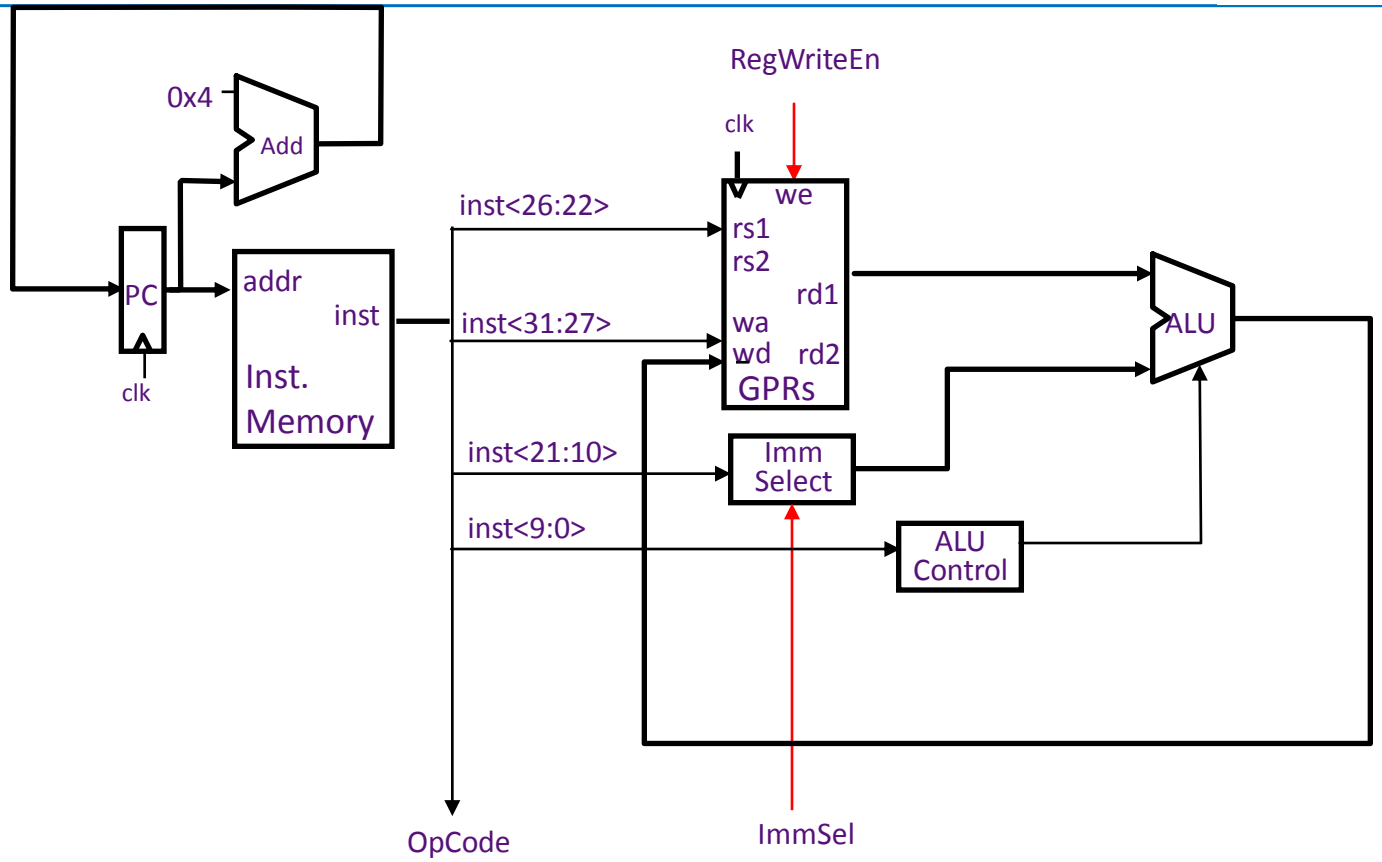


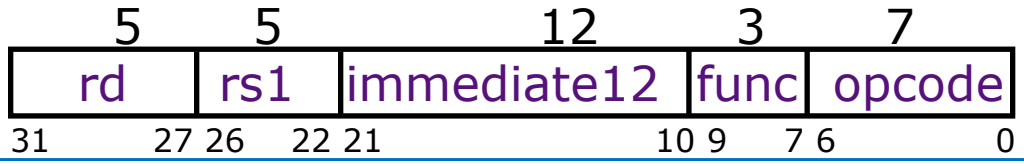
RegWrite Timing?



rd ← (rs1) func (rs2)

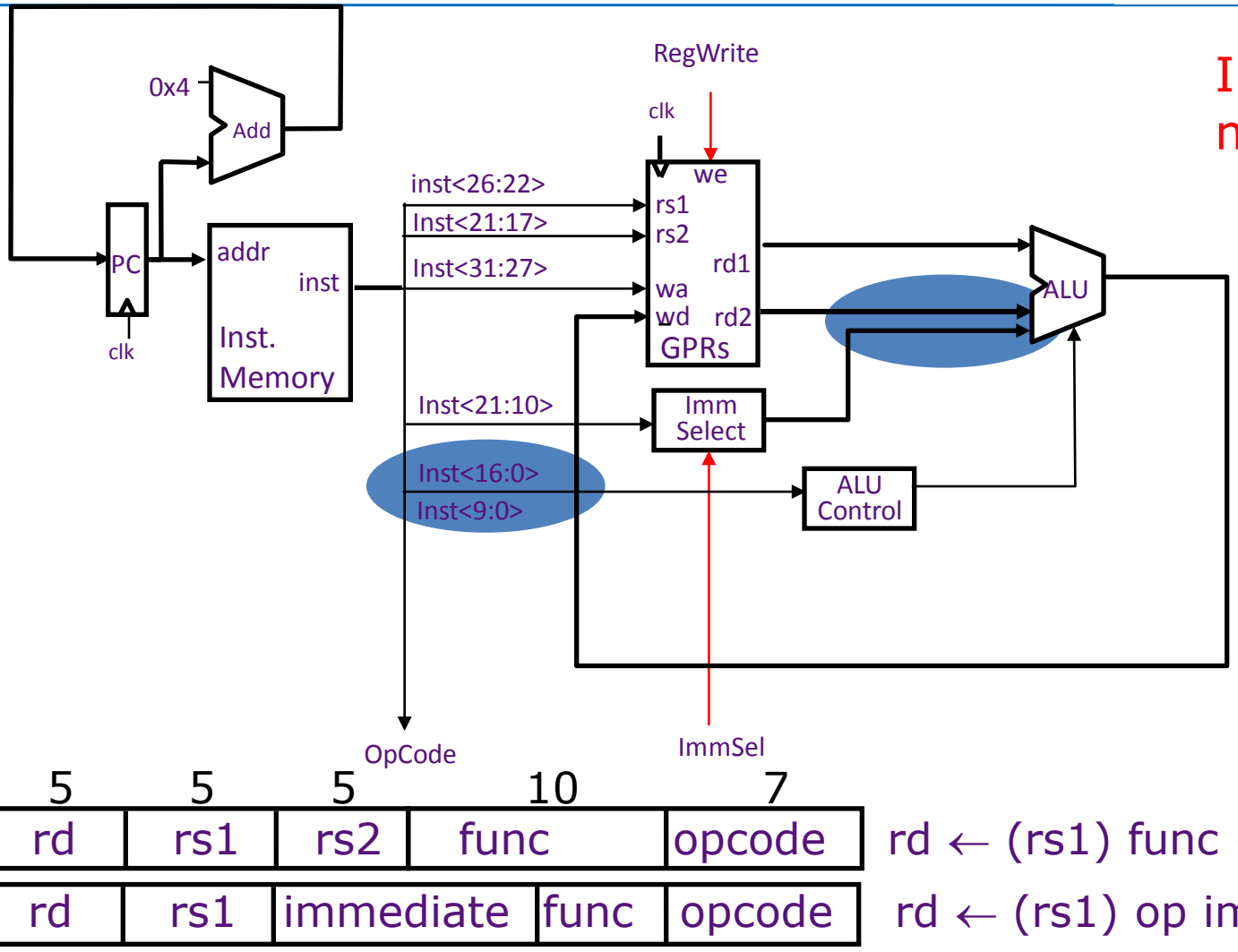
Transfer de date: Instrucțiuni Reg-Imm ALU





rd ← (rs1) op immediate

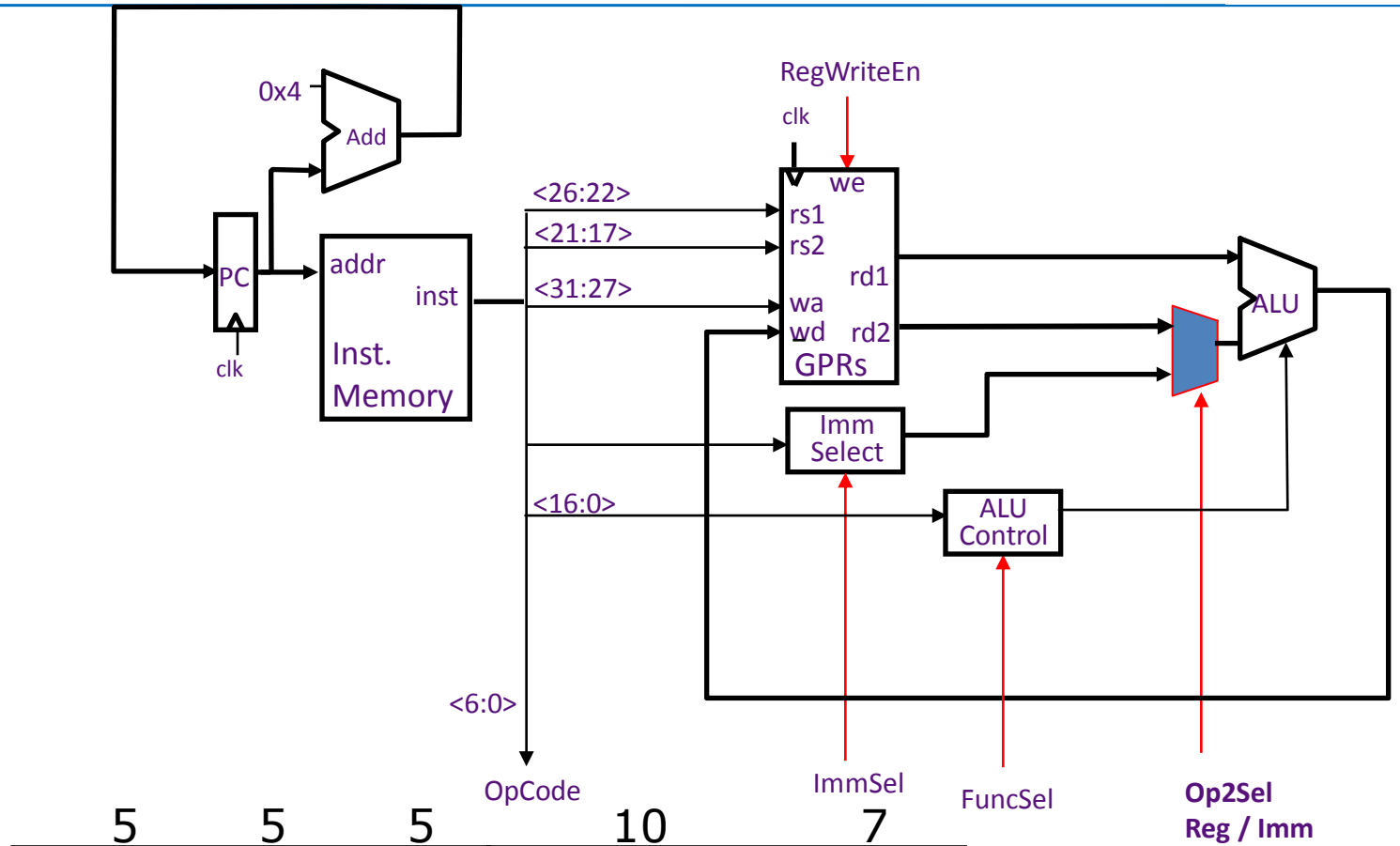
Conflicte în transferul de date



Introducem mux-uri

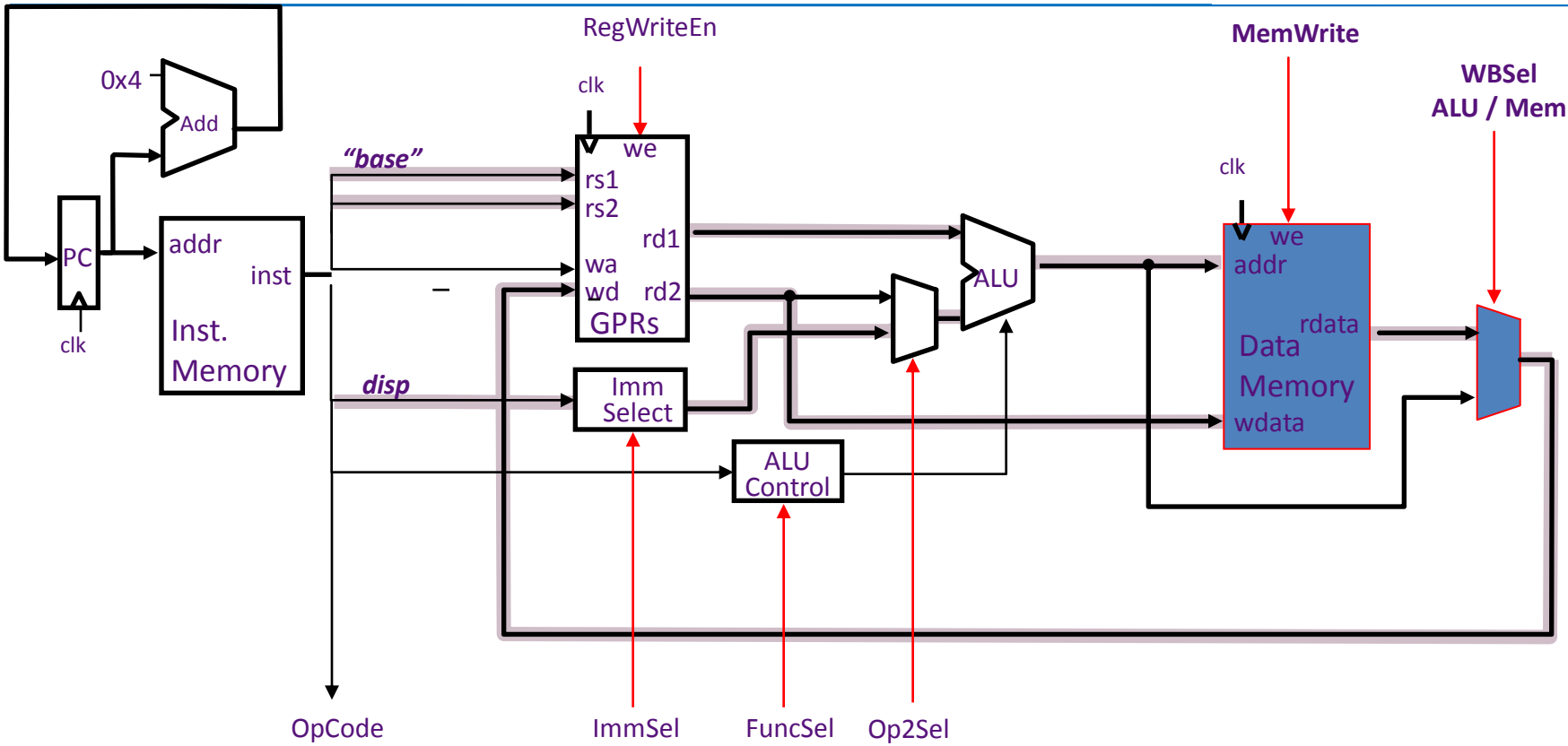
5	5	5	10	7	
rd	rs1	rs2	func	opcode	rd ← (rs1) func (rs2)
rd	rs1	immediate	func	opcode	rd ← (rs1) op immediate

Transfer date pentru instrucțiuni cu UAL



5	5	5	10	7	
rd	rs1	rs2	func	opcode	rd ← (rs1) func (rs2)
rd	rs1	immediate		func opcode	rd ← (rs1) op immediate

Instrucțiuni Load/Store



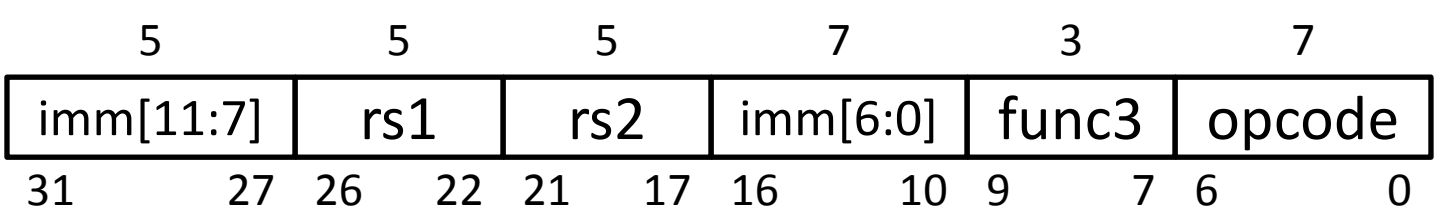
5	5	5	7	3	7	
imm	rs1	rs2	imm	func	opcode	Store
rd	rs1	immediate		func	opcode	Load

Addressing Mode
(rs) + displacement

rs1 este registrul de bază

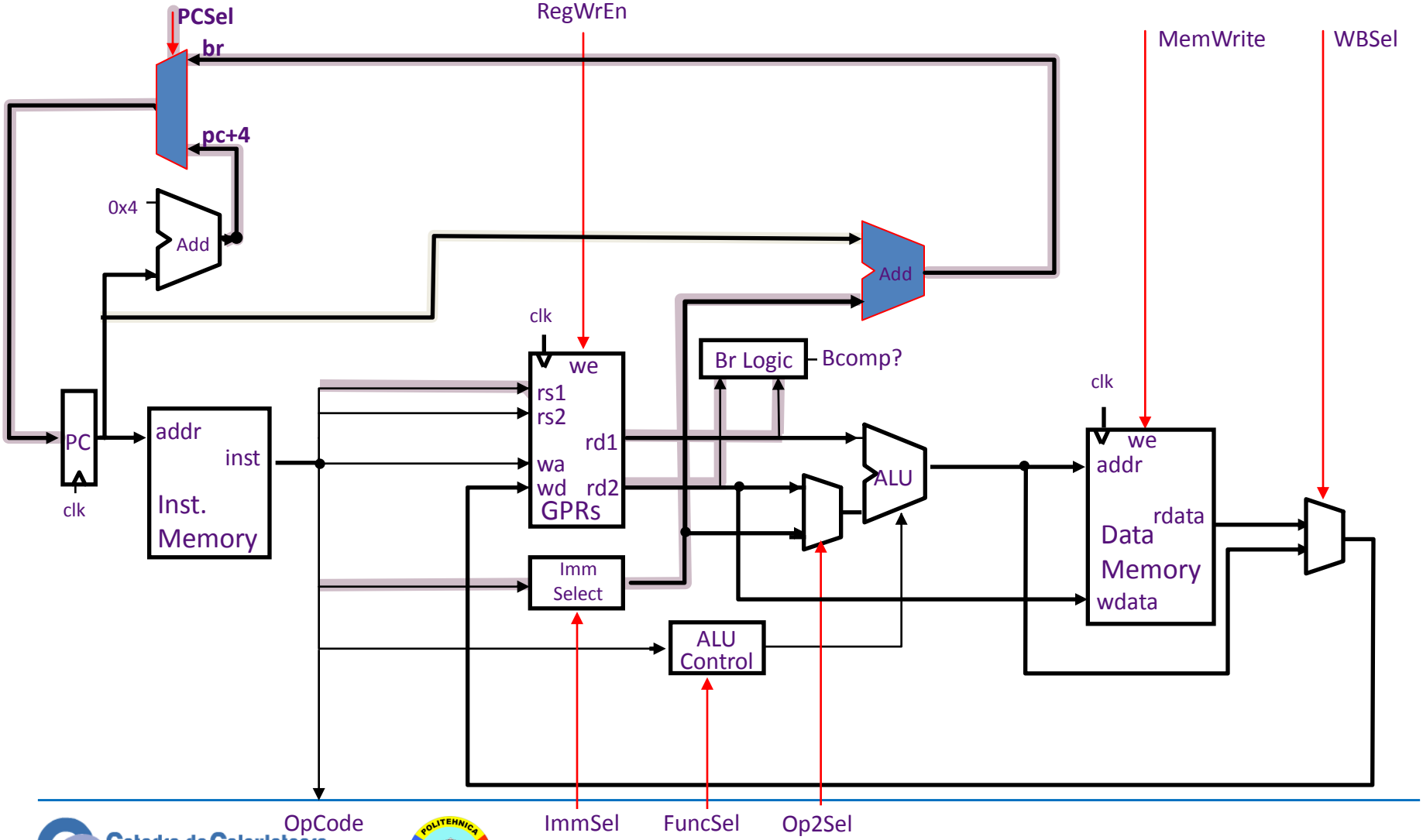
rd este destinația pt Load, rs2 este sursa de date pentru un Store

Salturi condiționale la RISC-V

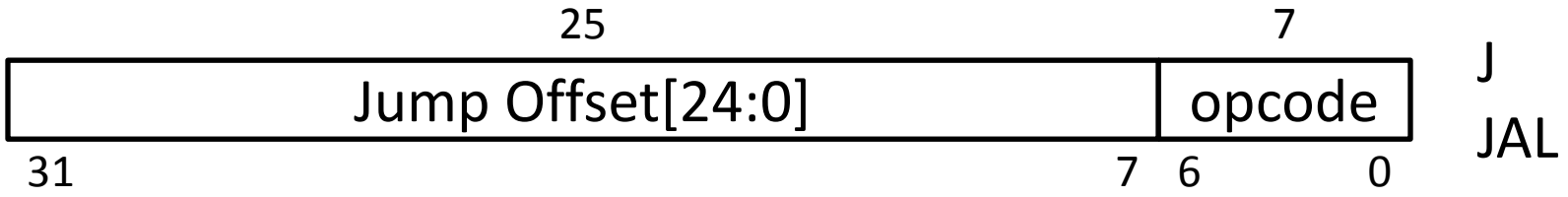


- Compară două registre întregi pentru egalitate (BEQ/BNE) sau mărime cu semn (BLT/BGE) sau fără semn (BLTU/BGEU)
- Valoarea imediată de 12 biți stipulează adresa țintă de salt ca fiind un offset cu semn pentru PC, în unități de 16 biți (i.e., shift left cu 1 apoi adaugă la PC).

Salturi condiționale (BEQ/BNE/BLT/BGE/BLTU/BGEU)

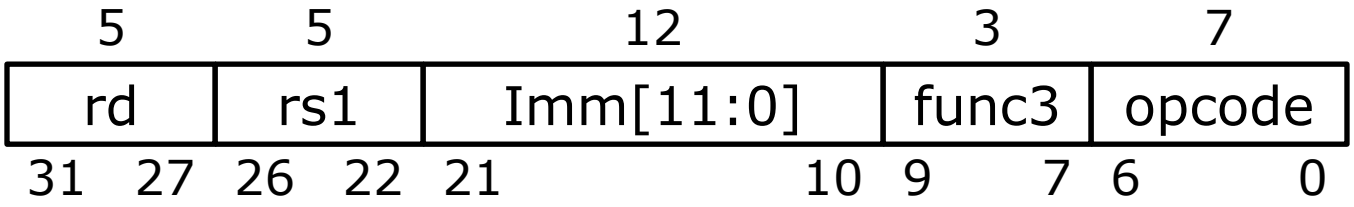


Salturi necondiționale la RISC-V



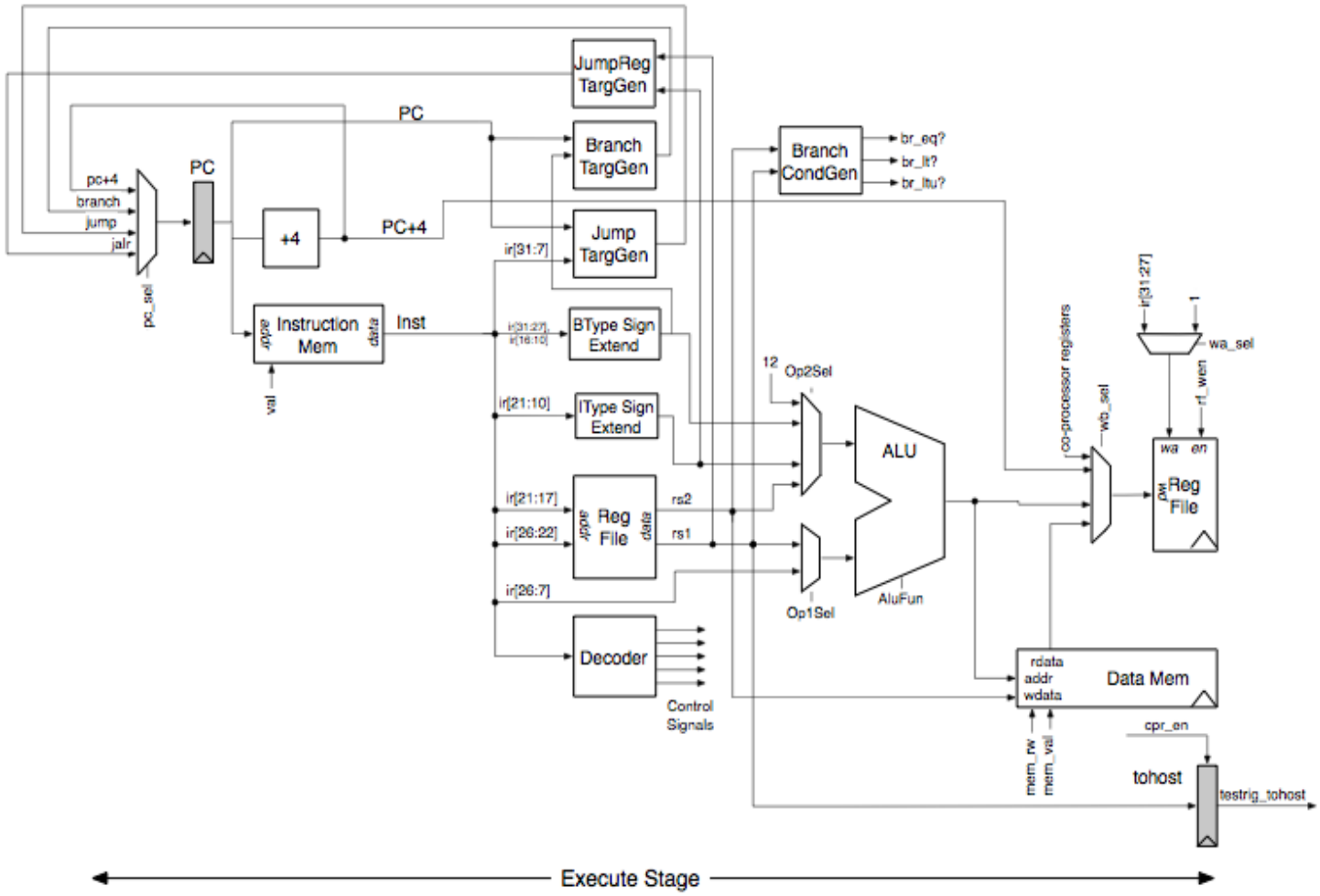
- Valoarea imediată de 25 biți specifică adresa de jump ca fiind un offset cu semn al PC, în unități de 16 biți cu semn (i.e., shift left cu 1 apoi adună la PC). (+/- 16MB)
- JAL este o subrutină care salvează și adresa de întoarcere (PC+4) în registrul x1

Salturi indirecte prin registre la RISC-V

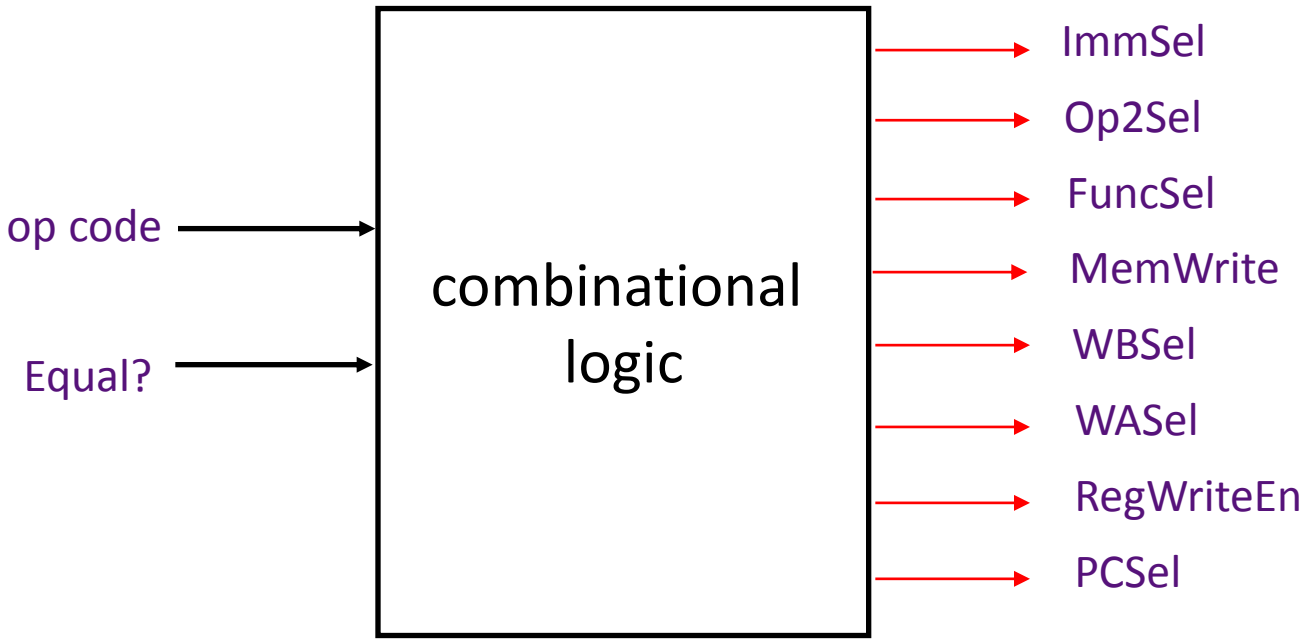


- Salturile la adresa țintă sunt făcute prin adunarea unui offset de 12 biți (*ne-shiftat* cu 1)registrului rs1
- Adresa de întoarcere (PC+4) este scrisă în registrul rd (poate fi **x0** dacă valoarea nu este necesară)
- Instrucțiunea RDNPC scrie adresa de întoarcere în rd fără să facă saltul efectiv (folosită pentru dynamic linking)

Full RISCV1Stage Datapath



Controlul cablat este logică combinațională pură



Controlul UAL și extensia pentru valori imediate

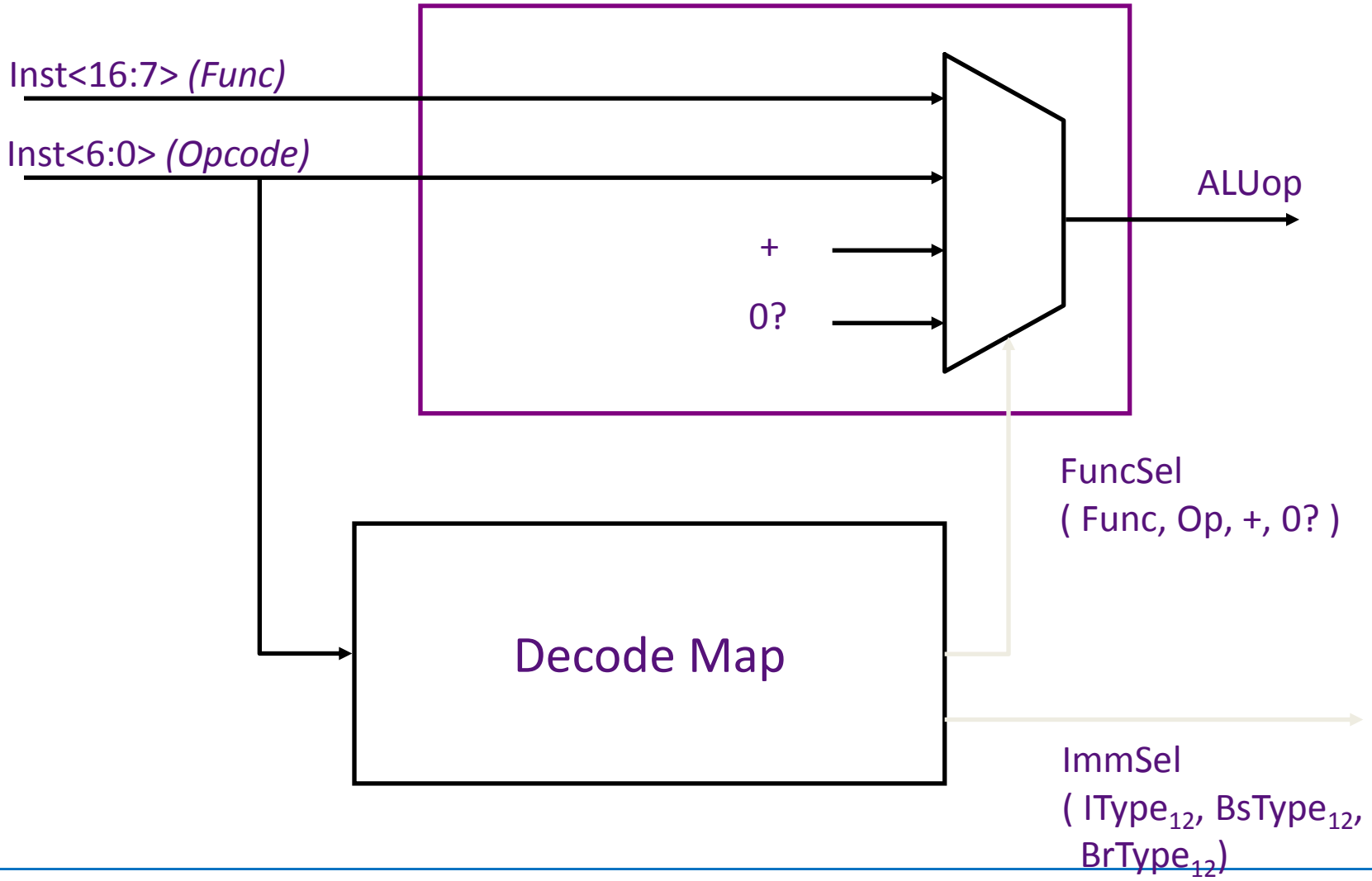


Tabela pentru controlul cablat

Embedded Systems Laboratory

Opcode	ImmSel	Op2Sel	FuncSel	MemWr	RFWen	WBSel	WASel	PCSel
ALU	*	Reg	Func	no	yes	ALU	rd	pc+4
ALUi	$IType_{12}$	Imm	Op	no	yes	ALU	rd	pc+4
LW	$IType_{12}$	Imm	+	no	yes	Mem	rd	pc+4
SW	$BsType_{12}$	Imm	+	yes	no	*	*	pc+4
BEQ_{true}	$BrType_{12}$	*	*	no	no	*	*	br
BEQ_{false}	$BrType_{12}$	*	*	no	no	*	*	pc+4
J	*	*	*	no	no	*	*	jabs
JAL	*	*	*	no	yes	PC	X1	jabs
JALR	*	*	*	no	yes	PC	rd	rind

Op2Sel= Reg / Imm
WASel = rd / X1

WBSel = ALU / Mem / PC
PCSel = pc+4 / br / rind / jabs

Controlul cablat într-un singur ciclu

Embedded Systems Laboratory

Presupunem că perioada ceasului este suficient de lungă pentru ca toți pașii următori să fie executați:

1. Instruction fetch
2. Decode and register fetch
3. ALU operation
4. Data fetch if required
5. Register write-back setup time

$$\Rightarrow t_C > t_{IFetch} + t_{RFetch} + t_{ALU} + t_{DMem} + t_{RWB}$$

Pe frontul pozitiv al semnalului următor de ceas sunt actualizate PC, tabela de registre și memoria

Recapitulare

- Microcodarea a devenit mai neatragatoare pe măsură ce diferențele de viteză dintre RAM și ROM s-au anulat și circuitele logice au putut fi implementate în aceeași tehnologie ca și memoria
- Seturile complexe de instrucțiuni sunt dificil de folosit intr-un pipeline așa că, pe măsură ce complexitatea circuitelor a crescut era foarte greu să crească și performanța lor
- Legea de fier descrie foarte bine diferența dintre RISC și CISC
  - Interschimbi instrucțiuni/program, cicli/instrucțiune și timp/ciclu
- ISA RISC este de tip load/store și e proiectată pentru implementări eficiente de pipeline
  - Foarte similar cu microcodul verticalizat
  - Inspirat de masinile Cray timpurii (CDC 6600/7600)
- ISA RISC-V va fi folosită în curs pentru a descrie restul conceptelor
  - Berkeley RISC chips: RISC-I, RISC-II, SOAR (RISC-III), SPUR (RISC-IV)

Acknowledgements

- Aceste slide-uri conțin materiale aparținând:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)
  - Behrooz Parhami (UCSB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

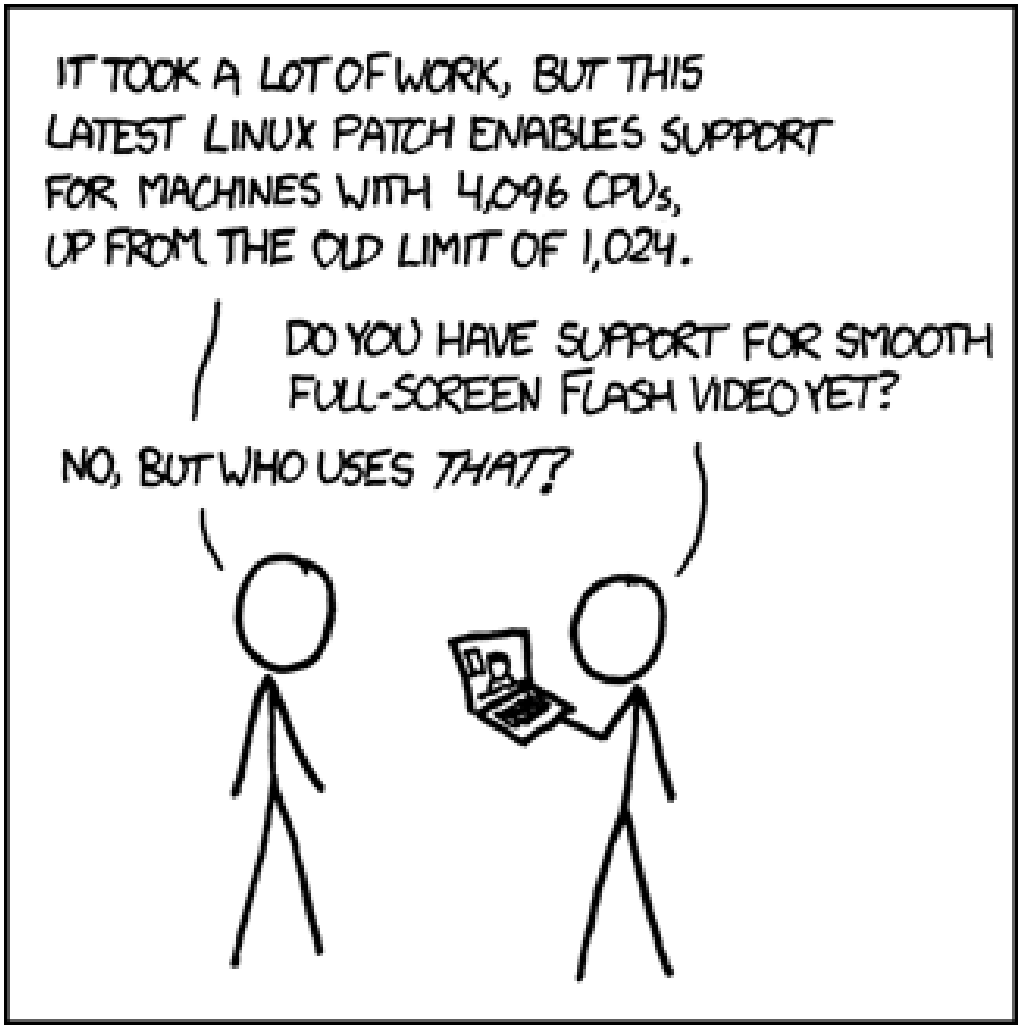
Calculatoare Numerice

– Cursul 11 –

Banda de asamblare

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Comic of the day

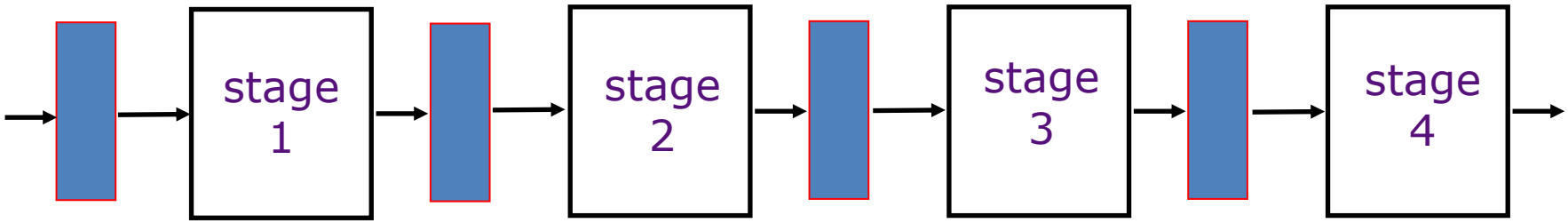


http://xkcd.com/619/

Din cursurile anterioare

- Microcodificarea a devenit din ce în ce mai puțin folosită pe măsură ce diferențele de performanță dintre RAM și ROM s-au micșorat
- Seturile complexe de instrucțiuni sunt dificil de executat în pipeline, așa că este foarte dificil să mărești performanța odată cu creșterea numărului de porți per procesor.
- ISA RISC Load-Store proiectate pentru eficiență maximă și execuție pipeline
  - Foarte similar cu microcod verticalizat
  - Inspirat de mașinile Cray (vom vorbi despre ele mai târziu)
- Legea de fier a performanței descrie destul de eficient spațiul de lucru
  - Echilibru între instrucțiuni/program, cicli/instrucțiune și timp/ciclu

O bandă de asamblare ideală



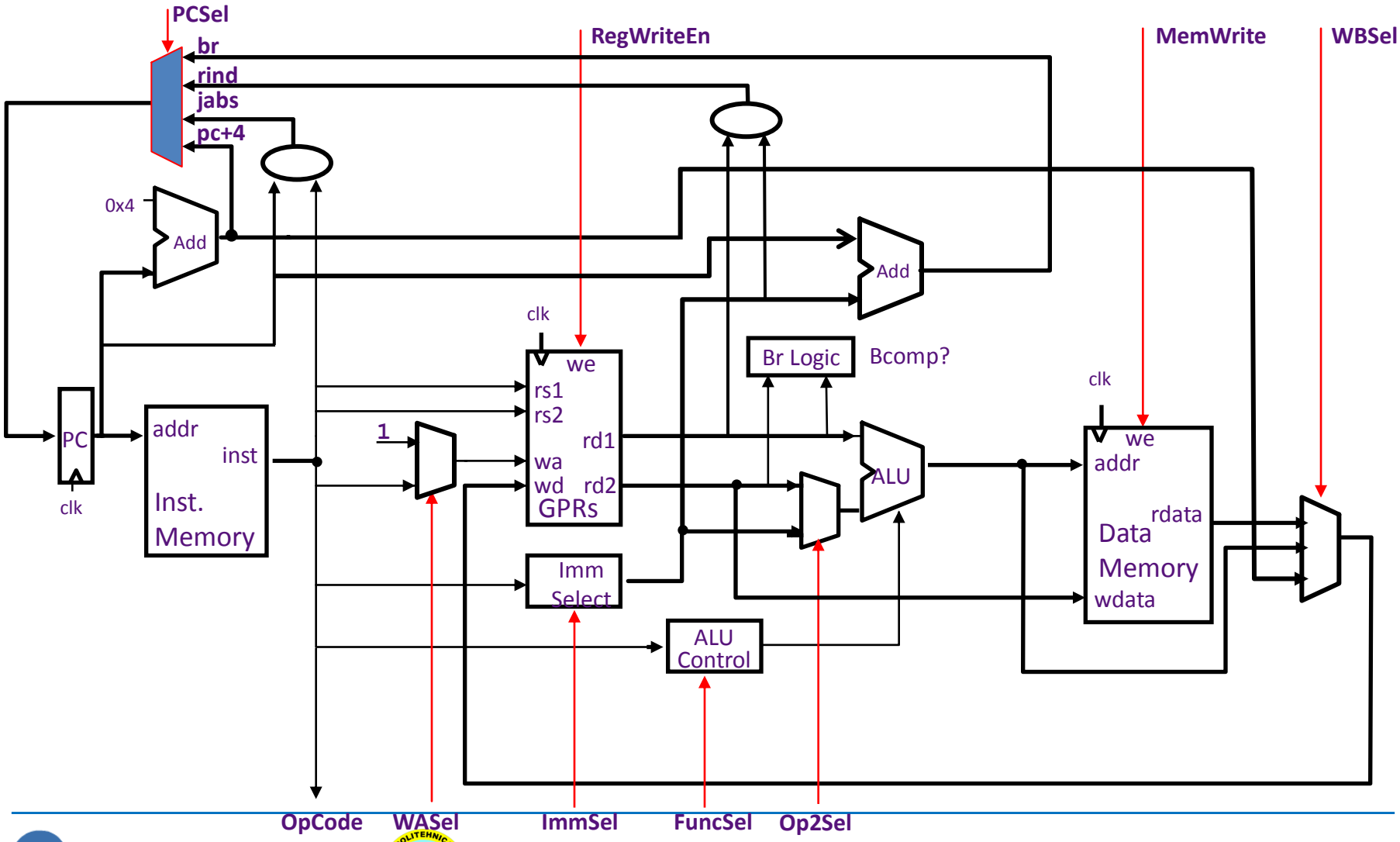
- Toate obiectele trec prin același niveluri
- Fără resurse partajate între oricare două niveluri
- Întârzierea de proapagare prin toate etapele benzii de asamblare este egală
- Planificarea intrării unui obiect în pipeline nu este afectată de obiectele din alte etape

Aceste condiții sunt valabile pentru liniile de asamblare industriale, dar instrucțiunile depind unele de celalalte!

RISC-V în bandă de asamblare

- Pentru a implementa RISC-V în bandă de asamblare:
- Întâi construim RISC-V fără pipeline cu CPI=1
- Pe urmă, adaugăm registre pentru pipeline pentru a reduce timpul de execuție, menținând CPI=1

Cursul anterior: implementare fără bandă de asamblare pentru RISC-V



Cursul anterior: Tabela de semnale pentru unitatea de control a RISC-V

Embedded Systems Laboratory

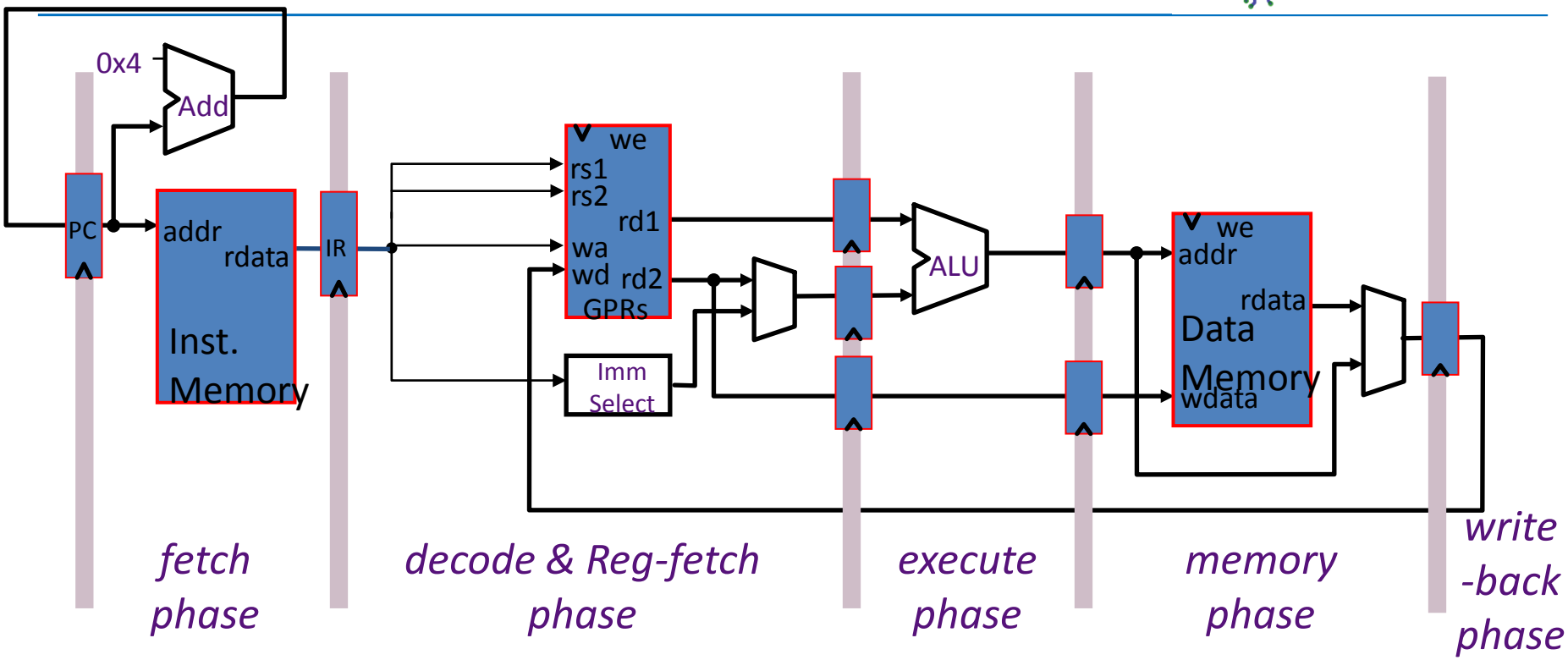
Opcode	ImmSel	Op2Sel	FuncSel	MemWr	RFWen	WBSel	WASel	PCSel
ALU	*	Reg	Func	no	yes	ALU	rd	pc+4
ALUi	$IType_{12}$	Imm	Op	no	yes	ALU	rd	pc+4
LW	$IType_{12}$	Imm	+	no	yes	Mem	rd	pc+4
SW	$BsType_{12}$	Imm	+	yes	no	*	*	pc+4
BEQ_{true}	$BrType_{12}$	*	*	no	no	*	*	br
BEQ_{false}	$BrType_{12}$	*	*	no	no	*	*	pc+4
J	*	*	*	no	no	*	*	jabs
JAL	*	*	*	no	yes	PC	X1	jabs
JALR	*	*	*	no	yes	PC	rd	rind

Op2Sel= Reg / Imm
WASel = rd / X1

WBSel = ALU / Mem / PC
PCSel = pc+4 / br / rind / jabs

Fetch
Reg Read
ALU
Data Memory
Reg Write

Magistrală în bandă de asamblare



Perioada ceasului poate fi redusă prin împărțirea execuției unei instrucțiuni în mai multe etape

$$t_C > \max \{t_{IM}, t_{RF}, t_{ALU}, t_{DM}, t_{RW}\} \; (= t_{DM} \textit{ probabil})$$

Cu toate acestea, CPI va crește dacă instrucțiunile nu sunt executate în pipeline

“Legea de fier” a performanței procesoarelor

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} * \frac{\text{Cycles}}{\text{Instruction}} * \frac{\text{Time}}{\text{Cycle}}$$

- Instructions per program depinde de codul sursă, compilator și ISA
- Cycles per instructions (CPI) depinde de ISA și μarchitectură
- Time per cycle depinde de μarchitectură și tehnologia de bază în care e implementat procesorul

Microarchitectură	CPI	cycle time
Microcodificată	>1	short
Single-cycle unpipelined	1	long
Pipelined	1	short

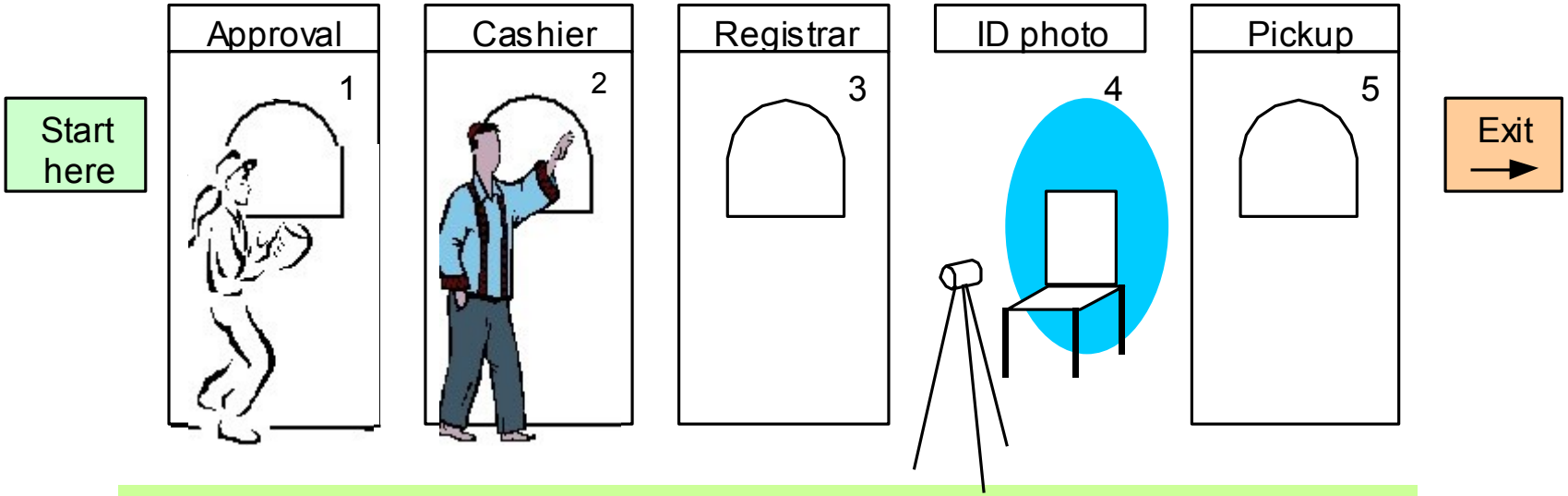
Concepte de pipelining

Embedded Systems Laboratory

Strategii pentru îmbunătățirea performanței

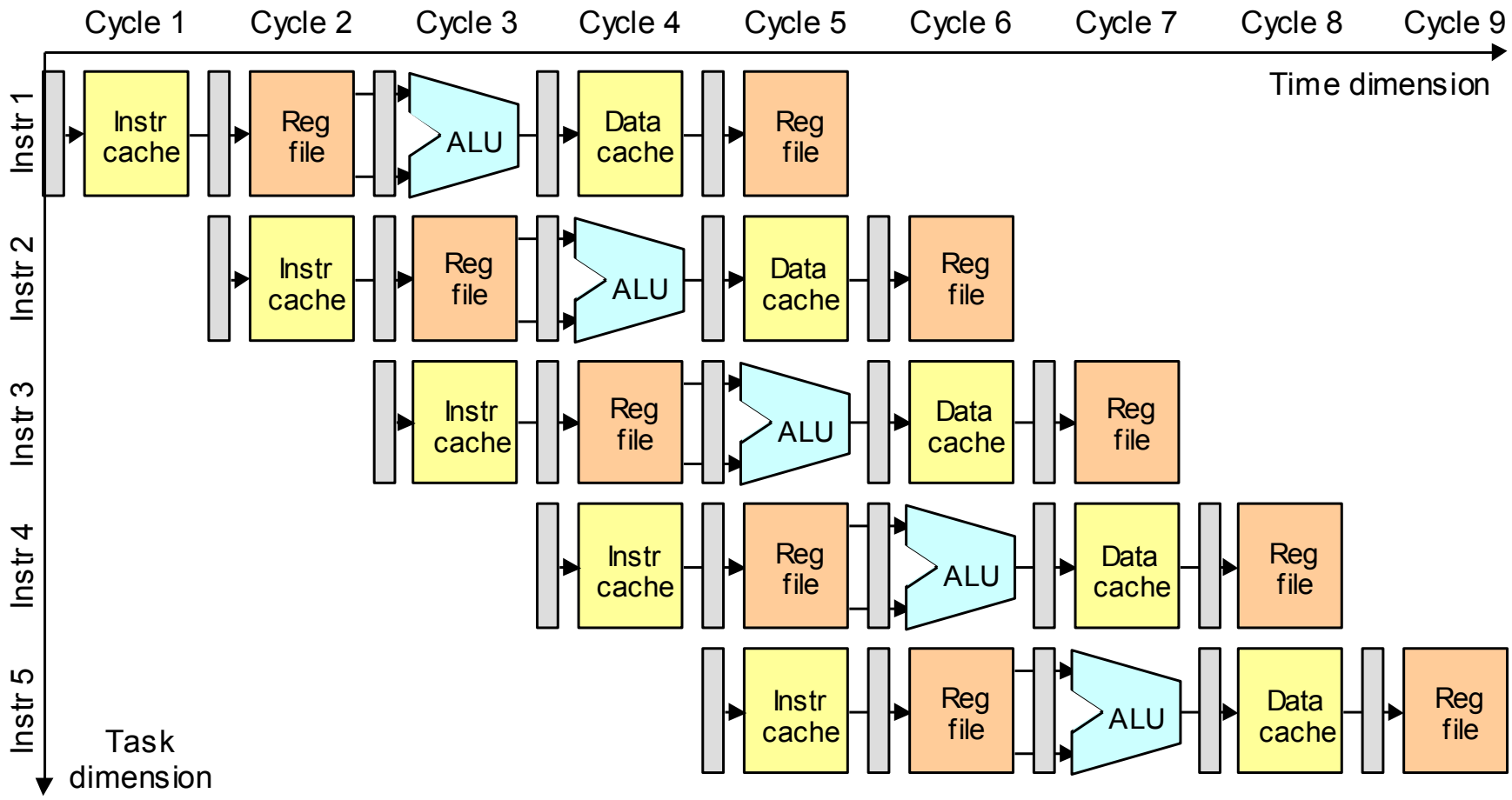
1 – Folosim căi de date independente care acceptă citirea mai multor instrucțiuni în paralel: *multiple-instruction-issue* sau *superscalar*

2 – Suprapunem execuția diferitelor instrucțiuni prin pornirea următoarei instrucțiuni înainte de terminarea celei în curs de execuție:*(super)pipelined*



Banda de asamblare aplicată unui proces de înregistrare persoane

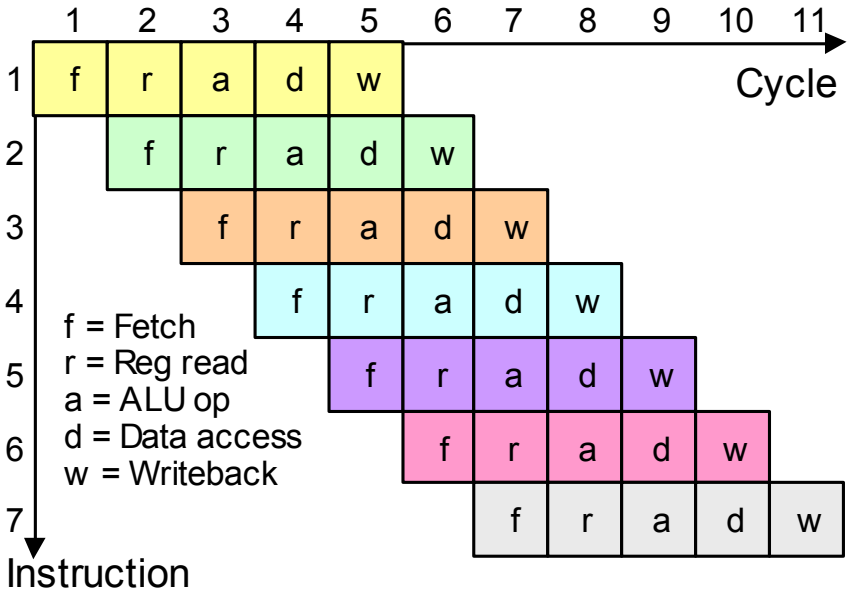
Execuția instrucțiunilor în banda de asamblare



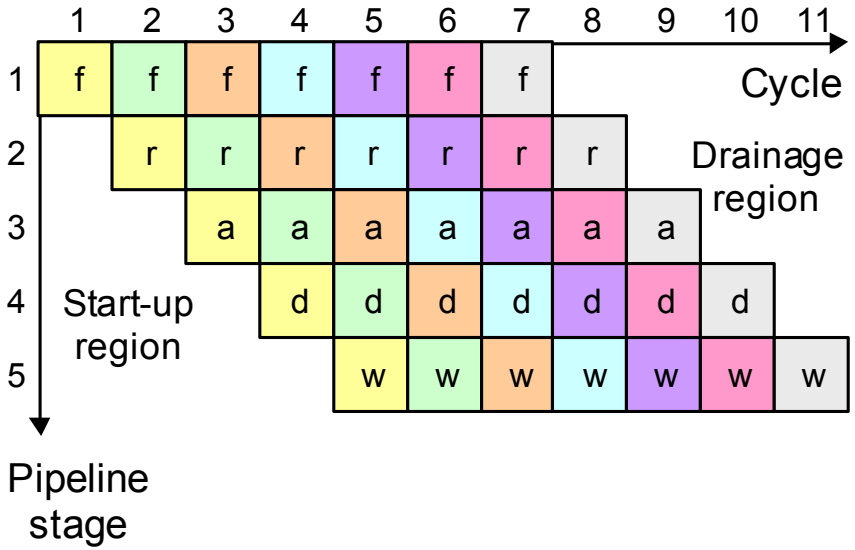
Exemplu de execuție în b.a. pentru un procesor general

Reprezentări alternative ale benzii de asamblare

O bandă de asamblare poate să execute de obicei o instrucțiune pe front de ceas. IPS este dictată de frecvența ceasului sistemului.



(a) Task-time diagram



(b) Space-time diagram

Două reprezentări grafice abstracte a unei benzi de asamblare în cinci etape ce execută șapte instrucțiuni.

Exemplu de b.a. într-un fotocopiator

Un fotocopiator cu o tavă de x foi de hârtie copiază prima foaie în 4s și apoi celelalte foi în câte 1s. Calea pentru copiator este o b.a. în 4 etape, în care fiecare etapă are o latență de 1s. Prima pagină trece prin toate cele 4 etape ale b.a. Și iese după 4s. Fiecare pagină următoare iese la 1s după anterioară. Cum variază productivitatea copiatorului în funcție de x, presupunând că încărcarea și descărcarea copiatorului durează 15s.

Soluție

Fiecare teanc de x foi este copiat în $15 + 4 + (x - 1) = 18 + x$ secunde.

Un copiator fără b.a. necesită $4x$ secunde ca să copieze x foi.

Pentru $x > 6$, versiunea cu b.a. are o performanță mai bună.

Dacă $x = 50$, îmbunătățirea de performanță este

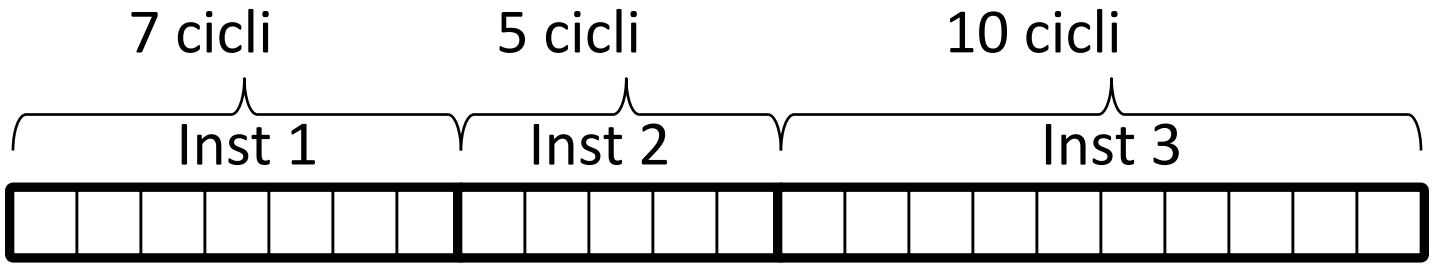
$$(4 \times 50) / (18 + 50) = 2.94.$$

Exemple CPI

Embedded Systems Laboratory

Mașină cu microcod

Timp →

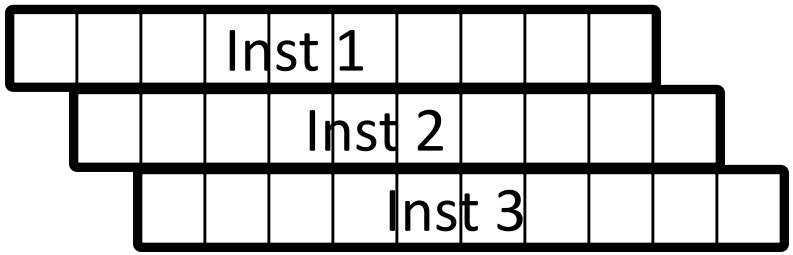


3 instrucțiuni, 22 cicli, CPI=7.33

Mașină fără pipeline

3 instrucțiuni, 3 cicli, CPI=1

Mașină cu pipeline



3 instrucțiuni, 3 cicli, CPI=1

5 niveluri de pipeline CPI≠5!!!

Presupuneri legate de tehnologie

- Memorie mică dar foarte rapidă (cache) susținută de o memorie lentă dar semnificativ mai mare
- UAL rapid (cel puțin pentru calcul pe întregi)
- Tabele de registre multiport (mai lente!)

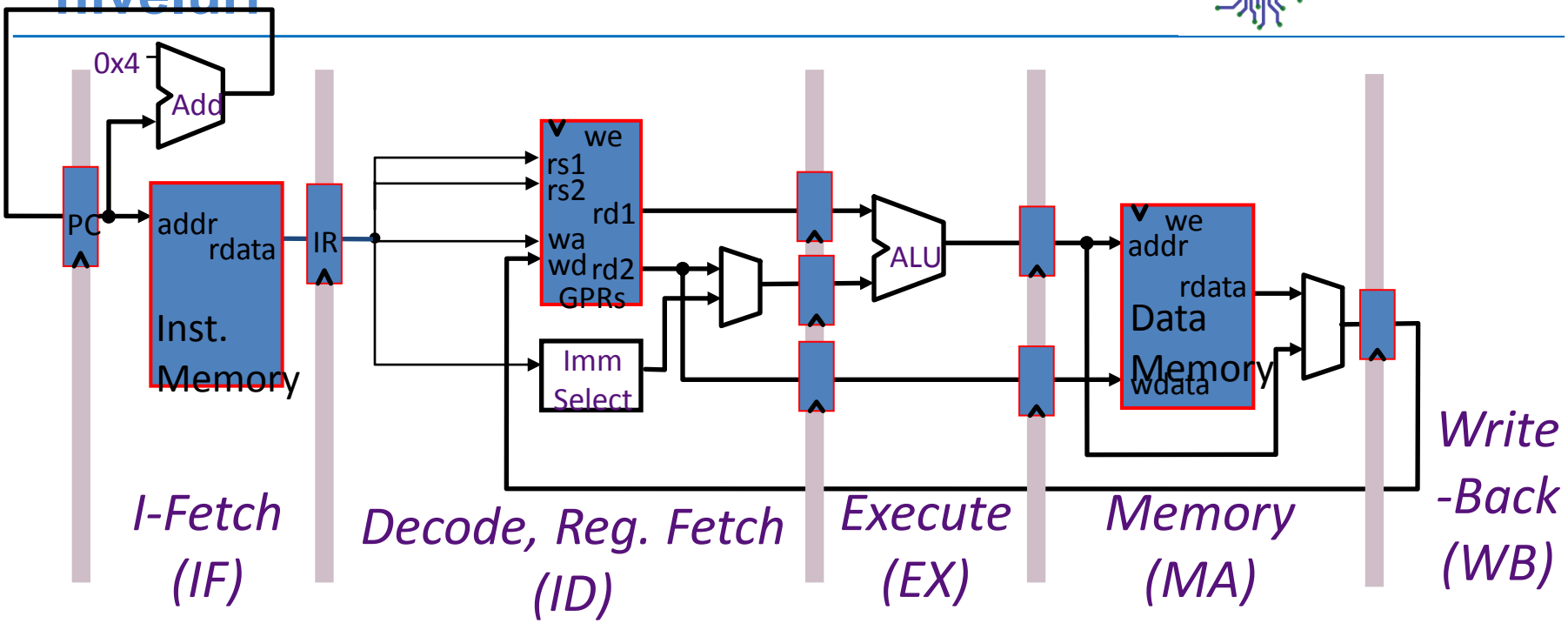
Prin urmare, următoarea presupunere este rezonabilă

$$t_{IM} \approx t_{RF} \approx t_{ALU} \approx t_{DM} \approx t_{RW}$$

Vom proiecta o bandă de asamblare în 5 etape

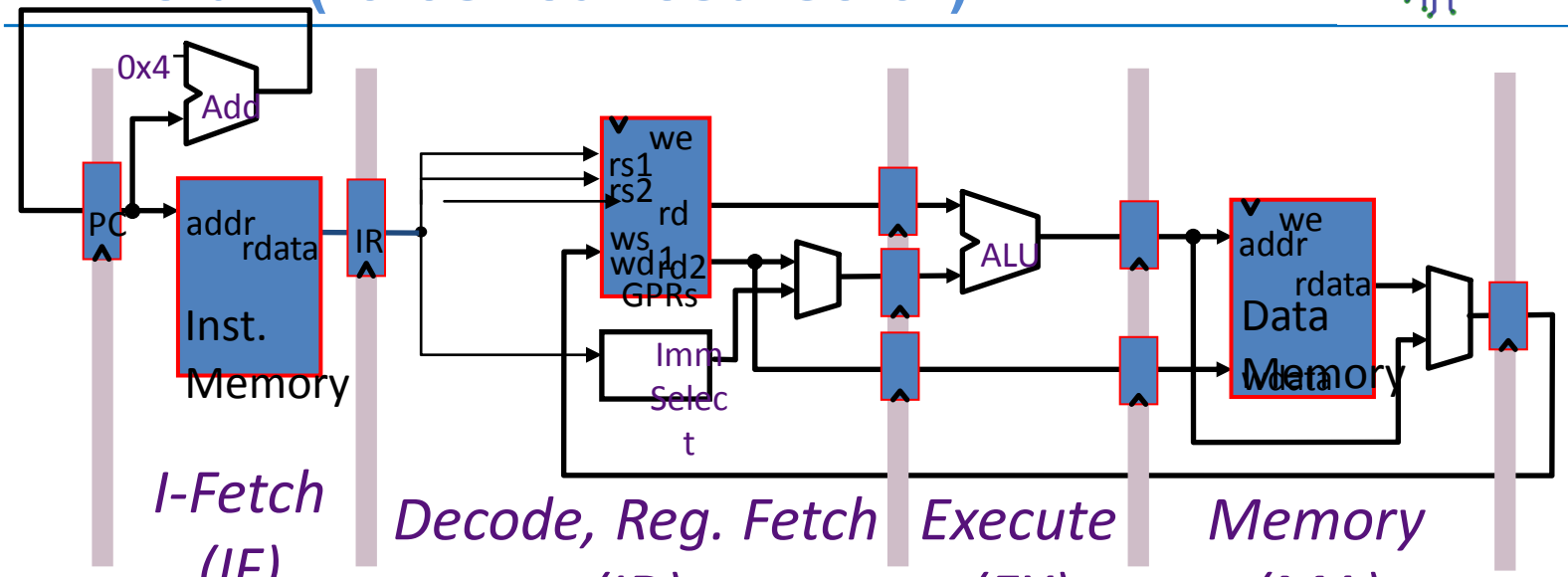
- unele design-uri comerciale de procesor au peste 30 de etape pentru a face o singură adunare de întregi!

Execuție în bandă de asamblare cu 5 niveluri



timp	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
instruction1	IF_1	ID_1	EX_1	MA_1	WB_1				
instruction2		IF_2	ID_2	EX_2	MA_2	WB_2			
instruction3			IF_3	ID_3	EX_3	MA_3	WB_3		
instruction4				IF_4	ID_4	EX_4	MA_4	WB_4	
instruction5					IF_5	ID_5	EX_5	MA_5	WB_5

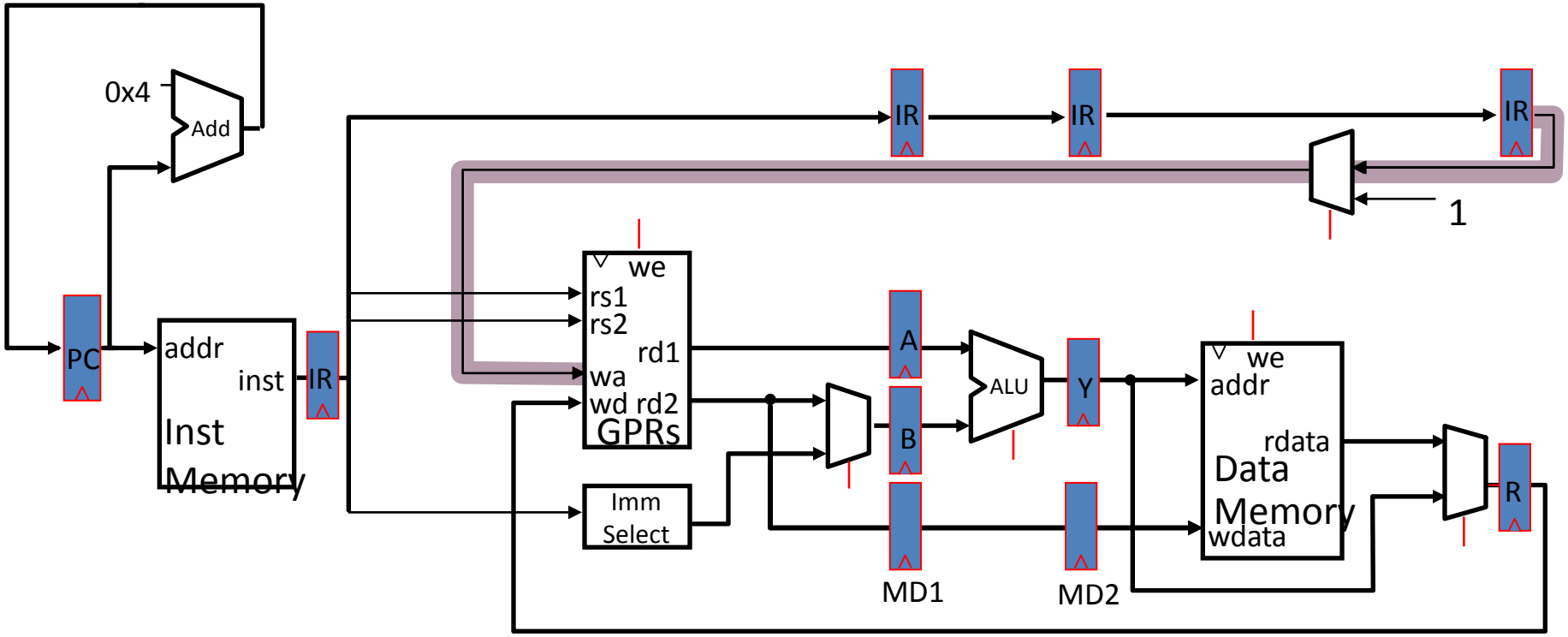
Execuție în bandă de asamblare cu 5 niveluri (folosirea resurselor)



Resurse

	timp	t0	t1	t2	t3	t4	t5	t6	t7	. .
	IF	I_1	I_2	I_3	I_4	I_5				
	ID		I_1	I_2	I_3	I_4	I_5			
	EX			I_1	I_2	I_3	I_4	I_5		
	MA				I_1	I_2	I_3	I_4	I_5	
	WB					I_1	I_2	I_3	I_4	I_5

Execuție în pipeline: Instrucțiuni UAL

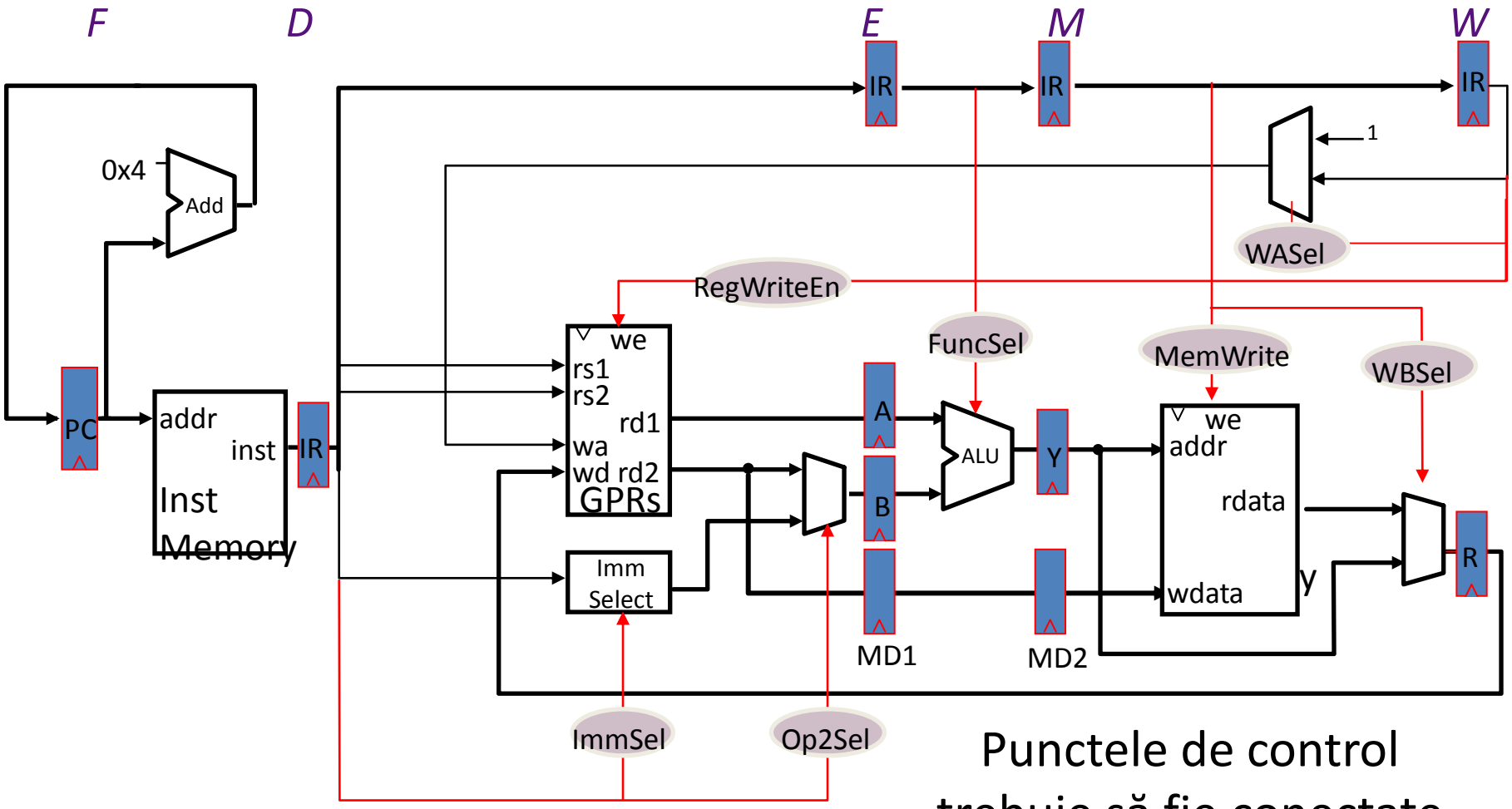


Nu e foarte corect!

Avem nevoie de un Instruction Reg (IR) pentru fiecare etapă

Magistrala în b.a. Pentru RISC-V

fără instrucțiuni de jump



Punctele de control trebuie să fie conectate

Instrucțiunile interacționează unele cu celalalte în pipeline

- O instrucțiune din b.a. Poate avea nevoie de o resursă folosită de altă instrucțiune din b.a.

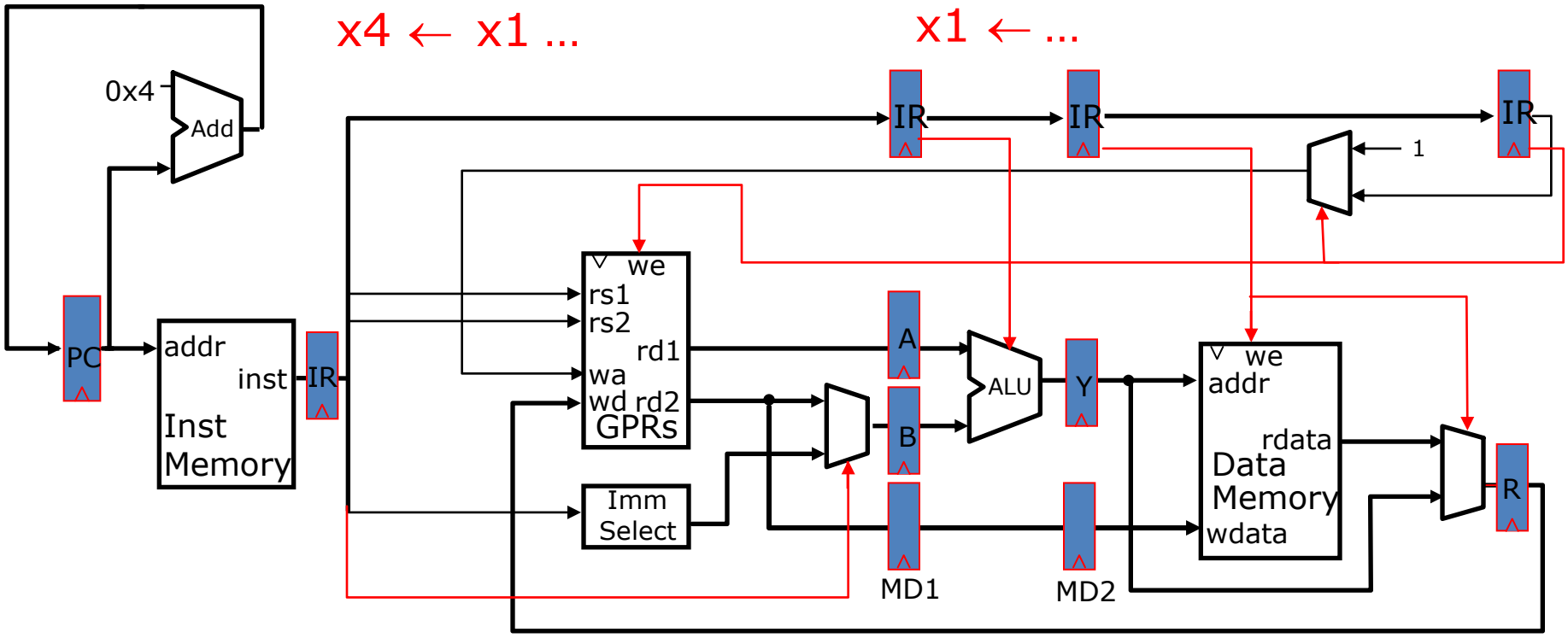
→ *Hazard structural*

- O instrucțiune poate să depindă de rezultatul unei instrucțiuni anterioare
  - Dependența poate să fie pentru valoarea unei variabile
    → *hazard de date*
  - Dependența poate să fie pentru adresa următoarei instrucțiuni
    → *hazard de control (branch-uri, excepții)*

Soluționarea hazardurilor structurale

- Hazardurile structurale apar atunci când două instrucțiuni necesită aceeași resursă hardware în același timp
  - Poate fi rezolvat în hardware prin oprirea execuției noii instrucțiuni până când instrucțiunea mai veche a eliberat resursa folosită
- Un hazard hardware structural poate fi întotdeauna evitat prin adăugarea de mai mult hardware în design
  - E.g., dacă două instrucțiuni au nevoie de acces la memorie în același timp, se poate soluționa prin folosirea unei memorii bi- sau multi-port
- Implementarea noastră în 5 etape nu are hazarduri structurale de la bun început
  - Datorită RISC-V ISA, care a fost proiectată pentru execuția în b.a.

Hazarduri de date



...
x1 ← x0 + 10
x4 ← x1 + 17
...

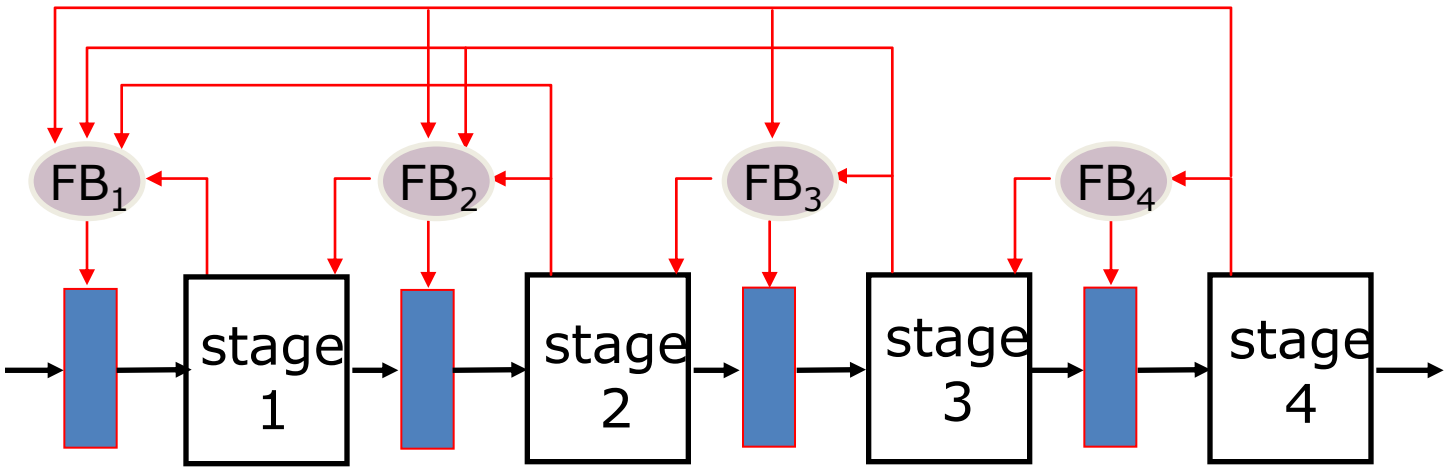
x1 este învechit. Oops!

Soluționarea hazardurilor de date (1)

Strategia 1:

Aștept ca rezultatul să fie disponibil prin "înghețarea" tuturor etapelor anterioare ale b.a.
➔ *interlocking*

Feedback pentru soluționarea hazardurilor

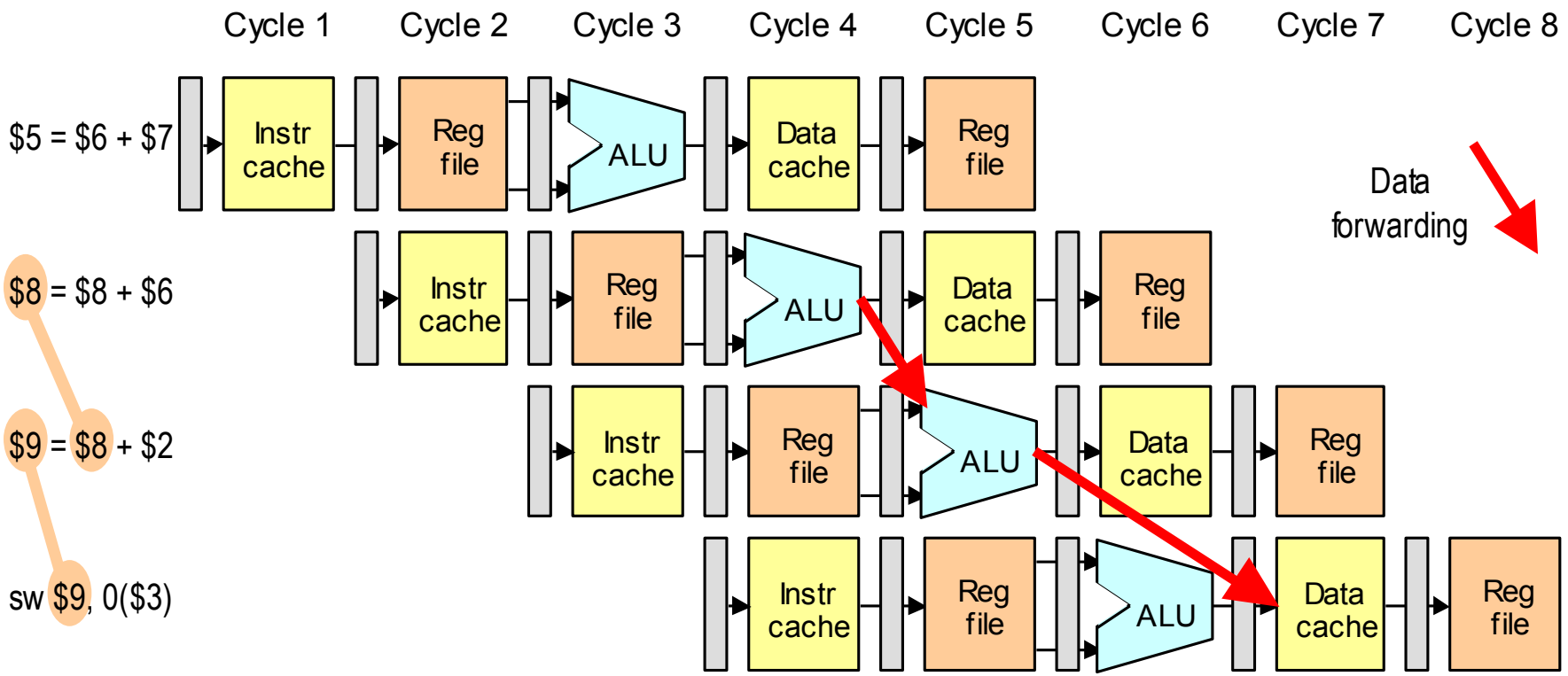


- Etapele superioare furnizează informații legate de dependență stagiilor inferioare, care pot încetini sau opri instrucțiunile
- Controlul unei b.a. în acest fel funcționează dacă instrucțiunea de la nivelul i+1 poate să comepleteze execuția fără nici o interferență cu instrucțiunile din nivelele 1..i (altfel pot apare deadlock-uri)

Hazarduri în b.a. Stalls & Bubbles

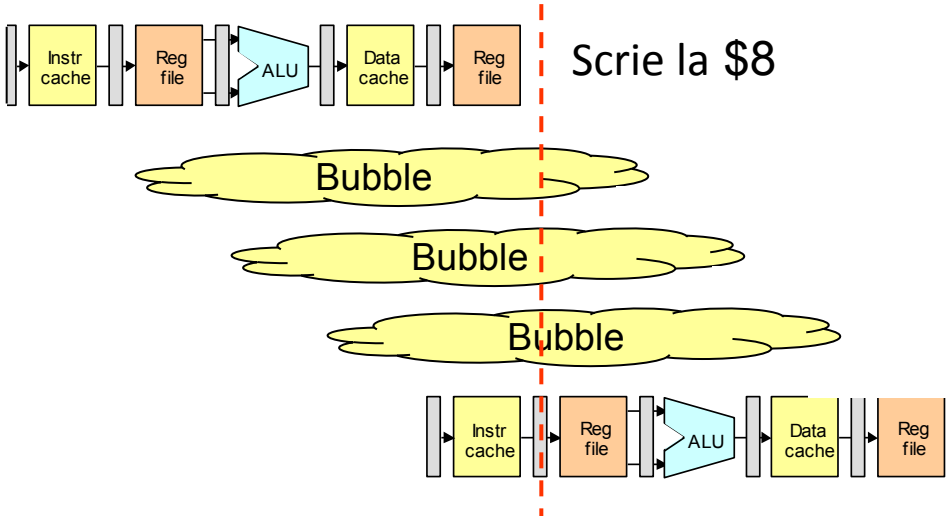
Embedded Systems Laboratory

Generate din cauza dependențelor de date

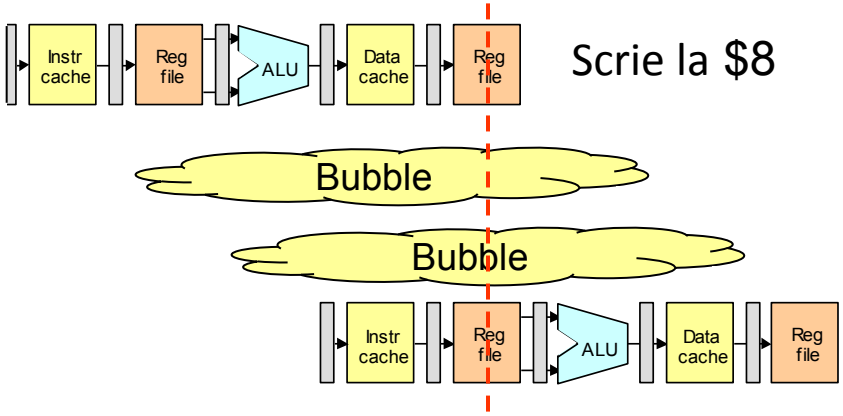


Dependență Read-After-Write și soluționarea ei prin data forwarding

Inserarea de bule într-o b.a.

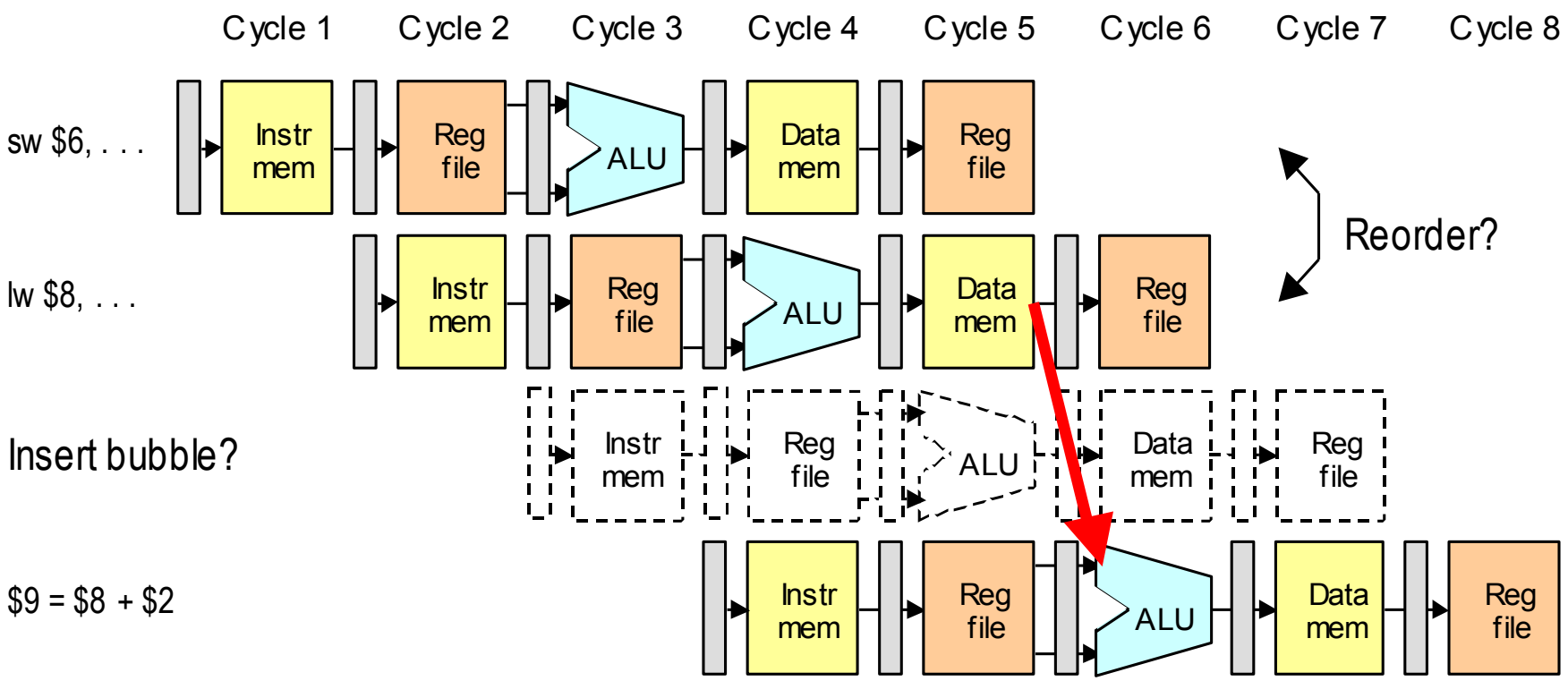


Fără data forwarding avem nevoie de aceste trei "bule" pentru a soluționa dependența read-after-write



Doar două bule dacă presupunem că un registru poate fi scris și citit într-un singur ciclu de ceas.

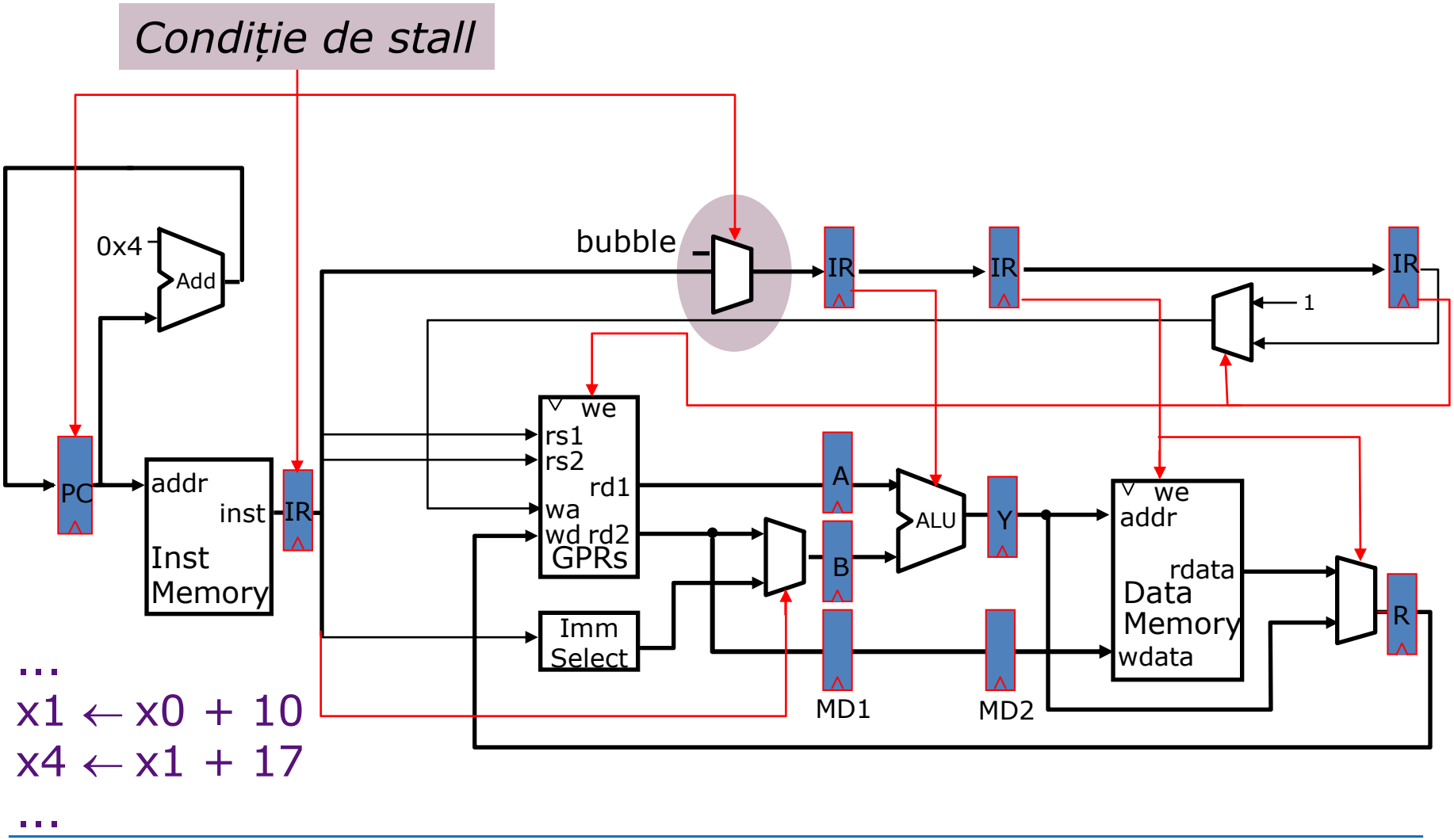
Al doilea tip de dependență de date



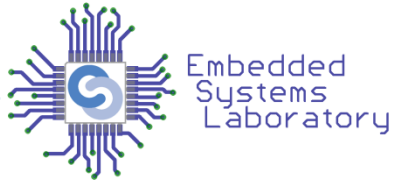
Fără data forwarding, este nevoie de trei (două) bule pentru a rezolva un conflict read-after-load.

Dependența Read-After-Load și soluționarea ei prin inserarea de bule și data forwarding

Interlocking pentru a rezolva hazardurile de date



Stalled Stages & Pipeline Bubbles



	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .		
(I_1) x1 ← (x0) + 10	IF_1	ID_1	EX_1	MA_1	WB_1						
(I_2) x4 ← (x1) + 17		IF_2	ID_2	ID_2	ID_2	ID_2	EX_2	MA_2	WB_2		
(I_3)			IF_3	IF_3	IF_3	IF_3	ID_3	EX_3	MA_3	WB_3	
(I_4)							IF_4	ID_4	EX_4	MA_4	WB_4
(I_5)								IF_5	ID_5	EX_5	MA_5

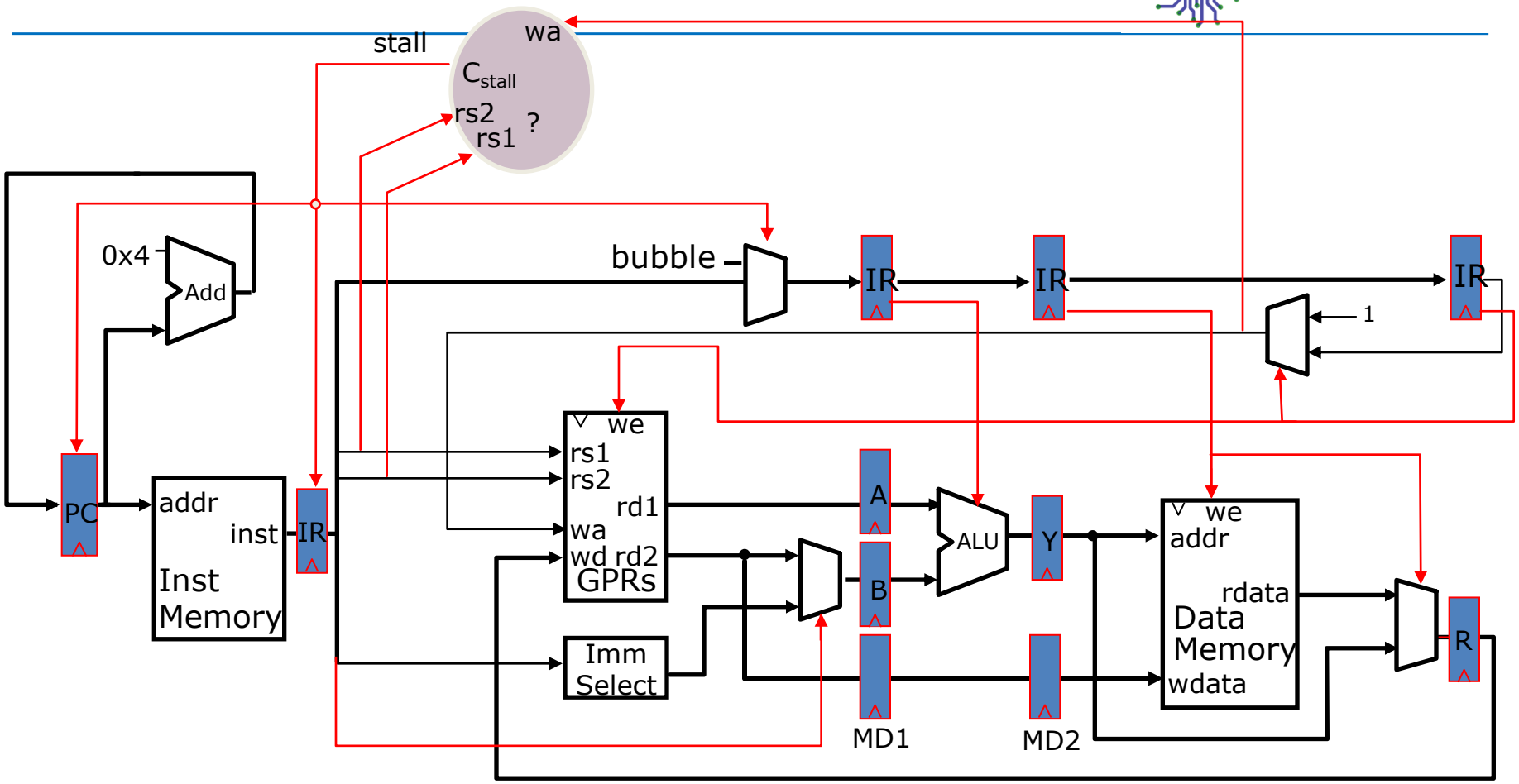
stalled stages

Resource Usage

timp	t0	t1	t2	t3	t4	t5	t6	t7	. . . .			
IF	I_1	I_2	I_3	I_3	I_3	I_3	I_4	I_5				
ID		I_1	I_2	I_2	I_2	I_2	I_3	I_4	I_5			
EX			I_1	-	-	-	I_2	I_3	I_4	I_5		
MA				I_1	-	-	-	I_2	I_3	I_4	I_5	
WB					I_1	-	-	-	I_2	I_3	I_4	I_5

- ⇒ *pipeline bubble*

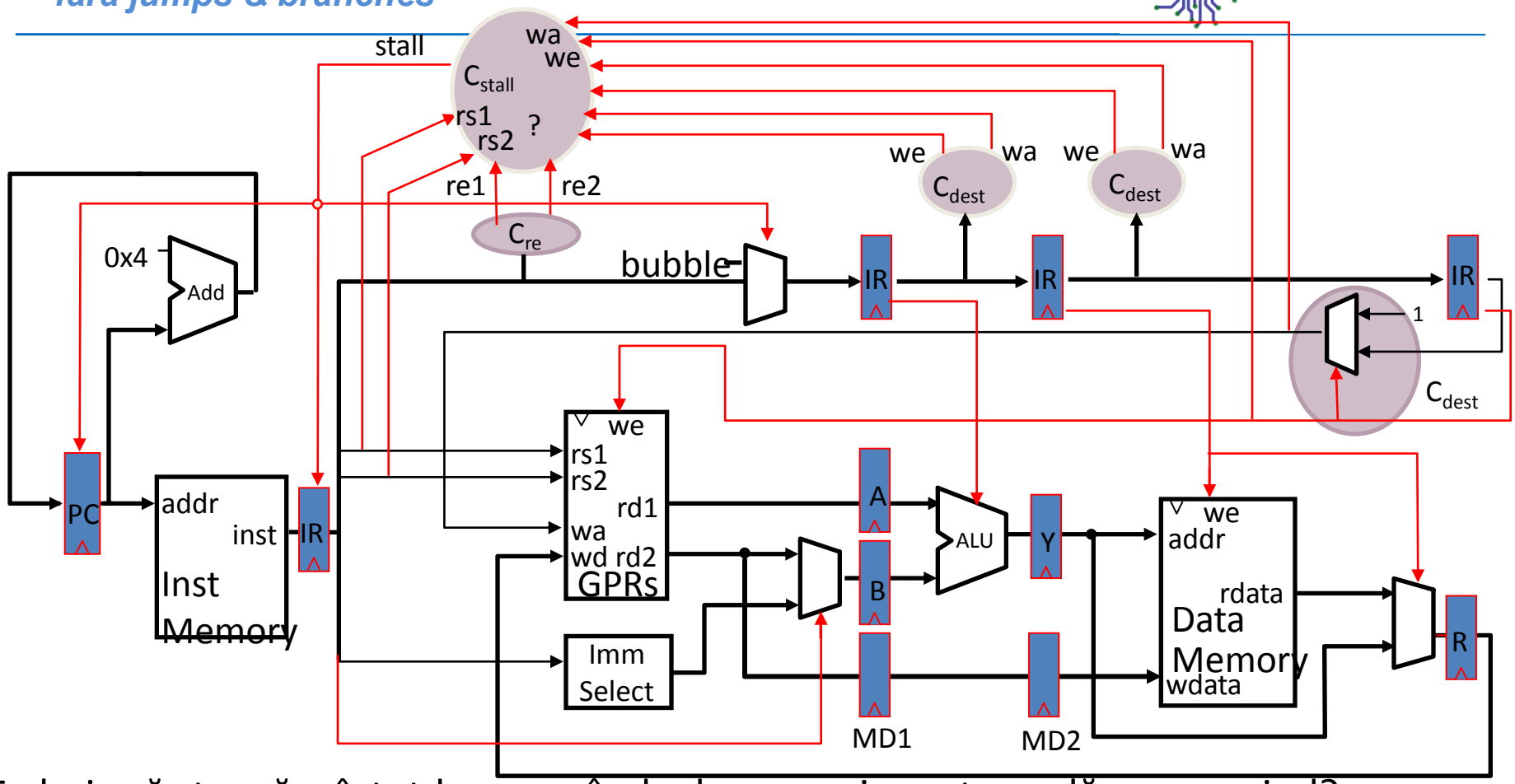
Logica de control pentru interlocking



Compară *registrele sursă* ale instrucțiunii din faza de *decode* cu registrul destinație al instrucțiunilor în *curs de execuție*

Logica de control pentru interlocking

fără jumps & branches

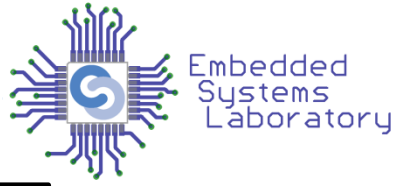


Trebuie să stagnăm întotdeauna când adresa unui rs este egală cu a unui rd?

nu orice instrucțiune scrie un registru -> we

nu orice instrucțiune citește un registru -> re

Registrele sursă și destinație



rd	rs1	rs2	func10		opcode	ALU
rd	rs1	Imm[11:0]		func3	opcode	ALUI/LW/JALR
Imm[11:7]	rs1	rs2	Imm[6:0]	func3	opcode	SW/Bcond
Jump offset[24:0]					opcode	

		source(s)	*destination*
ALU	rd <- rs1 func10 rs2	rs1, rs2	rd
ALUI	rd <- rs1 op imm	rs1	rd
LW	rd <-M [rs1 + imm]	rs1	rd
SW	M [rs1 + imm] <- rs2	rs1, rs2	-
B*cond*	rs1,rs2	rs1, rs2	-
	true: PC <- PC + imm		
	false: PC <- PC + 4		
J	PC <- PC + imm	-	-
JAL	x1 <- PC, PC <- PC + imm	-	x1
JALR	rd <- PC, PC <- rs1 + imm	rs1	rd

Determinarea semnalului de stall pentru b.a.

Embedded Systems Laboratory

C_{dest}

ws = *Case* opcode

JAL	-> X1
else	-> rd

we = *Case* opcode

ALU, ALUi, LW,JALR	->(ws != 0)
JAL	->on
...	->off

C_{re}

re1 = *Case* opcode

ALU, ALUi, LW, SW, Bcond, JALR	->on
J, JAL	->off

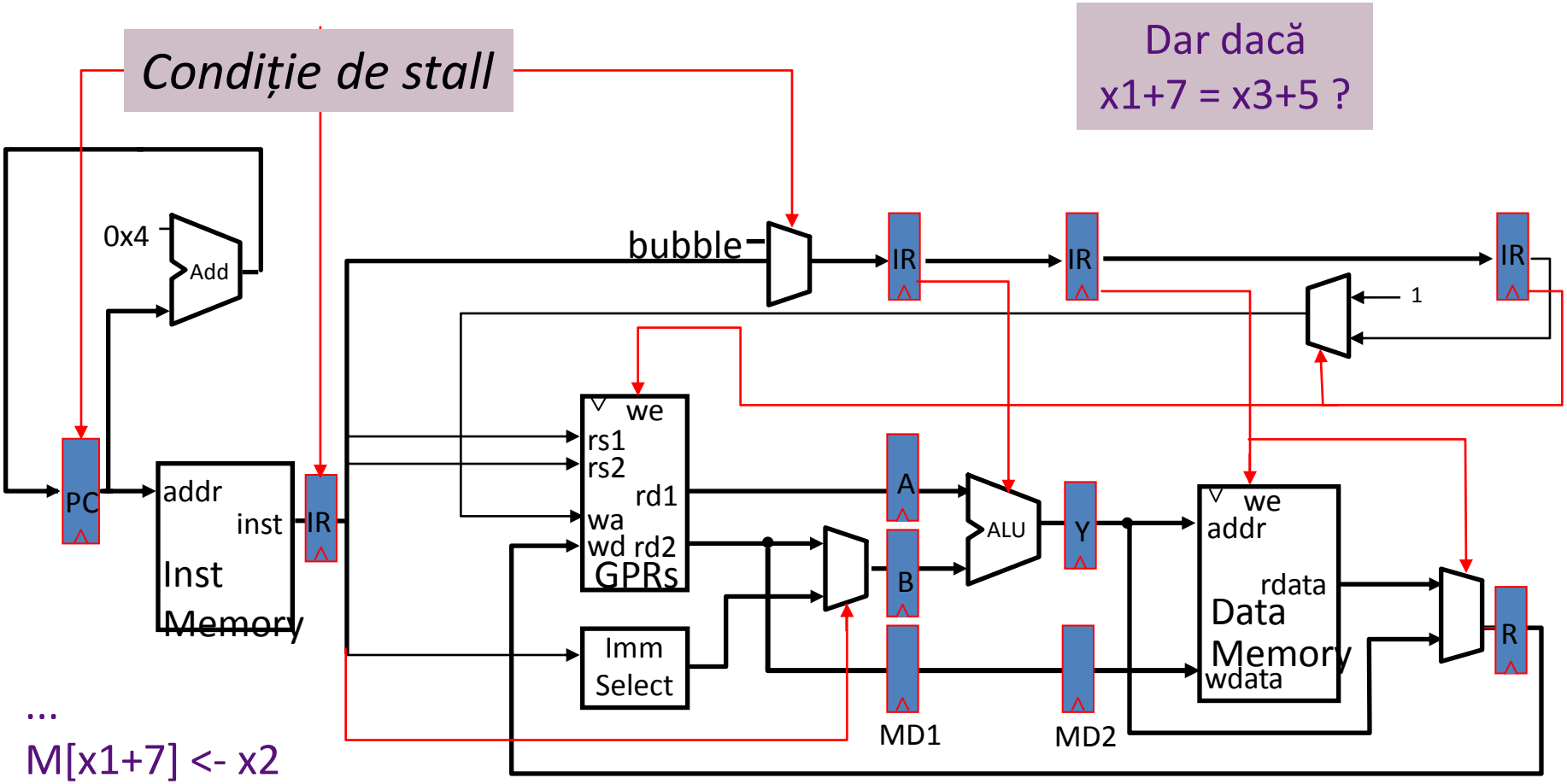
re2 = *Case* opcode

ALU, SW,Bcond	->on
...	->off

C_{stall} $stall = ((rs1_D = ws_E).we_E + (rs1_D = ws_M).we_M + (rs1_D = ws_W).we_W) . re1_D + ((rs2_D = ws_E).we_E + (rs2_D = ws_M).we_M + (rs2_D = ws_W).we_W) . re2_D$

This is not the full story !

Hazarduri determinate de instrucțiuni Load & Store



...
M[x1+7] <- x2
x4 <- M[x3+5]
...

Este posibil vreun hazard de date în cadrul acestei secvențe de instrucțiuni?

Hazarduri Load & Store

```
...
M[x1+7] <- x2
x4 <- M[x3+5]
...
```

x1+7 = x3+5 -> *hazard de date*

Cu toate acestea, hazardul este evitat pentru că sistemul de memorie face un write într-un singur ciclu de ceas*!*

Hazardurile Load/Store sunt câteodată soluționate în pipeline și câteodată în sistemul de memorii.

Vom vorbi mai multe despre asta în curs.

Soluționarea hazardurilor de date(2)

Strategia 2:

Rutează datele imediat ce ele sunt calculate către nivelurile inferioare din banda de asamblare
→ *bypass*

Bypassing

timp	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) x1 ← x0 + 10	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) x4 ← x1 + 17		IF_2	ID_2	ID_2	ID_2	ID_2	EX_2	MA_2	WB_2
(I_3)			IF_3	IF_3	IF_3	IF_3	ID_3	EX_3	MA_3
(I_4)			*stalled stages*				IF_4	ID_4	EX_4
(I_5)								IF_5	ID_5

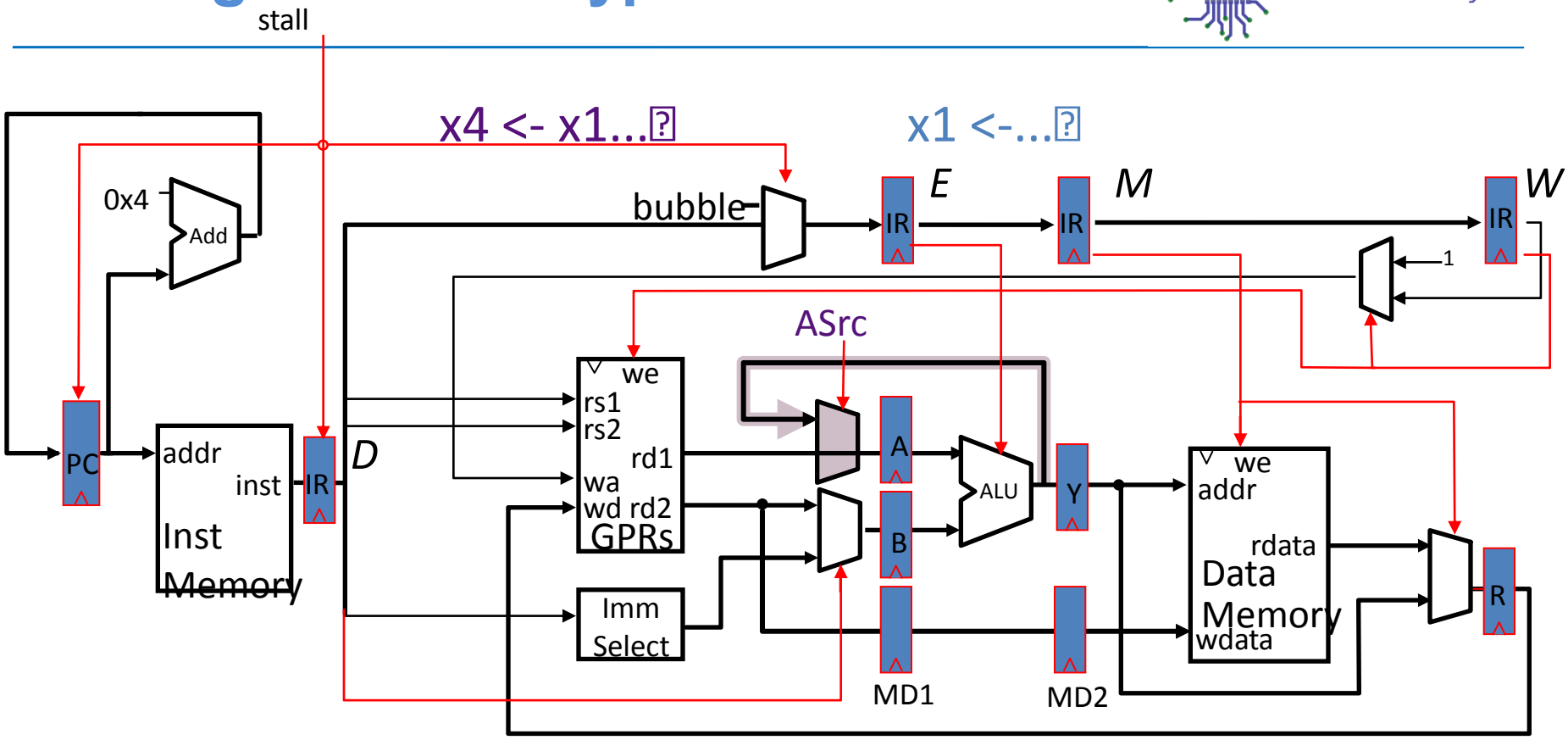
Fiecare *stall sau kill* introduce o bulă în banda de asamblare

$$\Rightarrow CPI > 1$$

O nouă cale de date, i.e., *un bypass*, poate aduce datele de la ieșirea UAL la intrarea acesteia

timp	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) x1 ← x0 + 10		IF_1	ID_1	EX_1	MA_1	WB_1			
(I_2) x4 ← x1 + 17			IF_2	ID_2	EX_2	MA_2	WB_2		
(I_3)				IF_3	ID_3	EX_3	MA_3	WB_3	
(I_4)					IF_4	ID_4	EX_4	MA_4	WB_4
(I_5)						IF_5	ID_5	EX_5	MA_5 WB_5

Adăugarea unui bypass



Când ajută acest bypass?

	...		
(I_1)	x1 <- x0 + 10	x1 <- M[x0 + 10]	JAL 500
(I_2)	x4 <- x1 + 17	x4 <- x1 + 17	x4 <- x1 + 17
	da	*nu*	*nu*

Semnalul de bypass
Derivarea semnalului din cel pentru Stall

Embedded Systems Laboratory

$$stall = (((\sout{rs1_D = ws_E).we_E} + (rs1_D = ws_M).we_M + (rs1_D = ws_W).we_W).re1_D$$
$$+((rs2_D = ws_E).we_E + (rs2_D = ws_M).we_M + (rs2_D = ws_W).we_W).re2_D)$$

ws = *Case* opcode
JAL ->X1
else ->rd

we = *Case* opcode
ALU, ALUi, LW, JALR ->(ws != 0)
JAL ->on
... ->off

$ASrc = (rs1_D = ws_E).we_E.re1_D$

Este corect?

Nu, pentru că doar instrucțiunile tip ALU și ALUi pot beneficia de un astfel de bypass

Împărțim we_E în două componente: we-bypass, we-stall

Semnalele Bypass și Stall

Împărțim we_E în două componente: we-bypass, we-stall

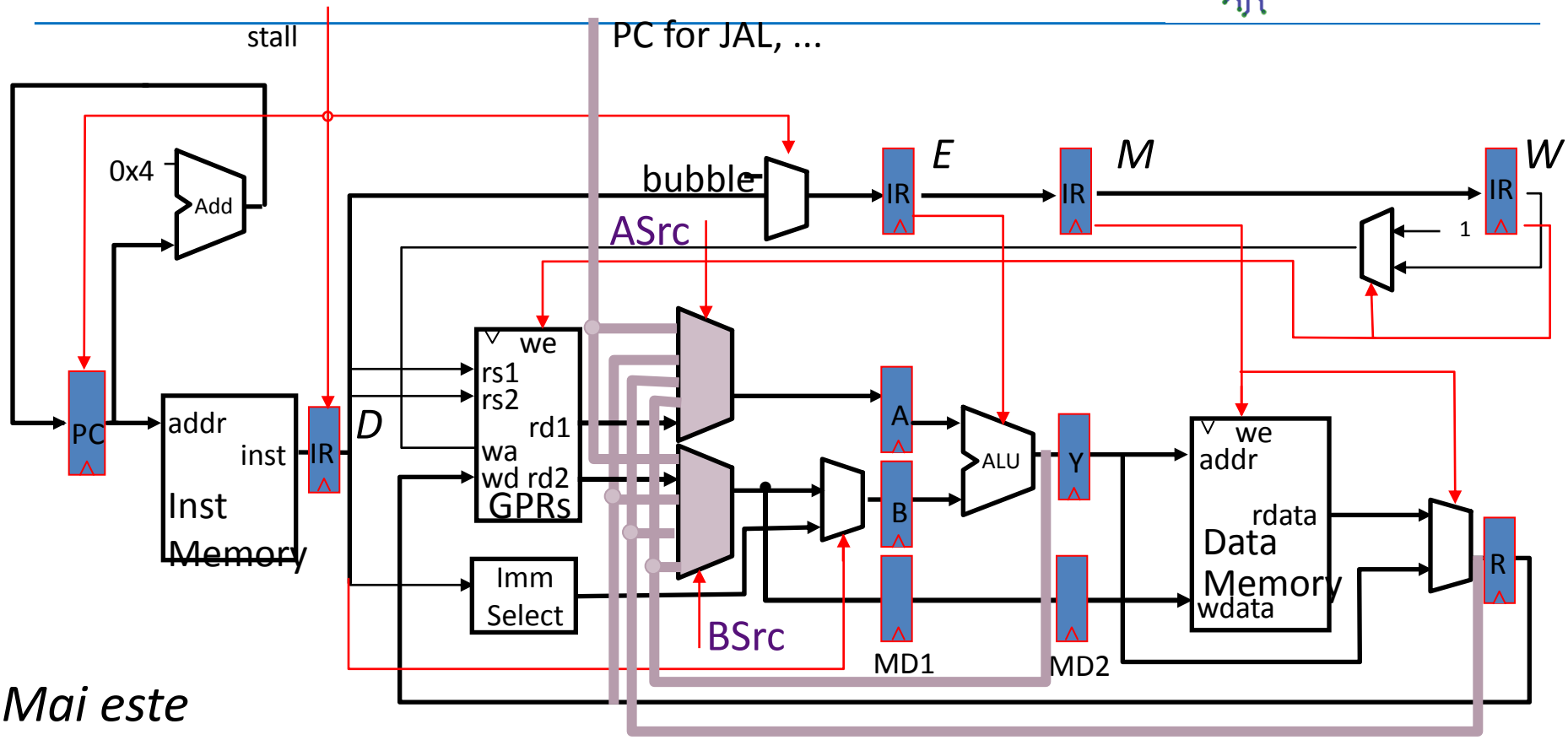
$we\text{-}bypass_E$ = *Case* $opcode_E$
ALU, ALUi -> (ws != 0)
... ->off

$we\text{-}stall_E$ = *Case* $opcode_E$
LW, JALR -> (ws != 0)
JAL ->on
... ->off

$ASrc = (rs1_D = ws_E).we\text{-}bypass_E . re1_D$

$stall = ((rs1_D = ws_E).we\text{-}stall_E + (rs1_D = ws_M).we_M + (rs1_D = ws_W).we_W).re1_D + ((rs2_D = ws_E).we_E + (rs2_D = ws_M).we_M + (rs2_D = ws_W).we_W).re2_D$

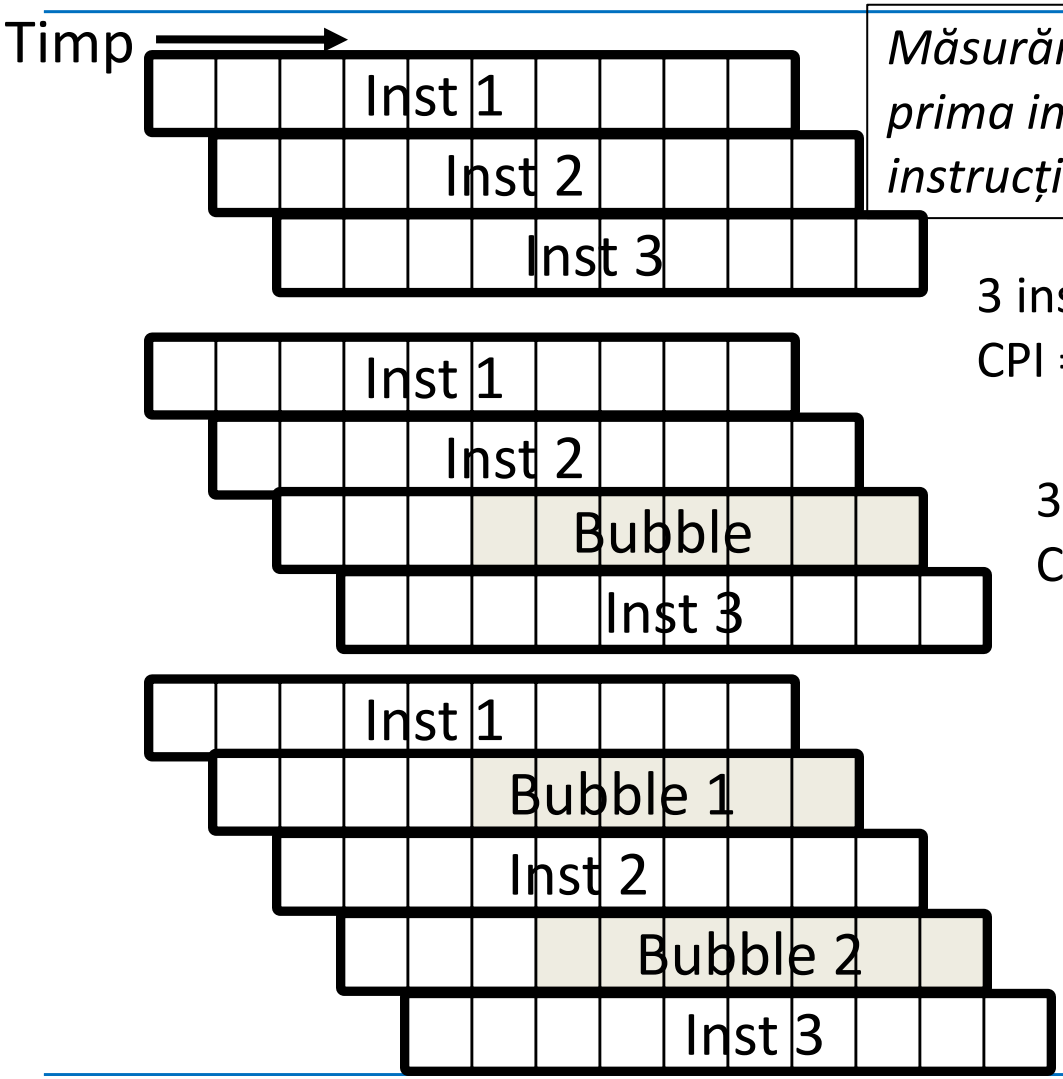
Căi de date complet separate



Mai este nevoie de un semnal de stall ?

$$stall = (rs1_D = ws_E).(opcode_E = LW_E).(ws_E != 0).re1_D + (rs2_D = ws_E).(opcode_E = LW_E).(ws_E != 0).re2_D$$

Exemple de CPI pentru b.a.



Măsurăm din momentul în care termină prima instrucțiune până când ultima instrucțiune își incheie execuția.

3 instrucțiuni termină în 3 cicli
CPI = 3/3 =1

3 instrucțiuni termină în 4 cicli
CPI = 4/3 = 1.33

3 instrucțiuni termină în 5 cicli
CPI = 5/3 = 1.67

Soluționarea hazardelor de date (3)

Strategia 3: Speculează legat de dependență!

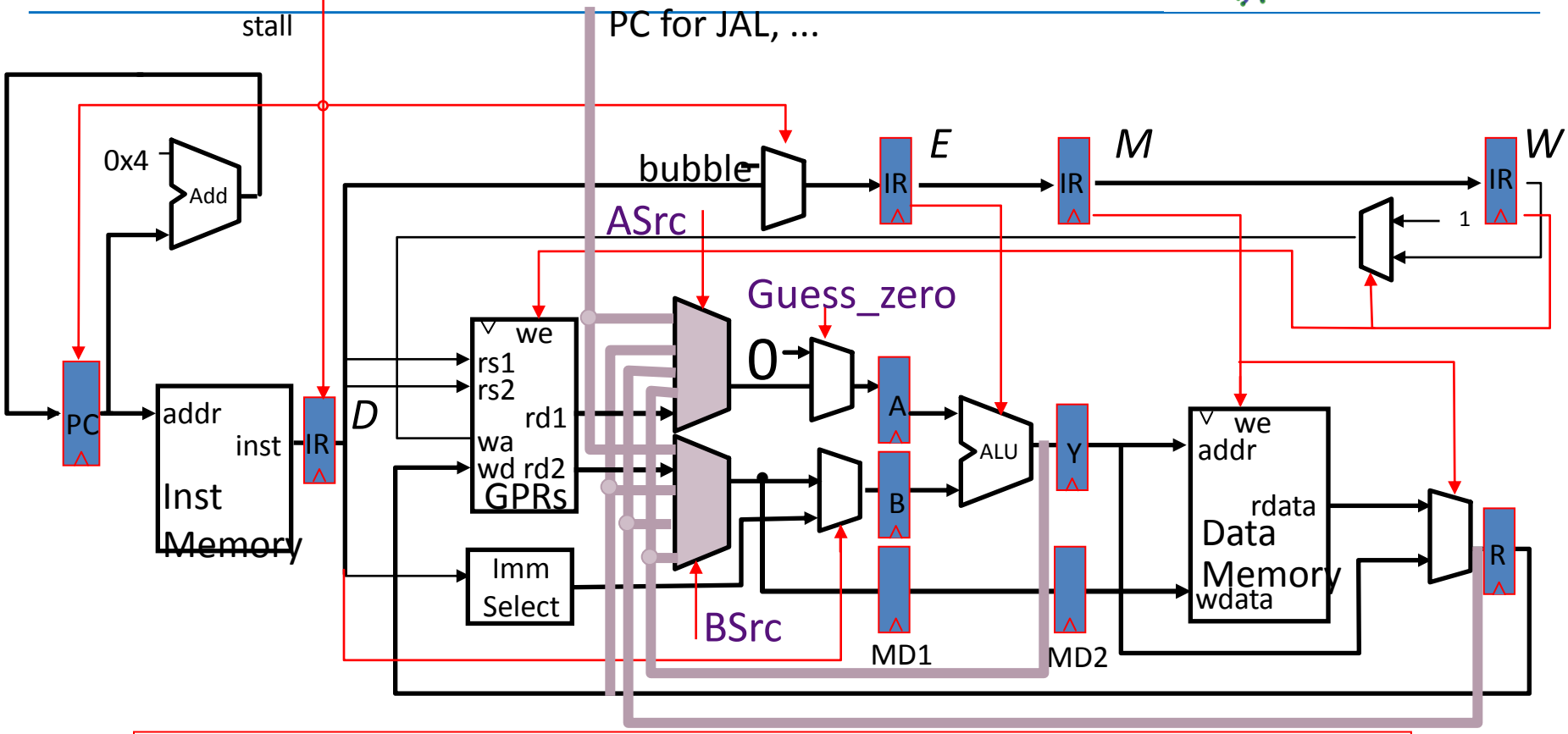
Două cazuri:

Am ghicit corect → nu face nimic

Am ghicit incorect → kill și restart

…. Vom vedea mai târziu exemple de astfel de comportament în procesoare mai complexe.

Speculăm ca valoarea pentru load=zero



$$Guess_zero = (rs1_D = ws_E).(opcode_E = LW_E).(ws_E != 0).re1_D$$

De asemenea, avem nevoie de circuite adiționale pentru a ne aduce aminte că a fost o presupunere și să golim b.a. dacă load != zero!

Nu este indicată o astfel de implementare în practică – de ce?

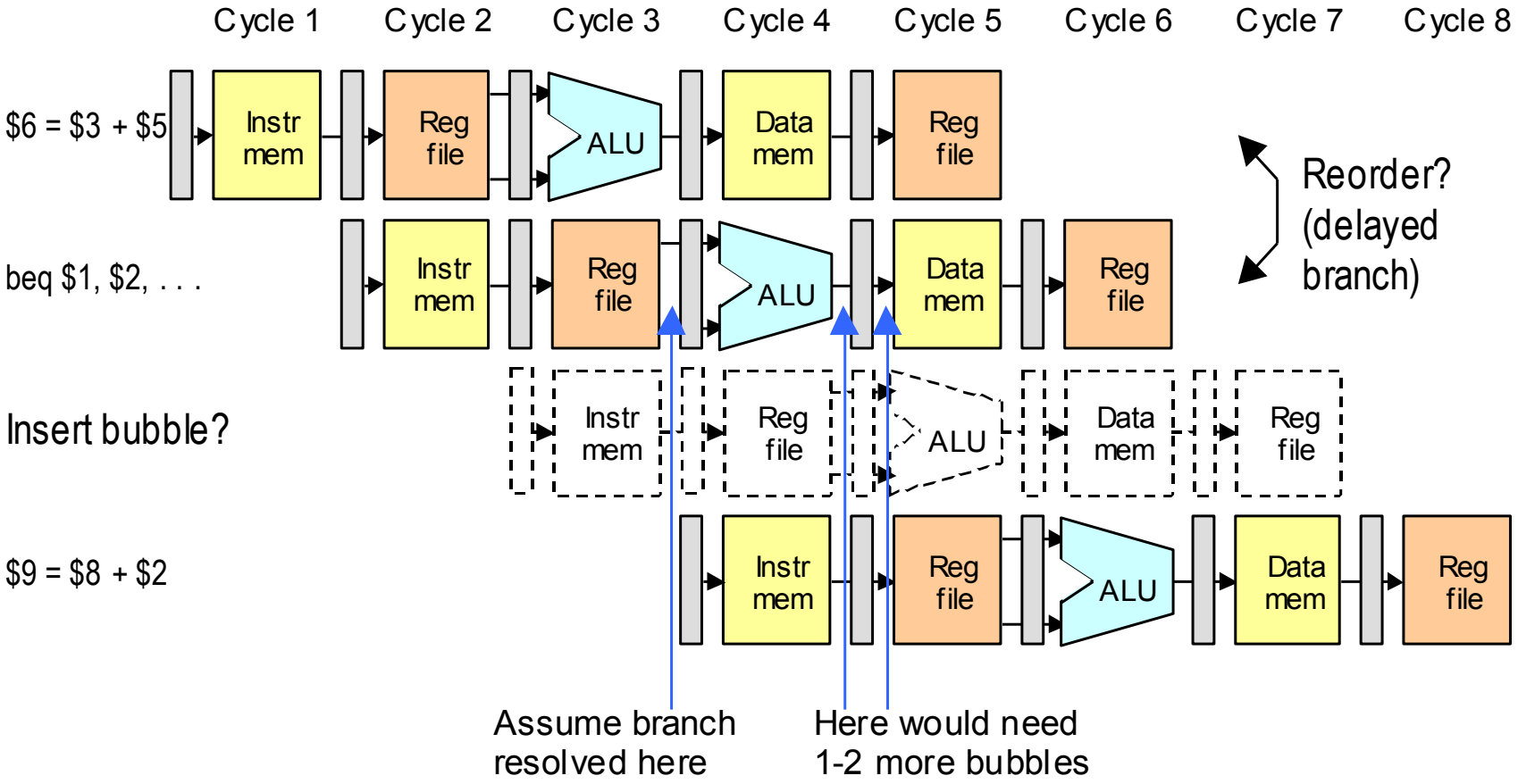
Hazarde de control

Embedded Systems Laboratory

De ce avem nevoie pentru a calcula următorul PC?

- Pentru Jump-uri
  - Opcode, PC și offset
- Pentru Jump Register
  - Opcode, Register value și PC
- Pentru Branch-uri Condiționale
  - Opcode, Register (pentru condiție), PC și offset
- Pentru toate celalalte instrucțiuni
  - Opcode și PC (și trebuie să știm dinainte că nu este una dintre cele de mai sus)

Hazarde de control într-o b.a.



Hazard de control datorat unui branch condițional.

Bule în calculul PC

timp

	t0	t1	t2	t3	t4	t5	t6	t7	. . . .		
(I_1) x1 ← x0 + 10	IF_1	ID_1	EX_1	MA_1	WB_1						
(I_2) x3 ← x2 + 17		IF_2	IF_2	ID_2	EX_2	MA_2	WB_2				
(I_3)				IF_3	IF_3	ID_3	EX_3	MA_3	WB_3		
(I_4)						IF_4	IF_4	ID_4	EX_4	MA_4	WB_4

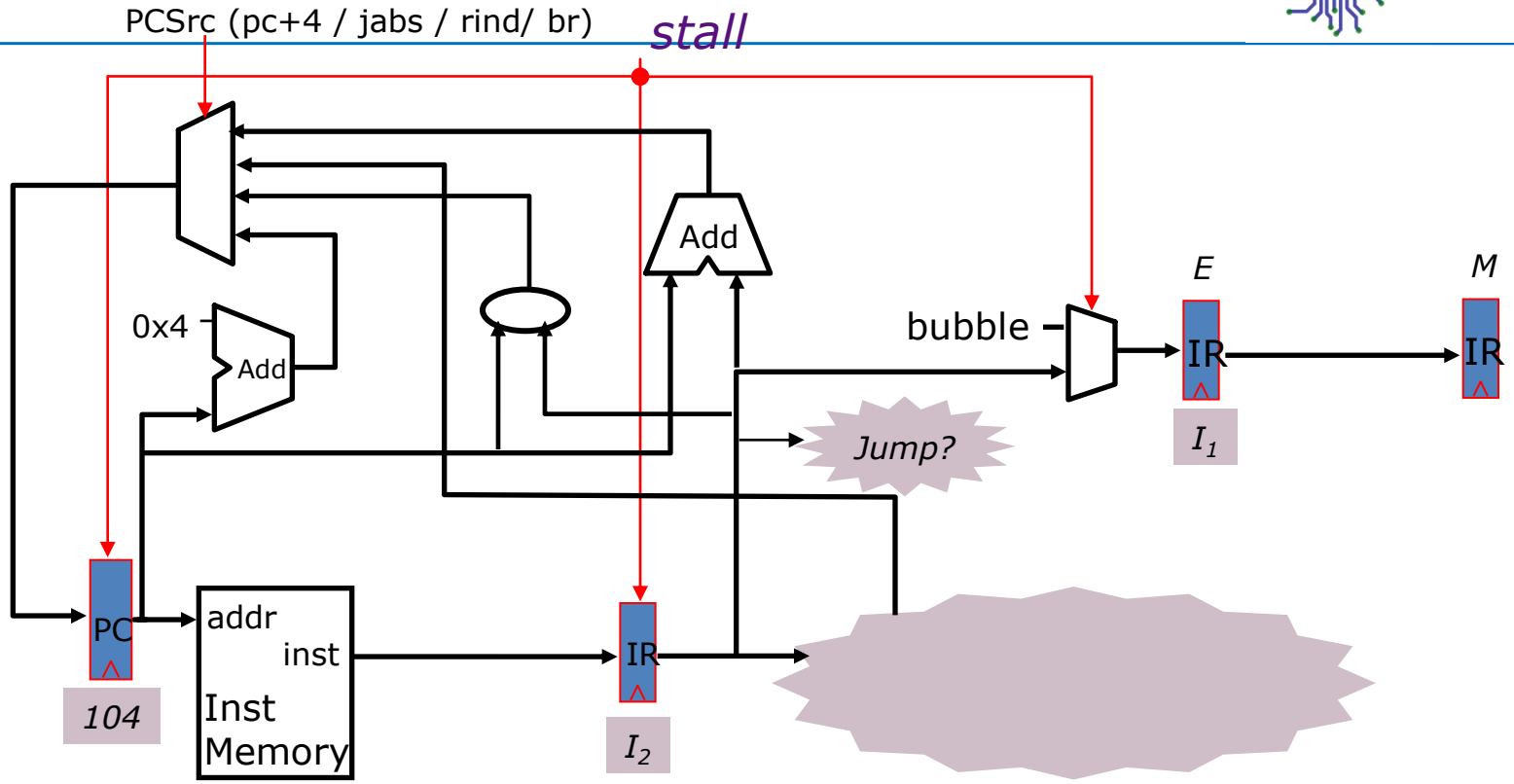
Resource Usage

time

	t0	t1	t2	t3	t4	t5	t6	t7	. . . .		
IF	I_1	-	I_2	-	I_3	-	I_4				
ID		I_1	-	I_2	-	I_3	-	I_4			
EX			I_1	-	I_2	-	I_3	-	I_4		
MA				I_1	-	I_2	-	I_3	-	I_4	
WB					I_1	-	I_2	-	I_3	-	I_4

- ⇒ *pipeline bubble*

Speculăm că următoarea adresă e PC+4



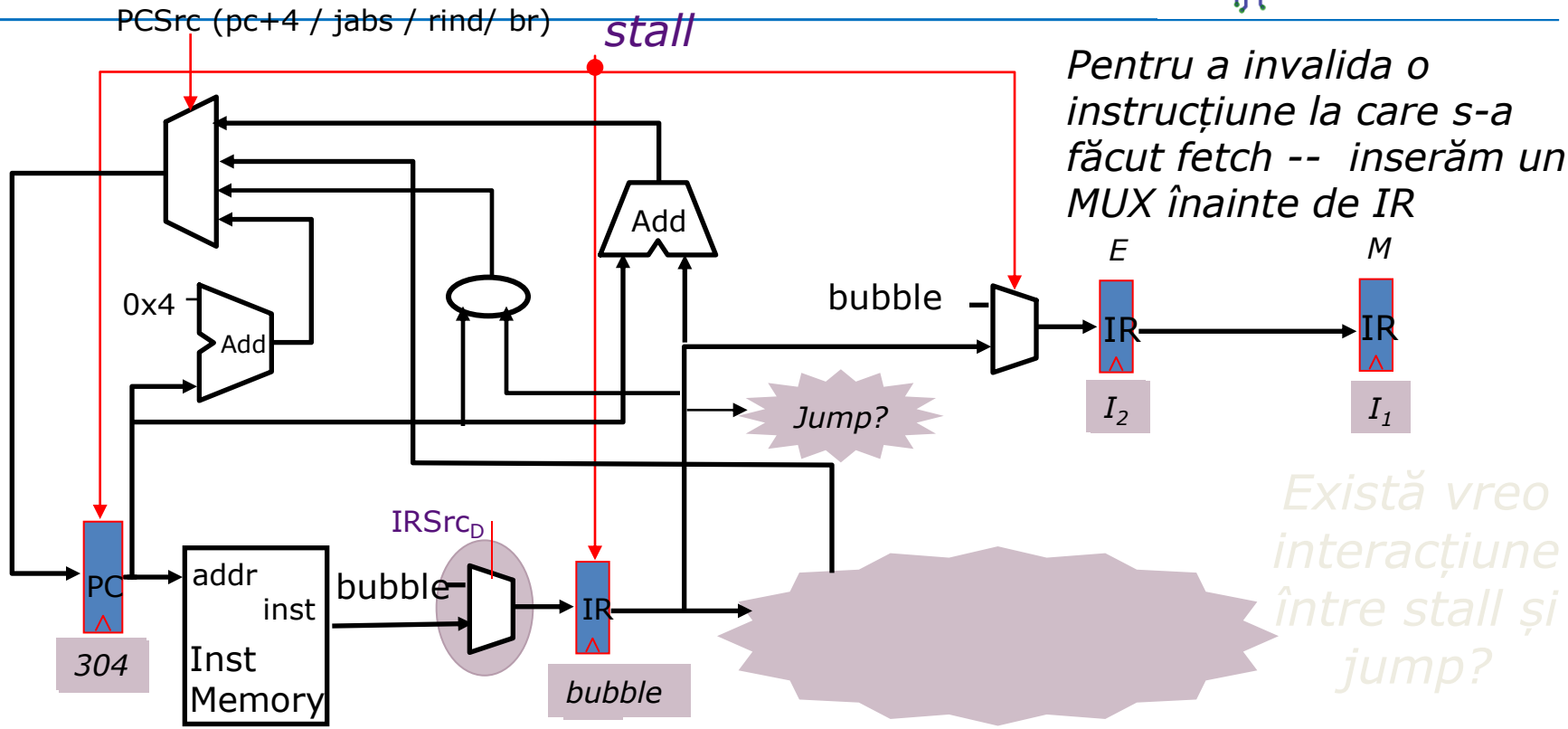
I_1	096	ADD
I_2	100	J 304
I_3	~~104~~	~~ADD~~
I_4	304	ADD

kill

O instrucțiune de jump omoară (nu stagnează) următoarea instrucțiune

Cum?

Implementarea salturilor în b.a.



Pentru a invalida o instrucțiune la care s-a făcut fetch -- inserăm un MUX înainte de IR

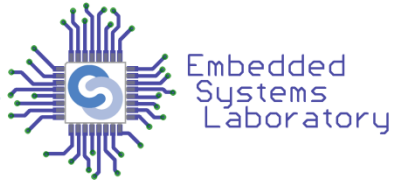
Există vreo interacțiune între stall și jump?

I_1	096	ADD
I_2	100	J 304
I_3	~~104~~	~~ADD~~
I_4	304	ADD

kill

$IRSrc_D$ = *Case* $opcode_D$
J, JAL ⇒ bubble
... ⇒ IM

Diagrame pipeline pentru Jump



timp

	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) 100: J 304		IF_2	ID_2	EX_2	MA_2	WB_2			
(I_3) 104: ADD			IF_3	-	-	-	-		
(I_4) 304: ADD				IF_4	ID_4	EX_4	MA_4	WB_4	

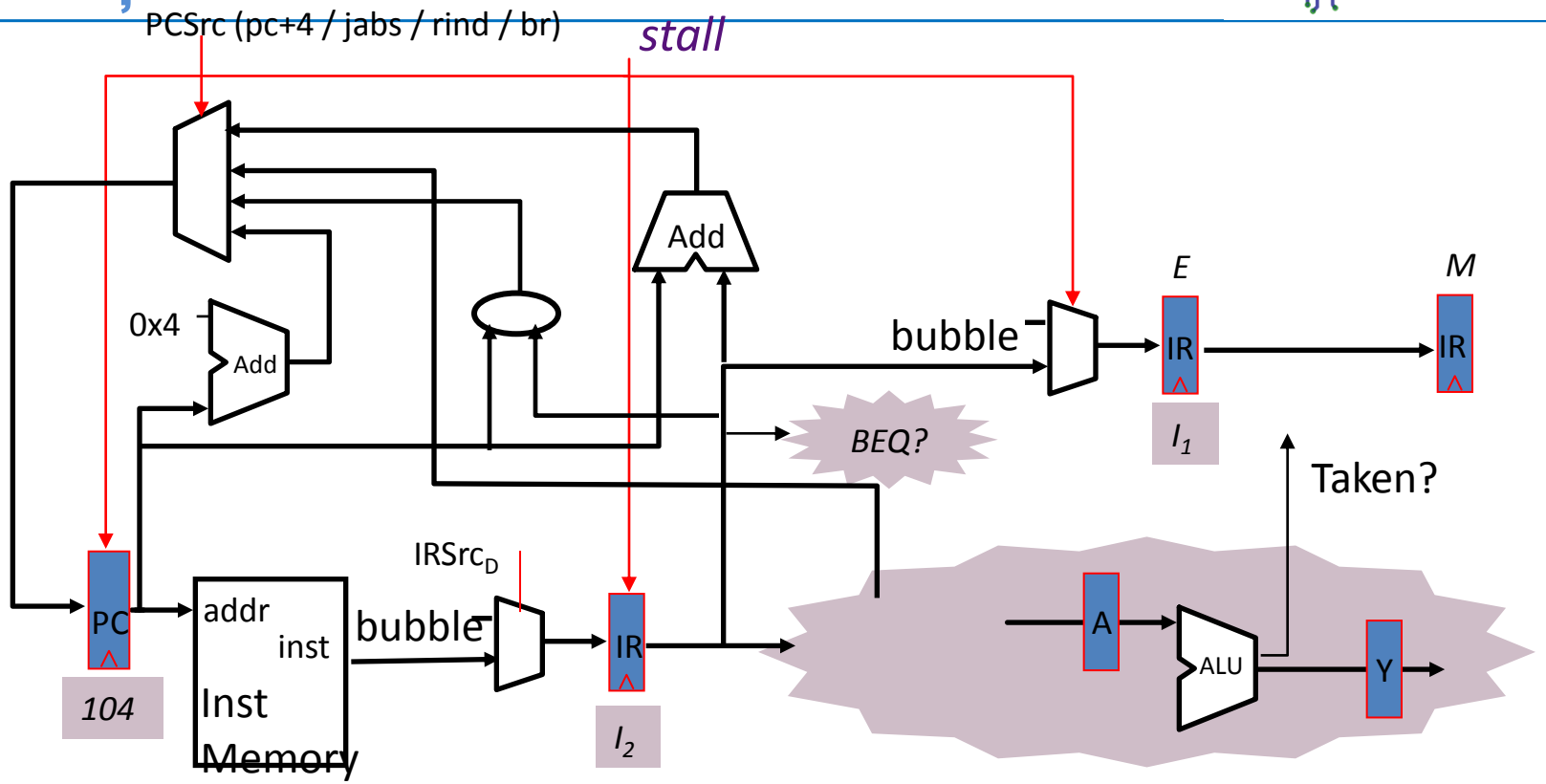
Resource Usage

timp

	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	-	I_4	I_5			
EX			I_1	I_2	-	I_4	I_5		
MA				I_1	I_2	-	I_4	I_5	
WB					I_1	I_2	-	I_4	I_5

- $\Rightarrow$ *pipeline bubble*

Implementarea în b.a. a salturilor condiționale

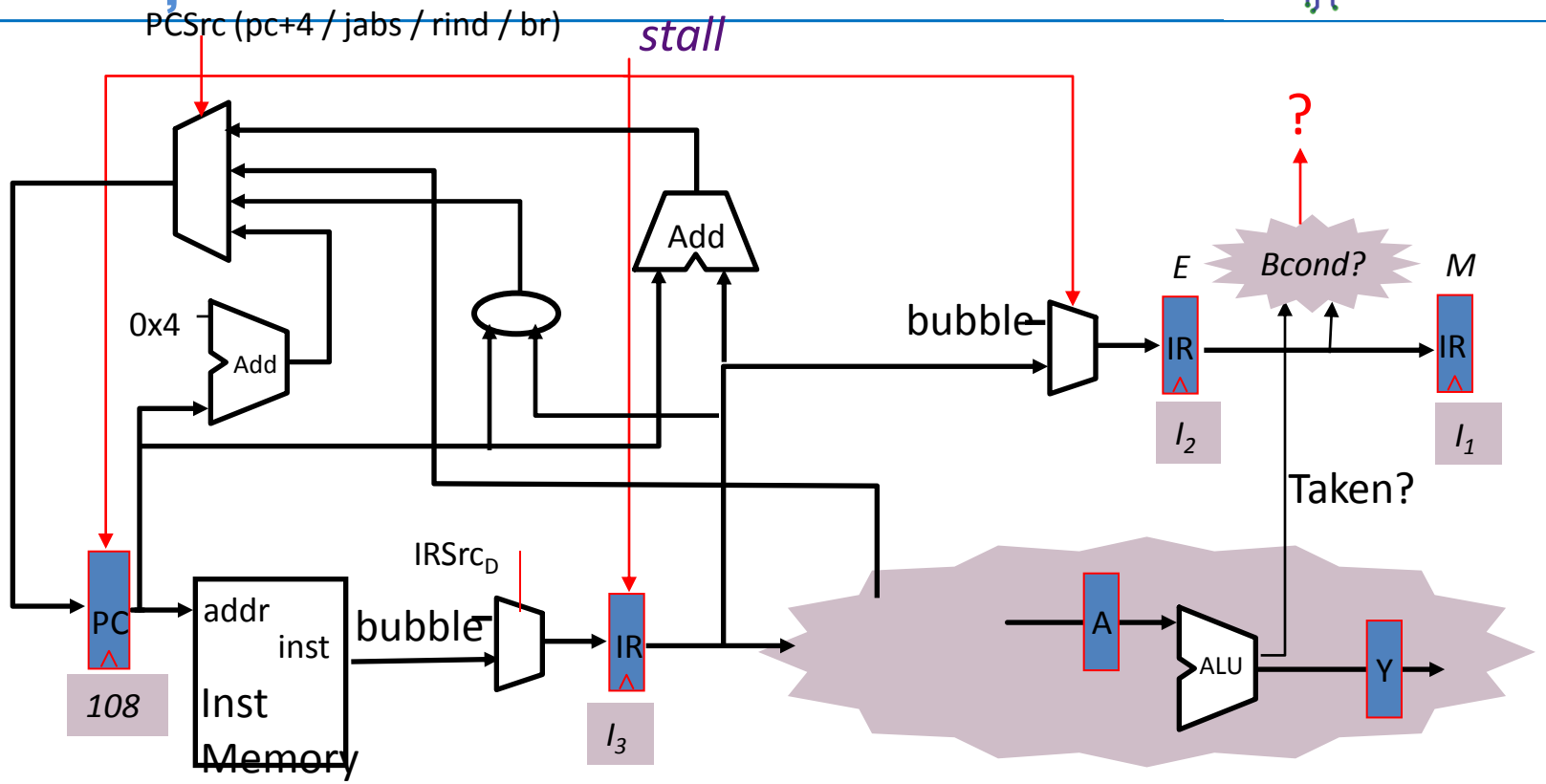


I_1	096	ADD
I_2	100	BEQ x1,x2 +200
I_3	104	ADD
I_4	304	ADD

Condiția de brach nu e cunoscută până la faza de execute

Ce acțiuni putem întreprinde în faza de decode?

Implementarea în b.a. a salturilor condiționale



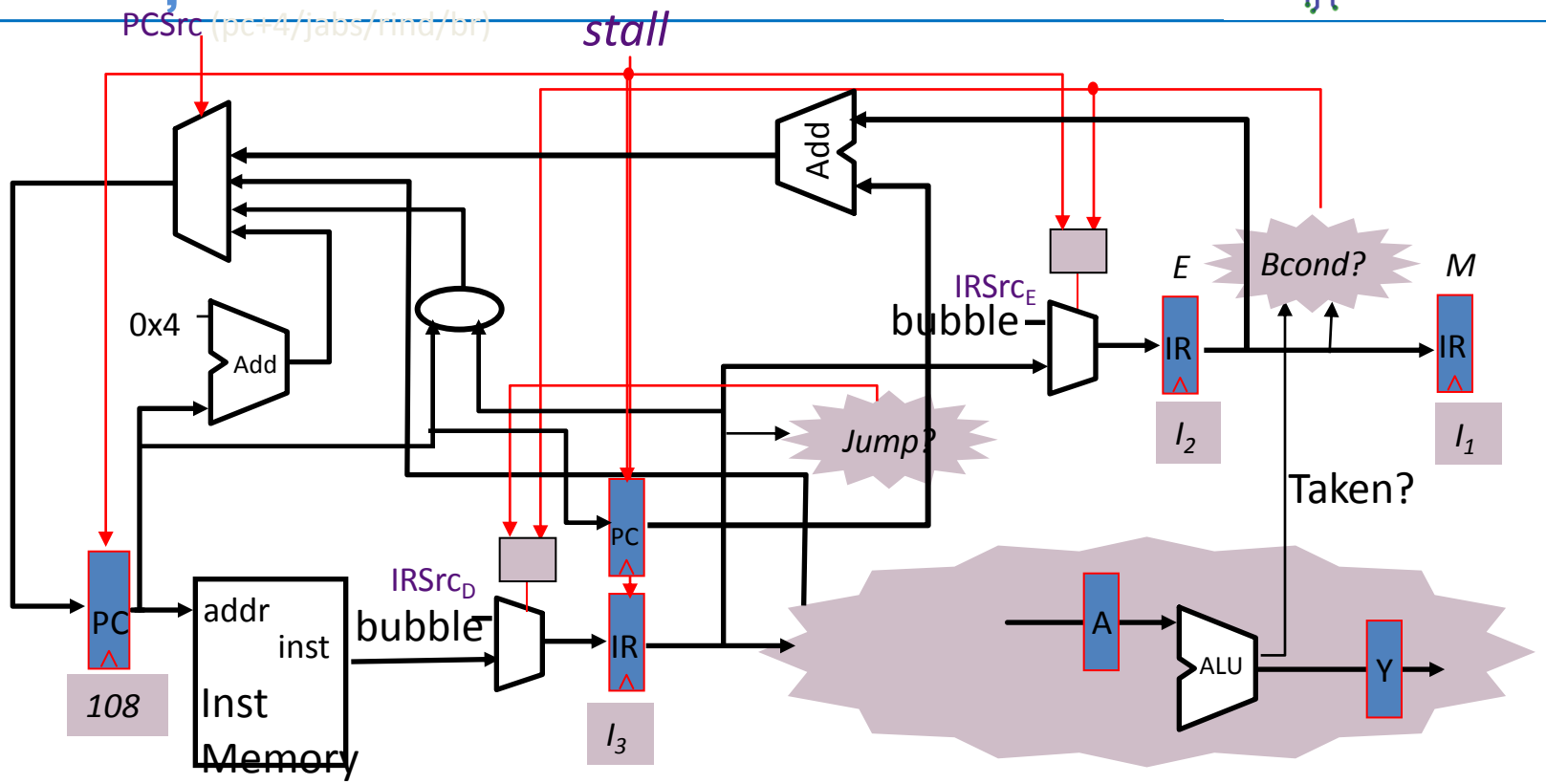
I_1	096	ADD
I_2	100	BEQ x1,x2 +200
I_3	104	ADD
I_4	304	ADD

Dacă branch-ul este valid

- Invalidează următoarele 2 instrucțiuni
- Instrucțiunea din faza de decode nu este validă

⇒ *semnalul stall nu este valid*

Implementarea în b.a. a salturilor condiționale



$I_{1:}$	096	ADD
$I_{2:}$	100	BEQ x1,x2 +200
$I_{3:}$	104	ADD
$I_{4:}$	304	ADD

Dacă branch-ul este valid

- Omoară următoarele 2 instrucțiuni
- Instrucțiunea din faza de decode nu este validă

⇒ *semnalul stall nu este valid*

Diagrame pentru execuția în b.a. a branch-urilor

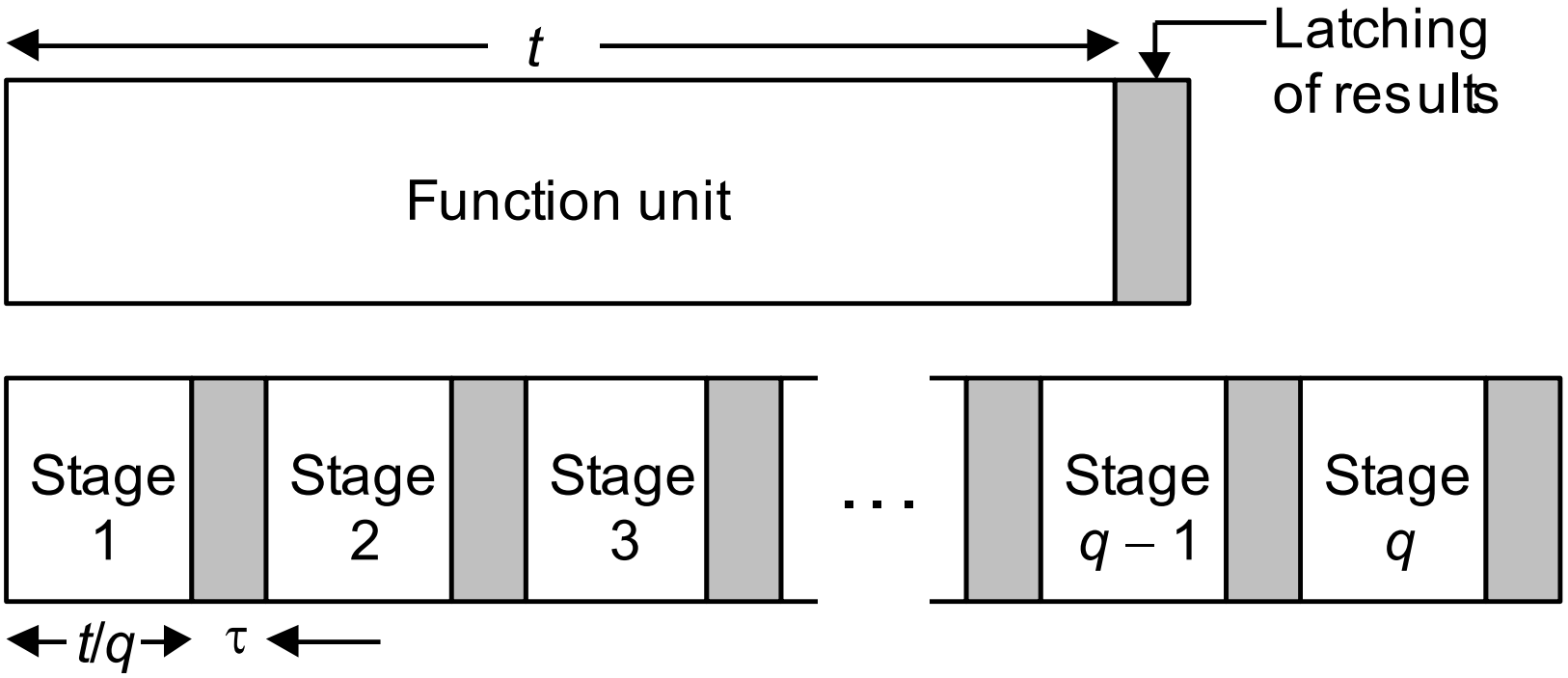
	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) 100: BEQ +200		IF_2	ID_2	EX_2	MA_2	WB_2			
(I_3) 104: ADD			IF_3	ID_3	-	-	-		
(I_4) 108:				IF_4	-	-	-	-	
(I_5) 304: ADD					IF_5	ID_5	EX_5	MA_5	WB_5

Resource Usage

	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	I_3	-	I_5			
EX			I_1	I_2	-	-	I_5		
MA				I_1	I_2	-	-	I_5	
WB					I_1	I_2	-	-	I_5

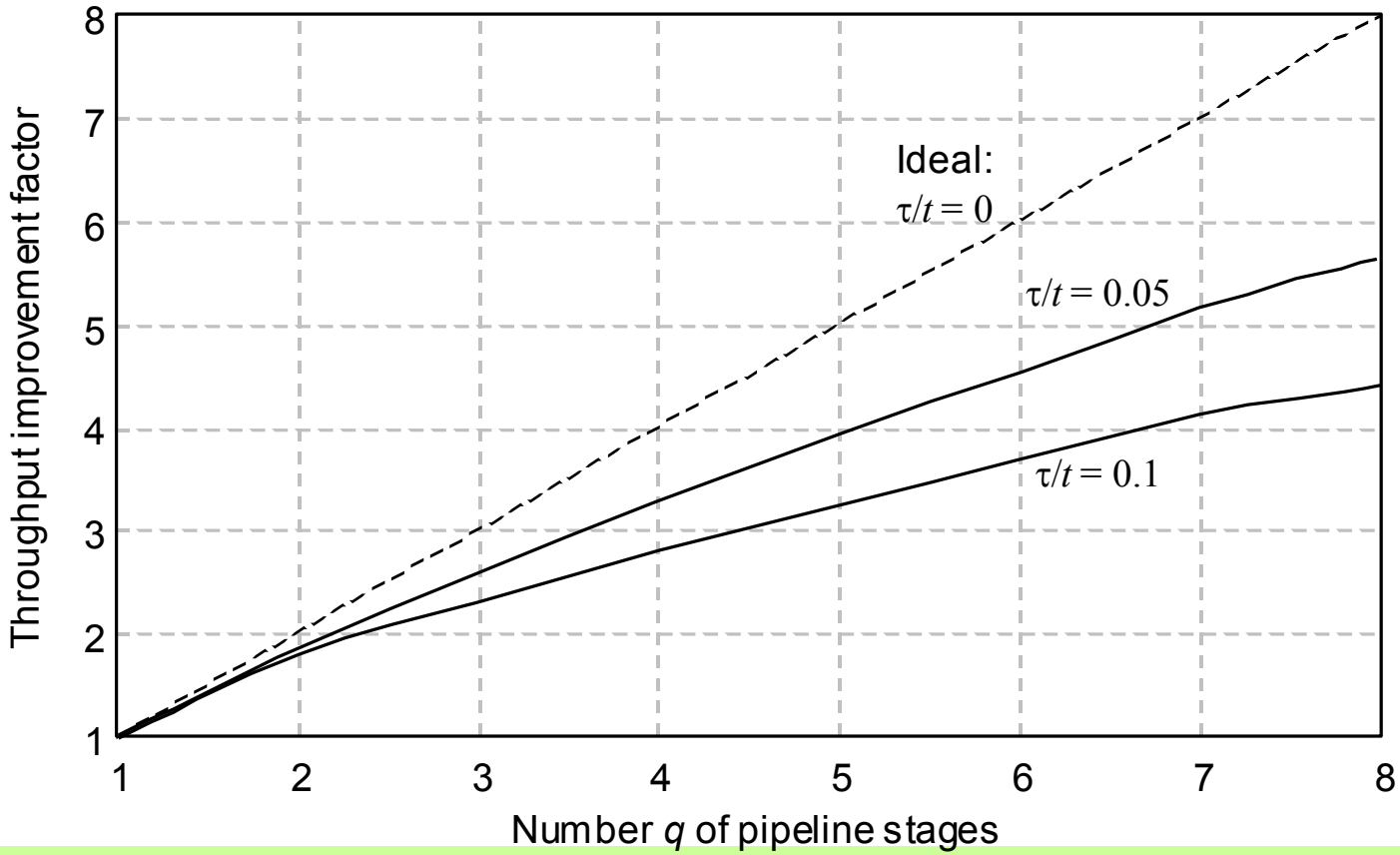
- $\Rightarrow$ *pipeline bubble*

Temporizarea și performanța unei b.a.



Forma în b.a. a unei unități funcționale cu întârzierile introduse de latch-uri

Creșterea de productivitate a unei b.a. cu q stagii



$$\frac{t}{t/q + \tau}$$

sau

$$\frac{q}{1 + q\tau/t}$$

Creșterea în productivitate în funcție de numărul de etape din b.a. și de întârzierile datorate propagării semnalelor prin diferitele latch-uri ale b.a.

Productivitatea b.a. afectată de hazarde

Embedded Systems Laboratory

Presupunem că o bulă trebuie să fie inserată din cauza unei dependențe read-after-load și după un branch.
Notăm cu β procentul de instrucțiuni care sunt urmate de o bulă.

Pipeline speedup = $$\frac{q}{(1 + q\tau / t)(1 + \beta)}$$

R-type	44%
Load	24%
Store	12%
Branch	18%
Jump	2%

Exemplu:

Calculați CPI efectiv pentru un procesor MIPS, presupunând ca un sfert din toate instrucțiunile de branch și load sunt urmate de bule

Soluție Procent de bule: $\beta = 0.25(0.24 + 0.18) = 0.105$

CPI = $1 + \beta = 1.105$ (foarte aproape de valoarea ideală 1)

Acknowledgements

- These slides contain material developed and copyright by:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

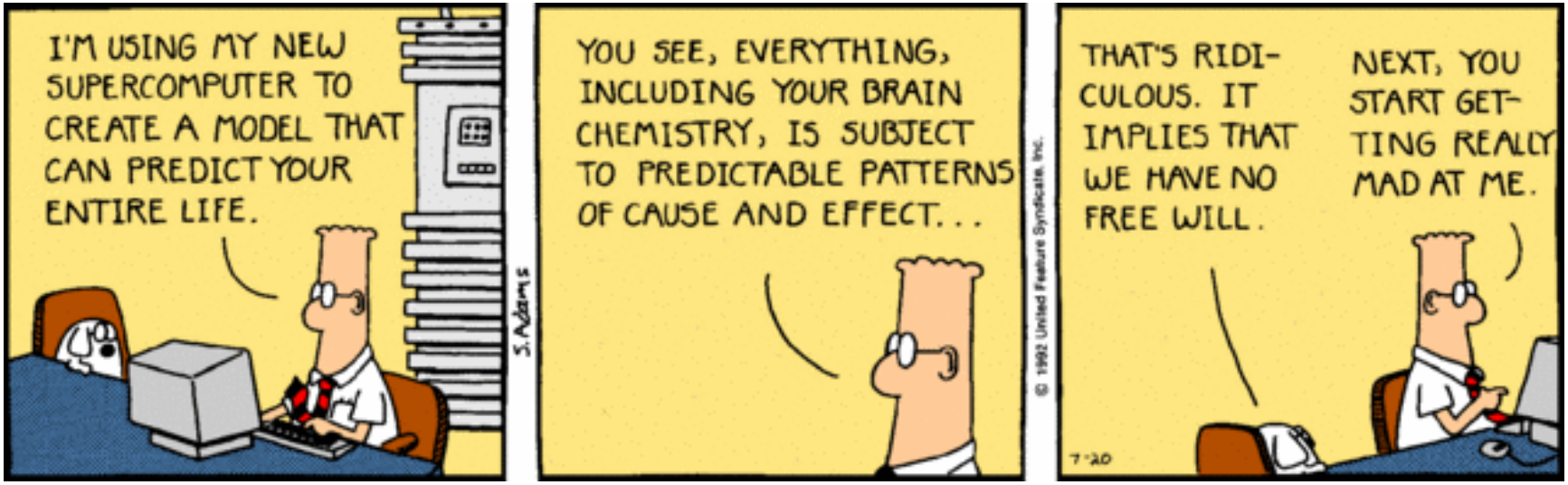
Calculatoare Numerice

– Cursul 12 –

Banda de asamblare (2)

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Comic of the day



http://dilbert.com/strips/comic/1992-07-20/

Din cursul anterior

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} * \frac{\text{Cycles}}{\text{Instruction}} * \frac{\text{Time}}{\text{Cycle}}$$

Crește datorită bulelor din pipeline

Scade, deoarece sunt mai puține porți logice pe căile critice dintre flip-flop-uri

- Pipelining mărește frecvența de operare iar CPI cresc mai lent -> performanță mărită
- Implementarea în b.a. este complicată de HAZARDE:
  - Hazarde structurale (două istrucțiuni vor aceeași resursă hardware)
  - Hazarde de date (instrucțiunea anterioară produce o valoare de care are nevoie instrucțiunea în curs de execuție)
  - Hazarde de control(instrucțiunile schimbă semnalele de control, e.g., branches & exceptions)
- Tehnici de rezolvare:
  1) Interlock-uri (oprim execuția noilor instrucțiuni până când vechile instrucțiuni ies din b.a. și fac write-back)
  2) Bypass (transferăm valoarea vechii instrucțiuni către cea nouă imediat ce rezultatul este disponibil)
  3) Speculează (ghicește efectul instrucțiunii prealabile)

Hazarde de control

De ce avem nevoie să calculăm următorul PC?

- Pentru Jump-uri
  - Opcode, PC & offset
- Pentru Jump Register
  - Opcode, Register value și PC
- Pentru Conditional Branches
  - Opcode, Register (pentru condiție), PC și offset
- Pentru toate celelalte instrucțiuni
  - Opcode și PC (și trebuie să știm că nu sunt cele de sus)

Calculul PC - bule

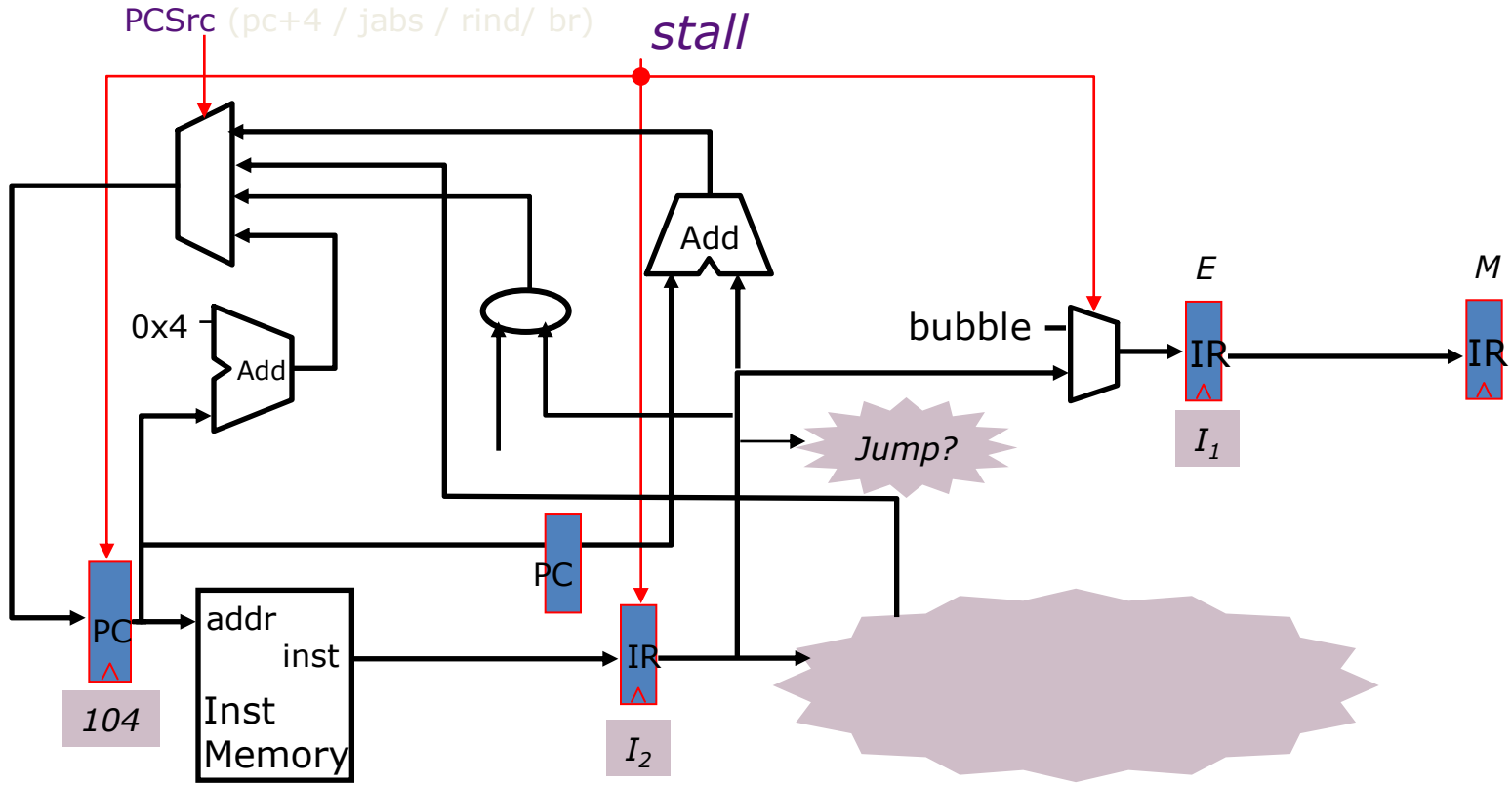
	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .		
(I_1) x1 ← x0 + 10	IF_1	ID_1	EX_1	MA_1	WB_1						
(I_2) x3 ← x2 + 17		IF_2	IF_2	ID_2	EX_2	MA_2	WB_2				
(I_3)				IF_3	IF_3	ID_3	EX_3	MA_3	WB_3		
(I_4)						IF_4	IF_4	ID_4	EX_4	MA_4	WB_4

Resource Usage

	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .		
IF	I_1	-	I_2	-	I_3	-	I_4				
ID		I_1	-	I_2	-	I_3	-	I_4			
EX			I_1	-	I_2	-	I_3	-	I_4		
MA				I_1	-	I_2	-	I_3	-	I_4	
WB					I_1	-	I_2	-	I_3	-	I_4

- ⇒ *pipeline bubble*

Speculăm că următoarea adresă e PC+4



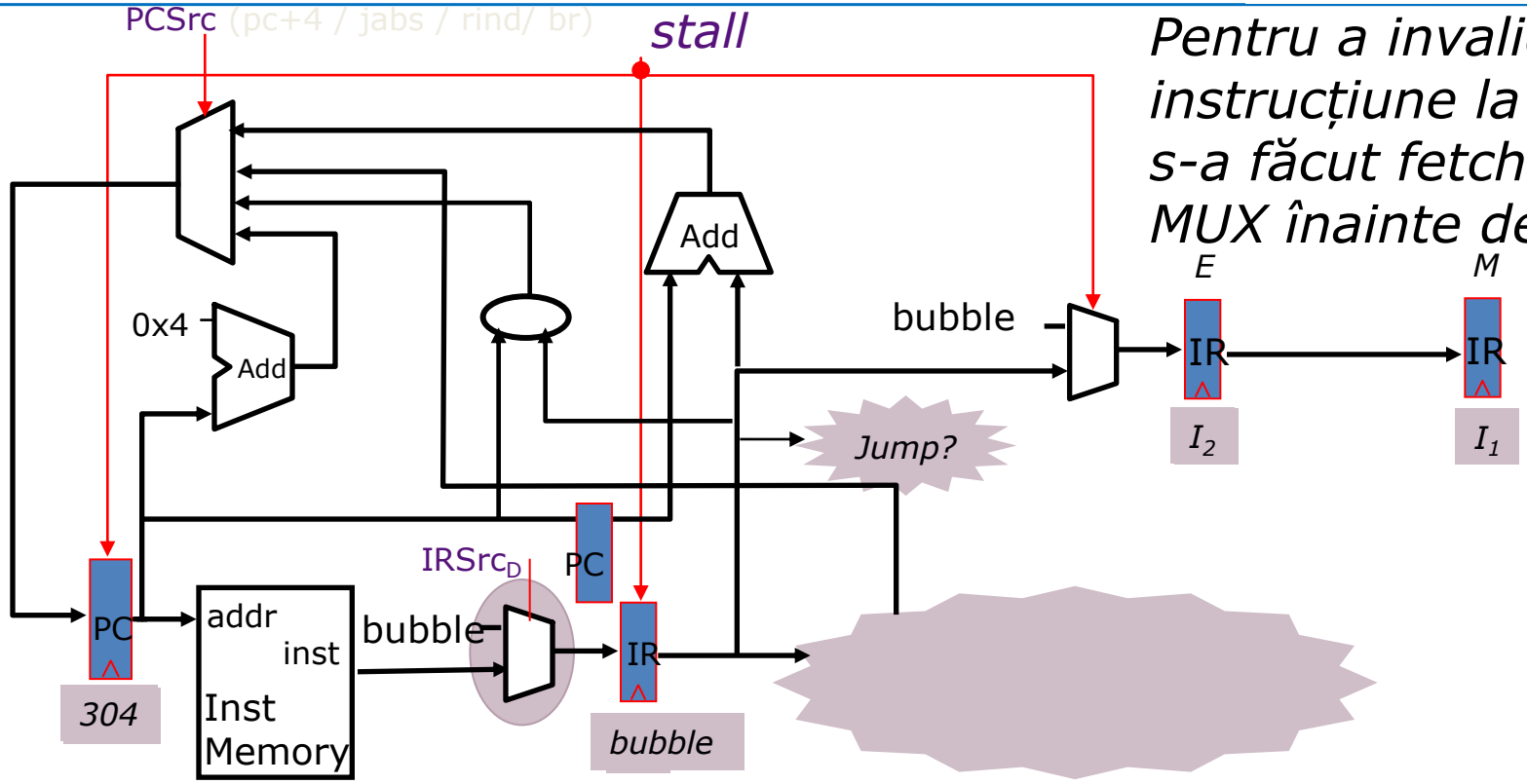
I_1	096	ADD
I_2	100	J 304
I_3	~~104~~	~~ADD~~
I_4	304	ADD

kill

Un jump invalidează (nu oprește) instrucțiunea următoare

Cum?

Implementarea de jump-uri în b.a.



Pentru a invalida o instrucțiune la care s-a făcut fetch -- MUX înainte de IR

I_1	096	ADD
I_2	100	J 304
I_3	~~104~~	~~ADD~~
I_4	304	ADD

kill

$IRSrc_D$ = *Case* $opcode_D$
- J, JAL ⇒ bubble
- ... ⇒ IM

Diagrama Jump în b.a.

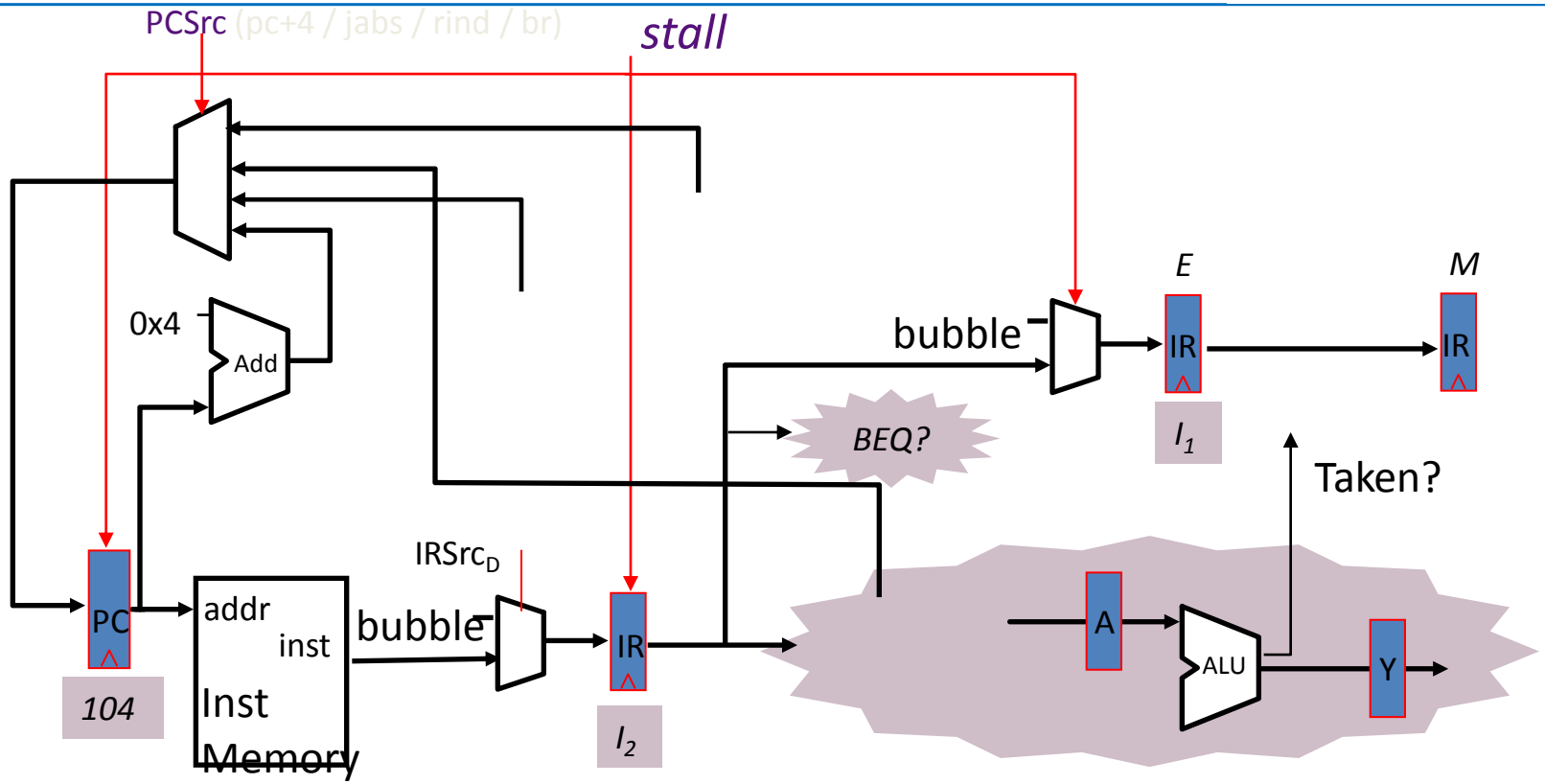
	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) 100: J 304		IF_2	ID_2	EX_2	MA_2	WB_2			
(I_3) 104: ADD			IF_3	-	-	-	-		
(I_4) 304: ADD				IF_4	ID_4	EX_4	MA_4	WB_4	

Resource Usage

	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	-	I_4	I_5			
EX			I_1	I_2	-	I_4	I_5		
MA				I_1	I_2	-	I_4	I_5	
WB					I_1	I_2	-	I_4	I_5

- $\Rightarrow$ *pipeline bubble*

Implementarea în b.a. a branch-urilor

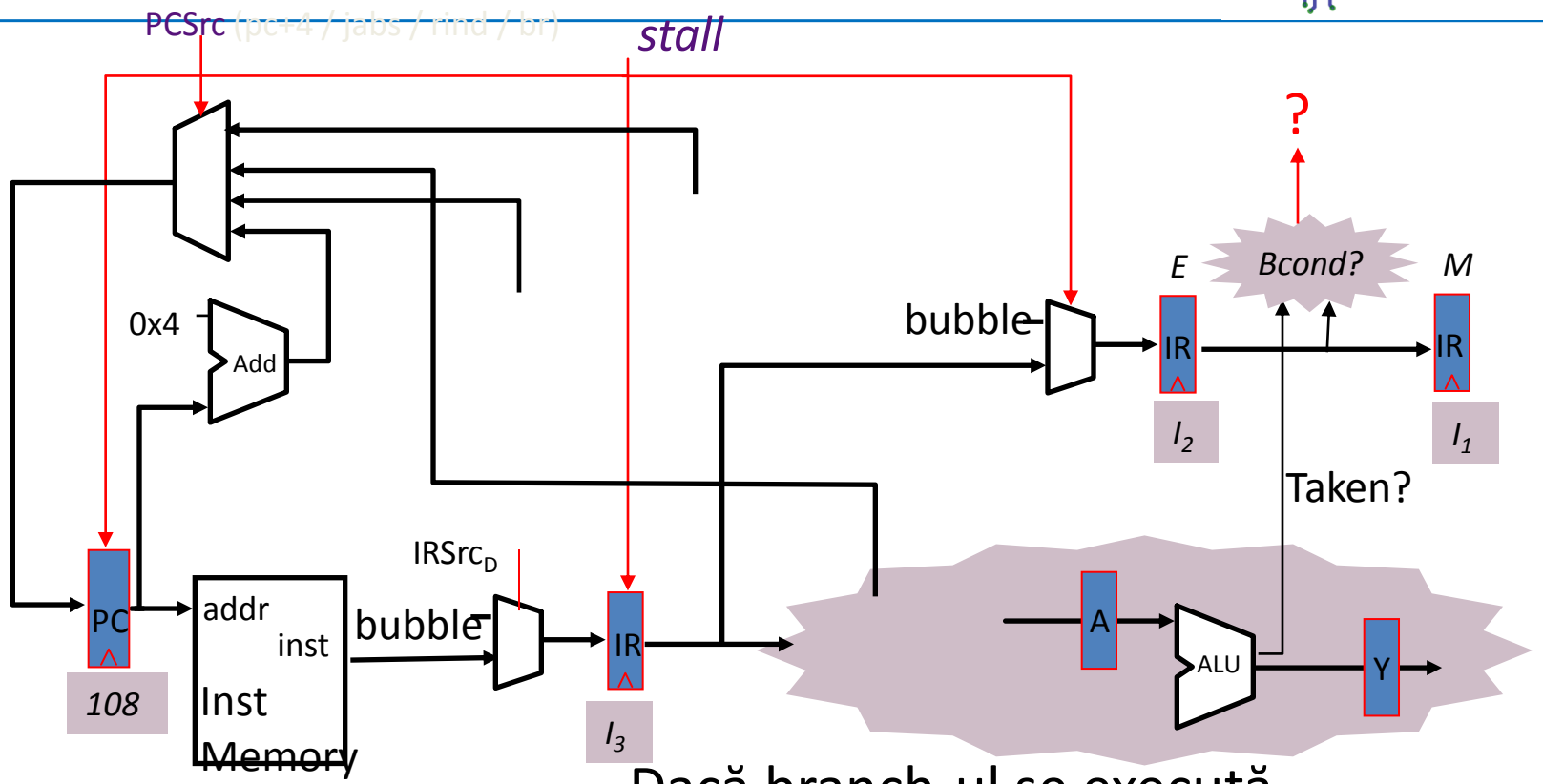


I_1 096 ADD
I_2 100 BEQ x1,x2 +200
I_3 104 ADD
I_4 300 ADD

Condiția de branch nu e cunoscută până la faza de execute

Ce putem face în faza de decode?

Implementarea în b.a. a branch-urilor



I_1 096 ADD

I_2 100 BEQ x1,x2 +200

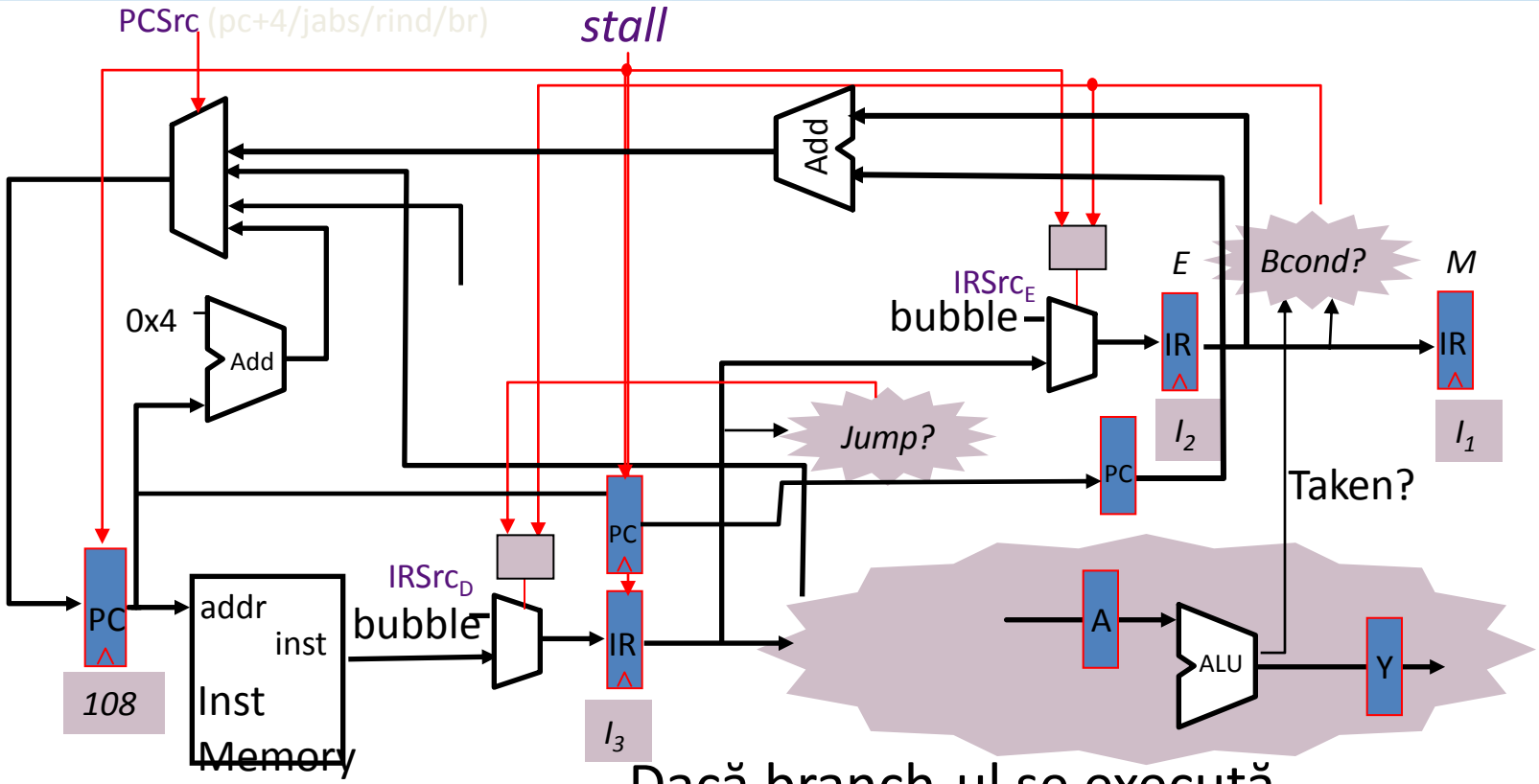
I_3 104 ADD

I_4 300 ADD

Dacă branch-ul se execută

- Invalidăm următoarele 2 instrucțiuni
- instrucțiunea din faza de decode nu e validă $\Rightarrow$ semnalul de stall nu e valid

Implementarea în b.a. a branch-urilor



$I_{1:}$ 096 ADD

$I_{2:}$ 100 BEQ x1,x2 +200

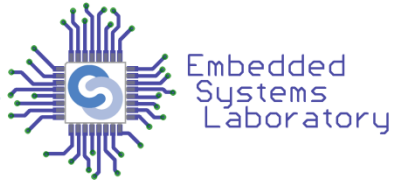
$I_{3:}$ 104 ADD

$I_{4:}$ 300 ADD

Dacă branch-ul se execută

- Invalidăm următoarele 2 instrucțiuni
- instrucțiunea din faza de decode nu e validă ⇒ semnalul de stall nu e valid

Diagrame în b.a. pentru branch



timp

	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) 100: BEQ +200		IF_2	ID_2	EX_2	MA_2	WB_2			
(I_3) 104: ADD			IF_3	ID_3	-	-	-		
(I_4) 108:				IF_4	-	-	-	-	
(I_5) 300: ADD					IF_5	ID_5	EX_5	MA_5	WB_5

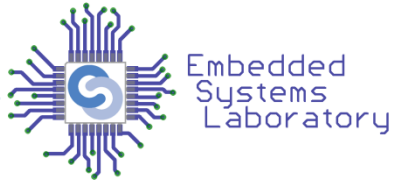
Resource Usage

timp

	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	I_3	-	I_5			
EX			I_1	I_2	-	-	I_5		
MA				I_1	I_2	-	-	I_5	
WB					I_1	I_2	-	-	I_5

- ⇒ *pipeline bubble*

Folosim branch-uri mai simple (e.g.,comparăm un registru cu zero) cu comparația în faza de decode



	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) 100: BEQZ +200		IF_2	ID_2	EX_2	MA_2	WB_2			
(I_3) 104: ADD			IF_3	-	-	-	-		
(I_4) 300: ADD				IF_4	ID_4	EX_4	MA_4	WB_4	

Resource Usage

	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	-	I_4	I_5			
EX			I_1	I_2	-	I_4	I_5		
MA				I_1	I_2	-	I_4	I_5	
WB					I_1	I_2	-	I_4	I_5

- $\Rightarrow$ *pipeline bubble*

Branch Delay Slots
(expunem hazardul de control software-ului)

- Schimbăm semnatica ISA ca instrucțiunea de după un jump sau branch este executată întotdeauna
  - Dă compilatorului oportunitatea de-a insera o instrucțiune utilă unde in mod normal ar fi fost o bulă în b.a.

I_1	096	ADD
I_2	100	BEQZ r1, +200
I_3	104	ADD
I_4	300	ADD

Delay slot instruction executată indiferent de rezultatul branch-ului

Diagrama în b.a. pentru branch
(branch delay slot)

	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) 100: BEQZ +200		IF_2	ID_2	EX_2	MA_2	WB_2			
(I_3) 104: ADD			IF_3	ID_3	EX_3	MA_3	WB_3		
(I_4) 300: ADD				IF_4	ID_4	EX_4	MA_4	WB_4	

Resource Usage

timp	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4					
ID		I_1	I_2	I_3	I_4				
EX			I_1	I_2	I_3	I_4			
MA				I_1	I_2	I_3	I_4		
WB					I_1	I_2	I_3	I_4	

De ce nu putem să executăm tot timpul o instrucțiune nouă la fiecare ciclu de ceas? (CPI>1)

- Un sistem de bypass complet s-ar putea să fie prea dificil de implementat (și prea costisitor)
  - De obicei, sunt rutate cele mai utilizate căi de bypass
  - Pentru alte rute este posibil să avem nevoie de să mărim ciclul de ceas, ceea ce ar contracara beneficiile date de reducerea CPI
- Un load are latență de doi cicli
  - O instrucțiune imediat după load nu poate să utilizeze rezultatul load-ului
  - MIPS-I ISA definește *load delay slots*, un hazard vizibil software (compilatorul planifică o instrucțiune independentă sau inserează un NOP pentru a evita hazardul). Înlăturat la MIPS-II (prin adăugarea de pipeline interlocks în hardware)
    - MIPS:"**M**icroprocessor without **I**nterlocked **P**ipeline **S**tages"
- Branch-urile condiționale pot cauza bule
  - Invalidează următoarea instrucțiune dacă nu avem delay slots.

RISC-V Branches & Jumps

Embedded Systems Laboratory

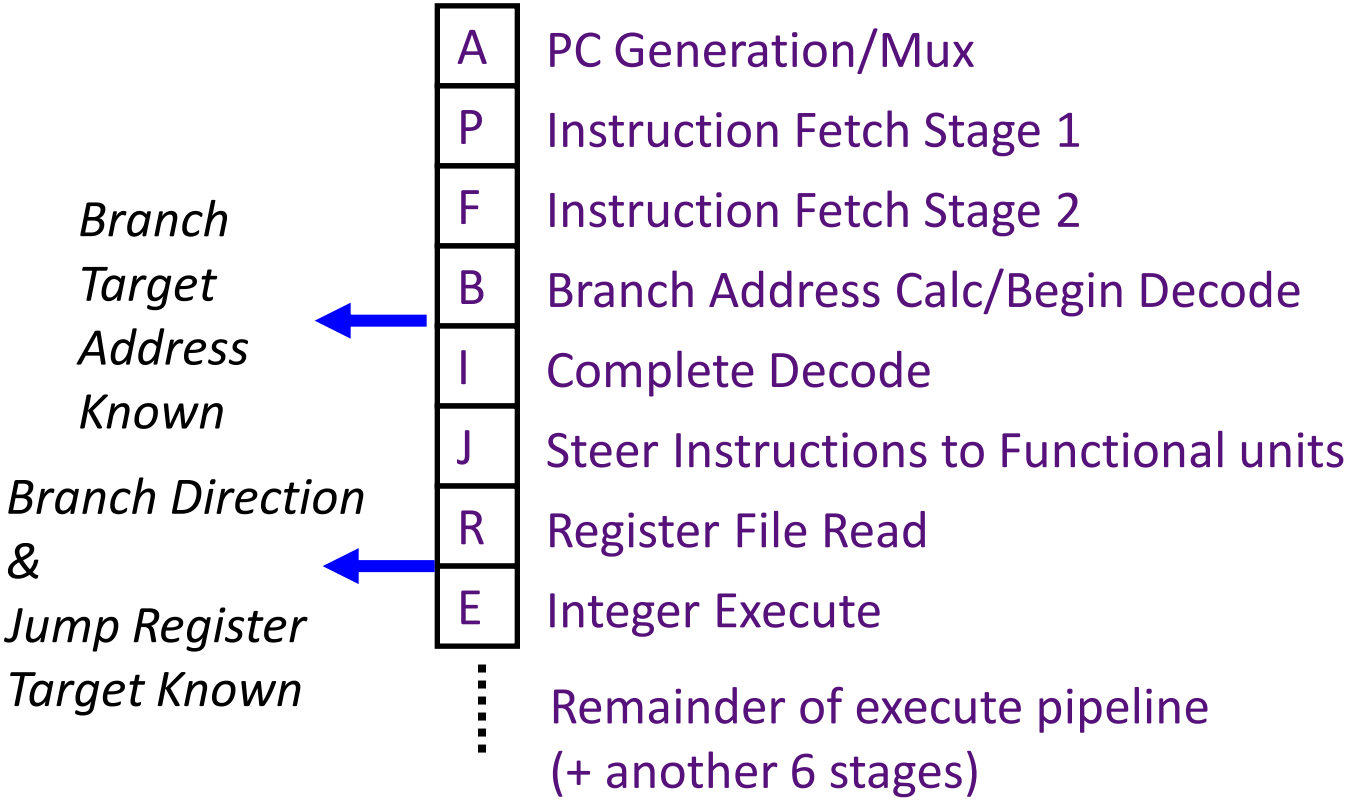
Fiecare instruction fetch depinde de una sau două informații de la instrucțiunea precedentă:

1) Este instrucțiunea precedentă un branch valid?
2) Dacă da, care e adresa țintă?

Instruction	*Taken known?*	*Target known?*
J	After Inst. Decode	After Inst. Decode
JR	After Inst. Decode	After Reg. Fetch
B<cond.>	After Execute	After Inst. Decode

Penalizările pentru branch în b.a. moderne

Etapele pentru instruction fetch la UltraSPARC-III (in-order issue, 4-way superscalar, 750MHz, 2000)



Reducerea penalizărilor în lanțul de control

- Soluții software
  - Eliminarea branch-urilor - loop unrolling
    - Mărește lungimea programului
  - Reducerea timpului de decizie – instruction scheduling
    - Calculează condiția pentru branch cât de curând posibil

- Soluții hardware
  - Găsește altceva de făcut - delay slots
    - Umple bulele din b.a. cu instrucțiuni utile (necesită cooperare din partea software-ului)
  - Speculează - branch prediction
    - Execuția speculativă a instrucțiunilor de după branch

Branch Prediction

Embedded Systems Laboratory

Motivație:

Penalizările pentru branch limitează profund performanța procesoarelor în b.a.

Predictoarele moderne au o acuratețe destul de mare (>95%) și pot reduce penalizările semnificativ

Suport hardware necesar:

Structuri pentru predicție:

- Branch history tables, branch target buffers, etc.

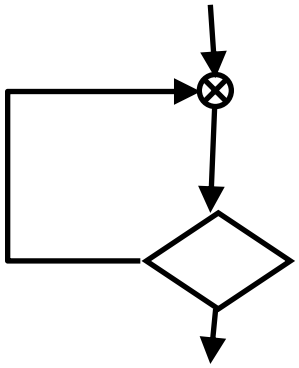
Mecanisme de recuperare în cazul unei predicții proaste:

- *Ține rezultatele calculelor separate de commit*
- Invalidează instrucțiunile de după branch din b.a.
- Reface starea pentru branch-ul respectiv

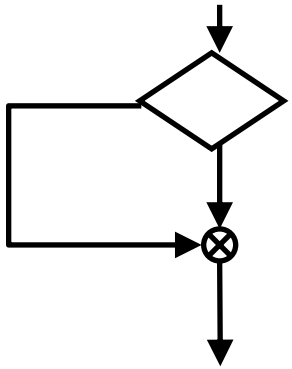
Static Branch Prediction

Probabilitatea totală ca un branch să fie valid este de ~60-70% dar:

înapoi
90%



înainte
50%



ISA poate să atașeze o semantică potrivită la direcțiile pentru branch-uri, e.g., Motorola MC88110

bne0 *(direcția preferată)* beq0 *(mai puțin folosită)*

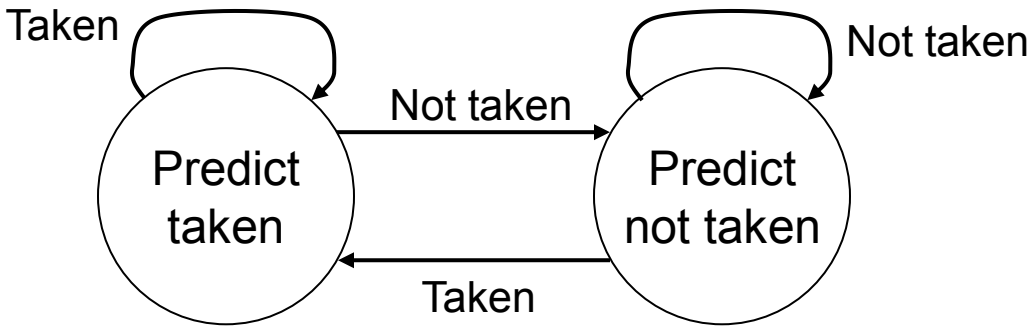
ISA poate permite alegerea arbitrară a direcției prezise static, e.g., HP PA-RISC, Intel IA-64 sunt de obicei ~80% precise

Dynamic Branch Prediction
- învățând din experiențele anterioare

Embedded Systems Laboratory

- Corelație temporală
  - Felul în care se execută un branch poate fi un bun indicator despre modul în care se va executa la următoarea trecere prin program.
- Corelație spațială
  - O structură cu mai multe branch-uri se poate executa într-o manieră corelată (cale preferată de execuție)

Predicția branch-urilor



Automat pentru predicția branch-urilor cu două stări.

Probleme cu această abordare:

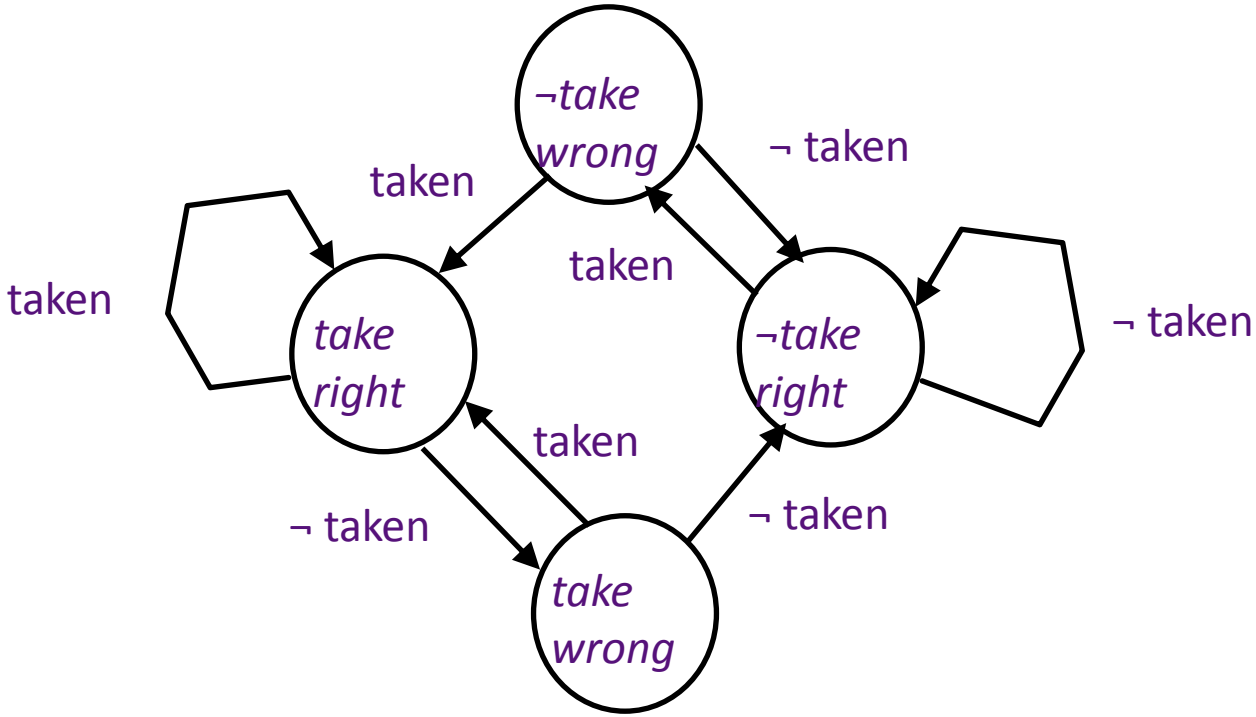
Dacă avem branch-uri într-o buclă, avem automat 2 previziuni greșite:

La prima iterație (bucla este repetată dar istoricul indică ieșirea din buclă)

La ultima interație (când bucla se termină dar istoricul arată că există repetiție)

Elemente de predicție a branch-urilor

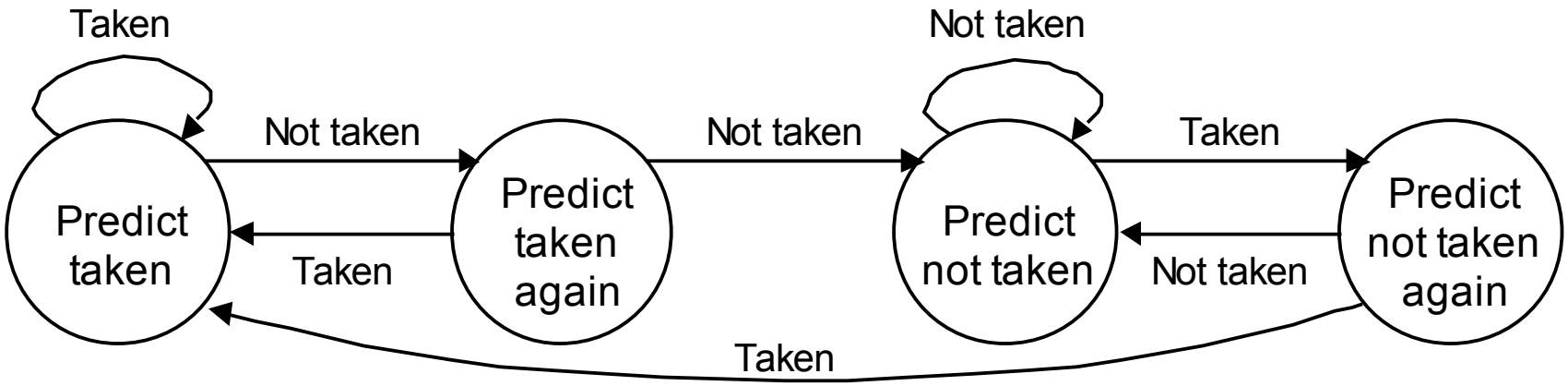
- Presupunem că avem doi biți BP per instrucțiune
- Schimbăm politica de predicție după două greșeli consecutive!



BP state:

(*predict* take/¬take) x (*last prediction* right/wrong)

Elemente de predicție a branch-urilor



Schemă de predicție cu patru stări

```
L1:  ----
     ----
L2:  ----
     ----
     br <c2> L2
     ----
     br <c1> L1
```

10 iter's

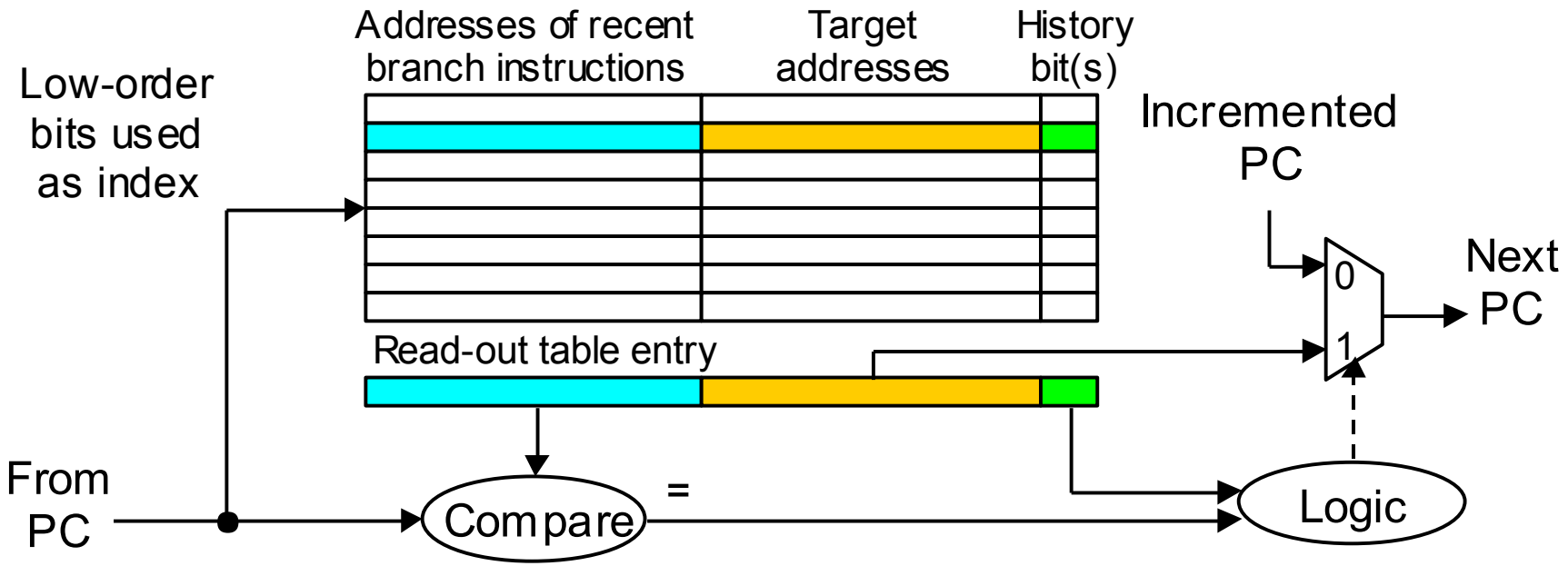
20 iter's

Impactul diferitelor scheme de predicție

Soluție

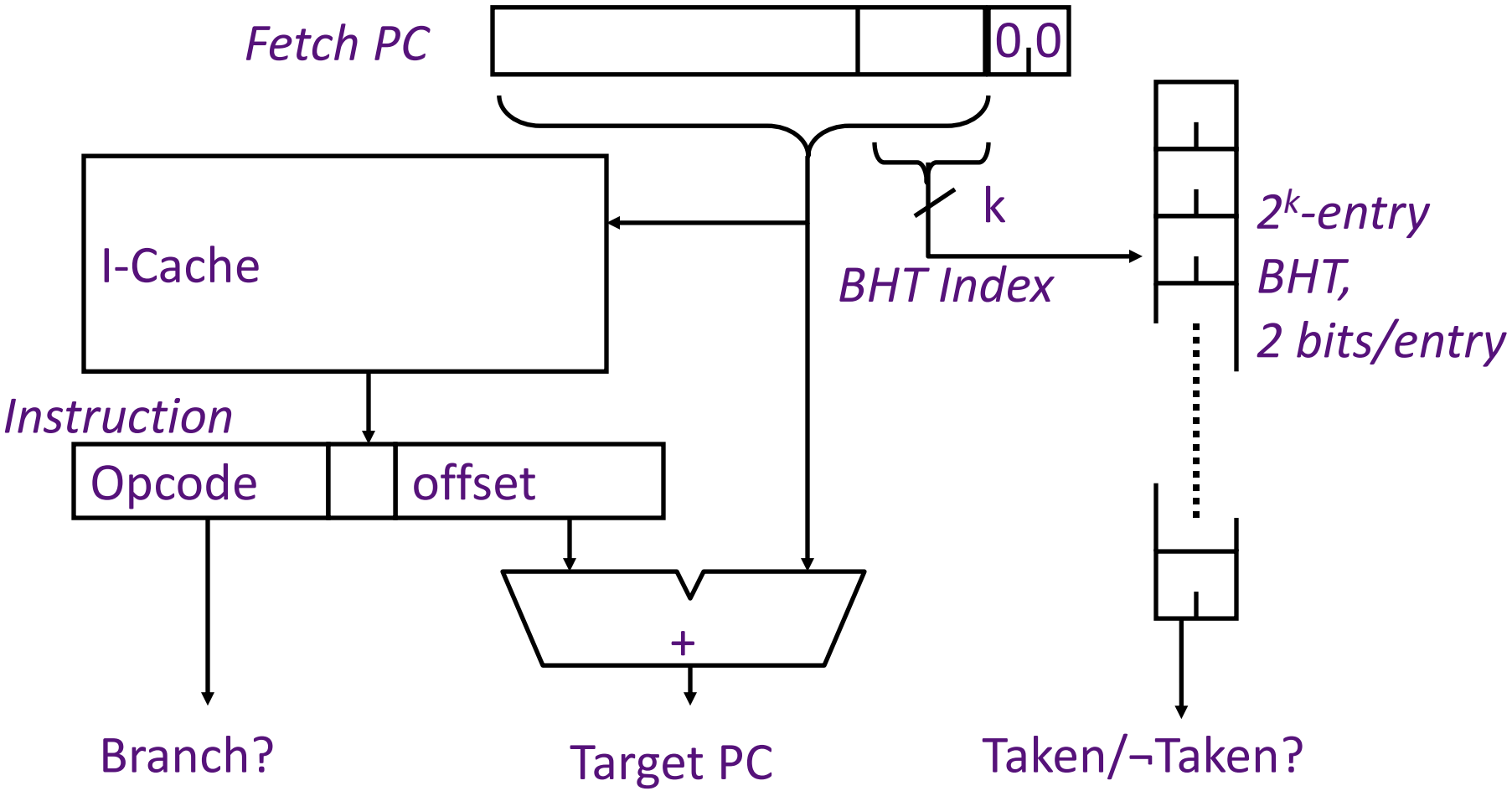
Always taken: 11 predicții proaste, 94.8% precis
1-bit history: 20 predicții proaste, 90.5% precis
2-bit history: la fel ca Always taken

Implementarea hardware pentru branch prediction



Elementele hardware ale unei scheme de branch prediction

Branch History Table



4K-entry BHT, 2 bits/entry, ~80-90% correct predictions

Exploatarea corelării spațiale

Yeh and Patt, 1992

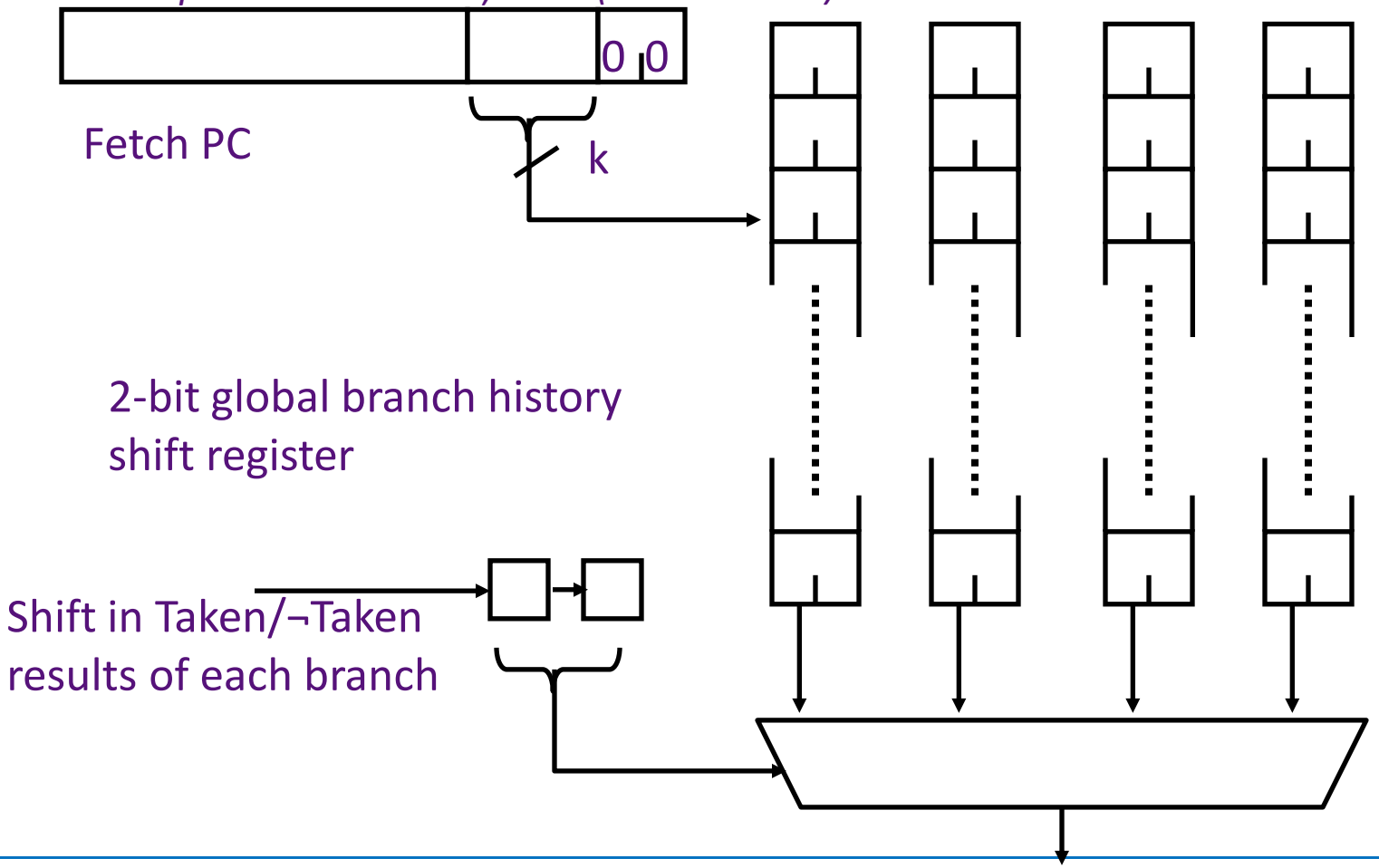
```
if (x[i] < 7) then
        y += 1;
if (x[i] < 5) then
        c -= 4;
```

Dacă prima condiție e falsă atunci și a doua este falsă

History register, H, înregistrează direcția ultimelor N branch-uri executate de către procesor

Branch prediction pe două niveluri

Pentium Pro folosește rezultatul ultimelor două branch-uri pentru a selecta unul din cele patru seturi de biți BHT (~95% corect)

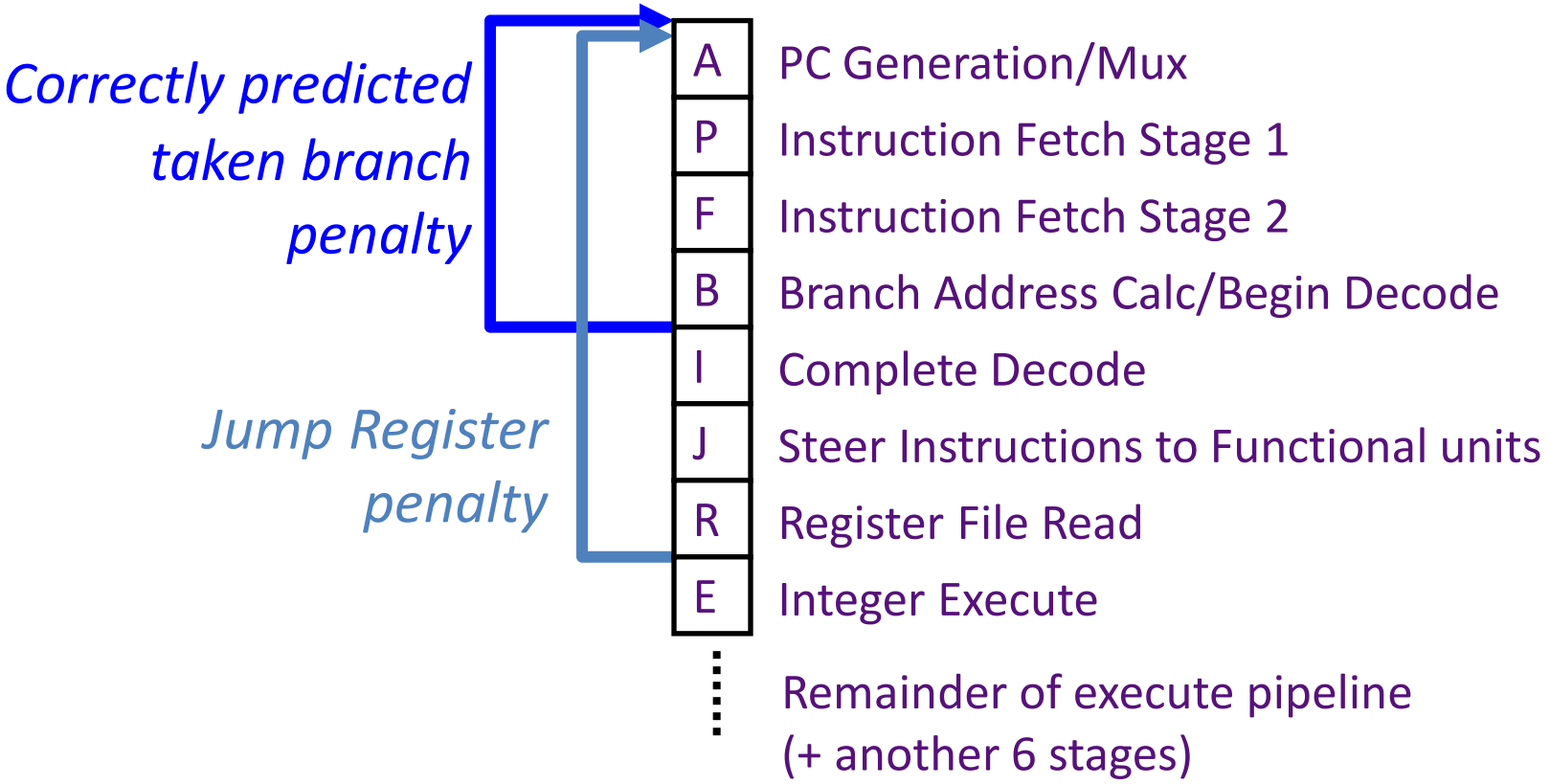


Speculații în ambele direcții

- O alternativă la branch prediction este execuția speculativă a ambelor direcții ale branch-ului
  - Cerințele de resurse sunt proporționale cu numărul de execuții speculative concurente
  - Doar jumătate din resurse vor face muncă utilă când ambele direcții ale unui branch sunt executate speculativ
  - Un branch prediction clasic folosește mai puține resurse decât o execuție speculativă concurentă
- Dacă avem un branch predictor cu o precizie foarte bună, e mai eficient din punct de vedere al costului să dedicăm toate resursele direcției prezise!

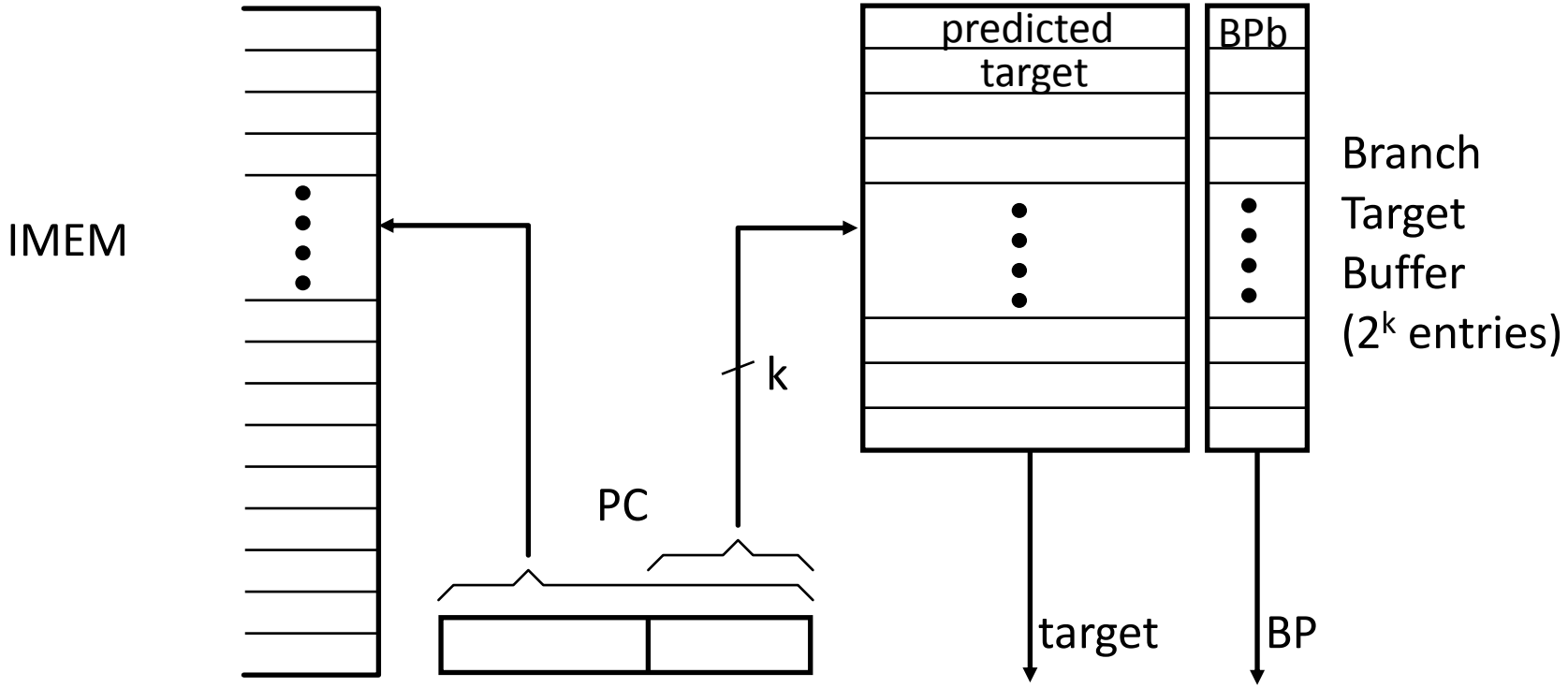
Limitările BHT

Prezice doar direcția branch-ului. Prin urmare, nu poate să redirecteze operațiile de fetch decât după ce s-a determinat ținta branch-ului.



UltraSPARC-III fetch pipeline

Branch Target Buffer

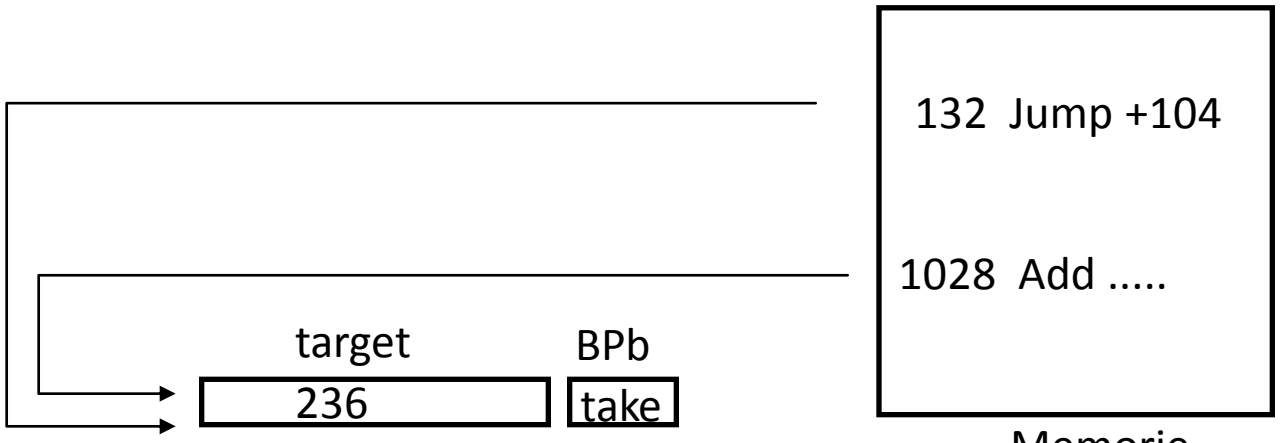


Biții BP sunt stocați împreună cu adresa țintă prezisă pentru salt.

IF stage: *If (BP=taken) then nPC=target else nPC=PC+4*
Later: *check prediction, if wrong then kill the instruction and update BTB & BPb else update BPb*

Coliziunea adreselor

Presupunem un BTB cu 128 intrări



La ce se va face fetch după instrucțiunea de la adr. 1028?

BTB prediction = 236

Correct target = 1032

-> *kill* PC=236 and *fetch* PC=1032

E o întâmplare frecventă?

Pot fi evitate astfel de bule?

BTB este folosit numai pentru instrucțiunile de control

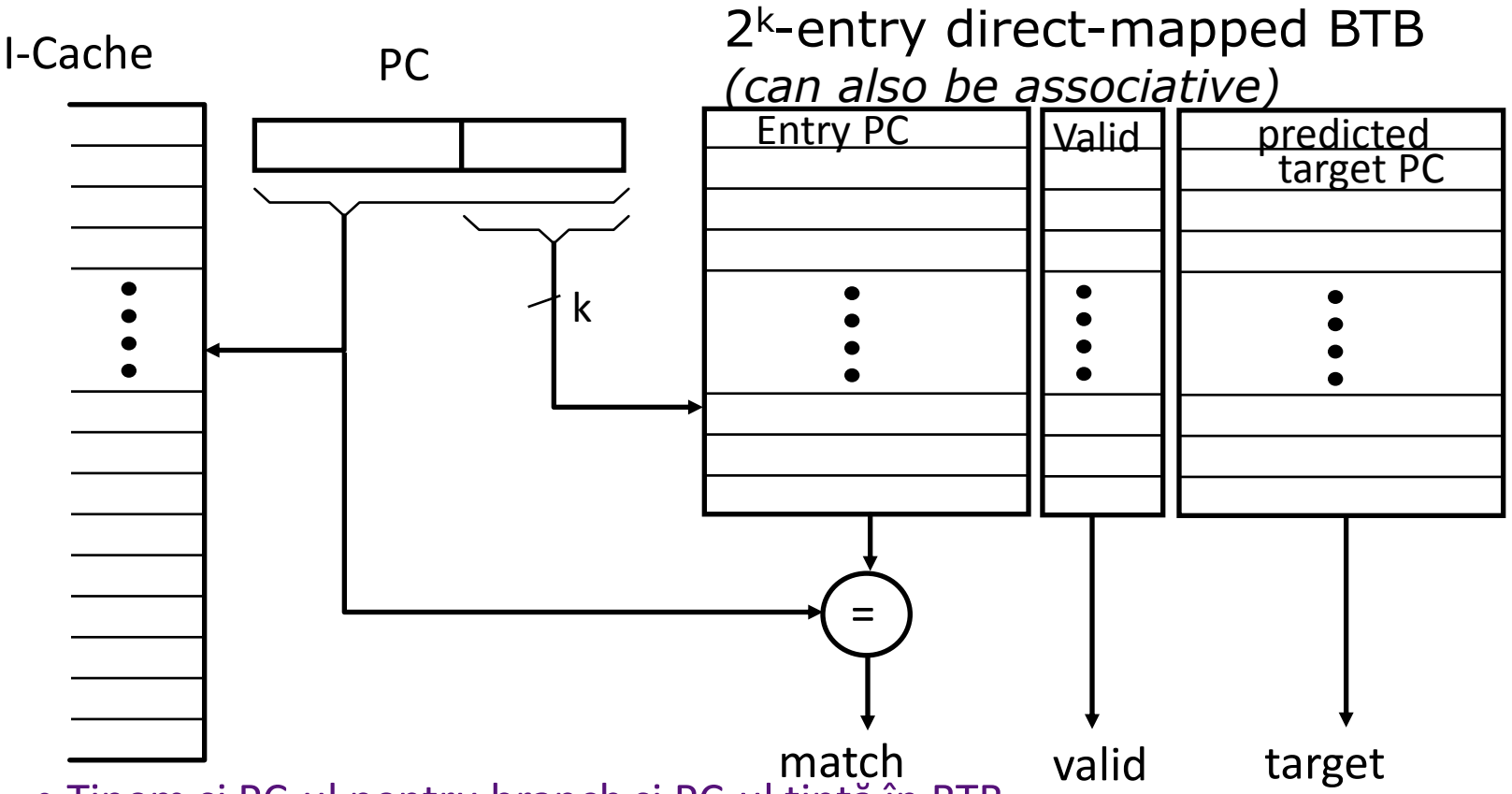
- BTB conține informații utile doar pentru instrucțiunile de branch și jump

  -> Nu face update pentru celalalte instrucțiuni

- Pentru toate celalalte instrucțiuni, următoarea adresă e PC+4 !

- *Cum obținem un astfel de comportament fără a decodifica instrucțiunea?*

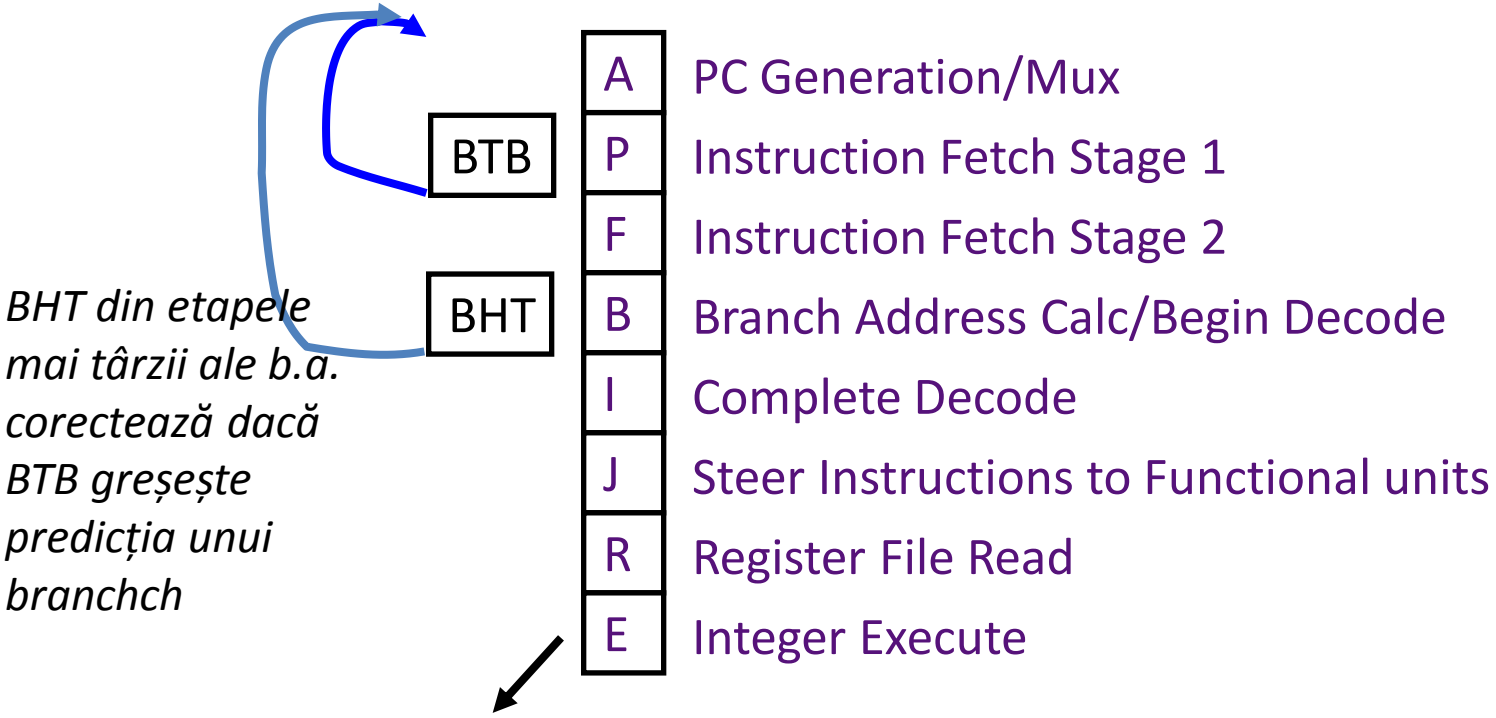
Branch Target Buffer (BTB)



- Ținem și PC-ul pentru branch și PC-ul țintă în BTB
- Facem fetch la PC+4 dacă cele două nu coincid
- Doar branch-urile și jump-urile active sunt ținute în BTB
- Următorul PC e determinat înainte să se facă fetch și decode la un branch

Combinăm BTB și BHT

- Intrările în BTB sunt considerabil mai costisitoare decât cele din BHT, dar pot să redirecționeze fetch-urile mai devreme în b.a.și pot să accelereze branch-urile indirecte (JR)
- BHT poate să țină mai multe înregistrări și este mai precisă



BHT din etapele mai târzii ale b.a. corectează dacă BTB greșește predicția unui branchch

BTB/BHT sunt actualizate după ce branch-ul se execută (etapa E)

Utilizarea Jump Register (JR)

Embedded Systems Laboratory

- Switch-uri (jump la adresa pentru case-ul corespunzător)

  BTB funcționează bine dacă folosim același case în mod repetat

- Apelare dinamică de funcții (jump la adresa de run-time a funcției)

  BTB funcționează bine dacă apelăm aceeași funcție în mod repetat, (e.g., în C++/Java, când obiectele au același tip într-un apel virtual de funcție)

- Întoarceri din subrutine (jump to return address)

  BTB funcționează bine dacă avem return la aceeași adresă

  $\Rightarrow$ *De obicei, aceeași funcție este apelată din mai multe locuri într-un program!*

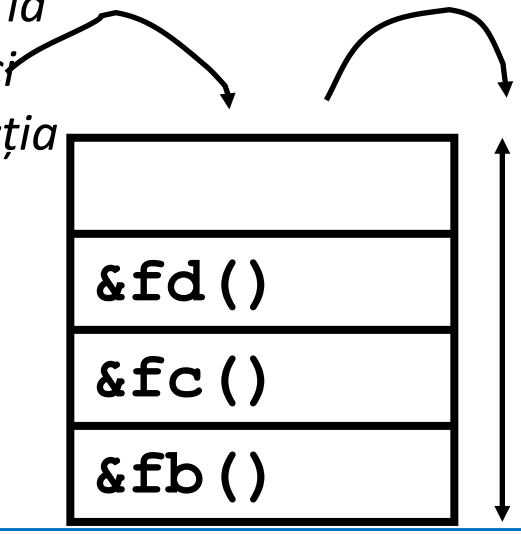
Cât de bine funcționează BTB pentru fiecare din aceste cazuri?

Stiva pentru întoarcere din subrutine

Structură de mici dimensiuni pentru accelerarea JR în cazul return-urilor din subrutine, de obicei mai precisă decât un BTB.

```
fa() { fb(); }
fb() { fc(); }
fc() { fd(); }
```

Facem push la adresa la care se face call atunci când se apelează funcția



Pop întoarce adresa atunci când este decodificată instrucțiunea de return din subrutină

k intrări
(de obicei k=8-16)

Variația CPI în funcție de diferitele arhitecturi

Embedded Systems Laboratory

Efectul pe care le au arhitectura, metodele de branch prediction și execuție speculativă asupra CPI.

Architectură	Metode utilizate în practică	CPI
Nonpipelined, multicycle	Strict in-order instruction issue and exec	5-10
Nonpipelined, overlapped	In-order issue, with multiple function units	3-5
Pipelined, static	In-order exec, simple branch prediction	2-3
Superpipelined, dynamic	Out-of-order exec, adv branch prediction	1-2
Superscalar	2- to 4-way issue, interlock & speculation	0.5-1
Advanced superscalar	4- to 8-way issue, aggressive speculation	0.2-0.5

3.3 inst / cycle × 3 Gigacycles / s
≅ 10 GIPS

Dezvoltarea procesoarelor Intel

Embedded Systems Laboratory

La început a fost 8080; urmat de 80x86 = IA32 ISA

În jur de cinci etape în banda de asamblare:

- 80286
- 80386
- 80486
- Pentium (80586)

↓ Tehnologie mai avansată

În jur de zece etape în banda de asamblare, out-of-order execution

- Pentium Pro
- Pentium II
- Pentium III
- Celeron

↓ Tehnologie mai avansată

Instrucțiunile sunt sparte în microinstrucțiuni care sunt executate out-of-order dar sunt retrase în ordine.

20-30 de etape în banda de asamblare

- Pentium 4

Starea curentă a performanței calculatoarelor

Embedded Systems Laboratory

Multi-GIPS/GFLOPS pentru desktop-uri și laptop-uri

Foarte puțini utilizatori au nevoie de mai multă putere de calcul
Utilizatorii nu mai sunt atât de doritori să facă upgrade doar pentru a avea un procesor mai rapid
Accentul se pune mai mult pe reducerea consumului și ergonomie

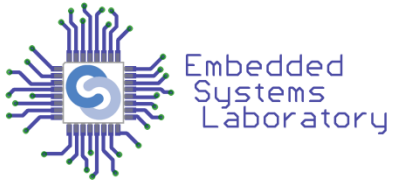
Multi-TIPS/TFLOPS în centrele mari de calcul

Top 500 al supercalculatoarelor, http://www.top500.org
Următoarea listă în iunie 2016; din noiembrie 2015:
Toate 500 >> 75 TFLOPS, ≈30 > 700 TFLOPS, 4 > 10 PFLOPS
(în 2008) Toate 500 >> 10 TFLOPS, ≈30 > 100 TFLOPS, 1 > PFLOPS

Multi-PIPS/PFLOPS supercalculatoare în faza de proiectare

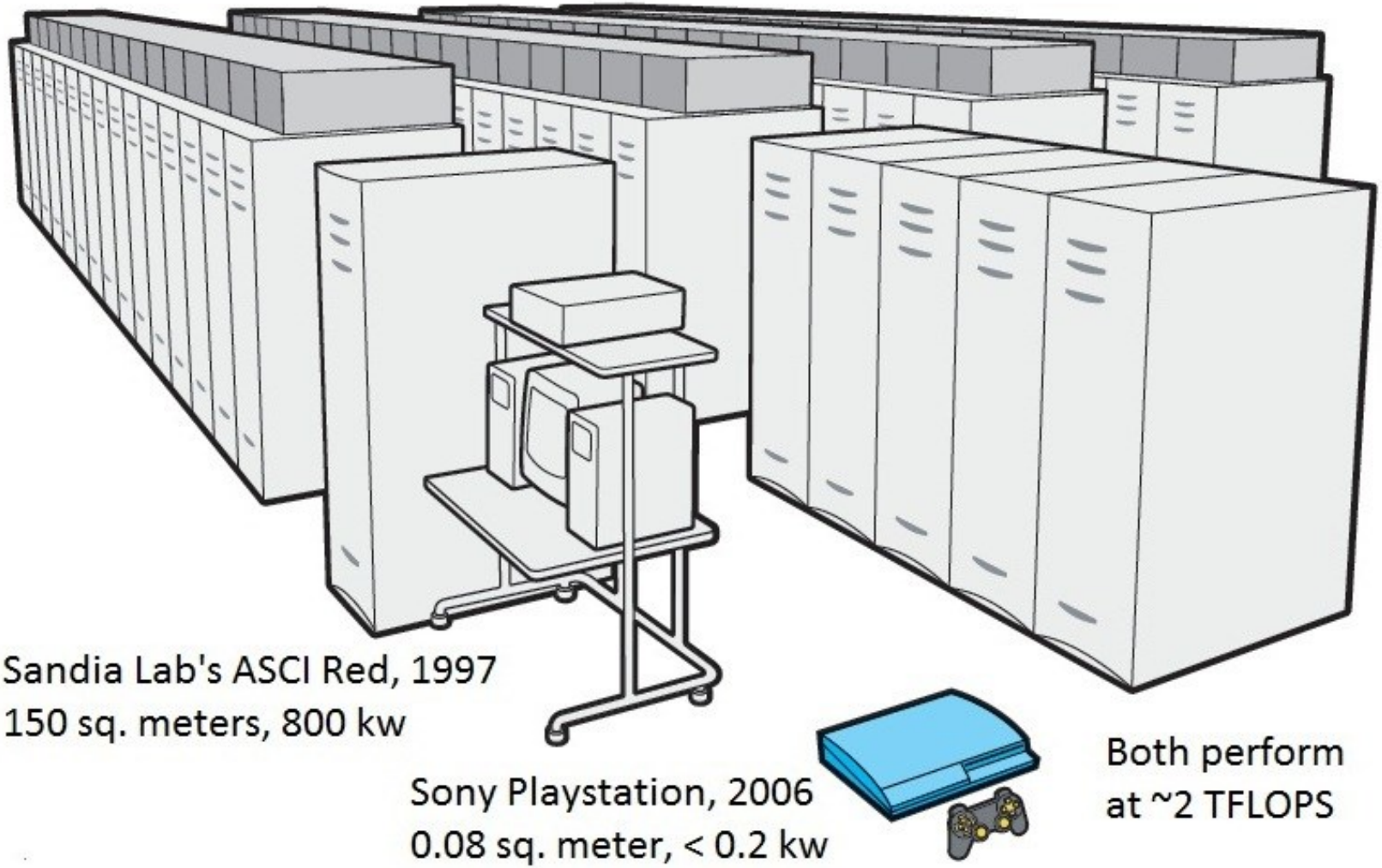
Exascale – momentan doar un concept

Primele trei supercalculatoare din lume (noiembrie 2015)



Rank	Site	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	National Super Computer Center in Guangzhou China	**Tianhe-2 (MilkyWay-2)** - TH-IVB-FEP Cluster, Intel Xeon E5-2692 12C 2.200GHz, TH Express-2, Intel Xeon Phi 31S1P NUDT	3,120,000	33,862.7	54,902.4	17,808
2	DOE/SC/Oak Ridge National Laboratory United States	**Titan** - Cray XK7 , Opteron 6274 16C 2.200GHz, Cray Gemini interconnect, NVIDIA K20x Cray Inc.	560,640	17,590.0	27,112.5	8,209
3	DOE/NNSA/LLNL United States	**Sequoia** - BlueGene/Q, Power BQC 16C 1.60 GHz, Custom IBM	1,572,864	17,173.2	20,132.7	7,890

The Shrinking Supercomputer



Acknowledgements

- These slides contain material developed and copyright by:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

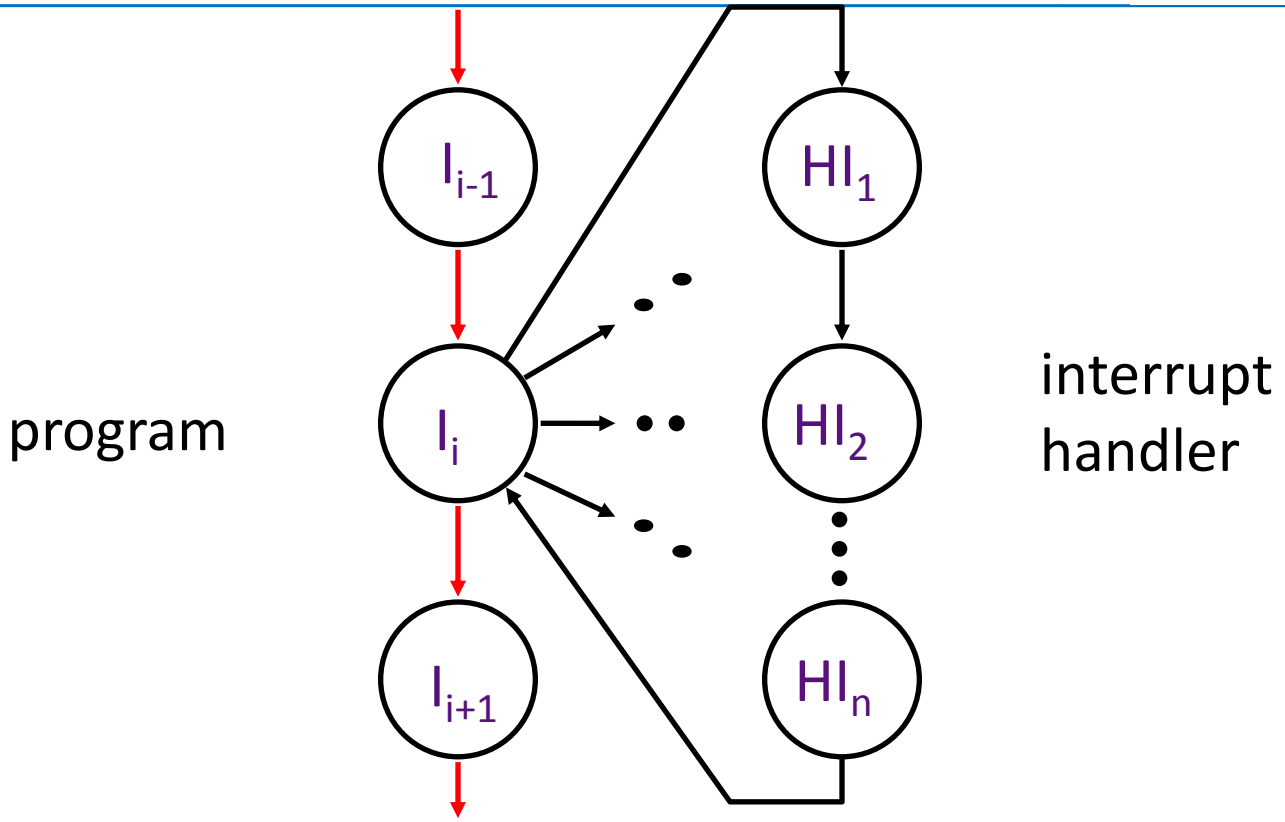
Calculatoare Numerice

– Cursul 13 –

Întreruperi

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Întreruperi: schimbarea fluxului normal de execuție



O **întrerupere** este un eveniment *extern* sau *intern* care trebuie procesat de o rutină (sau program) separată. Evenimentul este de obicei neașteptat (asincron față de ceasul sistemului) și/sau rar din punctul de vedere al programului principal.

Surse pentru întreruperi

Embedded Systems Laboratory

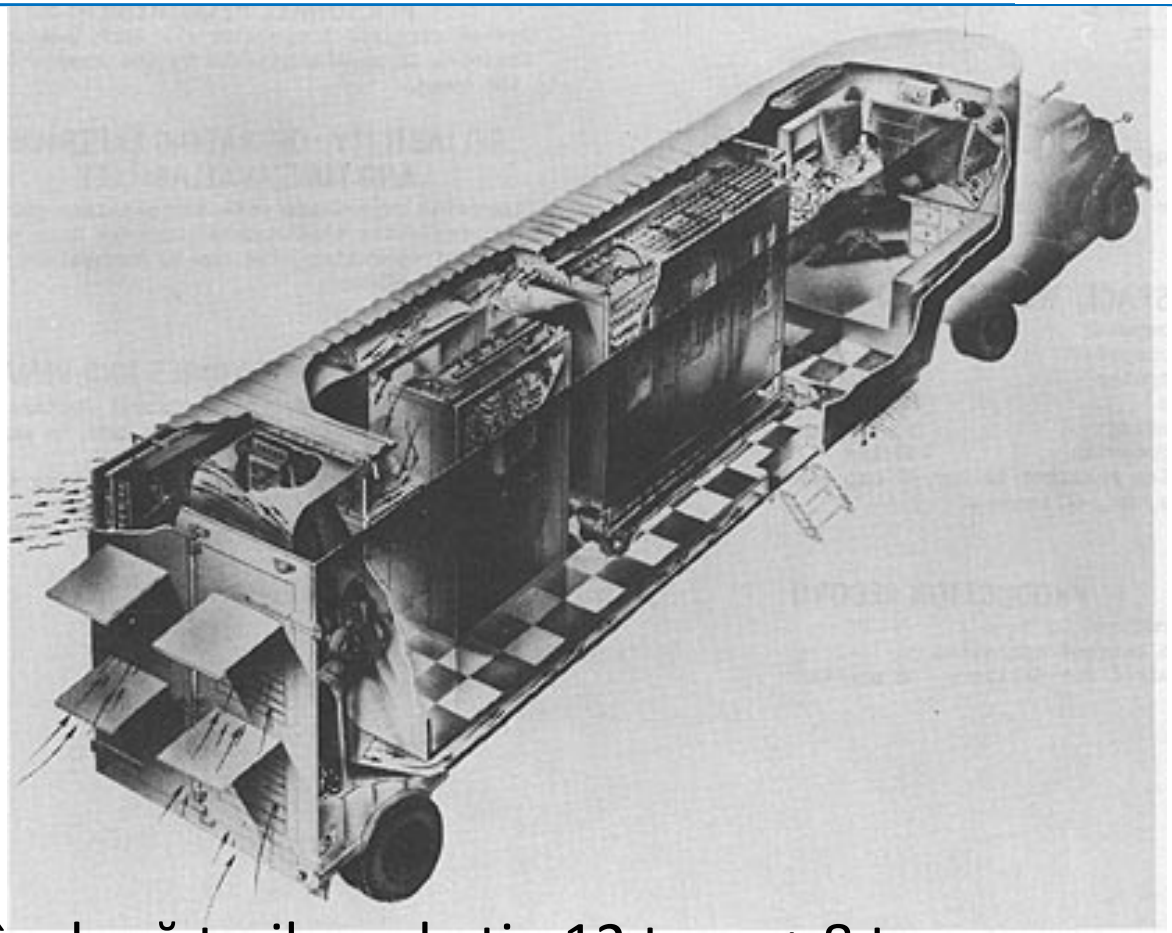
Întrerupere: un *eveniment* ce necesită atenția procesorului

- Asincron: un eveniment extern
  - Cerere de la un dispozitiv input/output
  - Expirarea unui timer
  - Întrerupere în alimentarea cu energie, cădere hardware
- Sincron: un eveniment intern (*a.k.a. Trap sau Excepție)*
  - Opcode nedefinit, instrucțiune privilegiată
  - Overflow aritmetic, excepție FPU
  - Acces prost aliniat la memorie
  - *Excepții de memorie virtuală:* page faults, TLB misses, protection violations
  - System calls (e.g., jump-uri în kernel)

O scurtă istorie pentru exception handling

- Primul sistem care trata excepțiile Univac-I, 1951
  - Overflow aritmetic, care avea următoarele roluri:
    1. Declanșează execuția unei rutine de fix-up de două instrucțiuni situată la adresa 0, sau
    2. La alegerea programatorului, cauzează mașina de calcul să se oprească
  - Mai târziu, Univac 1103, 1955, modificat pentru a trata întreruperi externe
    - Folosit pentru a culege informații de timp real dintr-un tunel aerodinamic
- Primul sistem cu întreruperi pe I/O a fost DYSEAC, 1954
  - Avea două PC-uri și o întrerupere de la I/O cauza schimbarea PC-ului curent
  - De asemenea, era primul sistem cu DMA (Direct Memory Access pentru un dispozitiv I/O)

DYSEAC, primul computer mobil!



- Instalat în două trailere de tir, 12 tone + 8 tone
- Construit pentru US Army Signal Corps *[Courtesy Mark Smotherman]*

Întreruperi asincrone: invocarea rutinei de tratare a întreruperii

Embedded Systems Laboratory

- Un dispozitiv I/O cere atenție prin semnalizarea pe o linie dedicată de interrupt request (IRQ)

- Când un procesor decide să proceseze întreruperea
  - Oprește execuția programului curent la instrucțiunea I_i, completând execuția tuturor instrucțiunilor până la I_{i-1} (*întrerupere precisă)*
  - Salvează PC-ul instrucțiunii I_i într-un registru special (EPC)
  - Maschează toate întreruperile și transferă controlul unei rutine speciale pentru tratarea întreruperii (pentru un OS ca Linux/Windows aceasta e situată de obicei în kernel)

Rutina de tratare a întreruperii

- Salvează EPC înainte să permită execuția altor întreruperi (întreruperi imbricate) ⇒
  - Avem nevoie de o instrucțiune pentru a muta EPC în GPR-uri
  - Avem nevoie de un mecanism pentru a masca toate întreruperile până când EPC este salvat
- Trebuie să citească un Registru de Stare care indică ce periferic sau subsistem a cauzat întreruperea
- Folosește o instrucțiune specială de salt indirect RFE (*return-from-exception*) care
  - Demaschează întreruperile
  - Întoarce procesorul înapoi în modul utilizator
  - Reface statusul inițial hardware și starea unității de control

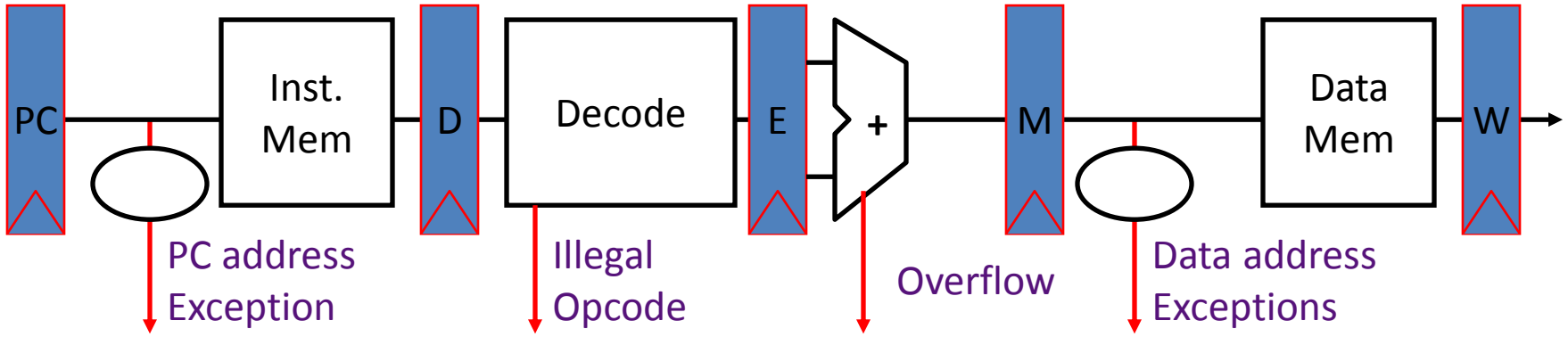
Întreruperi sincrone

- O întrerupere sincronă (excepție) este cauzată de o anumită instrucțiune

- În general, instrucțiunea nu poate fi completată și trebuie să fie restartată după tratarea excepției pe care a generat-o
  - Necesită refacerea efectului unei sau mai multor instrucțiuni executate parțial

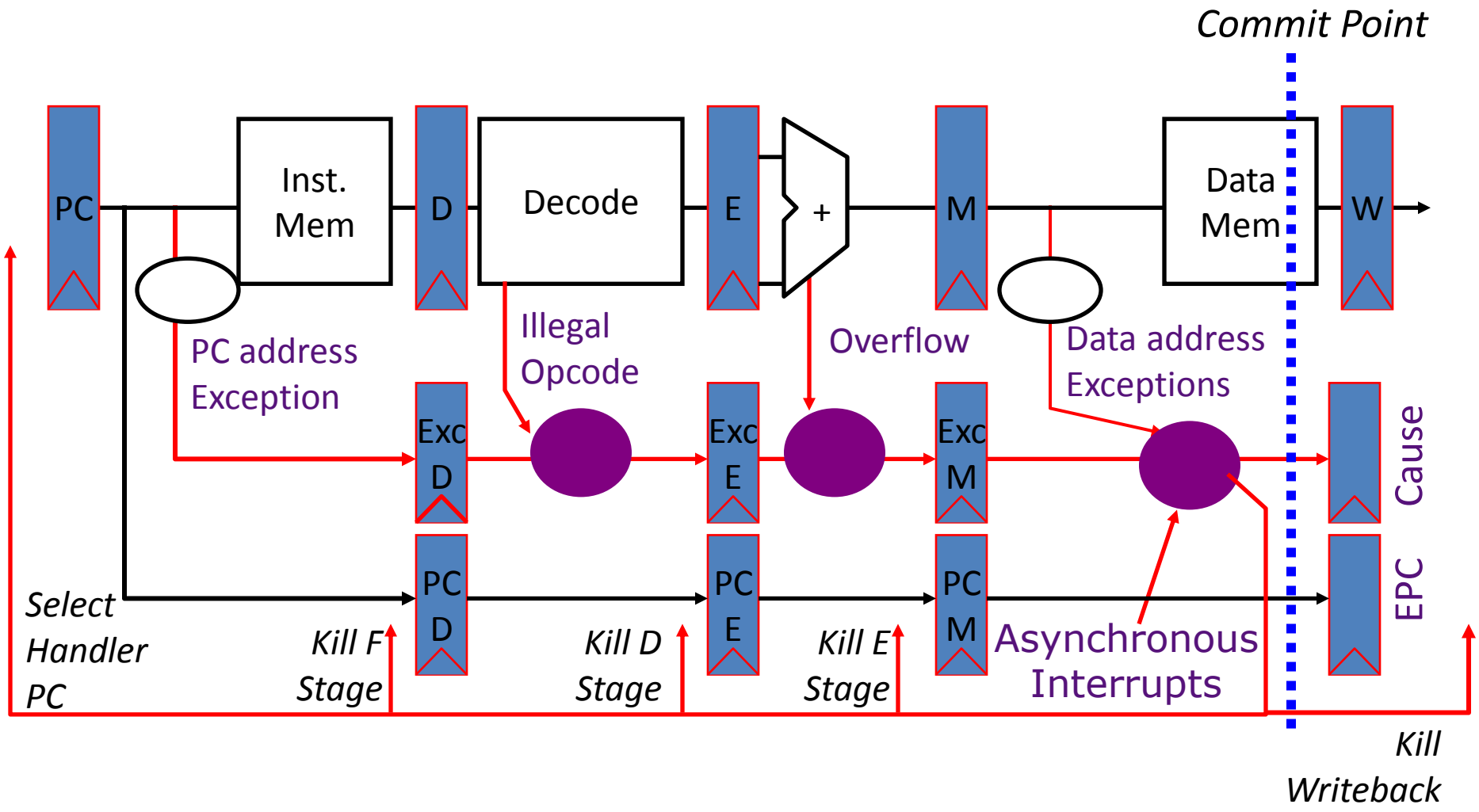
- În cazul unui system call trap, se consideră că instrucțiunea și-a incheiat execuția
  - O instrucțiune de jump specială ce face o trecere a procesorului în modul kernel privilegiat

Tratarea întreruperilor b.a. cu 5 etape



- Cum putem să tratăm excepții multiple simultane în diferitele etape ale b.a.?
- Cum și unde putem să tratăm întreruperile asincrone externe?

Tratarea întreruperilor b.a. cu 5 etape



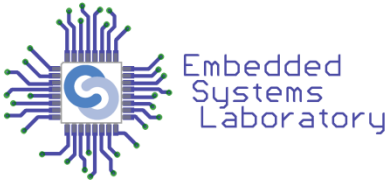
Tratarea întreruperilor b.a. cu 5 etape

- Ținem flag-urile de excepție în b.a. până la commit point (etapa M)
- Excepțiile din etapele anterioare suprascriu excepțiile din etapele superioare pentru o instrucțiune dată
- Injectează întreruperile externe la commit point (suprascrie toate celalalte)
- Dacă excepția se petrece la commit: actualizează registrele Cause și EPC, invalidează toate etapele din b.a., injectează adresa rutinei de tratare a întreruperii în PC

Speculații la excepții

- Mecanism de predicție
  - Excepțiile sunt foarte rare, așa că un mecanism care prezice că nu există nici o excepție este foarte precis!
- Verificarea mecanismului de predicție
  - Excepțiile sunt detectate la sfârșitul execuției în b.a., hardware special pentru diferitele tipuri de excepții
- Mecanism de revenire din excepție
  - Scriem doar starea generală la commit point, ca să putem să neglijăm toate instrucțiunile executate parțial după apariția excepției
  - Lansează rutina de tratare a întreruperii după ce golim întreaga bandă de asamblare

Diagrame b.a. pentru tratarea excepțiilor



	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	-		*overflow!*		
(I_2) 100: XOR		IF_2	ID_2	EX_2	-	-			
(I_3) 104: SUB			IF_3	ID_3	-	-	-		
(I_4) 108: ADD				IF_4	-	-	-	-	
(I_5) Exc. Handler code					IF_5	ID_5	EX_5	MA_5	WB_5

Resource Usage

	timp t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	I_3	-	I_5			
EX			I_1	I_2	-	-	I_5		
MA				I_1	-	-	-	I_5	
WB					-	-	-	-	I_5

Apeluri de sistem pentru I/O

Embedded Systems Laboratory

De ce utilizatorul trebuie să fie izolat de toate detaliile operațiilor I/O?

Protecție: Utilizatorul trebuie să nu acceseze anumite zone de memorie

Convenienţă: Nu trebuie să știe toate detaliile de operare pentru fiecare device

Eficienţă: Majoritatea utilizatorilor sunt incapabili să găsească cel mai bun mod de acces la I/O

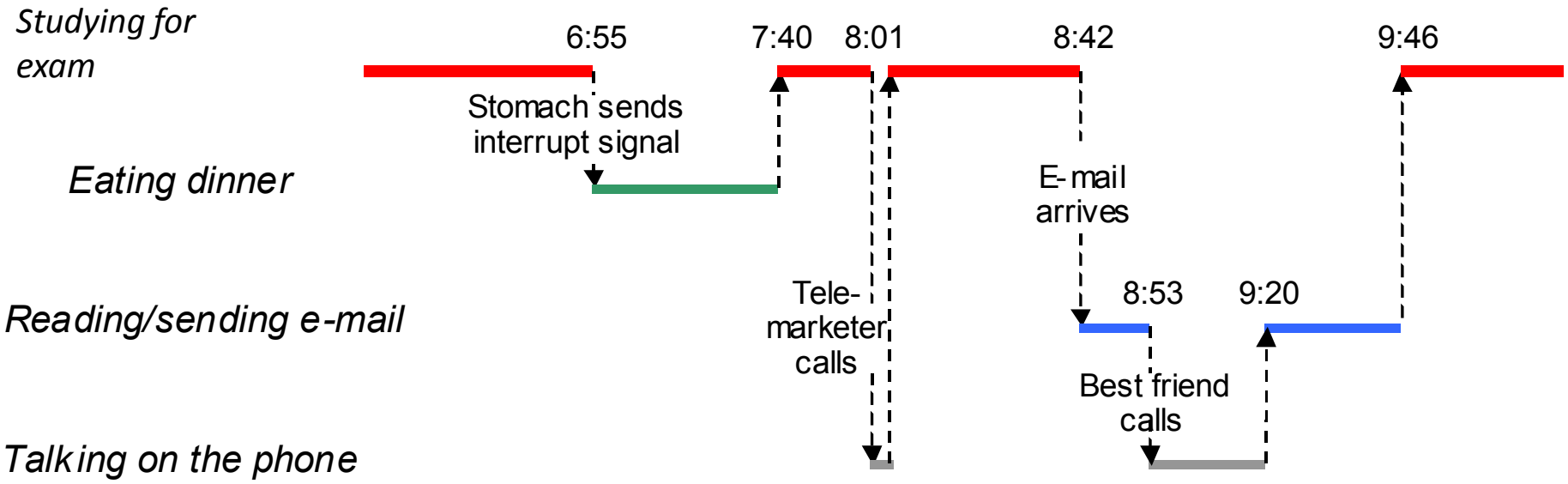
Abstractizarea I/O: grupăm dispozitivele I/O într-un număr mic de clase generice pentru a face operațiile de I/O independente de dispozitive

Character stream I/O: get(•), put(•) – e.g., keyboard, printer
Block I/O: seek(•), read(•), write(•) – e.g., disk

Network Sockets: create socket, connect, send/receive packet
Clocks or timers: set up timer (notificare prin întrerupere)

Întreruperi, Excepții și System traps

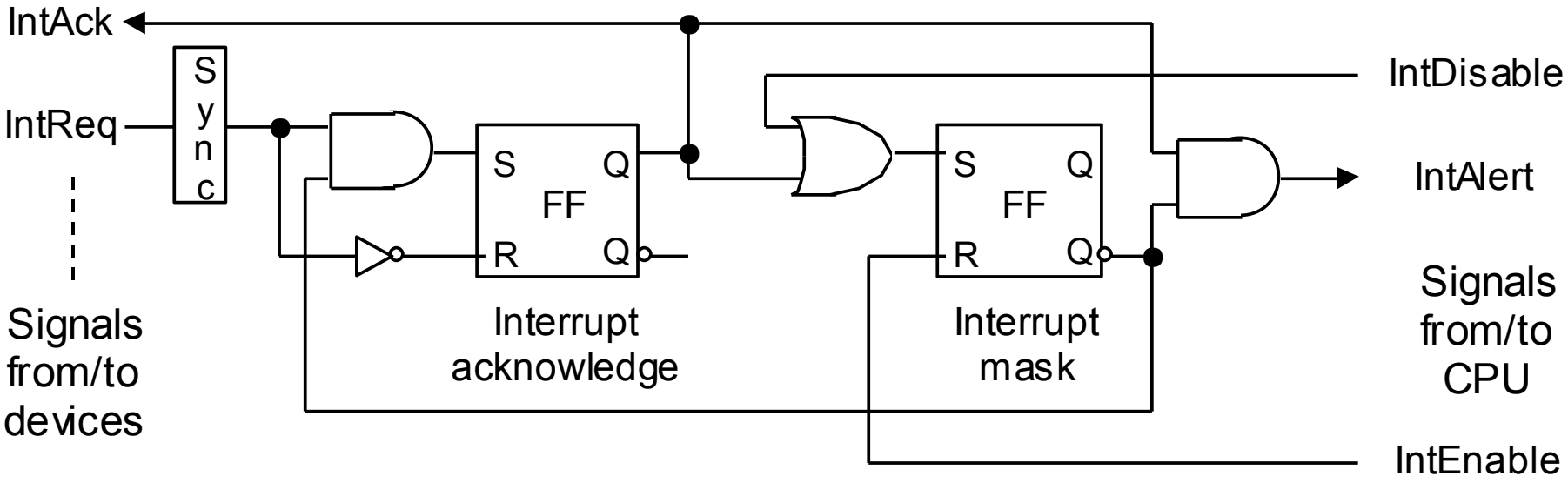
Intrerupere	Termen general pentru orice cerere sau de la I/O
Excepție	Cauzate de o operație ilegală (imprevizibile)
Trap	AKA “software interrupt” (pre-planificată)



Noțiunea de întrerupere și întreruperi imbricate

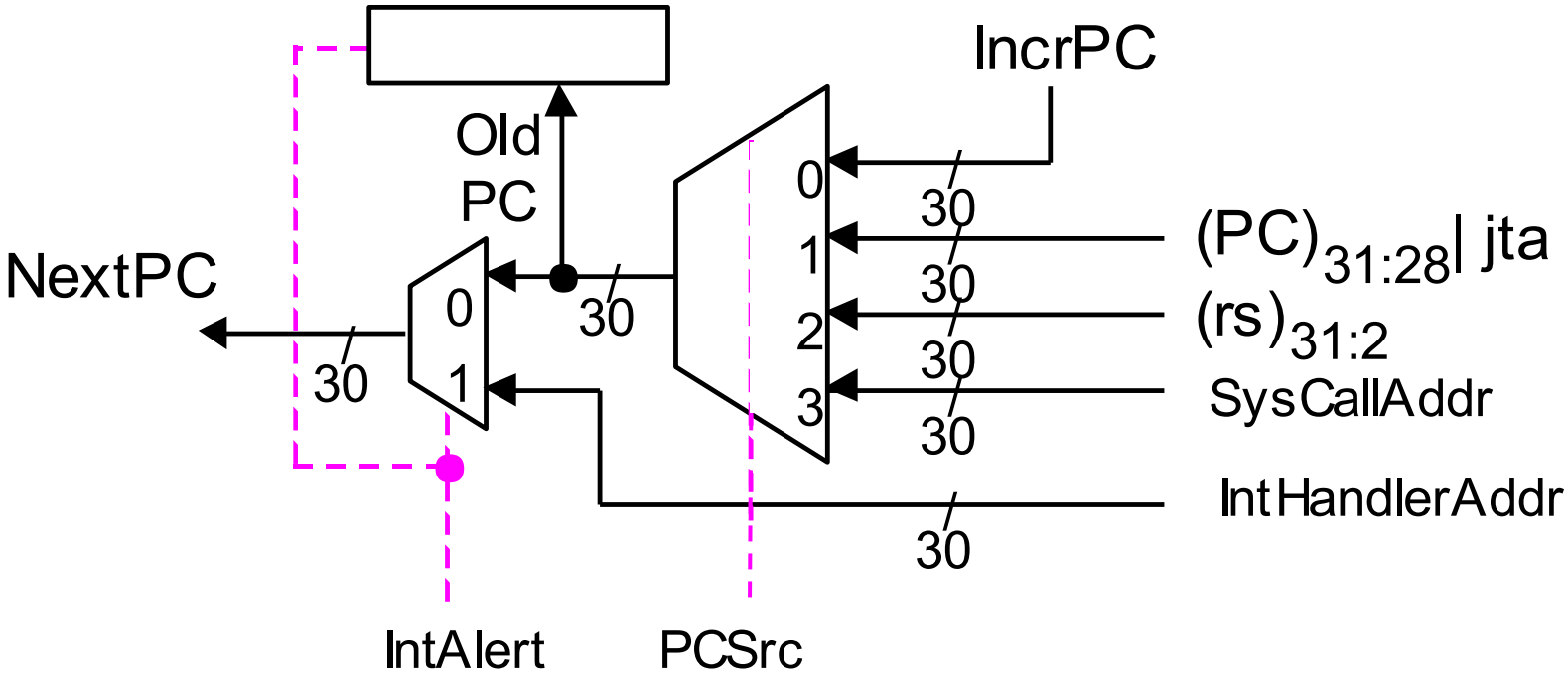
Tratarea întreruperilor

Răspundem la cererea de întrerupere prin semnalul IntAck
Notificăm procesorul că avem o întrerupere de tratat
Setăm masca de întreruperi a.î. noua întrrupere să fie demascată



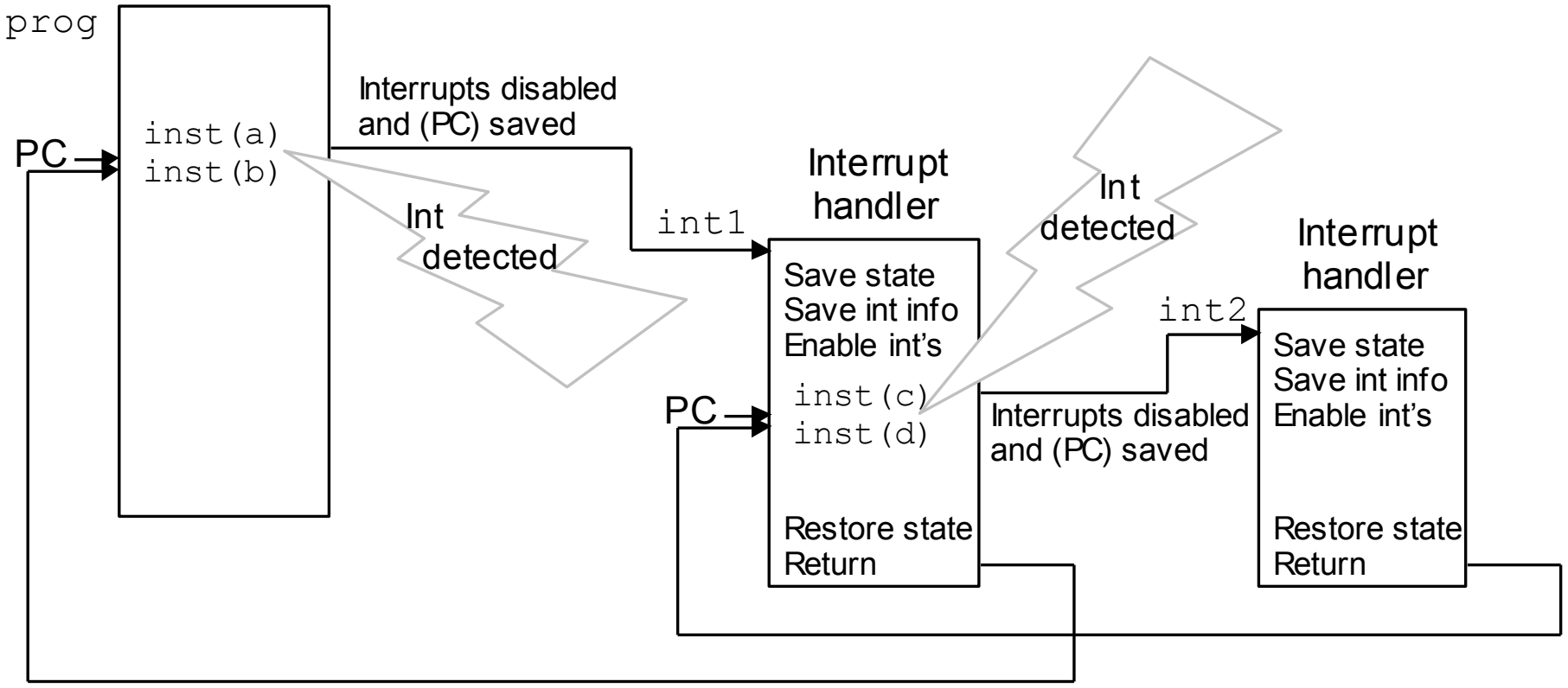
Logică simplă de semnalizarea a unei întreruperi pentru un procesor

Logica pentru calculul rutinelor de tratare a întreruperilor



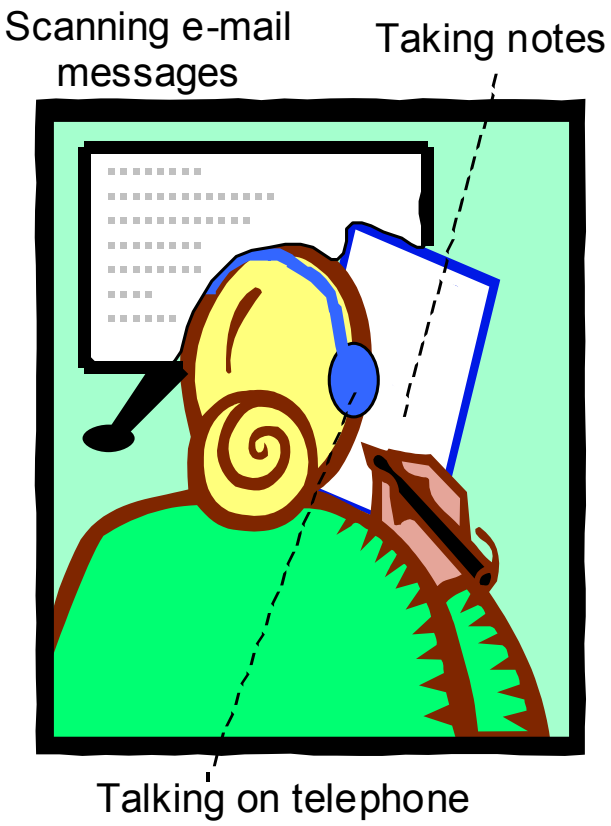
Logica pentru calculul adresei următoare de program la care s-a adăugat calculul adreselor pentru rutinele de tratare a întreruperilor

Întreruperi imbricate

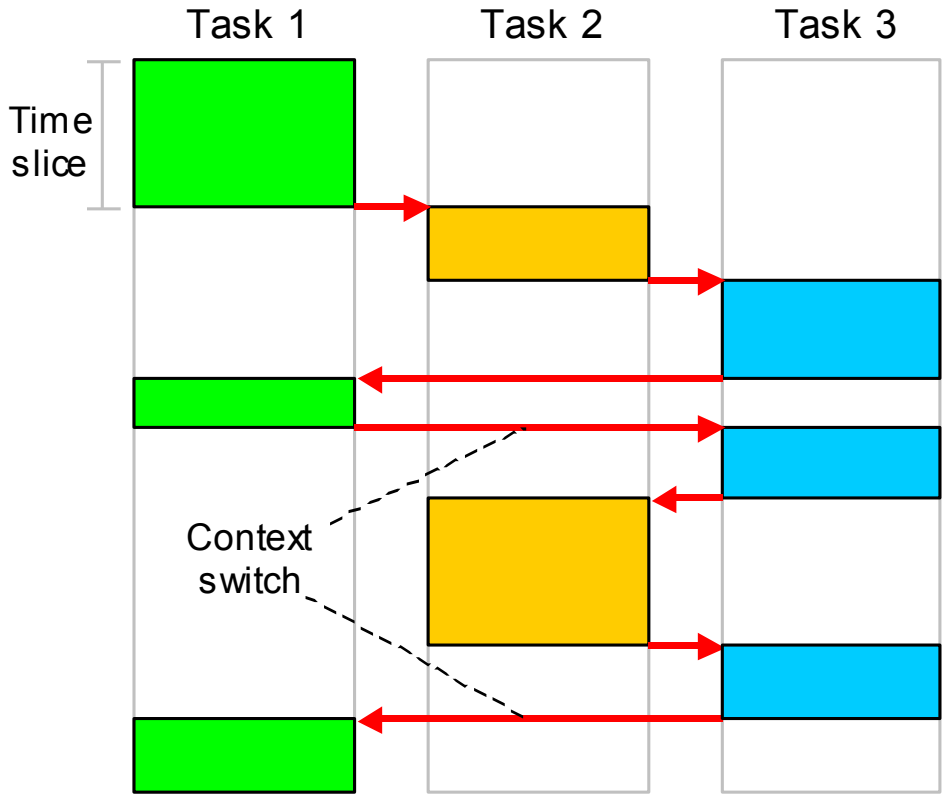


Exemplu de întrerupere imbricată.

Context switching



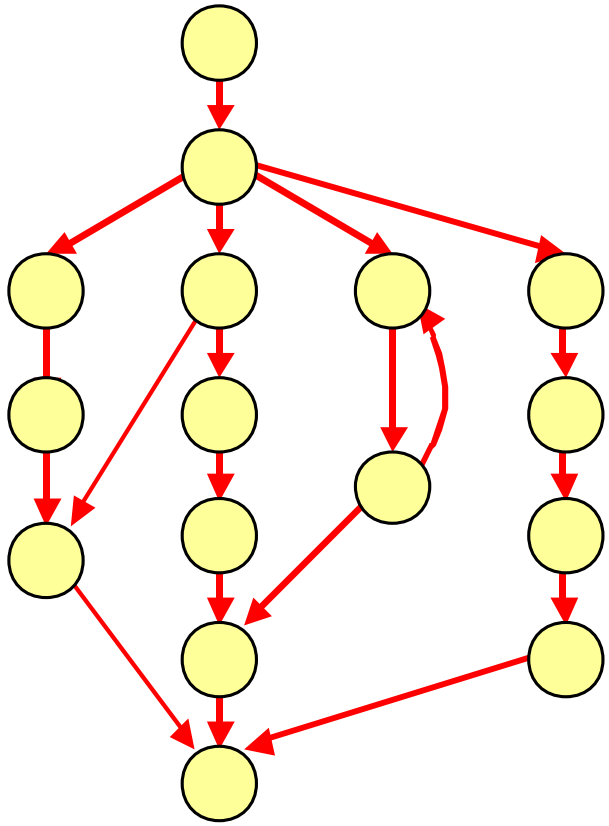
(a) Human multitasking



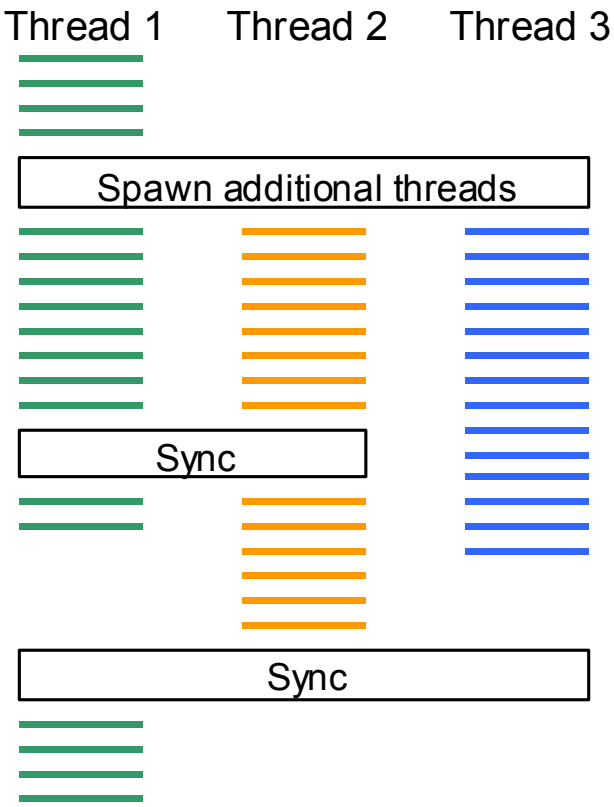
(b) Computer multitasking

Multitasking la oameni și la calculatoare

Threaduri și multithreading



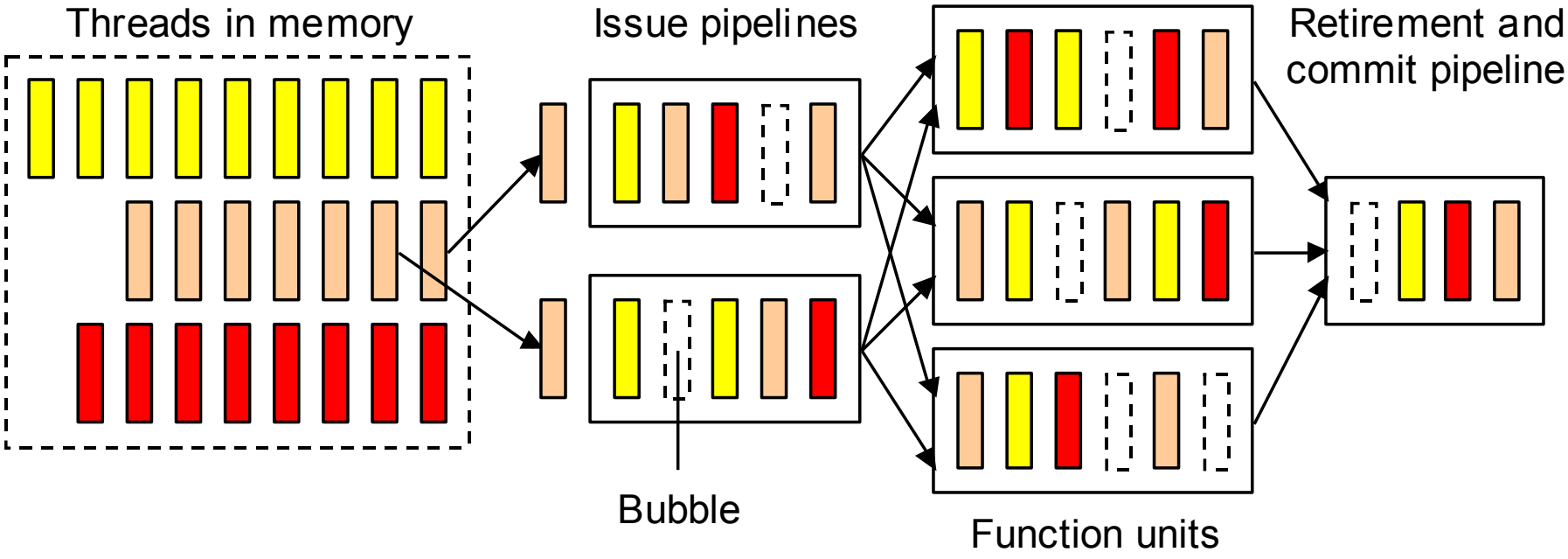
(a) Task graph of a program



(b) Thread structure of a task

Un program poate fi împărțit în task-uri sau thread-uri

Procesoare cu mai multe fire de execuție



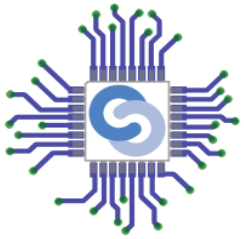
Instrucțiuni din mai multe thread-uri pe măsură ce își găsesc drumul prin banda/benzile de asamblare ale unui procesor modern.

Unde mergem mai departe?

Proiectarea memoriei:
Cum putem proiecta o memorie care să răspundă într-un singur ciclu de ceas?

Input and Output:
Periferice, programarea I/O, interfațare, întreruperi

Performanță (și) mai mare:
Procesare vectorială
Procesare paralelă



Calculatoare Numerice

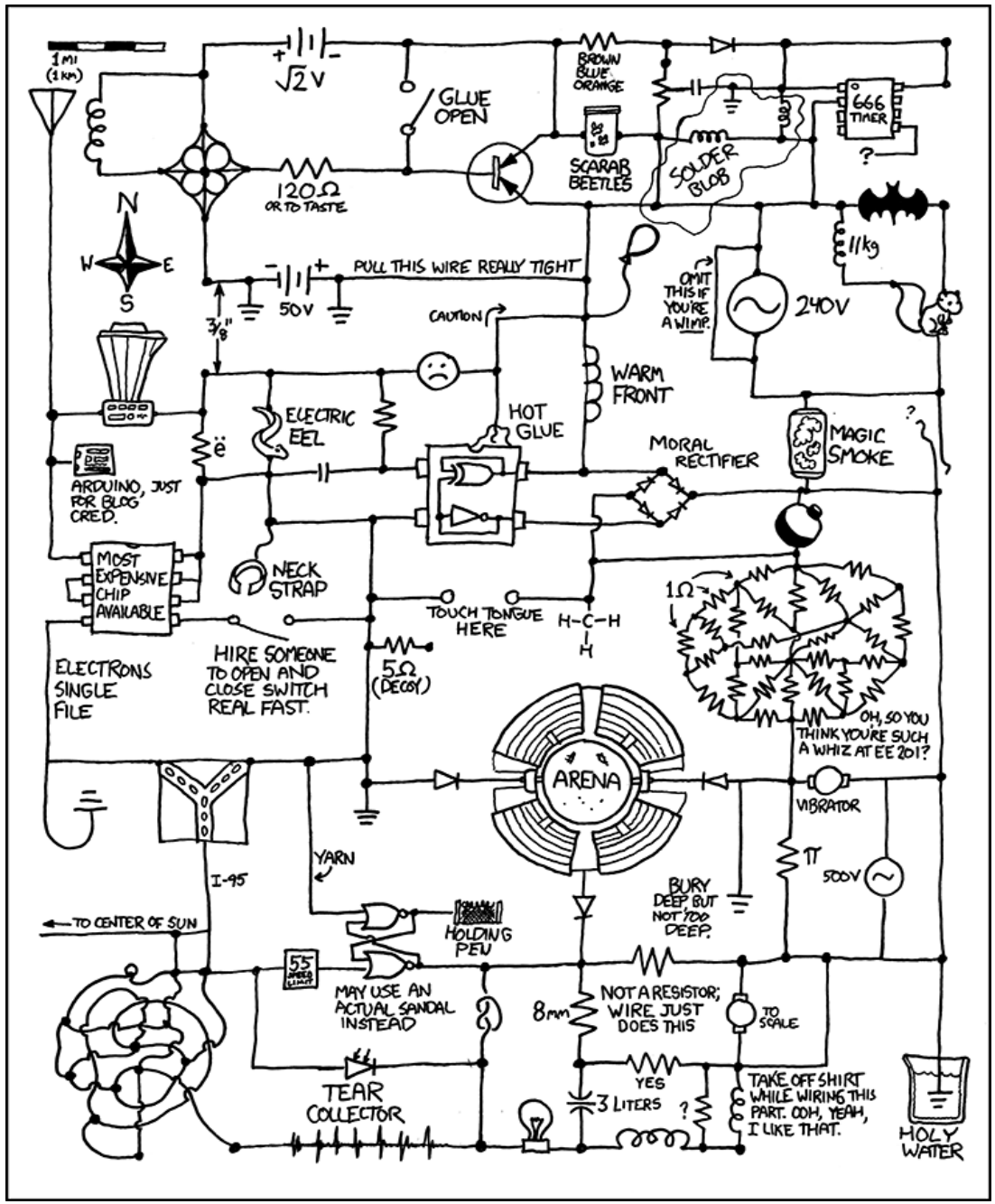
-Cursul 1 –

De la porți logice la procesoare

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Comic of the day

Embedded Systems Laboratory

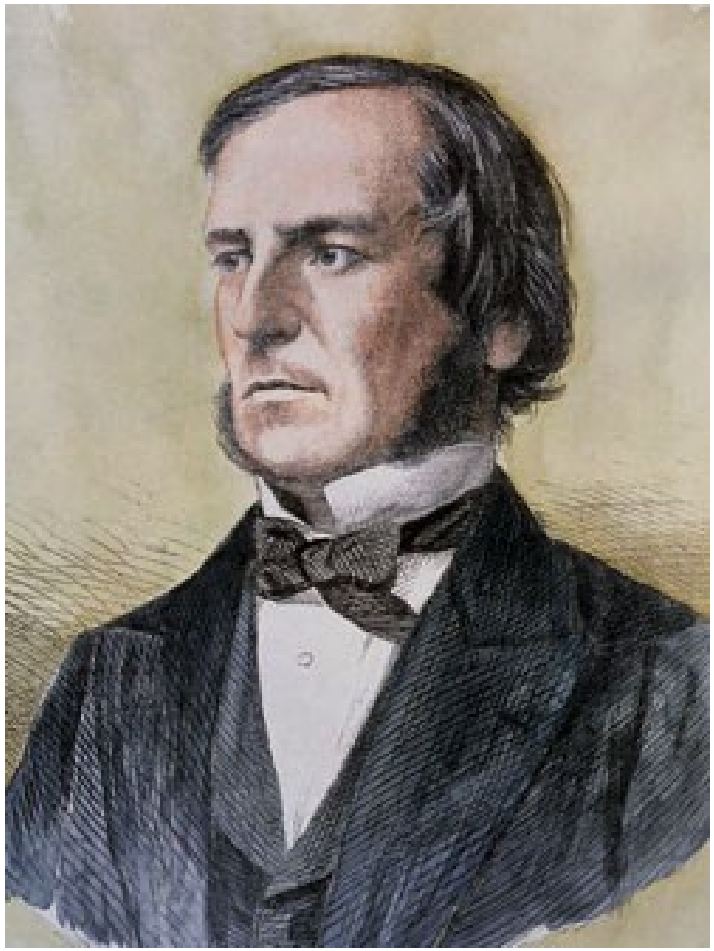


http://xkcd.com/730/

Cum construiesc un calculator?

George Boole, 1815-1864

- Matematician, filozof si logician englez
- “The Laws of Thought”(1854) – algebra booleană
- Părintele erei informației



Manipularea expresiilor logice

Embedded Systems Laboratory

Legile algebrei binare

Numele legii	Forma SAU	Forma ȘI
Identitate	$x \vee 0 = x$	$x \wedge 1 = x$
Unu/Zero	$x \vee 1 = 1$	$x \wedge 0 = 0$
Idempotență	$x \vee x = x$	$x \wedge x = x$
Negare	$x \vee x' = 1$	$x \wedge x' = 0$
Comutativitate	$x \vee y = y \vee x$	$x \wedge y = y \wedge x$
Asociativitate	$(x \vee y) \vee z = x \vee (y \vee z)$	$(x \wedge y) \wedge z = x \wedge (y \wedge z)$
Distributivitate	$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$
DeMorgan	$(x \vee y)' = x' \wedge y'$	$(x \wedge y)' = x' \vee y'$

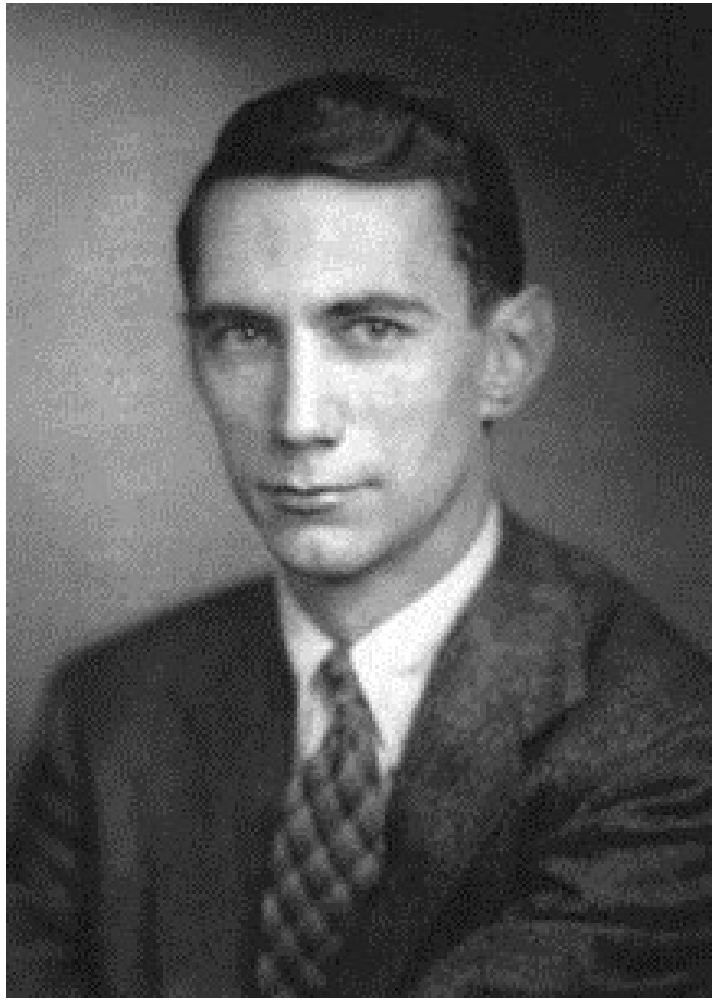
Funcții și expresii booleene

Moduri de a specifica o funcție logică

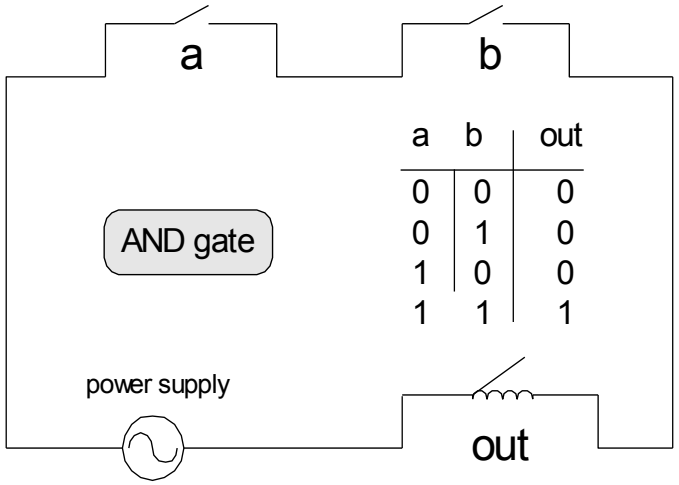
- Tabelă de adevăr: 2^n rânduri, “don’t-care” input/output
- Expresie logică: $w'(x \vee y \vee z)$, sumă de produse, produs de sume, expresii echivalente
- Limbaj natural: Alarma va suna dacă ușa se deschide cât timp sistemul de securitate este armat sau când senzorul de fum este activat
- Diagrama logică a circuitului: Sinteză vs. analiză

Claude Shannon, 1916-2001

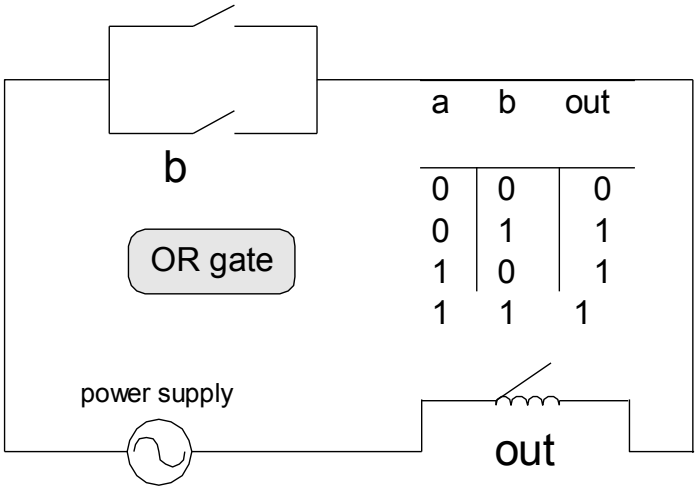
- Matematician și inginer american
- ”A Symbolic Analysis of Relay and Switching Circuits” (1938) – teză de master la MIT, aplică logica booleană circuitelor de calcul
- "A Mathematical Theory of Communication” (1948) – pune bazele teoriei informației



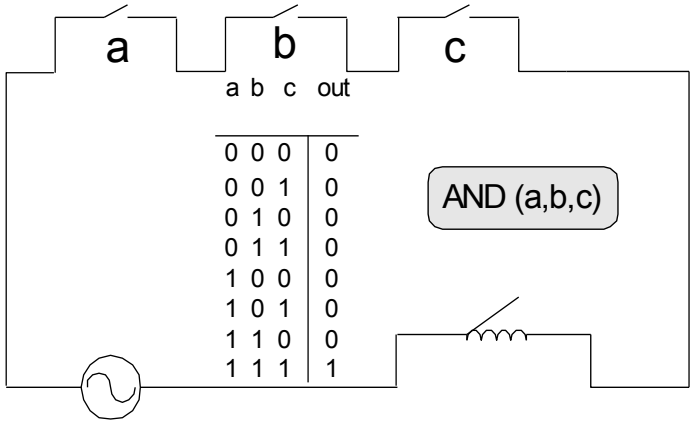
Porți logice cu relee (Shannon)



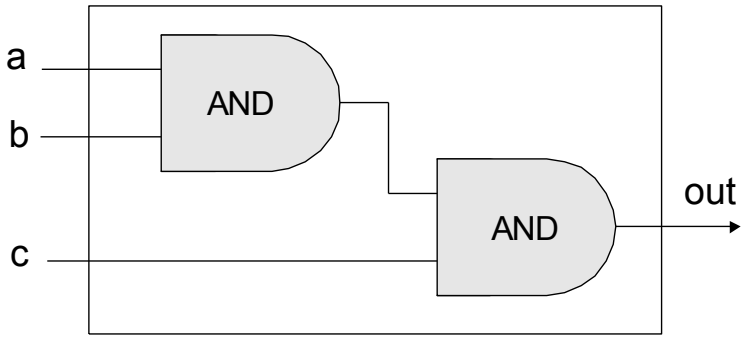
a	b	out
0	0	0
0	1	0
1	0	0
1	1	1



a	b	out
0	0	0
0	1	1
1	0	1
1	1	1



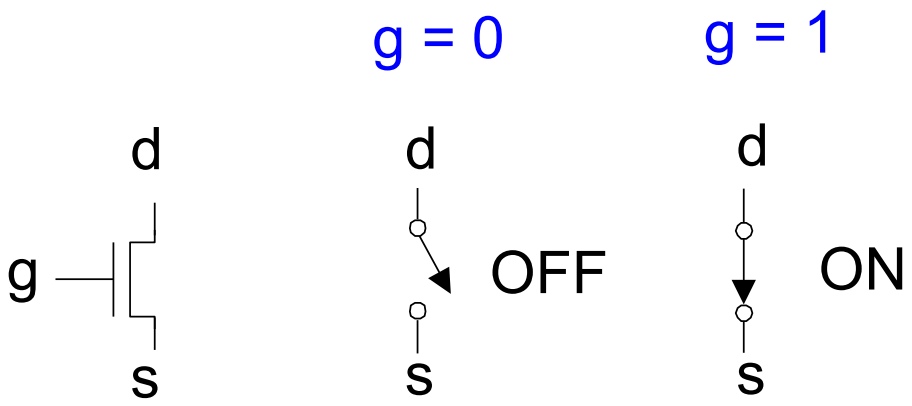
a	b	c	out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1



- Din perspectiva teoretică, este irelevantă tehnologia în care porțile logice sunt construite, cât timp ele respectă regulile algebrei binare.

Tranzistoare

- Toate porțile logice sunt construite din tranzistoare
- Comutator controlat în tensiune cu trei porturi
  - 2 sunt conectate împreună în funcție de tensiunea aplicată pe al treilea port
  - d și s sunt conectate (ON) dacă g este 1 logic

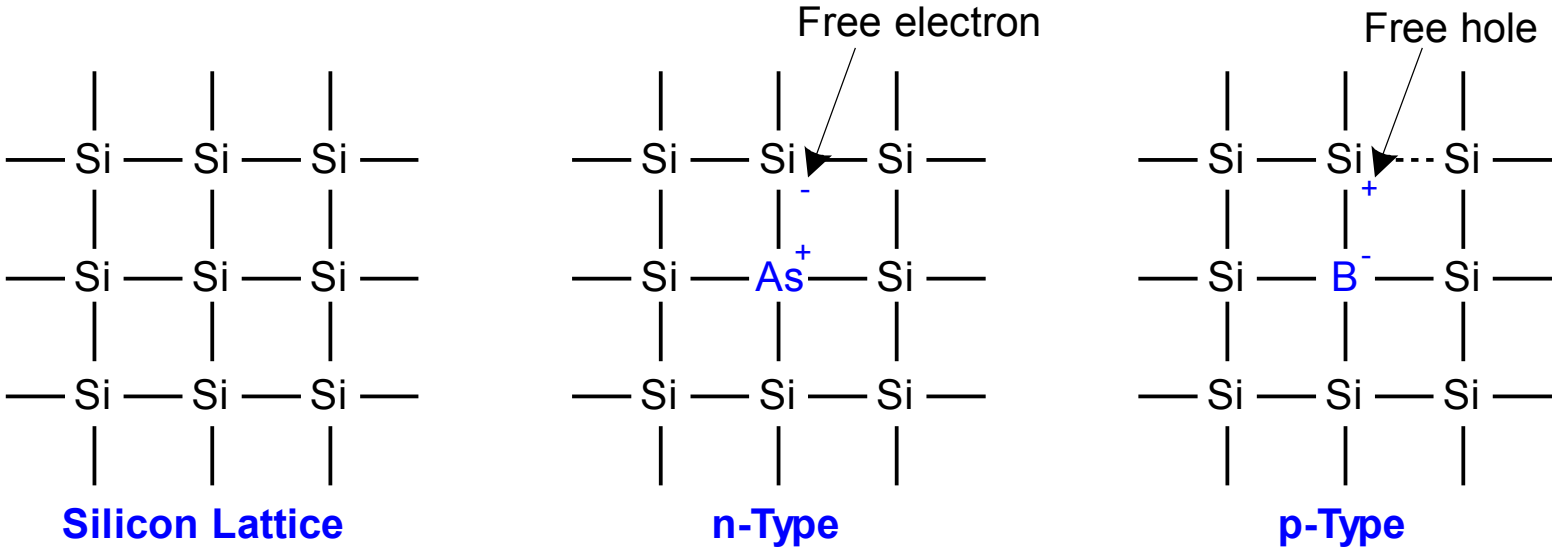


Robert Noyce, 1927-1990

- Poreclit “Primarul din Silicon Valley”
- Co-fondator Fairchild Semiconductor în 1957
- Co-fondator Intel în 1968
- Co-inventatorul circuitului integrat

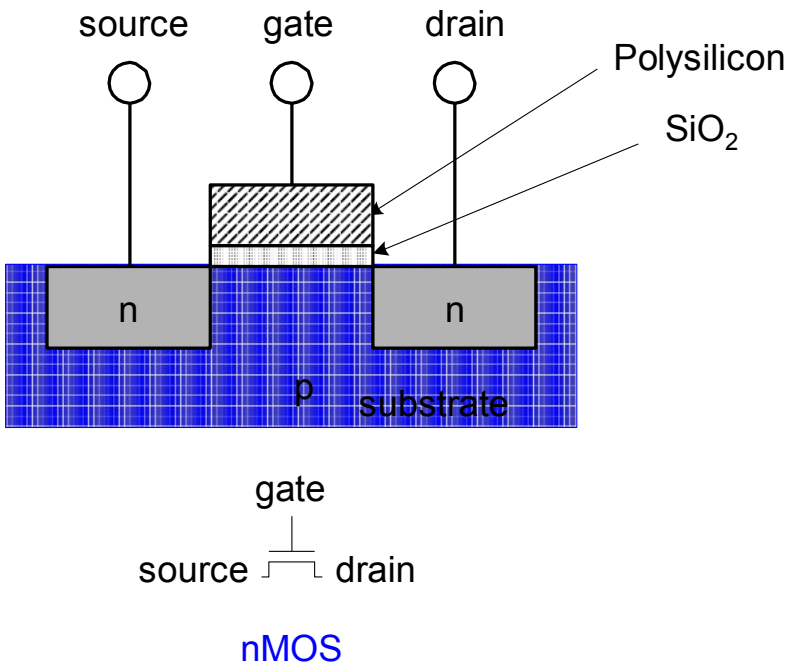
Siliciul

- Tranzistoare construite din siliciu, un semiconductor
- Siliciul pur nu este un conductor foarte bun (nu are sarcini libere)
- Siliciul dopat este un bun conductor (sarcini libere)
  - Tip-n (sarcini *n*egative libere, electroni)
  - Tip-p (sarcini *p*ozitive libere, goluri)



Tranzistoare MOS

- **Metal oxide silicon (MOS):**
  - Poartă din polisiliciu (sau **Metal**)
  - Izolator din **Oxid** (dioxid de siliciu)
  - **Siliciu** dopat

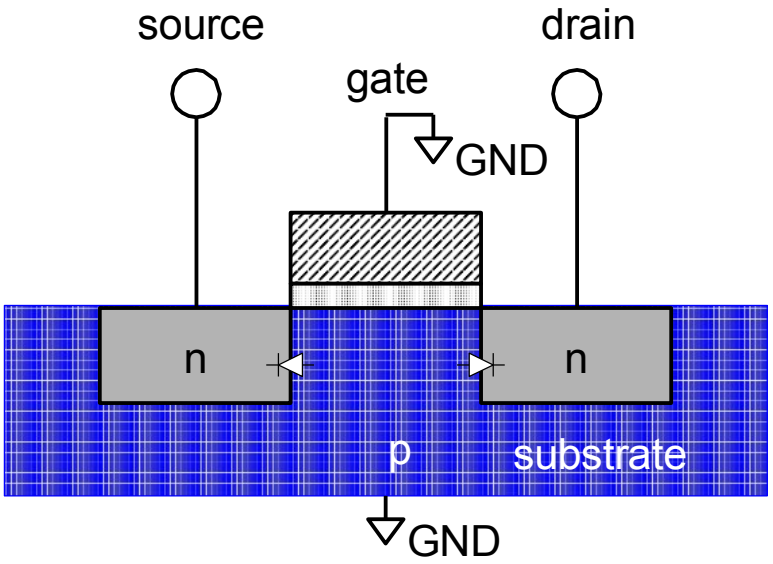


Tranzistoare: nMOS

Embedded Systems Laboratory

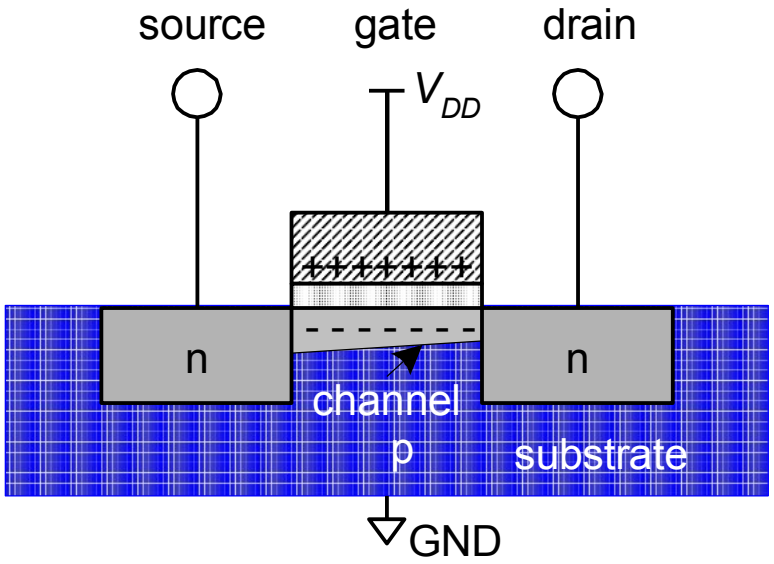
Gate = 0

OFF (nu avem conexiune între sursă și drenă)



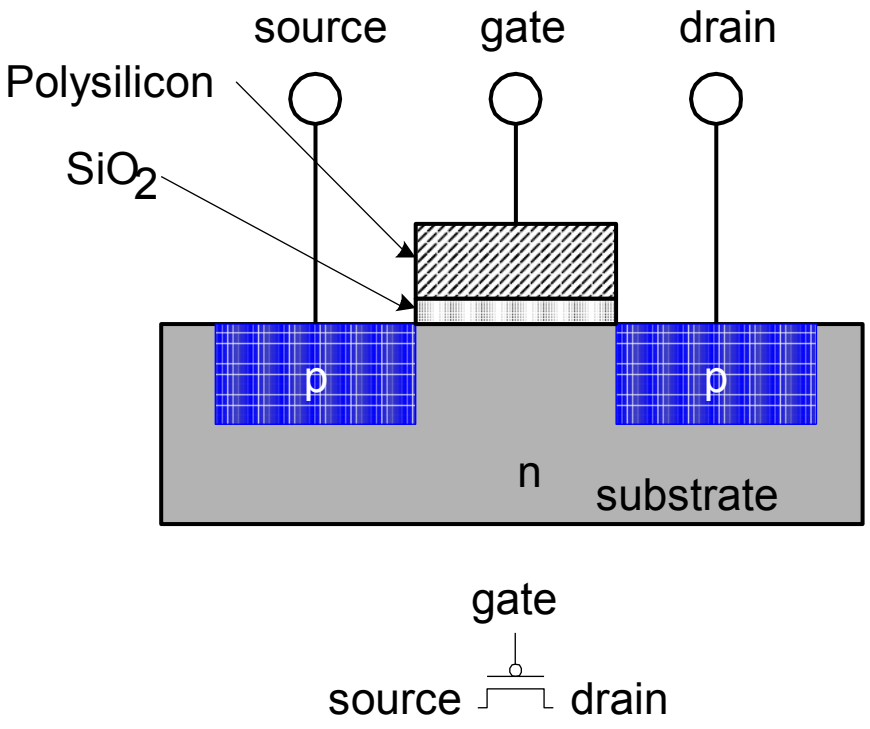
Gate = 1

ON (canal de conducție între sursă și drenă)

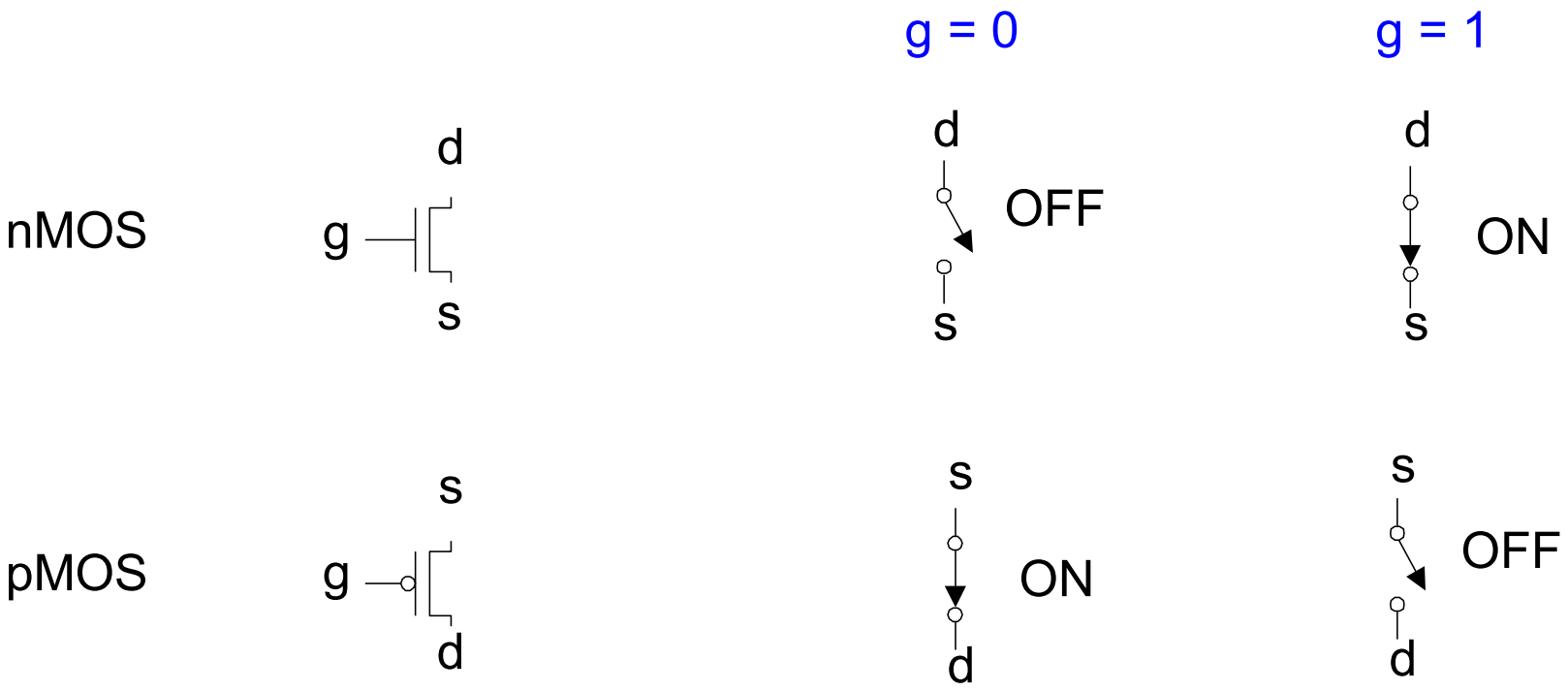


Tranzistoare: pMOS

- pMOS este opusul nMOS
  - ON când Poarta = 0
  - OFF când Poarta = 1

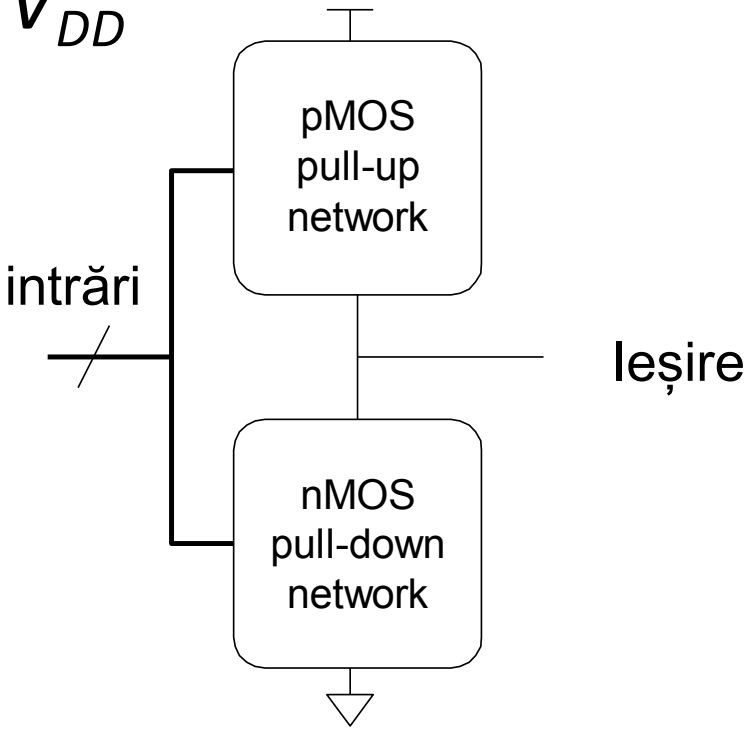


Funcționarea tranzistoarelor



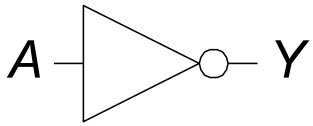
Funcționarea tranzistoarelor

- **nMOS:** lasă să treacă 0-uri, deci conectăm sursa la GND
- **pMOS:** lasă să treacă 1-uri, deci conectăm sursa la V_{DD}



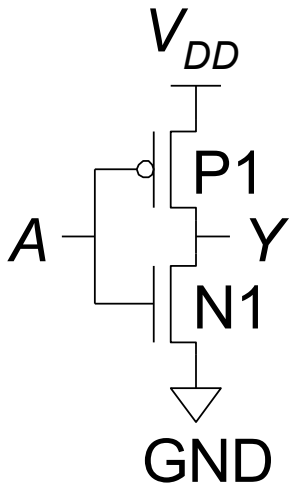
Porți CMOS: Poarta NOT

NOT



$Y = \overline{A}$

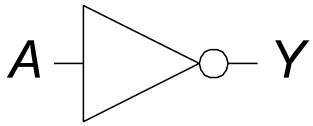
A	Y
0	1
1	0



A	P1	N1	*Y*
0			
1			

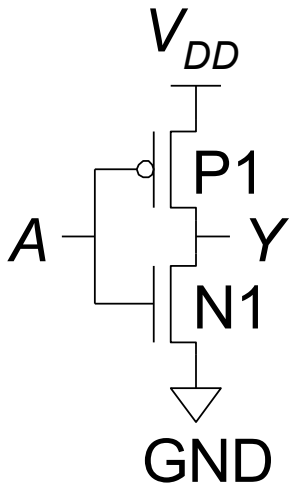
Porți CMOS: Poarta NOT

NOT



$Y = \overline{A}$

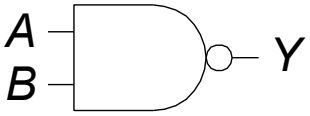
A	Y
0	1
1	0



A	P1	N1	*Y*
0	ON	OFF	1
1	OFF	ON	0

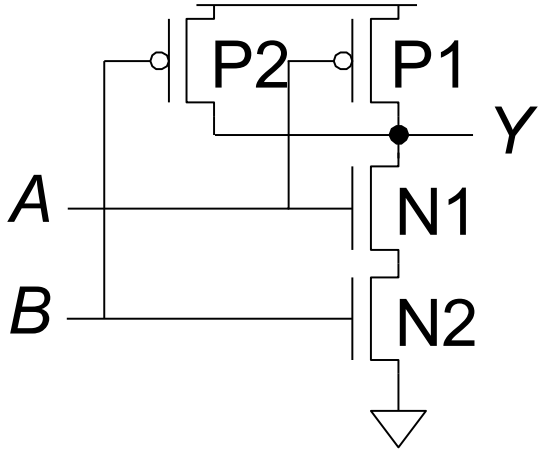
Porți CMOS: Poarta NAND

NAND



$Y = \overline{AB}$

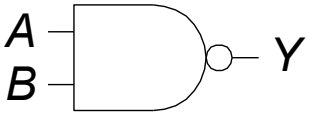
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



A	B	P1	P2	N1	N2	Y
0	0					
0	1					
1	0					
1	1					

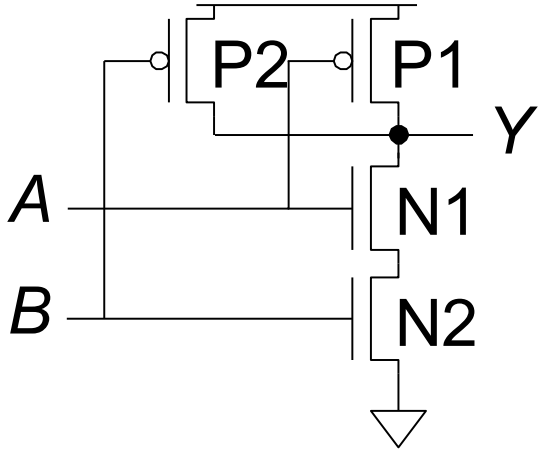
Porți CMOS: Poarta NAND

NAND



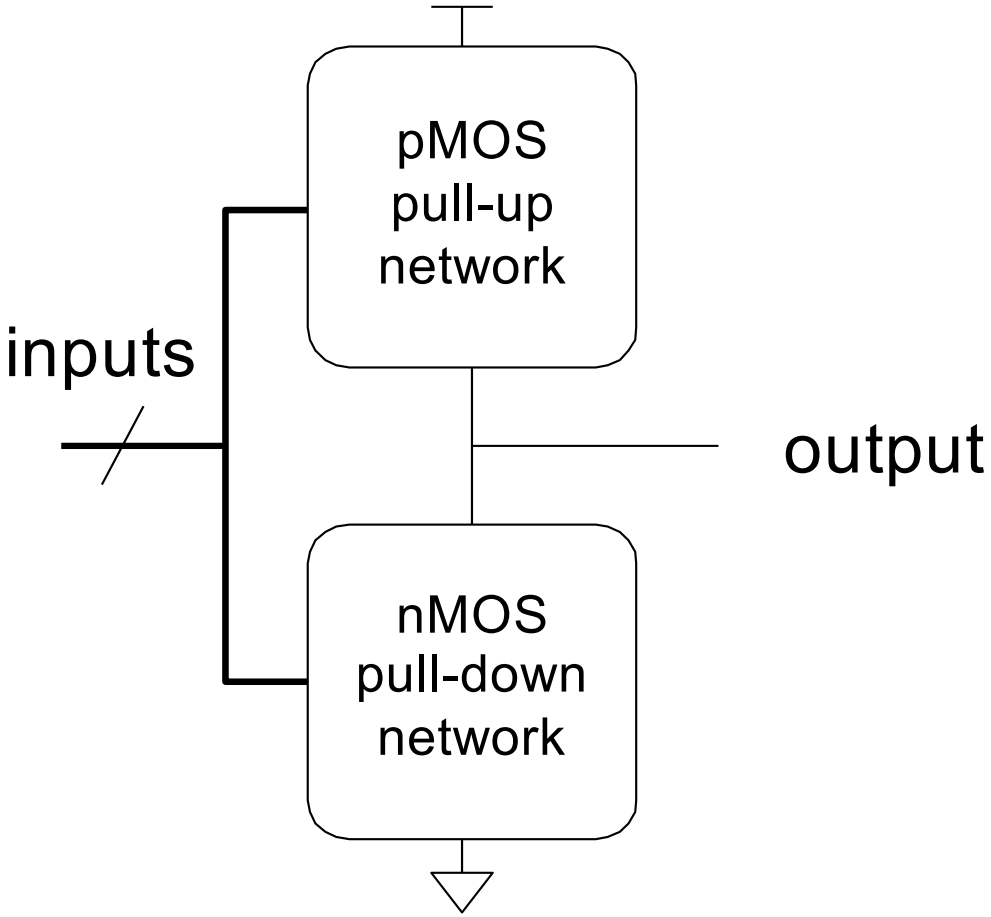
$Y = \overline{AB}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



A	*B*	P1	P2	N1	N2	*Y*
0	0	ON	ON	OFF	OFF	1
0	1	ON	OFF	OFF	ON	1
1	0	OFF	ON	ON	OFF	1
1	1	OFF	OFF	ON	ON	0

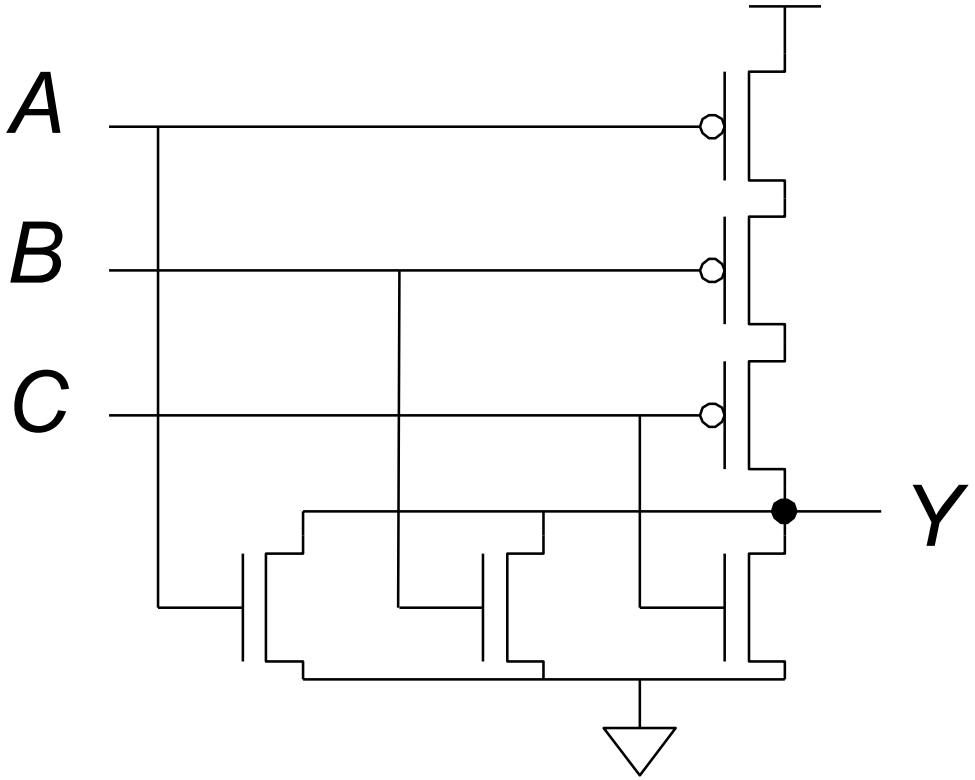
Structura unei porți CMOS



Poarta NOR

Cum construim o poartă NOR cu trei intrări?

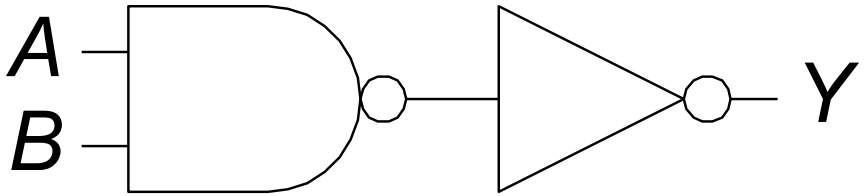
Poarta NOR3



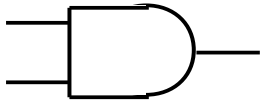
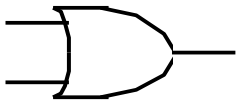
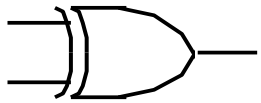
Alte porți CMOS

Cum construim o poartă AND cu două intrări?

Poarta AND2



Semnale, operatori și porți logice

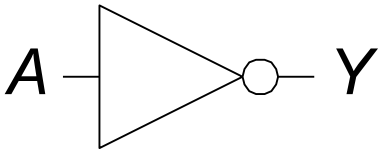
Name	NOT	AND	OR	XOR
Graphical symbol				
Operator sign and alternate(s)	x' $\neg x$ or $\overline{x}$	xy $x \wedge y$	$x \vee y$ $x + y$	$x \oplus y$ $x \not\equiv y$
Output is 1 iff:	Input is 0	Both inputs are 1s	At least one input is 1	Inputs are not equal
Arithmetic expression	$1 - x$	$x \times y$ or xy	$x + y - xy$	$x + y - 2xy$

Porți logice

- **Implementează funcții logice:**
  - inversiune (NOT), AND, OR, NAND, NOR, etc.
- **O intrare:**
  - Poarta NOT, buffer
- **Două intrări:**
  - AND, OR, XOR, NAND, NOR, XNOR
- **Intrări multiple**

Porți logice cu o intrare

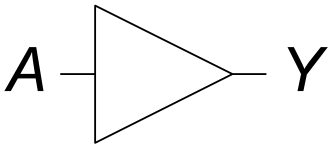
NOT



$Y = \overline{A}$

A	Y
0	
1	

BUF

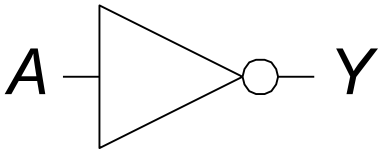


$Y = A$

A	Y
0	
1	

Porți logice cu o intrare

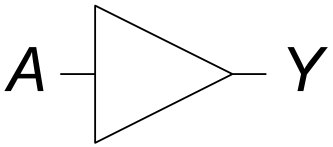
NOT



$Y = \overline{A}$

A	Y
0	1
1	0

BUF

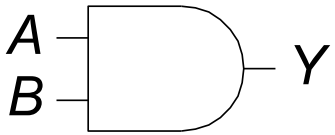


$Y = A$

A	Y
0	0
1	1

Porți logice cu două intrări

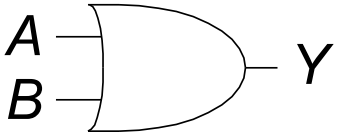
AND



$Y = AB$

A	B	Y
0	0	
0	1	
1	0	
1	1	

OR

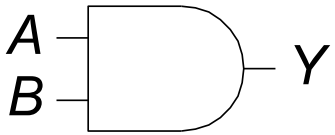


$Y = A + B$

A	B	Y
0	0	
0	1	
1	0	
1	1	

Porți logice cu două intrări

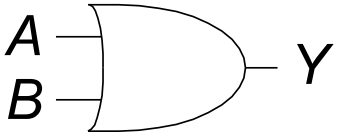
AND



$Y = AB$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR



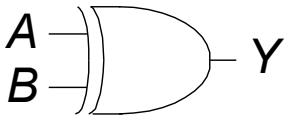
$Y = A + B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Mai multe porți logice cu două intrări

Embedded Systems Laboratory

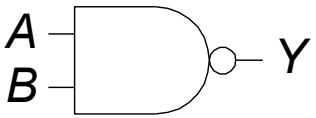
XOR



$Y = A \oplus B$

A	B	Y
0	0	
0	1	
1	0	
1	1	

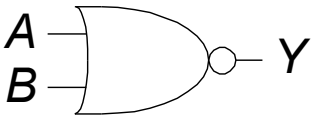
NAND



$Y = \overline{AB}$

A	B	Y
0	0	
0	1	
1	0	
1	1	

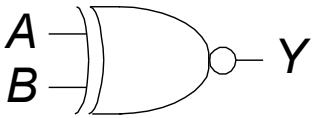
NOR



$Y = \overline{A + B}$

A	B	Y
0	0	
0	1	
1	0	
1	1	

XNOR



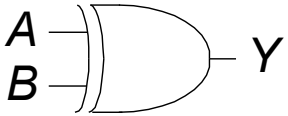
$Y = \overline{A \oplus B}$

A	B	Y
0	0	
0	1	
1	0	
1	1	

Mai multe porți logice cu două intrări

Embedded Systems Laboratory

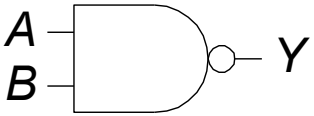
XOR



$Y = A \oplus B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

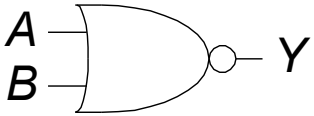
NAND



$Y = \overline{AB}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

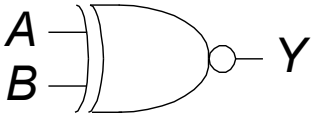
NOR



$Y = \overline{A + B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR

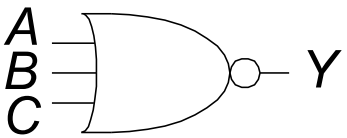


$Y = \overline{A \oplus B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Porți logice cu mai multe intrări

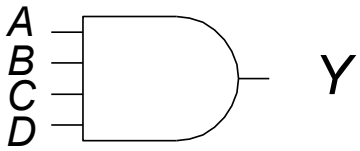
NOR3



$$Y = \overline{A+B+C}$$

A	B	C	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

AND4

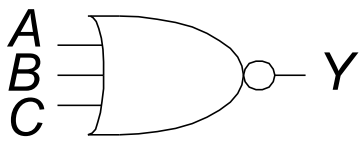


$$Y = ABCD$$

A	B	C	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Porți logice cu mai multe intrări

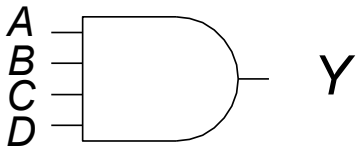
NOR3



$$Y = \overline{A+B+C}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

AND4



$$Y = ABCD$$

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

- Multi-input XOR: Odd parity

Niveluri Logice

- Valori discrete de tensiune reprezintă 1 și 0
- De exemplu:
  - 0 = *ground* (GND) sau 0 volți
  - 1 = V_{DD} sau 5 volți
- Dar 4.99 volți? Este un 0 sau un 1?
- Dar 3.2 volți?

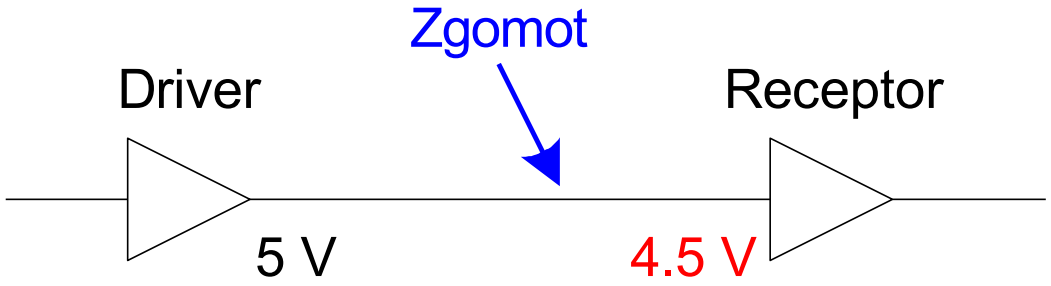
Niveluri Logice

- *Domeniu* de tensiuni pentru 1 și 0
- Domenii diferite pentru intrări și ieșiri, pentru a lua în calcul și *zgomotul*

Ce este zgomotul?

Ce este zgomotul?

- **Orice degradează semnalul**
  - E.g., rezistența, zgomotul sursei de alimentare, cuplajele parazite de la conductorii învecinați etc.
- **Exemplu:** o poartă (driver) pune la ieșirea sa 5 V dar, din cauza rezistenței liniei lungi, receptorul primește 4.5 V

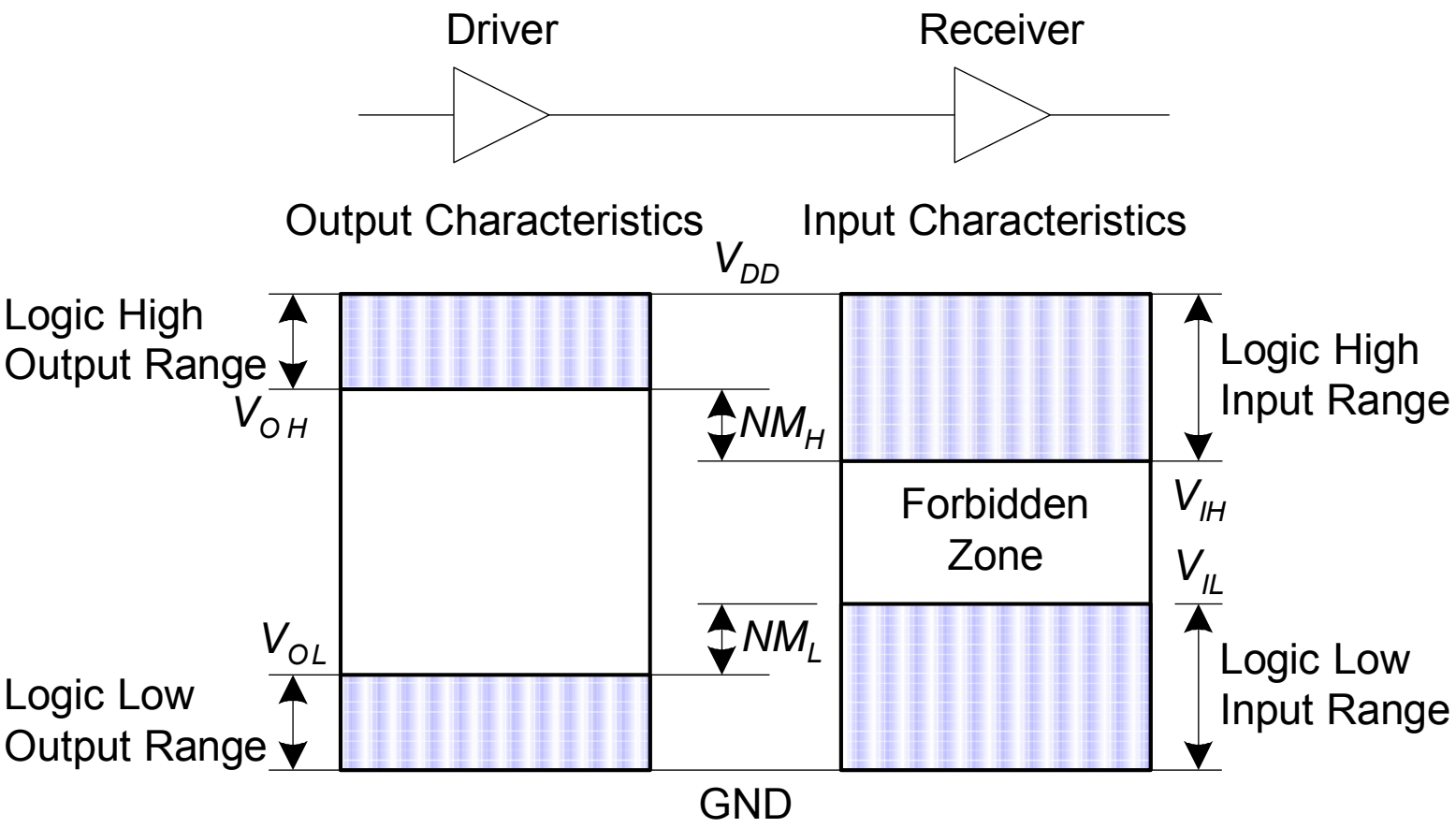


Analiza statică

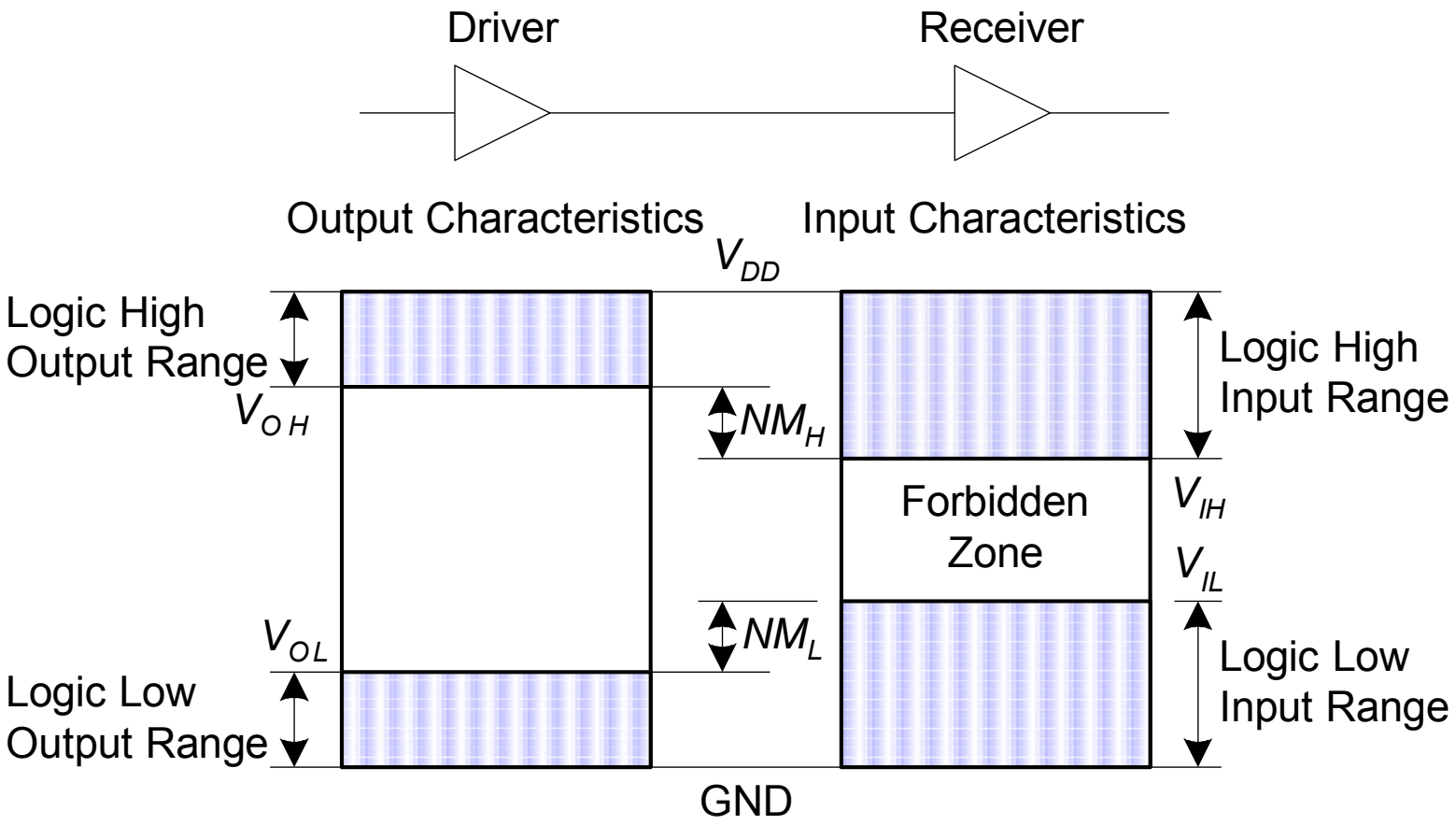
- Pornind de la niște intrări logice valide, fiecare element de circuit trebuie să producă ieșiri valide din punct de vedere logic

- Trebuie să folosim o gamă restrânsă de tensiuni pentru a reprezenta valorile discrete

Niveluri logice



Limite de zgomot



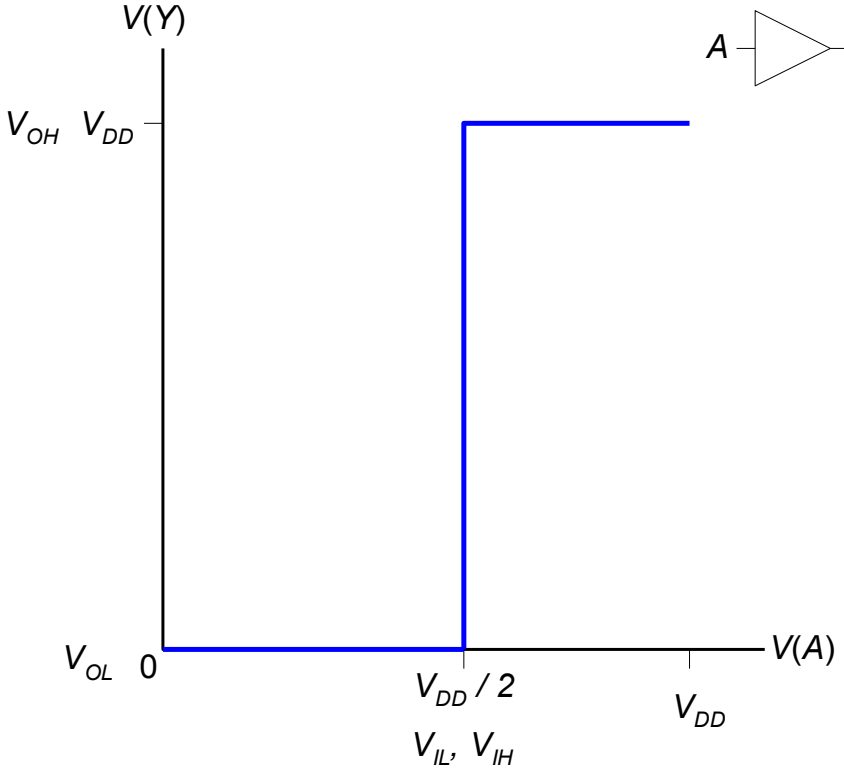
$$NM_H = V_{OH} - V_{IH}$$

$$NM_L = V_{IL} - V_{OL}$$

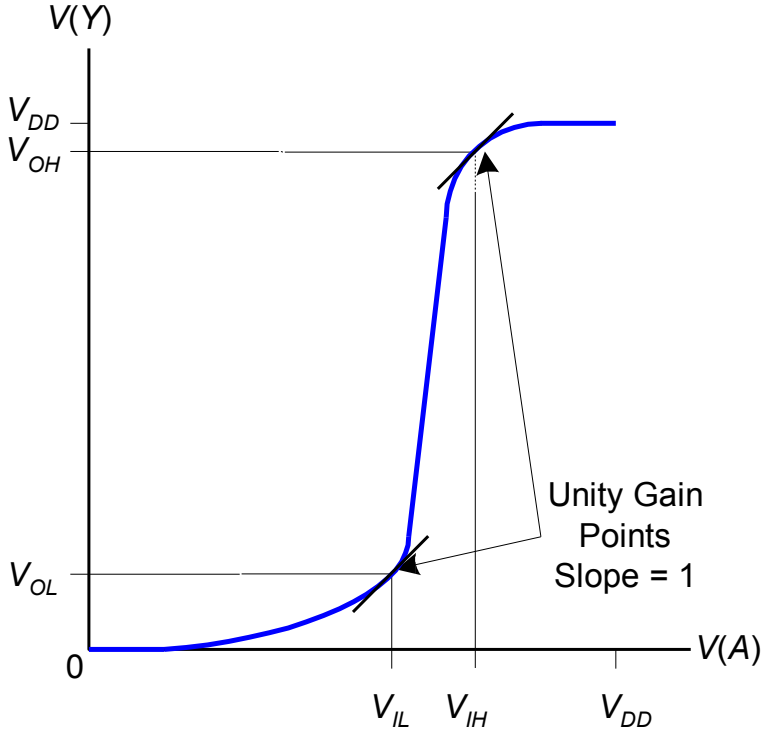
Caracteristici de transfer în c.c.

Embedded Systems Laboratory

Buffer ideal:



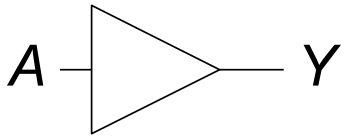
Buffer real:

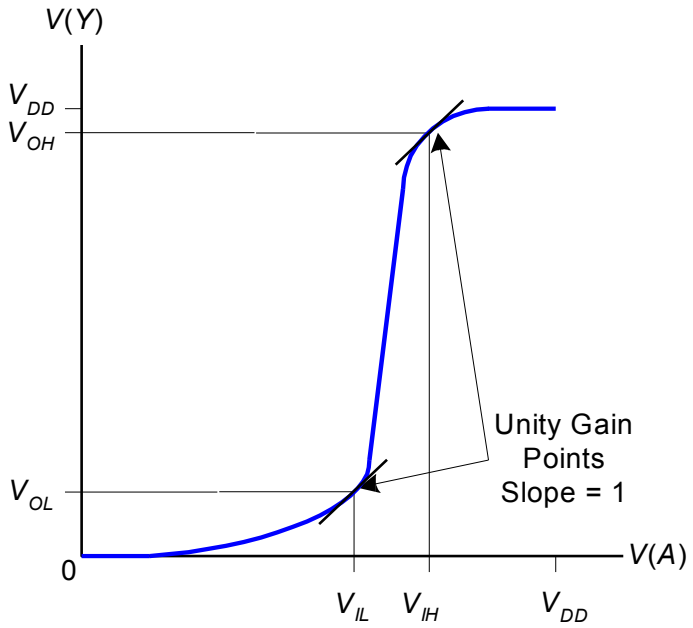


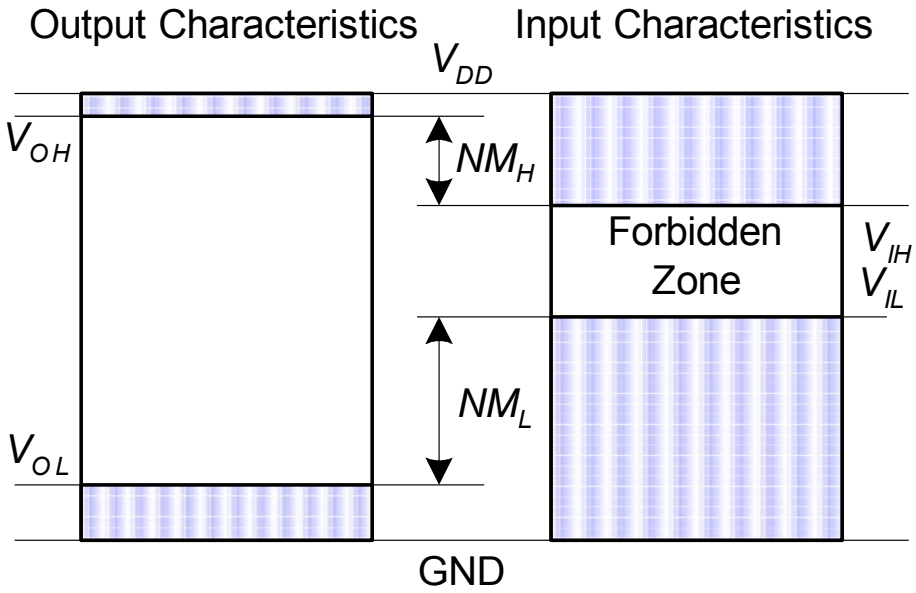
$NM_H = NM_L = V_{DD}/2$

$NM_H , NM_L < V_{DD}/2$

Caracteristici de transfer în c.c.







Scalarea V_{DD}

- In anii 1970 și 1980, V_{DD} = 5 V
- V_{DD} a scăzut
  - Tranzistoarele s-au micșorat
  - Consumă mai puțină putere
- 3.3 V, 2.5 V, 1.8 V, 1.5 V, 1.2 V, 1.0 V, …
- Aveți grijă când conectați componente ce funcționează la diferite nivele de tensiune

Teorie alternativă:

Circuitele funcționează pe bază de fum magic!

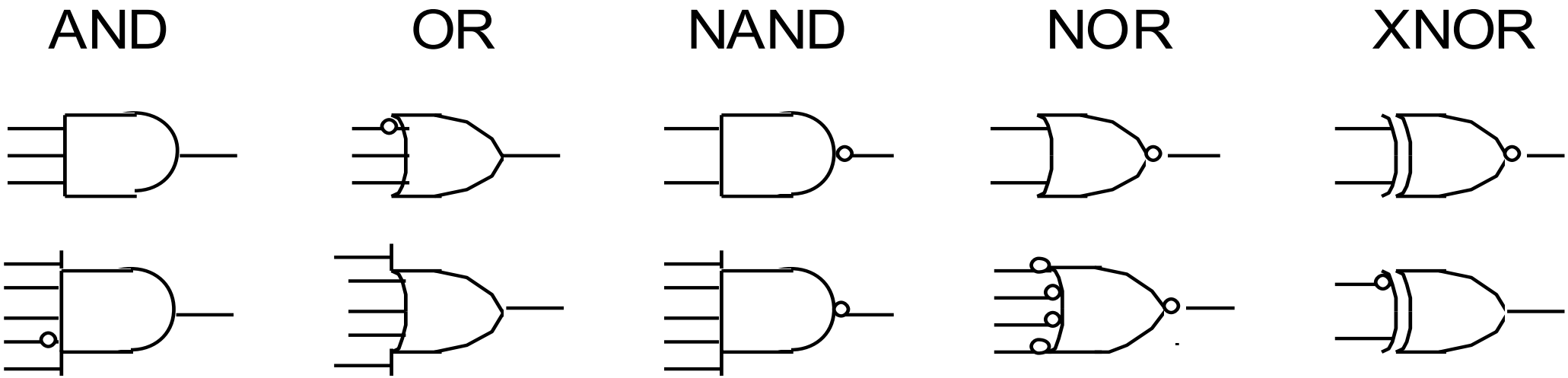
Dovada:

- Dacă iese fumul magic, circuitul nu mai funcționează!

Exemple de familii logice

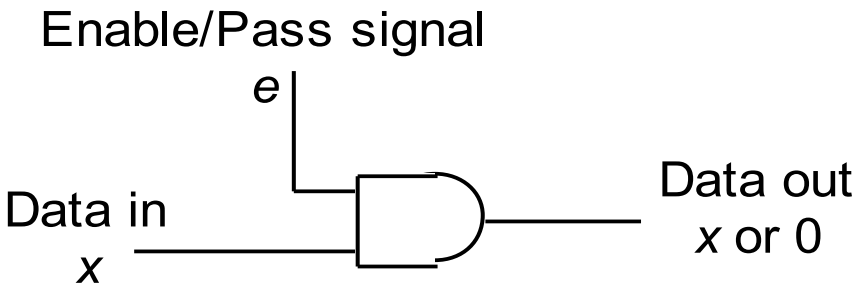
Familie Logică	V_{DD}	V_{IL}	V_{IH}	V_{OL}	V_{OH}
TTL	5 (4.75 - 5.25)	0.8	2.0	0.4	2.4
CMOS	5 (4.5 - 6)	1.35	3.15	0.33	3.84
LVTTL	3.3 (3 - 3.6)	0.8	2.0	0.4	2.4
LVCMOS	3.3 (3 - 3.6)	0.9	1.8	0.36	2.7

Variațiuni ale simbolurilor logice

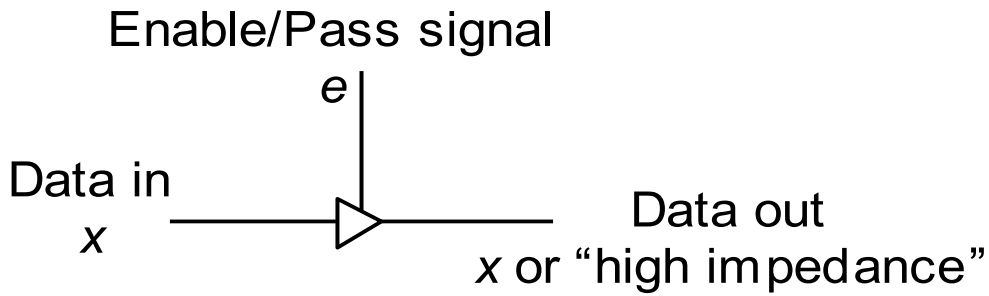


Porți logice cu mai mult de două intrări și, sau semnale inversate la intrare sau ieșire.

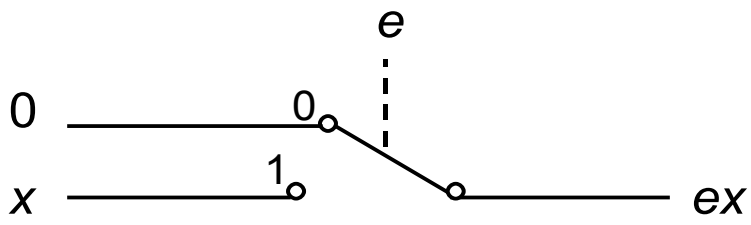
Folosirea porților logice ca elemente de control



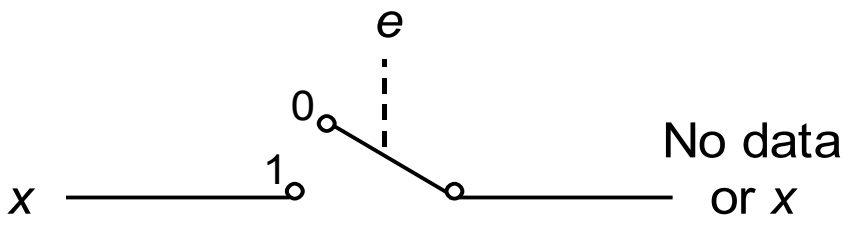
(a) AND gate for controlled transfer



(b) Tristate buffer



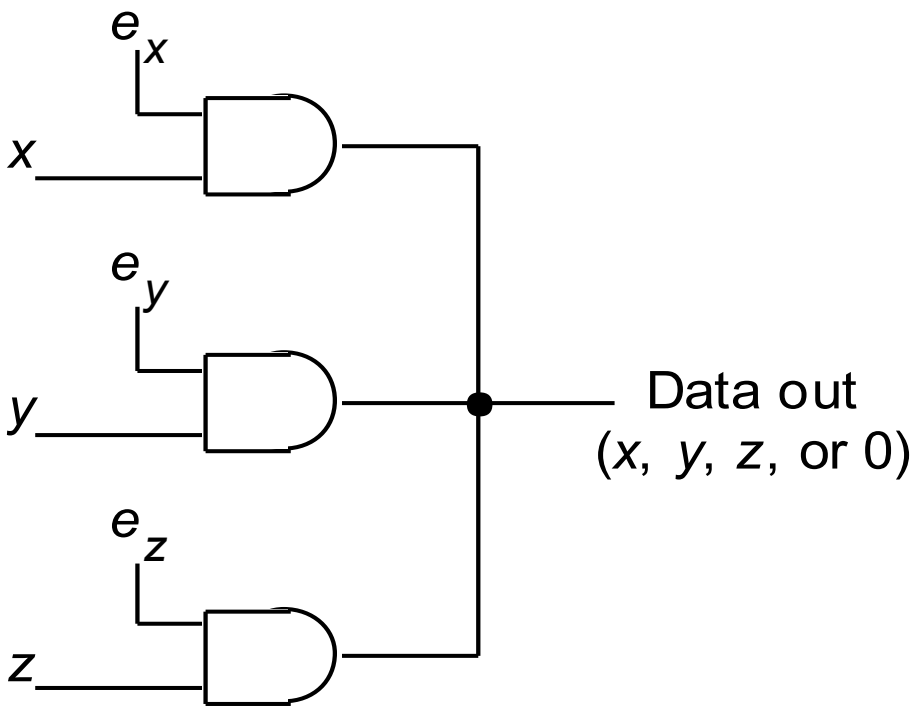
(c) Model for AND switch.



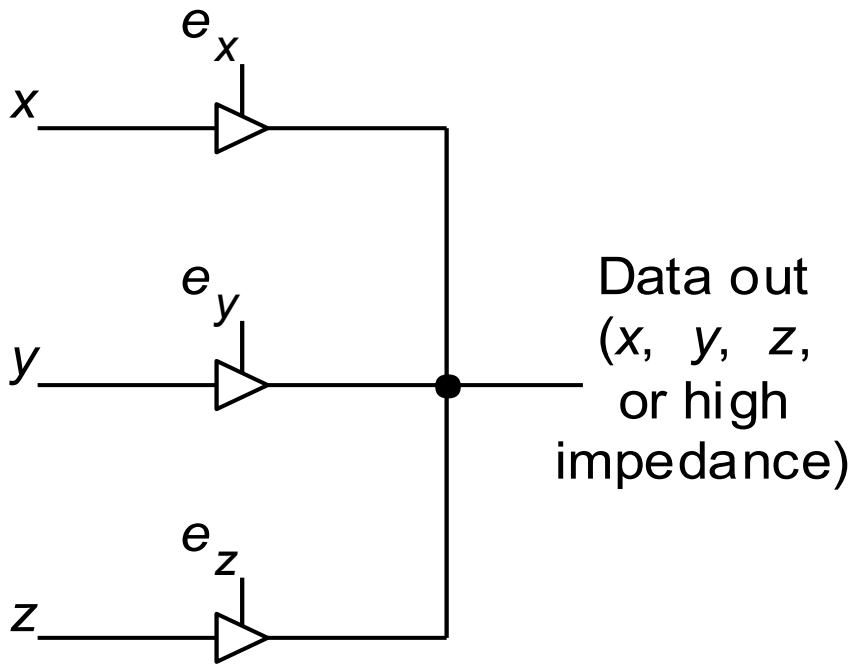
(d) Model for tristate buffer.

O poartă AND și un buffer tristate funcționează ca un întrerupător controlat. Un buffer inversor este identic din punct de vedere logic cu un inversor (poartă NOT).

Logică cablată: poarta SAU



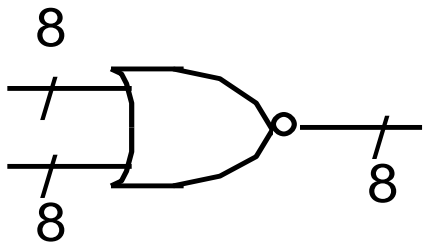
(a) Wired OR of product terms



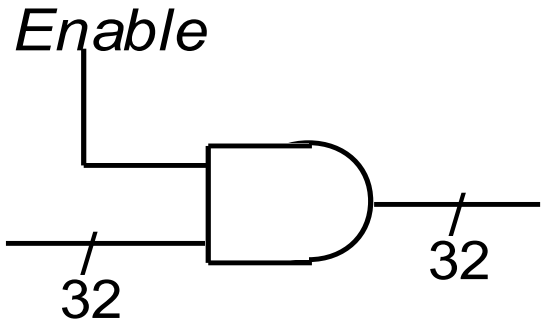
(b) Wired OR of tristate outputs

Poarta SAU cablată permite legarea mai multor semnale și a logicii de control.

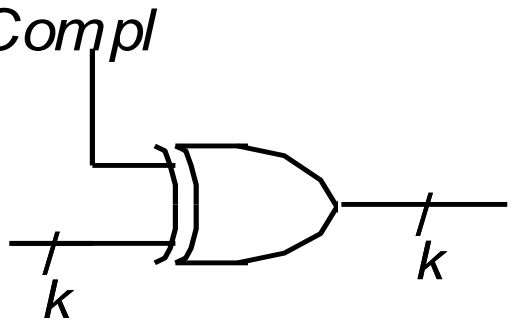
Agregarea semnalelor de date și control



(a) 8 NOR gates



(b) 32 AND gates

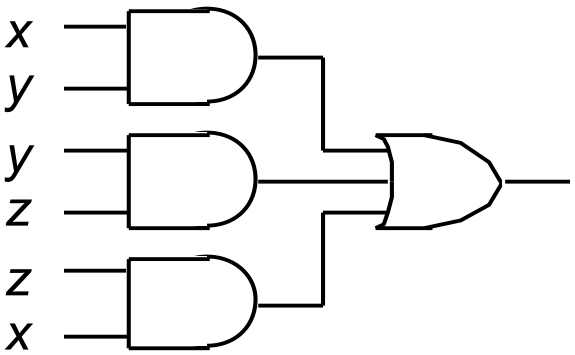


(c) *k* XOR gates

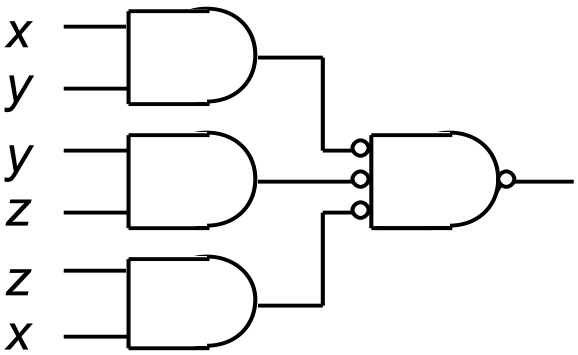
Circuite de porți logice pot fi reprezentate schematic printr-un singur simbol de poartă logică.

Proiectarea rețelelor de porți logice

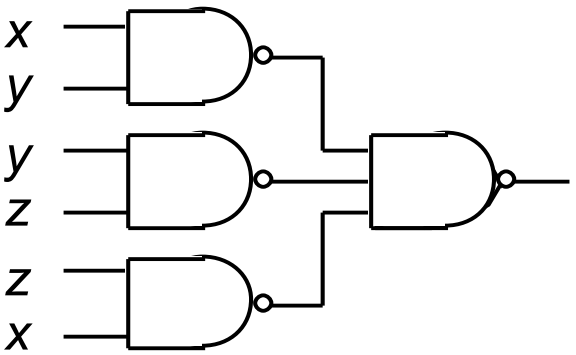
- AND-OR, NAND-NAND, OR-AND, NOR-NOR
- Optimizare logică: cost, viteză de execuție, putere disipată



(a) AND-OR circuit



(b) Intermediate circuit

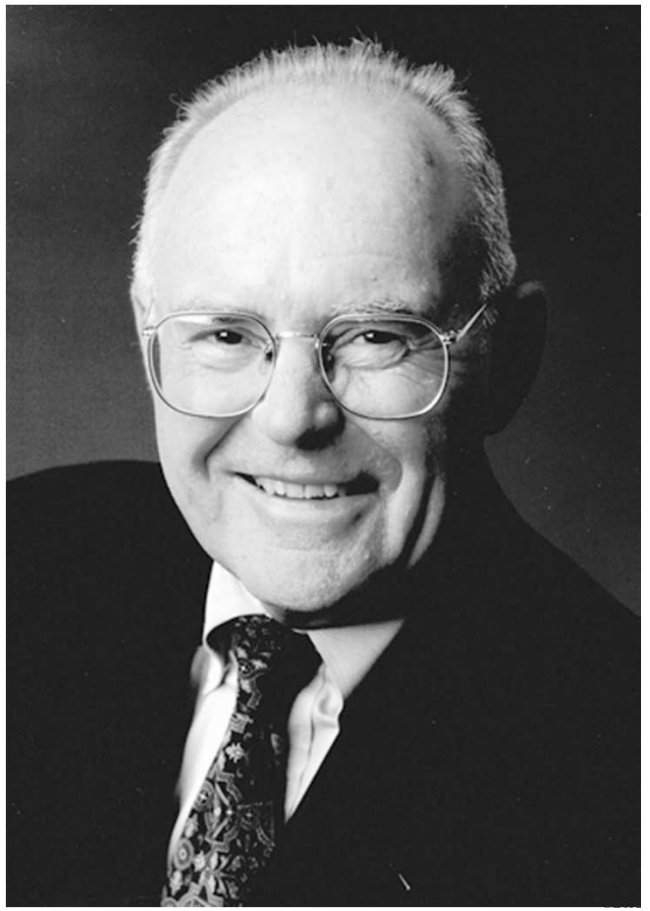


(c) NAND-NAND equivalent

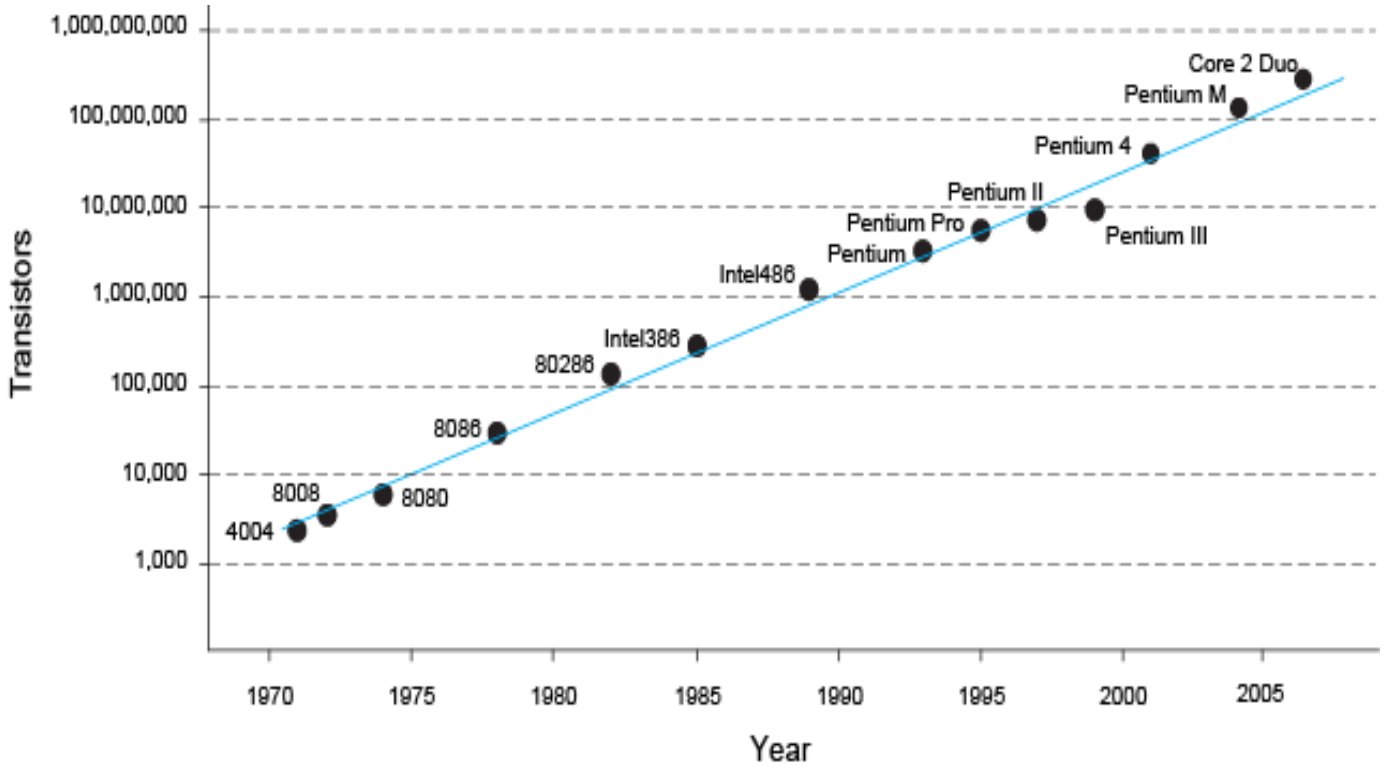
Un circuit AND-OR pe 2 niveluri și două circuite echivalente

Gordon Moore, 1929-

- Co-fondator Intel în 1968 cu Robert Noyce.
- **Legea lui Moore:** numărul de tranzistoare de pe un circuit integrat se dublează în fiecare an (observat în 1965)
- Din 1975, numărul s-a dublat la fiecare doi ani.



Legea lui Moore



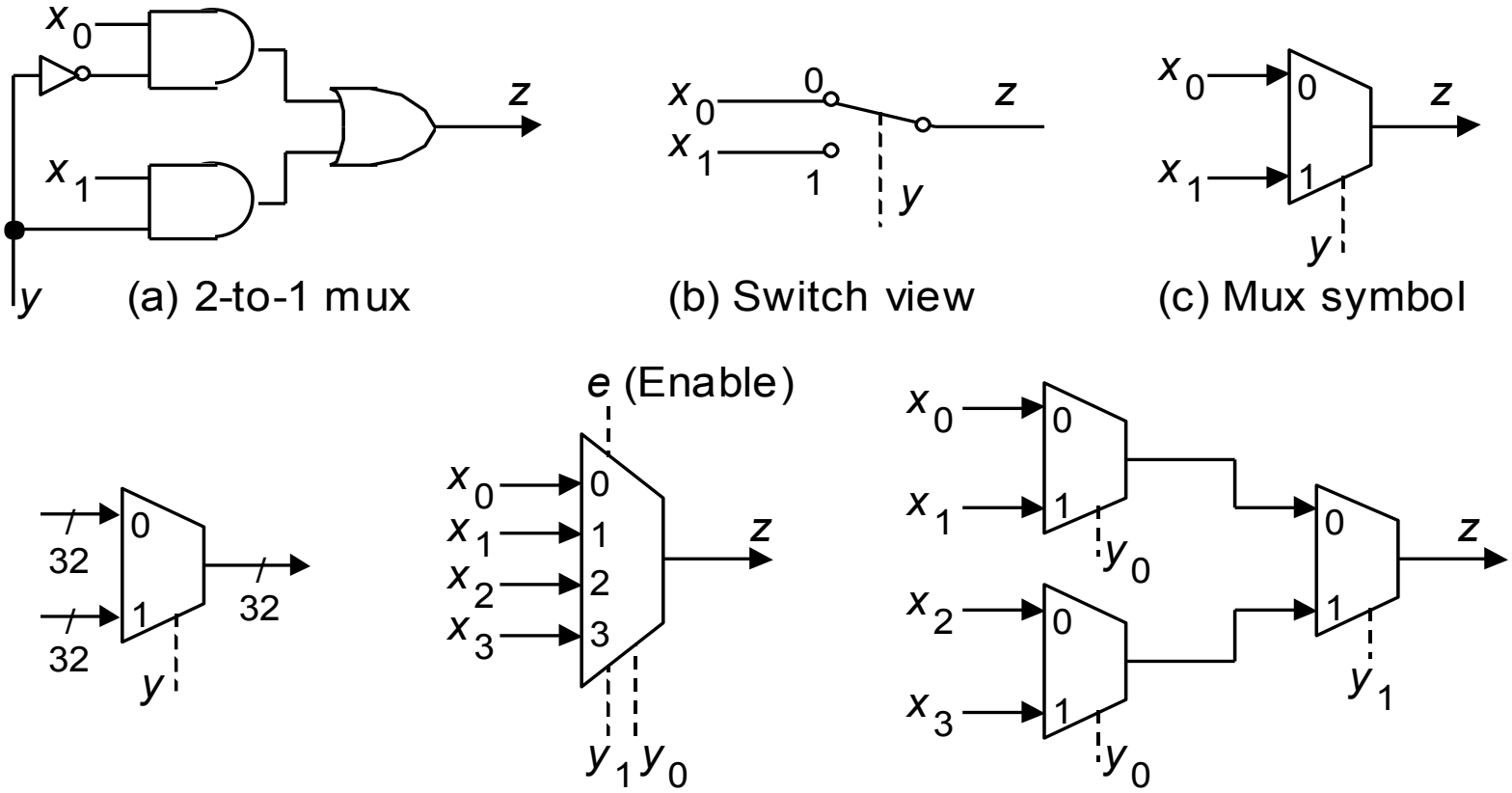
- *"If the automobile had followed the same development cycle as the computer, a Rolls-Royce would today cost $100, get one million miles to the gallon, and explode once a year . . ."*

  – Robert Cringley

Circuite combinaționale utile

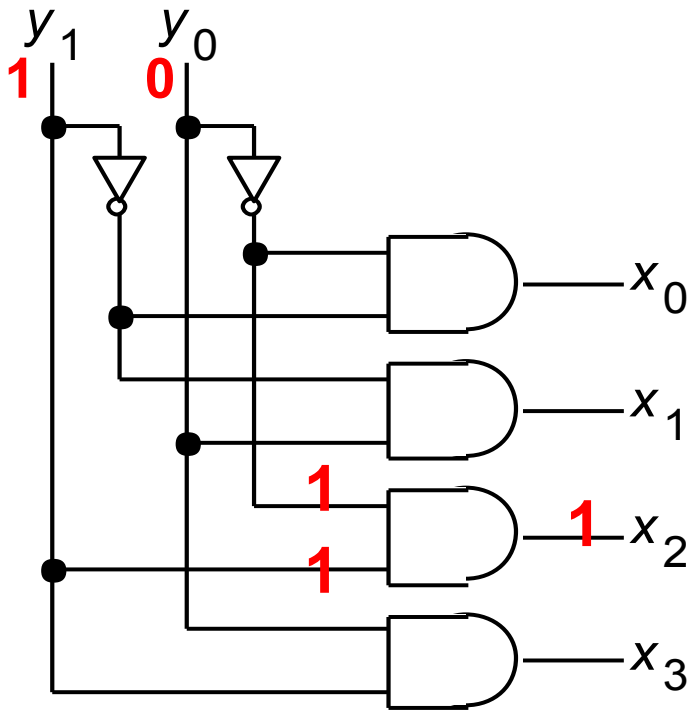
- Blocuri de nivel înalt
- Seamănă cu blocurile prefabricate folosite în construcția unei case
- Componente aritmetice (Sumatoare, Multiplicatoare, Unități Aritmetice-Logice)
- În continuare vom vorbi despre trei componente utile: multiplexoare, decodificatoare/demultiplexoare și codificatoare

Multiplexoare

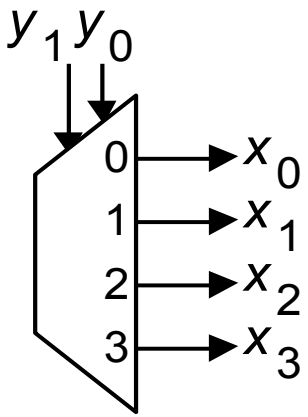


Un multiplexor(mux), sau selector, permite selectarea uneia sau mai multor intrări și rutarea acestora la ieșire, în funcție de valoarea binară a intrărilor de selecție/adresă furnizate.

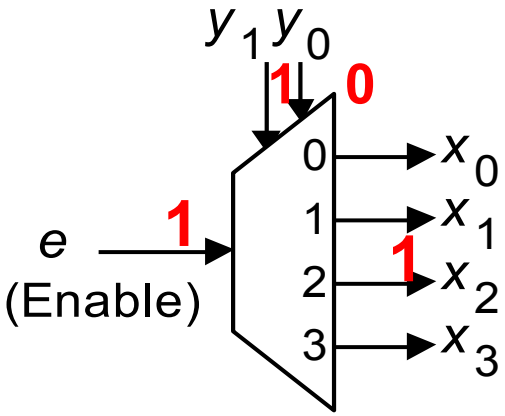
Decodificatoare/demultiplexoare



(a) 2-to-4 decoder



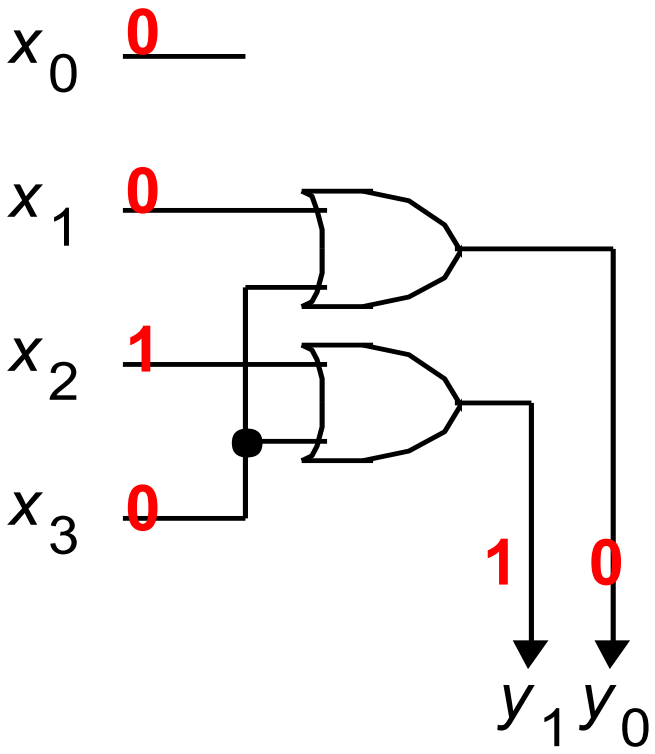
(b) Decoder symbol



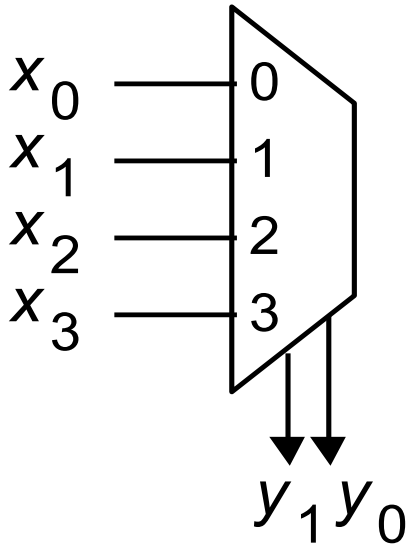
(c) Demultiplexer, or decoder with “enable”

Un decodificator permite selecția a uneia dintre cele 2^a intrări cu ajutorul a unei intrări de selecție pe *a* biți. Un demultiplexor (demux) este un decodificator care selectează o ieșire în funcție de semnalele de adresă și o intrare suplimentară de Enable.

Codificatoare



(a) 4-to-2 encoder

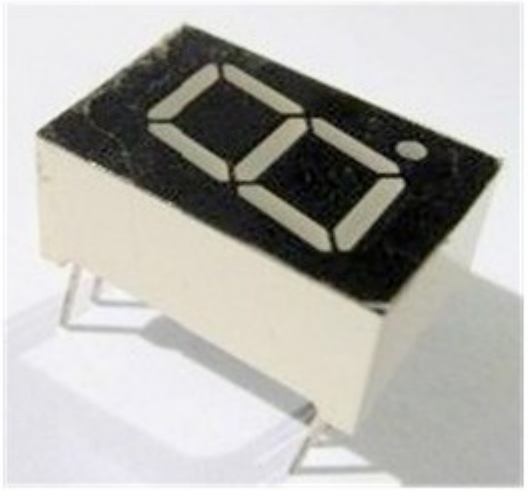


(b) Encoder symbol

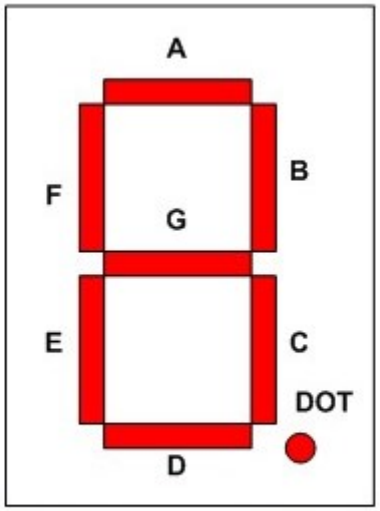
Un codificator 2^a-la-a furnizează la ieșire un număr binar pe a-biți egal cu rangul valorii unice de 1 de la intrare.

Afișarea cifrelor zecimale pe un display cu 7 segmente

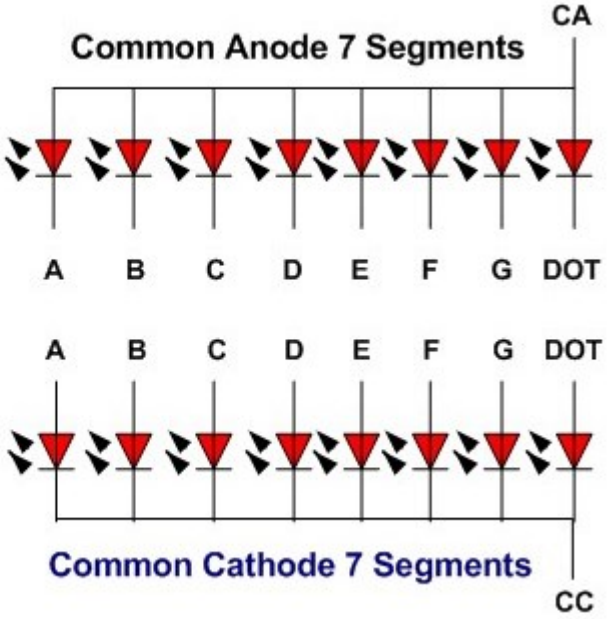
Afișaj seven-segment pentru toate cifrele zecimale. Fiecare din cele 7 segmente corespunde unui bit care poate fi activ (=> segment aprins) sau inactiv (segment stins). In acest fel putem afisa cifre pe un ecran.



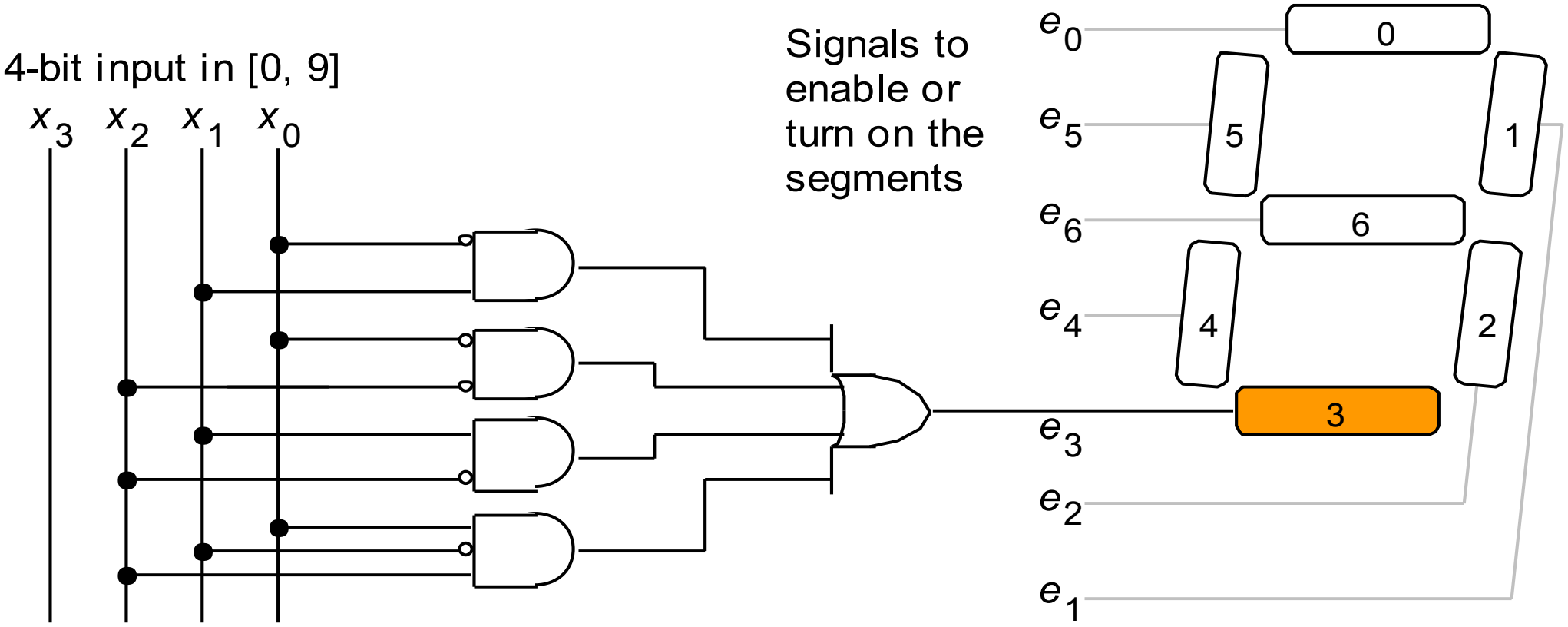
Typical 7 Segments Display



The 7 Segment's Name and the DOT

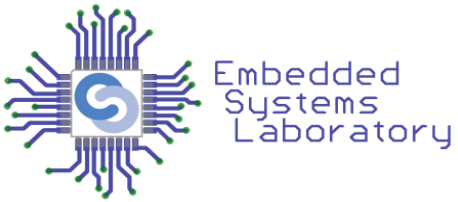


Decodificator BCD-to-Seven-Segment



Circuitul logic care generează semnalul de Enable pentru segmentul cel mai de jos al unui digit 7-segment

Logică combinațională programabilă

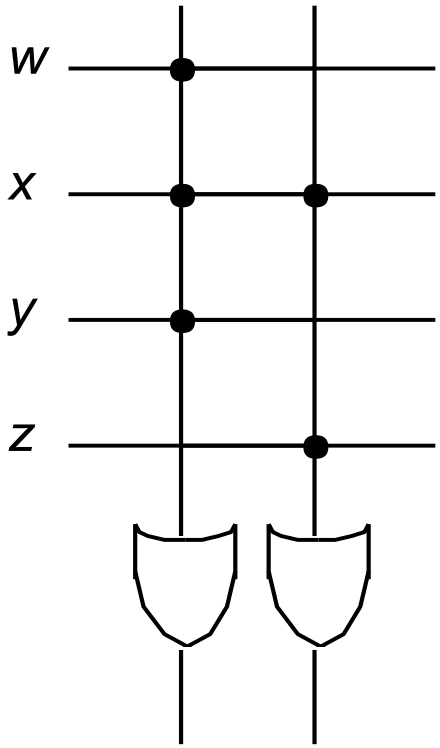


Un circuit combinațional programabil poate să înlocuiască funcționalitatea mai multor porți logice

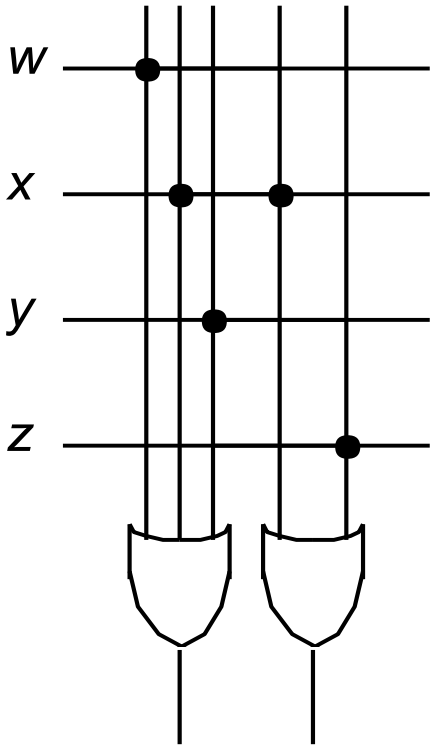
Programate prin întreruperea unor conexiuni existente (*fuses*) sau stabilirea de noi conexiuni(*antifuses*)

- Programmable ROM (PROM)
- Programmable array logic (PAL)
- Programmable logic array (PLA)

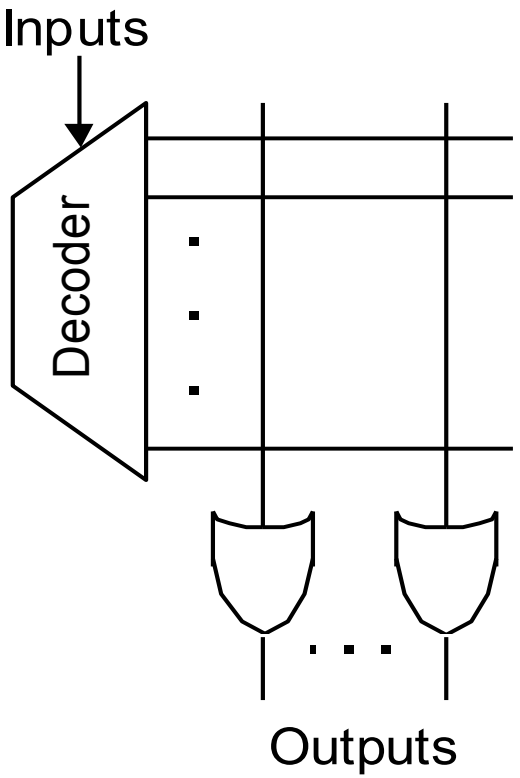
Circuite PROM



(a) Programmable OR gates



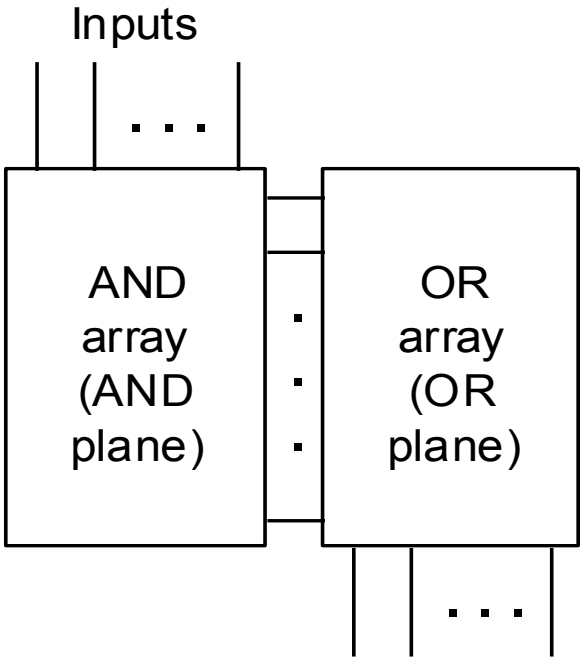
(b) Logic equivalent of part a



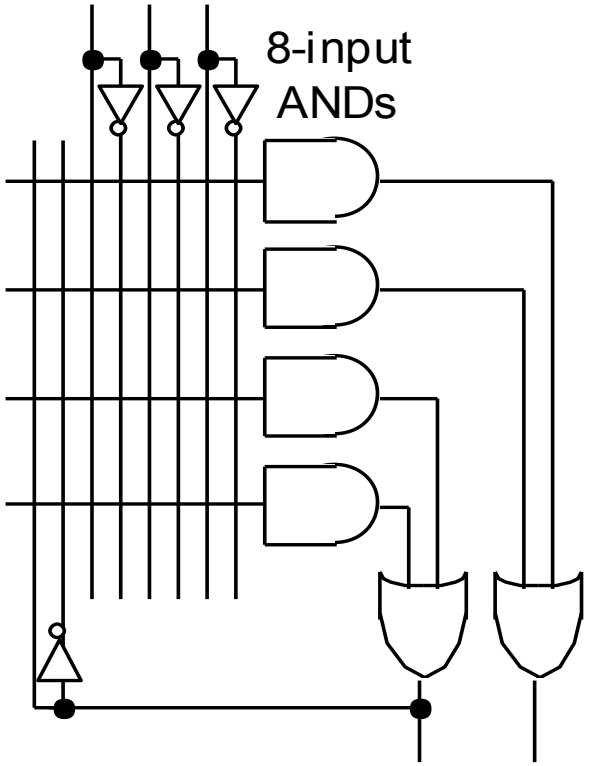
(c) Programmable read-only memory (PROM)

Conexiuni programabile și folosirea lor într-un circuit PROM.

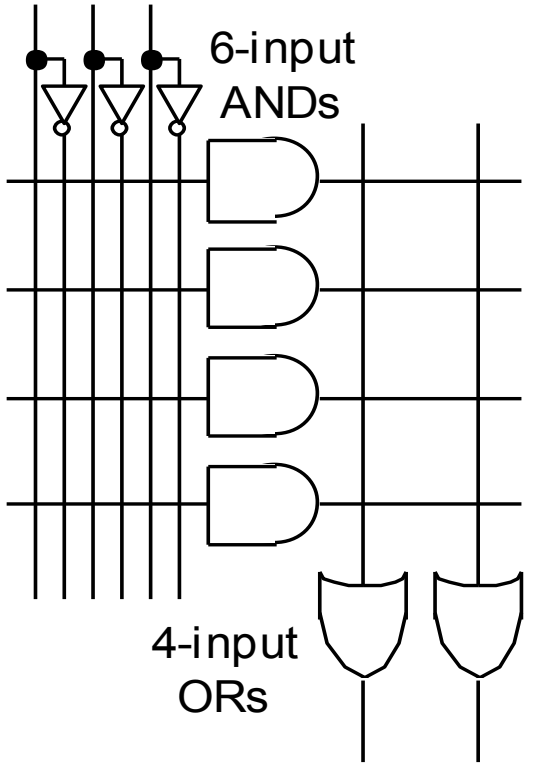
Circuite PAL & PLA



(a) General programmable combinational logic



(b) PAL: programmable AND array, fixed OR array



(c) PLA: programmable AND and OR arrays

Logică combinațională programabilă: structură generală și două clase de circuite (PAL și PLA). În același circuit este nevoie și de un PROM cu un plan AND fix (rol de decodificator) și un plan OR programabil.

Constrângeri de design și temporizare

Schimbările valorilor ieșirilor oricărui circuit logic, determinate de schimbările valorilor intrărilor, nu sunt instantanee!

- Gate delay δ: de ordinul nanosecundelor
- Wire delay, inițial era neglijabil, este acum f. Important (semnalele electrice circulă cu 15cm/ns)
- Simularea circuitului pentru a determina funcționarea corectă și respectarea constrângerilor de timing.

Glitching

Folosim PAL-ul descris anterior pentru $f = x \vee y \vee z$

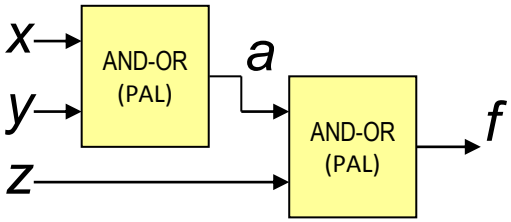
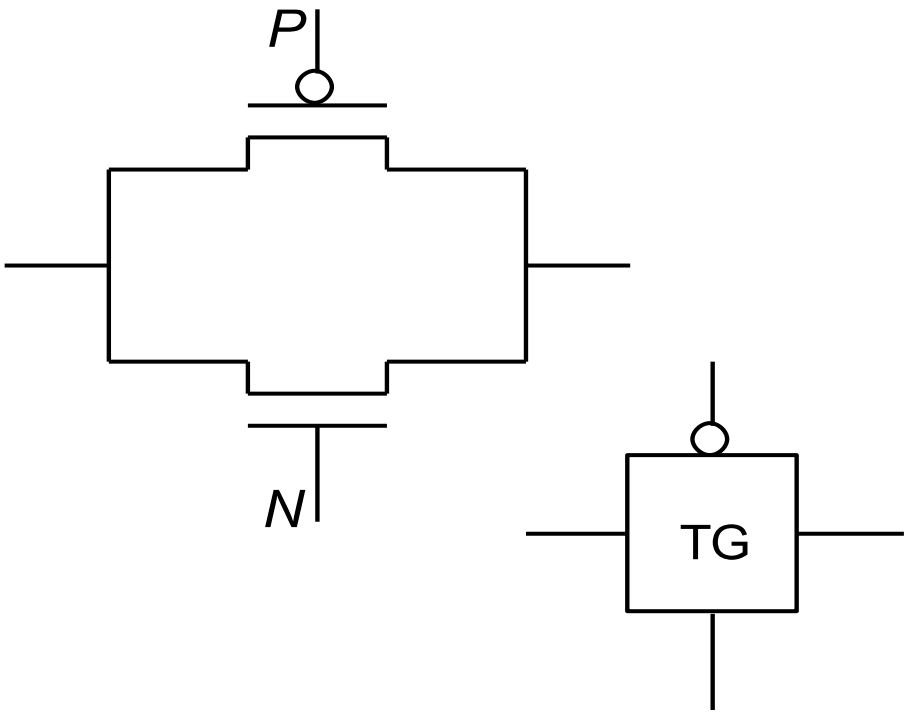
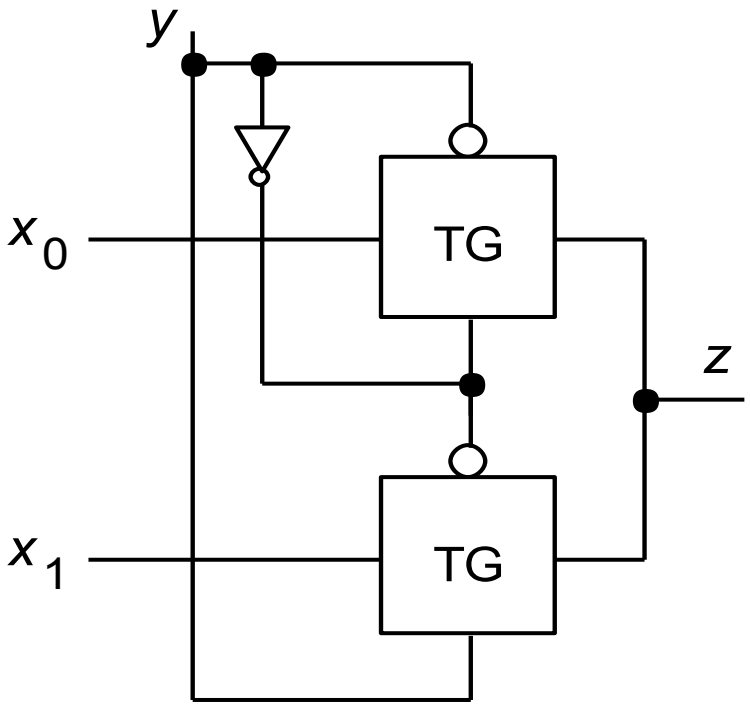


Diagrama de semnale arată fenomenul de gliching.

CMOS Transmission Gate



(a) CMOS transmission gate: circuit and symbol

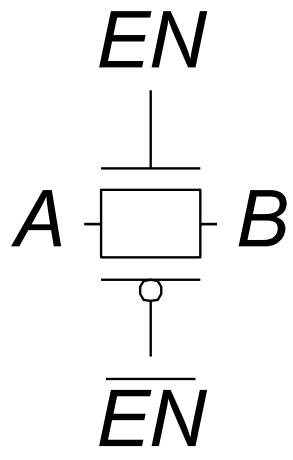


(b) Two-input mux built of two transmission gates

O poartă de transmisie CMOS și folosirea ei ca un mux 2-la-1.

Porți cu transmisie

- nMOS lasă să treacă 1 slab
- pMOS lasă să treacă 0 slab
- Poarta de transmisie e un comutator mai bun
  - Conduce bine în ambele stări
- Când *EN* = 1, comutatorul e închis (ON):
  - *EN* = 0 și *A e conectat la* B
- Când *EN* = 0, comutatorul e deschis (OFF):
  - *A nu e conectat la B*



Consumul de putere

- Putere = Energia consumată per unitatea de timp
  - Consum de putere în regim dinamic
  - Consum de putere în regim static

Puterea dinamică

- **Puterea necesară pentru a încărca capacitățile parazite ale porților**
  - Energia necesară pentru a încărca o capacitate C, la V_{DD} este CV_{DD}^2
    - Circuitul lucrează la frecvența f: tranzistoarele comută (din 1 în 0 și vice versa) la acea frecvență
    - Condensatorul este încărcat de $f/2$ ori pe secundă (descărcarea din 1 în 0 este gratis)
- Puterea dinamică:

$$P_{dynamic} = \tfrac{1}{2} f C V_{DD}^2$$

Puterea statică

- Puterea consumată când nici o poartă nu comută
- Cauzată de *curentul de mers în gol*, I_{DD} (numis și curent de *leakage*)
- Puterea statică:

$$P_{static} = I_{DD}V_{DD}$$

Exemplu de consum de putere

- Estimați consumul de putere pentru un calculator portabil cu următoarele specificații:
  - V_{DD} = 1.2 V
  - C = 20 nF
  - f = 1 GHz
  - I_{DD} = 20 mA

Exemplu de consum de putere

- Estimați consumul de putere pentru un calculator portabil cu următoarele specificații:
  - V_{DD} = 1.2 V
  - C = 20 nF
  - f = 1 GHz
  - I_{DD} = 20 mA

$$P = \tfrac{1}{2}CV_{DD}^2 f + I_{DD}V_{DD}$$

$$= \tfrac{1}{2}(20\text{ nF})(1.2\text{ V})^2(1\text{ GHz}) + (20\text{ mA})(1.2\text{ V})$$

$$= \mathbf{14.4\text{ W}}$$

Embedded Systems Laboratory

Catedra de Calculatoare
Facultatea de Automatica si Calculatoare

Calculatoare Numerice

-Cursul 2 –

De la porți logice la procesoare (2)

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Comic of the day

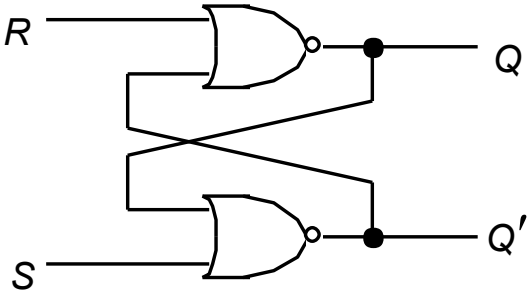
http://dilbert.com/strips/comic/2010-05-04/

Circuite digitale cu memorie

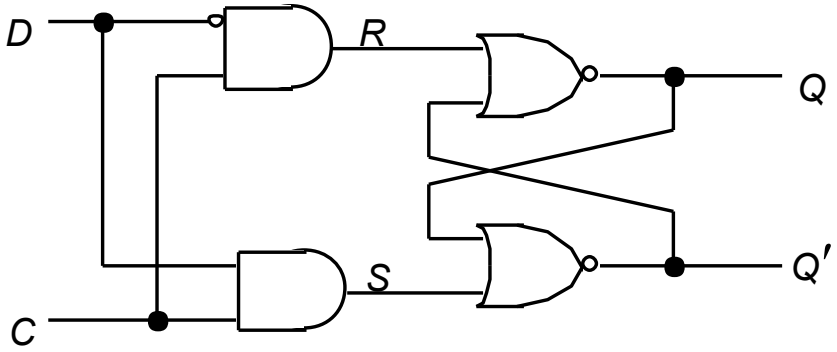
Embedded Systems Laboratory

Topics
1. Latch-uri, Flip-Flop-uri și Registre
2. Automate FSM (Finite-State Machines)
3. Proiectarea circuitelor secvențiale
4. Circuite secvențiale utile
5. Circuite secvențiale programabile
6. Semnale de sincronizare si temporizare

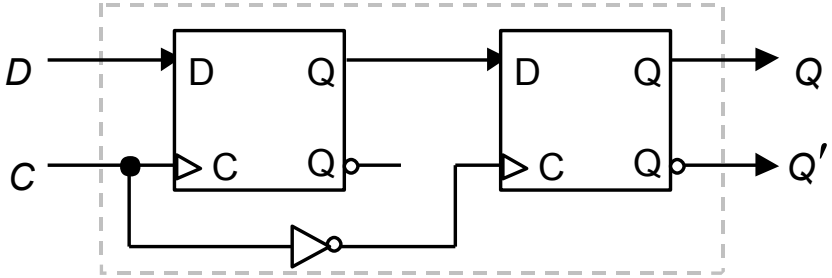
Latch-uri, Flip-Flop-uri și Registre



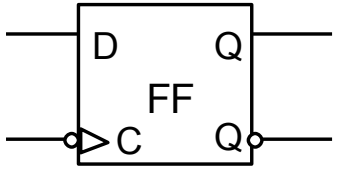
(a) SR latch



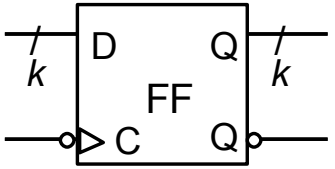
(b) D latch



(c) Master-slave D flip-flop



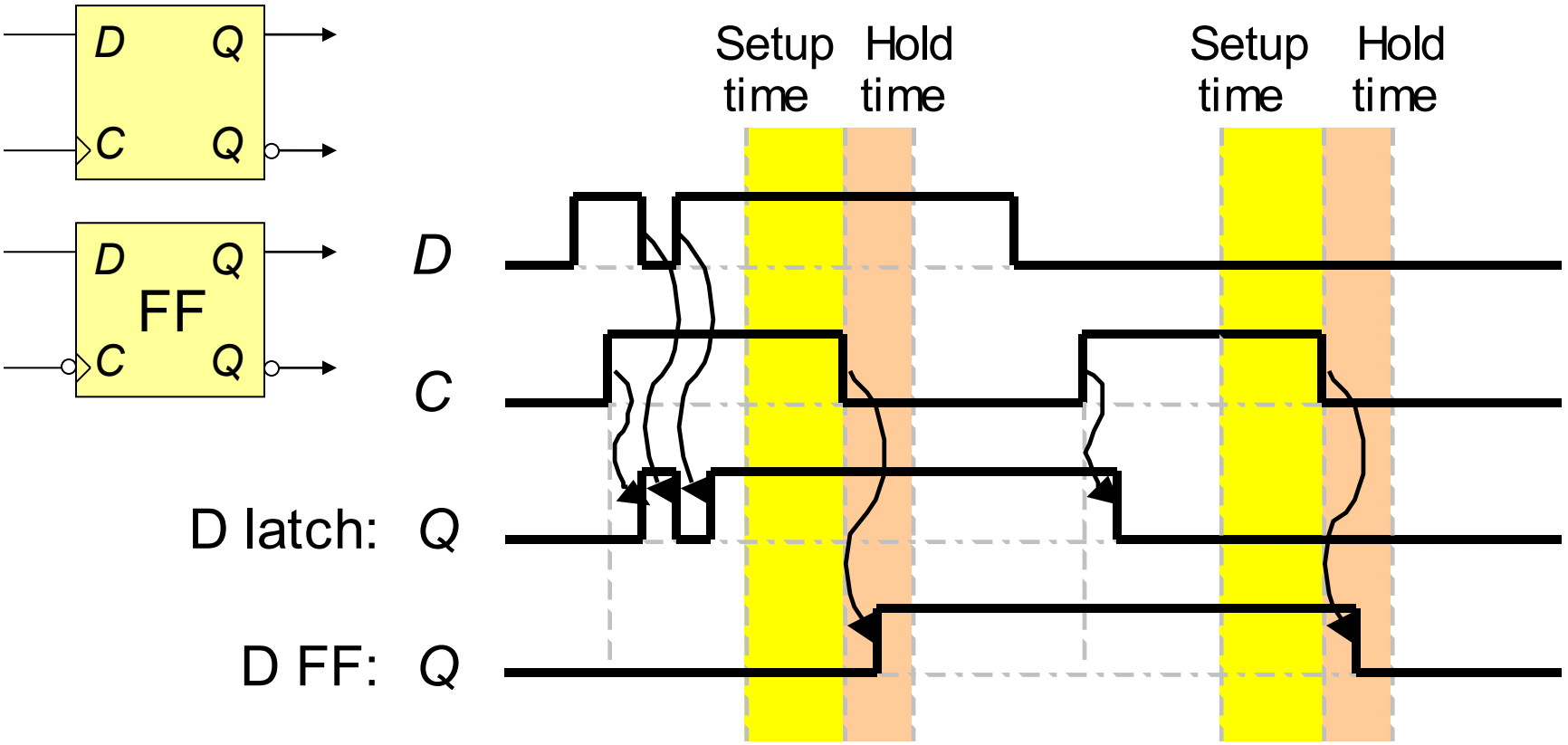
(d) D flip-flop symbol



(e) *k*-bit register

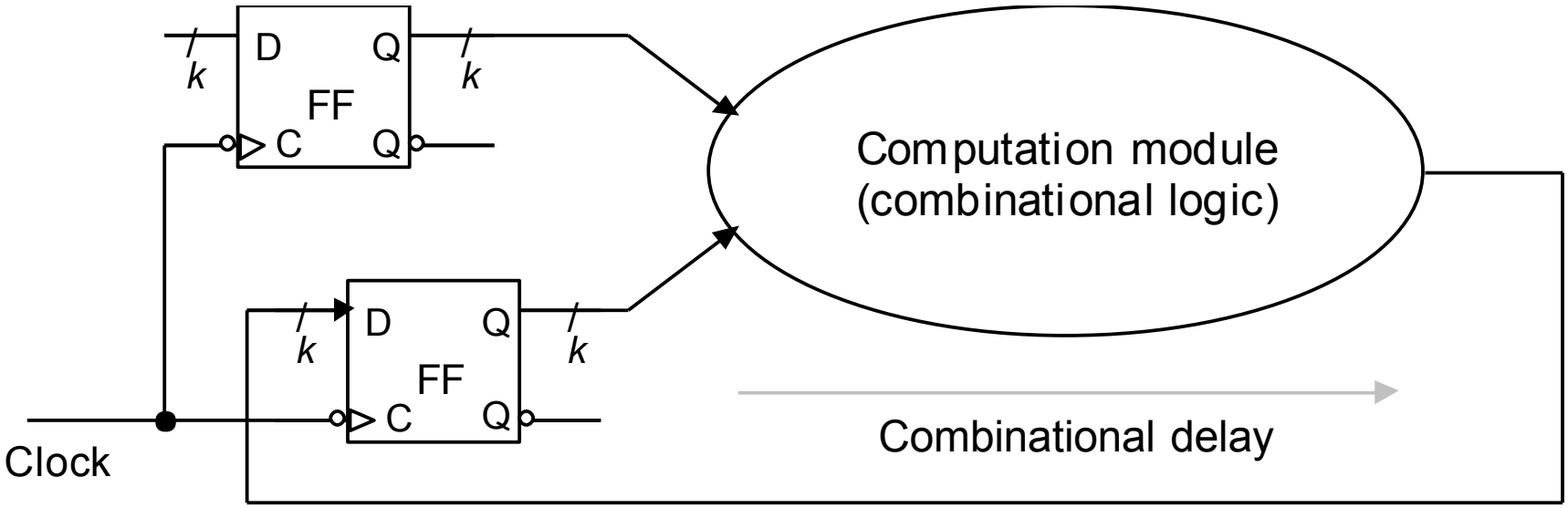
Latch-uri, flip-flop-uri și registre.

Latch-uri vs. Flip-Flop-uri



Funcționarea unui latch D și a unui flip-flop D activ pe front negativ

Citirea și modificarea bistabilelor în același ciclu de ceas



Transfer de registre folosind bistabile active pe tranzițiile semnalului de ceas.

Automate FSM

Exemplu

Current state	Input: Dime	Input: Quarter	Input: Reset
S_{00}	S_{10}	S_{25}	S_{00}
S_{10}	S_{20}	S_{35}	S_{00}
S_{20}	S_{30}	S_{35}	S_{00}
S_{25}	S_{35}	S_{35}	S_{00}
S_{30}	S_{35}	S_{35}	S_{00}
S_{35}	S_{35}	S_{35}	S_{00}

Next state

S_{00} is the initial state
S_{35} is the final state

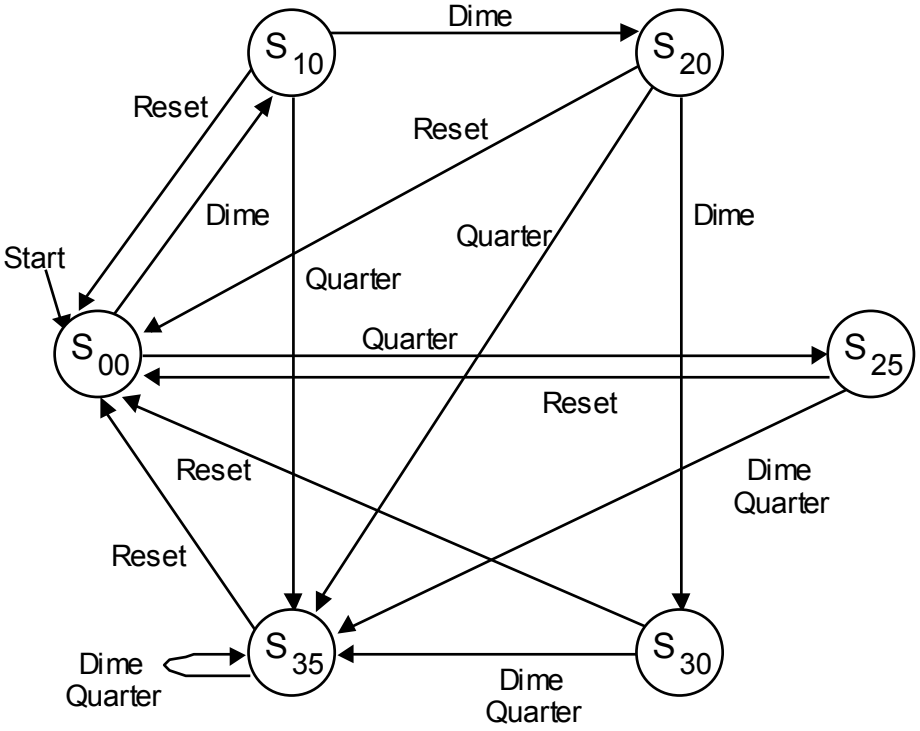
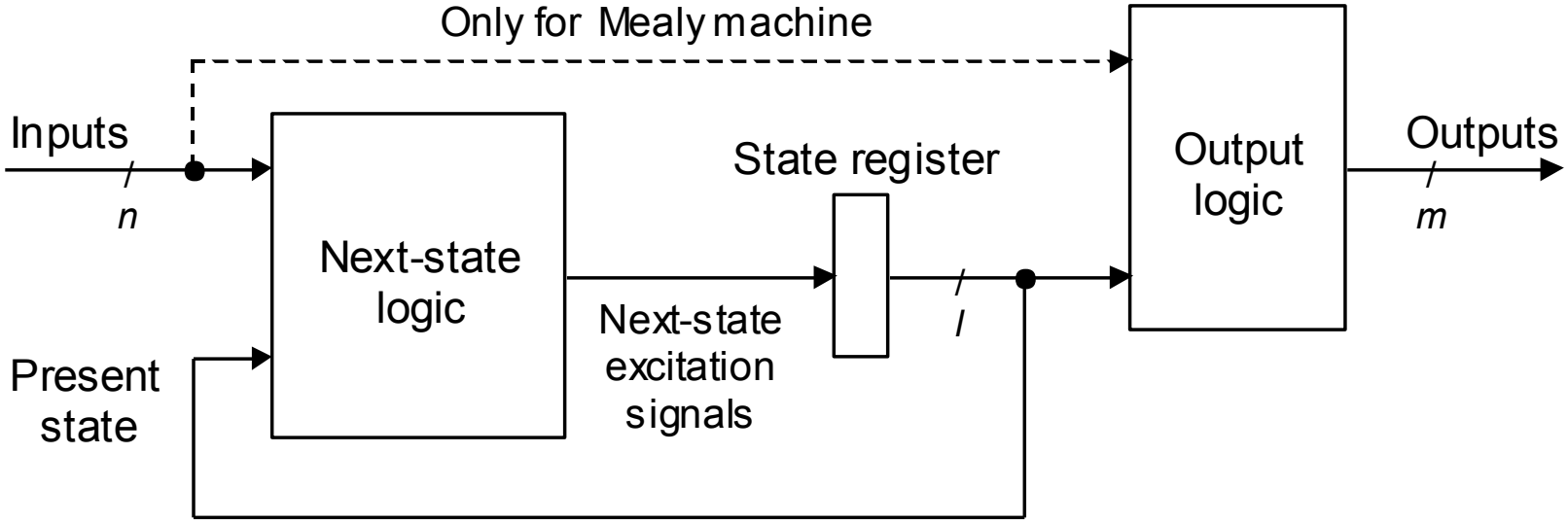


Tabela de stări și diagrama pentru un automat de băuturi răcoritoare.

Implementarea logicii secvențiale



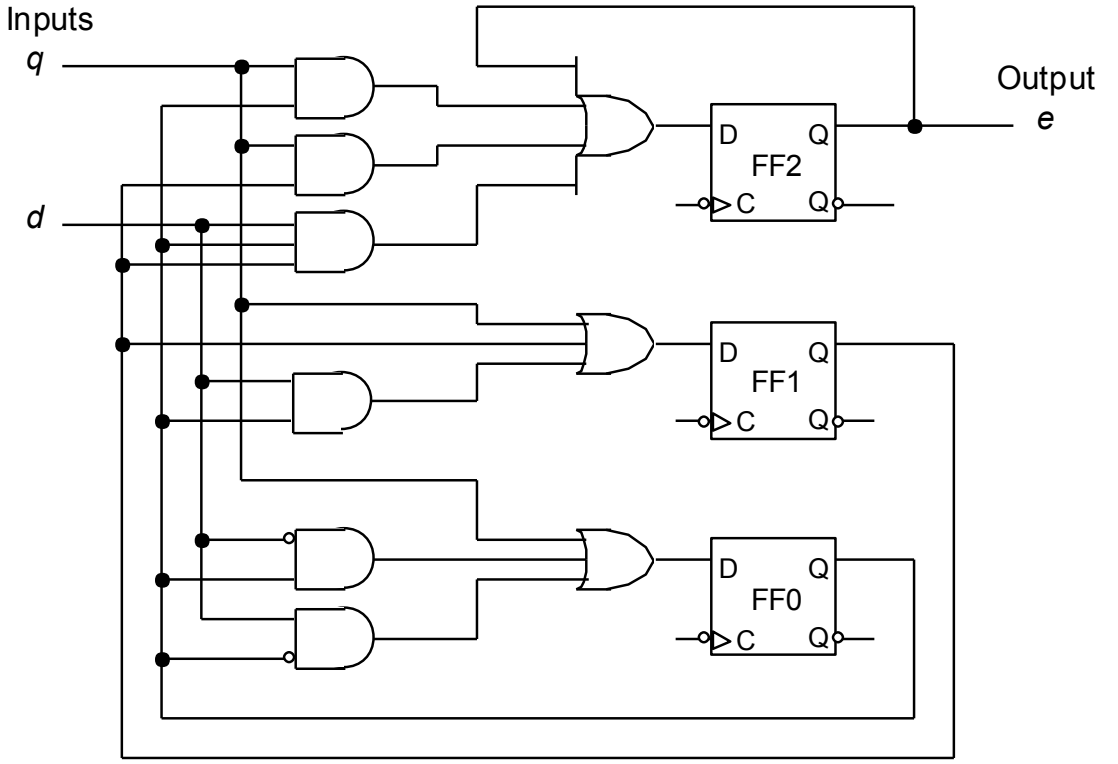
Realizarea hardware a automatelor secvențiale Moore și Mealy.

Proiectarea circuitelor secvențiale

Exemplu

Quarter in

Dime in



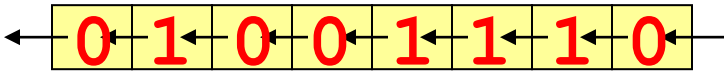
Final state is 1xx

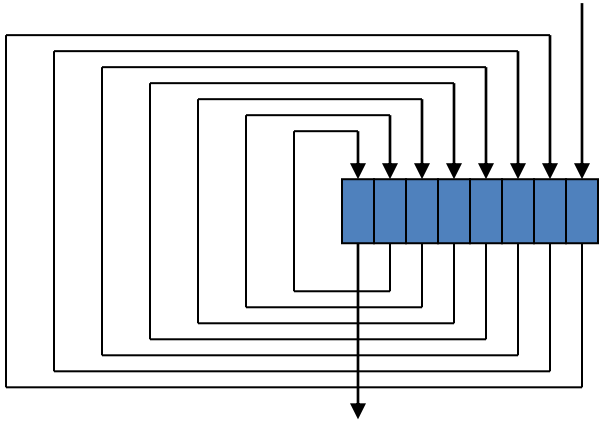
Implementarea hardware a automatului de răcoritoare.

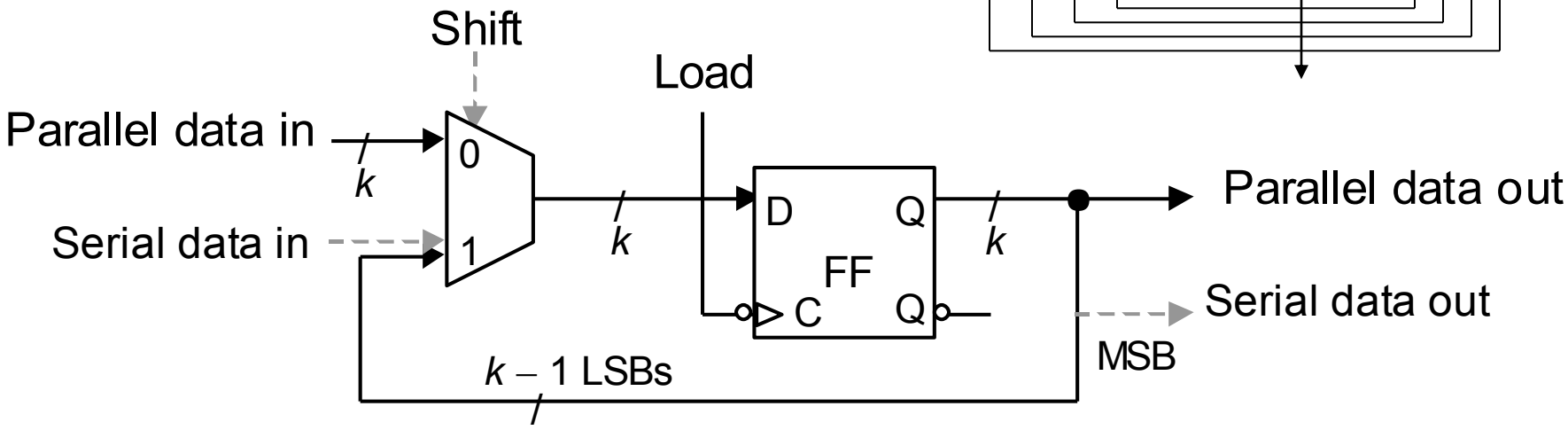
Circuite secvențiale utile

- Blocuri de nivel înalt
- Similar cu niște dulapuri prefabricate folosite în construcția unei case
- Componente de memorie(SRAM, DRAM, Flash)
- În continuare discutăm despre urmatoarele circuite: registru cu deplasare, registru (celulă SRAM), numărător

Registru deplasare

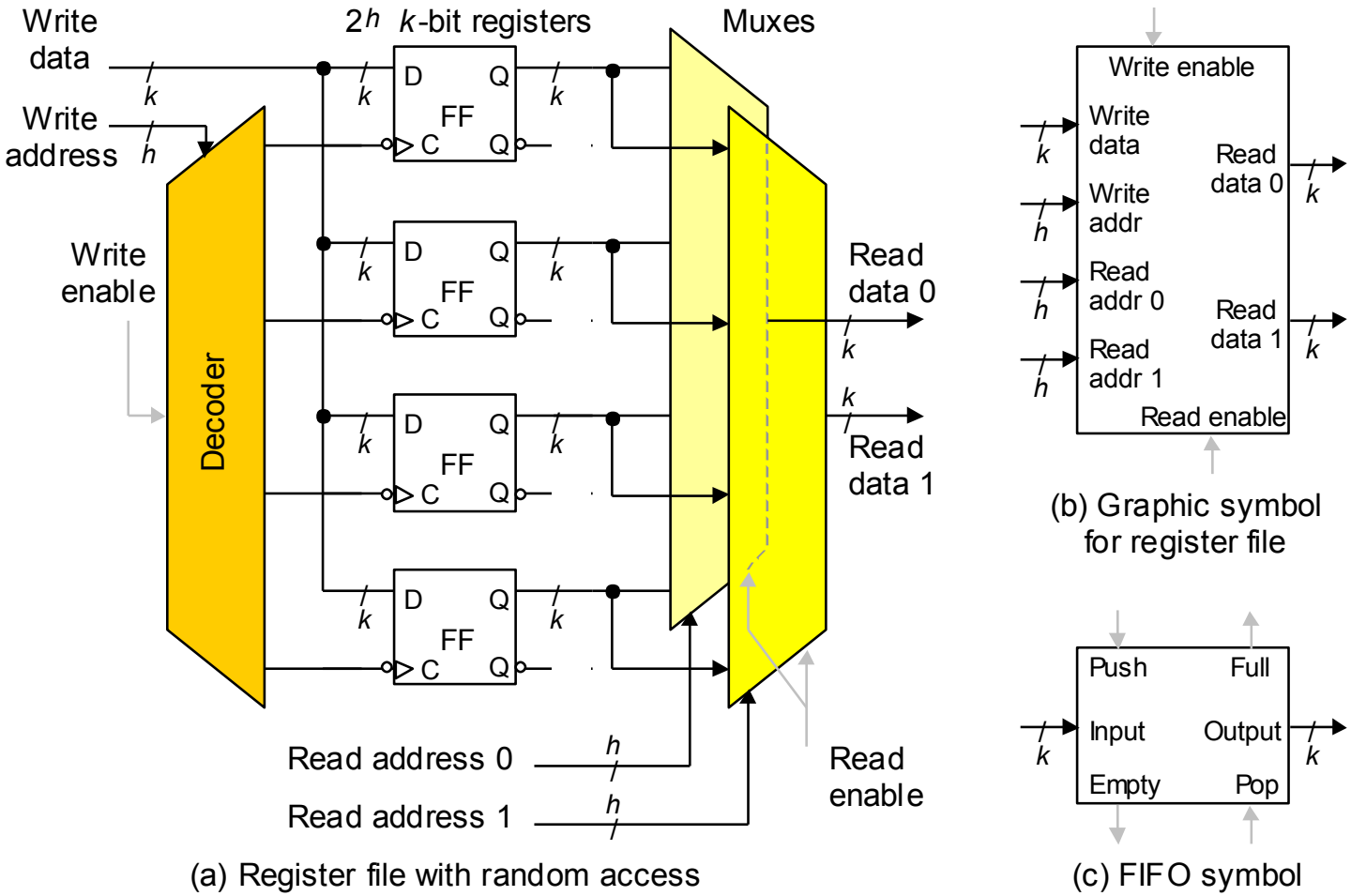






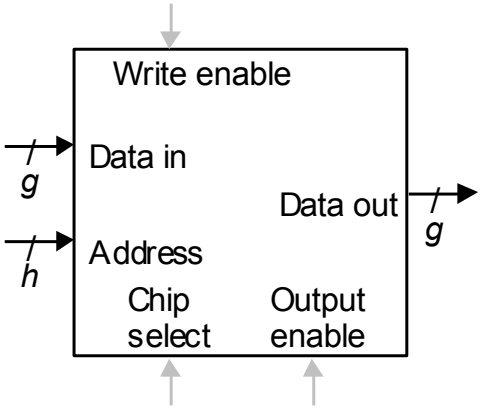
Registru cu shiftarea la stânga a unui singur bit și încărcare paralelă.

Registre și buffere FIFO

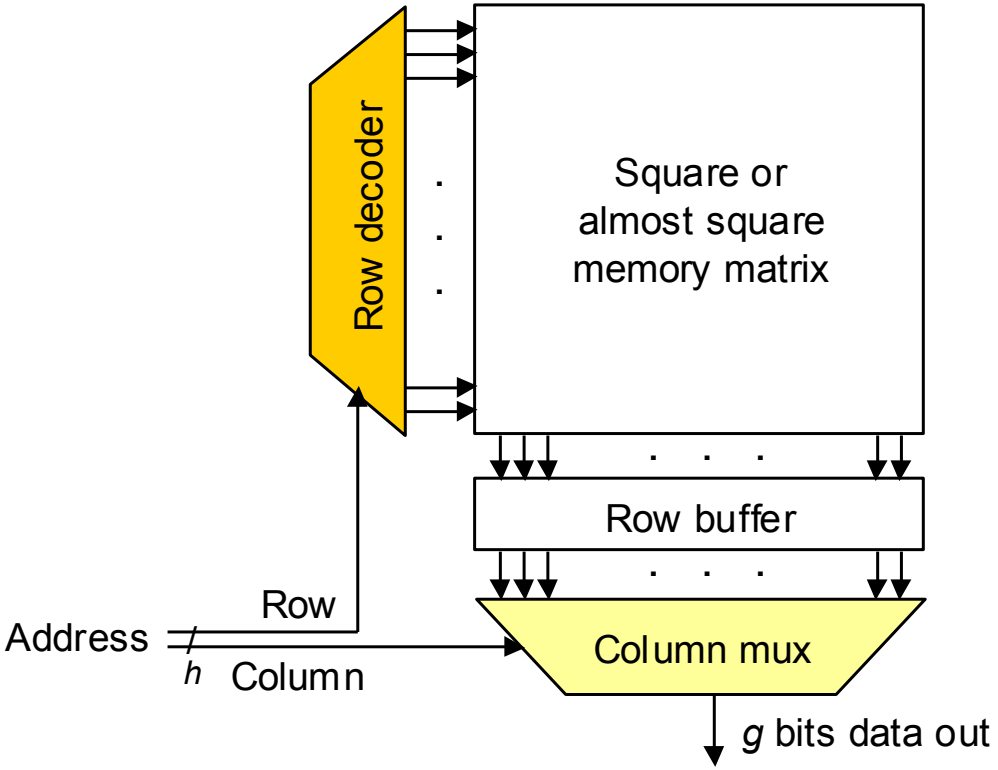


Implementarea unui registru cu acces random și a unui FIFO.

SRAM



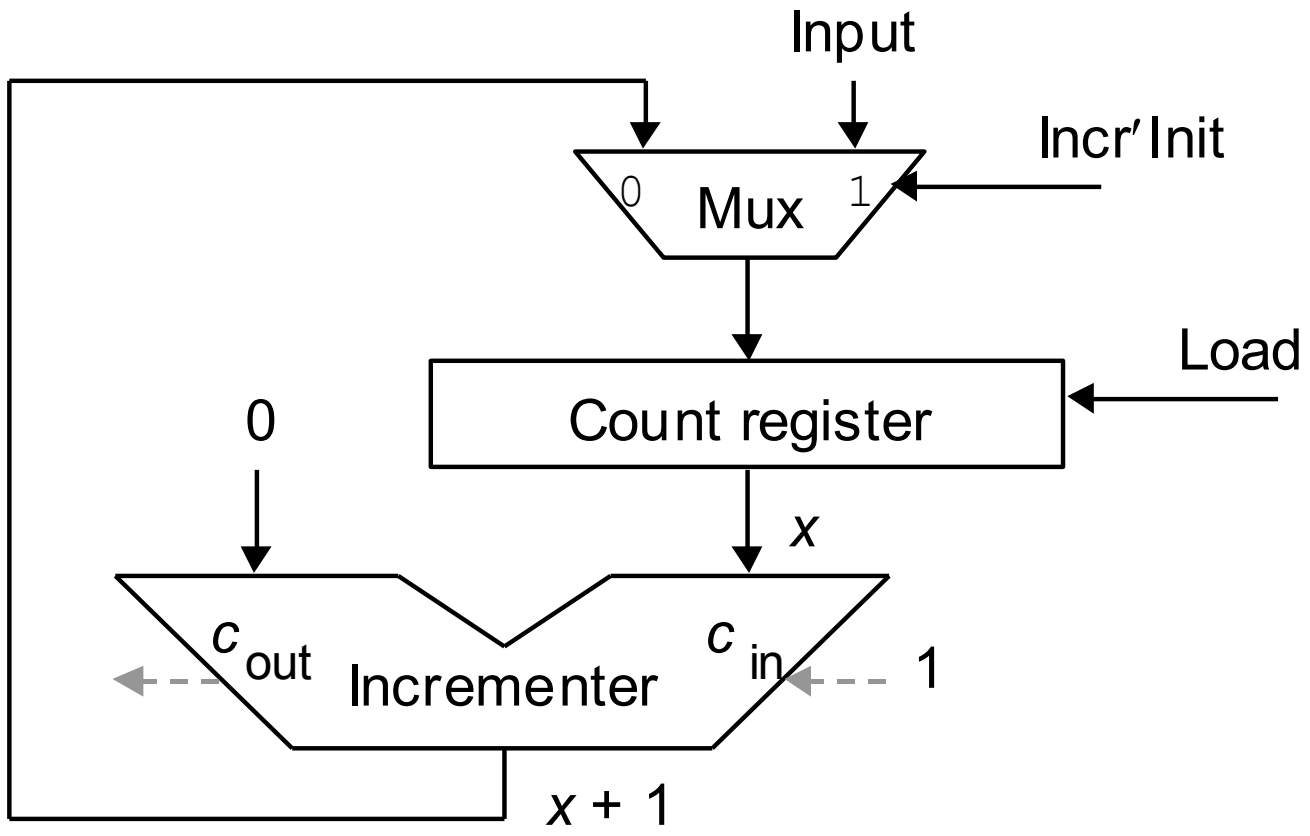
(a) SRAM block diagram



(b) SRAM read mechanism

Memoria SRAM nu este altceva decât o matrice foarte mare de registre single-port.

Numărător Binar



Numărător binar sincron cu capabilități de inițializare.

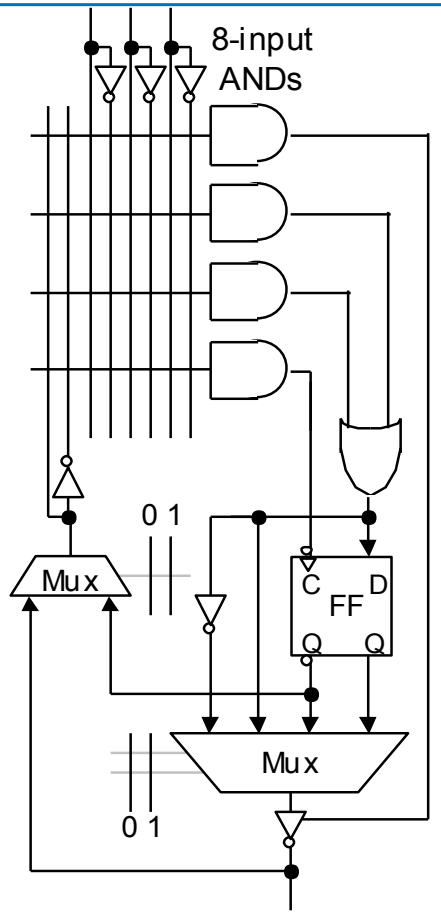
Componente programabile secvențiale

O componentă programabilă secvențială conține porți logice și elemente de memorare

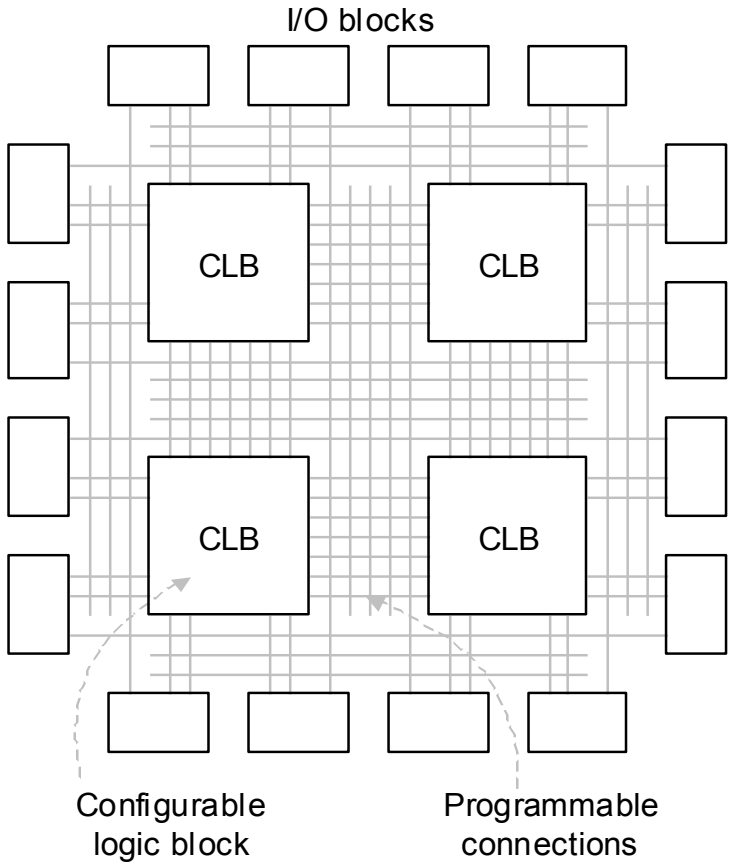
Programate prin tăierea conexiunilor existente (*fuses*) sau prin stabilirea de noi conexiuni (*antifuses*)

- Programmable array logic (PAL)
- Field-programmable gate array (FPGA)
- Ambele tipuri conțin macrocelule și rețele de interconectare

PAL și FPGA



(a) Portion of PAL with storable output

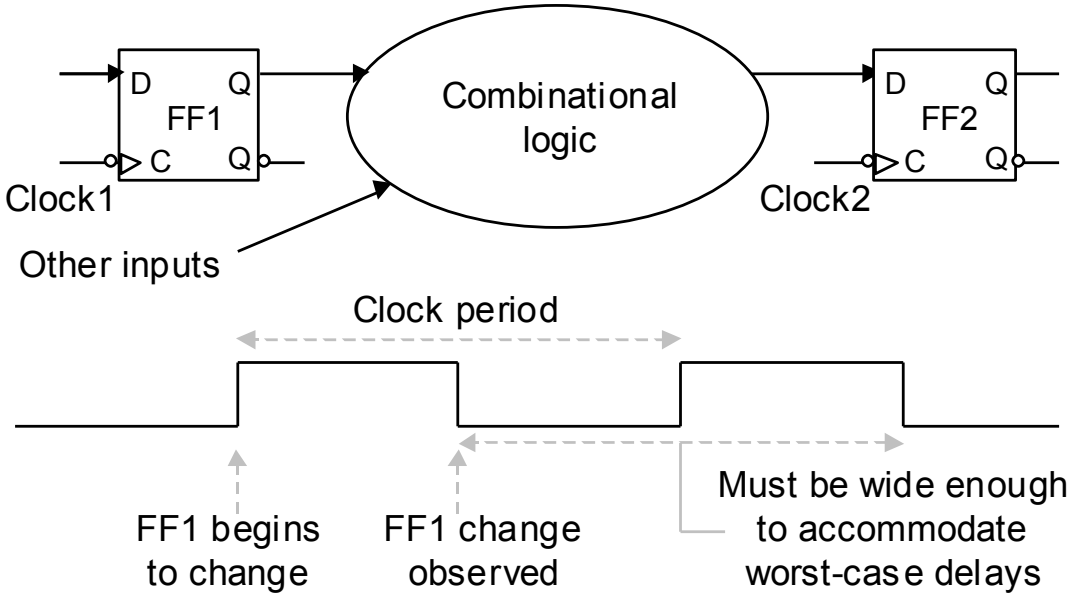


(b) Generic structure of an FPGA

Exemple de circuite logice secvențiale programabile.

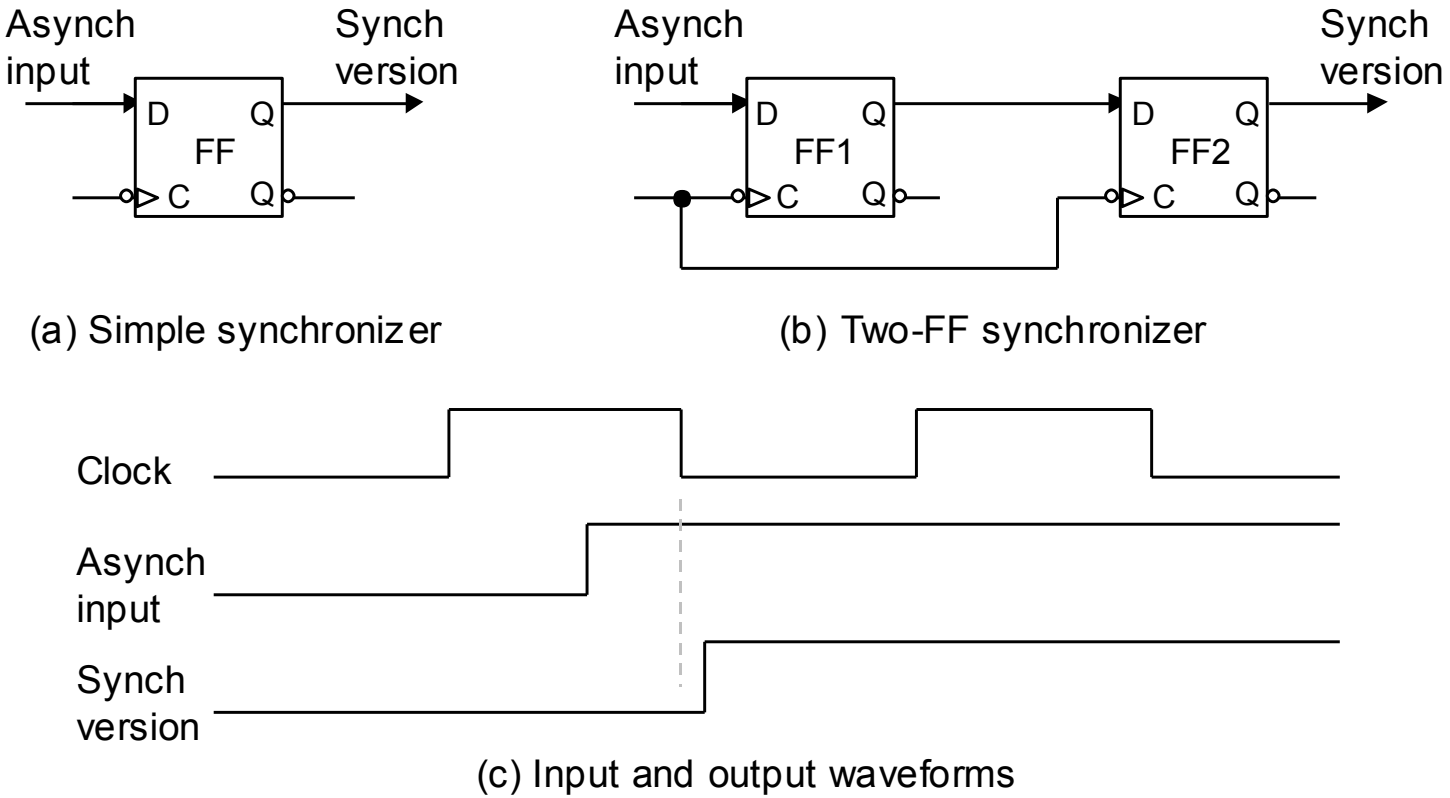
Sincronizarea și temporizarea evenimentelor

Semnalul de ceas este unul periodic: clock rate = clock frequency
Inversul frecvenței de ceas este perioada ceasului: 1 GHz ↔ 1 ns
Constrângere: Clock period ≥ t_{prop} + t_{comb} + t_{setup} + t_{skew}



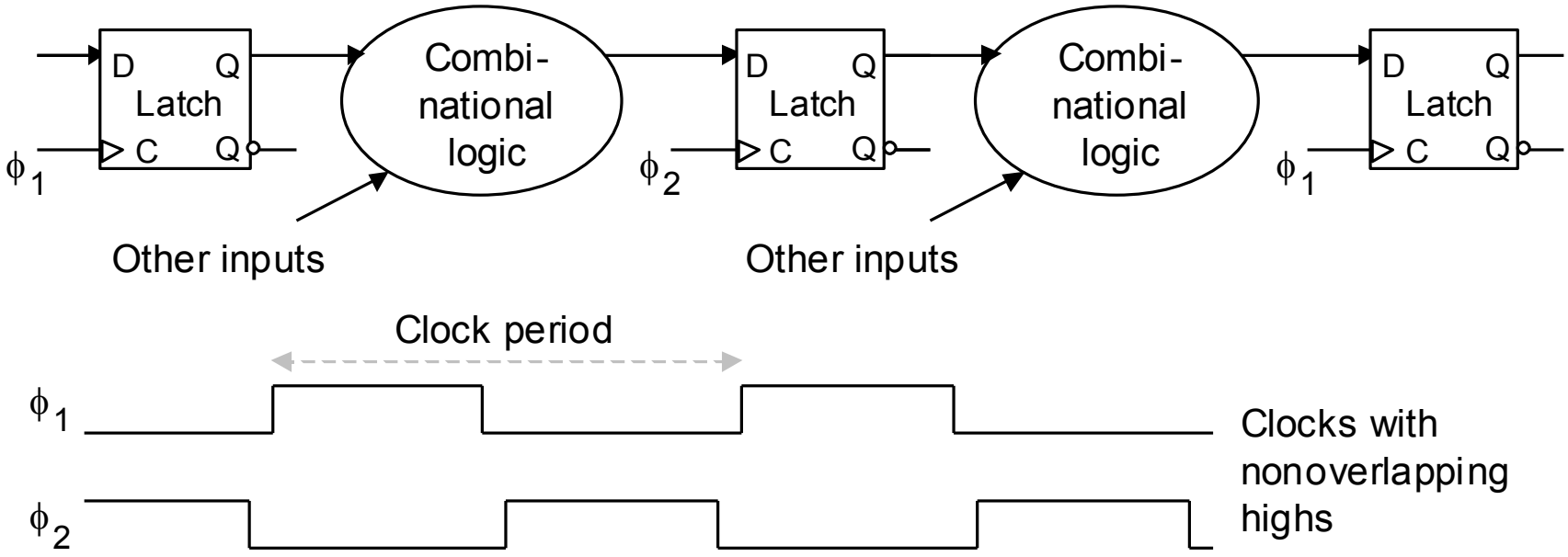
Determinarea lungimii necesare pentru perioada unui semnal de ceas.

Sincronizare



Circuitele de sincronizare sunt folosite pentru a preveni problemele care apar din cauza întârzierilor nedorite ale semnalelor asincrone.

Operare cu mai multe semnale de sincronizare



Circuit cu două semnale de ceas defazate.

Acknowledgements

- Aceste slide-uri conțin materiale aparținând:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)
  - Behrooz Parhami (UCSB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

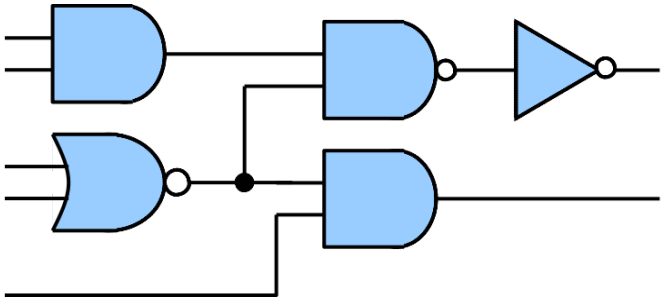
Calculatoare Numerice

- Cursul 3 -

Introducere în Verilog HDL

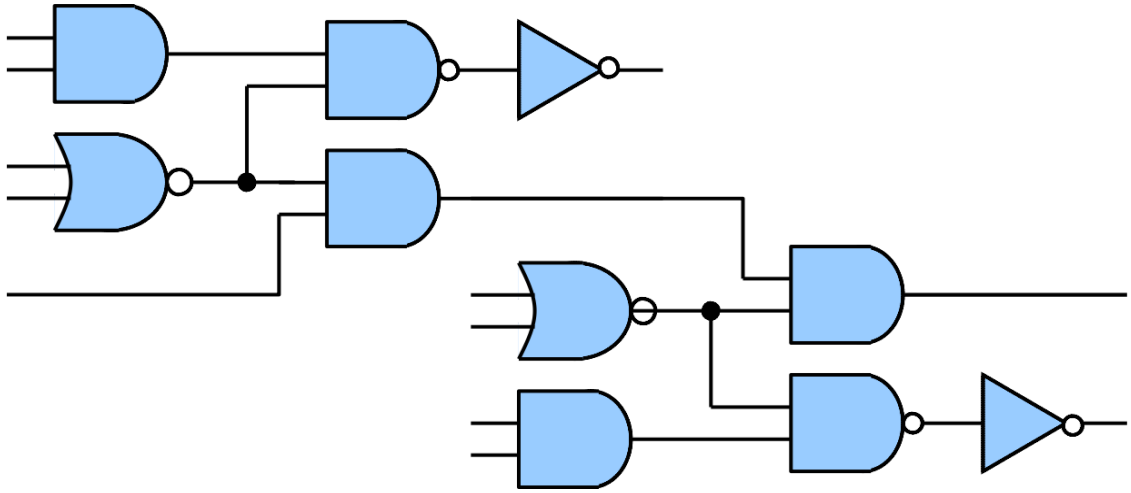
Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Hardware Description Languages



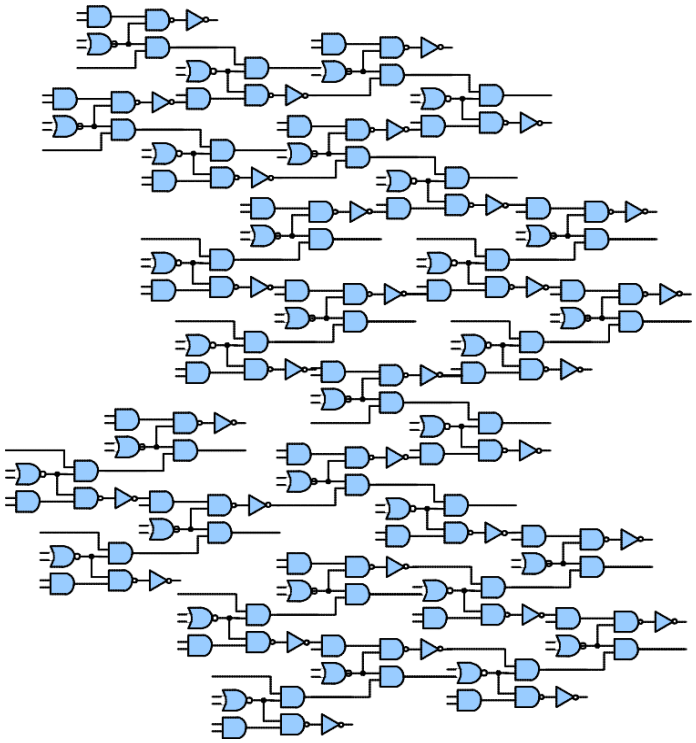
- La început, design-urile logice includeau doar câteva porți logice
- Era foarte ușor de verificat un astfel de design pe hârtie sau pe un breadboard

Hardware Description Languages



Pe măsură ce schemele logice ale dispozitivelor au devenit mai complicate, proiectanții au început să folosească modele logice la nivelul porților logice. Acestea descriau într-un limbaj Hardware Description Language (HDL) comportamentul circuitului și ajutau la verificarea corectitudinii circuitului înainte de fabricație

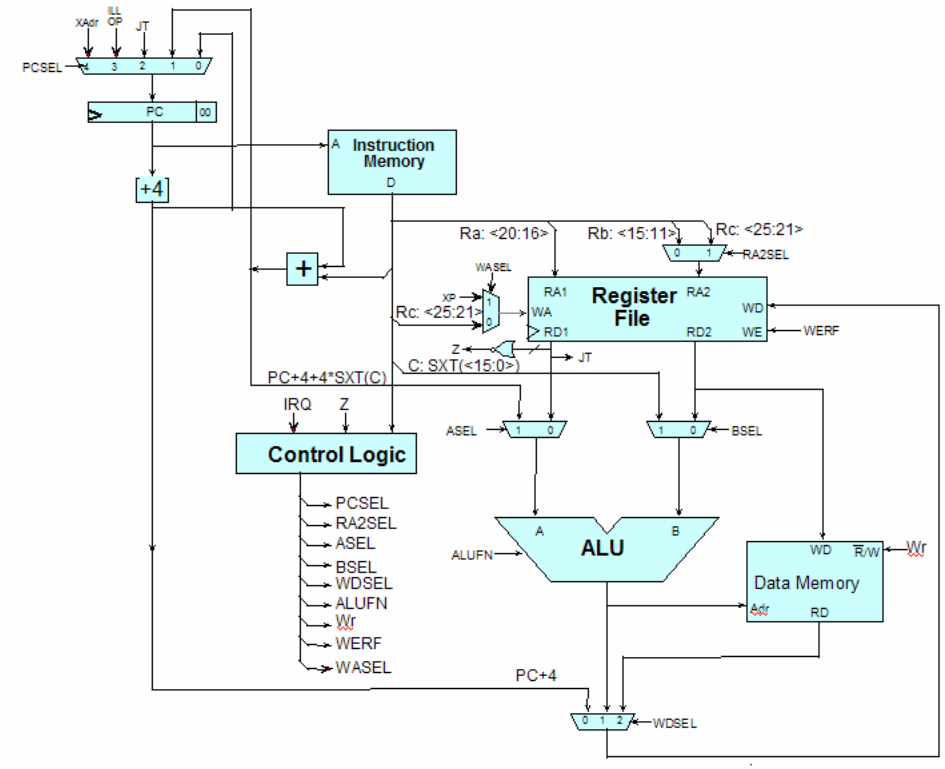
Hardware Description Languages



Atunci când proiectanții au început să lucreze cu design-uri de peste 100,000 porți lgice, modelele la nivel de poartă logică erau prea simple.

S-a trecut la limbaje de descriere hardware de nivel înalt.

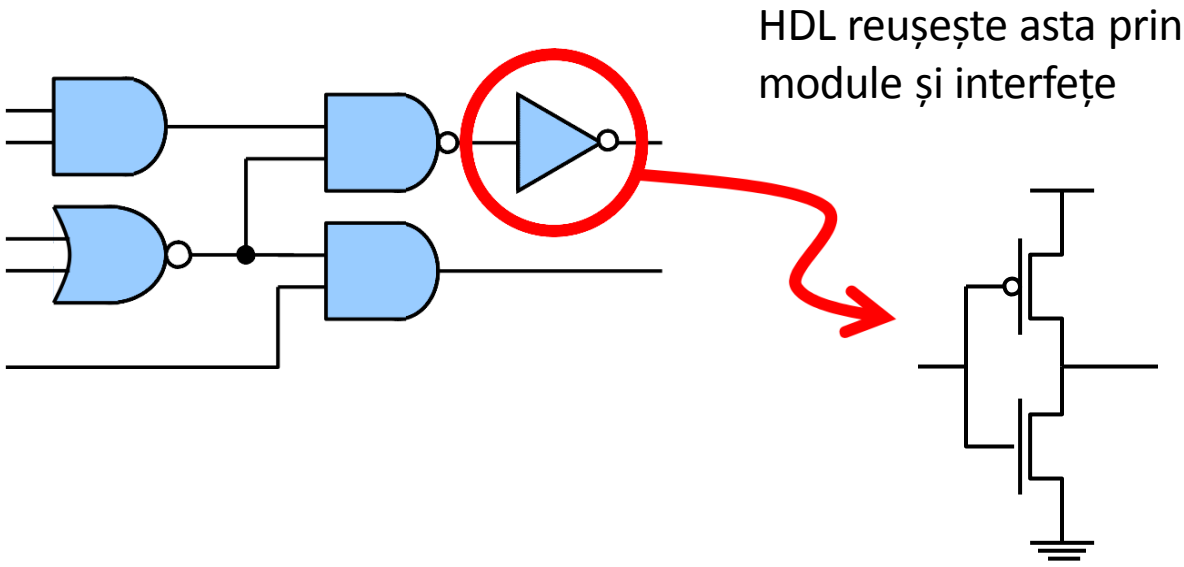
Hardware Description Languages



Design-ul HDL s-a dovedit util și pentru nivelul acesta de abstractizare pentru că un limbaj HDL poate genera o specificație precisă și un fundament pentru construirea de noi și noi design-uri, din ce în ce mai complicate.

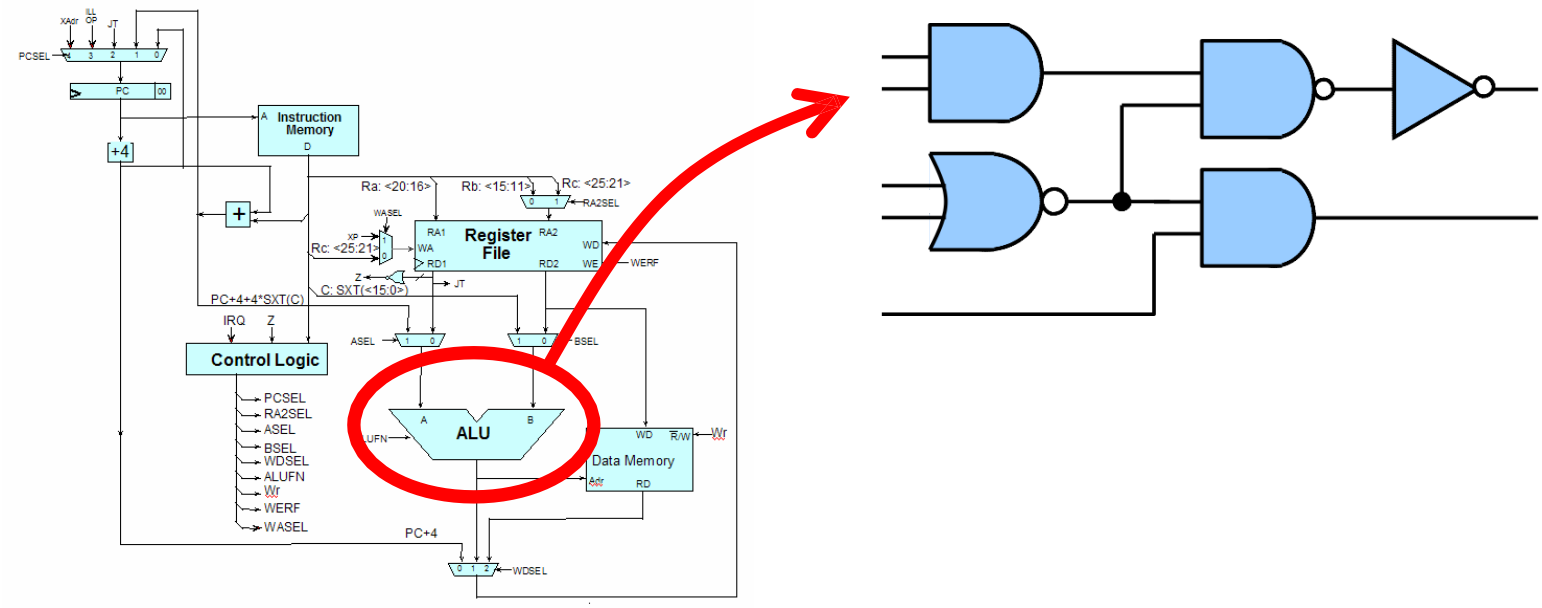
Avantajele HDL

Permite proiectanților să discute despre ce ar trebui să facă hardware-ul fără să proiecteze hardware-ul în sine. Cu alte cuvinte, un HDL separă comportamentul unui circuit de implementarea acestuia, prin diferite nivele de abstractizare.



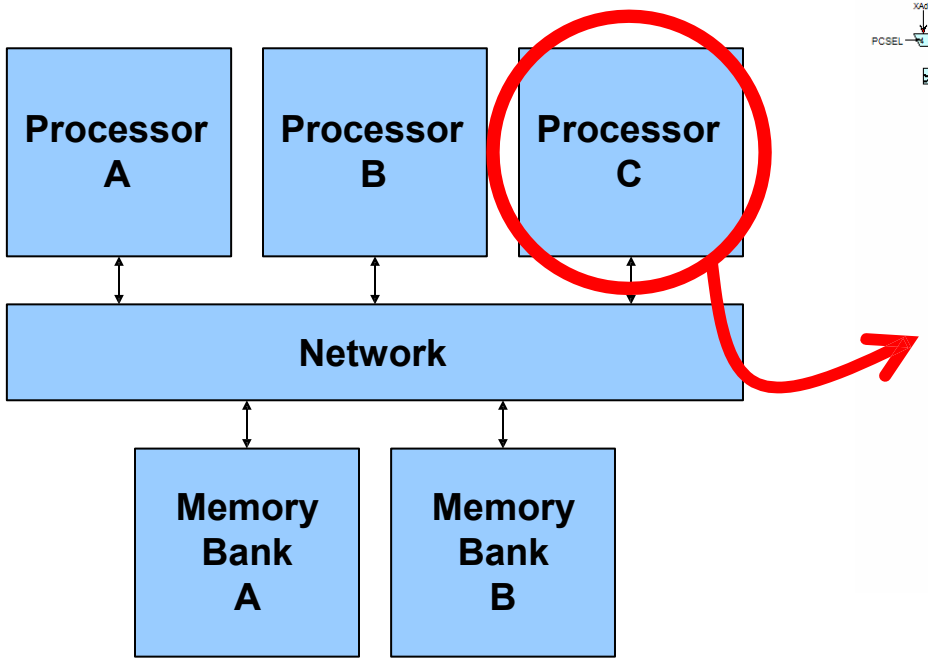
Avantajele HDL

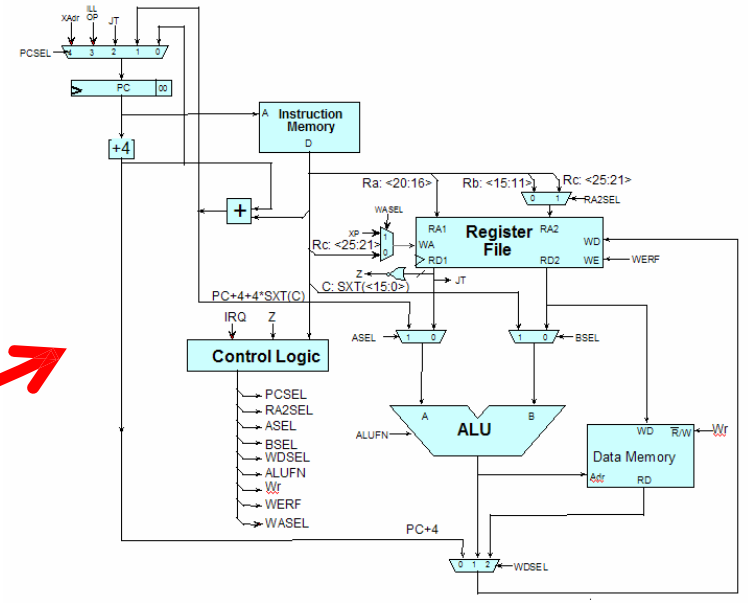
Permite proiectanților să discute despre ce ar trebui să facă hardware-ul fără să proiecteze hardware-ul în sine. Cu alte cuvinte, un HDL separă comportamentul unui circuit de implementarea acestuia, prin diferite nivele de abstractizare.



Avantajele HDL

Permite proiectanților să discute despre ce ar trebui să facă hardware-ul fără să proiecteze hardware-ul în sine. Cu alte cuvinte, un HDL separă comportamentul unui circuit de implementarea acestuia, prin diferite nivele de abstractizare.

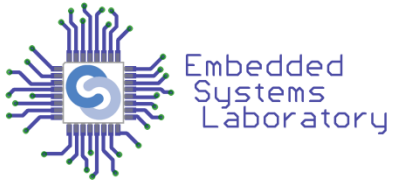




Avantajele HDL

- Proiectanții pot să dezvolte specificații funcționale complete care descriu cu exactitate comportamentul tuturor componentelor și interfețelor acestora
- Proiectanții pot să ia decizii legate de cost,performanță, consum de energie încă din primele etape ale proiectarii unui circuit logic
- Proiectanții pot să creeze diferite programe și unelte care să manipuleze automat design-ul în etapele de verificare, sinteză, optimizare etc.

A tale of two HDLs



VHDL

Sintaxă ADA-like, multă redundanță

Tipuri extensibile și motor pentru simulare

Un design este compus din entități, fiecare entitate poate avea arhitecturi multiple

Gate-level, dataflow, modelare comportamentală. Sintetizabil

Mai greu de învățat și folosit

Verilog

Sintaxă C-like, concisă

Tipuri și reprezentări logice prestabilite

Un design este compus din module care au o singură implementare

Gate-level, dataflow, modelare comportamentală. Sintetizabil

Ușor de învățat și folosit, simulare rapidă

Vom folosi Verilog…

Avantaje

- Alegerea majorității echipelor de design
- Sunteți deja familiarizați cu sintaxa C
- Sintaxă simplă modul/port organizează un proiect în mod intuitiv și ierarhic, chiar și dacă acesta descrie o structură complexă (procesor)
- Potrivit pentru verificarea și sinteza circuitelor logice

Dezavantaje

- Câteva ”chichițe” care sunt mai greu de înțeles de către începători
- Sintaxa C poate să facă pe începători să creadă că Verilog funcționează și la nivel semantic ca C-ul
- E foarte ușor să scrii cod foarte urât. Este mai important decât la C ca programele să fie ușor de înțeles și parcurs.

HDL NU ESTE un limbaj de programare software!!!

Embedded Systems Laboratory

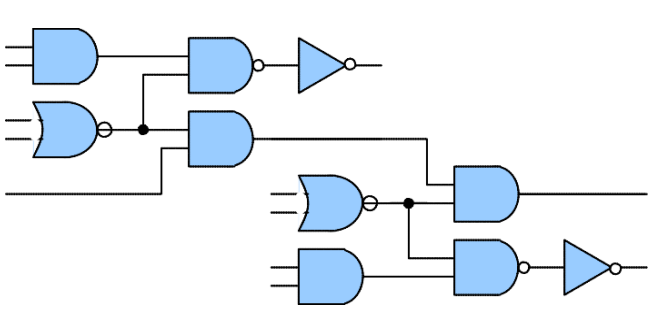
Limbaj programare software

- Limbaj ce poate fi translatat în instrucțiuni în cod mașină, ce poate fi apoi executat de un procesor.

Hardware Description Language

- Limbaj pentru structurii spațiale și a comportamentului temporal a unui sistem hardware

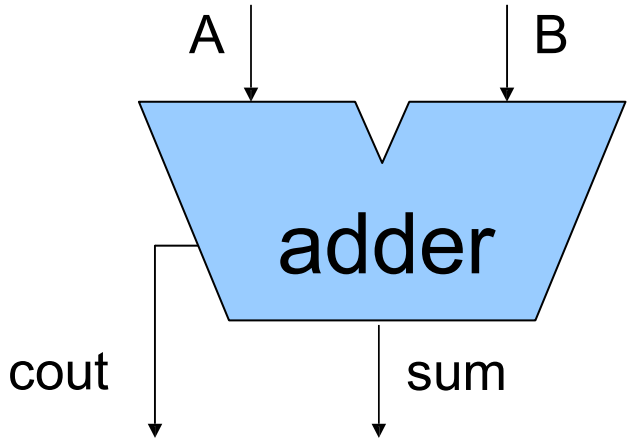
```
module foo(clk,xi,yi,done); input
   [15:0] xi,yi; output done;

   always @(posedge clk) begin:
         if (!done) begin
            if (x == y) cd <= x; else (x > y) x <=
            x - y;
         end
      end
endmodule
```



Modelare ierarhică cu Verilog

Un modul Verilog are un nume și o interfață sub forma unei liste de porturi

- Trebuie să specificăm direcția și lățimea pe biți a fiecărui port



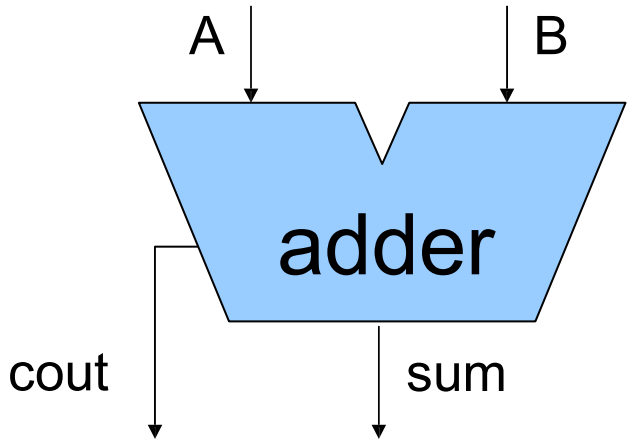

```
module adder( A, B, cout,  sum );
  input  [3:0] A, B;
  output       cout;
  output [3:0] sum;

  // HDL modeling of
  // adder functionality

endmodule
```

Nu uitați punct și virgulă!!!

Modelare ierarhică cu Verilog

Un modul Verilog are un nume și o interfață sub forma unei liste de porturi

- Verilog-2001 a introdus o exprimare succintă tip ANSI C pentru porturi

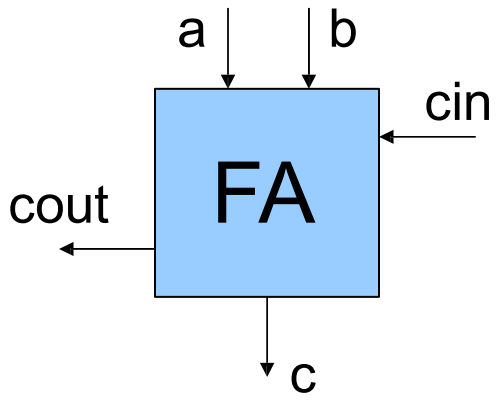



```
module adder( input   [3:0] A, B,
              output        cout,
              output [3:0] sum );

  // HDL modeling of 4 bit
  // adder functionality

endmodule
```

Modelare ierarhică cu Verilog

Un modul poate conține alte module prin instanțierea lor. Asftel, se creează o ierarhie de module

- Modulele sunt conectate prin linii de legatură
- Porturile sunt atașate la aceste linii fie prin poziționare, fie prin numele lor

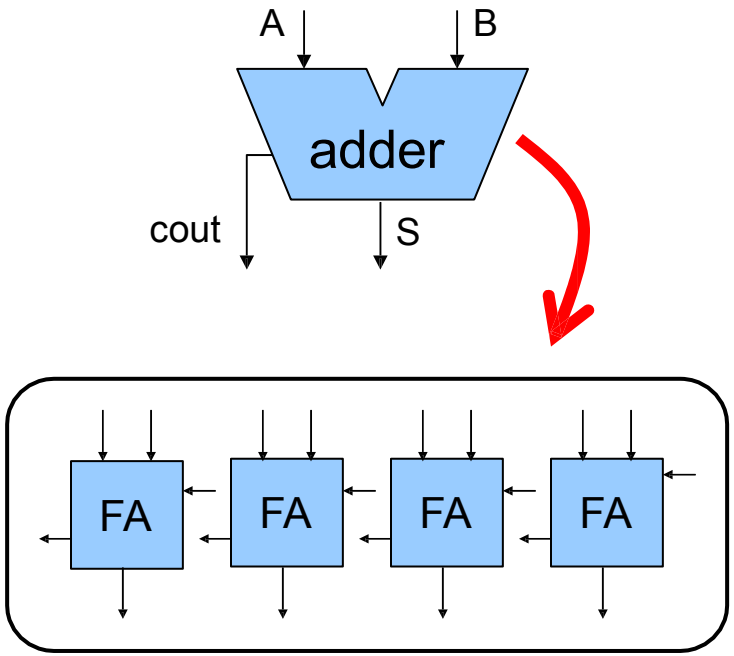



```
module FA( input   a, b, cin
           output  cout, sum );

  // HDL modeling of 1 bit
  // adder functionality

endmodule
```

Modelare ierarhică cu Verilog

Un modul poate conține alte module prin instanțierea lor. Asftel, se creează o ierarhie de module

- Modulele sunt conectate prin linii de legatură
- Porturile sunt atașate la aceste linii fie prin poziționare, fie prin numele lor

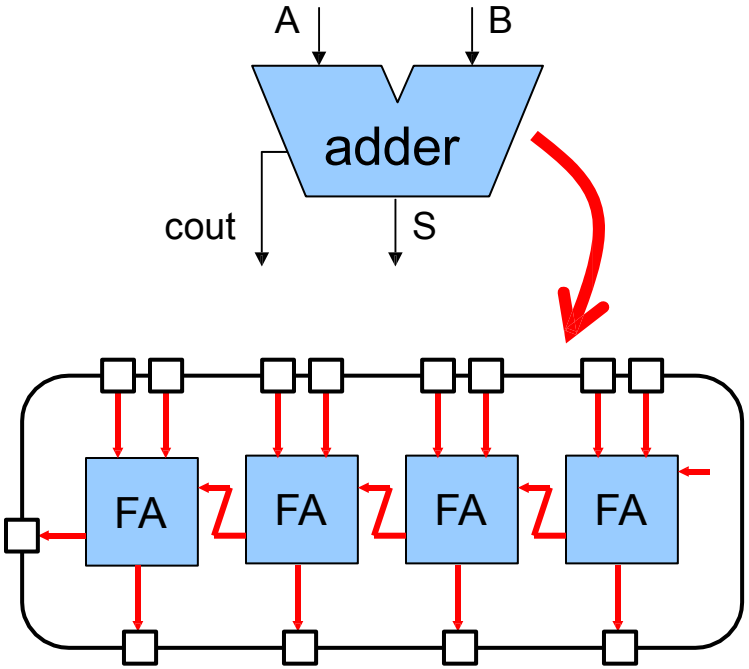



```
module adder(  input [3:0] A,B,
               output cout,
               output [3:0] S );
  FA fa0( ... );
  FA fa1( ... );
  FA fa2( ... );
  FA fa3( ... );

endmodule
```

Modelare ierarhică cu Verilog

Un modul poate conține alte module prin instanțierea lor. Asftel, se creează o ierarhie de module

- Modulele sunt conectate prin linii de legatură
- Porturile sunt atașate la aceste linii fie prin poziționare, fie prin numele lor



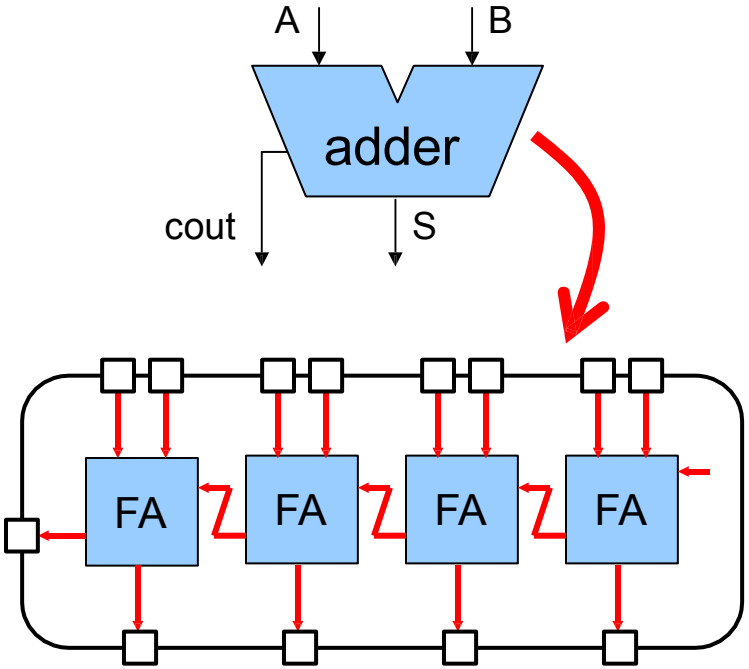

```
module   adder( input   [3:0] A, B,
                output        cout,
                output [3:0] S );

  wire c0, c1, c2;
  FA fa0( A[0], B[0], 0,  c0, S[0] );
  FA fa1( A[1], B[1], c0, c1, S[1] );
  FA fa2( A[2], B[2], c1, c2, S[2] );
  FA fa3( A[3], B[3], c2, cout, S[3] );

endmodule
```

Carry Chain

Modelare ierarhică cu Verilog

Un modul poate conține alte module prin instanțierea lor. Asftel, se creează o ierarhie de module

- Modulele sunt conectate prin linii de legatură
- Porturile sunt atașate la aceste linii fie prin poziționare, fie prin numele lor




```
module adder( input   [3:0] A, B,
              output        cout,
              output  [3:0] S );

  wire c0, c1, c2;
  FA fa0( .a(A[0]), .b(B[0]),
          .cin(0), .cout(c0),
          .sum(S[0] );

  FA fa1( .a(A[1]), .b(B[1]),
          ...

endmodule
```

Verilog Basics

Embedded Systems Laboratory

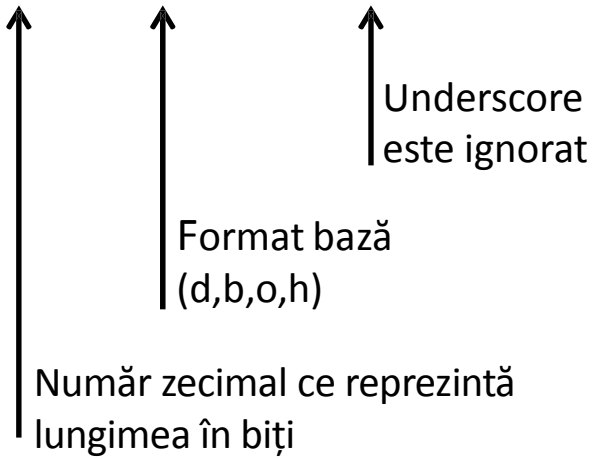
Valori pentru date

0 1

X Z

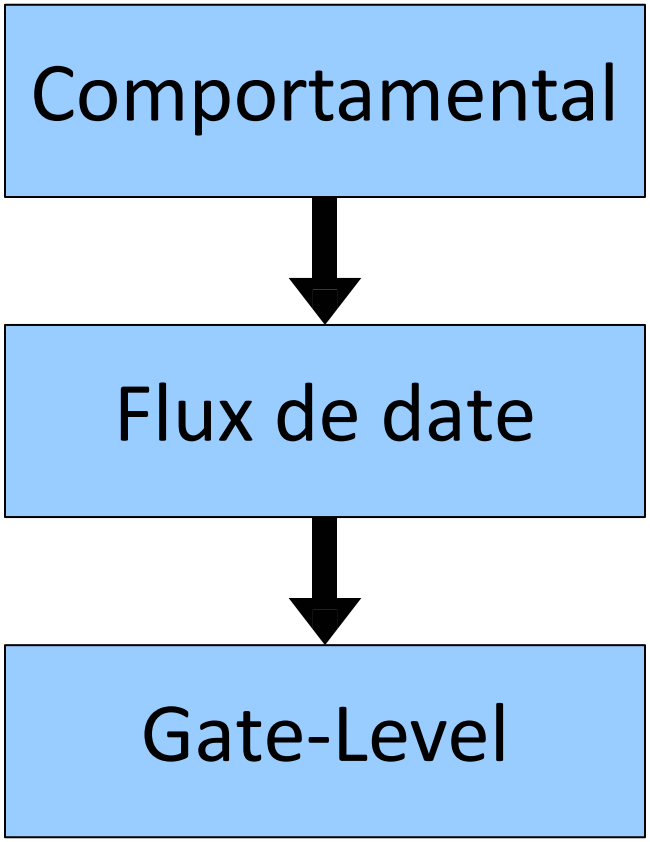
Constante numerice

4’b10_11



32’h8XXX_XXA3

3 Niveluri comune de abstractizare

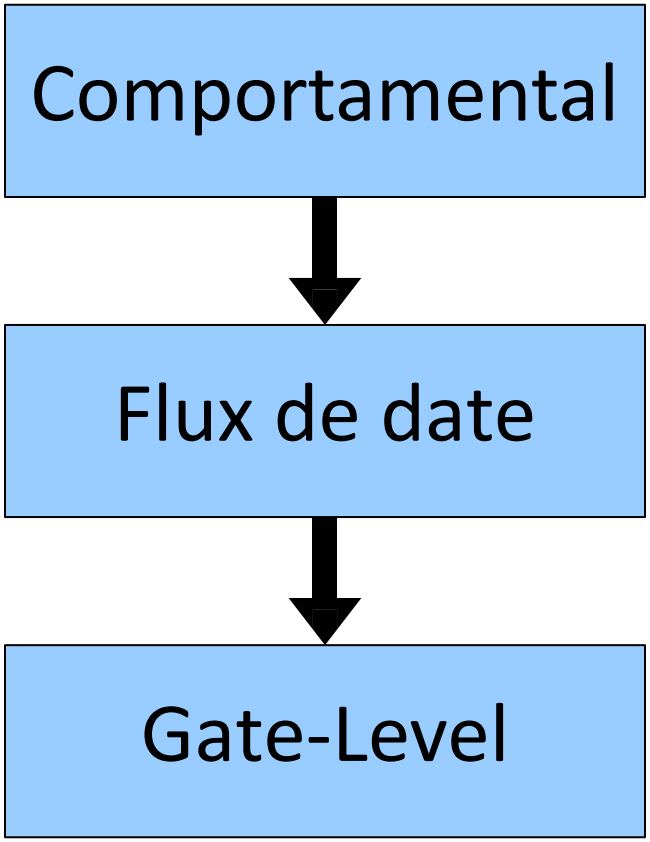


Algoritmul de nivel înalt al modulului este implementat fără să se țină prea mult cont de hardware

Modulul este implementat prin specificarea felului în caredatele curg între registre

Modulul este implemetat folosind porți logice (AND, OR, NOT) și interconectările lor

3 Niveluri comune de abstractizare



Proiectanții pot crea modele de nivel scăzut din modelele de nivel înalt, fie manual sau automat

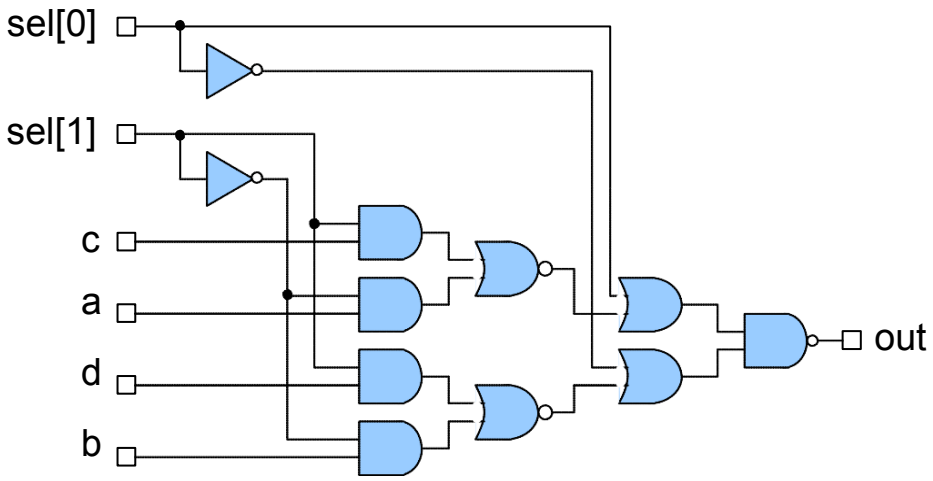
Procesul generării automate a unui model la nivel de porți logice dintr-un nivel comportamental sau de flux de date se numește

Sinteză Logică

Gate-Level : 4-input Multiplexer

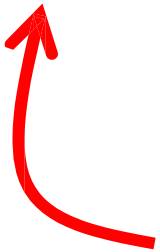

```
module mux4( input   a, b, c, d
             input [1:0] sel,
             output out );

  wire [1:0] sel_b;
  not not0( sel_b[0], sel[0] );
  not not1( sel_b[1], sel[1] );

  wire n0, n1, n2,  n3;
  and  and0( n0, c, sel[1]    );
  and  and1( n1, a, sel_b[1] );
  and  and2( n2, d, sel[1]    );
  and and3(  n3, b, sel_b[1] );

  wire x0, x1;
  nor nor0( x0,  n0, n1 );
  nor nor1( x1,  n2, n3 );

  wire y0, y1;
  or or0( y0,  x0, sel[0]    );
  or or1( y1,  x1, sel_b[0] );

  nand nand0(  out, y0, y1 );

endmodule
```

Porțile logice de bază sunt declarate ca primitive, deci nu este nevoie să definim module pentru ele



Dataflow : 4-input Multiplexer

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  wire out, t0, t1;
  assign t0  = ~( (sel[1] & c) | (~sel[1] & a) );
  assign t1  = ~( (sel[1] & d) | (~sel[1] & b) );
  assign out = ~( (t0 | sel[0]) & (t1 | ~sel[0]) );

endmodule
```



Aceasta este o atribuire continuă deoarece partea dreaptă este evaluată tot timpul și rezultatul este atribuit la orice moment de timp părții stângi a expresiei

Dataflow : 4-input Multiplexer

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  wire t0  = ~( (sel[1] & c) | (~sel[1] & a) );

  wire t1  = ~( (sel[1] & d) | (~sel[1] & b) );
  wire out = ~( (t0 | sel[0]) & (t1 | ~sel[0]) );

endmodule
```

O atribuite continuă implicită combină declararea liniei de date cu o atribuite de tip assign, deci este mai succintă

Dataflow : 4-input Mux & Adder


```
// Four input muxltiplexor
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  assign out = ( sel == 0 ) ? a :
               ( sel == 1 ) ? b :
               ( sel == 2 ) ? c :
               ( sel == 3 ) ? d : 1'bx;

endmodule

// Simple four bit adder
module adder( input  [3:0] op1, op2,
              output [3:0] sum );

  assign sum = op1 + op2;

endmodule
```

Exprimarea la nivel de flux de date permite descrieri mai abstracte decât cele la nivel de porți logice în Verilog

Dataflow : Key Points

Modelarea la nivel de flux de date permite proiectantului să se concentreze asupra stării design-ului și asupra felului în care datele circulă în circuit fără a se poticni în detalii de implementare la nivelul porților logice

- **Atribuirile continue sunt folosite pentru a conecta logica combinațională**

- **Este disponibilă o mare varietate de operatori:**

```
Arithmetic:     + - * / % **
Logical:        ! && ||
Relational:     > < >= <=
Equality:       == != === !===
Bitwise:        ~ & | ^ ^~
Reduction:      & ~& | ~| ^ ^~
Shift:          >> << >>> <<<
Concatenation:  { }
Conditional:    ?:
```

Evitați acești operatori pentru că se sintetizează prost!

Dataflow : Key Points

Modelarea la nivel de flux de date permite proiectantului să se concentreze asupra stării design-ului și asupra felului în care datele circulă în circuit fără a se poticni în detalii de implementare la nivelul porților logice

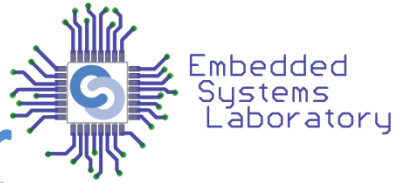
- **Atribuirile continue sunt folosite pentru a conecta logica combinațională**

- **Este disponibilă o mare varietate de operatori:**

```
Arithmetic:      + - * / % **
Logical:         ! && ||
Relational:      > < >= <=
Equality:        == != === !===
Bitwise:         ~ & | ^ ^~
Reduction:       & ~& | ~| ^ ^~
Shift:           >> << >>> <<<
Concatenation:   { }
Conditional:     ?:
```

```
assign signal[3:0]
         = { a, b, 2'b00 }
```

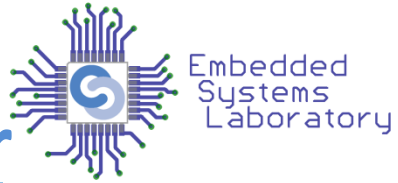
Comportamental: 4-input Multiplexer




```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( a or b or c or d or sel)
  begin
    if ( sel == 0 )
      out = a;
    else if ( sel == 1 )
      out = b
    else if ( sel == 2 )
      out = c
    else if ( sel == 3 )
      out = d
  end

endmodule
```

Un bloc always este un bloc comportamental ce conține expresii ce sunt de obicei evaluate secvențial Codul dintr-un astfel de bloc poate fi foarte abstract (similar cu C) – aici implementăm un MUX cu declarații if/else

Comportamental: 4-input Multiplexer




```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( a or b or c or d or sel )
  begin
    if ( sel == 0 )
      out = a;
    else if ( sel == 1 )
      out = b
    else if ( sel == 2 )
      out = c
    else if ( sel == 3 )
      out = d
  end

endmodule
```

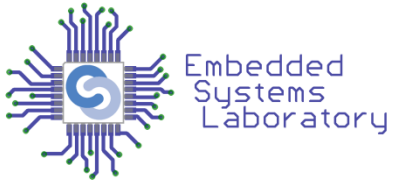
Un bloc always poate include un sensitivity list – dacă oricare din aceste semnale se schimbă, atunci se execută blocul always

Comportamental: 4-input Multiplexer

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( a, b, c, d, sel )
  begin
    if ( sel == 0 )
      out = a;
    else if ( sel == 1 )
      out = b
    else if ( sel == 2 )
      out = c
    else if ( sel == 3 )
      out = d
  end

endmodule
```

În Verilog-2001 putem folosi o virgulă în loc de or

Comportamental: 4-input Multiplexer




```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( a, b, c, d, sel )
  begin
    if ( sel == 0 )
      out = a;
    else if ( sel == 1 )
      out = b
    else if ( sel == 2 )
      out = c
    else if ( sel == 3 )
      out = d
  end

endmodule
```

Ce se întâmplă dacă uităm din greșeală un semnal din listă?

Blocul always nu se va executa dacă doar *d* se schimbă - deci dacă *sel==3* și *d* se schimbă, *out* nu va fi actualizat
Asta va crea discrepanțe între hardware-ul simulat și cel sintetizat - un hardware real nu are sensitivity lists, deci în realitate va funcționa cum trebuie!

Comportamental: 4-input Multiplexer

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( * )
  begin
    if ( sel == 0 )
      out = a;
    else if ( sel == 1 )
      out = b
    else if ( sel == 2 )
      out = c
    else if ( sel == 3 )
      out = d
  end

endmodule
```

În Verilog-2001 putem să folosim o exprimare *@(*)* care creează un sensitivity list pentru toate semnalele din acel bloc always

Comportamental: 4-input Multiplexer

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( * )
  begin
    case ( sel )
      0 : out = a;
      1 : out = b;
      2 : out = c;
      3 : out = d;
    endcase
  end

endmodule
```

Blocurile always pot conține declarații tip case statements, bucle for și while, chiar și funcții – în acest fel permit modelarea comportamentală de nivel înalt

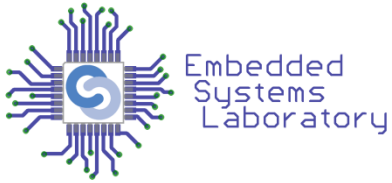
Comportamental: 4-input Multiplexer

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( * )
  begin
    case ( sel )
      0 : out = a;
      1 : out = b;
      2 : out = c;
      3 : out = d;
    endcase
  end

endmodule
```

În Verilog, o declarație reg este doar o variabilă – de fiecare dată când vedeți reg trebue să vă gândiți la o variabilă, NU la un registru hardware!

TOATE atribuirile dintr-un bloc always TREBUIE să aibă ca destinație o variabilă reg!!!

Comportamental: 4-input Multiplexer




```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( * )
  begin
    case ( sel )
      0 : out = a;
      1 : out = b;
      2 : out = c;
      3 : out = d;
    endcase
  end

endmodule
```

Ce se întâmplă în situația asta?
Hardware-ul sintetizat va include un latch pentru out?

Comportamental: 4-input Multiplexer

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( * )
  begin
    case ( sel )
      0 : out = a;
      1 : out = b;
      2 : out = c;
      3 : out = d;
    endcase
  end

endmodule
```

Poate! Ce se întâmplă dacă *sel == xx*? Out va fi neinițializat și hardware-ul va trebui să mențină cumva ultima valoare a lui *out*!

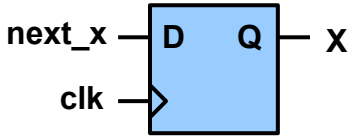
Comportamental: 4-input Multiplexer

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  reg out;

  always @( * )
  begin
    case ( sel )
      default : out = 1'bx;
      0 : out = a;
      1 : out = b;
      2 : out = c;
      3 : out = d;
    endcase
  end

endmodule
```

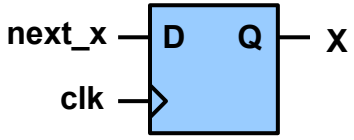
Putem repara asta cu o clauză default în case – atunci nu avem nevoie de nici un latch hardware

Atribuiri non-blocante

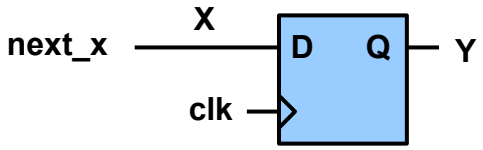
```
always @( posedge clk )
begin
  x = next_x;
end
```



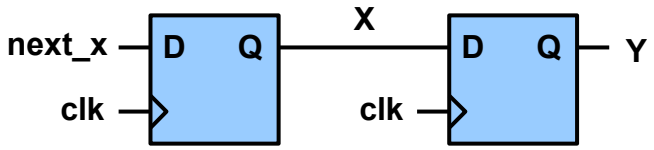

```
always @( posedge clk )
begin
  x <= next_x;
end
```




```
always @( posedge clk )
begin
  x = next_x;
  y = x;
end
```




```
always @( posedge clk )
begin
  x <= next_x;
  y <= x;
end
```



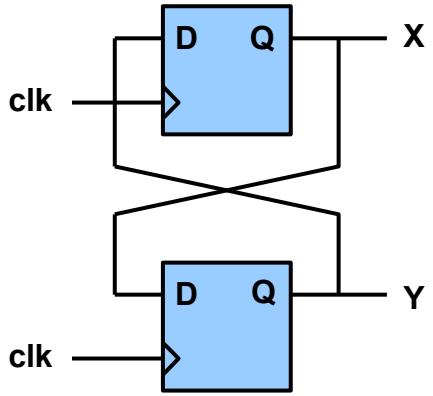
Atribuiri non-blocante

```
always @( posedge clk )
begin
   y = x;
   x = y;
end
```

```
always @( posedge clk )
begin
   y <= x;
   x <= y;
end
```

X Y

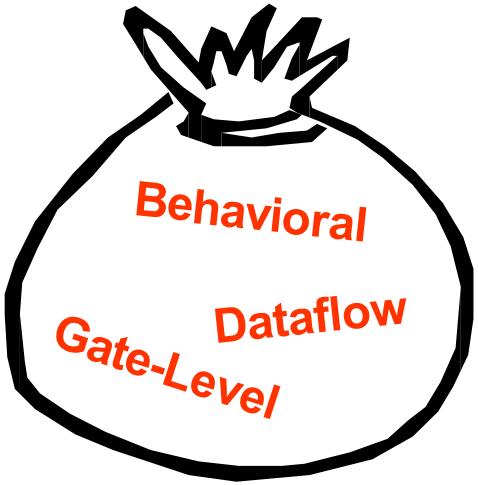
Take Away Point – trebuie să vă întrebați întotdeauna: ”Am nevoie de atribuiri blocante sau non-blocante pentru acest bloc always?”
Nu le puneți la întâmplare!



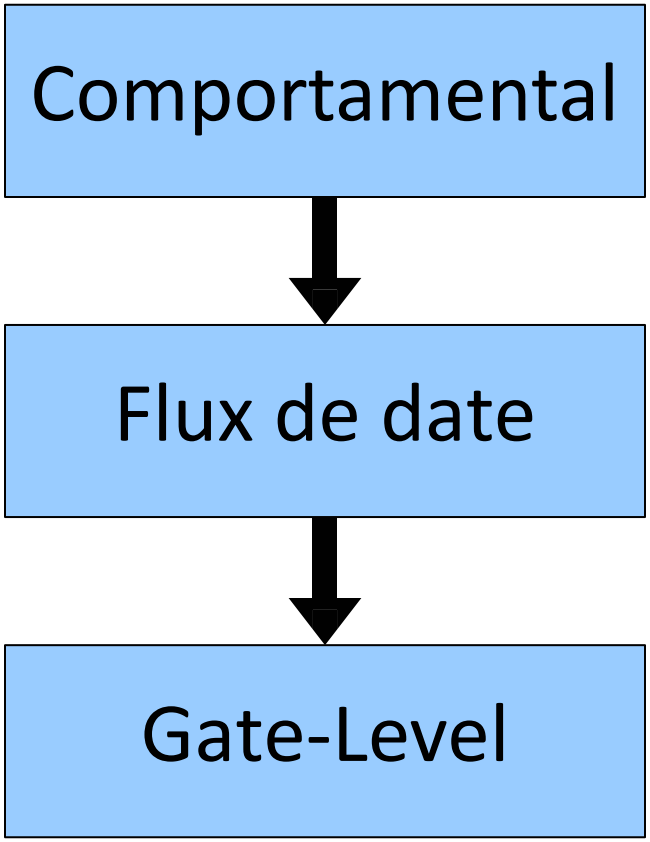
Care abstractizare este cea mai potrivită?

Embedded Systems Laboratory

Proiectanții folosesc de obicei o combinație de toate trei! La începutul etapei de design este posibil să folsoească mai mult modele comportamentale. Pe măsură ce design-ul este rafinat, acestea sunt înlocuite de modele data-flow. În cele din urmă, proeictanții folosesc programe automate pentru a sintetiza un model gate-level.



Sinteza Logică



Compilatoarele moderne sunt capabile să sintetizeze din ce în ce mai mult cod comportamental Verilog în cod gate-level

Problema este că este foarte greu de prezis cum va arăta hardware-ul generat în urma sintezei

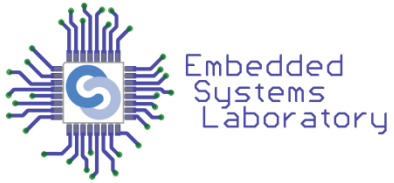
Acest lucru îngreunează foarte mult procedeul de proiectare a unui circuit și găsirea formei optime a acestuia

Modele parametrizate


```
module mux4 #( parameter width )
             (  input [width-1:0] a, b, c, d
               input  [1:0] sel,
               output [width-1:0] out );

  ...

endmodule

// Specify parameters at instantiation time
mux4 #( .width(32) )
      alu_mux( .a(op1), .b(bypass), .c(32'b0),  .d(32'b1),
               .sel(alu_mux_sel), .out(alu_mux_out) );
```

Parametrii permit configurarea statică a modulelor la instanțiere și pot să mărească foarte mult utilitatea moduleleor respective.

Conectarea modelelor parametrizate




```
module adder #( parameter width )
              ( input  [width-1:0] op1,op2,
                output cout,
                output [width-1:0] sum );

  wire [width-1:0] carry;
  assign carry[0] = 0;
  assign cout = carry[width];

  genvar i;
  generate
    for ( i = 0; i < width; i = i+1 )
    begin : ripple
      FA fa( op1[i], op2[i],
             carry[i], carry[i+1] );
    end
  endgenerate

endmodule
```

Blocurile generate pot folosi parametrii pentru a instanția un număr variabil de submodule pentru a crea un număr variabil de legături

Drept concluzie

Limbajele de descriere hardware constituie o parte esențială a proiectării circuitelor digitale

- HDL poate furniza o specificație funcțională executabilă
- HDL permite o flexibilitate mărită în alegrea design-ului potrivit
- HDL încurajează dezvoltarea de programe automate de sinteză
- HDL ajută la micșorarea complexității unui design digital modern

Verilog nu este un limbaj de programare software, sdeci trebue să fiți tot timpul conștienți de faptul că programul vostru se va translata într-un circuit hardware real!

Planificați-vă cu atenție ierarhia modulelor, deoarece asta va influența multe alte părți ale design-ului vostru

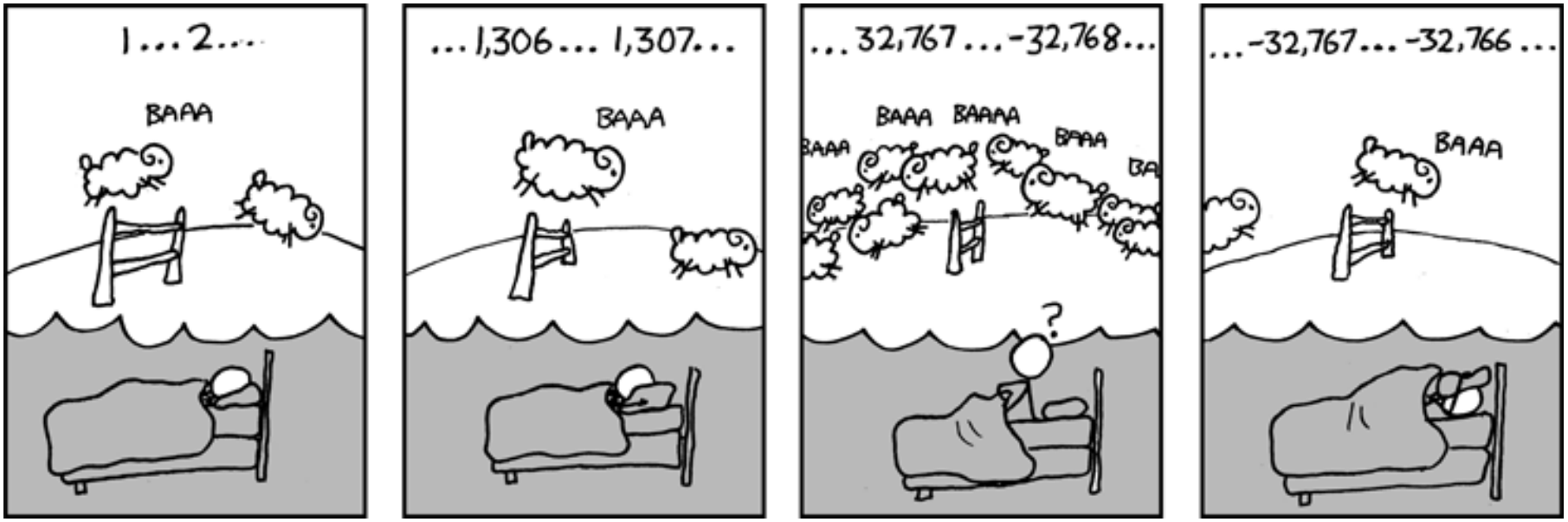
Calculatoare Numerice

-Cursul 4 –

Reprezentarea numerelor

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Comic of the day



If androids someday DO dream of electric sheep, don't forget to declare sheepCount as a long int.

http://xkcd.com/571

Numere

`int`s are not integers,

`float`s are not reals

- Is $x^2 \geq 0$?
  - floats: Da!
  - ints:
    - 40000 * 40000 → 1600000000
    - 50000 * 50000 → ??
- Is (x + y) + z = x + (y + z)?
  - unsigned & signed ints: Da!
  - floats:
    - (1e20 + -1e20) + 3.14 → 3.14
    - 1e20 + (-1e20 + 3.14) → ??

Exemplu Code Security

Embedded Systems Laboratory

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

- Similar to code found in a version of FreeBSD's implementation of **`getpeername`**
- There are legions of smart people trying to find vulnerabilities in programs

Utilizare tipică

Embedded Systems Laboratory

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

```
void getstuff() {
    char mybuf[MSIZE];
    copy_from_kernel(mybuf, MSIZE);
    printf("%s\n", mybuf);
}
```

Utilizare malițioasă

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

```
void getstuff() {
    char mybuf[MSIZE];
    copy_from_kernel(mybuf, -MSIZE);
    . . .
}
```

Operațiile matematice

- Nu generează valori aleatoare
  - Operațiile aritmetice au proprietăți matematice importante
- Nu putem să acoperim toate proprietățile matematice ”uzuale”
  - Reprezentarea numerelor este finită
  - Operațiile cu întregi satisfac proprietățile unui inel
    - Commutativitate associativitate, distributivitate
  - Operațiile în virgulă mobilă satisfac proprietățile de ordonare
    - Monotonie,
- Observație
  - Trebuie să înțelegem care abstractizări funcționează în anumite contexte
  - Problemă importantă pentru cei care scriu compilatoare și pentru dezvoltatorii de aplicații

Codificarea binară

- Byte = 8 bits
  - Binar 00000000_2 to 11111111_2
  - Zecimal: 0_{10} to 255_{10}
    - Prima cifră diferită de 0 în C
  - Hexazecimal 00_{16} to FF_{16}
    - Reprezentare în baza 16
    - Folosim caracterele ‘0’ ..‘9’ și ‘A’ ..‘F’
    - Scriem $FA1D37B_{16}$ în C: **`0xFA1D37B`**
      - Sau **`0xfa1d37b`**

Hex	Dec	Binar
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Machine words

- Calculatoarele au “word size”
  - Dimensiunea nominală a datelor pe întregi
    - Inclusiv adresele
  - Multe calculatoare folosesc cuvinte de 32-biți (4 byte)
    - Limitează adresele la 4GB
    - Devine prea mic pentru aplicațiile memory-intensive
  - Mașinile moderne folosesc cuvinte de 64-biți (8 byte)
    - Spațiu potențial de adrese ~ 1.8 X 10^{19} bytes
    - x86-64 au adresare pe 48-biți: 256 Terabytes
  - Mașinile suportă formate multiple de date
    - Fracții sau multipli de word size
    - Întotdeauna un număr întreg de biți

Embedded Systems Laboratory

Dimensiunea tipurilor de date C (în octeți)

C data type	Typical 32-bit	ia32	Intel x86-64
`char`	1	1	1
`short`	2	2	2
`int`	4	4	4
`long`	4	4	8
`long long`	8	8	8
`float`	4	4	4
`double`	8	8	8
`long double`	8	10/12	10/16
`char *`	4	4	8

Ordonarea octeților

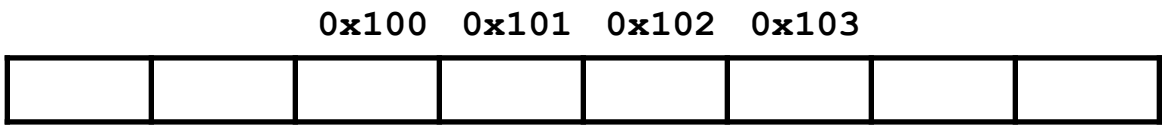
- Cum ar trebui să fie ordonați în memorie octeții unui cuvânt de mai mulți octeți?
  - Big Endian: Sun, PPC, Internet
    - Cel mai puțin semnificativ octet are adresa cea mai mare
  - Little Endian: x86
    - Cel mai puțin semnificativ octet are adresa cea mai mică
- Origine:Călatoriile lui Gulliver (Gulliver's Travels)
- La ce capăt trebuie să spargi un ou fiert?

Ou în configurație "Little Endian" (Wikipedia)

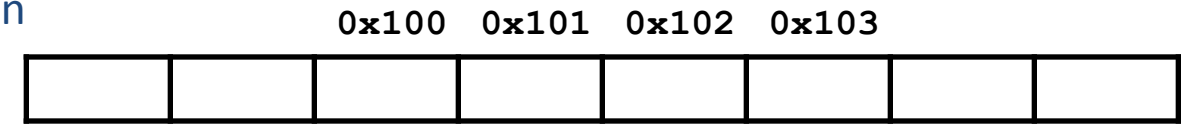
Exemplu

- Big Endian
  - Least significant byte has highest address
- Little Endian
  - Least significant byte has lowest address
- Exemplu
  - Variabila **`x`** de 4 octeți are reprezentarea **`0x01234567`**
  - Adresa dată de **`&x`** este **`0x100`**

Big Endian

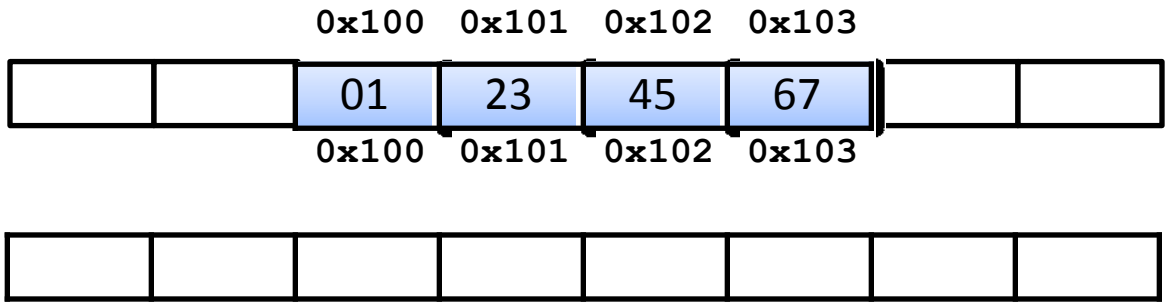


Little Endian



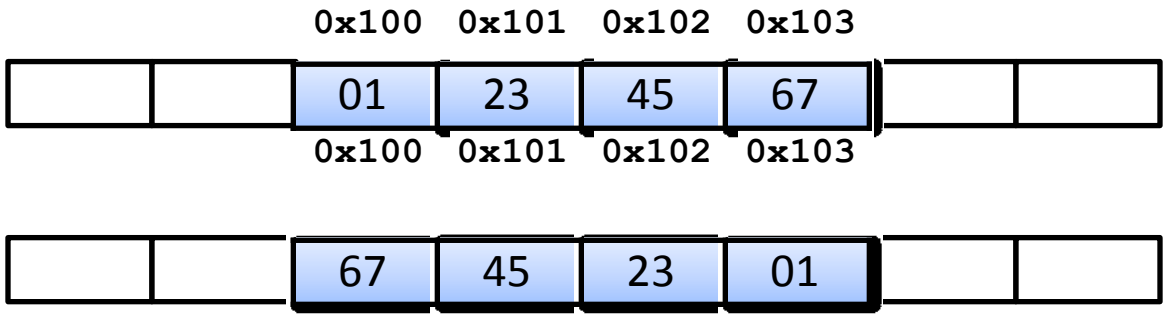
Exemplu

- Big Endian
  - Least significant byte has highest address
- Little Endian
  - Least significant byte has lowest address
- Exemplu
  - Variabila **`x`** de 4 octeți are reprezentarea **`0x01234567`**
  - Adresa dată de **`&x`** este **`0x100`**



Exemplu

- Big Endian
  - Least significant byte has highest address
- Little Endian
  - Least significant byte has lowest address
- Exemplu
  - Variabila **`x`** de 4 octeți are reprezentarea **`0x01234567`**
  - Adresa dată de **`&x`** este **`0x100`**



Citirea listing-ului

- Dezasamblare
  - Reprezentarea text a codului mașină
  - Generat de programul care citește codul mașină
- Exemplu de fragment:

Adresă	Instruction Code	Cod Assembly	
8048365:	5b 00 00	pop	%ebx
8048366:	81 c3 ab 12	add	$0x12ab,%ebx
804836c:	83 bb 28 00 00 00 00	cmpl	$0x0,0x28(%ebx)

- Descifrarea numerelor:

Valoare:	0x12ab
Pad la 4 octeți:	0x000012ab
Despărțire in octeți:	00 00 12 ab
Oglindire (endian):	ab 12 00 00

Examinarea reprezentărilor

Embedded Systems Laboratory

Cod care printează reprezentarea datelor

- Cast pointer la **`unsigned char*`** creează un vector de octeți

```
typedef unsigned char *pointer;

void show_bytes(pointer start, int len)
{
  int i;
  for (i = 0; i < len; i++) {
    printf("0x%p\t0x%.2x\n", start+i, start[i]);
  }
}
```

`printf` directives:

`%p`: print pointer

`%x`: print hexadecimal

Exemplu show_bytes()

```
int a = 15213;
printf("int a = 15213;\n");
show_bytes((pointer) &a, sizeof(int));
```

Rezultat folosind Linux pe Intel x86:

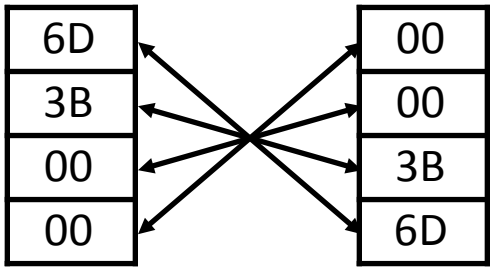
```
int a = 15213;
0x11ffffcb8   0x6d
0x11ffffcb9   0x3b
0x11ffffcba   0x00
0x11ffffcbb   0x00
```

Reprezentarea întregilor

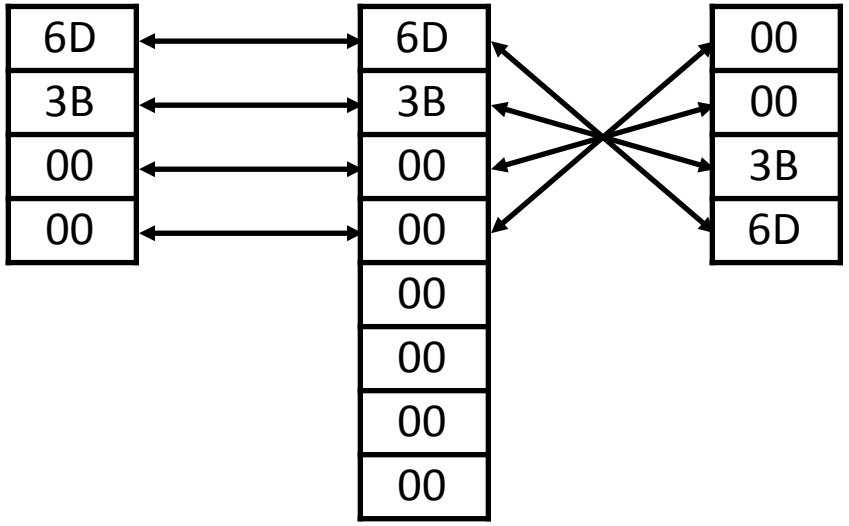
```
int A = 15213;
int B = -15213;
long int C = 15213;
```

Zecimal:	15213
Binar:	0011 1011 0110 1101
Hex:	3 B 6 D

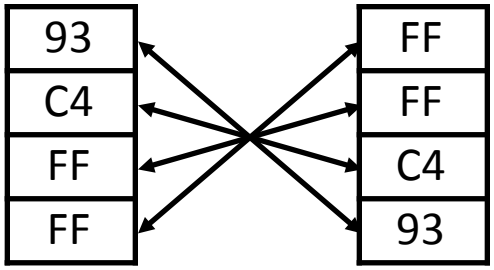
A on ia32, x86-64 A on SPARC



C on ia32 C on x86-64 C on SPARC



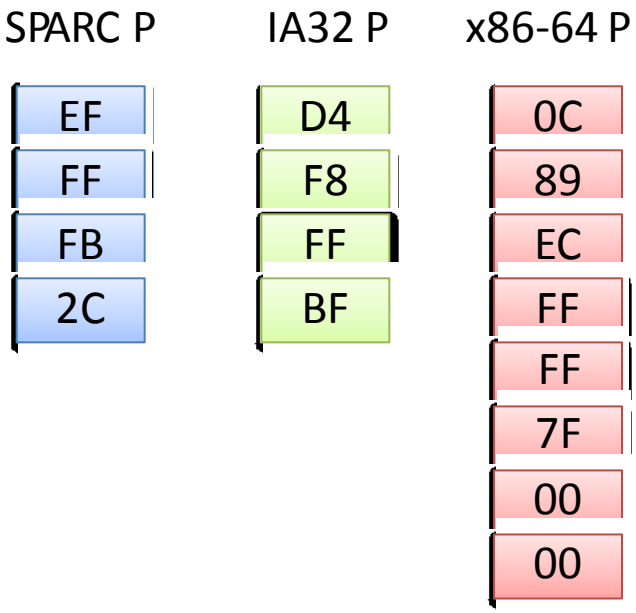
B on ia32, x86-64 B on SPARC



Two's complement representation

Reprezentarea pointer-ilor

```
int B = -15213;
int *P = &B;
```

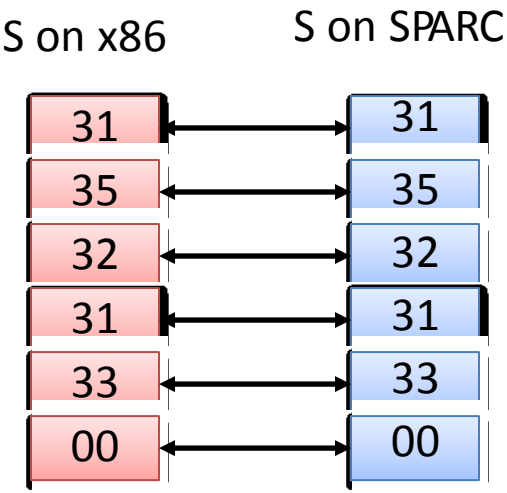


Compilatoare și mașini diferite vor atribui locații diferite obiectelor

Reprezentarea string-urilor

- String în C
  - Reprezentate ca un vector de caractere
  - Fiecare caracter codificat în format ASCII
    - Codificare standard pe 7 biți
    - Caracterul “0” are codul 0x30
      - Cifra i are codul 0x30+i
  - Orice string trebuie să fie null-terminated
    - Caracterul final = 0
- Compatibilitate
  - Ordonarea octeților nu e o problemă

```
char S[6]  = "15213";
```



Sisteme de numerație

- Numerele sunt reprezentate în calculator în binar
  - Numere pozitive
    - Unsigned binary
  - Numere negative
    - Two's complement
    - Sign/magnitude numbers

- Cum reprezentăm **fracțiile**?

Numere fracționare

- Două notații consacrate:

– **Fixed-point (Virgulă fixă):** poziția virgulei este fixă

– **Floating-point (Virgulă mobilă):** virgula se poate afla oriunde și "plutește" către dreapta celui mai semnificativ 1

Numere în virgulă fixă

- 6.75 folosind 4 biți pentru întreg și pentru fracție:
  - Virgula nu este specificată explicit

$$01101100$$

$$0110.1100$$

$$2^2 + 2^1 + 2^{-1} + 2^{-2} = 6.75$$

- Numărul biților pt partea întreagă și fracționară trebuie stabilit dinainte

Exemplu de calcul în v.f.

Embedded Systems Laboratory

Reprezentați 7.5_{10} folosind câte 4 biți pentru partea întreagă și cea fracționară

01111000

Numere cu semn în virgulă fixă

- **Reprezentări:**

– Semn/magnitudine

– Cod complement al lui 2

- **Exemplu:** Reprezentați -7.5_{10} folosind câte 4 biți pentru partea întreagă și cea fracționară

– Semn/magnitudine:

11111000

– Complement al lui 2:

1. +7.5:	01111000
2. Invert bits:	10000111
3. Add 1 to lsb: +	1
	10001000

Numere în virgulă mobilă

- Virgula“plutește” către dreapta celui mai semnificativ 1
- Similar cu notația științifică zecimală
- De exemplu, scrieți 273_{10} în notație științifică:

$$273 = 2.73 \times 10^2$$

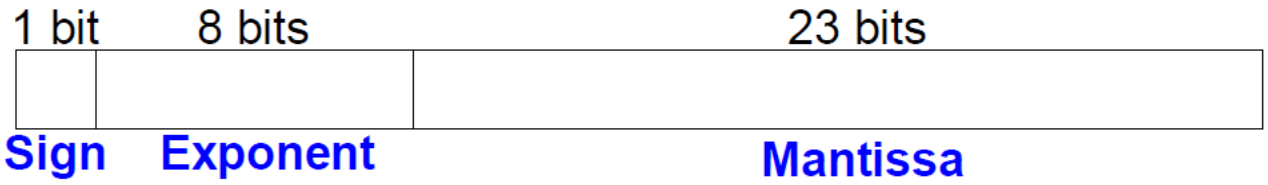
- În general, un număr este scris în notație științifică ca și:

$$\pm M \times B^E$$

– **M** = mantisă
– **B** = bază
– **E** = exponent

– În exemplul de mai sus, M = 2.73, B = 10, and E = 2

Numere în virgulă mobilă



- **Exemplu:** reprezentați valoarea 228_{10} folosind notația în v.m. 32-biți

Arătăm trei versiuni – ultima este numită **IEEE 754 floating-point standard**

Reprezentarea v.m. 1

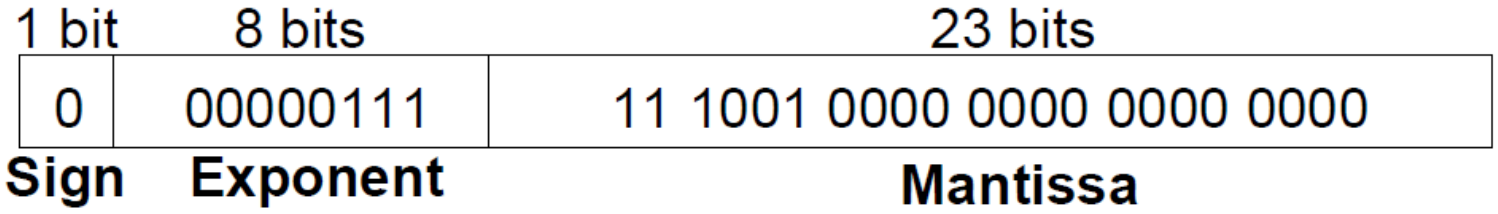
1. Convertim din zecimal în binar:

$$228_{10} = 11100100_2$$

2. Scriem numărul binar în notația științifică:

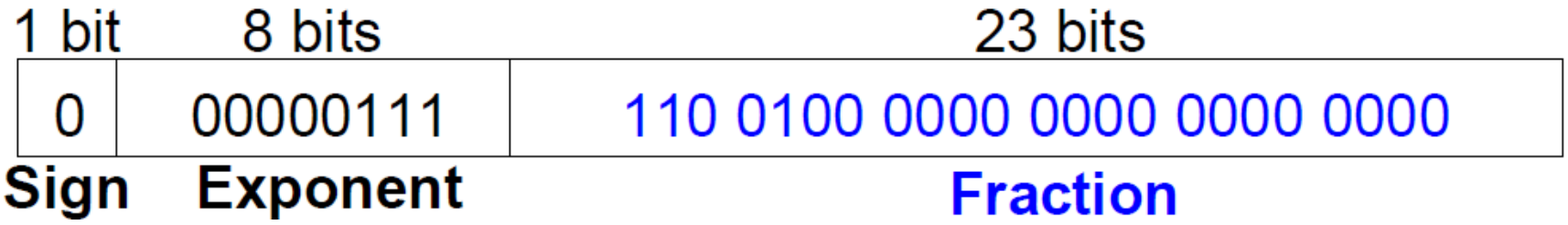
$$11100100_2 = 1.11001_2 \times 2^7$$

3. Completăm fiecare câmp al numărului în virgulă mobilă:
   - Semnul e pozitiv (0)
   - Exponentul are valoarea 7
   - Ceilalți 23 de biți reprezintă mantisa



Reprezentarea v.m. 2

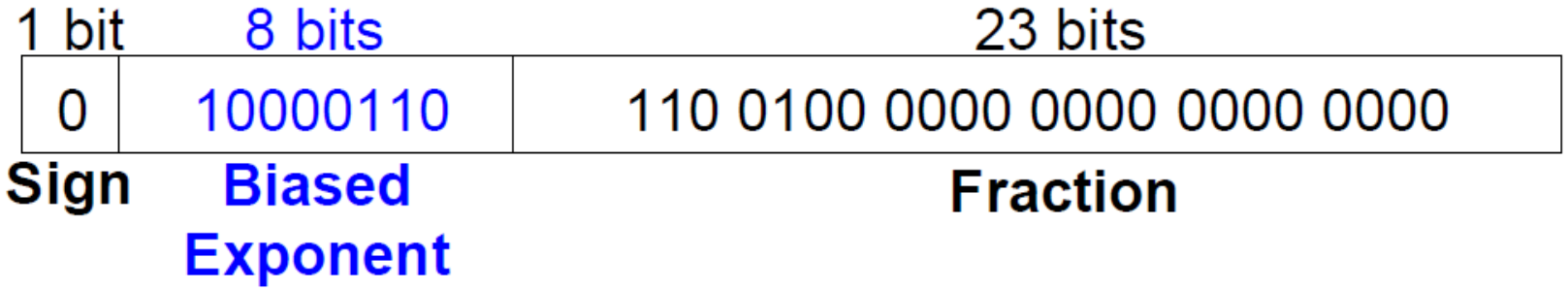
- Primul bit al mantisei este întotdeuna 1:
  - $228_{10} = 11100100_2 = \mathbf{1}.11001 \times 2^7$
- Deci, nu e nevoie să îl stocăm: *1 implicit*
- Stocăm doar biții de fracție în câmpul pt mantisă



Reprezentarea în v.m. 3

- *Exponent deplasat*: deplasament = 127 (01111111_2)
  - Exponentul deplasat = deplasament + exponent
  - Exponentul 7 este stocat ca:

    $127 + 7 = 134 = 0x10000110_2$
- **IEEE 754 32-bit floating-point representation** a 228_{10}



în hexazecimal: **0x43640000**

Exemplu în v.m.

Embedded Systems Laboratory

Scrieți -58.25_{10} în floating point (IEEE 754)

1. Convertim în binar:

   $58.25_{10} = \mathbf{111010.01_2}$

2. Scriem în notația științifică:

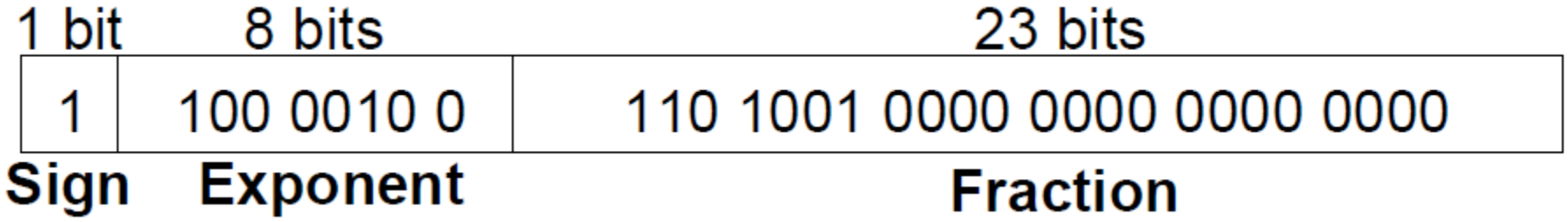
   $\mathbf{1.1101001 \times 2^5}$

3. Completăm câmpurile:

   Bit de semn: 1 (negativ)

   8 biți exponent: (127 + 5) = 132 = $\mathbf{10000100_2}$

   23 biți fracție: **110 1001 0000 0000 0000 0000**



în hexazecimal: **0xC2690000**

Virgula mobilă: Cazuri speciale

Number	Sign	Exponent	Fraction
0	X	00000000	00000000000000000000000
∞	0	11111111	00000000000000000000000
- ∞	1	11111111	00000000000000000000000
NaN	X	11111111	non-zero

Precizia în v.m.

- **Single-Precision:**
  - 32-bit
  - 1 bit semn, 8 biți exponent, 23 biți fracție
  - deplasament = 127
- **Double-Precision:**
  - 64-bit
  - 1 bit semn, 11 biți exponent, 52 biți fracție
  - deplasament = 1023

Floating-Point: Rounding

- **Overflow:** number too large to be represented
- **Underflow:** number too small to be represented
- **Rounding modes:**
  - Down
  - Up
  - Toward zero
  - To nearest
- **Exemplu:** rotunjiți 1.100101 (1.578125) la doar 3 biți parte fracționară
  - Down: 1.100
  - Up: 1.101
  - Toward zero: 1.100
  - To nearest: 1.101 (1.625 is closer to 1.578125 than 1.5 is)

Adunarea în v.m.

1. Extragem biții pentru exponent și mantisă
2. Adăugăm un bit de 1 în fața mantisei
3. Comparăm exponenții
4. Shiftăm mantisa mai mică dacă e necesar
5. Adunăm mantisele
6. Normalizăm mantisele și ajustăm exponentul dacă e necesar
7. Rotunjim rezultatul
8. Asamblăm exponentul și fracția înapoi în formatul pentru virgulă mobilă

Exemplu de adunare în v.m.

Embedded Systems Laboratory

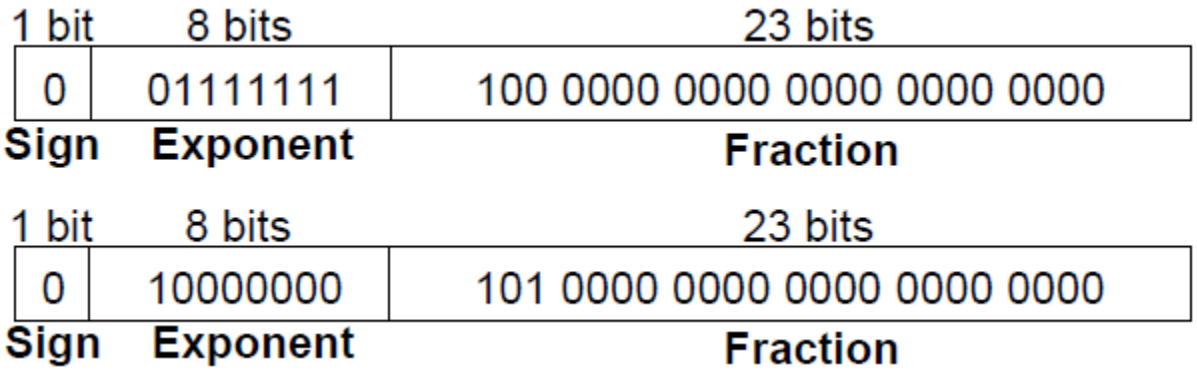
Adunați următoarele numere în v.m.:
0x3FC00000
0x40500000

Exemplu de adunare în v.m.

1. Extragem biții pentru exponent și fracție

Pentru primul număr (N1): S = 0, E = 127, F = .1

Pentru al doilea număr (N2): S = 0, E = 128, F = .101



2. Anexăm 1 pentru a forma mantisa

N1: 1.1

N2: 1.101

Exemplu de adunare în v.m.

3. Comparăm exponenții

127 – 128 = -1, așa că shiftăm N1 dreapta cu 1 bit

4. Shiftăm mantisa mai mică dacă e necesar

shift mantisa lui N1: 1.1 >> 1 = 0.11 $(\times 2^1)$

5. Adunăm mantisele

$$\begin{array}{r} 0.11 \times 2^1 \\ +\ 1.101 \times 2^1 \\ \hline 10.011 \times 2^1 \end{array}$$

Exemplu de adunare în v.m.

6. Normalizăm mantisa și ajustăm exponentul, dacă e necesar

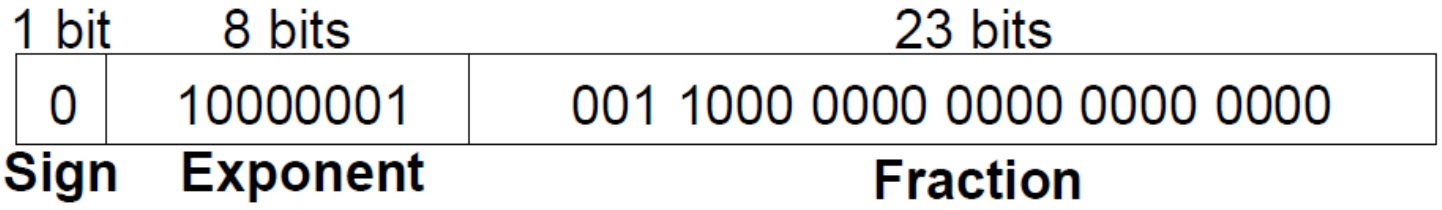
$10.011 \times 2^1 = 1.0011 \times 2^2$

7. Rotunjim rezultatul

Nu e nevoie (intră în 23 biți)

8. Asamblăm exponentul și fracția înapoi în formatul v.m.

$S = 0$, $E = 2 + 127 = 129 = 10000001_2$, $F = 001100..$



în hexazecimal: **0x40980000**

Acknowledgements

- Aceste slide-uri conțin materiale aparținând:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)
  - Behrooz Parhami (UCSB)

- MIT material derived from course 6.823
- UCB material derived from course CS252

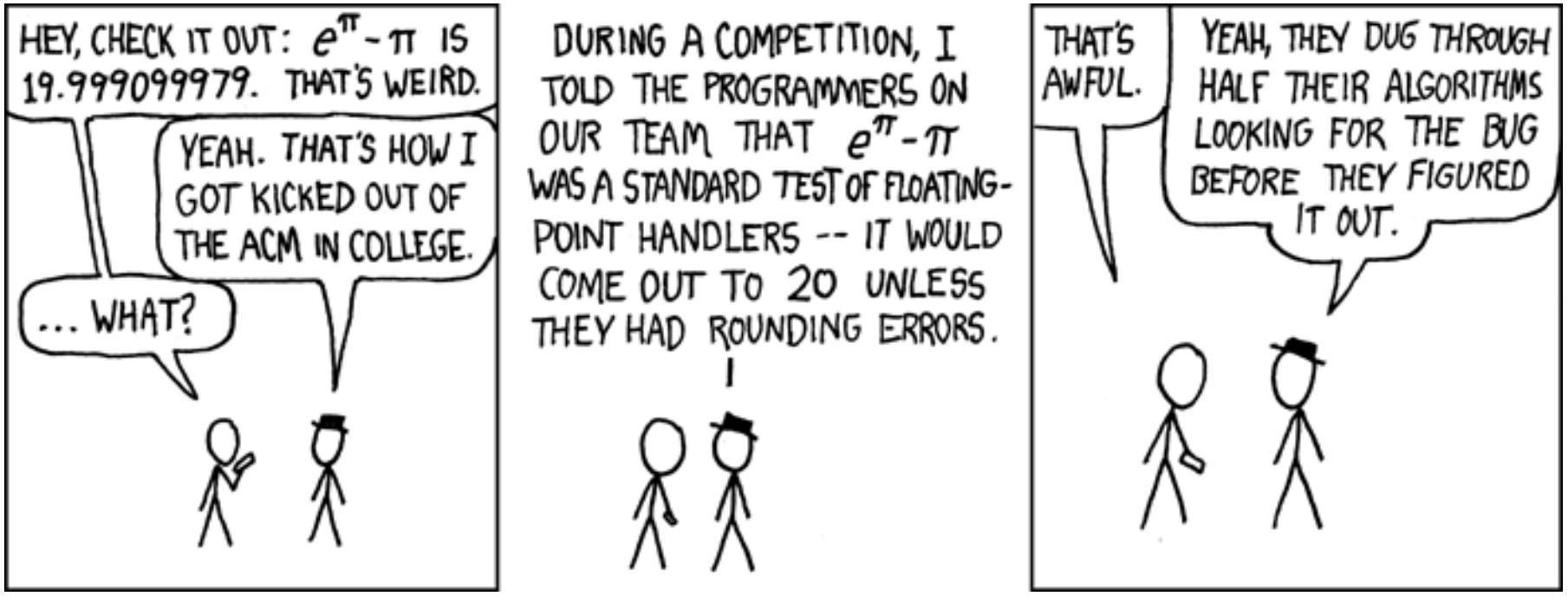
Calculatoare Numerice

– Cursul 5 –

Operații aritmetice

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

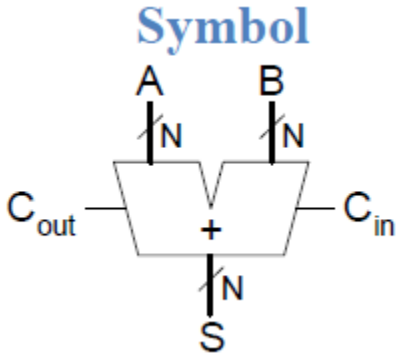
Comic of the day



http://xkcd.com/217/

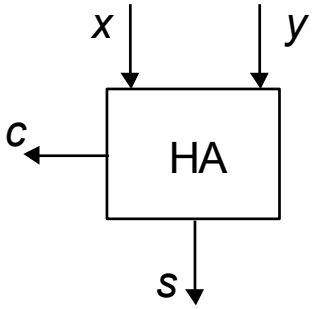
Sumatoare pe mai mulți biți

- Tipuri de sumatoare cu propagare de carry
  - Ripple-carry (lent)
  - Carry-lookahead (rapid)
  - Prefix (și mai rapid)
- Sumatoarele carry-lookahead și cu prefix sunt mai rapide pentru numere pe mai mulți biți dar necesită mai mult hardware



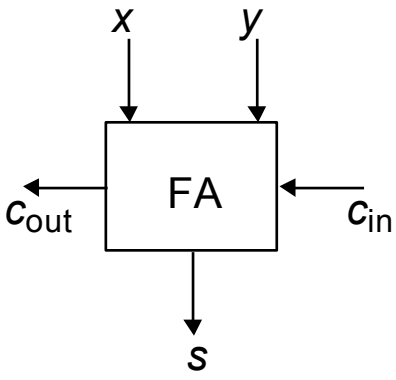
Sumatoare simple

Inputs		Outputs	
x	*y*	*c*	*s*
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



Interpretarea setului de cifre binare:
{0, 1} + {0, 1}
= {0, 2} + {0, 1}

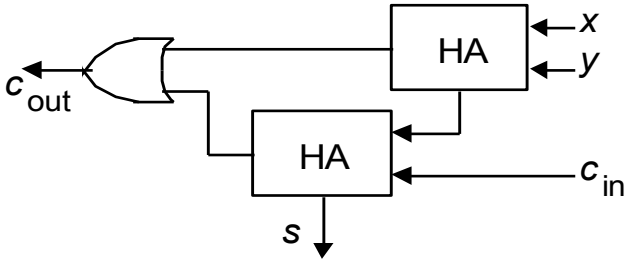
Inputs			Outputs	
x	*y*	c_{in}	c_{out}	*s*
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



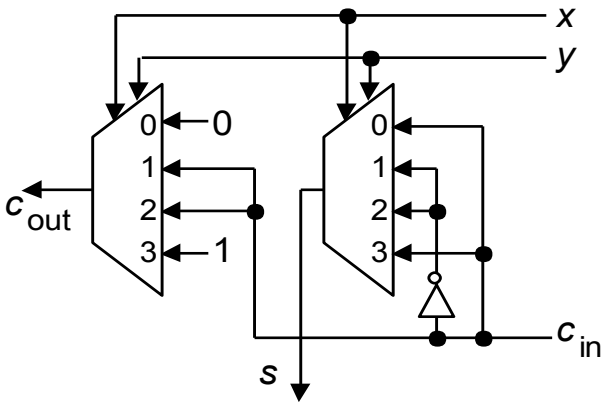
Interpretarea setului de cifre binare:
{0, 1} + {0, 1} + {0, 1}
= {0, 2} + {0, 1}

Binary half-adder (HA) & full-adder (FA).

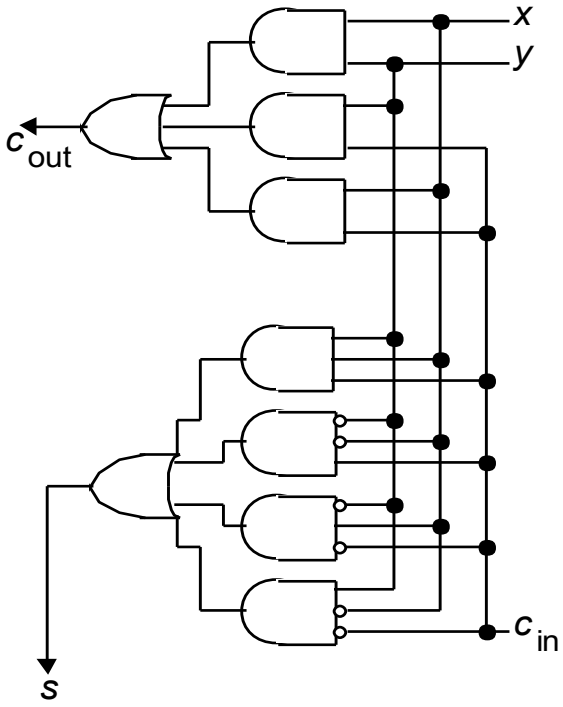
Implementarea unui sumator complet



(a) FA built of two HAs



(b) CMOS mux-based FA

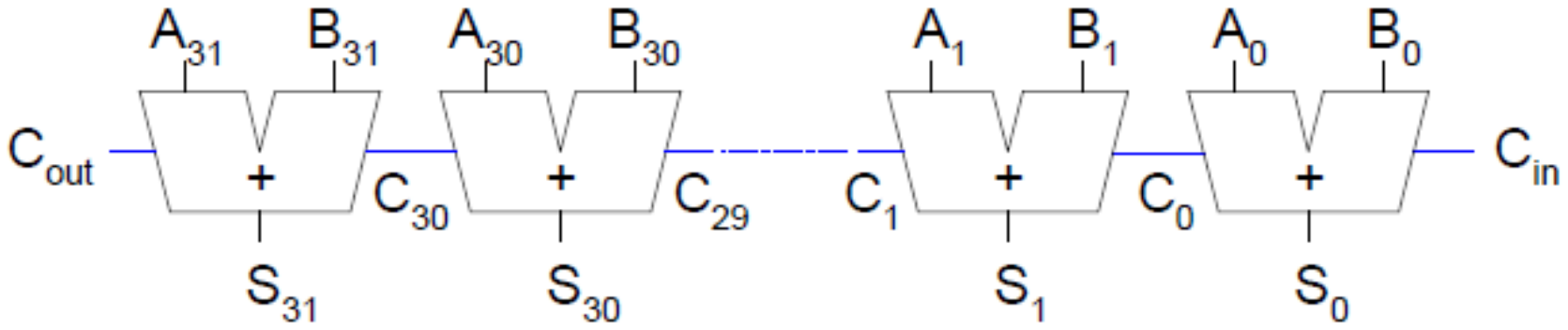


(c) Two-level AND-OR FA

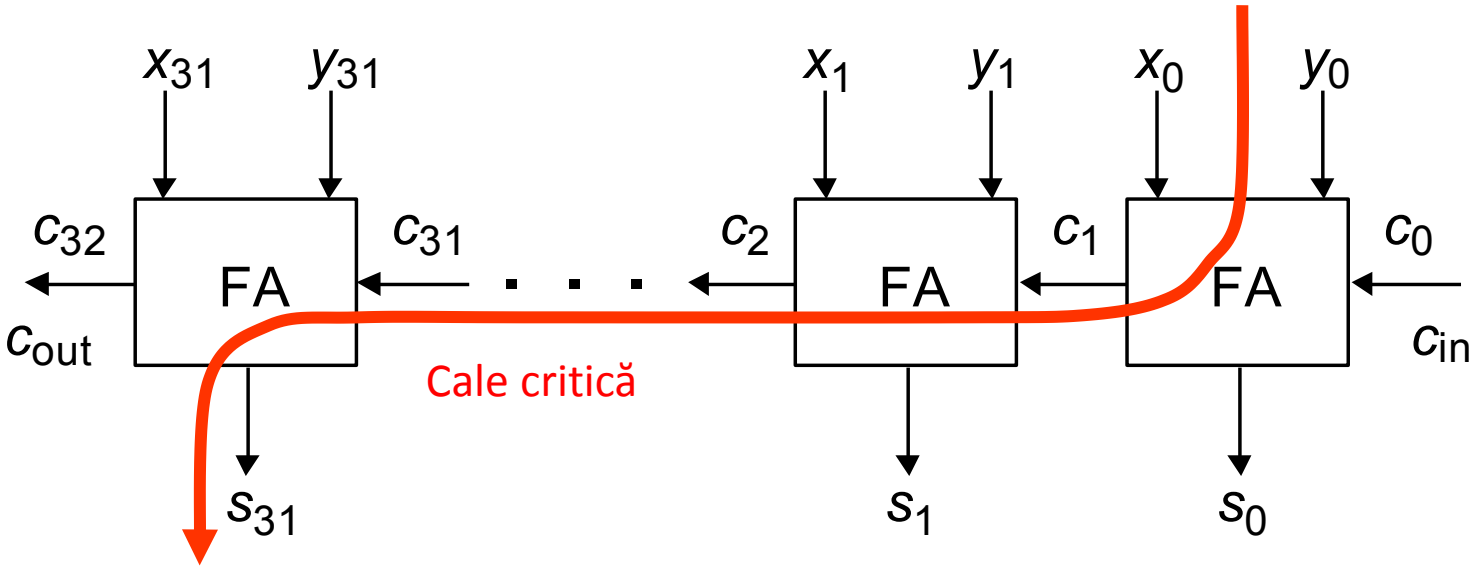
Sumator complet implementat folosind două sumatoare tip half-adder cu ajutorul a două multiplexoare cu 4 intrări sau un circuit cu porți pe două niveluri.

Sumator Ripple-Carry

- Înlănțuim mai multe sumatoare de un bit
- Semnalul de carry se propagă prin întregul lanț
- Dezavantaj: **lent**



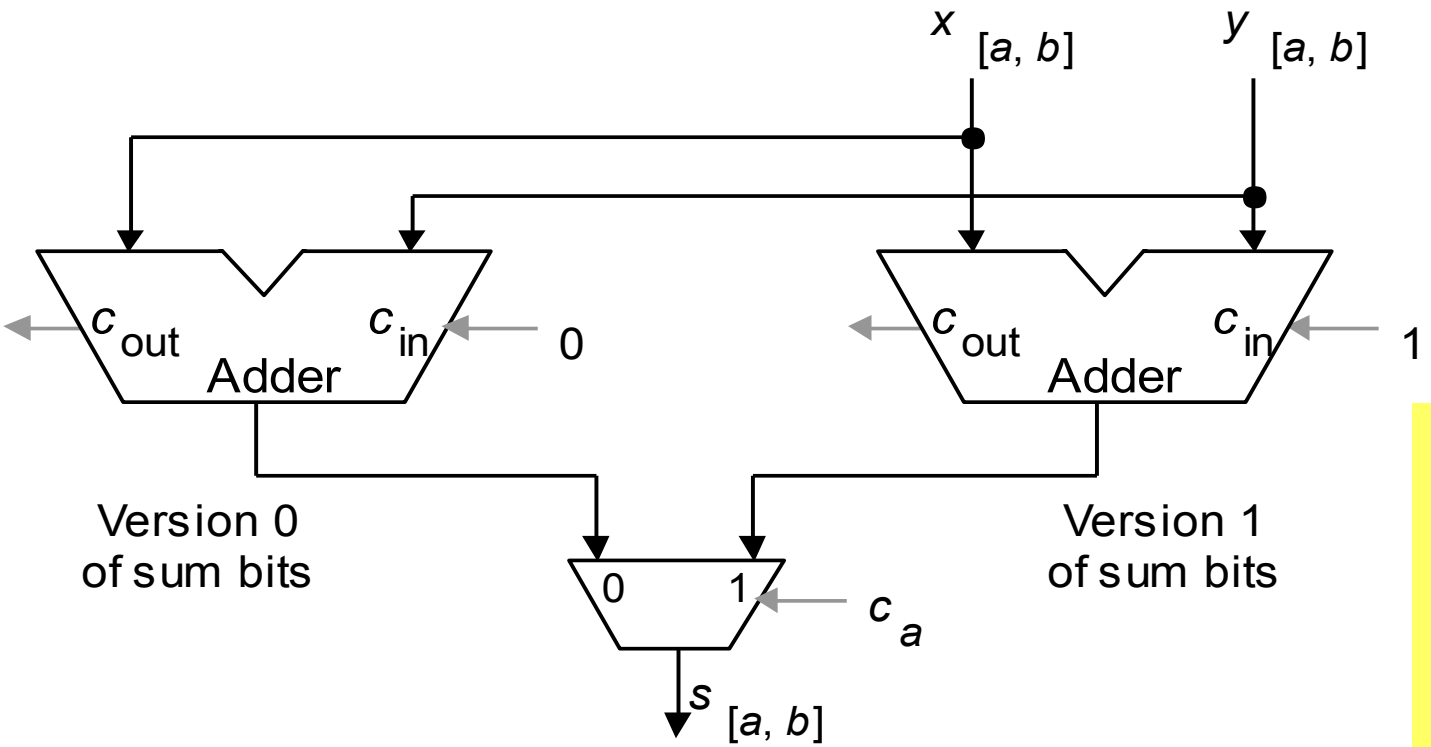
Sumator în riplu: lent dar foarte simplu



Sumator ripple-carry pentru numere de 32 de biți.

Sumator Carry Select

Presupune dublarea sumatorului inițial și introduce o întârziere suplimentară printr-un singur MUX



Rangurile inferioare ale lui a, (0 la $a-1$) sunt sumate normal

Principiul de funcționare al sumatorului Carry-select

Întârzierea prin ripple-carry

$$t_{ripple} = Nt_{FA}$$

unde t_{FA} este întârzierea printr-un sumator complet (Full Adder)

Sumator Carry-Lookahead

Embedded Systems Laboratory

- Calculează carry out (C_{out}) pentru blocuri de k biți folosind semnalele de *generare* și *propagare*
- **Definiții:**

– Coloana *i* produce un carry out fie prin *generarea* unui carry out sau prin *propagarea* unui carry in la ieșirea de carry out

– Semnalele Generate (G_i) și Propagate (P_i) petru fiecare coloană:

- Coloana *i* va genera un carry out dacă A_i ȘI B_i sunt ambele 1.

$$G_i = A_i B_i$$

- Coloana *i* va propaga un carry in la carry out dacă A_i SAU B_i sunt 1.

$$P_i = A_i + B_i$$

- Semnalul carry out al coloanei *i* (C_i) este:

$$C_i = A_i B_i + (A_i + B_i) C_{i-1} = G_i + P_i C_{i-1}$$

Adunarea Carry-Lookahead

- **Pas 1:** Calculează G_i și P_i pentru toate coloanele
- **Pas 2:** Calculează G și P pentru blocuri de k-biți
- **Pas 3:** C_{in} se propagă prin fiecare bloc de k-biți de propagate/generate

Sumatorul Carry-Lookahead

- **Exemplu:** blocuri de 4 biți ($G_{3:0}$ și $P_{3:0}$):

$$G_{3:0} = G_3 + P_3 (G_2 + P_2 (G_1 + P_1 G_0))$$

$$P_{3:0} = P_3 P_2 P_1 P_0$$

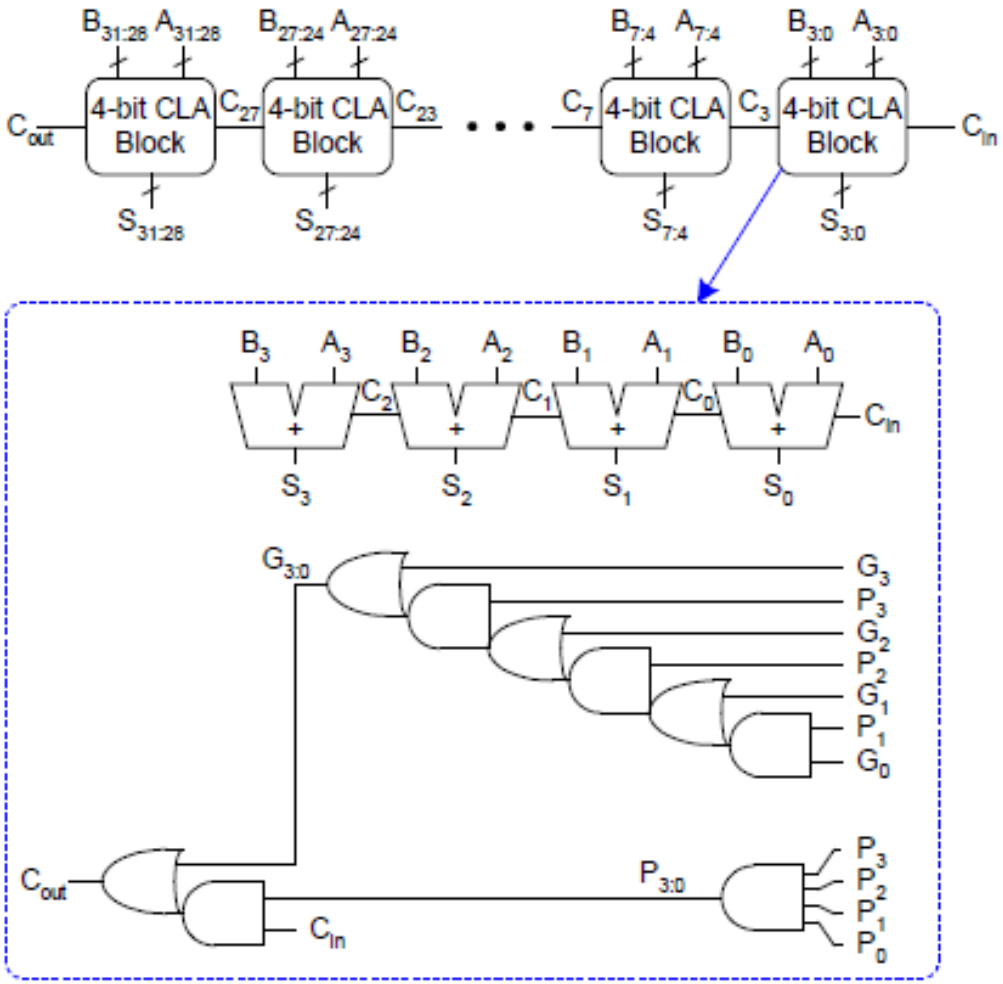
- **În general,**

$$G_{i:j} = G_i + P_i (G_{i-1} + P_{i-1} (G_{i-2} + P_{i-2} G_j))$$

$$P_{i:j} = P_i P_{i-1} P_{i-2} P_j$$

$$C_i = G_{i:j} + P_{i:j} C_{i-1}$$

CLA pe 32-biți cu blocuri de 4-biți



Întârzierile printr-un sumator Carry-LookAhead

Embedded Systems Laboratory

Pentru un CLA pe N biți din blocuri de k biți:

$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$

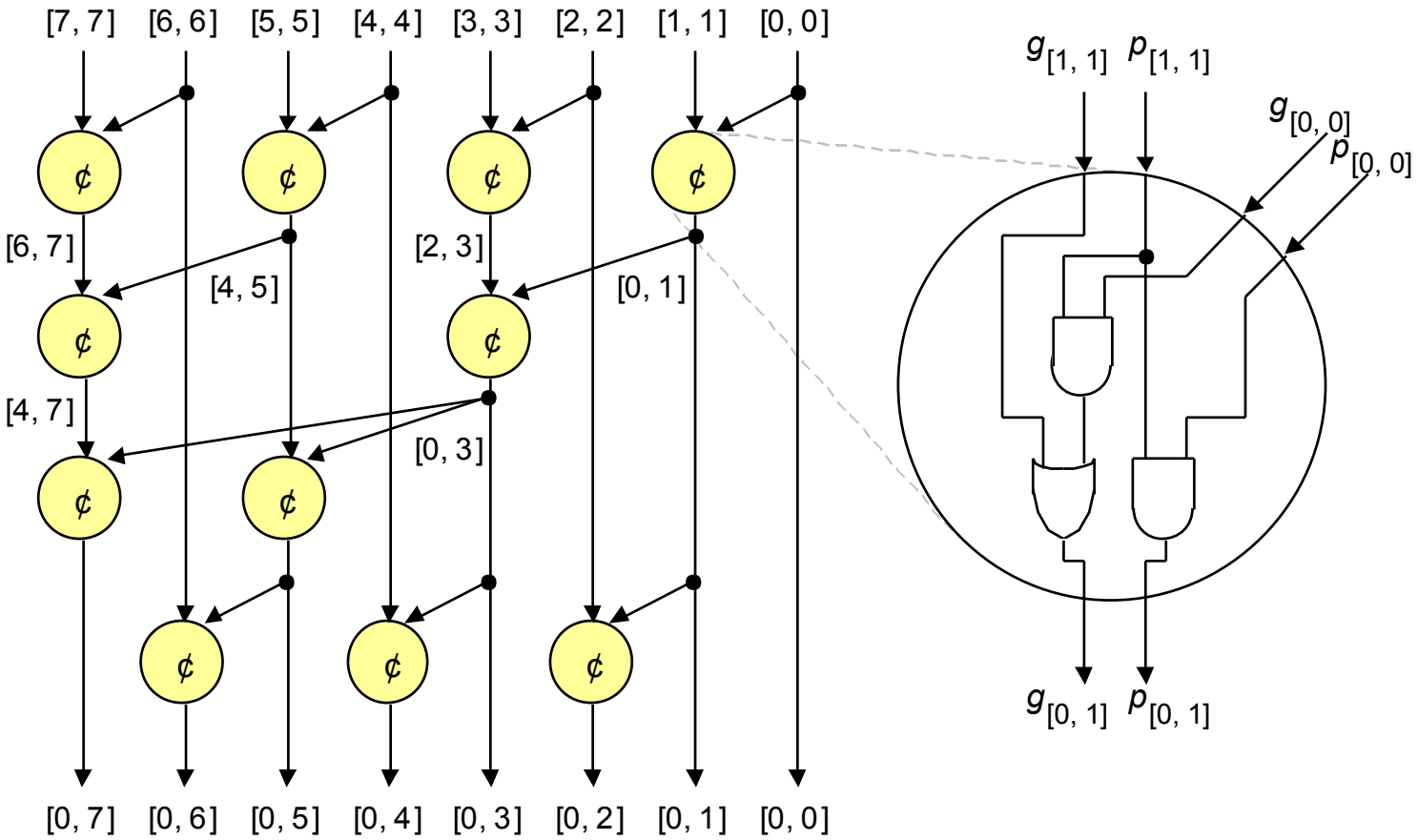
– t_{pg} : delay pentru generarea P_i, G_i

– t_{pg_block} : delay pentru generarea tuturor $P_{i:j}$, $G_{i:j}$

– t_{AND_OR} : delay de la C_{in} la C_{out} a ultimei porți AND/OR din blocul de k-biți al CLA

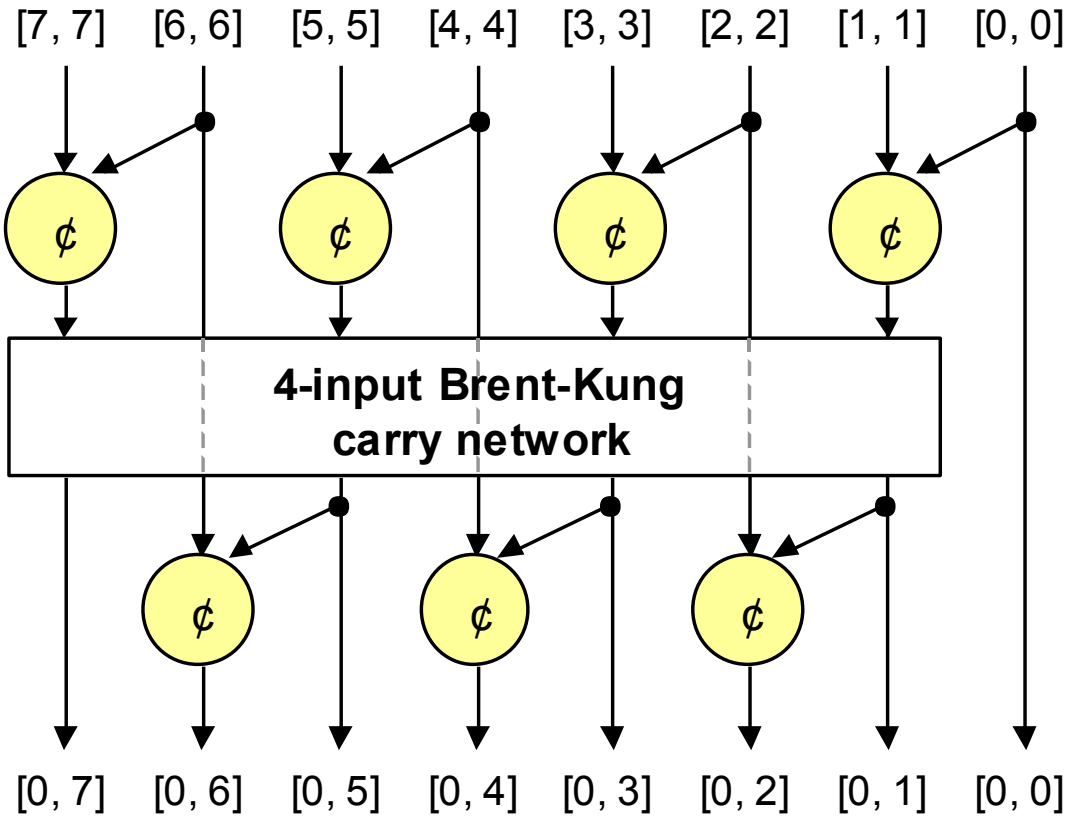
Un sumator carry-lookahead de N biți este mult mai rapid decât un sumator ripple-carry dacă $N > 16$

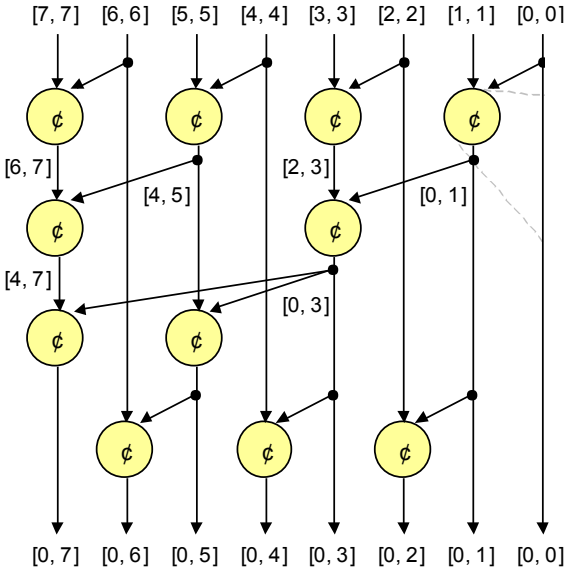
Rețele Carry - Carry Lookahead



Rețea carry lookahead Brent-Kung pentru un sumator pe 8 biți și detaliul de implementare pentru un bloc de carry.

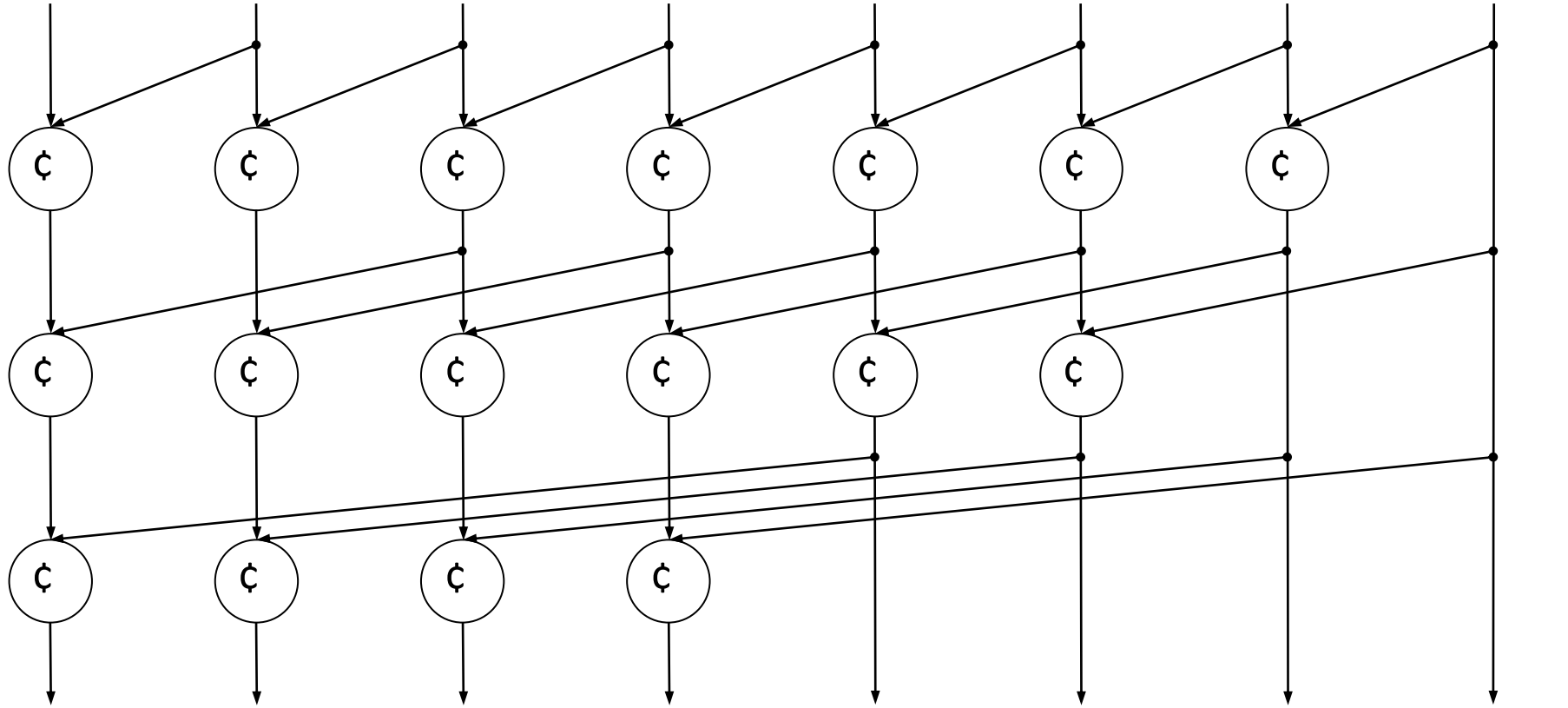
Structura recursivă a unei rețele carry Brent-Kung





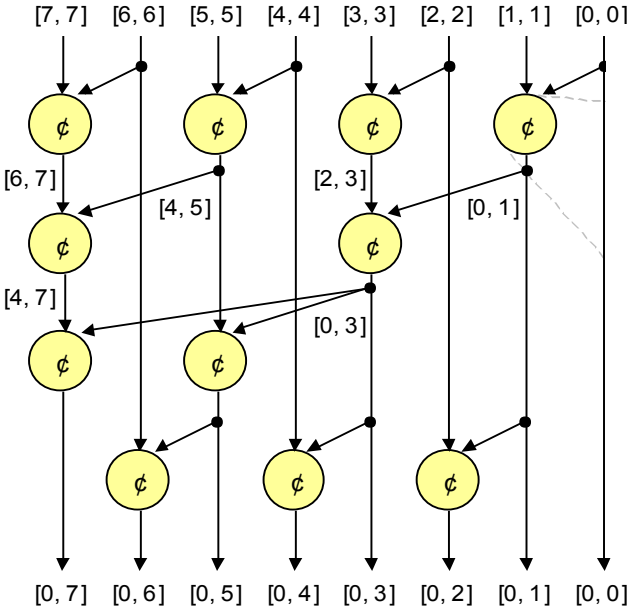
Rețea carry-lookahead Brent-Kung pentru un sumator pe 8 biți. Numai primul și ultimul nivel de operatori carry sunt evidențiați.

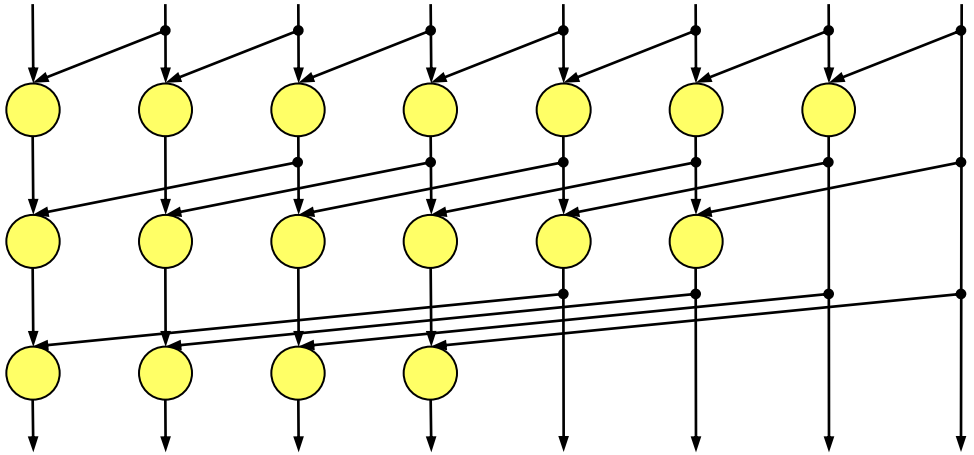
Un alt design: Rețeaua Kogge-Stone



Rețea carry-lookahead Kogge-Stone pentru un sumator pe 8 biți.

Brent-Kung vs. Kogge-Stone Carry Network

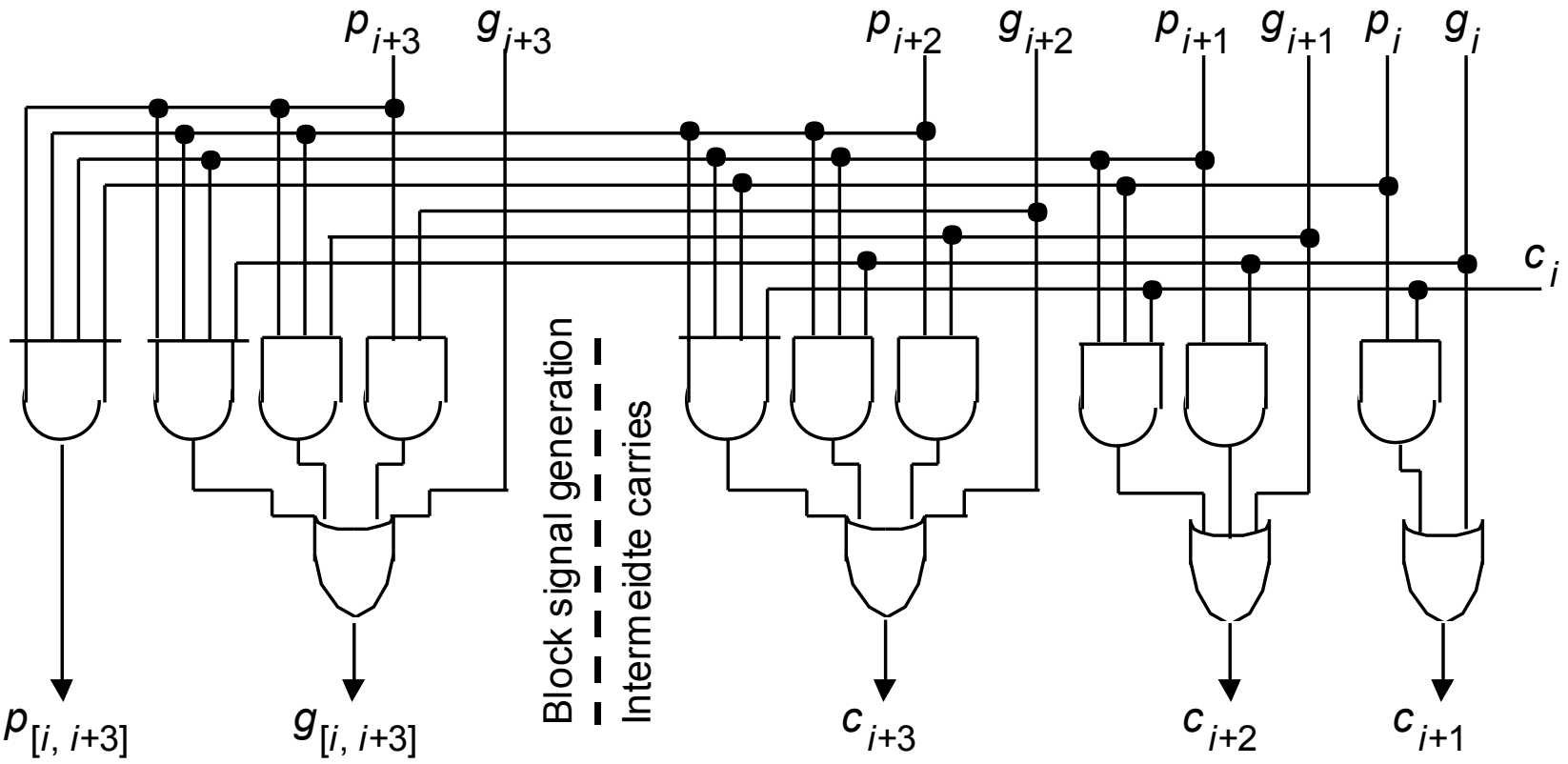




11 operatori carry
4 niveluri

17 operatori carry
3 niveluri

Logică Carry-Lookahead cu blocuri de 4 biți



Blocurile folosite pentru implementarea unui sumator carry-lookahead pot fi grupate în unități de câte 4 biți.

http://xkcd.com/676/

Sumator cu prefixe

Embedded Systems Laboratory

- Calculează carry in (C_{i-1}) pentru fiecare coloană apoi calculează suma:

$$S_i = (A_i \text{ XOR } B_i) \text{ XOR } C_i$$

- Calculează G și P pentru blocuri de 1-, 2-, 4-, 8- biți, etc.

Până când toate G_i (carry in) sunt cunoscute

- $\log_2 N$ etape

Sumator cu prefixe

Embedded Systems Laboratory

- Carry in este ori *generat* de o coloană sau *propagat* de la o coloană anterioară
- Coloana -1 ține C_{in}, așa că:

$$G_{-1} = C_{in},\ P_{-1} = 0$$

- Carry in al coloanei i = carry out al coloanei i-1:

$$C_{i-1} = G_{i-1:-1}$$

$G_{i-1:-1}$: semnal generate de la coloana i-1 la -1

- Ecuația sumei:

$$S_i = (A_i \text{ XOR } B_i) \text{ XOR } G_{i-1:-1}$$

- **Scop: Calculează rapid** $G_{0:-1}$, $G_{1:-1}$, $G_{2:-1}$, $G_{3:-1}$, $G_{4:-1}$, $G_{5:-1}$, ...(numite ***prefixe***)

Sumator cu prefixe

Embedded Systems Laboratory

• Semnalele Generate și Propagate pentru un bloc de la biții i:j:

$$G_{i:j} = G_{i:k} + P_{i:k} G_{k-1:j}$$

$$P_{i:j} = P_{i:k} P_{k-1:j}$$

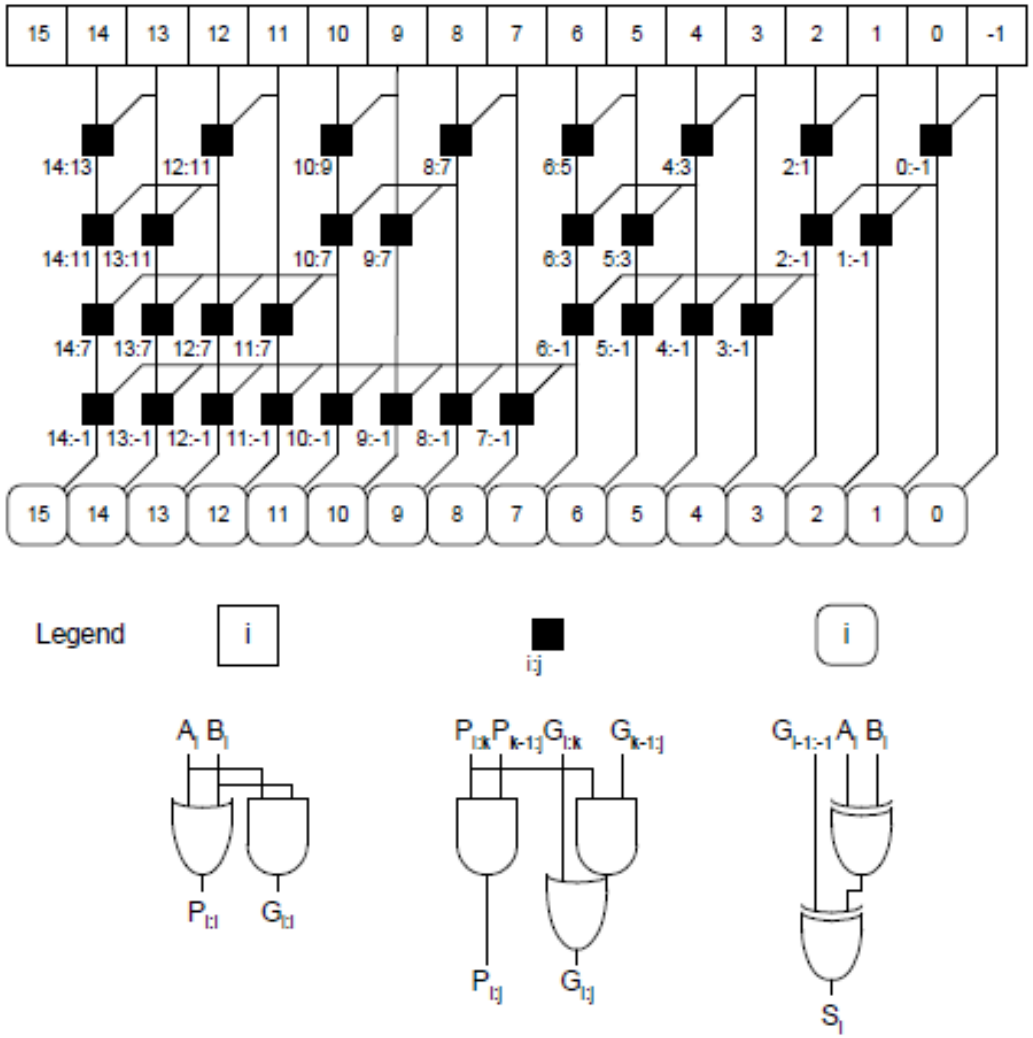
• În limba română:

– **Generate:** blocul i:j va genera carry dacă:

- partea superioară (i:k) generează carry, sau
- partea superioară propagă un carry generat de partea inferioară (k-1:j)

– **Propagate:** blocul i:j va propaga carry dacă și blocul superior și cel inferior propagă carry-ul

Schema unui sumator cu prefixe



Întârzierea unui sumator cu prefixe

$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR}$$

- t_{pg}: delay pentru a produce P_i G_i (poartă AND sau OR)
- t_{pg_prefix}: delay introdus de celula (neagră) pentru calculul semnalelor combinate (i,j) (porți AND-OR)

Compararea performanțelor

Embedded Systems Laboratory

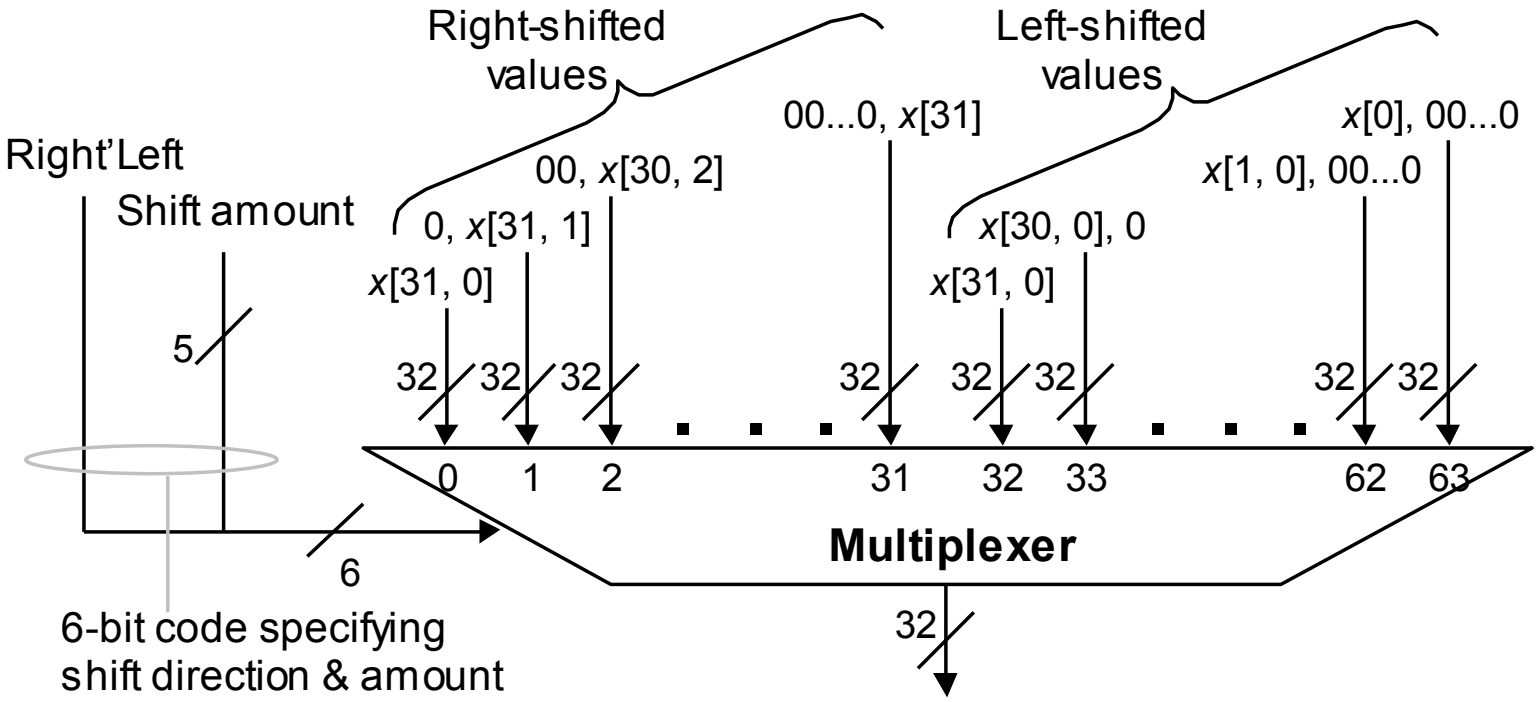
Comparați întârzierile prin trei sumatoare de 32 de biți cu ripple-carry, carry-lookahead si prefixe

- CLA are blocuri de 4-biți
- 2-input gate delay = 100 ps; full adder delay = 300 ps

$$t_{\text{ripple}} = Nt_{FA} = 32(300 \text{ ps})$$
$$= \mathbf{9.6 \text{ ns}}$$
$$t_{CLA} = t_{pg} + t_{pg_\text{block}} + (N/k - 1)t_{\text{AND_OR}} + kt_{FA}$$
$$= [100 + 600 + (7)200 + 4(300)] \text{ ps}$$
$$= \mathbf{3.3 \text{ ns}}$$
$$t_{PA} = t_{pg} + \log_2 N(t_{pg_\text{prefix}}) + t_{\text{XOR}}$$
$$= [100 + \log_2 32(200) + 100] \text{ ps}$$
$$= \mathbf{1.2 \text{ ns}}$$

Operații logice și de shiftare

Conceptual, shiftările pot fi implementate prin multiplexoare

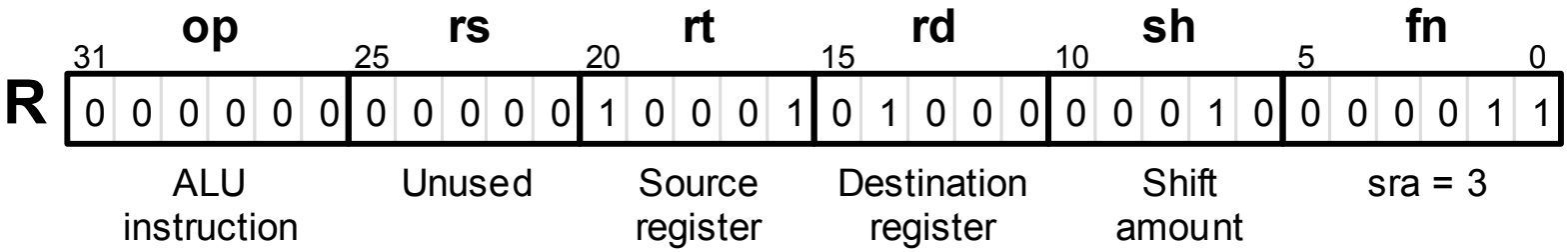


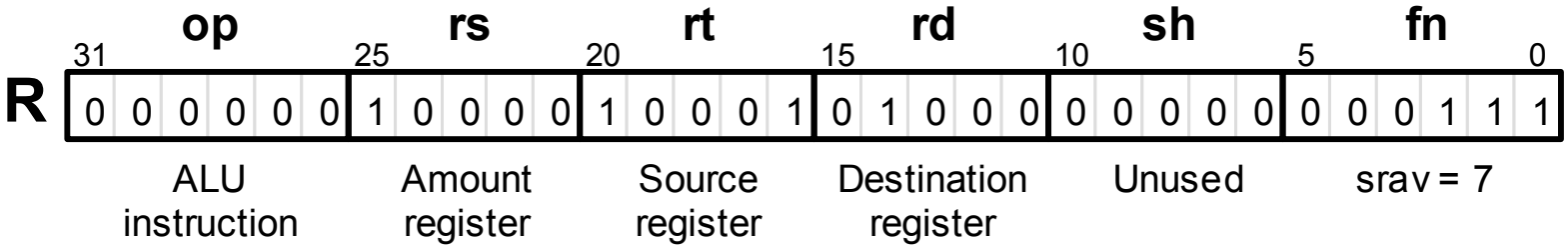
Unitate de shiftare logică folosind multiplexoare.

Shiftari aritmetice

Scop: înmulțirea și împărțirea cu puteri ale lui 2

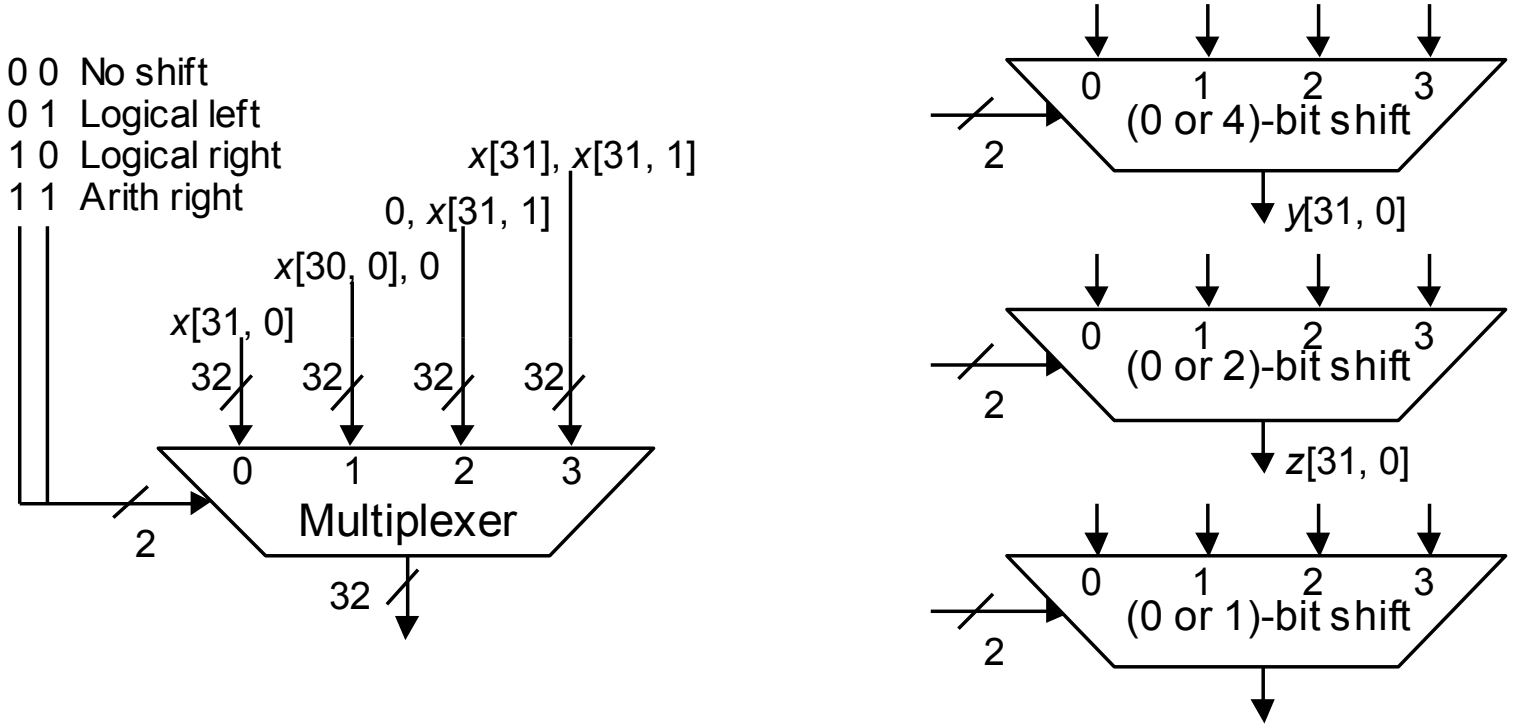
```
sra  $t0,$s1,2     # $t0 ← ($s1) right-shifted by 2
srav $t0,$s1,$s0   # $t0 ← ($s1) right-shifted by ($s0)
```





Operații de shiftare în cod mașină.

Implementarea shiftării pe mai multe niveluri



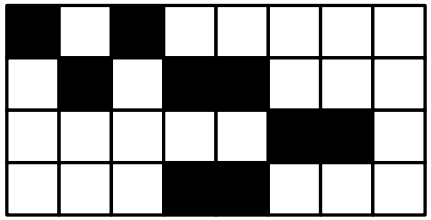
Shiftare pe mai multe niveluri într-un barrel shifter.

Manipularea biților prin shiftări si operații logice

Bits 10-15

ȘI cu o mască izolează un câmp: 0000 0000 0000 0000 1111 1100 0000 0000
Shiftează rezultatul cu 10 poziții pentru a alinia câmpul la dreapta cuvântului de date
Rezultatul poate să aibă valori între 0 și 63, în funcție de valoarea izolată

32-pixel (4 × 8) block of black-and-white image:



Representation as 32-bit word:

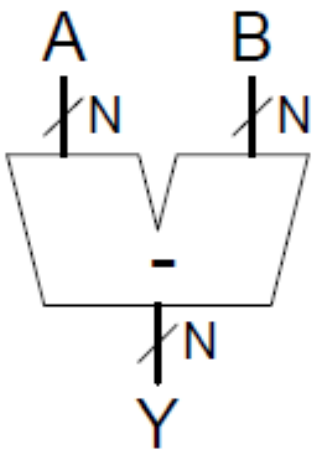
Row 0	Row 1	Row 2	Row 3
1010 0000	0101 1000	0000 0110	0001 0111

Hex equivalent: `0xa0a80617`

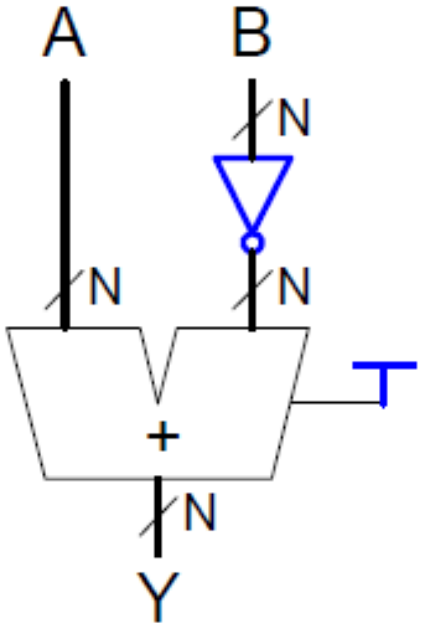
Un bloc de imagine alb-negru de 4 × 8 pixeli reprezentat ca un cuvânt de 32 de biți.

Scăzător

Simbol



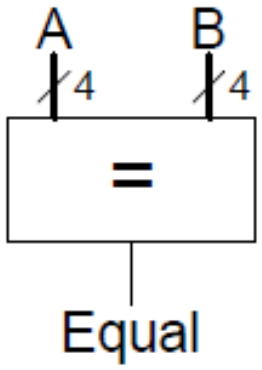
Implementare



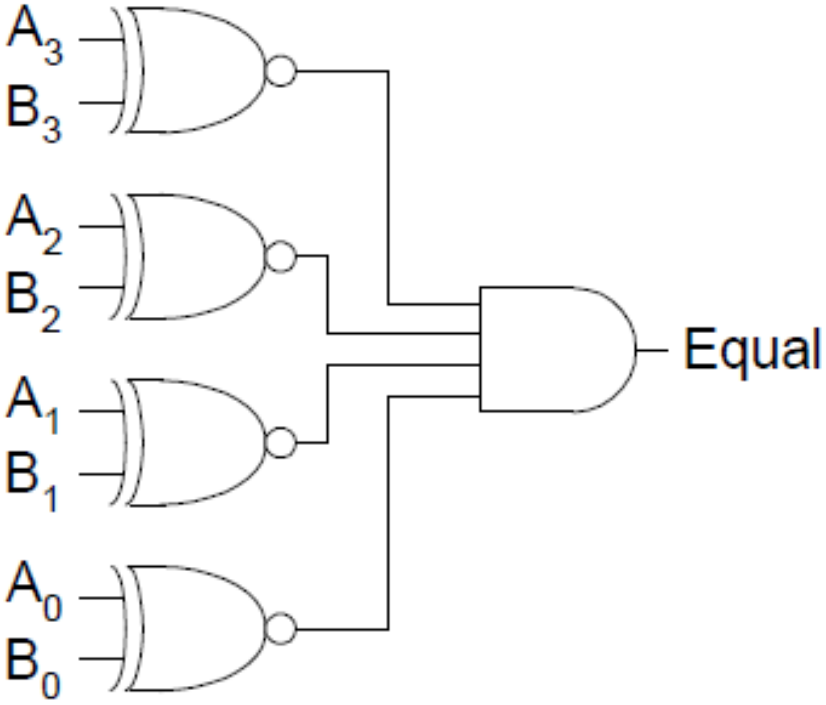
Comparator: Egalitate

Embedded Systems Laboratory

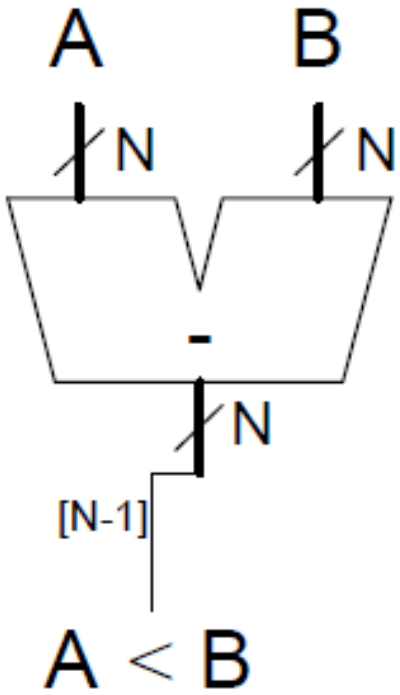
Simbol



Implementare



Comparator: “Mai mic decât”



Acknowledgements

- Aceste slide-uri conțin materiale aparținând:
  - Arvind (MIT)
  - Krste Asanovic (MIT/UCB)
  - Joel Emer (Intel/MIT)
  - James Hoe (CMU)
  - John Kubiatowicz (UCB)
  - David Patterson (UCB)
  - Behrooz Parhami (UCSB)

- MIT material derived from course 6.823
- UCB material derived from course CS252